C++ 程序设计

原理与实践

基础篇

Bjarne Stroustrup 著

据原书第 2 版（Programming: Principles and Practice Using C++, Second Edition）译出

原生中文重排版

2026

版权说明

本书原名为 *Programming: Principles and Practice Using C++, Second Edition*，作者 Bjarne Stroustrup，由 Pearson Education（Addison-Wesley）出版，版权归原作者与出版者所有。本文件是基于原书内容制作的非官方中文重排版，仅供个人学习与研究使用，不得用于商业发行。原书中的商标、插图与代码示例的权利归其各自权利人所有。

译文中对原书印刷错误的更正均在各章制作记录中留有证据；技术内容以原书为准。

目录

前言 **7**

给学生的话 . . . 8
给教师的话 . . . 9
ISO 标准 C++ . . . 9
支持与资源 . . . 10
致谢 . . . 10

0 读者须知 **11**

0.1 本书的结构 . . . 11
0.2 教与学的理念 . . . 14
0.3 编程与计算机科学 . . . 17
0.4 创造力与问题求解 . . . 18
0.5 征求反馈 . . . 18
0.6 参考文献 . . . 18
0.7 作者小传 . . . 19

1 计算机、人与编程 **21**

1.1 引言 . . . 21
1.2 软件 . . . 22
1.3 人 . . . 23
1.4 计算机科学 . . . 25
1.5 计算机无处不在 . . . 25
1.6 程序员的理想 . . . 31

2 Hello, World! **1**

2.1 程序 . . . 1
2.2 经典的第一个程序 . . . 2
2.3 编译 . . . 4
2.4 链接 . . . 6
2.5 编程环境 . . . 7

3 对象、类型与值 **11**

3.1 输入 . . . 11
3.2 变量 . . . 13
3.3 输入与类型 . . . 14
3.4 运算与运算符 . . . 15
3.5 赋值和初始化 . . . 19
3.6 复合赋值运算符 . . . 21

3.7 名字 . . . 23
3.8 类型与对象 . . . 24
3.9 类型安全 . . . 25

4 计算 33
4.1 计算 . . . 33
4.2 目标与工具 . . . 34
4.3 表达式 . . . 36
4.4 语句 . . . 42
4.5 函数 . . . 52
4.6 vector . . . 56
4.7 语言特性 . . . 62

5 错误 66
5.1 引言 . . . 66
5.2 错误的来源 . . . 68
5.3 编译时错误 . . . 68
5.4 链接时错误 . . . 70
5.5 运行时错误 . . . 71
5.6 异常 . . . 76
5.7 逻辑错误 . . . 82
5.8 估算 . . . 84
5.9 调试 . . . 85
5.10 前置条件与后置条件 . . . 89
5.11 测试 . . . 91

6 编写程序 96
6.1 问题 . . . 96
6.2 思考问题 . . . 97
6.3 回到计算器! . . . 99
6.4 文法 . . . 107
6.5 把文法变成代码 . . . 112
6.6 试运行第一版 . . . 119
6.7 试运行第二版 . . . 123
6.8 记号流 . . . 124
6.9 程序结构 . . . 128

7 完成程序 133
7.1 引言 . . . 133
7.2 输入与输出 . . . 133
7.3 错误处理 . . . 135
7.4 负数 . . . 139

7.5 余数：% 140
7.6 清理代码 141
7.7 从错误中恢复 147
7.8 变量 150

8 技术细节：函数等 161
8.1 技术细节 161
8.2 声明与定义 162
8.3 头文件 167
8.4 作用域 169
8.5 函数调用与返回 174
8.6 求值顺序 190
8.7 命名空间 192

9 技术细节：类等 199
9.1 用户自定义类型 199
9.2 类与成员 200
9.3 接口与实现 201
9.4 演化一个类 202
9.5 枚举 211
9.6 运算符重载 214
9.7 类接口 215
9.8 `Date` 类 224

10 输入与输出流 1
10.1 输入与输出 1
10.2 I/O 流模型 2
10.3 文件 3
10.4 打开文件 4
10.5 读写文件 6
10.6 I/O 错误处理 7
10.7 读取单个值 10
10.8 用户自定义输出运算符 15
10.9 用户自定义输入运算符 16
10.10 一个标准输入循环 17
10.11 读取结构化文件 19

11 定制输入与输出 29
11.1 规则性与不规则性 29
11.2 输出格式化 29
11.3 文件打开与定位 36
11.4 字符串流 40

11.5 行导向输入 . . . 41
11.6 字符分类 . . . 42
11.7 使用非标准分隔符 . . . 44
11.8 还有太多太多的内容 . . . 50

17 vector 与自由存储 1
17.1 引言 . . . 1
17.2 vector 基础 . . . 3
17.3 内存、地址与指针 . . . 4
17.4 自由存储与指针 . . . 7
17.5 析构函数 . . . 14
17.6 访问元素 . . . 18
17.7 指向类对象的指针 . . . 18
17.8 摆弄类型：void* 与类型转换 . . . 20
17.9 指针与引用 . . . 22
17.10 this 指针 . . . 29

18 向量与数组 36
18.1 引言 . . . 36
18.2 初始化 . . . 37
18.3 拷贝 . . . 38
18.4 基本操作 . . . 46
18.5 vector 元素的访问 . . . 50
18.6 数组 . . . 52
18.7 示例：回文 . . . 60

19 向量、模板与异常 67
19.1 问题 . . . 67
19.2 改变大小 . . . 69
19.3 模板 . . . 75
19.4 范围检查与异常 . . . 86
19.5 资源与异常 . . . 90

附录 A 语言概要 1
A.1 总则 . . . 1
A.2 字面量 . . . 4
A.3 标识符 . . . 7
A.4 作用域、存储类别与生存期 . . . 8
A.5 表达式 . . . 12
A.6 语句 . . . 19
A.7 声明 . . . 21
A.8 内建类型 . . . 23

A.9 函数 26
A.10 用户自定义类型 29
A.11 枚举 29
A.12 类 30
A.13 模板 41
A.14 异常 45
A.15 命名空间 46
A.16 别名 47
A.17 预处理指令 48

附录 C Visual Studio 入门 50
C.1 让程序运行起来 50
C.2 安装 Visual Studio 50
C.3 创建并运行程序 51
C.4 后续 52

前言

“该死的鱼雷！
全速前进。”
——Farragut 海军上将

编程是一门艺术：把问题的解法表达出来，让计算机去执行这些解法。编程所花的精力，很大一部分用在寻找和改进解法上。而且，一个问题常常只有动手为它编写解法，才能真正弄明白。

本书写给这样的读者：此前从未编过程序，却愿意下苦功去学。它会帮你理解编程的原理，并借助 C++ 程序设计语言掌握编程的实用技能。我的目标是让你获得足够的知识和经验，能用当前最好的技术完成简单而有用的编程任务。这要花多长时间？如果作为大学一年级课程的一部分来学，你可以在一个学期里学完本书（假定你同时修四门难度中等的课程）。如果是自学，就不要指望比这更省时间（大约每周 15 小时，共 14 周）。

三个月听起来也许很长，但要学的东西很多；而且，大约一个小时之后，你就能写出第一个简单程序。此外，学习总是循序渐进的：每一章都会介绍有用的新概念，并用取自真实应用的例子加以说明。用代码表达想法的能力——让计算机按你的意愿行事的能力——会随着学习的推进稳步提高。我从不主张“先学一个月的理论，再看看会不会用”。

人为什么要编程？我们的文明建立在软件之上。不懂软件，你就只能相信“魔法”，还会被挡在许多最有趣、最赚钱、也最有社会价值的技术工作之外。我谈编程时，想到的是计算机程序的整个谱系：从带图形用户界面（GUI）的个人计算机应用，到工程计算与嵌入式系统控制应用（例如数码相机、汽车和手机），再到许多人文学科和商业应用中常见的文本处理程序。与数学一样，编程——只要做得好——是一种极有价值的思维训练，能让我们的思考能力更敏锐。不过，因为有计算机给出的反馈，编程比大多数数学分支更具体，因而能为更多的人所掌握。它是一种影响世界、改变世界的方式——理想情况下是让世界变得更好。最后，编程还可以其乐无穷。

为什么用 C++？不借助某种程序设计语言，你无法学会编程；而 C++ 直接支持真实软件中使用的关键概念与技术。C++ 是使用最广的程序设计语言之一，其应用领域之宽，罕有其他语言能及。从海底到火星表面，C++ 应用随处可见。C++ 由一份非专有的国际标准精确而全面地定义。在各类计算机上，都能找到高质量的、和/或免费的实现。你用 C++ 学到的大多数编程概念，都可以直接用于其他语言，例如 C、C#、Fortran 和 Java。最后，我本人就是喜欢 C++，喜欢用它写出优雅而高效的代码。

本书不是最轻松的入门编程书，我也没打算把它写成那样。我只希望：在教你真实世界编程基础的书里，它是最容易读的一本。这个目标相当高，因为现代软件所依赖的许多技术，在几年前还被视作高深技术。

我的基本假设是：你希望写出供别人使用的程序，并且负责任地写，达到像样的系统质量；也就是说，我假定你想达到专业水准。因此，本书的选题覆盖真实世界编程入门所需的内容，而不只是那些容易教、容易学的东西。如果某项技术是把基本工作做对所需要的，我就会讲它，说明支撑它的概念和语言设施，配上习题，并指望你去做这些习题。如果你只想看懂玩具程序，需要的东西远比我讲的少。反过来说，我也不会拿实际价值不大的材料浪费你的时间。凡是本书讲解的想法，几乎都是你迟早会用到的。

如果你只想使用别人的成果，既不想弄懂事情是怎么做的，也不想自己给代码添上多少东西，那么本书不适合你。若果真如此，请想一想：换一本书、换一种语言，是不是对你更有帮助？如果那是

你对编程的大致看法，也请想一想这种看法从何而来，以及它是否真的能满足你的需要。人们常常既低估编程的复杂性，也低估它的价值。我不希望因为你的需要与我描述的这部分软件现实对不上，就让你对编程心生反感。“信息技术”领域里有许多方向并不需要编程知识。本书服务的是那些确实想编写或理解不那么简单的程序的人。

由于本书的结构和实用目标，它也可以用作第二本编程书：读者可以是懂一点 C++ 的人，也可以是用别的语言编程、想学 C++ 的人。如果你属于其中一类，我就不去猜你读完本书要多久了，但我确实建议你多做些习题。这有助于你避免一个常见毛病：沿用自己熟悉的旧风格写程序，而在更合适的地方不采用新技术。如果你是按某种比较传统的方式学的 C++，那么在读到第 7 章之前，你就会发现一些既意外又有用的东西。除非你也姓 Stroustrup，否则本书讨论的绝不是“你父亲那辈的 C++”。

编程是靠写程序学会的。在这一点上，编程与其他带实践成分的行当一样。光靠读书，学不会游泳，学不会演奏乐器，也学不会开车——你必须练。同样，不读、不写大量代码，也学不会编程。本书着重给出与讲解文字、插图紧密配合的代码示例。你需要这些示例来理解编程的理想、概念和原理，掌握用来表达它们的语言构造。这些固然必不可少，但仅靠它们，你还得不到编程的实用技能。要做到这一点，你得做习题，还得熟悉编写、编译和运行程序的工具。你需要自己犯错，并学会改正错误。写代码这件事，没有替代品。何况，乐趣正在于此！

另一方面，编程远不止是遵守几条规则、读读手册。本书明确不以“C++ 的语法”为中心。理解基本的理想、原理和技术，才是一名优秀程序员的本色。只有设计良好的代码，才有可能成为正确、可靠、易维护的系统的一部分。而且，“基本原理”才是长久的东西：等到今天的语言和工具演变或更替之后，它们仍然必不可少。

那么计算机科学、软件工程、信息技术等等呢？它们全都是编程吗？当然不是！编程是计算机相关领域中支撑一切的基础课题之一，在均衡的计算机科学课程里自有其位置。本书简要介绍了算法、数据结构、用户界面、数据处理和软件工程的关键概念与技术。不过，本书不能替代对这些课题进行全面而均衡的学习。

代码可以既有用又优美。本书写作的目的，就是帮你看到这一点，理解“代码优美”意味着什么，并帮你掌握原理、获得写出这种代码的实用技能。祝你编程顺利！

给学生的话

我们在 Texas A&M University 用本书教过的大学一年级学生已有数千人之多，其中约 60% 以前编过程序，约 40% 则从未见过一行代码。大多数人最后都学成了，所以你也能行。

你不必非把这本书当作课程的一部分来读。本书被广泛用于自学。不过，无论是跟着课程系统学习，还是独自钻研，都尽量和别人一起学。编程背上了一个“孤独的活儿”的名声——这不公平。大多数人处在一个目标相同的群体中时，做得更好，学得更快。和同伴一起学习、讨论问题不算作弊！这是最有效率、也最愉快的进步方式。退一步说，和同伴一起做事会逼你把想法讲清楚，而这差不多是检验自己是否真懂、并确保记得住的最有效办法。语言和编程环境里的那些冷僻问题，你其实不必每个都亲自找到答案。但是，请不要自己骗自己：小试身手（drill）要做，习题也要做相当数量（即使没有老师逼你）。请记住：编程（除了别的之外）是一门实用技能，要掌握它就得练。如果你不写代码（每章做上几道习题），读这本书就只是一场毫无意义的理论演练。

大多数学生——尤其是那些肯动脑筋的好学生——都有过这样的时刻：怀疑自己的苦功是否值得。当这种事发生在你身上时（它一定会发生，而非“也许”），不妨歇一歇，重读这篇前言，再看看第 1

章（“计算机、人与编程”）和第 22 章（“理想与历史”）。在那两章里，我试着说明编程究竟哪里让我兴奋，以及为什么我认为它是为世界做出积极贡献的关键工具。如果你想了解我的教学理念和总体思路，请翻一翻第 0 章（“读者须知”）。

你也许会觉得这本书太厚，有点吓人。不过，下面两点应该能让你安心。一是书之所以厚重，部分原因在于我宁可把某处解释再讲一遍、或者多加一个例子，也不愿让你去找那唯一的解释。另一个主要原因是，本书后半部分是参考资料和“附加材料”，只有在你对某个具体编程领域（比如嵌入式系统编程、文本分析或数值计算）想了解更多时，才需要去钻研。

还有，请别太心急。学习任何重要的、有价值的新技能都要花时间，而这时间是值得的。

给教师的话

不是。这并不是一门传统的“计算机科学 101”课程。本书讲的是怎样把能用的软件构造出来。正因如此，它略去了计算机科学专业学生传统上要接触的许多内容（图灵完备性、状态机、离散数学、乔姆斯基文法，等等）。连硬件也不谈，因为假定学生从幼儿园起就以各种方式用过计算机。本书甚至不打算提及大部分重要的计算机科学课题。它讲的是编程（更一般地说，是怎样开发软件），因此涉及的题目比别人少，讲得却比许多传统课程细。它只想把一件事做好，而计算机科学并不是一门课就能讲完的。如果本书/本课程被用作计算机科学、计算机工程、电气工程（我们最早几届学生中有不少是电气工程专业的）、信息科学或别的什么培养方案的一部分，我希望它与其他课程配合讲授，共同构成一个完整的入门训练。

关于我的教学理念、总体思路等，请读第 0 章（“读者须知”）。也请你在教学过程中把这些想法传达给学生。

ISO 标准 C++

C++ 由一项 ISO 标准定义。第一份 ISO C++ 标准于 1998 年获得批准，所以那个版本的 C++ 称为 C++98。本书第一版是我在参与设计 C++11 期间写的。当时无法用那些新特性（例如统一初始化、范围 for 循环、移动语义、lambda 表达式和 concepts）来简化原理与技术的讲述，这让人十分懊恼。不过，本书在构思时就以 C++11 为念，所以把新特性“塞回”它们该在的上下文里相对容易。截至写作之时，现行标准是 2011 年的 C++11，而即将发布的 2014 年 ISO 标准 C++14 中的设施，正陆续进入主流的 C++ 实现。本书使用的语言是 C++11，另含少量 C++14 特性。例如，如果你的编译器对下面这段代码报错：

```
vector<int> v1;
vector<int> v2 {v1};    // C++14 风格的复制构造
```

请改用：

```
vector<int> v1;
vector<int> v2 = v1;    // C++98 风格的复制构造
```

来代替它。

如果你的编译器不支持 C++11，那就换一个新编译器。好的现代 C++ 编译器可以从多家供应商处下载，见 www.stroustrup.com/compilers.html。用更早期、支持更少的语言版本去学编程，会带来不必要的困难。

支持与资源

本书的支持网站 www.stroustrup.com/Programming 上汇集了多种材料，供使用本书讲授与学习编程时取用。这些材料会日臻完善；不过作为起步，你可以在上面找到：

- 基于本书的讲课幻灯片
- 教师指南
- 本书所用各库的头文件与实现
- 书中示例的代码
- 部分习题的解答
- 可能有用的一些链接
- 勘误表

我们随时欢迎你提出改进建议。

致谢

我要特别感谢已故的同事、与我共同授课的 Lawrence “Pete” Petersen。他鼓励我去承担教初学者这件事，那比我原本觉得可以胜任的时间要早得多；他还贡献了实际的教学经验，使这门课得以成功。没有他，最初版本的课程会以失败告终。本书所服务的那门课，最初几个版本是我们一起做出来的；我们一起反复讲授，从中汲取经验，不断改进课程，也改进本书。本书中我用的“我们”，起初指的就是“Pete 和我”。

感谢 Texas A&M University 的 ENGR 112、ENGR 113 和 CSCE 121 各课程的学生、助教和同伴教师，他们以直接或间接的方式帮助我们构筑了这本书；也感谢 Walter Daugherity、Hyunyoung Lee、Teresa Leyk、Ronnie Ward 和 Jennifer Welch，他们也讲授过这门课。还要感谢 Damian Dechev、Tracy Hammond、Arne Tolstrup Madsen、Gabriel Dos Reis、Nicholas Stroustrup、J. C. van Winkel、Greg Versoonder、Ronnie Ward 和 Leor Zolman 对本书各版草稿提出的建设性意见。感谢 Mogens Hansen 讲解发动机控制软件。感谢 Al Aho、Stephen Edwards、Brian Kernighan 和 Daisy Nguyen 帮我躲开杂事，让我在夏天把书写完。

感谢 Art Werschulz，他在纽约市 Fordham University 的课程中使用本书第一版，提出了许多建设性意见；感谢 Nick Maclaren，他在剑桥大学使用本书第一版，对习题提出了许多细致的意见。他的学生与 TAMU 的一年级学生相比，背景和专业需求都迥然不同。

感谢 Addison-Wesley 为我找到的各位审稿人。他们的意见弥足珍贵，大多基于他们在大学讲授 C++ 或“计算机科学 101”的经验：Richard Enbody、David Gustafson、Ron McCarty 和 K. Narayanaswamy。也感谢我的编辑 Peter Gordon，他提了许多有用的意见，而且（同样重要的是）很有耐心。我非常感激 Addison-Wesley 组建的制作团队，他们为本书的质量增色不少：Linda Begley（校对）、Kim Arney（排版）、Rob Mauhar（插图）、Julie Nahil（制作编辑）和 Barbara Wood（文字编辑）。

感谢本书第一版的各位译者，他们发现了许多问题，帮助澄清了许多地方。尤其是 Loïc Joly 和 Michel Michaud，他们对法文译本做了彻底的技术审校，带来了许多改进。

我还要感谢 Brian Kernighan 和 Doug McIlroy，他们为如何写作编程书籍树立了极高的标准；也要感谢 Dennis Ritchie 和 Kristen Nygaard，他们在实用的语言设计方面提供了宝贵的教益。

0 读者须知

“当地形与地图不符时，要相信地形。”
——瑞士军谚

本章是一袋杂七杂八的信息，目的是让你大致了解本书其余部分会讲些什么。请随手翻一翻，只读你感兴趣的地方。教师会发现其中大部分内容马上就能派上用场。如果你没有一位好老师领路，就不要试图读懂本章的每一处；只看看“本书的结构”和“教与学的理念”两节的前半部分即可。等你觉得编写并运行小程序已经得心应手了，不妨再回来重读本章。

0.1 本书的结构
- **0.1.1** 一般方法
- **0.1.2** 小试身手、习题等
- **0.1.3** 本书之后学什么？

0.2 教与学的理念
- **0.2.1** 主题的顺序
- **0.2.2** 编程与编程语言
- **0.2.3** 可移植性

0.3 编程与计算机科学

0.4 创造力与问题求解

0.5 征求反馈

0.6 参考文献

0.7 作者小传

0.1 本书的结构

本书由四个部分和一组附录组成：

- *第一部分“基础”*讲解编程的基本概念与技术，以及起步写代码所需的 C++ 语言和标准库设施。内容包括类型系统、算术运算、控制结构、错误处理，还有函数与用户自定义类型的设计、实现和使用。
- *第二部分“输入与输出”*说明怎样从键盘和文件读取数值与文本数据，又怎样把相应的结果写到屏幕和文件。接着，它演示如何把数值数据、文本和几何图形呈现为图形输出，以及如何通过图形用户界面（GUI）把输入送进程序。
- *第三部分“数据与算法”*聚焦 C++ 标准库的容器与算法框架（STL，标准模板库）。它展示容器（如 `vector`、`list` 和 `map`）如何借助指针、数组、动态内存、异常和模板实现、又如何使用，同时演示标准库算法（如 `sort`、`find` 和 `inner_product`）的设计与用法。
- *第四部分“开阔视野”*给出一个审视编程的视角：讨论理想与历史，举出若干实例（如矩阵计算、文本处理、测试和嵌入式系统编程），并简要介绍 C 语言。
- *附录*提供一些不适合写进教程正文的有用信息，例如 C++ 语言与标准库设施的概览，以及如何开始使用集成开发环境（IDE）和图形用户界面（GUI）库的说明。

可惜，编程世界并不真的能分成四个界限分明的部分。因此，本书的“部分”只是对主题的粗略分类。我们认为这是个有用的分类（否则也不会采用它），但现实总有办法逃出整齐的框框。举个例子：我们用到输入操作的时间，远早于我们能详细讲解 C++ 标准 I/O 流（输入/输出流）的时候。当讲解某

个想法所需的主题集合与整体分类发生冲突时，我们只讲清把问题说明白所必需的最少内容，而不是简单地把读者推给别处的完整解释。僵硬的分类适合手册，远胜过适合教程。

主题的顺序由编程技术决定，而不是由编程语言特性决定；参见 §0.2。若想看按语言特性组织的讲解，见附录 A。

为了便于复习，也为了在你初次阅读、还分不清哪类信息至关重要而漏掉要点时帮你一把，我们在页边放了三种“提示标记”：

- 蓝色：概念与技术（本段就是一个例子）
- 绿色：建议
- 红色：警告

0.1.1 一般方法

在本书中，我们直接对你说话。这比多数科学论文里那种“专业”的间接口吻更简单，也更清楚。我们说“你”，指的是“你，读者”；说“我们”，指的要么是“我们自己，也就是作者和教师”，要么是你和我们一起对付某个问题——就好像我们在同一间屋子里共事一样。

本书按从头到尾、一章一章的顺序设计。你常常会想回头再看第二遍、第三遍。事实上，这是唯一明智的读法，因为你总会匆匆掠过一些细节，而当时还看不出它们有什么用。遇到这种情况，你最终还是要回头再看。然而，尽管有索引和交叉引用，这并不是一本随便翻到哪一页都能读下去、还能指望读懂的书。每一节、每一章都以读懂前面的内容为前提。

每一章都是一个相当自足的单位，本意是“一口气”读完（逻辑上如此，尽管在学生紧张的日程里未必总能做到）。这是我们把正文分章的一条主要标准。其他标准还有：一章是安排小试身手和习题的合适单位；每章都要讲清某个特定的概念、想法或技术。这些标准凑在一起，让少数几章长得有点不像话，所以请不要把“一口气”太当真。特别是，在你思考过要点回顾的问题、做完小试身手、又动手做了几道习题之后，往往会发现还得回头重读几节，而好几天已经过去了。我们把各章聚成若干个“部分”，每部分围绕一个主题，比如输入/输出。这些部分很适合作为复习单位。

对一本教科书常见的赞誉是：“我刚想到的问题，它都回答了！”对于细小的技术问题，这确实是个理想；本书的早期读者也说有过这种体验。然而，这不可能成为全部的理想。我们提出的问题，初学者多半想不到。我们想提出并回答的，是你为别人编写高质量软件时必须考虑的问题。学会提出正确的问题（往往是难题），是学会像程序员那样思考的重要一环。只问那些容易的、显而易见的问题，会让你自我感觉良好，却无助于把你变成程序员。

我们尽量尊重你的智力，也尽量体谅你的时间。在讲述上，我们追求专业，而不是花哨；宁可把话说得轻一点，也不愿炒作。我们尽量不夸大某项编程技术或某个语言特性的重要性，但也请你不要低估“这往往很有用”这样一句朴素的话。如果我们不声不响地强调某件事很重要，意思是：你要是不掌握它，迟早会白白浪费好几天。我们用幽默的频率比我们希望的要低，因为经验表明，各人对什么好笑看法差别极大，而一次失败的幽默尝试只会让人困惑。

我们并不假装自己的想法或提供的工具完美无缺。面对程序员遇到的种种难题，没有任何工具、库、语言或技术是“终极答案”。它顶多能帮你开发并表达你自己的解法。我们尽力避免“善意的谎言”，也就是不给出那种清楚易懂、但在真实语言和真实问题的语境里并不成立的简化解释。另一方面，本书不是参考手册；要更精确、更完整地了解 C++，请看 Bjarne Stroustrup 的 *The C++ Programming Language, Fourth Edition*（Addison-Wesley，2013）和 ISO C++ 标准。

0.1.2 小试身手、习题等

编程不只是一种智力活动，所以要想掌握编程技能，就必须动手写程序。我们提供两个层次的编程练习：

- *小试身手*（*Drill*）：小试身手是一种很简单的练习，用来培养实用的、近乎机械的技能。它通常是对同一个程序作一连串修改。每一次小试身手都应该做。它并不要求你有多么深刻的理解、多么机灵或多么主动。我们把小试身手看作本书的基本构件。没做过小试身手，就谈不上“读完了”这本书。
- *习题*（*Exercise*）：有些习题很简单，有些则很难，但大多数都想给主动性和想象力留出余地。如果你认真对待，就多做几道；至少要做到能看出哪些题对你来说很难，然后再多做一些那样的题，这样收获最大。习题的设计目标是：不需要特别聪明也能应付，而不是故意设成刁钻的谜题。不过，我们希望提供的习题难到足以让任何人都感到挑战，也多到足以耗尽哪怕最好的学生的全部时间。我们并不指望你把它们全做完，但不妨放手去试。

此外，我们建议你（每一位学生）参加一个小项目（时间允许的话，多参加几个）。项目的目标是做出一个完整而有用的程序。理想情况下，项目由一个小团队（比如三个人）共同完成，花一个月左右，同时学完第三部分各章。多数人觉得项目最有意思，也最能把这些内容串成一体。

有些人喜欢把书放下，先试几个例子，再往下读；另一些人则愿意先读到章末，然后才动手让代码跑起来。为了照顾前一种读者，我们在正文自然停顿的地方给出一些简单的动手建议，标为“试一试”（Try this）。试一试大体上相当于一次小试身手，只不过紧紧围绕它前面的主题。如果你路过一个试一试却没有动手——也许因为身边没有计算机，也许因为正读得入迷——那么在完成本章小试身手时，请回头补上；试一试要么是小试身手的补充，要么就是它的一部分。

每章末尾都有一组要点回顾（Review）问题，用意是指出本章讲解的关键思想。可以把要点回顾看作习题的补充：习题着眼于编程的实践层面，而要点回顾则试图帮你把想法和概念讲清楚。就此而言，它们很像好的面试题。

每章末尾的“术语”一节列出编程和 C++ 的基本词汇。如果你想听懂别人谈论编程话题，也想把自己的想法讲清楚，就该知道每个术语是什么意思。

学习离不开重复。我们的理想是：每个要点至少讲两遍，再用习题加以巩固。

0.1.3 本书之后学什么？

读完本书，你就能成为编程和 C++ 的专家吗？当然不能！编程做得好，是一门精细、深奥、技术要求很高的技艺，建立在多种专业技能之上。你不该指望四个月就成为编程专家，就像不该指望四个月——或者半年、一年——就成为生物学、数学、一门自然语言（如汉语、英语或丹麦语）的专家，或者成为小提琴演奏专家一样。你应当期望、并且在认真对待本书的前提下可以指望的，是一个真正良好的开端：能写出比较简单而有用的程序，能读懂更复杂的程序，并为后续的深入工作打下扎实的概念基础和实践基础。

这门入门课之后最好的延续，是参加一个真实项目，开发供别人使用的代码。在这之后，或者（更好的做法是）与真实项目同时进行，可以读一本专业水准的通用教材（如 Stroustrup 的 *The C++ Programming Language*），一本切合你项目需要的专门书籍（如讲 Qt 做 GUI 的，或讲 ACE 做分布式编程的），或者一本聚焦 C++ 某一方面的教材（如 Koenig 和 Moo 的 *Accelerated C++*、Sutter 的 *Exceptional C++*，或 Gamma 等人的 *Design Patterns*）。更多文献见 §0.6 或书末的参考文献（Bibliography）部分。

最终，你还应该再学一门编程语言。我们认为，在软件领域里做一名专业人士——即使你并不以编程为主业——却只会一门语言，是不可能的。

0.2 教与学的理念

我们想帮你学会什么？我们又怎样看待教学这件事？我们试图只给出最少的概念、技术和工具，够你写出有效的实用程序，其中包括：

- 程序组织
- 调试与测试
- 类设计
- 计算
- 函数与算法设计
- 图形（仅限二维）
- 图形用户界面（GUI）
- 文本处理
- 正则表达式匹配
- 文件与流的输入输出（I/O）
- 内存管理
- 科学、数值与工程计算
- 设计与编程的理想
- C++ 标准库
- 软件开发策略
- C 语言编程技术

沿着这些主题一路讲下来，我们涵盖了被称为过程式编程（procedural programming，如 C 语言中的做法）、数据抽象（data abstraction）、面向对象编程（object-oriented programming）和泛型编程（generic programming）的几种编程技术。本书的主题是编程，也就是用代码表达想法的那些理想、技术和工具。C++ 是我们主要的工具，所以我们相当详细地讲述 C++ 的许多设施。但请记住：C++ 只是工具，不是本书的主题。这本书是“用 C++ 编程”，不是“C++ 加一点编程理论”。

我们讲的每个主题至少有两个用处：既介绍一种技术、概念或原则，又介绍一项实用的语言或库特性。例如，我们用一个二维图形系统的接口来说明类和继承的用法。这样既能节省篇幅（也节省你的时间），又能强调：编程并不只是把代码胡乱堆在一起，好尽快得出结果。C++ 标准库正是这种“一箭双雕”例子的主要来源——很多例子甚至一箭三雕。比如我们引入标准库的 `vector`，用它说明广泛有用的设计技术，同时展示实现它时用到的许多编程技术。我们的目标之一是让你看到：主要的库设施怎样实现，又怎样对应到硬件上。我们坚持认为，手艺人必须了解自己的工具，而不能只把它们当成“魔法”。

有些主题，某些程序员会比另一些更感兴趣。不过我们劝你不要预判自己的需要（你怎么知道将来需要什么？），至少把每一章都看一看。如果你是把本书作为课程的一部分来读，老师会指导你如何取舍。

我们把自己的讲法概括为“深度优先”。它同时也是“具体优先”和“概念为本”的。首先，我们很快（嗯，相对很快，第 1—11 章）凑齐写小型实用程序所需的一套技能。在此过程中，我们以最少

的细节介绍大量工具和技术。我们专注于简单、具体的代码示例，因为人掌握具体的东西比掌握抽象的更快——人类大多就是这么学习的。在这个初始阶段，你不该指望弄懂每一个细枝末节。特别是你会发现，在刚刚跑通的代码上稍微改动一点，就可能有“神秘”的效果。不过，还是去试！也请把我们提供的小试身手和习题做完。只需记住：在早期，你还没有足够的概念和技能去准确判断什么简单、什么复杂；那就期待意外，并从意外中学。

在这个初始阶段我们推进得很快——我们想尽快把你带到能写有意思的程序的地步。有人会争辩：“我们必须慢慢来、小心来；得先会走才能跑！”可你见过婴儿学走路吗？婴儿确实是先自己跑起来，然后才学会慢而受控地行走这类更精细的本领。同样，你会一路冲在前面，偶尔绊一下，先找到编程的感觉，然后再慢下来，练出必要的精细控制力和理解力。你得先会跑，才能会走！

千万不要卡在试图把某个语言细节或技术“全部”学会的路上。举个例子，你可以把 C++ 的所有内置类型以及使用它们的全部规则都背下来。你当然做得到，这样做也许还让你觉得自己很有学问。但它不会把你变成程序员。跳过细节偶尔会让你因知识不足而“吃亏”，但这是获得写好程序所需眼界的最快途径。请注意，我们的做法本质上就是儿童学母语的做法，也是外语教学中最有效的做法。在难免卡住的场合，我们鼓励你向老师、朋友、同事、讲师、导师（Mentor）等求助。请放心，这些早期章节里没有什么是根本困难的。不过，很多内容对你来说是陌生的，所以一开始可能觉得难。

到后面，我们在这些初始技能之上扩展你的知识和技能基础。我们用例子和习题巩固你的理解，并为编程打下概念基础。

我们格外强调理想和理由。寻找实际解法时，你需要理想来指引——判断一个解法何时算好、何时算有原则。你还需要理解这些理想背后的理由，才能明白它们为什么该成为你的理想，为什么朝着它们努力对你、对你代码的使用者都有好处。谁都不该满足于“事情本来就是这样”这样的解释。更重要的是，理解了理想和理由，你就能从已知推广到新情境，并把想法和工具以新的方式组合起来，去应对新问题。懂得“为什么”，是获得编程技能不可缺少的一环。反过来，只是死记大量似懂非懂的规则和语言设施，会限制你的发展，成为错误的来源，也是极大的时间浪费。我们视你的时间为宝贵之物，尽量不去浪费它。

我们把许多 C++ 语言技术细节挪到了附录和手册里，需要时再去查。我们假定你有主动查找信息的劲头。请使用索引和目录。别忘了编译器的联机帮助，还有网络。不过要记住：在你找到理由相信某个网页资源更好之前，应把它视为高度可疑。许多看起来权威的网站，是编程新手或者想卖东西的人建的；另一些则干脆已经过时。我们在支持网站上收集了一批链接和资料：www.stroustrup.com/Programming。

请不要太急于要“真实”的例子。我们心目中的理想例子，是能直接说明某项语言设施、某个概念或某种技术的最短、最简单的代码。现实世界的大多数例子比我们的例子乱得多，但它们也不过是我们演示过的那些东西的组合。那些成功的商业程序有几十万行代码，其基础技术，我们在十来个 50 行的程序里就演示过了。理解真实代码最快的路，是把基础理解透。

另一方面，我们不用“毛茸茸的可爱动物”之类的例子来说明观点。我们假定你的目标是写出供真人使用的真实程序，所以凡不是作为语言技术讲解而给出的例子，都取自真实用途。我们的基本口吻，是专业人士对（未来的）专业人士说话。

0.2.1 主题的顺序

教人编程有很多种方法。显然，我们并不认同那种流行的“我是这么学编程的，所以这就是最好的学法”的理论。为了便于学习，我们很早就介绍几年前还会被视为高级的主题。我们的理想是：所讲的主题由你学编程时遇到的问题来驱动，随着你的理解和实践技能提高，从一个主题平滑地过渡到下一个。本书的主线更像一个故事，而不是一部词典或一套层级次序。

要写一个程序，需要掌握的原则、技术和语言设施不可能一次学完。因此，我们必须挑出一个子集作为起点。更一般地说，一本教科书或一门课程必须带领学生走过一系列子集。我们认为，挑选主题并决定轻重是我们的责任。我们没法把所有东西都摆出来，所以必须选择；而舍弃什么，至少和留下什么同样重要——在旅程的每个阶段都是如此。

为了形成对照，不妨看看我们决定不采用的那些路子，下面是它们（经过大幅简化）的特征：

- “*C* 优先”：这种学 C++ 的路子浪费学生的时间，还逼着学生用比实际需要更少的设施、技术和库去处理问题，从而养成糟糕的编程习惯。C++ 有比 C 更强的类型检查，有对新手支持更好的标准库，还有用于错误处理的异常。
- 自底向上：这种路子会妨碍学生学习良好而有效的编程实践。它逼着学生在语言和库支持不足的条件下解决问题，反而助长糟糕、浪费的编程习惯。
- “讲了什么，就得讲全”：这种路子意味着自底向上（对每个碰到的主题都越钻越深）。它用新手毫无兴趣、很可能多年都用不上的技术细节让他们厌烦。一旦你会编程了，技术细节可以到手册里查。手册擅长这个，在概念的入门学习上却糟糕透顶。
- 自顶向下：这种路子从第一原理出发走向细节，容易让读者忽视编程的实践层面，逼他们在还没机会体会高层概念的重要性之前就专注于这些概念。举例来说，在你亲身体会到程序里犯个错误有多容易、改正它又有多难之前，你根本无法领会良好的软件开发原则。
- “抽象优先”：专注于一般原则，把学生保护起来、不接触现实世界那些难缠的约束，会让学生轻视现实世界的问题、语言、工具和硬件限制。这种路子常常靠“教学语言”来支撑，而这些语言日后没法用，还（有意地）把学生与硬件和系统问题隔离开。
- “软件工程原则优先”：这种路子和抽象优先的路子往往有自顶向下路子的毛病：没有具体例子和实践经验，你根本无法领会抽象的价值和良好的软件开发实践。
- “第一天就面向对象”：面向对象编程是组织代码和编程工作的最佳方式之一，但不是唯一有效的方式。特别是我们认为，在类型和算法代码方面打好基础，是领会类与类层次设计的前提。我们从第一天起就使用用户自定义类型（有些人会称之为“对象”），但直到第 6 章才讲怎样设计一个类，直到第 12 章才讲类层次。
- “只管相信魔法”：这种路子靠演示强大的工具和技术，却不把背后的技术和设施介绍给新手。结果学生只能猜——而且常常猜错——事情为什么是这样、使用它们要付出什么代价、在哪里可以合理地使用。这会让人死板地照搬熟悉的工作套路，成为进一步学习的障碍。

自然，我们并不声称这些别的路子从来没用。事实上，对其中的某些具体子主题，我们也采用其中几种做法，因为在那里能看出它们的优点。但作为面向实际应用的编程入门总路线，我们不取它们，而采用我们的替代方案：具体优先、深度优先，并强调概念与技术。

0.2.2 编程与编程语言

我们先教编程，把所选定的编程语言放在次要位置，当作工具。我们这套总体思路可以用于任何通用编程语言。我们的首要目标是帮你学会一般的概念、原则和技术。然而，这些东西无法孤立地体会。举例来说，语法细节、能直接表达的想法种类、工具支持，都因编程语言而异。但是，许多用来产出无缺陷代码的基本技术——比如写出逻辑简单的代码（第 5、6 章）、建立不变量（invariant，§9.4.3）、把接口与实现细节分开（§9.7 和 §14.1-2）——在各种编程语言之间差别很小。

编程技术和设计技术必须借助一门编程语言来学。设计、代码组织和调试不是光靠空想就能获得的技能。你得用某种编程语言写代码，并在其中积累实践经验。这意味着你必须学会一门编程语言的基础。我们说“基础”，是因为那种几周就能学会一门主流工业语言的全部内容的时代已经一去不返了。我们讲授的 C++ 部分，是挑选出来最能直接支撑写出好代码的子集。同时，我们也讲一些你无法回避的 C++ 特性：它们要么是逻辑完整性所必需，要么在 C++ 社区中很常见。

0.2.3 可移植性

人们通常希望写出的 C++ 程序能在多种机器上运行。一些重要的 C++ 应用运行在我们从没听说过的机器上！我们认为可移植性（portability）以及使用多种机器体系结构和操作系统非常重要。本书中几乎每个例子不仅是 ISO 标准 C++，而且可移植。除非特别说明，我们给出的代码应当能在每一种 C++ 实现上工作，并且已经在若干机器和操作系统上测试过。

编译、链接和运行 C++ 程序的具体做法，因系统而异。每逢涉及实现层面的问题就要把每个系统、每个编译器的做法都交代一遍，未免太啰嗦。附录 C 给出了最基本的信息：如何在 Windows 机器上开始使用 Visual Studio 和 Microsoft C++。

如果你在某个流行却相当繁复的 IDE（集成开发环境）上遇到麻烦，我们建议你试试从命令行工作；它简单得令人意外。例如，在 Unix 或 Linux 系统上，用 GNU C++ 编译器编译、链接并运行一个由两个源文件 `my_file1.cpp` 和 `my_file2.cpp` 组成的简单程序，所需的全部命令就是：

```
c++ -o my_program my_file1.cpp my_file2.cpp
./my_program
```

是的，确实就这么多。

0.3 编程与计算机科学

计算机科学就只有编程吗？当然不是！我们之所以提这个问题，只是因为确实有人在这件事上犯糊涂。我们涉及计算机科学的一些重要主题，比如算法和数据结构，但我们的目标是教编程：程序的设计与实现。这既多于也少于大多数人所理解的计算机科学：

- 多于，因为编程涉及许多通常不被看作任何一门科学之组成部分的技术技能；
- 少于，因为我们并不系统地讲解我们所用到的那部分计算机科学的基础。

本书的目标是：成为计算机科学课程的一部分（如果你的志向是成为计算机科学家）；成为软件构建与维护的众多课程中第一门课的基础（如果你的志向是成为程序员或软件工程师）；总而言之，成为更大整体的一部分。

我们通篇依赖计算机科学，也强调原则，但我们把编程当作一门以理论和经验为基础的实用技能来教，而不是当作一门科学。

0.4 创造力与问题求解

本书的首要目标是帮你用代码表达自己的想法，而不是教你怎么产生这些想法。在讲述过程中，我们给出了许多处理问题的实例，通常先分析问题，再逐步改进解法。我们把编程本身看作一种问题求解：只有完整理解了一个问题及其解法，你才能为它写出正确的程序；也只有通过构造并测试程序，你才能确定自己的理解是完整的。因此，编程天生就是获取理解的努力的一部分。不过，我们想通过例子来展示这一点，而不是靠“说教”，也不是靠开出一份详细的问题求解处方。

0.5 征求反馈

我们不认为完美的教科书可能存在；个人的需要差别太大，容不下这样一本书。不过，我们愿意把本书及其配套材料做到我们所能做到的最好。为此，我们需要反馈；一本好教科书不可能脱离读者闭门写成。请把错误、错字、讲得不清楚的地方、缺失的解释等报告给我们。我们也欢迎关于更好的习题、更好的例子，以及该增加哪些主题、该删掉哪些主题等等的建议。建设性的意见会造福后来的读者，我们也会把勘误发布在支持网站上：www.stroustrup.com/Programming。

0.6 参考文献

本节除了列出本章提到的出版物，还包括一些你也许会觉得有用的出版物。

Becker, Pete, ed. *The C++ Standard*. ISO/IEC 14882:2011.

Blanchette, Jasmin, and Mark Summerfield. *C++ GUI Programming with Qt 4, Second Edition*. Prentice Hall, 2008. ISBN 0132354160.

Koenig, Andrew, and Barbara E. Moo. *Accelerated C++: Practical Programming by Example*. Addison-Wesley, 2000. ISBN 020170353X.

Meyers, Scott. *Effective C++: 55 Specific Ways to Improve Your Programs and Designs, Third Edition*. Addison-Wesley, 2005. ISBN 0321334876.

Schmidt, Douglas C., and Stephen D. Huston. *C++ Network Programming, Volume 1: Mastering Complexity with ACE and Patterns*. Addison-Wesley, 2001. ISBN 0201604647.

Schmidt, Douglas C., and Stephen D. Huston. *C++ Network Programming, Volume 2: Systematic Reuse with ACE and Frameworks*. Addison-Wesley, 2002. ISBN 0201795256.

Stroustrup, Bjarne. *The Design and Evolution of C++*. Addison-Wesley, 1994. ISBN 0201543303.

Stroustrup, Bjarne. “Learning Standard C++ as a New Language.” *C/C++ Users Journal*, May 1999.

Stroustrup, Bjarne. *The C++ Programming Language, Fourth Edition*. Addison-Wesley, 2013. ISBN 0321563840.

Stroustrup, Bjarne. *A Tour of C++*. Addison-Wesley, 2013. ISBN 0321958314.

Sutter, Herb. *Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions*. Addison-Wesley, 1999. ISBN 0201615622.

更全面的文献列表见书末的参考文献（Bibliography）部分。

0.7 作者小传

你也许会合理地发问：“这两个想教我编程的家伙是谁？”下面是一些小传。本书由我，Bjarne Stroustrup，撰写；我和 Lawrence “Pete” Petersen 一起设计并讲授了一门大学程度的初学者（一年级）课程，这门课与本书同步开发，用的就是本书的草稿。

Bjarne Stroustrup

我是 C++ 编程语言的设计者和最初的实现者。过去大约 40 年里，我用这门语言以及许多别的编程语言，处理过各种各样的编程任务。我就是喜欢看到优雅而高效的代码用在富有挑战性的应用里，比如机器人控制、图形、游戏、文本分析和网络。我教过设计、编程和 C++，学生几乎涵盖各种能力和兴趣。我是 C++ ISO 标准委员会的创始成员，并担任语言演化工作组的主席。这是我的第一本入门书。我的其他著作，如 *The C++ Programming Language* 和 *The Design and Evolution of C++*，都是为有经验的程序员写的。



我出生在丹麦奥胡斯（Århus）的一个蓝领（工人阶级）家庭，在家乡的大学取得数学与计算机科学方向的硕士学位。我的计算机科学博士学位来自英国剑桥大学。我在 AT&T 工作了大约 25 年，先是在贝尔实验室著名的计算机科学研究中心——Unix、C、C++ 以及许许多多别的东西都在那里诞生——后来在 AT&T 实验室研究部。

我是美国国家工程院院士、ACM Fellow 和 IEEE Fellow。作为有史以来第一位计算机科学家，我获得了 Sigma Xi（科学研究学会）颁发的 2005 年度 William Procter 科学成就奖。2010 年，我获得了奥胡斯大学授予与该大学有关人士的、历史最悠久也最具声望的科学贡献荣誉——Rigmor og Carl Holst-Knudsens Videnskapspris。2013 年，俄罗斯圣彼得堡的国立研究型大学 ITMO 授予我计算机科学名誉博士学位。

工作之外我也有自己的生活。我已婚，有两个孩子，一个是医生，一个是博士后研究员。我读书很多（包括历史、科幻、侦探小说和时事），喜欢大多数种类的音乐（包括古典、摇滚、布鲁斯和乡村）。与朋友共享美食是生活中不可或缺的一部分，我也喜欢到世界各地走访有趣的地方、结识有趣的人。为了能享受美食，我跑步。

更多信息见我的主页：www.stroustrup.com。特别地，在那里你可以查到我的名字怎么读。

Lawrence “Pete” Petersen

2006 年底，Pete 这样介绍自己：“我是一名教师。将近 20 年里，我在德克萨斯 A&M 大学教授编程语言。我五次被学生选为教学优秀奖获得者，并于 1996 年获得工学院校友会颁发的杰出教学奖。我是 Wakonse 教学卓越项目的 Fellow，也是教育者发展学院的 Fellow。

“作为军官的儿子，我在不断搬迁中长大。在华盛顿大学取得哲学学位后，我在军队服役 22 年，当过野战炮兵军官，也做过作战测试的研究分析员。1971 到 1973 年，我在俄克拉何马州锡尔堡的野战炮兵军官高级课程任教。1979 年，我协助组织了一门测试军官培训课程，并于 1978 至 1981 年、1985 至 1989 年间，在美国各地九个不同地点作为首席教官讲授该课程。

“1991 年，我创办了一家小型软件公司，为大学各院系开发管理软件，直到 1999 年。我的兴趣在于教学、设计以及编写真人可用的软件。我在佐治亚理工学院取得工业工程硕士学位，在德克萨斯 A&M 大学取得教育课程与教学硕士学位。我还在 NTS 完成了微型计算机方向的硕士项目。我的博士学位是德克萨斯 A&M 大学的信息与运营管理。



“我和妻子 Barbara 住在德克萨斯州布莱恩。我们喜欢旅行、园艺和招待客人；也尽可能多花时间与儿子们及他们的家人在一起，尤其是和我们的孙辈 Angelina、Carlos、Tess、Avery、Nicholas 和 Jordan。”

令人悲痛的是，Pete 于 2007 年因肺癌去世。没有他，这门课绝不会成功。

后记　大多数章都附有一篇简短的“后记”，试着为本章所讲的内容提供一些视角。我们这样做，是因为明白这些内容可能——而且常常确实——令人生畏，只有在做完习题、读过后面各章（它们会用到本章的想法）、再回过头复习之后，才能完全领会。别慌！放轻松，这是自然而然的，也在意料之中。你不会一天就成为专家，但只要你一路学下去，就能成为一个相当有能力的程序员。在这一路上，你会遇到许多信息、许多例子和许多技术，很多程序员都觉得它们既启发人又有趣。

1 计算机、人与编程

“专精是昆虫的事。”

——R. A. Heinlein

本章要讲的，是我们认为让编程显得重要、有趣、好玩的一些事情，此外还有几条基本的理念与理想。我们还想戳破两个关于编程和程序员的流行神话。这一章现在不妨略读，等以后你被某个编程问题卡住、怀疑这一切是否值得时，再回来重读。

1.1 引言
1.2 软件
1.3 人
1.4 计算机科学
1.5 计算机无处不在
  1.5.1 有屏幕与没屏幕
  1.5.2 航运
  1.5.3 电信
  1.5.4 医学
  1.5.5 信息
  1.5.6 纵向视角
  1.5.7 那又怎样？
1.6 程序员的理想

1.1 引言

和大多数学习一样，学编程也是个先有鸡还是先有蛋的问题：我们想赶紧动手，却又想知道即将学的东西为什么重要。我们想学一门实用技能，却又想确认它不只是一时的时髦。我们想知道自己不会白费时间，却又不想再被一通吹嘘和说教烦到。眼下，你只需把本章中看起来有意思的部分读一读；等日后需要提醒自己这些技术细节在课堂之外有什么用时，再回来翻翻。

本章是我们的一份个人陈述，说的是我们认为编程中哪些地方有意思、哪些地方重要。它解释了是什么让我们在这个领域干了几十年还愿意接着干下去。读这一章，是为了对大致的终极目标有个概念，也对程序员可能是怎样一种人有个概念。一本给初学者的技术书，难免要塞进许多相当基本的东西。本章则把目光从技术细节上抬起来，看看大局：编程为什么是一件值得做的事？编程在我们的文明中扮演什么角色？程序员在哪里能做出让自己引以为傲的贡献？在软件开发、部署与维护这个更大的世界里，编程处在什么位置？当人们谈论“计算机科学”“软件工程”“信息技术”等等时，编程又在其中占哪一块？程序员到底做什么？一名优秀的程序员具备哪些技能？

对学生来说，理解一个想法、一项技术或一章内容，最迫切的理由也许是以好成绩通过考试——但学习的理由绝不止于此！对在软件行业工作的人来说，理解这些东西最迫切的理由，也许是找到能帮上当前项目、又不会惹恼那位掌管下一份薪水、晋升与解雇的老板的东西——但学习的理由同样绝不止于此！当我们觉得自己所做的工作能以某种微小的方式让世界变得更宜于居住时，我们干得最好。

对于那些要持续做上好几年的工作（也就是构成职业与事业的那些“事情”），理想以及更抽象的观念至关重要。

我们的文明运行在软件之上。改进软件、为软件找到新用途，是一个人帮助改善许多人生活的两条途径。编程在其中起着不可或缺的作用。

1.2　软件

好的软件是看不见的。你看不见它，摸不着它，称不出它的重量，也敲不出声响。软件是一组在某台计算机上运行的程序。有时我们看得见那台计算机；更多时候，我们只看得见装着计算机的某个东西，比如电话、相机、面包机、汽车或风力发电机。我们看得见软件做了什么。如果它没做该做的事，我们会恼火，甚至受到伤害；如果它该做的事不合我们的需要，我们同样会恼火，甚至受到伤害。

世界上有多少台计算机？我们不知道，少说也有几十亿台吧。世界上的计算机可能比人还多。服务器、台式机、笔记本电脑、平板电脑、智能手机，还有嵌在各类“小玩意儿”里的计算机，都得算进去。

你每天（或多或少直接地）用到多少台计算机？我的汽车里有三十多台，手机里有两台，MP3 播放器里有一台，相机里有一台。此外还有我的笔记本电脑（你正在读的这页就是用它在写）和台式机。把夏天的暑热和潮湿挡在外面的空调控制器，就是一台简单的计算机。计算机科学系那部电梯也有一台在控制。你若用一台现代电视机，里面某处至少有一台计算机。上上网，你就会通过一个由成千上万台计算机——电话交换机、路由器等等——组成的电信系统，直接接触几十台、甚至几百台服务器。

不，我并没有在后座上放着三十台笔记本电脑开车到处跑！要点在于：大多数计算机并不像大众心目中计算机的样子（带屏幕、键盘、鼠标等等）；它们是嵌在我们所用设备里的小小“零件”。所以，那辆车里没有任何东西看上去像计算机，连一块用来显示地图和行车路线的屏幕都没有（不过这类玩意儿在别的车上很流行）。可它的发动机里装着相当多的计算机，负责燃油喷射控制、温度监测之类的事。助力转向至少涉及一台计算机，收音机和防盗系统里也有几台；我们甚至怀疑，连车窗的升降控制都是由计算机操纵的。更新的车型里还有持续监测胎压的计算机。

你一天里做的事，有多少要靠计算机？你要吃饭；如果你住在现代城市，把食物送到你面前是件大工程，需要规划、运输和仓储方面的小小奇迹。配送网络的管理当然是计算机化的，把它们串起来的通信系统也是。现代农业高度计算机化：牛棚旁边就有用来监测牛群（年龄、健康状况、产奶量等等）的计算机，农机也越来越计算机化，而政府部门各个分支要求的表格多到能让任何一位老实的农民落泪。出了什么事，你可以在报纸上读到详情；那份报纸上的文章当然是在计算机上写的，版面是用计算机排的，（如果你还在读“死树版”）还是用计算机化的设备印出来的——往往先经电子方式传到印厂。书籍的生产也是如此。如果你得通勤，交通流量由计算机监测，为的是（通常是徒劳地）避免堵车。你更愿意坐火车？那列火车同样是计算机化的；有些甚至不用司机就能运行，而列车的各个子系统，比如广播、制动和售票，都涉及大量计算机。今天的娱乐产业（音乐、电影、电视、舞台演出）是计算机的最大用户之一。连非动画电影也大量使用（计算机）动画；音乐和摄影在录制与发行两端也都数字化（也就是用计算机）了。你要是病了，医生开的检查会用到计算机，病历往往是计算机化的，而被送进医院治疗时会遇到的大部分医疗设备里都装着计算机。除非你恰好住在林中一间没有电力驱动的小玩意儿（连灯泡也没有）的小屋里，你就在消耗能源。石油的勘探、开采、加工和配送，每一步都经过一个使用计算机的系统，从深埋地下的钻头一直到你街角的加油机。如果你用信用卡付油钱，又动用了一大群计算机。煤、天然气、太阳能和风能也是同样的情形。

到目前为止举的例子都是“运行层面”的：它们直接参与你正在做的事。比这更远一层，是重要而有趣的设计领域。你穿的衣服、你对着说话的电话、为你煮出最爱那口味咖啡的咖啡机，都是用计算机设计和制造的。现代摄影镜头之所以品质出众，现代日常器具和用品之所以造型精美，几乎全靠基于计算机的设计与生产方法。那些设计我们生活环境的工匠、设计师、艺术家和工程师，已经从许多过去被视为天经地义的物质限制中解放出来。你要是病了，用来治好你的药物也将是用计算机设计出来的。

最后，研究——科学本身——严重依赖计算机。探测遥远恒星奥秘的望远镜，没有计算机就无法设计、建造和操作；它们产生的海量数据，没有计算机也无法分析和理解。一位在野外工作的生物学研究者或许用不着多少计算机（当然，除非他用相机、数字录音机、电话等等），但一回到实验室，数据就得存储、分析、对照计算机模型检验，并传达给同行。现代化学和生物学——包括医学研究——使用计算机的程度，是几年前难以想象的，至今大多数人仍想象不到。人类基因组是由计算机测序的。或者——说得精确些——人类基因组是由人使用计算机测序的。在所有这些例子里，我们都看到计算机是这样一个东西：它使我们能够做到那些没有计算机就会难得多的事情。

这些计算机中的每一台都在运行软件。没有软件，它们只是一堆昂贵的硅、金属和塑料：挡门的砖、船锚和取暖器。那些软件里的每一行，都是某个具体的人写出来的。其中真正被执行过的每一行，起码都还算说得过去，即使谈不上正确。这一切居然能运转，实在令人惊叹！我们说的是数百种编程语言写成的几十亿行代码（程序文本）。让这一切运转起来，花费了惊人数量的工夫，涉及多得无法想象的技能。我们希望对所依赖的几乎每一项服务和每一件器具都做进一步的改进。随便想一项你依赖的服务或器具：你希望它改进成什么样？别的且不说，我们总希望服务与器具更小（或更大）、更快、更可靠、功能更多、更易用、容量更大、更好看、更便宜。你想到的那项改进，多半需要一些编程。

1.3　人

计算机是人造的，供人使用。计算机是一种非常通用的工具，能用于多得无法想象的任务。但要让它对某个人有用，还得有程序。换句话说，在有人——某个程序员——为它写出代码、让它做点有用的事之前，计算机只是一件硬件。我们常常忘记软件。更常忘记的是程序员。

好莱坞以及类似的“流行文化”误导来源，给程序员安上了大体负面的形象。比如，我们都见过那种孤僻、肥胖、丑陋的书呆子，毫无社交能力，沉迷电子游戏，还热衷于侵入别人的计算机。他（几乎总是“他”）想毁灭世界的可能性和想拯救世界的可能性一样大。显然，现实生活中有这类漫画形象温和一些的版本；但据我们的经验，他们在软件开发人员中出现的频率，并不比在律师、警察、汽车销售员、记者、艺术家或政客中更高。

想想你自己生活中知道的那些计算机应用。它们是某个独处暗室的人做出来的吗？当然不是；做出一件成功的软件、一台计算机化的器具或一个系统，要几十、几百甚至几千人扮演一大堆令人眼花缭乱的角色：比如程序员、（程序）设计员、测试员、动画师、焦点小组主持人、实验心理学家、用户界面设计师、分析师、系统管理员、客户关系人员、音响工程师、项目经理、质量工程师、统计学家、硬件接口工程师、需求工程师、安全官、数学家、销售支持人员、故障排查员、网络设计师、方法学家、软件工具管理员、软件库管理员，等等。角色的范围大得惊人，而各组织的头衔又各不相同，这就更让人眼花缭乱：一个组织里的“工程师”，可能是另一个组织里的“程序员”，再换一个组织又成了“开发人员”“技术人员”或“架构师”。甚至有些组织让员工自己挑头衔。这些角色并非都直接涉及编程。不过，我们亲眼见过有人在做上述每一种角色时，把读代码或写代码当作工作中不可或缺的一部

分。此外，一名程序员（承担上述任何一种角色乃至更多角色）可能在很短一段时间里，与应用领域的各类人打交道，比如生物学家、发动机设计师、律师、汽车销售员、医学研究者、历史学家、地质学家、宇航员、飞机工程师、木料场经理、火箭科学家、保龄球馆建造商、记者和动画师（没错，这份名单来自亲身经历）。一个人也可能此时是程序员，在职业生涯的其他阶段又做着与编程无关的工作。

程序员都很孤僻，这个神话就只是个神话。喜欢独自工作的人，会选择最适合独自工作的领域，而且通常会为“打扰”和会议太多而大吐苦水。喜欢与人打交道的人则轻松一些，因为现代软件开发是一项团队活动。这意味着，社交与沟通能力必不可少，其受重视的程度远超那些刻板印象所显示的。在一名程序员（不管你怎么现实地定义“程序员”）最被看重的技能清单上，你会找到这样一种能力：与背景各异的人良好沟通——非正式地谈，在会上谈，书面沟通，以及做正式报告。我们确信，在你完成一两个团队项目之前，你根本不知道编程是什么，也不知道自己是否真的喜欢它。我们喜欢编程的原因之一，就是会遇到许多友善而有趣的人，并且因职业之便可以走访各种各样的地方。

这一切的一个含义是：要做出好的软件，必须有具备各种技能、兴趣和工作习惯的人。我们的生活质量取决于这些人——有时甚至我们自己的性命也取决于他们。没有哪一个人能填满我们在这里提到的所有角色；也没有哪个明智的人会想要所有角色。要点在于，你的选择比你所能想象的宽广得多，而不是你必须做出某个特定的选择。作为个体，你会“漂”向与自己的技能、天赋和兴趣相匹配的工作领域。

我们谈论“程序员”和“编程”，但编程显然只是整体图景的一部分。设计一艘船或一部手机的人，并不把自己看作程序员。编程是软件开发的重要部分，但不是软件开发的全部。同样，对大多数产品来说，软件开发是产品开发的重要部分，但不是产品开发的全部。

我们并不假定你——我们的读者——想成为职业程序员，把余生都用来写代码。即使是最好的程序员——尤其是最好的程序员——大部分时间也不在写代码。理解问题要花大量时间，往往还需要相当的脑力。许多程序员说编程有意思，指的就是这种智力挑战。许多最优秀的程序员还拥有通常不被视为计算机科学范畴的学位。比如，如果你做基因组研究方面的软件，懂一点分子生物学就会让你高效得多。如果你做分析中世纪文学的程序，读一点那类文学、甚至懂一两门相关语言，会大有好处。特别地，一个抱着“我只关心计算机和编程”态度的人，将无法与不做程序员的同事打交道。这样的人不仅会错过人际交往（也就是生活）中最好的部分，还会是个糟糕的软件开发人员。

那么，我们假定什么？编程是一套富有智力挑战的技能，是许多重要而有趣的技术学科的一部分。此外，编程是我们这个世界不可或缺的一部分，所以不懂编程基础，就像不懂物理、历史、生物或文学的基础一样。对编程完全无知的人，只能相信魔法，在许多技术岗位上是危险的。如果你读过《迪尔伯特》（Dilbert），就把那个尖头发的老板当成你不想遇到、更不想变成的那种管理者。另外，编程可以很好玩。

但我们假定你可能会用编程来做什么？也许你会把编程当作继续学习和工作中的关键工具，而不成为职业程序员。也许你会在职业和个人生活中与人打交道，而编程的基本知识会让你占优，比如作为设计师、作家、管理者或科学家。也许你会在学习或工作中以专业水准编程。即使你真的成了职业程序员，也不太可能除了编程什么都不做。

你也许会成为专注计算机的工程师，或者计算机科学家，但即便如此，你也不会“一直在编程”。编程是一种用代码表达想法的方式——一种帮助解决问题的方式。除非你有值得表达的想法、有值得解决的问题，否则它什么都不是，纯粹是浪费时间。

这是一本讲编程的书，我们答应过帮你学会编程，那为什么还要强调非编程的科目、强调编程有限的作用？因为一名好的程序员懂得代码和编程技术在项目中的位置。好的程序员（在大多数时候）是

好的团队成员，会努力弄清代码及其生产过程如何最好地支撑整个项目。举个例子：假设我在做一款新的 MP3 播放器（也许要装进智能手机或平板电脑），而我只关心自己代码的漂亮，以及能提供多少巧妙的功能。我大概会坚持用最大、最强的计算机来跑我的代码。我可能看不起声音编码理论，因为那“不算编程”。我会待在自己的实验室里，而不去走访潜在用户——反正他们的音乐品味肯定很糟，也不会欣赏 GUI（图形用户界面）编程的最新进展。这样做的结果，多半是项目的灾难。更大的计算机会让 MP3 播放器更贵，电池续航多半也更短。编码是数字方式处理音乐必不可少的一环，所以不注意编码技术的进展，就可能让每首歌占用更多内存（同样的输出质量，不同编码的体积相差可达 100%）。无视用户的偏好——不管这些偏好在你看上去多么古怪、多么陈旧——通常只会让用户去买别的产品。写好程序的关键之一，是理解用户的需要，以及这些需要对实现（也就是代码）施加的约束。要补齐这幅糟糕程序员的漫画，只需再加上两点：因为执着于细节而习惯性拖延交付，以及对自己草草测试过的代码的正确性过分自信。我们希望你成为一名好的程序员，对“做出好软件需要什么”有宽广的视野。对社会的价值与个人满足感的钥匙，都在这里。

1.4 计算机科学

即便按最宽泛的定义，编程也最好被看作某种更大事物的一部分。我们可以把它看作计算机科学、计算机工程、软件工程、信息技术或任何其他与软件相关的学科的一个分支。我们把编程看作这些计算机与信息科学与工程领域的使能技术（enabling technology），对物理、生物、医学、历史、文学以及任何其他学术或研究领域也是如此。

以计算机科学为例。1995 年美国政府的“蓝皮书”这样定义它：“对计算系统与计算的系统研究。这一学科所产生的知识体系包括：用于理解计算系统与方法的理论；设计方法学、算法与工具；概念检验的方法；分析与验证的方法；以及知识表示与实现。”如我们所料，维基百科的条目没那么正式：“计算机科学，或称计算科学，研究信息与计算的理论基础，以及它们在计算机系统中的实现与应用。计算机科学有许多子领域；有些强调特定结果的计算（如计算机图形学），另一些（如计算复杂性理论）则关乎计算问题的性质。还有一些专注于实现计算时的挑战。例如，编程语言理论研究描述计算的方法，而计算机编程则把具体的编程语言用于解决具体的计算问题。”

编程是一种工具；它是表达根本问题与实际问题之解法的基础工具，使这些解法可以被检验、通过实验改进并投入使用。编程是想法与理论同现实相遇的地方。正是在这里，计算机科学才能成为一门实验学科，而不只是纯理论，并对世界产生影响。在这个语境中，如同在许多别的语境中一样，编程既要是经过验证的实践的表达，也要是理论的表达。它绝不能沦落为纯粹的胡乱拼凑：随便按什么老办法把代码写出来，只要应付眼前的急需就行。

1.5 计算机无处不在

没有人通晓关于计算机或软件的一切。本节只给你几个例子。也许你会看到自己喜欢的东西。至少，你可能会相信：计算机应用的范围——以及由此而来的编程——远远超出任何个人所能完全把握的程度。

大多数人想到计算机，就是一台接在屏幕和键盘上的灰色小盒子。这类计算机往往擅长游戏、消息与电子邮件，以及播放音乐。另一些计算机叫笔记本电脑，被无聊的商务人士在飞机上用来查看电子表格、玩游戏、看视频。这幅漫画只是冰山一角。大多数计算机在我们看不见的地方工作，是维持

我们文明运转的那些系统的一部分。有些占满整个房间，有些比一枚小硬币还小。许多最有意思的计算机，并不通过键盘、鼠标或类似装置直接与人交互。

1.5.1　有屏幕与没屏幕

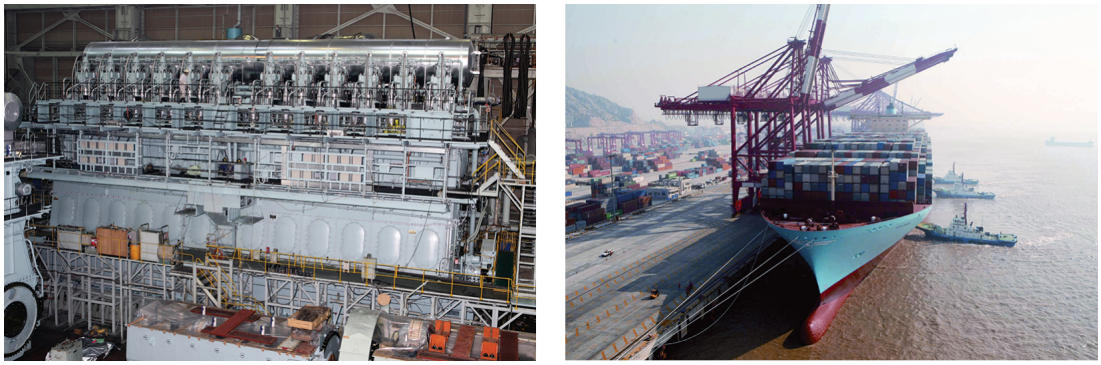
把计算机想象成一个相当大的长方盒子、上面带一块屏幕和一个键盘——这种观念很常见，也常常难以摆脱。不过，请看这两台计算机：



这两件“小玩意儿”（恰好都是手表）首先是计算机。事实上，我们猜想它们基本上是同一种型号的计算机，只是 I/O（输入/输出）系统不同。左边那台驱动一块小屏幕（类似常规计算机的屏幕，只是更小），右边那台驱动几个小电动机，去带动传统的指针，以及一个用于显示日期的数字盘。它们的输入系统是四个按钮（在右边那块表上更容易看到）和一个无线电接收器，后者用于与精度极高的“原子”钟对时。控制这两台计算机的程序，大部分是共用的。

1.5.2　航运

下面两张照片是一台大型船用柴油机和它可能驱动的那种巨轮：



想想计算机和软件在这里哪些地方起关键作用：

- 设计：当然，船和发动机都是用计算机设计的。用途清单几乎列不完，包括建筑与工程图纸、一般计算、空间与零件的可视化，以及零件性能的仿真。
- 建造：现代船厂高度计算机化。船的装配用计算机精心规划，作业也由计算机引导。焊接由机器人完成。特别是，现代双壳油轮若没有在两层船壳之间作业的小型焊接机器人，根本就造不出来——那里实在没有容得下人的空间。为船舶切割钢板，是世界上最早的 CAD/CAM（计算机辅助设计与计算机辅助制造）应用之一。

- *发动机*：这台发动机采用电子燃油喷射，由几十台计算机控制。对于一台 10 万马力的发动机（就像照片里这台），这可不是件小事。例如，发动机管理计算机不断调整燃油混合比，以尽量减少因发动机失调而造成的污染。与发动机（以及船上其他部分）相关的许多泵本身也是计算机化的。
- *管理*：船开往有货可装、有货可卸的地方。船队的调度是一个持续进行的过程（当然由计算机完成），航线会随天气、供求以及港口的空间与装载能力而变化。甚至有些网站可以让你随时查看主要商船的位置。照片里这艘恰好是集装箱船（世界上最大的集装箱船之一；长 397m，宽 56m），但其他类型的大型现代船舶的管理方式也类似。
- *监控*：远洋船在很大程度上是自治的，也就是说，船员能应付抵达下一港口之前可能出现的多数意外。不过，他们也是一个环球网络的一部分。船员能获得相当准确的天气信息（来自——并通过——计算机化的卫星）。他们有 GPS（全球定位系统）以及计算机控制、计算机增强的雷达。如果船员需要休息，多数系统（包括发动机、雷达等）都可以由航运公司的控制室（通过卫星）监控。一旦发现异常，或者与“家里”的连接断了，船员就会收到通知。

想想这段简短描述中明说或隐含的几百台计算机里，有一台出了故障会怎样。第 25 章（“嵌入式系统编程”）会更详细地讨论这一点。为现代船舶写代码，是一项需要技艺而又有趣的活动。它也有用。海运的成本实在低得惊人。你买到一件并非本地制造的东西时，就能体会到这一点。海运一直比陆运便宜；如今原因之一是认真使用了计算机与信息。

1.5.3　电信

下面两张照片是一台电话交换机和一部电话（它恰好也是相机、MP3 播放器、FM 收音机、网页浏览器等等）：





想想计算机和软件在其中起关键作用的地方。你拿起电话“拨号”，被叫的人接听，你们交谈。或者你听到语音信箱，或者用手机相机发一张照片，或者发一条短信（按下“发送”，让电话去拨号）。显然，电话就是一台计算机。如果这部电话（像大多数手机一样）有屏幕，并且支持超出传统“老式电话服务”的功能，比如网页浏览，这一点就尤其明显。实际上，这类电话往往含有好几台计算机：一台管理屏幕，一台与电话系统通信，也许还有更多。

电话里负责管理屏幕、浏览网页等等的那部分，大概是计算机用户最熟悉的：它只是运行一个图形用户界面，去处理“所有那些常见的事”。大多数用户不知道、也基本想不到的是，这部小电话在工作时对话的那个庞大系统。我在德克萨斯拨一个号码，而你正在纽约市度假，几秒钟之内你的电话就响了，我在城市交通的喧嚣中听到你那声“喂！”。许多电话在地球上任意两个地点之间都能玩这个把

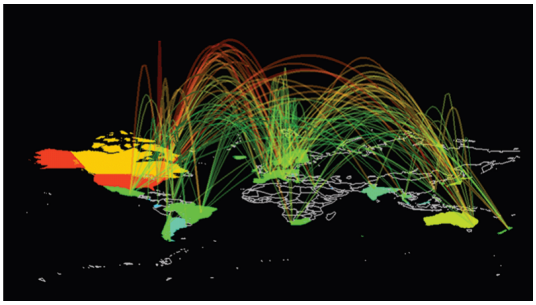
戏，而我们只当它理所当然。我的电话是怎么找到你的？声音是怎么传过去的？声音是怎么编码成数据包的？答案能写满许多本比这本厚得多的书，但它涉及散布在相关地理区域内的几百台计算机上的硬件与软件的组合。如果你运气不好，还会牵涉几颗电信卫星（它们本身也是计算机化系统）——“运气不好”，是因为我们无法完美补偿往返太空那 2 万英里的绕行；光速（因而你的声音的速度）是有限的（光纤电缆要好得多：路径更短、速度更快、承载的数据也多得多）。这套东西大部分运行得非常好；骨干电信系统的可靠性达 99.9999%（举例来说，20 年里有 20 分钟停机——也就是 20/(20*365*24*60)）。我们遇到的麻烦，往往出在手机与最近的主电话交换机之间的通信上。

有软件负责接通电话，把我们说的话切成数据包，通过有线和无线电链路发送，为这些消息选路，从各种故障中恢复，持续监控服务的质量与可靠性，当然还有计费。连记录系统里所有物理部件，都需要大量聪明的软件：什么与什么相连？新系统里要装哪些部件？什么时候该做预防性维护？

可以说，由半独立而又互连的各个系统组成的世界骨干电信系统，是最大、最复杂的人造物。说得更实在一点：请记住，这不只是无聊的老式电话加上几个新花哨。各种基础设施已经合流。它们也是互联网（万维网）赖以运行的底座，是我们的银行与交易系统赖以运行的底座，也是把电视节目送到广播台的通道。所以，我们还可以再加两张照片来说明电信：

这个房间是纽约华尔街美国证券交易所的“交易大厅”，那张地图是互联网骨干网部分线路的表示（完整的地图会乱得没法用）。





碰巧，我们也喜欢数字摄影，喜欢用计算机绘制专门的地图来把知识可视化。

1.5.4　医学

下面两张照片是一台 CAT（计算机轴向断层扫描）扫描仪，以及一间用于计算机辅助手术（也称“机器人辅助手术”或“机器人手术”）的手术室：





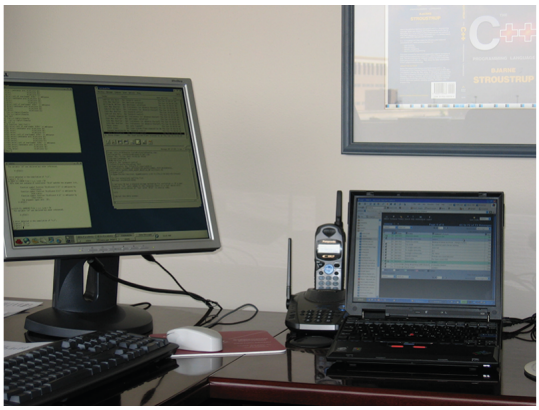
想想计算机和软件在其中起关键作用的地方。扫描仪基本上就是计算机；它发出的脉冲由计算机控制，而读数在施加相当复杂的算法之前只是一堆乱码，算法把它们转换成我们认得出的（三维）人体相关部位图像。要做计算机化手术，还得再往前走几步。各种成像技术被用来让外科医生看到病人

体内，把手术部位大幅放大，或者用比别的方式更好的光照明。借助计算机，外科医生可以使用人手难以握持的精细工具，或者在人手不经过多余切割就到不了的位置操作。微创手术（腹腔镜手术）就是一个简单的例子，它让千百万人的痛苦和康复时间降到最低。计算机还能帮外科医生稳住“手”，从而完成原本做不到的更精细的操作。最后，“机器人”系统可以远程操作，于是医生可以（通过互联网）远程帮助某个人。其中涉及的计算机与编程令人瞠目，复杂而有趣。单是用户界面、设备控制和成像方面的挑战，就足以让成千上万的研究人员、工程师和程序员忙上几十年。

我们听说过一群医生的一次讨论：哪一种新工具对他们的工作帮助最大？CAT 扫描仪？MRI 扫描仪？自动血液分析仪？高分辨率超声仪？PDA？讨论了一阵，这场“竞赛”冒出一个出人意料的“赢家”：即时调阅病历。了解一位病人的病史（既往疾病、以前试过的药物、过敏、遗传问题、总体健康状况、当前用药等等），能简化诊断，把出错的可能降到最低。

1.5.5　信息

下面两张照片是一台普通的 PC（嗯，是两台）和一处服务器农场的一角：





我们一直聚焦于“小玩意儿”，原因很平常：你看不见、摸不着、也听不到软件。我们没法给你看一张漂亮程序的照片，于是就给你看一台运行它的“小玩意儿”。不过，很多软件直接处理的是“信息”。所以，不妨来看看“普通计算机”运行“普通软件”的“普通用途”。

“服务器农场”（server farm）是一批提供网络服务的计算机。运营最先进服务器农场的机构（如 Google、Amazon 和 Microsoft）对自家服务器的细节有些讳莫如深，而服务器农场的规格又不断变化（所以你在网上找到的信息大多已经过时）。不过，那些规格很惊人，足以让任何人相信，编程远不止是在笔记本电脑上算几个数：

- Google 在 25 到 50 个“数据中心”里使用约 100 万台服务器（每一台都比你的笔记本电脑强）。
- 这样一个数据中心设在一座仓库里，仓库的尺寸可能是 60m×100m（约合 200ft×330ft）甚至更大。
- 2011 年，《纽约时报》报道说 Google 的数据中心持续耗电约 2.6 亿瓦（能耗大致相当于拉斯维加斯）。
- 假设一台服务器是 3GHz 四核、24GB 主存。那大约意味着 12×10^{15}Hz 的算力（约每秒 12,000,000,000,000,000 条指令）、24×10^{15} 字节的主存（约 24,000,000,000,000,000 个 8 位字节），每台服务器也许还有 4TB 磁盘，合计 4×10^{18} 字节存储。

我们可能低估了这些数量；到你读这本书的时候，几乎可以肯定低估了。尤其，降低能耗的努力似乎正驱使机器架构朝每台服务器更多处理器、每个处理器更多核的方向发展。GB 是吉字节，也就是约 10^9 个字符。TB 是太字节，约 1000GB，也就是约 10^{12} 个字符。PB 是拍字节（即 10^{15} 字节），正成

为更常见的度量。这是个相当极端的例子，但每一家大公司都在网上运行程序，与用户/客户互动。例子有 Amazon（图书及其他商品销售）、Amadeus（机票与租车）和 eBay（在线拍卖）。数以百万计的公司、组织和个人也在网上有自己的存在。多数并不运行自己的软件，但许多确实在运行，其中不少并不简单。

另一项更传统的庞大计算工作涉及会计、订单处理、工资、记录保存、开票、库存管理、人事档案、学生档案、病人档案等等——也就是几乎每个组织（营利的与非营利的、政府的与私人的）都要保存的记录。这些记录是各自组织的支柱。就计算工作而言，处理这类记录似乎很简单：大多只是把一些信息（记录）存进去、取出来，很少对它做什么加工。例子包括：

- 我 12:30 飞芝加哥的航班还准点吗？
- Gilbert Sullivan 得过麻疹吗？
- Juan Valdez 订的那台咖啡机发货了吗？
- Jack Sprat 在 1996 年（大约那时候）买的是哪种厨房椅？
- 2012 年 8 月，有多少通电话是从 212 区号打出的？
- 一月份卖出了多少台咖啡壶，总价多少？

所涉数据库的庞大规模，使这些系统高度复杂。此外还要加上快速响应（对单个查询往往不到两秒）和正确（至少大多数时候正确）的要求。如今，人们谈论 TB 级数据已不罕见（一个字节是容纳一个普通字符所需的内存量）。那是传统的“数据处理”，它正与“网络”合流，因为如今对数据库的大多数访问都通过网页界面进行。

这类计算机应用常被称为信息处理。它聚焦于数据——往往是大量数据。这就在数据的组织与传输上带来挑战，也在“如何把海量数据以可理解的形式呈现出来”上带来大量有趣的工作：“用户界面”是处理数据时非常重要的一个方面。例如，设想分析一部古代文学作品（比如乔叟的《坎特伯雷故事集》或塞万提斯的《堂吉诃德》），通过比对几十个版本来弄清作者到底写了什么。我们需要按做分析的人给出的各种标准检索文本，并把结果显示得有助于发现要点。说到文本分析，就想到出版：今天，几乎每一篇文章、每一本书、每一本小册子、每一份报纸等等，都是在计算机上制作的。设计能很好支撑这一点的软件，对大多数人来说仍是一个缺少真正好解法的问题。

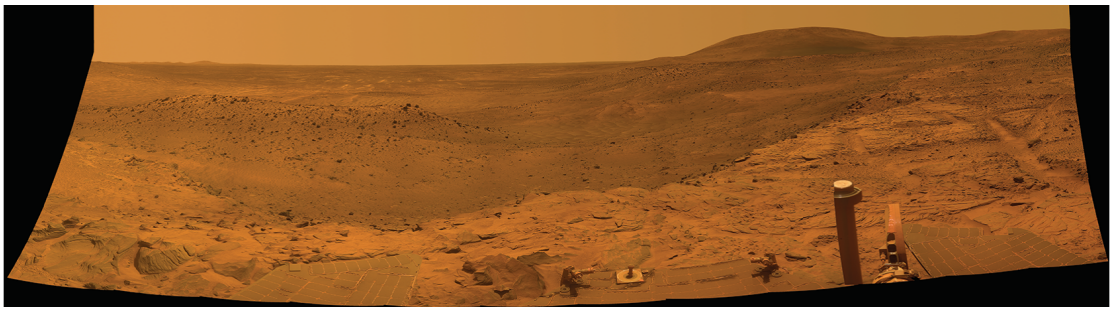
1.5.6 纵向视角

有时据说，古生物学家能从研究一小块骨头重建出完整的恐龙，并描述它的生活方式与自然环境。这也许是夸大，但“看着一件简单的物品，想想它意味着什么”这个想法是有道理的。请看这张由 NASA 的一辆火星车上的相机拍摄的火星地貌照片：

如果你想搞“火箭科学”，成为优秀的程序员是一条路。各个航天计划都雇用大量软件设计师，尤其是那些还懂一点物理、数学、电子工程、机械工程、医学工程等等的人——这些正是载人与不载人航天计划的基础。让那两辆火星车在火星上行驶多年，是我们文明最伟大的技术成就之一。一辆（*Spirit*）传回了六年的数据，另一辆（*Opportunity*）在写作本书时仍在工作，并将在 2014 年 1 月迎来它在火星上的十周年。它们的设计寿命估计是三个月。

这张照片通过一条单程有 25 分钟传输延迟的通信信道传回地球；要让图像以最少的比特数传输而不丢失任何一个比特，需要大量巧妙的编程和高等数学。到了地球，再用算法渲染照片，恢复色彩，并把光学系统和电子传感器造成的畸变降到最低。

火星车的控制程序当然是程序——火星车一次自主行驶 24 小时，执行前一天从地球发去的指令。传输由程序管理。



火星车、传输和照片重建所涉及的各种计算机所用的操作系统是程序，用来写这一章的应用程序也是。运行这些程序的计算机，是用 CAD/CAM（计算机辅助设计与计算机辅助制造）程序设计和生产的。装进这些计算机的芯片，是在用精密工具搭建的计算机化装配线上生产的，而那些工具在设计与制造中也要用到计算机（和软件）。这些漫长制造过程的质量控制涉及大量计算。所有这些代码都是人用高级编程语言写成的，再由编译器翻译成机器码，而编译器本身就是这样一个程序。其中许多程序通过 GUI 与用户交互，并用输入/输出流交换数据。

最后，大量编程投入了图像处理（包括处理火星车传回的照片）、动画和照片编辑（网上流传着一些加了“火星人”的火星车照片版本）。

1.5.7 那又怎样？

这些“花哨而复杂”的应用和软件系统，与学习编程、使用 C++ 有什么关系？联系很简单：许多程序员确实有机会做这类项目。这些正是好的编程能帮助实现的事情。而且，本章用到的每个例子都涉及 C++，以及本书所讲的至少一部分技术。是的，MP3 播放器里、船上、风力发电机里、火星上以及人类基因组计划中，都有 C++ 程序。更多使用 C++ 的应用，见 www.stroustrup.com/applications.html。

1.6 程序员的理想

我们希望程序具备什么？不是某个程序的某个特定功能，而是一般而言我们希望什么？我们希望正确，并且作为正确的一部分，希望可靠。如果程序不做它该做的事，而且做不到让我们可以依赖的程度，那它轻则是严重的麻烦，重则是危险。我们希望它设计良好，能很好地满足真实的需要；如果一个程序所做之事与我们无关，或者它正确地做了某件让我们厌烦的事，那么它正确与否其实无所谓。我们还希望它负担得起；比起平常的交通方式，我也许更愿意要一辆劳斯莱斯或一架公务机，但除非我是个大富翁，成本总会在我的选择中起作用。

这些是软件（器具、系统）的方面，非程序员也能从外部欣赏。它们必须成为程序员的理想，我们必须时刻记着，尤其在开发的早期阶段，如果我们想做出成功的软件。此外，我们还必须关心与代码本身有关的理想：代码必须可维护，也就是说，它的结构必须让没写过它的人也能看懂并做出修改。一个成功的程序会“活”很久（往往是几十年），会被一再修改。例如，它会被移植到新硬件上，会被加上新功能，会被改造以使用新的 I/O 设施（屏幕、视频、声音），用新的自然语言交互，等等。只有失败的程序才永远不会被修改。要做到可维护，程序相对于它的需求必须简单，代码必须直接表达所表达的想法。复杂性——简单性与可维护性之敌——可能是问题本身固有的（那就只好应付），但也可能来自代码中对想法的不良表达。我们必须靠良好的编码风格来避免后者——风格很重要！

这听起来不太难，其实很难。为什么？编程从根本上说是简单的：只要告诉机器它该做什么。那为什么编程可能极具挑战？计算机从根本上说是简单的；它们只会做少数几种操作，比如把两个数相加，

以及根据两个数的比较选择下一条要执行的指令。问题在于，我们并不想让计算机做简单的事。我们想让“机器”做那些难到我们需要帮助的事，可计算机却是吹毛求疵、毫不留情、笨头笨脑的牲口。再者，世界比我们愿意相信的更复杂，所以我们其实并不知道自己所要求的东西会带来什么后果。我们只想让程序“做点类似这样的事”，不想被技术细节烦扰。我们还倾向于假定“常识”。不幸的是，常识在人类当中并不那么常见，在计算机里则完全不存在（尽管一些设计得非常好的程序能在特定且理解充分的场合模仿它）。

顺着这个思路，就有了“编程即理解”的说法：当你能把一个任务编成程序，你就理解了它。反过来，当你彻底理解一个任务，你就能写出程序来完成它。换句话说，我们可以把编程看作彻底理解某个主题的努力的一部分。程序是我们对某个主题之理解的精确表示。

编程时，你会花大量时间去努力理解你想要自动化的那个任务。

我们可以把开发程序的过程描述为四个阶段：

- 分析：问题是什么？用户想要什么？用户需要什么？用户能负担什么？我们需要什么样的可靠性？
- 设计：我们怎样解决问题？系统的总体结构应当是什么？它由哪些部分组成？这些部分之间怎样通信？系统怎样与用户通信？
- 编程：把问题的解法（设计）用代码表达出来。写代码时要满足所有约束（时间、空间、金钱、可靠性等等）。确保代码正确且可维护。
- 测试：通过系统地试用，确保系统在所需的一切情况下都能正确工作。

编程加测试常被称为实现（implementation）。显然，把软件开发这样简单地分成四部分是一种简化。关于这四个主题，每一个都写过高厚的书；关于它们彼此如何关联，写的书更多。有一点很重要：这些开发阶段并不彼此独立，也不严格按顺序发生。我们通常从分析开始，但测试的反馈可以帮助改进编程；让程序跑起来时遇到的问题可能说明设计有问题；而在做设计时，又可能发现分析中一直忽略的问题的某些方面。真正使用系统，通常会暴露分析的弱点。

这里的关键概念是反馈。我们从经验中学习，并根据学到的东西调整行为。这对有效的软件开发至关重要。对任何大项目，我们在开始之前都不可能知道关于问题及其解法的全部。我们可以通过编程来试验想法、获得反馈，但在开发的较早阶段，写下设计想法、试着推演这些想法、拿场景去朋友身上演练，获得反馈更容易（也更快）。我们知道的最好的设计工具是黑板（如果你更喜欢化学气味而不是粉笔灰，就用白板）。能避免的话，绝不要独自做设计！在把想法讲给别人听、试过之前，不要开始写代码。在动手敲键盘之前，先与朋友、同事、潜在用户等等讨论设计和编程技术。你会惊讶地发现，仅仅试着把一个想法讲清楚，就能学到多少东西。毕竟，程序不过是某些想法（用代码）的表达。

同样，当你在实现一个程序时卡住了，就从键盘上抬起头来。想想问题本身，而不是你不完整的解法。找人聊聊：解释你想做什么，以及它为什么行不通。你会惊讶地发现，仅仅把问题仔细讲给别人听，就常常能找到解法。若非必要，不要独自调试（查找程序错误）！

本书的重点是实现，尤其是编程。除了给你大量问题及其解法的例子，我们并不教“问题求解”。问题求解的大部分内容，是认出已知的问题并套用已知的解法技术。只有当大多数子问题都以这种方式处理时，你才有时间去沉溺于激动人心、富有创造性的“跳出框框思考”。所以，我们专注于展示如何用代码清晰地表达想法。

用代码直接表达想法，是编程的一项基本理想。这其实相当明显，但到目前为止，好例子还不够多。我们会反复回到这一点。当代码里需要一个整数，我们把它存进一个 `int`，它提供基本的整数运算。当需要一串字符，我们把它存进一个 `string`，它提供最基本的文本操作。在最基本的层面上，理想是这样的：当我们有一个想法、一个概念、一个实体，一个我们当作“东西”来想、能在白板上画出

来、能在讨论中提及、我们的（非计算机科学的）教科书会谈到的东西，我们就希望那个东西在程序里作为一个有名字的实体（一个类型）存在，并提供我们认为合适的操作。如果要做数学，我们希望有一个表示复数的 `complex` 类型，和一个表示线性代数的 `Matrix` 类型。如果要做图形，我们希望有 `Shape` 类型、`Circle` 类型、`Color` 类型，以及 `Dialog_box`。如果要处理数据流，比如来自温度传感器的数据流，我们希望有一个 `istream` 类型（i 表示 input）。显然，每一个这样的类型都应提供合适的操作，而且只提供合适的操作。这些只是本书中的几个例子。除此之外，我们还提供工具和技术，让你自己构建类型，直接表示你程序中想要的任何概念。

编程一半是实践，一半是理论。只讲实践，你会弄出无法扩展、无法维护的乱摊子。只讲理论，你会弄出没法用（或买不起）的玩具。

想看关于编程理想的另一种视角，以及几位通过与编程语言有关的工作对软件做出重大贡献的人物，见第 22 章“理想与历史”。

要点回顾

要点回顾的问题意在指出一章中讲解的关键思想。看待它们的一种方式，是把它当作习题的补充：习题着眼于编程的实践层面，而要点回顾的问题则试图帮你把想法和概念讲清楚。就此而言，它们很像好的面试题。

1. 什么是软件？
2. 软件为什么重要？
3. 软件在哪些地方重要？
4. 如果某些软件失效，可能出什么问题？举几个例子。
5. 软件在哪些地方起重要作用？举几个例子。
6. 与软件开发相关的工作有哪些？举几个。
7. 计算机科学与编程有什么区别？
8. 在船舶的设计、建造和使用中，哪些地方用到了软件？
9. 什么是服务器农场？
10. 你在网上会问哪几类查询？举几个。
11. 软件在科学中有哪些用途？举几个。
12. 软件在医学中有哪些用途？举几个。
13. 软件在娱乐中有哪些用途？举几个。
14. 我们期望好的软件具备哪些一般性质？
15. 软件开发人员是什么样子？
16. 软件开发的各个阶段是什么？
17. 软件开发为什么会困难？举几个原因。
18. 有哪些软件用途让你的生活更轻松？
19. 有哪些软件用途让你的生活更困难？

术语 这些术语是编程与 C++ 的基本词汇。如果你想听懂别人谈论编程话题，并把自己的想法讲清楚，就该知道每一个的意思。

- `affordability`：负担能力；软件或设备的价格是否在用户可承受的范围内。
- `analysis`：分析；弄清问题是什么、用户想要什么、需要什么、负担得起什么。
- `blackboard`：黑板；做设计时最有效的工具。
- `CAD/CAM`：计算机辅助设计与计算机辅助制造。

- communication：沟通；与背景各异的人交流想法的能力，是程序员的重要技能。
- correctness：正确性；程序确实做它该做的事。
- customer：客户；购买或使用软件产品的人。
- design：设计；决定系统由哪些部分组成、各部分如何协作、如何与用户交流。
- feedback：反馈；从试验与使用中得到的、用于改进的线索。
- GUI：图形用户界面。
- ideals：理想；程序员长期坚持的追求，如正确、可维护、直接表达想法。
- implementation：实现；编程加测试。
- programmer：程序员；编写并维护程序的人。
- programming：编程；用代码表达问题的解法。
- software：软件；在某台计算机上运行的一组程序。
- stereotype：刻板印象；对人的简单化成见，如“程序员都是孤僻的书呆子”。
- testing：测试；系统地检验系统是否在所需的各种情况下都能正确工作。
- user：用户；使用程序的人。

习题

1. 选一项你大多数日子都会做的活动（比如去上课、吃晚饭或看电视），列出计算机直接或间接参与其中的各种方式。
2. 选一个职业，最好是你有些兴趣或有些了解的职业。列出该职业的人所做的、涉及计算机的活动。
3. 把你对习题 2 的清单与一位选了别的职业的朋友交换，并改进他或她的清单。两人都做完后，比较各自的结果。记住：开放式的习题没有完美答案，总有改进的余地。
4. 根据你自己的经历，描述一项没有计算机就不可能完成的活动。
5. 列出你直接使用过的程序（软件应用）。只列你显然在与程序交互的例子（比如在 MP3 播放器上选一首新歌），不列只是碰巧有台计算机参与的情况（比如转动汽车的方向盘）。
6. 列出十项人们做的、完全不涉及计算机（连间接涉及也没有）的活动。这可能比你想的更难！
7. 找出五项目前不用计算机、但你认为将来某个时候会用上的任务。就你选的每一项写几句话加以说明。
8. 写一段说明（至少 100 词，但少于 500 词），讲讲你为什么愿意做一名计算机程序员。反过来，如果你确信自己不愿意做程序员，就把这一点讲清楚。无论哪种情况，都要给出经过深思熟虑的、合乎逻辑的论证。
9. 写一段说明（至少 100 词，但少于 500 词），讲讲除了程序员之外，你愿意在计算机行业里扮演什么角色（不管“程序员”是不是你的首选）。
10. 你认为计算机会不会发展到有意识、能思考、足以与人竞争的地步？写一小段（至少 100 词）支持你的立场。
11. 列出大多数成功程序员共有的若干特征，然后列出人们普遍认为程序员具有的若干特征。
12. 找出本章提到的至少五种计算机程序应用，挑出你觉得最有意思、最想有朝一日参与的那一种。写一小段（至少 100 词）说明你为什么选它。
13. 存储以下内容需要多少内存：(a) 这一页文字，(b) 这一章，(c) 莎士比亚的全部作品？假定一个字节的内存存放一个字符，尽量精确到 20% 左右。
14. 你的计算机有多少内存？主存呢？磁盘呢？

后记　我们的文明运行在软件之上。软件是一个多样性无与伦比的领域，充满了做有趣的、对社会有用的、有利可图的工作的机会。当你接触软件时，请以有原则、认真的态度对待它：你要成为解法的一部分，而不是给问题添乱。

我们显然对渗透在我们技术文明中的软件范围之广心怀敬畏。当然，并非所有软件应用都在行善，但那是另一回事。这里我们想强调的是软件多么无处不在，我们日常生活中所依赖的东西有多少要靠软件。那些软件都是由像我们一样的人写出来的。所有参与构建这里简要提及的那些软件的科学家、数学家、工程师、程序员等等，起步时都和你现在一样。

现在，让我们回到学习编程所需技术技能这一脚踏实地的事情上来。如果你开始怀疑这一切是否值得你辛苦付出（大多数爱思考的人迟早都会这样怀疑），就回来重读本章、前言以及第 0 章（“读者须知”）的片段。如果你开始怀疑自己能否应付这一切，请记住：已经有千百万人成功地成为称职的程序员、设计师、软件工程师等等。你也可以。

第一部分

基础

2 Hello, World!

“编程是靠写程序学会的。”

——Brian Kernighan

这里，我们给出一个真正能干点事的最简单的 C++ 程序。写这个程序的目的是：

- 让你试一试自己的编程环境
- 让你对“怎样才能让计算机为你做事”有第一次亲身感受

为此，我们介绍程序（program）的概念、用编译器（compiler）把程序从人类可读的形式翻译成机器指令的想法，以及最后执行这些机器指令的过程。

2.1 程序

2.2 经典的第一个程序

2.3 编译

2.4 链接

2.5 编程环境

2.1 程序

要让计算机做点什么，你（或别的什么人）必须确切地——以事无巨细的细节——告诉它要做什么。这样一份“要做什么”的说明就叫作程序（program），而编程（programming）就是编写并测试这类程序的活动。

从某种意义上说，我们都已经“编过程”了。毕竟，我们都给过别人任务说明，比如“怎么开车去最近的电影院”“怎么找到楼上的卫生间”“怎么用微波炉热一顿饭”。这类说明与程序之间的差别只是精确程度的问题：人们会用常识来弥补说明的含糊，计算机却不会。比如，“在走廊右转，上楼梯，就在你左手边”很可能是去楼上卫生间的一份不错的说明。然而细看这几句简单的说明，你会发现文法松垮、指示不全。人类很容易将其补全。举例来说，假如你正坐在桌旁，向别人问去卫生间的路，没有人需要告诉你得先从椅子上起身才能走到走廊，怎么绕过（而不是跨过或从桌子底下钻过）桌子，不要踩到猫，等等。也不必有人叮嘱你别带着刀叉就往那边走，或者记得开灯，免得看不清楼梯。进卫生间之前先把门打开，多半也不用别人教。

与此相反，计算机实在很笨。每件事都必须精确、详尽地向它描述。再想想“在走廊右转，上楼梯，就在你左手边”这句说明。走廊在哪儿？什么是走廊？什么是“右转”？什么楼梯？怎么上楼梯？（一级一级上？两级两级上？顺着扶手滑上去？）我左手边是什么？它什么时候在我左手边？要能对计算机精确地描述“事物”，我们需要一种精确定义的语言，它有特定的文法（英语的结构太松散，做不到这一点），并且有一套针对我们想让计算机执行的那些动作类型的良定义词汇表。这样的语言就叫作程序设计语言（programming language），而 C++ 就是一种程序设计语言，它是为很宽范围内的编程任务而设计的。

如果你想更多地从哲学层面了解计算机、程序和编程，请（重）读第 1 章。这里，我们来看看一些代码，先从一个非常简单的程序出发，再看看让它在你的计算机上运行所需的工具和技术。

2.2 经典的第一个程序

下面是经典的第一个程序的一个版本。它把“Hello, World!”写到你的屏幕上：

```
// 本程序向显示器输出消息 "Hello, World!"
#include "std_lib_facilities.h"

int main()                    // C++ 程序从执行函数 main 开始
{
    cout << "Hello, World!\n";    // 输出 "Hello, World!"
    return 0;
}
```

把这段文本看作我们交给计算机去执行的一组指令，就像我们把一份菜谱交给厨子照着做，或者照着一份组装说明书一步步把一个新玩具装起来。下面从这一行开始，逐行讨论这个程序做的事情：

```
cout << "Hello, World!\n";    // 输出 "Hello, World!"
```

这一行才是真正产生输出的语句。它打印字符 Hello, World!，然后再打出一个换行；也就是说，写出 Hello, World! 之后，光标会移到下一行的开头。光标（cursor）就是一个闪烁的小字符或小横线，指示你下一个字符的键入位置。

在 C++ 中，字符串字面值（string literal）用双引号定界；也就是说，`"Hello, World!\n"` 是一个字符串。`\n` 是一个“特殊字符”，表示换行。名字 cout 指的是标准输出流。用输出运算符 `<<` “放进 cout”的字符会出现在屏幕上。名字 cout 读作“see-out”，是“character output stream”（字符输出流）的缩写。你会发现缩写在编程中相当常见。当然，第一次见到缩写并不得不记住它时，会觉得有点讨厌；但一旦开始反复使用，它们就成了第二天性，而且对保持程序文本简短、易管理至关重要。

那一行的末尾

```
// 输出 "Hello, World!"
```

是一条注释。在一行内，写在记号 `//`（也就是两个称为“斜杠”的字符 /）之后的任何内容都是注释。注释会被编译器忽略，写它是为了阅读代码的程序员。这里，我们用这条注释告诉你那一行的开头部分实际做了什么。

写注释，是为了说明程序打算做什么，并总体上提供一些对人有用、却无法直接用代码表达的信息。最能从你代码的注释中受益的人，多半就是你自己——等你下周或明年回到这份代码前，已经想不起当初为什么那样写的时候。所以，请好好为你的程序写文档。在 §7.6.4 中，我们会讨论什么样的注释才算好注释。

程序是写给两类读者的。显然，我们写代码是让计算机执行的。然而，我们还要花大量时间阅读和修改代码。因此，程序员也是程序的一类受众。所以说，写代码也是人与人交流的一种形式。实际上，把代码的人类读者当作首要受众来看待是有道理的：如果他们（我们）都觉得代码不易理解，这代码多半永远也变不成正确的。所以请别忘了一点：代码是给人读的——尽你所能让它可读。无论如何，注释只是为了人类读者而写的；计算机根本不会看注释里的文字。

程序的第一行是一条典型的注释；它只是告诉人类读者这个程序打算做什么：

```
// 本程序向显示器输出消息 "Hello, World!"
```

这类注释很有用，因为代码本身说的是程序做什么，而不是我们想让它做什么。而且，把一个程序（大致）应该做什么解释给一个人听，通常比把它（细致地）用代码表达给计算机简洁得多。这样的注释往往是我们写程序时写下的第一部分内容。就算别的用处没有，它至少提醒我们想做什么。

下一行

```
#include "std_lib_facilities.h"
```

是一条“`#include` 指令”。它指示计算机把一个名叫 std_lib_facilities.h 的文件中的设施提供出来（“包含进来”）。这个文件是我们为了便于使用所有 C++ 实现（“C++ 标准库”）中可用的设施而写的；其内容我们会随进度解释。它是完全普通的标准 C++，只是包含了一些我们不想再让你烦心十几个章节的细节。对这个程序来说，std_lib_facilities.h 的重要性在于，它让标准 C++ 的流 I/O 设施可用；这里我们只用到标准输出流 cout 和它的输出运算符 `<<`。用 `#include` 包含进来的文件通常带后缀 `.h`，称为头（header）或头文件（header file）。头文件中包含 cout 之类的术语的定义，我们的程序会用到它们。

计算机怎么知道从哪里开始执行程序？它会找一个名叫 main 的函数，从那里开始执行它找到的指令。这就是我们的“Hello, World!”程序中的 main 函数：

```
int main()                      // C++ 程序从执行函数 main 开始
{
    cout << "Hello, World!\n";     // 输出 "Hello, World!"
    return 0;
}
```

每个 C++ 程序都必须有一个叫 main 的函数，来告诉程序从哪里开始执行。函数基本上是一组有名字的指令，计算机按它们写出的顺序依次执行。函数有四个部分：

- 一个返回类型（return type），这里是 `int`（意为“整数”），它指明函数执行后——如果有的话——会向调用者返回什么类型的结果。`int` 这个词是 C++ 的保留字（关键字），因此 `int` 不能再用作任何其他东西的名字（参见 §A.3.1）。
- 一个名字，这里是 `main`。
- 一个用圆括号括起来的参数列表（parameter list，参见 §8.2 与 §8.6），这里是 `()`；本例中参数列表为空。
- 一个由一对“花括号”`{ }` 括起来的函数体（function body），其中列出函数要执行的动作（称为语句）。

由此可知，最小的 C++ 程序就是

```
int main() { }
```

不过它没什么用，因为它什么都不做。我们的“Hello, World!”程序中的 main()（“main 函数”）在其函数体中有两条语句：

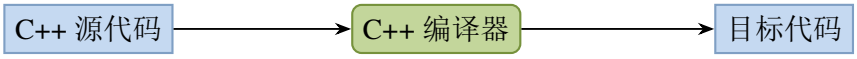
```
cout << "Hello, World!\n";    // 输出 "Hello, World!"
return 0;
```

它会先把 Hello, World! 写到屏幕上，然后向调用它的那一方返回值 0（零）。由于 main() 是由“系统”调用的，我们不会去用这个返回值。不过，在某些系统（尤其是 Unix/Linux）上，它可以用来检查程序是否成功。main() 返回零（0）表示程序成功终止。

C++ 程序中指明某个动作、又不是 `#include` 指令（或其他预处理指令；参见 §4.4 和 §A.17）的部分叫作语句（statement）。

2.3 编译

C++ 是一种编译型语言。这意味着，要让程序运行起来，你必须先把它从人类可读的形式翻译成机器能够“理解”的东西。这一翻译由一个叫作编译器（compiler）的程序完成。你读和写的东西叫作源代码（source code）或程序文本（program text），计算机执行的东西叫作可执行代码（executable）、目标代码（object code）或机器代码（machine code）。通常，C++ 源代码文件用后缀 `.cpp`（例如 hello_world.cpp）或 `.h`（如 std_lib_facilities.h），目标代码文件用后缀 `.obj`（Windows 上）或 `.o`（Unix 上）。单说“代码（code）”这个词是有歧义的，容易引起混淆，只有在含义显而易见时才谨慎使用。除非特别指明，我们用 code 表示“源代码”，甚至指“除去注释之外的源代码”，因为注释只是给我们人类看的，生成目标代码的编译器并不看它。



编译器读入你的源代码，并试图理解你写的东西。它会检查程序在文法上是否正确、每个词是否都有定义过的含义，以及有没有不实际执行程序就能发现的明显错误。你会发现编译器对语法相当挑剔。我们的程序若少掉任何细节——比如少了一个 `#include` 文件、一个分号或一个花括号——都会引发错误。同样，编译器对拼写错误绝对的零容忍。下面用一系列例子来说明这一点，每个例子都只含一个小错误。每个错误都是我们常犯的一类错误的例子：

```
// 这里没有 #include
int main()
{
    cout << "Hello, World!\n";
    return 0;
}
```

我们没有包含某个东西来告诉编译器 cout 是什么，于是编译器报错。要改正，就加上一个头文件：

```
#include "std_facilities.h"
int main()
{
    cout << "Hello, World!\n";
    return 0;
}
```

遗憾的是，编译器又抱怨了：我们把 std_lib_facilities.h 拼错了。编译器同样会挑下一段代码的毛病：

```
#include "std_lib_facilities.h"
int main()
{
    cout << "Hello, World!\n;
    return 0;
}
```

我们没有用 " 来终止字符串。编译器同样会挑下一段代码的毛病：

```
#include "std_lib_facilities.h"
integer main()
{
    cout << "Hello, World!\n";
    return 0;
}
```

在 C++ 中用的是缩写 `int`，而不是单词 integer。编译器也不喜欢下面这段代码：

```
#include "std_lib_facilities.h"
int main()
{
    cout < "Hello, World!\n";
    return 0;
}
```

我们用了 `<`（小于运算符），而不是 `<<`（输出运算符）。编译器还会挑下一段代码的毛病：

```
#include "std_lib_facilities.h"
int main()
{
    cout << 'Hello, World!\n';
    return 0;
}
```

我们用了单引号而不是双引号来定界字符串。最后，编译器会对下面的代码报错：

```
#include "std_lib_facilities.h"
int main()
{
    cout << "Hello, World!\n"
    return 0;
}
```

我们忘了用分号终止输出语句。请注意，许多 C++ 语句都以分号（`;`）终止。编译器需要这些分号，才能知道一条语句在哪里结束、下一条从哪里开始。目前还没有一种真正简短、完全正确又非技术化的办法来概括哪里需要分号。眼下请照抄我们的用法，它可以概括为：“在每个不以右花括号（`}`）结尾的表达式后面放一个分号。”

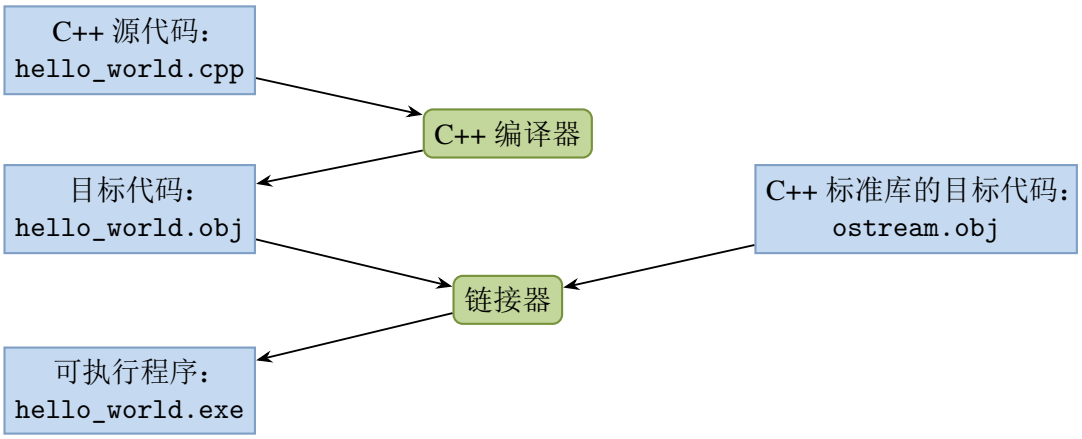
我们为什么要花上两页宝贵的篇幅、占用你几分钟宝贵的时间，在一个平凡的程序上展示一堆琐碎错误的例子？关键在于说明一点：你——和所有程序员一样——会花很多时间在程序源文本中找错误。大多数时候，我们看的就是带错误的文本。毕竟，如果确信某段代码是正确的，我们通常早转去看别的代码，或者干脆歇着去了。计算机先驱们当年竟会犯错误、还得把相当大一部分时间花在找错上，这是一件让当年的很多人大吃一惊的事。如今大多数编程新手对此仍然吃惊。

编程的时候，你有时会被编译器气得不轻。它有时似乎在抱怨一些无关紧要的细节（比如漏了一个分号），或者抱怨一些你认为“显然正确”的东西。然而，编译器通常是对的：当它给出错误信息、拒绝从你的源代码生成目标代码时，你的程序就确实有不太对劲的地方；也就是说，你所写内容的含义没有被 C++ 标准精确定义。

编译器没有常识（它不是人），而且对细节非常挑剔。正因为它没有常识，你也不希望它去猜：某个“看上去凑合”但不符合 C++ 定义的东西，你到底是什么意思。如果它猜了，而且猜出来的意思和你的不一样，你可能会花上大量时间，去追究为什么程序没做你以为已经告诉它要做的事。归根结底，编译器替我们免除了许多自己招来的麻烦；它省下的麻烦，远比它带来的多。所以请记住：编译器是你的朋友；编程时它可能就是你最好的朋友。

2.4 链接

一个程序通常由几个彼此独立的部分组成，这些部分往往由不同的人开发。例如，“Hello, World!”程序就由我们写的部分加上 C++ 标准库中的部分组成。这些独立的部分（有时称为翻译单元，translation unit）必须分别编译，得到的目标代码文件还必须链接（link）在一起，构成一个可执行程序。把这些部分链接起来的程序（不出所料）就叫作链接器（linker）：



请注意，目标代码和可执行文件在不同系统之间是不可移植的。例如，为 Windows 机器编译得到的是 Windows 的目标代码，它在 Linux 机器上运行不了。

库（library）就是一些代码——通常由别人写好——我们借助在一个被 `#include` 的文件里找到的声明（declaration）来使用它们。声明是一种程序语句，它规定一段代码如何使用；我们稍后会详细考察声明（例如 §4.5.2）。

编译器发现的错误叫编译期错误（compile-time error），链接器发现的错误叫链接期错误（link-time error），程序运行之后才发现的错误叫运行期错误（run-time error）或逻辑错误（logic error）。一般来说，编译期错误比链接期错误更容易理解和修复；而链接期错误通常又比运行期错误和逻辑错误更容易发现和修复。第 5 章会更详细地讨论错误及其处理方法。

2.5 编程环境

编程时，我们使用程序设计语言；还用编译器把源代码翻译成目标代码，用链接器把目标代码链接成可执行程序。此外，还要用某个程序把源代码文本输入计算机并编辑它。这些还只是构成程序员工具集（或称“程序开发环境”）的头几样、也是最关键的几样工具。

如果你在命令行窗口中工作——许多职业程序员就是这么做的——就必须自己动手发出编译和链接命令。而如果你使用 IDE（“交互式开发环境”或“集成开发环境”）——许多职业程序员也同样这么做——那么只要在正确的按钮上单击一下就行了。如何在你所用的 C++ 实现上编译和链接，参见附录 C。

IDE 通常带一个编辑器，其上有一些有用的功能，比如用颜色编码来帮助区分注释、关键字和程序源代码的其他部分，此外还有帮你调试（debug）代码、编译并运行它的设施。调试（debugging）就是在程序中找出错误并清除它们的活动；一路学下去，你会不断听到这个词。

使用本书时，你可以用任何提供了最新的、符合标准的 C++ 实现的系统。我们所说的内容，只需极小的改动，就对所有 C++ 实现都成立，代码在任何地方都能运行。在我们的工作中，我们使用了好几种不同的实现。

小试身手

到目前为止，我们谈了编程、代码和工具（比如编译器）。现在，你得让一个程序真正跑起来。这是本书中、也是学习编程过程中至关重要的一步。正是从这里，你开始培养实用的技能和良好的编程习惯。本章的这部分练习围绕熟悉你的软件开发环境展开。一旦让“Hello, World!”程序跑了起来，你就跨过了作为程序员的第一个重要里程碑。

小试身手的目的，是帮你建立或巩固实用的编程技能，并让你积累使用编程环境工具的经验。典型的小试身手是对同一个程序做的一系列修改，把它从完全平凡的东西“养成”某个真实程序中有用的一部分。传统的一组习题是用来考验你的主动性、机敏或创造性的；与之相反，小试身手几乎不需要你发挥创造性。通常，步骤的先后次序至关重要，而每一步都应当容易（甚至被视为平凡）。请不要自作聪明跳过任何步骤；一般而言，那样只会拖慢你，甚至把你绕糊涂。

你也许觉得自己把书里的每句话都读懂了，也把指导教师或老师讲给你的每件事都听懂了，但要发展编程技能，重复和练习必不可少。就这一方面而言，编程与体育、音乐、舞蹈或者任何以技能为本的手艺一样。想象有人在这些领域中不坚持定期练习就去参加比赛，你自然知道他们会有怎样的表现。持续的练习——对专业人士来说，意味着终身不断的练习——是培养并维持高水平实用技能的唯一途径。

所以，无论多么想抄近道，都不要跳过小试身手；它们是学习过程中必不可少的一环。就从第一步开始，一步一步做下去，每做一步就测试一步，确保自己做对了。

如果你对所用语法的某些细节还不能全弄明白，不必惊慌，也不妨大胆向教师或朋友求助。坚持下去，把所有小试身手和相当数量的习题都做完，到时候一切自会明朗。

那么，这就是你的第一个小试身手：

1. 前往附录 C，按其中要求的步骤设置一个工程。建立一个名为 `hello_world` 的空 C++ 控制台工程。

2. 按照下述规定的样子，一字不差地键入 hello_world.cpp，把它保存到你的练习目录（文件夹）中，并把它加入 hello_world 工程。

```
#include "std_lib_facilities.h"
int main()                  // C++ 程序从执行函数 main 开始
{
    cout << "Hello, World!\n";    // 输出 "Hello, World!"
    keep_window_open();         // 等待输入一个字符
    return 0;
}
```

调用 keep_window_open() 在某些 Windows 机器上是必要的，用来防止窗口在你还来不及读输出之前就关闭。这是 Windows 的怪癖/“特性”，不是 C++ 的。我们在 std_lib_facilities.h 中定义了 keep_window_open()，为的是让编写简单的文本程序少费点事。

到哪里去找 std_lib_facilities.h？如果你在上课，问你的老师；否则，从我们的支持网站 www.stroustrup.com/Programming 下载它。可是，假如既没有老师也上不了网，怎么办？只有在这种情况下，才把 #include 指令替换成

```
#include<iostream>
#include<string>
#include<vector>
#include<algorithm>
#include<cmath>
using namespace std;
inline void keep_window_open() { char ch; cin>>ch; }
```

其中直接使用了标准库，可以支撑你用到第 5 章；详细解释在后面（§8.7）。

3. 编译并运行“Hello, World!”程序。很可能有地方不对劲。第一次使用新的编程语言或新的编程环境就一切顺利的情况极为罕见。找出问题，把它改正！在这个节骨眼上向更有经验的人求助是明智的，但务必弄懂人家给你演示的东西，好在继续往前走之前能独立完成全过程。

4. 到现在，你多半已经碰上过一些错误，并且不得不改正它们。现在该进一步熟悉你的编译器的错误检测和错误报告设施了！试试 §2.3 里的那六个错误，看看你的编程环境会有什么反应。再想出至少五种你键入程序时可能犯的错误（比如漏写 keep_window_open()、键入单词时把 Caps Lock 键一直开着，或者把分号误敲成逗号），逐个试一遍，看看这些版本的程序在编译和运行时会出现什么。

要点回顾

这些要点回顾问题的基本用意，是给你一个机会，检视自己是否注意到了并理解了本章的关键内容。回答问题可能需要回头查阅正文；这很正常，也在预料之中。可能需要重读整节的内容；这同样正常，不必意外。不过，如果你不得不重读整章，或者每个回顾问题都让你犯难，那就应该考虑一下自己的学习方式是否有效：是不是读得太快了？是不是该停下来做做“试一试”？是不是该约位朋友一块学，好一起讨论书中对各种问题的解释？

1. “Hello, World!”程序的目的是什么？

2. 说出函数的四个组成部分。

3. 说出一个必须出现在每个 C++ 程序中的函数。

4. 在“Hello, World!”程序中，`return 0;` 这一行起什么作用？

5. 编译器的用途是什么？

6. #include 指令的用途是什么？

7. 在 C++ 中，文件名末尾的 .h 后缀意味着什么？

8. 链接器为你的程序做什么？

9. 源文件和目标文件有什么区别？

10. 什么是 IDE？它为你做了什么？

11. 既然你已经理解了教科书中的全部内容，为什么还有必要练习？

大多数要点回顾问题都能在其所在的章节中找到明确的答案。不过，我们偶尔也会安排一些问题，提醒你回忆来自其他章节的相关内容，有时甚至涉及本书之外的世事。我们认为这很公平：写好软件、并思考这样做意味着什么，其间的门道远比单独一章或一本书所能容纳的多。

术语

这些术语收集的是编程和 C++ 的基本词汇。如果你想听懂别人谈论编程话题时说的话，也想把自己的想法讲清楚，就应当知道每个术语的含义。

//	executable	main()
<<	function	object code
C++	header	output
comment	IDE	program
compiler	#include	source code
compile-time error	library	statement
cout	linker	

你也许愿意逐步编一份用自己的话写成的词汇表来充实自己。做法是对每一章都重复做下面的习题 5。

习题

我们把小试身手和习题分开列出；做任何习题之前，先完成本章的小试身手。这样做会为你省下时间。

1. 修改刚才的程序，让它输出下面两行：

```
Hello, programming!
Here we go!
```

2. 在你所学内容的基础上，写一个程序，为计算机列出找到楼上卫生间的操作步骤（§2.1 中讨论过的那件事）。你能想到哪些步骤是人会想当然、计算机却不会的吗？把它们加进你的清单。这是“像计算机一样思考”的良好开端。警告：对大多数人来说，“去卫生间”是一条相当够用的指示。可对一个没见过房子或卫生间的人（想象一个石器时代的人，不知怎么被传送到了你的餐厅里）来说，所需指示的清单可能会非常长。请最多写一页。为了便于读者理解，你可以加上对你所设想那所房子格局的简短描述。

3. 写一份说明：如何从你宿舍、公寓、房子（不管是什么住处）的前门走到你教室的门（假定你在某所学校上学；若不是，就另选一个目标）。请一位朋友试着照说明走一走，并在走的过程中一路标注改进意见。为了保住朋友，把这份说明交给朋友之前先“实地测试”一下，也许是个好主意。

4. 找一本好菜谱。读一读烤蓝莓玛芬的做法（如果你所在的国家把“蓝莓玛芬”当成一道陌生稀奇的外国点心，就换一种更熟悉的点心）。请注意：只要有一点帮助和指点，世界上大多数人都能烤出美味的蓝莓玛芬；这算不上高级或困难的精细烹饪。然而对本书作者来说，本书习题中很少有比这更难的。多加练习，你能做到的事会连你自己都吃惊。

- 重写这些说明，使每一个单个动作都自成一个带编号的段落。小心列出每一步用到的所有原料和所有厨具。当心那些关键细节，比如所需的烤箱温度、预热烤箱、准备玛芬烤盘、给烹饪计时的方式，以及从烤箱里取出玛芬时对双手的保护。
- 从烹饪新手的视角看待这些说明（如果你不是新手，请一位不会做饭的朋友帮忙）。把书的作者（几乎必然是一位有经验的厨师）因为觉得“显而易见”而省略的步骤补上。
- 为用到的术语建一份词汇表。（什么是玛芬烤盘？预热有什么用？你说的“烤箱”是什么意思？）
- 现在就烤上一些玛芬，享受你的成果。

5. 为“术语”一节中的每个术语写一个定义。先试试不看本章能不能写出来（多半不行），然后翻阅本章去找定义。你会发现自己的第一版和书里的版本有差别，这多半会挺有意思。也可以查阅某个合适的在线词汇表，比如 www.stroustrup.com/glossary.html。先凭自己之力写下定义、再去查证，这会强化你通过阅读学到的东西。如果为了写出定义不得不重读某一节，那只会更有助于你理解。放心用自己的话来写定义，其详细程度以你自己觉得合理为宜。通常，在主定义后面给出一个例子会很有帮助。你也许愿意把定义存到一个文件里，以便日后能从后续各章的“术语”一节中继续补充。

后记

“Hello, World!”程序有什么了不起的？它的目的，是让我们熟悉编程的基本工具。每当接触一个新工具，我们都倾向于先做一个极其简单的例子，比如“Hello, World!”。这样，我们把学习分成两步：先用一个平凡的程序学会工具的基本用法，然后再去学更复杂的程序，不受工具的干扰。同时学习工具和语言，比先学一样再学另一样要难得多。把一件复杂任务分解成一系列小的（因而更易掌控的）步骤来简化学习，这种思路并不只属于编程和计算机；它在生活的大多数领域都很常见、很有用，尤其是那些涉及实用技能的领域。

3 对象、类型与值

机缘偏爱有准备的人。
——路易·巴斯德

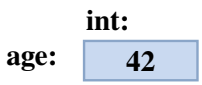
本章介绍在程序中存储和使用数据的基本方法。为此，我们首先集中讨论如何从键盘读入数据；在建立对象、类型、值和变量的基本概念之后，我们引入若干运算符，并给出大量使用 `char`、`int`、`double` 和 `string` 类型变量的例子。

3.1 输入
3.2 变量
3.3 输入与类型
3.4 运算与运算符
3.5 赋值与初始化
3.5.1 一个例子：检测重复的单词
3.6 复合赋值运算符
3.6.1 一个例子：查找重复的单词
3.7 名字
3.8 类型与对象
3.9 类型安全
3.9.1 安全转换
3.9.2 不安全转换

3.1 输入

“Hello, World!”程序只是往屏幕上写东西：它有输出，却不读取任何内容，也不会从用户那里得到输入。这实在有点乏味。真实的程序往往要根据我们给定的输入来产生结果，而不是每次执行都一成不变地做同一件事。

要读取数据，就得有个读取的目的地；也就是说，需要在计算机内存中有一个位置来存放读入的内容。我们把这样的“位置”称为对象（object）。对象是内存中的一块区域，它带有类型（type），类型说明了可以放入其中的信息种类。有名字的对象称为变量（variable）。例如，字符串放入 `string` 变量，整数放入 `int` 变量。你可以把对象想象成一个“盒子”，可以往里面放入该对象类型的值：



上图表示一个名为 `age`、类型为 `int` 的对象，其中存放着整数值 42。使用 `string` 变量，我们可以从输入中读入一个字符串并原样写出，如下所示：

```
// 读入并写出一个名字
#include "std_lib_facilities.h"

int main()
{
    cout << "Please enter your first name (followed by 'enter'):\n";
    string first_name;    // first_name 是一个 string 类型的变量
    cin >> first_name;    // 把字符读入 first_name
    cout << "Hello, " << first_name << "!\n";
}
```

#include 和 main() 在第 2 章已经见过了。由于这个 #include 是我们所有程序（直到第 12 章）都需要的，为了避免干扰，书中通常不再把它写出来。类似地，我们有时会给出一些代码片段，它们只有在放入 main() 或其他函数中才能运行，就像这样：

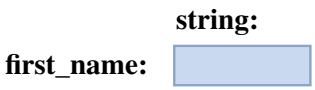
```
cout << "Please enter your first name (followed by 'enter'):\n";
```

我们假定你已经明白如何把这样的代码放进一个完整的程序中去做测试。

main() 的第一行只是写出一条消息，鼓励用户输入名字。这样的消息通常称为提示（prompt），因为它提示用户采取行动。随后几行定义了一个 string 类型的变量 first_name，从键盘读入内容存到该变量中，再写出一声问候。下面逐一分析这三行：

```
string first_name;    // first_name 是一个 string 类型的变量
```

这行代码在内存中划出一块区域来存放字符串，并给它命名为 first_name：

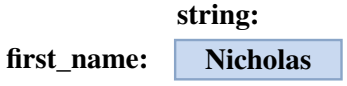


在程序中引入一个新名字并为变量分配内存的语句称为定义（definition）。

下一行把输入（键盘）中的字符读入该变量：

```
cin >> first_name;    // 把字符读入 first_name
```

名字 cin 指标准库中定义的标准输入流（读作“see-in”，即 character input，字符输入）。运算符 >>（“取自”）的第二个操作数指定输入的去处。因此，如果我们键入一个名字，比如 Nicholas，再按回车，字符串 "Nicholas" 就成了 first_name 的值：



必须输入换行符，机器才会开始处理。在敲下回车键（即输入换行）之前，计算机只是在收集字符。这段“延迟”给了你改变主意的余地：在按下回车之前，可以删除或替换已输入的字符。换行符本身不会存入内存中的字符串。

把输入字符串读入 first_name 之后，就可以使用它了：

```
cout << "Hello, " << first_name << "!\n";
```

它在屏幕上打印出 Hello,，随后是 Nicholas（first_name 的值），再是 ! 和一个换行符（'\n'）：

```
Hello, Nicholas!
```

如果我们不怕麻烦、愿意多打字，也可以写成三条独立的输出语句：

```
cout << "Hello, ";
cout << first_name;
cout << "!\n";
```

不过，我们对打字没什么热情；更重要的是，我们强烈反感不必要的重复（因为重复会带来出错的机会），所以把这三条输出语句合并成了一条。

请注意，`"Hello, "` 这几个字符外面加了引号，而 `first_name` 没有。想要字面字符串（literal string）时就用引号；不加引号时，引用的是某个名字所对应的值。请看：

```
cout << "first_name" << " is " << first_name;
```

这里，`"first_name"` 给我们的是十个字符 first_name，而不带引号的 `first_name` 给出的是变量 `first_name` 的值，在这里就是 Nicholas。于是我们得到

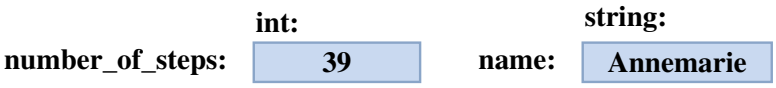
```
first_name is Nicholas
```

3.2 变量

基本上，如果不能把数据存入内存，计算机就做不了什么有趣的事情——上面例子中的输入字符串就是这么做的。存放数据的“位置”称为对象。要访问对象，需要一个名字。有名字的对象称为变量，变量有确定的类型（如 `int` 或 `string`），类型决定了什么可以放入对象（例如 123 可以放入 `int`，`"Hello, World!\n"` 可以放入 `string`），以及可以对它施加哪些运算（例如，`int` 可以用 `*` 运算符相乘，`string` 可以用 `<=` 运算符比较）。放入变量的数据项称为值（value）。定义变量的语句（不出所料）称为定义（definition），定义可以（而且通常应该）提供初始值。请看：

```
string name = "Annemarie";
int number_of_steps = 39;
```

可以把这些变量想象成这样：



不能把类型不对的值放进变量：

```
string name2 = 39;                  // 错误：39 不是字符串
int number_of_steps = "Annemarie";  // 错误："Annemarie" 不是 int
```

编译器记住每个变量的类型，并确保你按照定义中所规定的类型来使用它。

C++ 提供的类型相当多（参见附录 A.8 节）。不过，只用其中五种就能写出非常完善的程序：

```
int number_of_steps = 39;      // int 表示整数
double flying_time = 3.5;      // double 表示浮点数
char decimal_point = '.';      // char 表示单个字符
string name = "Annemarie";     // string 表示字符串
bool tap_on = true;            // bool 表示逻辑变量
```

double 这个名字的由来纯属历史原因：它是“双精度浮点数”（double-precision floating point）的简称。浮点数是计算机对数学中实数概念的近似。

注意，上述每种类型的字面值（literal）都有自己特定的书写风格：

```
39            // int: 整数
3.5           // double: 浮点数
'.'           // char: 单引号括起的单个字符
"Annemarie"   // string: 双引号括起的字符串
true          // bool: true 或 false
```

也就是说，一串数字（如 1234、2 或 976）表示整数；单引号括起的单个字符（如 '1'、'@' 或 'x'）表示字符；带小数点的数字串（如 1.234、0.12 或 .98）表示浮点数；双引号括起的一串字符（如 "1234"、"Howdy!" 或 "Annemarie"）表示字符串。字面值的详细说明见附录 A.2 节。

3.3　输入与类型

输入运算符 >>（“取自”）对类型是敏感的：它根据读取目标变量的类型来读取。例如：

```
// 读入名字和年龄
int main()
{
    cout << "Please enter your first name and age\n";
    string first_name;    // string 变量
    int age;              // 整数变量
    cin >> first_name;    // 读入一个字符串
    cin >> age;           // 读入一个整数
    cout << "Hello, " << first_name << " (age " << age << ")\n";
}
```

于是，如果键入 Carlos 22，>> 运算符会把 Carlos 读入 first_name，把 22 读入 age，并产生如下输出：

```
Hello, Carlos (age 22)
```

为什么它不会把整个 Carlos 22 都读入 first_name？因为按照约定，字符串的读取在遇到所谓空白（whitespace）处终止——空白即空格、换行和制表符。而在其他场合，>> 默认忽略空白。例如，你可以在待读取的数字前面随意添加任意多个空格，>> 会跳过它们并读取该数字。

如果键入的是 22 Carlos，你可能会看到一时想不通的结果。22 会被读入 first_name，因为毕竟 22 也是一串字符；而 Carlos 不是整数，所以不会被读入。输出将是 22，后面跟着 (age 和某个随机数，

比如 −96739 或 0。为什么？因为你没有给 age 初始值，也没有成功读入值。因此你得到的是程序开始执行时那块内存中恰好遗留的“垃圾值”。10.6 节会讨论处理“输入格式错误”的方法。眼下，我们先给 age 一个初始值，这样即使输入失败，得到的结果也是可预测的：

```
// 读入名字和年龄（第二版）
int main()
{
    cout << "Please enter your first name and age\n";
    string first_name = "???";     // string 变量
                                   //  ("???" 意为“不知道名字”)
    int age = -1;                  // 整数变量 (-1 意为“不知道年龄”)
    cin >> first_name >> age;      // 读入一个字符串和一个整数
    cout << "Hello, " << first_name << " (age " << age << ")\n";
}
```

现在键入 22 Carlos 将输出

```
Hello, 22 (age -1)
```

注意，我们可以在一条输入语句中读取多个值，正如可以在一条输出语句中写出多个值一样。还要注意，<< 同样对类型敏感，所以 int 变量 age、string 变量 first_name 以及字符串字面值 "Hello, "、" (age " 和 ")\n" 都能输出。

用 >> 读入的字符串（默认）以空白结束；也就是说，它一次只读一个词。但有时我们想读入多个词。当然，办法有很多。例如，可以像下面这样读入由两个词组成的名字：

```
int main()
{
    cout << "Please enter your first and second names\n";
    string first;
    string second;
    cin >> first >> second;    // 读入两个字符串
    cout << "Hello, " << first << " " << second << '\n';
}
```

我们只是把 >> 用了两次，每个名字一次。当我们要把名字写到输出时，必须在它们之间插入一个空格。

试一试

让“名字和年龄”这个例子运行起来。然后，修改它，把年龄按月写出来：以年为单位读入年龄，再（用 * 运算符）乘以 12。把年龄读入一个 double，这样五岁半的孩子（比起只有五岁）可以为此感到更加自豪。

3.4 运算与运算符

类型除了规定变量里能存放哪些值之外，还决定了我们可以对它施加哪些运算，以及这些运算的含义。例如：

```
int count;
cin >> count;       // >> 把一个整数读入 count
string name;
cin >> name;        // >> 把一个字符串读入 name


int c2 = count+2;          // + 把整数相加
string s2 = name + " Jr. ";    // + 把字符追加到后面


int c3 = count-2;         // - 把整数相减
string s3 = name - " Jr. ";       // 错误：字符串没有定义 -
```

这里说的“错误”，是指编译器会拒绝企图对字符串做减法的程序。编译器确切地知道每个变量能施加哪些运算，因此能防止许多错误。不过，编译器并不知道对你手头的值来说哪些运算才有意义，所以它会安然接受那些在语法上合法、结果却可能荒唐可笑的运算。例如：

```
int age = -100;
```

你一眼就能看出年龄不可能是负数（为什么不能?），可是没有人告诉编译器，所以它仍会为这个定义生成代码。

下面这张表列出了一些常用、实用的类型所支持的运算符：

	bool	char	int	double	string
赋值	=	=	=	=	=
加法			+	+	
串接					+
减法			-	-	
乘法			*	*	
除法			/	/	
余数（取模）			%		
加 1 递增			++	++	
减 1 递减			--	--	
递增 n			+= n	+= n	
在末尾追加					+=
递减 n			-= n	-= n	
乘法赋值			*=	*=	
除法赋值			/=	/=	
取余赋值			%		
把 x 从 s 读入	s >> x	s >> x	s >> x	s >> x	s >> x
把 x 写入 s	s << x	s << x	s << x	s << x	s << x
相等	==	==	==	==	==
不等	!=	!=	!=	!=	!=
大于	>	>	>	>	>
大于等于	>=	>=	>=	>=	>=
小于	<	<	<	<	<
小于等于	<=	<=	<=	<=	<=

表中的空白格表示该运算不能直接用于这种类型（不过仍可能有间接的使用方法；见 §3.9.1）。这些运算符以及更多的运算符，我们会随着讲解逐一说明。这里要抓住的关键点是：有用的运算符很多，而且对于相似的类型，这些运算符的含义往往也是相同的。

下面用一个涉及浮点数（floating-point number）的例子来试一试：

```
// 一个用来操练运算符的简单程序
int main()
{
    cout << "Please enter a floating-point value: ";
    double n;
    cin >> n;
    cout << "n == " << n
        << "\nn+1 == " << n+1
        << "\nthree times n == " << 3*n
        << "\ntwice n == " << n+n
        << "\nn squared == " << n*n
        << "\nhalf of n == " << n/2
        << "\nsquare root of n == " << sqrt(n)
        << '\n';  // 即输出中的“行结束”(换行符)的另一个名字
}
```

显然，常见的算术运算用的还是我们小学就熟悉的记法与含义。当然，我们想对浮点数做的并非每件事都有对应的运算符，比如取平方根就不是。许多运算是以有名字的函数（named function）的形式提供的。本例中，我们用标准库的 `sqrt()` 来求 `n` 的平方根，即 `sqrt(n)`——这个记号是从数学里继承来的。本书会一路使用函数，并在 §4.5 和 §8.5 里更详细地讨论它。

试一试

让这个小程序运行起来。然后，修改它，让它读入一个 `int` 而不是 `double`。注意 `sqrt()` 对 `int` 没有定义，所以要先把 `n` 赋给一个 `double`，再对它取 `sqrt()`。另外，也“操练”一下别的运算。注意，对 `int` 来说，`/` 是整数除法，`%` 是余数（取模），因此 5/2 是 2（而不是 2.5 或 3），5%2 是 1。整数 `*`、`/`、`%` 的定义保证了：对两个正 `int` 数 `a` 和 `b`，有 `a/b * b + a%b == a`。

字符串（string）的运算符要少一些，但正如第 23 章将看到的，它们有大量按名字调用的操作。不过，它们仅有的这几个运算符用起来符合惯例。例如：

```
// 读入名和姓
int main()
{
    cout << "Please enter your first and second names\n";
    string first;
    string second;
    cin >> first >> second;      // 读入两个字符串
    string name = first + ' ' + second;     // 把两个字符串连接起来
    cout << "Hello, " << name << '\n';
}
```

对字符串来说，`+` 的含义是串接（concatenation）：当 `s1` 和 `s2` 是字符串时，`s1+s2` 是一个新字符串，其中 `s1` 的字符后面跟着 `s2` 的字符。例如，若 `s1` 的值是”Hello”，`s2` 的值是”World”，则 `s1+s2` 的值是”HelloWorld”。字符串的比较特别有用：

```
// 读入并比较名字
int main()
{
    cout << "Please enter two names\n";
    string first;
    string second;
    cin >> first >> second;     // 读入两个字符串
    if (first == second) cout << "that's the same name twice\n";
    if (first < second)
        cout << first << " is alphabetically before " << second << '\n';
    if (first > second)
        cout << first << " is alphabetically after " << second << '\n';
}
```

这里我们用了一条 `if` 语句（if-statement），它将根据条件来选择执行哪些动作；§4.4.1.1 会详细解释它。

3.5　赋值和初始化

从很多方面看，最有意思的运算符是赋值（assignment），记作 `=`。它给变量一个新的值。例如：

`int a = 3;` // *a 一开始值为 3*	a: 3
`a = 4;` // *a 得到 4（“变成 4”）*	a: 4
`int b = a;` // *b 一开始是 a 的值的副本（即 4）*	a: 4　b: 4
`b = a+5;` // *b 得到 a+5（即 9）*	a: 4　b: 9
`a = a+7;` // *a 得到 a+7（即 11）*	a: 11　b: 9

最后那个赋值值得注意。首先，它清楚地表明 `=` 并不表示“相等”——显然，`a` 不等于 `a+7`。它表示的是赋值，也就是把一个新值放进变量。对 `a=a+7` 而言，发生的事情是：

1．先取出 `a` 的值，即整数 4。

2．再把 7 加到这个 4 上，得到整数 11。

3．最后，把 11 放进 `a`。

我们也可以用字符串来演示赋值：

`string a = "alpha";` // *a 一开始值为“alpha”*	a: alpha
`a = "beta";` // *a 得到“beta”（变成“beta”）*	a: beta
`string b = a;` // *b 一开始是 a 的值的副本（即“beta”）*	a: beta　b: beta
`b = a+"gamma";` // *b 得到 a+“gamma”（即“betagamma”）*	a: beta　b: betagamma
`a = a+"delta";` // *a 得到 a+“delta”（即“betadelta”）*	a: betadelta　b: betagamma

上面我们用了“一开始值为”（starts out with）和“得到”（gets）这两个说法，来区分两种相似但逻辑上不同的运算：

- 初始化（initialization）：给变量它的初始值
- 赋值（assignment）：给变量一个新的值

这两种运算非常相似，以至于 C++ 允许我们对两者使用同一种记号（`=`）：

```
int y = 8;      // 用 8 初始化 y
x = 9;          // 把 9 赋给 x

string t = "howdy!";      // 用 “howdy!” 初始化 t
s = "G'day";              // 把 “G'day” 赋给 s
```

不过，从逻辑上讲，赋值和初始化是不同的。区分二者的依据是：初始化总是以类型说明（int、string之类）开头，而赋值没有。原则上，初始化时变量总是空的；而赋值（在原则上）必须先把变量的旧值清掉，再放上新值。你可以把变量想象成一种小盒子，值则是放进盒子的具体物件，比如一枚硬币。初始化之前盒子是空的；但初始化之后盒子里总有一枚硬币，因此要放入新硬币，你（也就是赋值运算符）就得先取出旧的那枚（“销毁旧值”）。在计算机内存里事情并没有这么字面上地发生，但用这种方式理解正在发生的事并不坏。

3.5.1 一个例子：检测重复单词

当我们想把一个新值放进对象时，就需要赋值。仔细想想就知道：当你把一件事重复做很多次时，赋值才最有用武之地——想用一个不同的值再做一遍同样的事，就需要赋值。我们来看一个小程序，它检测一个单词序列中相邻的重复单词。这种代码是大多数语法检查程序的组成部分：

```
int main()
{
    string previous = " ";      // 上一个单词；初始化为 “不是一个单词”
    string current;             // 当前单词
    while (cin>>current) {      // 读入一串单词
        if (previous == current)    // 检查该单词是否与上一个相同
            cout << "repeated word: " << current << '\n';
        previous = current;
    }
}
```

这个程序算不上最有用，因为它没告诉你重复单词出现在文本的什么位置，但目前够用了。下面逐行分析这个程序，先看

```
string current;     // 当前单词
```

这就是把当前（也就是最近读入的）单词读进去的那个字符串变量，读入用的是

```
while (cin>>current)
```

这种构造称为 `while` 语句（while-statement），本身就很有意思，§4.4.2.1 会进一步考察它。`while` 表示：只要输入操作 `cin>>current` 成功，其后的语句就要反复执行；而只要标准输入里还有字符可读，`cin>>current` 就会成功。记住，对字符串来说，`>>` 读入的是以空白分隔的单词。要结束这个循环，就得给程序一个输入结束符（通常称为“文件结束”，end of file）。在 Windows 机器上是 Ctrl+Z（Control 和 Z 一起按下）再回车（Enter）；在 Unix 或 Linux 机器上是 Ctrl+D（Control 和 D 一起按下）。

于是，我们的做法是：读一个单词到 `current`，再拿它和上一个单词（存在 `previous` 里）比较。若二者相同，就报告出来：

```
if (previous == current)     // 检查该单词是否与上一个相同
    cout << "repeated word: " << current << '\n';
```

然后要为下一个词做好准备，把这一步再做一遍。做法是把 current 的词复制进 previous：

```
previous = current;
```

只要能正确起步，这就覆盖了所有情形。那么，处理第一个单词时该怎么做呢？那时还没有上一个单词可供比较。这个问题靠 previous 的定义来解决：

```
string previous = " ";      // 上一个单词；初始化为“不是一个单词”
```

这个 " " 里只有一个字符（空格字符，就是敲键盘空格键得到的那个）。输入运算 >> 会跳过空白，所以我们不可能从输入读到它。因此，第一次进入 while 语句时，测试

```
if (previous == current)
```

会像我们所希望的那样失败。理解程序流程的一种方法是“扮演计算机”（play computer）：逐行跟着程序走，它说什么就做什么。只要在纸上画出一些小盒子，把值写进去，并按程序的规定改写其中的值即可。

试一试

自己动手在纸上“执行”这个程序。输入用 The cat cat jumped。即便是经验丰富的程序员，也会用这种手法来推演那些不知为何总让人看不透的小段代码。

试一试

让“重复单词检测程序”跑起来。用句子 She she laughed He He He because what he did did not look very very good good 来测试它。一共发现了多少个重复单词？为什么？这里“单词”是怎么定义的？“重复单词”又是怎么定义的？（例如，She she 算是重复吗？）

3.6　复合赋值运算符

给变量加 1（即递增，increment）在程序里太常见了，以至于 C++ 为它提供了专门的语法。例如：

```
++counter
```

表示的就是

```
counter = counter + 1
```

基于变量当前值来改变它的值，还有许多常用的做法。例如，我们可能想给它加 7、减 9，或者把它乘以 2。这类运算 C++ 也直接支持。例如：

```
a += 7;     // 即 a = a+7
b -= 9;     // 即 b = b-9
c *= 2;     // 即 c = c*2
```

一般来说，对任何双目运算符 oper，`a oper= b` 就是 `a = a oper b`（§A.5）。就凭这一条规则，我们就得到了 `+=`、`-=`、`*=`、`/=` 和 `%=`。这套记法相当紧凑，能直接表达出我们的想法。例如，在很多应用领域，`*=` 和 `/=` 被称为“缩放”（scaling）。

3.6.1　一个例子：找出重复单词

回看上面检测重复相邻单词的例子。我们可以对它加以改进，给出重复单词在序列中的大致位置。一种简单的变体是把这些单词计数，并对重复的单词输出其序号：

```
int main()
{
    int number_of_words = 0;
    string previous = " ";      // 不是一个单词
    string current;
    while (cin>>current) {
        ++number_of_words;      // 增加单词计数
        if (previous == current)
            cout << "word number " << number_of_words
                 << " repeated: " << current << '\n';
        previous = current;
    }
}
```

我们把单词计数器初始化为 0。每遇到一个单词，就把它加 1：

```
++number_of_words;
```

这样第一个单词就是 1 号，下一个是 2 号，依此类推。同样的效果也可以写成

```
number_of_words += 1;
```

或者甚至写成

```
number_of_words = number_of_words+1;
```

但 `++number_of_words` 更短，而且直接表达了“递增”这个意思。体会一下这个程序与 §3.5.1 那个程序何其相似。显然，我们只是拿来 §3.5.1 的程序稍加修改，让它服务于新目的。这是很常用的手法：要解决一个问题时，先找一个相似的问题，把它的解法适当修改后拿来用。除非真的迫不得已，不要一切从零开始。以已有程序为基础进行修改常常省下大量时间，而且先前在原程序上投入的那些心血也能沾光。

3.7 名字

我们给变量起名字，为的是记住它们，并能在程序的其他地方引用它们。在 C++ 里，什么可以成为名字呢？在一个 C++ 程序中，名字以字母开头，且只能包含字母、数字和下划线。例如：

```
x
number_of_elements
Fourier_transform
z2
Polygon
```

下面这些不是名字：

```
2x                // 名字必须以字母开头
time$to$market    // $ 不是字母、数字或下划线
Start menu        // 空格不是字母、数字或下划线
```

说它们“不是名字”，意思是 C++ 编译器不会把它们当作名字接受。

如果你读系统代码或机器生成的代码，可能会见到以下划线开头的名字，比如 `_foo`。自己千万别这样起名：这类名字是为语言实现和系统实体保留的。只要不用下划线打头，你的名字就永远不会碰巧和实现生成的名字相冲突。

名字是区分大小写的（case sensitive），也就是说大写字母和小写字母互不相同，`x` 和 `X` 是两个不同的名字。下面这个小程序至少有四个错误：

```
#include "std_lib_facilities.h"

int Main()
{
    STRING s = "Goodbye, cruel world! ";
    cOut << S << '\n';
}
```

定义仅大小写不同的名字通常不是个好主意，比如 `one` 和 `One`；这不会让编译器犯糊涂，却很容易让程序员犯糊涂。

试一试

编译这个“Goodbye, cruel world!”程序，仔细看错误信息。编译器把全部错误都找出来了吗？它把哪些问题提示给你了？编译器会不会晕头转向，诊断出多于四个“错误”？从词法上最靠前的错误开始，一个一个地消掉它们，观察错误信息如何变化（并越变越好）。

C++ 语言把许多（约 85 个）名字保留为“关键字”（keyword），在 §A.3.1 中列出。你不能用这些关键字给自己的变量、类型、函数等命名。例如：

```
int if = 7;       // 错误：if 是关键字
```

你可以使用标准库设施的名字，比如 `string`，但最好别这么做。重用这种常见名字，一旦你真想用标准库，就会惹出麻烦：

```
int string = 7;     // 这会惹出麻烦
```

给变量、函数、类型等起名时，要选有意义的名字；也就是说，选那些有助于别人理解你程序的名字。要是程序里到处都是 x1、x2、s3、p7 这类“好敲”的名字，就连你自己都很难弄清它想干什么。缩写和首字母缩略词容易把人搞糊涂，所以要少用。我们写下面这些缩略词时觉得含义再显然不过，但你恐怕至少有一个看不懂：

```
mtbf
TLA
myw
NBV
```

我们预计，再过几个月，我们自己也会至少看不懂其中一个了。

x、i 这类短名字，只要按惯例使用就是有意义的：x 应当是局部变量或参数（见 §4.5 和 §8.4），i 应当是循环索引（见 §4.4.2.3）。

不要用过分长的名字：既难敲，又把行拉得太长，屏幕都放不下，也不好快速阅读。下面这些大概没问题：

```
partial_sum
element_count
stable_partition
```

下面这些恐怕就太长了：

```
the_number_of_elements
remaining_free_slots_in_symbol_table
```

我们“本社风格”（house style）是用下划线分隔标识符（identifier）中的单词，如 `element_count`，而不采用 `elementCount` 等替代写法。我们从不使用全大写的名字，如 `ALL_CAPITAL_LETTERS`，因为那在惯例上是留给宏（macro）的（§27.8 与 §A.17.2），而我们避免使用宏。自定义的类型用首字母大写，例如 `Square` 和 `Graph`。C++ 语言和标准库不采用首字母大写的风格，所以是 `int` 而不是 `Int`，是 `string` 而不是 `String`。这样，我们的约定就最大程度地避免了自定义类型与标准类型之间的混淆。

避免那些容易敲错、读错或混淆的名字。例如：

```
Name    names    nameS
foo     f00      fl
f1      fI       fi
```

字符 0（零）、o（小写 O）、O（大写 o）、1（一）、I（大写 i）和 l（小写 L）特别容易惹祸。

3.8　类型与对象

类型是 C++ 和其他大多数编程语言的核心概念。下面更仔细、也稍稍更技术化地审视类型，特别是我们在计算过程中存放数据所用的那些对象的类型。从长远看这会省不少工夫，也可能让你少一些困惑。

• 类型（type）为一组可能的值和一组（针对对象的）运算规定了范围。
• 对象（object）是保存某个给定类型的值的一些内存。
• 值（value）是内存中按类型加以解释的一组比特位。
• 变量（variable）是有名字的对象。
• 声明（declaration）是给对象一个名字的语句。
• 定义（definition）是为对象划出内存的声明。

非正式地说，我们把对象想成一个盒子，可以往里放给定类型的值。`int` 盒子能装整数，如 7、42 和 -399。`string` 盒子能装字符串值，如”Interoperability”、”tokens: !@#$%&*” 和”Old MacDonald had a farm”。图形化地看，可以是这样：

```
int a = 7;                      a: [7]
int b = 9;                      b: [9]
char c = 'a';                   c: [a]
double x = 1.2;                 x: [1.2]
string s1 = "Hello, World!";    s1: [13][Hello, World!]
string s2 = "1.2";              s2: [3][1.2]
```

`string` 的表示比 `int` 略微复杂些，因为字符串还得记住它保存了多少个字符。注意，`double` 存的是一个数，而 `string` 存的是字符。例如，`x` 存的是数 1.2，而 `s2` 存的是三个字符 `'1'`、`'.'` 和 `'2'`。字符字面值和字符串字面值外面的引号是不保存的。

每个 `int` 的尺寸都相同；也就是说，编译器为每个 `int` 划出同样大小的固定内存。在典型的台式机上，这一大小是 4 字节（32 位）。类似地，`bool`、`char` 和 `double` 也是固定尺寸的。你会发现台式机上一般用一个字节（8 位）存放 `bool` 或 `char`，用 8 字节存放 `double`。注意，不同类型的对象占用的空间不同。特别地，`char` 占的空间比 `int` 小；`string` 则不同于 `double`、`int` 和 `char`——不同字符串占用的空间可以不同。

内存中各比特位的含义，完全取决于访问它所用的类型。可以这样想：计算机内存根本不知道我们的类型；它就只是内存。内存里的比特，只有当我们决定如何解释这块内存时才获得含义。这和我们每天使用数字的方式类似。12.5 是什么意思？我们不知道。它可能是 12.5 美元、12.5 cm，也可能是 12.5 加仑。只有给出了单位，记法 12.5 才有意义。例如，同一块内存，那些比特位当作 `int` 来看时表示值 120；当作 `char` 来看时是 `'x'`；若当作 `string`，则根本不成体统，一旦试图使用还会变成一个运行时错误。可以用图把这一点表示如下，用 1 和 0 来表示内存中各比特位的值：

```
00000000 00000000 00000000 01111000
```

这就是内存中某个区域（一个字，word）的比特位的状态，把它当 `int` 读是 120；当 `char` 读是 `'x'`（只看最右边那 8 位）。比特（bit）是计算机内存的单位，可以取值 0 或 1。关于二进制数的含义，见 §A.2.1.1。

3.9 类型安全

每个对象在定义时都会获得一个类型。当一个程序——或一个程序片段——只在符合其类型规则的前提下使用对象时，它就是类型安全的（type-safe）。遗憾的是，有些运算的使用方式是类型不安全的。例如，使用尚未初始化的变量就不被认为类型安全：

```
int main()
{
    double x;          // 我们“忘了”初始化：
                       // x 的值是未定义的
    double y = x;      // y 的值是未定义的
    double z = 2.0+x;  // + 的含义和 z 的值都是未定义的
}
```

语言实现甚至被允许在使用未初始化的 x 时给出一个硬件错误。一定要初始化你的变量！这条规则只有很少很少的例外，比如那个立即用作输入操作目标的变量；但“总是初始化”是个好习惯，会替你挡掉许多伤心事。

完全的类型安全是理想，因而也是语言的一般规则。遗憾的是，C++ 编译器无法保证完全的类型安全，但我们可以把良好的编码实践和运行时检查结合起来，避免类型安全违例。理想的做法是：绝不使用编译器无法证明为安全的语言设施，即静态类型安全（static type safety）。遗憾的是，对大多数有趣的编程用途来说，这样做限制太严。另一个显然的退路——让编译器隐式生成检查类型安全违例并全部予以捕获的代码——超出了 C++ 的能力。当我们决定做（类型）不安全的操作时，就必须自己做些检查。遇到这类情形时，本书会随时点明。

类型安全的理想在写代码时至关重要，这正是我们在书中这么早就花时间讲它的原因。请注意这些陷阱，并设法避开。

3.9.1　安全的转换

在 §3.4 中我们看到，char 不能直接相加，double 也不能直接与 int 比较。不过，C++ 提供了间接达到这两个目的的途径：需要时，char 会被转换（convert）成 int，而 int 会转换成 double。例如：

```
char c = 'x';
int i1 = c;
int i2 = 'x';
```

这里 i1 和 i2 得到的值都是 120，即在最常用的 8 位字符集 ASCII 中字符 'x' 的整数值。这是获取字符数值表示的一种简单而安全的方式。我们之所以称这种 char 到 int 的转换为安全的（safe），是因为没有损失任何信息；也就是说，我们可以把得到的 int 再复制回一个 char，还原出原来的值：

```
char c2 = i1;
cout << c << ' ' << i1 << ' ' << c2 << '\n';
```

这将输出

```
x 120 x
```

在这个意义上——即一个值总是被转换成相等的值，或者（对 double 而言）是相等值的最佳近似——以下转换都是安全的：

```
bool to char
bool to int
bool to double
char to int
char to double
int to double
```

其中最有用的转换是 int 到 double，因为它让我们可以在表达式中混用 int 和 double：

```
double d1 = 2.3;
double d2 = d1+2;    // 加之前，2 被转换成 2.0
if (d1 < 0)          // 比较之前，0 被转换成 0.0
    cout << "d1 is negative";
```

对非常大的 int，在某些计算机上，转换成 double 时可能损失精度。这是个罕见的问题。

3.9.2 不安全的转换

安全转换通常是对程序员的恩惠，让代码好写不少。可遗憾的是，C++ 也允许（隐式的）不安全转换。所谓不安全，是指一个值可以被隐式地变成另一个类型的值，而这个值并不等于原来的值。例如：

```
int main()
{
    int a = 20000;
    char c = a;    // 试图把一个大的 int 塞进一个小的 char
    int b = c;
    if (a != b)    // != 意为“不等于”
        cout << "oops!: " << a << "!=" << b << '\n';
    else
        cout << "Wow! We have large characters\n";
}
```

这类转换也称为“收窄”（narrowing）转换，因为它们把一个值放进一个可能太窄（太小）而装不下它的对象。遗憾的是，很少有编译器会对用 int 不安全地初始化 char 提出警告。问题在于，int 通常比 char 大得多，所以它装得下（本案也正是如此）一个不能表示为 char 的 int 值。试一试，看看在你机器上 b 得到什么值（32 是常见的结果）；更好的是做个实验：

```
int main()
{
    double d = 0;
    while (cin>>d) {          // 反复执行下面的语句
                              // 只要我们输入的是数字
        int i = d;            // 试图把一个 double 塞进一个 int
        char c = i;           // 试图把一个 int 塞进一个 char
        int i2 = c;           // 取得该字符的整数值
        cout << "d==" << d    // 原来的 double
            << " i==" << i    // 转换成 int
            << " i2==" << i2  // char 的整数值
            << " char(" << c << ")\n";   // 字符本身
    }
}
```

我们用这条 `while` 语句来允许尝试多个输入值，§4.4.2.1 将解释它。

试一试

用各种各样的输入运行这个程序。试试小值（如 2 和 3）；试试大值（大于 127、大于 1000 的值）；试试负值；试试 56；试试 89；试试 128；试试非整数值（如 56.9 和 56.2）。除了展示你这台机器上 `double` 到 `int`、`int` 到 `char` 的转换是如何完成的之外，这个程序还会告诉你：对给定的整数值，你的机器会打出什么字符（如果有的话）。

你会发现很多输入值都会产生“不合理”的结果。说到底，我们是在把一加仑的东西往一品脱的罐子里装（大约是把 4 升水倒进一个 500 ml 的杯子）。以下所有转换

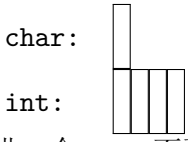
```
double to int
double to char
double to bool
int to char
int to bool
char to bool
```

尽管不安全，编译器却都会接受。所谓不安全，是说存进去的值可能与被赋的值不同。为什么这会是个问题？因为通常我们根本没察觉到有转换在悄悄发生。考虑：

```
double x = 2.7;
// 大量代码
int y = x;      // y 变成 2
```

等到定义 `y` 的时候，我们可能已经忘了 `x` 是一个 `double`，或者一时想不起 `double` 到 `int` 的转换是截断（总是向下舍入，朝零的方向）而不是习惯上的四舍五入。发生的事完全可预测，但 `int y = x;` 里没有任何东西提醒我们信息（那个.7）被扔掉了。

`int` 到 `char` 的转换没有截断问题——`int` 和 `char` 都表示不了整数的小数部分。不过，`char` 只能保存很小的整数值。在 PC 上，`char` 是 1 字节，而 `int` 是 4 字节：



因此，我们无法把 1000 这样的大数塞进一个 char 而不丢失信息：值被“收窄”了。例如：

```
int a = 1000;
char b = a;      // b 变成 -24（在某些机器上）
```

并非所有 int 值都有对应的 char 值，char 值的确切范围取决于具体实现。在 PC 上，char 值的范围是 $[-128:127]$，但只有 $[0:127]$ 可以拿来跨平台使用，因为并非每台计算机都是 PC，不同计算机的 char 值范围也不同，比如 $[0:255]$。

人们为什么容忍收窄转换带来的问题？主要原因是历史：C++ 从其祖先语言 C 那里继承了收窄转换，所以从 C++ 诞生的第一天起，就有大量依赖收窄转换的代码存在。此外，很多这样的转换实际上并不出事，因为牵涉的值碰巧都在范围内；而且许多程序员反感编译器“替他们拿主意”。特别地，在不大的程序里，不安全转换的问题往往可以应付，有经验的程序员更是如此。但在较大的程序里，它们可能成为错误的来源；对新手程序员来说，它们则是出问题的重要原因。不过，编译器可以对收窄转换提出警告——而且很多也确实这么做。

C++11 引入了一种初始化记法，把收窄转换定性为非法。例如，我们可以（而且应该）用 {} 列表记法来改写上面那些恼人的例子，而不是用 = 记法：

```
double x {2.7};      // OK
int y {x};           // 错误：double -> int 可能收窄


int a {1000};        // OK
char b {a};          // 错误：int -> char 可能收窄
```

当初始值是整数常量（integer literal）时，编译器能检查实际的值，并接受那些并不意味着收窄的值：

```
char b1 {1000};      // 错误：收窄（假定 char 为 8 位）
char b2 {48};        // OK
```

那么，如果你觉得某个转换可能出现坏值，该怎么办？用 {} 初始值来避免意外；而当你确实想要转换时，就像本节第一个例子那样，先检查值再赋。§5.6.4 和 §7.5 给出了做这类检查的简化方法。基于 {} 列表的记法称为通用统一初始化（universal and uniform initialization），后文还会大量见到它。

小试身手

小试身手的每一步做完之后，都运行一下你的程序，确认它真的在按你的预期行事。把自己犯的错误记成一张清单，以便今后尽力避免。

1. 本小试身手的任务是写一个程序，它根据用户的输入生成一封简单的模板信件。先把 §3.1 的代码敲进去：提示用户输入他/她的名（first name），然后写出“Hello, first_name”，其中 first_name 是用户输入的名字。随后按如下方式修改代码：把提示改为“Enter the name of the person you want to write to”，把输出改为“Dear first_name,”。别忘了逗号。

2. 加上一两句开场白，比如“How are you? I am fine. I miss you.”。注意给第一行缩进。再随意写上几行——这是你自己的信。

3．现在提示用户输入另一位朋友的名字，存入 `friend_name`。在信中加一行：“Have you seen `friend_name` lately?”

4．声明一个名为 `friend_sex` 的 `char` 变量，将其值初始化为 0。若这位朋友是男性，提示用户输入 `m`；若是女性，则输入 `f`。把输入的值赋给变量 `friend_sex`。然后用两条 `if` 语句写出下面两行。

若朋友是男性，写“If you see `friend_name` please ask him to call me.”。

若朋友是女性，写“If you see `friend_name` please ask her to call me.”。

5．提示用户输入收信人的年龄，存入一个名为 `age` 的 `int` 变量。让程序写出“I hear you just had a birthday and you are `age` years old.”。若 `age` 为 0 或更小，或为 110 或更大，用 `std_lib_facilities.h` 提供的 `simple_error()` 调用 `simple_error("you're kidding!")`。

6．把以下几行加进信中。

若朋友不满 12 岁，写“Next year you will be `age+1`.”。

若朋友是 17 岁，写“Next year you will be able to vote.”。

若朋友超过 70 岁，写“I hope you are enjoying retirement.”。

检查你的程序，确保它对每一种取值都恰到好处地做出响应。

7．加上“Yours sincerely,”，为其后的签名空两行，再写上你的名字。

要点回顾

1．术语“提示”（prompt）是什么意思？

2．读入到变量用哪个运算符？

3．如果你想让用户往程序中名为 `number` 的变量输入一个整数值，可以写哪两行代码——一行询问用户，一行把值读入程序？

4．`\n` 叫什么，有什么用处？

5．什么终止输入给一个字符串？

6．什么终止输入给一个整数？

7．你会如何把下面的代码

```
cout << "Hello, ";
cout << first_name;
cout << "!\n";
```

写成一整条语句？

8．什么是对象？

9．什么是字面值（literal）？

10．字面值有哪些种类？

11．什么是变量？

12．`char`、`int` 和 `double` 的典型尺寸各是多少？

13．衡量内存中 int、string 这类小实体的尺寸时，我们用什么单位？

14．= 和 == 有什么区别？

15．什么是定义？

16．什么是初始化？它与赋值有何不同？

17．什么是字符串串接？在 C++ 里怎么实现它？

18．下面哪些是 C++ 中的合法名字？对不合法的名字，为什么不合法？

```
This_little_pig     This_1_is_fine     2_For_1_special
latest_thing        the_$12_method     _this_is_ok
MiniMineMine        number             correct?
```

19．给出五个虽然合法、但因容易造成混乱而不该使用的名字的例子。

20．挑选名字有哪些好的规则？

21．什么是类型安全？它为什么重要？

22．为什么 `double` 到 `int` 的转换可能是件坏事？

23．定一条规则，帮助判定从一种类型到另一种类型的转换是安全的还是不安全的。

术语

assignment definition operation
cin increment operator
concatenation initialization type
conversion name type safety
declaration narrowing value
decrement object variable

习题

1．如果还没做，先完成本章的试一试练习。

2．写一个 C++ 程序，把英里换算成千米。程序应当给用户一个合理的提示，以便输入英里数。提示：1 英里合 1.609 千米。

3．写一个什么都不做的程序，只是声明一些名字有合法和不合法之分的变量（如 `int double = 0;`），看看编译器会有什么反应。

4．写一个程序，提示用户输入两个整数值。把这两个值存进名为 `val1` 和 `val2` 的 `int` 变量。让程序求出其中较小者、较大者、和、差、积以及比值，并把结果报告给用户。

5．修改上面的程序，改为让用户输入浮点数，并把它们存进 `double` 变量。对几个你选定的输入，比较两个程序的输出。结果相同吗？应当相同吗？差别在哪里？

6．写一个程序，提示用户输入三个整数值，然后按数值从小到大的顺序输出它们，值之间用逗号分隔。也就是说，若用户输入的值是 10 4 6，输出应当是 4, 6, 10。若两个值相同，就把它们并排放在一起。因此，输入 4 5 4 应输出 4, 4, 5。

7. 把第 6 题改用三个字符串值来做。也就是说，若用户输入的值是 Steinbeck, Hemingway, Fitzgerald，输出应当是 Fitzgerald, Hemingway, Steinbeck。

8．写一个程序，测试一个整数值，判断它是奇数还是偶数。一如既往，保证输出清楚、完整。换句话说，不要只输出 yes 或 no——你的输出应当能独立成句，比如 The value 4 is an even number.。提示：见 §3.4 中的余数（取模）运算符。

9．写一个程序，把拼写出来的数（如“zero”和“two”）转换成数字（如 0 和 2）。用户输入一个数时，程序应打印出对应的数字。对 0、1、2、3、4 这几个值都做一做；若用户输入的东西对不上，比如 stupid computer!，则输出 not a number I know。

10．写一个程序，读入一个运算，再读入两个操作数，输出其结果。例如：

```
+ 100 3.14
* 4 5
```

把运算读进名为 operation 的字符串，再用一条 if 语句判断用户想要哪个运算，例如 if (operation=="+")。把两个操作数读进 double 类型的变量。对名为 +、-、*、/、plus、minus、mul、div 的运算实现本程序，含义一目了然。

11．写一个程序，提示用户输入若干 1 美分硬币（penny）、5 美分硬币（nickel）、10 美分硬币（dime）、25 美分硬币（quarter）、50 美分硬币（half dollar）和 1 美元硬币（100 美分）的枚数。分别问用户每种硬币有多少。

比如问：“How many pennies do you have?”。然后程序应打印出类似下面的内容：

```
You have 23 pennies.
You have 17 nickels.
You have 14 dimes.
You have 7 quarters.
You have 3 half dollars.
The value of all of your coins is 573 cents.
```

再做些改进：若某种硬币只报告了一枚，要让输出语法正确，例如是 14 dimes 和 1 dime（而不是 1 dimes）。另外，把总额用美元和美分报出来，即 $5.73 而不是 573 cents。

后记

请不要低估类型安全这一概念的重要性。类型处于大多数“程序正确”观念的中心，而一些最有效的程序构造技术正是依赖于类型的设计与使用——你将在第 6、9 章以及第 II、III、IV 部分看到这一点。

4 计算

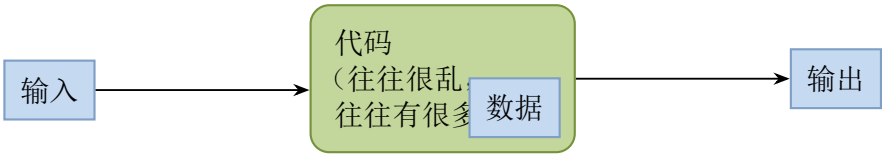
“如果不能产生正确的结果，我可以让它要多快有多快。”

——Gerald M. Weinberg

这一章介绍计算（computation）的基础。特别地，我们讨论如何由一组操作数（operand）计算出一个值（表达式，expression）、如何在备选的动作之间做出选择（选择，selection），以及如何对一列值重复进行同一个计算（迭代，iteration）。我们还将展示如何把一个特定的子计算单独命名并单独加以说明（即函数，function）。我们最关心的是：以能写出正确且组织良好的程序的方式来表达计算。为了帮助你进行更接近实际的计算，我们还会引入 `vector` 类型来存放一串值。

4.1 计算

从某种角度看，程序所做的一切就是计算；也就是说，程序接收一些输入，产生一些输出。毕竟，我们把运行程序的硬件就叫作计算机。只要我们对“构成输入和输出的东西”取足够宽泛的理解，这种看法就是准确而合理的：



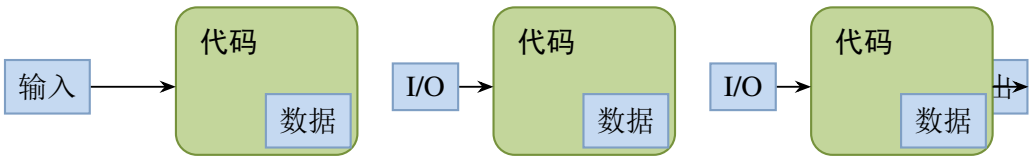
输入可以来自键盘、鼠标、触摸屏或文件，也可以来自其他输入设备、其他程序，甚至来自同一程序的其他部分。“其他输入设备”这一类中包含了大多数真正有趣的输入来源：音乐键盘、录像设备、网络连接、温度传感器、数码相机图像传感器，等等。这种多样性实际上是无限的。

为了处理输入，程序通常要包含一些数据，这些数据有时被称为程序的数据结构（data structure）或状态（state）。例如，一个日历程序可能保存着各个国家的节假日列表，以及你的日程安排列表。有些数据从一开始就是程序的一部分；另一些数据则是程序在读取输入、从中收集有用信息的过程中逐渐建立起来的。比如，日历程序多半会根据你给它的输入来建立你的日程列表。对日历程序而言，主要的输入是你要求查看某些月份和日期的请求（多半用鼠标点击发出），以及你交给它去记录的日程安排（多半用键盘输入）。输出则是日历和日程的显示，外加日历程序写在屏幕上的按钮和输入提示（prompt）。

4.1 计算
4.2 目标与工具
4.3 表达式
4.3.1 常量表达式
4.3.2 运算符
4.3.3 转换
4.4 语句
4.4.1 选择
4.4.2 迭代
4.5 函数
4.5.1 为什么要用函数？
4.5.2 函数声明
4.6 `vector`
4.6.1 遍历一个 vector
4.6.2 让 vector 增长
4.6.3 一个数值例子
4.6.4 一个文本例子
4.7 语言特性

输入来自多种多样的来源。类似地，输出也可以走向多种多样的目的地。输出可以送到屏幕、文件、网络连接、其他输出设备、其他程序，以及程序的其他部分。输出设备的例子有网络接口、音乐合成器、电动机、发光装置、加热器，等等。

从编程的角度看，最重要、最有趣的两类是“到/来自另一个程序”和“到/来自程序的其他部分”。本书其余的大部分内容，都可以看成在讨论后一类：我们怎样把一个程序表达为一组相互协作的部分？这些部分怎样共享与交换数据？这是编程中的关键问题。我们可以把它图示如下：



缩写 I/O 表示“输入/输出”（input/output）。在这里，一段代码的输出就是下一段代码的输入。这些“程序的部分”所共享的，是保存在主内存中、保存在持久存储设备（如磁盘）上，或经由网络连接传送的数据。所谓“程序的部分”，我们指的是这样一些实体：由一组输入实参（argument）算出一个结果的函数（例如，由一个浮点数算出它的平方根）、对某个物理对象执行一个动作的函数（例如，在屏幕上画一条线的函数），或者修改程序中某个表的函数（例如，往客户表里加一个名字的函数）。

当我们说“输入”和“输出”时，通常指的是进出计算机的信息；不过正如你所见，这两个词也可以指提供给程序的某一部分的信息，或它产生的信息。给程序某一部分的输入常常称为实参（argument），程序某一部分产生的输出则常常称为结果（result）。

所谓计算（computation），我们只是一般地指根据某些输入产生某些输出的行为，例如：用计算（函数）`square`，由实参（输入）7 得到结果（输出）49（见 §4.5）。顺便提一个也许有点帮助的冷知识：直到 20 世纪 50 年代，computer 一词的定义还是“做计算的人”，比如会计师、领航员或物理学家。今天，我们干脆把大多数计算托付给各种形态的计算机（器），袖珍计算器是其中最简单的一种。

4.2 目标与工具

我们作为程序员的工作，是把计算表达得

- 正确
- 简单
- 高效

请注意上面这些理想的次序：如果一个程序给出的结果是错的，它跑得再快也没有意义。同样，一个正确而高效的程序也可以复杂到不得不整个扔掉，或者为了推出新版本（release）而彻底重写。记住，有用的程序总会为了适应新需求、新硬件等而被修改。因此，一个程序——以及程序的任何一部分——都应当做到：为完成其任务而尽可能简单。例如，假设你为当地小学写了一个教儿童基础算术的完美程序，但它的内部结构一团糟。你用什么语言和这些儿童交流？英语？英语加西班牙语？如果我想在芬兰、在科威特用它呢？你会怎样更换程序与儿童交流所用的（自然）语言？如果程序的内部结构一团糟，那么“更换与用户交流的自然语言”这个逻辑上简单（但实践中几乎总是极其困难）的操作就会变成不可逾越的障碍。

从我们开始为别人写代码、并承担把这件事做好一以贯之的责任那一刻起，对正确性、简单性和效率的关心就成了我们的责任；也就是说，在我们决定成为专业人士时，就必须接受这份责任。落实到操作上，这意味着我们不能把代码胡乱堆起来，直到它“似乎能跑”为止；我们必须关心代码的结

构。吊诡的是，对结构和“代码质量”的关心，往往恰恰是让某样东西最快跑起来的途径。编程做得好时，这类关心会把编程中最令人沮丧的部分——调试（debugging）——的需要降到最低；也就是说，开发期间良好的程序结构，能把所犯错误的数量、以及找到并清除这些错误所需的时间都降到最少。

组织一个程序——以及在编程时组织我们的思路——的主要工具，是把一个大的计算拆成许多小的计算。这一技术有两种形式：

- 抽象（abstraction）：把使用某个设施时不需要了解的细节（“实现细节”）藏在一个方便而通用的接口后面。例如，与其去考虑怎么对电话簿排序的细节（关于排序已经写出了厚厚的大部头书），我们直接调用 C++ 标准库的 sort 算法。要排序，我们所需要知道的只是怎样调用（invoke，call）该算法：写 `sort(b)` 即可，其中 b 指电话簿；`sort()` 是在 std_library.h 中定义的标准库排序算法（§21.8、§B.5.4）的一个变体（§21.9）。另一个例子是我们使用计算机内存的方式：直接使用内存可能相当混乱，所以我们通过有类型、有名字的变量（§3.2）、标准库的 vector（§4.6、第 17–19 章）、map（第 21 章）等设施来访问内存。
- “分而治之”（divide and conquer）：把一个大问题拆成几个小问题。例如，要是需要建一个词典，我们可以把这件事拆成三件：读取数据、对数据排序、输出数据。得到的每一个问题，都比原来的问题小得多。

这为什么会有帮助？毕竟，用部件拼出来的程序，多半会比一个把一切都以最优方式合并在一起的程序略大一些。原因在于，我们并不是很擅长处理大的问题。我们在编程中（以及其他场合）实际处理大问题的方式，就是把它们拆成小问题，再不断把小问题拆成更小的部分，直到每一部分都简单到足以理解和求解为止。就编程而言，你会发现：一个 1000 行程序的错误，远不止 100 行程序的十倍，所以我们尽量把 1000 行的程序组合自一些不足 100 行的部件。对于很大的程序——比如 1000 万行——运用抽象和分而治之就不只是可选，而是不可或缺；我们根本写不了、也维护不了巨大的单体程序。把本书余下的内容看成“一长串需要拆成更小部分的问题的实例，外加完成拆分所需的工具与技术”，也是一种理解全书的方式。

考虑对程序做划分时，我们必须始终考虑手头有哪些工具可用，用来表达各个部分及其间的通信。一个良好的库，只要能提供表达想法的有用设施，就会对我们把功能分布到程序的哪些部分产生关键影响。我们不能坐在那里凭空“想象”把程序划分到哪里最好；我们必须考虑有哪些库可用，来表达各个部分及其通信。现在还是起步阶段，但指出这一点并不算太早：如果你能使用一个现成的库，比如 C++ 标准库，你就能省下大量工作——不只是编程，还包括测试和文档。iostream 使我们不必直接处理硬件的输入/输出端口。这就是用抽象划分程序的第一例。往后每一章都会提供更多的例子。

请注意这里对结构和组织的强调：光靠写出大量语句，得不到好代码。为什么现在就提这一点？在这个阶段，你（至少是许多读者）对什么是代码还没有多少概念，而且还要过好几个月，你才有能力写出别人可以拿性命和生计去托付的代码。现在提它，是为了帮你把学习的重点摆对位置。风风火火往前赶是很有诱惑力的：只顾编程中具体而马上有用的部分（就像本章其余部分所讲的内容），而忽视软件开发这门手艺中“更软”、更概念化的部分。然而，优秀的程序员和系统设计师都知道（往往是吃了苦头才学到的）：对结构的关心处在优秀软件的核心，无视结构会带来代价高昂的一团糟。没有结构，你（打个比方）就是在用泥砖砌房子。泥砖也能砌，但你永远砌不到第五层（泥砖不具备那样的结构强度）。如果你有志于建造相当持久的东西，就要在建造过程中一并留心代码的结构与组织，而不是等失败之后再回头来学。

4.3 表达式

程序最基本的构件是表达式（expression）。表达式由若干操作数计算出值。最简单的表达式就是一个字面值（literal value），例如 `10`、`'a'`、`3.14` 或 `"Norah"`。

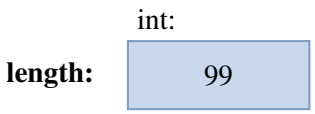
变量的名字也是表达式。一个变量代表它所命名的那个对象。考虑：

```
// 计算面积:
int length = 20;          // 一个字面整数（用于初始化变量）
int width = 40;
int area = length*width; // 一次乘法
```

这里，字面值 20 和 40 被用来初始化变量 `length` 和 `width`。然后，`length` 和 `width` 相乘；也就是说，我们把 `length` 和 `width` 中存放的值乘起来。这里的 `length` 只是“名为 length 的对象中存放的值”的简写。再考虑

```
length = 99;       // 把 99 赋给 length
```

这里，作为赋值的左操作数，`length` 表示“名为 length 的对象”，于是这个赋值表达式读作“把 99 放进 length 命名的对象里”。我们要区分出现在赋值或初始化左边的 `length`（“length 的左值（lvalue）”，或“length 所命名的对象”）与出现在赋值或初始化右边的 `length`（“length 的右值（rvalue）”、“名为 length 的对象的值”，或简称“length 的值”）。在这种语境下，我们发现把变量画成一个用名字做了标签的盒子很有帮助：



也就是说，`length` 是一个类型为 `int` 的对象的名字，其中存放着值 99。有时（作为左值）`length` 指的是那个盒子（对象），有时（作为右值）`length` 指的是盒中的值。

我们可以用运算符（operator）——比如 `+` 和 `*`——把表达式组合起来，构成更复杂的表达式，其方式我们早已熟悉。需要时，可以用括号把表达式分组：

```
int perimeter = (length+width)*2;     // 先加，后乘
```

如果不加括号，就不得不写成

```
int perimeter = length*2+width*2;
```

这已经很笨拙了；我们甚至可能犯下这样的错误：

```
int perimeter = length+width*2;     // 给 length 加上 width*2
```

最后这处错误属于逻辑错误，编译器查不出来。编译器看到的只是一个名叫 `perimeter`、由一个合法表达式初始化的变量。如果那个表达式的结果是一派胡言，那就是你自己的问题。你知道周长的数学定义，编译器可不知道。

通常的数学运算符优先级规则在此适用，因此 `length+width*2` 的意思是 `length+(width*2)`。类似地，`a*b+c/d` 的意思是 `(a*b)+(c/d)`，而不是 `a*(b+c)/d`。优先级表见 §A.5。

使用括号的第一条法则很简单：“有疑问，就加括号”。但请把表达式学到这样的程度：至少对 a*b+c/d 不再有疑问。过度使用括号（例如 (a*b)+(c/d)）会降低可读性。

为什么要关心可读性？因为你自己、还可能有别人，都要读你的代码，而丑陋的代码会拖慢阅读和理解。丑陋的代码不只是难读，它更难得写对。丑陋的代码常常把逻辑错误掩盖起来；读起来更慢，也更难让你自己——以及别人——相信这段丑代码是正确的。不要写出下面这类荒唐复杂的表达式：

```
a*b+c/d*(e-f/g)/h+7      // 太复杂了
```

并且总是尽量选用有意义的名字。

4.3.1 常量表达式

程序里通常要用到大量常量。例如，几何程序可能用到 pi，英寸—厘米换算程序则可能用到 2.54 这样的换算因子。显然，我们想给这些常量配上讲得通的名字（我们对 pi 就是这么做的；我们没有写 3.14159）。同样，我们也不想无意之间改动这些

常量。因此，C++ 提供了符号常量（symbolic constant）的概念，也就是一种已初始化之后便不能再赋予新值的命名对象。例如：

```
constexpr double pi = 3.14159;
pi = 7;              // 错误：给常量赋值
double c = 2*pi*r;   // OK：我们只是读 pi，并不试图修改它
```

这类常量对保持代码的可读性很有用。如果在某段代码里看到 3.14159，你或许能认出它是 pi 的近似值；可是 299792458 你还认得出来吗？另外，要是有人请你在代码里做修改、让计算中的 pi 达到 12 位数精度，你可以在代码里搜索 3.14；但如果有人一时不慎用了 22/7，你多半就找不到了。直接把 pi 的定义改成更合适的值，要省事得多：

```
constexpr double pi = 3.14159265359;
```

因此，除了极少数显而易见的值（比如 0 和 1）之外，我们情愿在代码中的大多数地方不用字面值，而是使用名字具有描述性的常量。出现在符号常量定义之外的非显然字面值，被讥称为魔法常量（magic constant）。

在有些地方，比如 case 标签（§4.4.1.3），C++ 要求使用常量表达式（constant expression），即完全由常量构成、取值为整数的表达式。例如：

```
constexpr int max = 17;    // 一个字面值是一个常量表达式
int val = 19;
max+2                      // 一个常量表达式（一个 const int 加一个字面值）
val+2                      // 不是常量表达式：它用到了变量
```

顺便说一句，299792458 是宇宙的基本常量之一：真空中光的速度，以米每秒计。如果你不能一眼认出它，凭什么指望自己不会被嵌在代码里的其他字面值搞得糊涂、拖慢速度？避开魔法常量！

constexpr 符号常量必须赋给一个在编译时已知的值。例如：

```
constexpr int max = 100;

void use(int n)
{
    constexpr int c1 = max+7;   // OK: c1 为 107
    constexpr int c2 = n+7;     // 错误: 我们不知道 c2 的值
    // ...
}
```

要处理那种初始化时用的值在编译时未知、但初始化之后不再变化的情形（也就是“变量”），C++提供了第二种常量形式（const）：

```
constexpr int max = 100;

void use(int n)
{
    constexpr int c1 = max+7;  // OK: c1 为 107
    const int c2 = n+7;        // OK, 但不要试图修改 c2 的值
    // ...
    c2 = 7;                    // 错误: c2 是一个 const
}
```

这类“const 变量”非常常见，原因有二：

- C++98 没有 constexpr，于是人们用 const。
- 那些虽不是常量表达式（其值在编译时未知）、但在初始化之后不再改变值的“变量”，本身就非常有用。

4.3.2 运算符

我们刚才用的都是最简单的运算符。然而，当你想表达更复杂的运算时，很快就会需要更多的运算符。大多数运算符都是约定俗成的，所以我们会随讲解按需解释；你什么时候觉得需要，再去查细节也不迟。下面是最常用的运算符列表：

	名称	注释
f(a)	函数调用	把 a 作为实参传给 f
++lval	前置递增	先递增，再使用递增后的值
--lval	前置递减	先递减，再使用递减后的值
!a	逻辑非	结果为 bool
-a	一元负号	
a*b	乘法	
a/b	除法	
a%b	取模（余数）	仅用于整数类型
a+b	加法	
a-b	减法	
out<<b	把 b 写到 out	其中 out 是一个 ostream
in>>b	从 in 读入到 b	其中 in 是一个 istream
a<b	小于	结果为 bool
a<=b	小于或等于	结果为 bool
a>b	大于	结果为 bool
a>=b	大于或等于	结果为 bool
a==b	相等	不要与 = 混淆
a!=b	不等	结果为 bool
a && b	逻辑与	结果为 bool
a || b	逻辑或	结果为 bool
lval = a	赋值	不要与 == 混淆
lval *= a	复合赋值	lval = lval*a；同样适用于 /、%、+、-

凡是运算符会修改其操作数的地方，我们都用了 lval（“能出现在赋值语句左边的值”的缩写）。完整的列表见 §A.5。

逻辑运算符 &&（与）、||（或）和 !（非）的使用示例，见 §5.5.1、§7.7、§7.8.2 和 §10.4。

注意，a<b<c 的意思是 (a<b)<c，而 a<b 求值得到一个布尔值：true 或 false。所以 a<b<c 会等价于 true<c 或 false<c 之一。特别地，a<b<c 并不意味着“b 是否介于 a 与 c 之间”，而许多人（可以说不无道理地）曾天真地这样理解。因此，a<b<c 基本上是个没用的表达式。不要写这种含两次比较运算的表达式；如果在别人的代码里看见这种表达式，要高度警惕——它多半是个错误。

递增至少可以用三种写法表达：

```
++a
a+=1
a=a+1
```

应该用哪种写法？为什么？我们偏好第一种写法 `++a`，因为它更直接地表达了“递增”这个想法。它写明我们想做什么（把 `a` 递增），而不是怎么做（把 1 加到 `a` 上，再把结果写回 `a`）。一般而言，如果一种表达想法的方式更直接，它就比另一种更好：结果更简练，读者也更容易理解。如果我们写 `a=a+1`，读者可能会很容易起疑：我们是当真想给 `a` 递增 1 吗？也许只是误敲成了 `a=b+1`、`a=a+2`，甚至是 `a=a-1`；而用 `++a`，产生这类疑义的机会就少得多。请注意，这是关于可读性和正确性的逻辑论证，而不是关于效率的论证。与流行的说法相反，当 `a` 是内置类型之一时，现代编译器对 `a=a+1` 与 `++a` 生成的代码恰恰相同。类似地，相比 `a=a*scale`，我们偏好 `a*=scale`。

4.3.3 转换

我们可以在表达式中“混用”不同的类型。例如，`2.5/2` 是一个 `double` 除以一个 `int`。这是什么意思？该做整数除法还是浮点除法？整数除法会把余数扔掉，例如 `5/2` 是 2。浮点除法不同：没有余数可扔，例如 `5.0/2.0` 是 2.5。于是，对“`2.5/2` 是整数除法还是浮点除法?”这个问题，最显然的答案是“当然是浮点除法，否则我们会丢掉信息”。我们希望答案是 1.25 而不是 1，而我们得到的正是 1.25。规则（就我们目前讲过的类型而言）是：只要一个运算符有 `double` 类型的操作数，我们便做浮点算术、得到 `double` 结果；否则，我们做整数算术、得到 `int` 结果。例如：

```
5/2 是 2 (不是 2.5)
2.5/2 即 2.5/double(2), 也就是 1.25
'a'+1 即 int{'a'}+1
```

记号 type(value) 和 type{value} 的意思是：“把 value 转换成 type，就像用值 value 去初始化一个 type 类型的变量那样”。换句话说，必要时，编译器会把 `int` 操作数转换（“提升”，promote）成 `double`，或把 `char` 操作数转换成 `int`。记号 type{value} 会防止收窄（narrowing，§3.9.2），而 type(value) 不会。结果算出来之后，编译器可能还必须（再次）对它加以转换，才能把它用作初始化式或赋值的右操作数。例如：

```
double d = 2.5;
int i = 2;

double d2 = d/i;     // d2 == 1.25
int i2 = d/i;        // i2 == 1
int i3 {d/i};        // 错误: double -> int 转换可能收窄 (§3.9.2)

d2 = d/i;            // d2 == 1.25
i2 = d/i;            // i2 == 1
```

当心：在一个同时含有浮点操作数的表达式里，很容易把整数除法忘在脑后。考虑常用的摄氏度转华氏度公式 $f = 9/5 \times c + 32$。我们可能写出

```
double dc;
cin >> dc;
double df = 9/5*dc+32;      // 当心!
```

不幸的是——虽然从逻辑上讲这完全顺理成章——这并不代表一次准确的温标换算：9/5 的值是 1，而不是我们希望得到的 1.8。要让代码在数学上正确，必须把 9 或 5（或两者）改成 double。例如：

```
double dc;
cin >> dc;
double df = 9.0/5*dc+32;     // 更好
```

4.4 语句

一个表达式用 §4.3 中提到的那类运算符、由一组操作数算出一个值。要是我们想产生好几个值呢？想把一件事重复做很多次呢？想在备选项中做出选择呢？想获取输入或产生输出呢？在 C++ 里——正如在很多语言里一样——你使用叫作语句（statement）的语言构造来表达这些事情。

到目前为止，我们见过两类语句：表达式语句（expression statement）和声明（declaration）。表达式语句就是一个表达式后面跟一个分号。例如：

```
a = b;
++b;
```

这两行就是两条表达式语句。注意，赋值 = 是一个运算符，因此 a=b 是一个表达式，需要加上结尾的分号，a=b; 才成其为一条语句。为什么需要这些分号？原因主要是技术性的。考虑：

```
a = b ++ b;     // 语法错误：缺分号
```

没有分号，编译器就不知道我们的意思是 a=b++; b; 还是 a=b; ++b;。这类问题不只存在于计算机语言里；想想那句惊呼“man eating tiger!”（“有老虎吃人!”）——到底谁吃谁？标点符号的存在，正是为了消除这类问题，例如写成“man-eating tiger!”（“吃人的老虎!”）。

语句相继排列时，计算机会按照书写顺序执行它们。例如：

```
int a = 7;
cout << a << '\n';
```

这里，带初始化的声明先于输出表达式语句执行。

一般说来，我们希望一条语句能产生某些效果。没有效果的语句通常毫无用处。例如：

```
1+2;        // 做一次加法，但不使用这个和
a*b;        // 做一次乘法，但不使用这个积
```

这类没有效果的语句通常属于逻辑错误，编译器也常常会就此发出警告。因此，表达式语句通常就是赋值、输入/输出语句或函数调用。

我们还要提到一种语句：“空语句”。考虑这段代码：

```
if (x == 5);
{ y = 3; }
```

这看起来像个错误，而且几乎可以肯定就是错误。第一行末尾的那个; 本不该在那里。但很不幸，这在 C++ 里是一个合法的构造，叫作空语句（empty statement），也就是什么都不做的语句。空语句很少有用。在这个例子里，它带来的不幸后果是：让一个几乎可以肯定的错误写法为编译器所接受——于是编译器不会提醒你这个错误，而你要找到它却难上加难。

运行这段代码会发生什么？编译器会测试 x，看它的值是不是 5。如果条件为真，就执行其后的那条语句（也就是那条空语句），没有效果。然后程序继续走到下一行，把 3 赋给 y（这正是当初 x 等于 5 时你想要发生的事）。反过来，如果 x 的值不是 5，编译器不会执行那条空语句（同样没有效果），仍旧照旧继续，把 3 赋给 y（除非 x 等于 5，否则这不是你想要的）。换句话说，这条 if 语句无关紧要；y 无论如何都会得到 3。这是新手程序员的常见错误，而且很难发现，所以要多加小心。

下一节专门讨论那些用来改变求值顺序的语句；有了它们，我们才能表达比“按书写顺序逐条执行”更有意思的计算。

4.4.1 选择

在程序中，正如在生活中一样，我们常常要在备选项之间做出选择。在 C++ 里，做这件事的方式要么是 if 语句，要么是 switch 语句。

4.4.1.1 if 语句

最简单的选择形式是 if 语句，它在两个备选项之间做出选择。例如：

```
int main()
{
    int a = 0;
    int b = 0;
    cout << "Please enter two integers\n";
    cin >> a >> b;

    if (a<b)        // 条件
                    // 第一种选择（条件为真时执行）:
        cout << "max(" << a << "," << b <<") is " << b <<"\n";

    else
                    // 第二种选择（条件为假时执行）:
        cout << "max(" << a << "," << b <<") is " << a << "\n";
}
```

if 语句在两个备选项之间做出选择：条件为真，就执行第一条语句；否则执行第二条。这个概念很简单。大多数基本的编程语言特性都是简单的。事实上，编程语言的大多数基本设施，不过是把你在小学——甚至更早就学会的东西用新记法写出来。例如，你在幼儿园大概被告知过：过马路要等红绿灯变绿：“如果红绿灯是绿的，就走”“如果红绿灯是红的，就等”。在 C++ 里，这差不多就成了

```
if (traffic_light==green) go();
```

以及

```
if (traffic_light==red) wait();
```

所以说，基本概念很简单；但把 if 语句用得过于“一根筋”也很容易。考虑一下这个程序有什么问题（照例不算省略 #include 这件事）：

```
// 把英寸转换为厘米，或把厘米转换为英寸
// 后缀 'i' 或 'c' 表明输入的单位

int main()
{
    constexpr double cm_per_inch = 2.54;    // 一英寸合
                                            // 多少厘米
    double length = 1;                      // 长度，单位英寸或
                                            // 厘米
    char unit = 0;
    cout<< "Please enter a length followed by a unit (c or i):\n";
    cin >> length >> unit;

    if (unit == 'i')
        cout << length << "in == " << cm_per_inch*length << "cm\n";
    else
        cout << length << "cm == " << length/cm_per_inch << "in\n";
}
```

实际上，这个程序干起活来大致与我们宣传的一样：输入 1i，你会得到 1in == 2.54cm；输入 2.54c，你会得到 2.54cm == 1in。亲自试试（doing so）——很好的练习。麻烦之处在于，我们没有对坏输入做测试。程序假定用户输入的是正当输入。条件 unit=='i' 区分了“单位为’i’”与其他各种情况，但它从不去找’c’。

如果用户“就是想看看会发生什么”而输入了 15f（feet，英尺）会怎样？条件 (unit == 'i') 会不成立，程序会执行 else 部分（第二种选择），把英寸的输入当成厘米来换算。这多半不是我们输入’f’时想要的结果。

我们必须始终用“坏”输入来测试程序，因为总有一天会有人——故意也好，无意也罢——输入坏输入。程序即使碰上不靠谱的用户，也应当表现得靠谱。

下面是一个改进版：

```
// 把英寸转换为厘米，或把厘米转换为英寸
// 后缀 'i' 或 'c' 表明输入的单位
// 其他任何后缀都是错误
int main()
{
    constexpr double cm_per_inch = 2.54;    // 一英寸合
                                            // 多少厘米
    double length = 1;                      // 长度，单位英寸或
                                            // 厘米
    char unit = ' ';                        // 空格不是单位
    cout<< "Please enter a length followed by a unit (c or i):\n";
    cin >> length >> unit;

    if (unit == 'i')
        cout << length << "in == " << cm_per_inch*length << "cm\n";
    else if (unit == 'c')
        cout << length << "cm == " << length/cm_per_inch << "in\n";
    else
        cout << "Sorry, I don't know a unit called '" << unit << "'\n";
}
```

我们先测试 `unit=='i'`，再测试 `unit=='c'`，若两者皆非，就输出“Sorry”。看上去好像用了一种“else-if 语句”，但 C++ 里并无这种东西；我们其实是把两条 if 语句组合了起来。if 语句的一般形式是

```
if ( expression ) statement else statement
```

也就是说：一个 if，跟着括号中的一个表达式，跟着一条语句，跟着一个 else，再跟着一条语句。我们所做的，是把一条 if 语句用作另一条 if 语句的 else 部分：

```
if ( expression ) statement else if ( expression ) statement else statement
```

对我们的程序来说，就得到了这样的结构：

```
if (unit == 'i')
    . . .           // 第一种选择
else if (unit == 'c')
    . . .           // 第二种选择
else
    . . .           // 第三种选择
```

这样，我们就能写出任意复杂的测试，并为每个备选项关联一条语句。不过请记住：代码的理想之一是简单，而不是复杂。写出最复杂的程序，并不能证明你有多聪明；写出完成此项任务的最简单的代码，才是能力的证明。

试一试

以上面的例子为样板，写一个把日元、欧元和英镑换算成美元的程序。如果你想把真实性做得再好一些，可以到网上查一查当前的汇率。

4.4.1.2 switch 语句

实际上，把 unit 和'i' 以及'c' 逐个比较，是选择问题中最常见形式的一个例子：把一个值与若干常量分别比较，依比较结果进行选择。这种选择太常见了，C++ 专门为它提供了一条语句：switch 语句。我们可以把例子重写成

```
int main()
{
    constexpr double cm_per_inch = 2.54;    // 一英寸合
                                            // 多少厘米
    double length = 1;                      // 长度，单位英寸或
                                            // 厘米

    char unit = 'a';
    cout<< "Please enter a length followed by a unit (c or i):\n";
    cin >> length >> unit;
    switch (unit) {
    case 'i':
        cout << length << "in == " << cm_per_inch*length << "cm\n";
        break;
    case 'c':
        cout << length << "cm == " << length/cm_per_inch << "in\n";
        break;
    default:
        cout << "Sorry, I don't know a unit called '" << unit << "'\n";
        break;
    }
}
```

switch 语句的语法很古老，但仍然比嵌套的 if 语句清楚，尤其是当我们要和很多常量比较时。写在 switch 后面括号中的那个值，要与一组常量做比较。每个常量都作为一个 case 标签（case label）的一部分。如果该值等于某个 case 标签中的常量，就选中这个 case 的语句。每个 case 都以一个 break 结尾。如果该值不匹配任何 case 标签，就选中由 default 标签标识的语句。default 并非必需，但除非你绝对确信自己已经列出了所有备选项，否则带上它是个好主意。如果你还不明白，编程会教你：对任何事都做到绝对确信（而且是正确的确信）有多难。

4.4.1.3 switch 的技术细节

下面是关于 switch 语句的一些技术细节：

1．switch 所依据的值，必须是整数、字符（char）或枚举（enumeration，§9.5）类型。特别地，不能对 string 用 switch。

2．case 标签中的值必须是常量表达式（§4.3.1）。特别地，case 标签里不能使用变量。

3．不能用同一个值为两个 case 标签赋值。

4．一条 case 可以挂上多个 case 标签。

5．别忘了用 break 结束每个 case。遗憾的是，就算忘了，编译器多半也不会提醒你。

例如：

```
int main()          // 只能对整数等做 switch
{
    cout << "Do you like fish?\n";
    string s;
    cin >> s;
    switch (s) {        // 错误：值必须是整数、字符或枚举类型
        case "no":
            // ...
            break;
        case "yes":
            // ...
            break;
    }
}
```

要依据一个 string 来选择，你就必须使用 if 语句或 map（第 21 章）。

switch 语句会针对“与一组常量做比较”生成优化的代码。对较大的常量集合而言，这通常比一堆 if 语句产生更高效的代码。不过这也意味着，case 标签的值必须是常量，而且必须互不相同。例如：

```
int main()          // case 标签必须是常量
{
    // 定义备选项：
    int y = 'y';        // 这会惹麻烦
    constexpr char n = 'n';
    constexpr char m = '?';
    cout << "Do you like fish?\n";
    char a;
    cin >> a;
    switch (a) {
        case n:
            // ...
            break;
        case y:         // 错误：case 标签中有变量
            // ...
            break;
        case m:
            // ...
            break;
        case 'n':       // 错误：case 标签重复（n 的值就是 'n'）
            // ...
            break;
        default:
            // ...
            break;
```

```
    }
}
```

经常有这样的情形：在 switch 里，我们希望对一组值执行同一个动作。重复去写这个动作会很啰嗦，所以你可以用一组 case 标签为同一条语句做标注。例如：

```
int main()      // 可以用多个 case 标签标注一条语句
{
    cout << "Please enter a digit\n";
    char a;
    cin >> a;

    switch (a) {
        case '0': case '2': case '4': case '6': case '8':
            cout << "is even\n";
            break;
        case '1': case '3': case '5': case '7': case '9':
            cout << "is odd\n";
            break;
        default:
            cout << "is not a digit\n";
            break;
    }
}
```

switch 语句最常见的错误，就是忘了用 break 结束一个 case。例如：

```
int main()      // 有问题的代码示例（少了一个 break）
{
    constexpr double cm_per_inch = 2.54;    // 一英寸合
                                            // 多少厘米
    double length = 1;                      // 长度，单位英寸或
                                            // 厘米

    char unit = 'a';
    cout << "Please enter a length followed by a unit (c or i):\n";
    cin >> length >> unit;

    switch (unit) {
        case 'i':
            cout << length << "in == " << cm_per_inch*length << "cm\n";
        case 'c':
            cout << length << "cm == " << length/cm_per_inch << "in\n";
    }
}
```

不幸的是，编译器会接受这段代码；当 case 'i' 执行完毕，程序会径直“掉下去”落进 case 'c'。这样一来，如果你输入 2i，程序将输出

```
2in == 5.08cm
2cm == 0.787402in
```

本人已经提醒过你了！

试一试

改写你上一个“试一试”中的货币换算程序，改用 switch 语句。再加上从人民币（yuan）和克朗（kroner）折算的换算。哪个版本的程序更容易编写、理解和修改？为什么？

4.4.2 迭代

我们很少只把某件事做一次。因此，程序设计语言都提供了便捷的手段，让人能把一件事做上好几遍。这称为重复（repetition），或者——尤其当作用对象是数据结构中的一系列元素时——称为迭代（iteration）。

4.4.2.1 while 语句

作为迭代的一个例子，看看有史以来第一台存储程序式计算机（EDSAC，stored-program computer）上运行的第一个程序。它由 David Wheeler 于 1949 年 5 月 6 日在英国剑桥大学的计算机实验室编写并运行，用来计算并打印一张像这样的简单平方数表：

```
0       0
1       1
2       4
3       9
4       16
    . . .
98      9604
99      9801
```

每一行都是一个数，后面跟一个“制表”字符（tab，即 `'\t'`），再跟这个数的平方。用 C++ 写出来大致是这样：

```
// 计算并打印 0~99 的平方数表
int main()
{
    int i = 0;          // 从 0 开始
    while (i<100) {
        cout << i << '\t' << square(i) << '\n';
        ++i;            // 递增 i（即 i 变成 i+1）
    }
}
```

记号 `square(i)` 的意思就是 `i` 的平方。它怎么会具有这个意义，我们稍后解释（§4.5）。

当然，这个最早的现代程序并不是真的用 C++ 写的，但它的逻辑一如这里所示：

• 我们从 0 开始。
• 我们看看是否已经到达 100，如果是，就完成了。
• 否则，我们打印这个数和它的平方（中间隔一个制表符 '\t'），把数加 1，然后再来一次。

显然，要做到这一点，我们需要

• 一种重复执行某条语句的手段（循环，loop）
• 一个用来记录已经绕循环走了多少次的变量（循环变量（loop variable）或控制变量（control variable）），这里就是那个名为 i 的 int
• 循环变量的初始化值，这里是 0
• 一个终止条件，这里是我们想在循环里走满 100 次
• 每绕循环一圈要做的事情（循环体（body），即循环的主体）

我们使用的语言构件称为 while 语句（while-statement）。紧跟在它标志性关键字 while 之后，顶部是一个条件，随后是它的循环体：

```
while (i<100)                // 循环条件：检测循环变量 i
{
    cout << i << '\t' << square(i) << '\n';
    ++i ;                    // 递增循环变量 i
}
```

循环体是一个块（block，由花括号界定），它写出表格中的一行，并递增循环变量 i。每绕循环一圈，我们都从检测 i<100 开始。若条件成立，说明还没做完，就可以执行循环体。如果已经到达终点，即 i 为 100，我们就离开 while 语句，去执行接下来的语句。在这个程序里，接下来就是程序结尾，于是便退出了程序。

while 语句的循环变量必须在 while 语句之外（之前）定义并初始化。不定义它，编译器会报错；定义了却忘了初始化，大多数编译器会警告我们，说类似“local variable i not set”（局部变量 i 未设置）的话，但只要我们坚持，它们也会放行这个程序。千万别坚持！编译器就未初始化变量发出的警告几乎总是对的。未初始化的变量是常见的错误来源。在这个例子里，我们写了

```
int i = 0;          // 从 0 开始
```

所以一切正常。

从原则上讲，写一个循环很简单。但要把它写到能正确应对现实世界的问题，却可能很棘手。特别是，把条件表达得正确、并把所有变量初始化到循环能正确启步的状态，都可能很难。

试一试

字符 'b' 是 char('a'+1)，'c' 是 char('a'+2)，如此等等。写一个循环，输出字符与其对应整数值的表：

```
a     97
b     98
. . .
z     122
```

4.4.2.2 块

注意我们是如何把 while 要执行的那两条语句归拢到一起的：

```
while (i<100)  {
    cout << i << '\t' << square(i) << '\n';
    ++i ;        // 递增 i (即 i 变成 i+1)
}
```

由花括号 { 和 } 界定的一串语句，称为一个块（block）或复合语句（compound statement）。块是一种语句。空块 { } 有时也很有用，用来表达不打算做任何事。例如：

```
if (a<=b)  {        // 什么都不做
}
else {              // 交换 a 和 b
    int t = a;
    a = b;
    b = t;
}
```

4.4.2.3 for 语句

对一串数字进行迭代实在太常见了，因此和其他大多数程序设计语言一样，C++ 专门为它准备了一种语法。for 语句（for-statement）就像一个 while 语句，不同的是，对控制变量的管理全部集中到顶部，让人一眼就能看清和读懂。其实，那个“最早程序”也可以这样写：

```
// 计算并打印 0~99 的平方数表
int main()
{
    for (int i = 0; i<100; ++i)
        cout << i << '\t' << square(i) << '\n';
}
```

它的意思是“让 i 从 0 起步执行循环体，每执行完一次循环体就把 i 递增一次，直到到达 100。”一个 for 语句总是等价于某个 while 语句。就此例而言，

```
for (int i = 0; i<100; ++i)
     cout << i << '\t' << square(i) << '\n';
```

的意义为

```
{
    int i = 0;              // for 语句的初始化
    while (i<100) {         // for 语句的条件
        cout << i << '\t' << square(i) << '\n';    // for 语句的循环体
        ++i;                // for 语句的递增
    }
}
```

初学者有的偏爱 while 语句，有的偏爱 for 语句。不过，只要一个循环能用初始化、条件和递增操作都很简单的 for 语句表达出来，用 for 语句写出的代码就更容易理解，也更容易维护。只有在这一前提不成立时，才退而使用 while 语句。

永远不要在 for 语句的循环体内修改循环变量。那会违背每位读者对循环行为的合理预期。看一看这个例子：

```
int main()
{
    for (int i = 0; i<100; ++i) {   // i 走遍 [0:100) 范围
        cout << i << '\t' << square(i) << '\n';
        ++i;     // 这里在搞什么名堂？闻着就像个错误！
    }
}
```

任何看到这个循环的人都会自然而然地认定，循环体会执行 100 次。然而并非如此。循环体里的 `++i` 确保 `i` 每转一圈被递增两次，于是我们只在 `i` 为偶数的 50 个值上得到输出。如果看到这样的代码，我们会认定它是个错误，多半是把 while 语句转换过来时手不干净所致。如果你想每次加 2，就明说：

```
// 计算并打印 [0:100) 范围内偶数的平方数表
int main()
{
    for (int i = 0; i<100; i+=2)
        cout << i << '\t' << square(i) << '\n';
}
```

请注意，更干净、更显式的版本反而比那个乱糟糟的版本更短。这很典型。

试一试

把上一个“试一试”中的字符值示例改写成使用 for 语句的版本。再修改你的程序，让它同时也输出大写字母和数字的整数值表。

还有一种更简单的“范围 for 循环”（range-for-loop），用来遍历 vector 之类的数据集合；见 §4.6。

4.5 函数

在上面这个程序里，`square(i)` 是什么？它是对一个函数（function）的调用。确切地说，是用实参 `i` 去调用名为 `square` 的函数。函数是一个有名字的语句序列。函数可以返回一个结果（也称为返回值（return value））。标准库提供了许多有用的函数，例如我们在 §3.4 用过的平方根函数 `sqrt()`。不过，我们自己动手写函数的场合更多。下面是 square 的一个可信的定义：

```
int square(int x)        // 返回 x 的平方
{
    return x*x;
}
```

这个定义的第一行告诉我们：这是一个函数（圆括号就是标志）、它名叫 square、它接受一个 int 实参（这里叫 x）、它返回一个 int（结果类型在函数声明中总是写在最前面）；也就是说，我们可以这样用它：

```
int main()
{
    cout << square(2) << '\n';      // 打印 4
    cout << square(10) << '\n';     // 打印 100
}
```

函数调用的结果可以弃之不用，但函数要求的实参，我们必须不多不少、如数奉上。看看下面的代码：

```
square(2);               // 多半是个错误：返回值没拿去用
int v1 = square();       // 错误：缺少实参
int v2 = square;         // 错误：缺少括号
int v3 = square(1,2);    // 错误：实参太多
int v4 = square("two");  // 错误：实参类型不对——要求是 int
```

许多编译器会就“返回值未被使用”发出警告，而上述各处错误都如所注地报出来。你可能觉得，计算机应该聪明到能猜出你写下字符串 "two" 时心里想的其实是整数 2。然而 C++ 编译器偏偏就选择不这么聪明。编译器的本分，是在核实你的代码按 C++ 的定义格式无误之后，不折不扣地照你说的去做。如果编译器靠猜来揣摩你的本意，它偶尔就会猜错；到那时，恼火的就是你——或者你的程序的用户。没有编译器“好心”替你揣摩心思，你要预测自己的代码会做什么本来就已经够难了。

函数体（function body）是真正干活的那个块（§4.4.2.2）。

```
{
    return x*x;     // 返回 x 的平方
}
```

对 square 来说，这个活儿微不足道：算出实参的平方，把它作为结果返回。这件事用 C++ 来说，倒比用英语说还容易。对简单的想法而言，这是典型情形——毕竟，设计程序设计语言的目的，就是把这些简单的想法简单而精确地陈述出来。

函数定义的语法可以这样描述：

type identifier (parameter-list) function-body

也就是说：先是一个类型（返回类型），接着是一个标识符（函数名），随后是圆括号括起来的参数列表，再接着是函数体（要执行的语句）。函数要求的实参列表称为参数列表（parameter list），其中的元素称为参数（parameter，或形式参数（formal argument））。参数列表可以为空；如果我们不想返回结果，返回类型就写成 void（意为“空”）。例如：

```
void write_sorry()      // 不接受实参；不返回值
{
    cout << "Sorry\n";
}
```

函数在语言技术层面的细节，第 8 章会作更仔细的考察。

4.5.1 为什么要用函数？

当我们想要一个有名字的独立计算时，就把它定义成函数，因为这样做

- 让这个计算在逻辑上相互分离
- 让程序文本更清晰（通过给计算命名）
- 使函数有可能用在程序的多个地方
- 让测试更省事

在学习过程中，我们会见到体现这些理由的大量例子，偶尔也会点明其中的某条理由。注意，真实世界的程序会用到成千上万个函数，有的甚至达到几十万个。显然，如果程序的各个部件（例如计算）没有被清晰地分离并命名，这样的程序谁也写不出来、更读不明白。另外你很快会发现，许多函数是反复有用的，等价的代码抄来抄去，很快就腻了。比如，你可能乐意写 `x*x` 和 `7*7` 和 `(x+7)*(x+7)` 等等，而不愿写 `square(x)` 和 `square(7)` 和 `square(x+7)`。不过，这仅仅是因为 square 是个非常简单的计算。换成平方根（在 C++ 中叫 `sqrt`）：你肯定更愿意写 `sqrt(x)`、`sqrt(7)`、`sqrt(x+7)` 等等，而不愿把（颇为复杂、长达数行的）计算平方根的代码重复抄上若干遍。更进一步：你甚至根本不必看平方根是怎么算的——知道 `sqrt(x)` 给出 `x` 的平方根就足够了。

在 §8.5 我们会讲到函数的许多技术细节；眼下，我们只再举一个例子。

假使我们想把 `main()` 里的那个循环弄得特别简单，本来可以写成

```
void print_square(int v)
{
    cout << v << '\t' << v*v << '\n';
}

int main()
{
    for (int i = 0; i<100; ++i) print_square(i);
}
```

那我们为什么没采用 `print_square()` 那个版本？它并不比用 `square()` 的版本明显简单，而且注意

- `print_square()` 是个相当专用的函数，别指望日后还能派上用场；而 `square()` 显然是可另作他用的候选
- `square()` 几乎不需要任何文档说明，而 `print_square()` 显然需要解释

这两条背后的根源是：`print_square()` 执行了两个在逻辑上各自分开的动作：

- 它打印。
- 它计算平方。

如果每个函数各自只执行一个逻辑动作，程序通常就更容易写、也更容易懂。从根本上讲，square() 那个版本才是更好的设计。

最后想想：在第一版程序里，我们为什么用 square(i) 而不干脆写 i*i？嗯，函数的用途之一，就是把复杂的计算拆分出去、做成有名有姓的函数。而在 1949 年的那个程序里，还没有硬件直接实现“乘法”，因此 i*i 实际上还是个相当复杂的计算，就像你在纸上手算乘法那样。另外，那版程序的作者 David Wheeler 正是现代计算机中函数（当时称子例程（subroutine））的发明者，所以在这里用上它也算顺理成章。

试一试

实现一个不用乘法运算符的 square()；也就是说，用重复累加来做 x*x（定义一个变量 result，从 0 起步，把 x 加上去，加 x 次）。然后用这个 square() 运行“最早程序”的某个版本。

4.5.2 函数声明

你注意到了吗？调用一个函数所需的全部信息，都在它定义的第一行里。例如：

```
int square(int x)
```

有了这一行，我们就足以下这样的判断：

```
int x = square(44);
```

其实我们根本不必去看函数体。在真实程序里，我们多半也根本不想看。为什么非要看标准库 sqrt() 函数的函数体？我们知道它计算实参的平方根。为什么非要看自己的 square() 函数体？当然，可能只是出于好奇。但几乎所有时候，我们只想知道怎么调用一个函数——看到定义反而分神。所幸，C++ 提供了一种把这类信息单独给出、不随完整函数定义一齐给出的办法，它叫函数声明（function declaration）：

```
int square(int);          // square 的声明
double sqrt(double);      // sqrt 的声明
```

注意结尾处的分号。函数声明用分号收尾；相应的函数声明里用的是函数体：

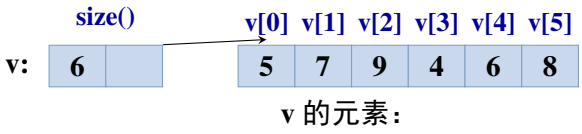
```
int square(int x)         // square 的定义
{
    return x*x;
}
```

因此，如果你只是想用某个函数，就直接写出它的声明——或者更常见的做法是 #include 它。函数的定义可以放在别处。“别处”可以放到哪里，我们在 §8.3 和 §8.7 讨论。声明与定义的这一区分在较大的程序中至关重要：我们靠声明把大多数代码挡在视线之外，让自己一次只专注于程序的一个部分（§4.2）。

4.6 vector

程序要想做点像样的事情，就需要有数据集合可供操作。比如，我们可能需要电话号码的列表、球队队员的列表、课程的列表、去年读过的书的列表、下载歌曲的目录、一组汽车的付款方案、接下来一周的天气预报列表、某个相机在不同网店中的价格列表，等等。可选项真是无穷无尽，因此数据集合在程序中无处不在。我们以后会见到存储数据集合的多种方式（各种各样的容器（container），见第20、21章）。这里先从或最简单、且公认最实用的存储方式讲起：vector。

vector就是一串可以按索引（index）访问的元素。例如，这里有一个名为v的vector：



v的元素：

也就是说，第一个元素的索引是0，第二个是1，如此等等。我们用元素索引去下标（subscript）向量名来指称元素，所以这里v[0]的值是5，v[1]的值是7，如此等等。vector的索引总是从0开始、每次加1。这应当眼熟：标准库vector只不过把这个古老而广为人知的概念搬进了C++标准库。我把这个vector画成那样，就是为了强调它“知道自己的大小”；也就是说，vector不仅存放元素，还存放自己的大小。

我们可以像这样造出这样一个vector：

```
vector<int> v = {5, 7, 9, 4, 6, 8};     // 6 个 int 的 vector
```

可见，创建vector时需要指明元素的类型和初始的一批元素。元素类型写在vector后面的尖括号（<、>）中，这里是<int>。再举一个例子：

```
vector<string> philosopher
        = {"Kant", "Plato", "Hume", "Kierkegaard"};    // 4 个 string 的 vector
```

自然，vector只接受其声明的元素类型的元素：

```
philosopher[2] = 99;    // 错误：想把 int 赋给 string
v[2] = "Hume";          // 错误：想把 string 赋给 int
```

我们也可以定义一个指定了大小、但不给元素值的vector。这时用(n)记法，其中n是元素个数，元素再按元素类型取默认值。例如：

```
vector<int> vi(6);      // 6 个 int 的 vector，初始化为 0
vector<string> vs(4);   // 4 个 string 的 vector，初始化为 ""
```

不含任何字符的字符串""称为空字符串（empty string）。

另外请注意，你不能随手就引用一个vector中不存在的元素：

```
vi[20000] = 44;     // 运行时错误
```

运行时错误和下标操作的原因，我们将在下一章讨论。

4.6.1 遍历 vector

vector 是“知道自己大小”的，所以我们可以这样打印一个 vector 的元素：

```
vector<int> v = {5, 7, 9, 4, 6, 8};
for (int i=0; i<v.size(); ++i)
        cout << v[i] << '\n';
```

调用 `v.size()` 得到名为 v 的 vector 的元素个数。一般地，`v.size()` 让我们能够访问一个 vector 的元素，而不至于不小心碰到其范围之外的元素。vector v 的范围是 `[0:v.size())`。这是半开序列（half-open sequence）的数学记法：v 的第一个元素是 `v[0]`，最后一个是 `v[v.size()-1]`。若 `v.size()==0`，则 v 没有元素，也就是说 v 是空 vector。这个半开序列的概念贯穿整个 C++ 与 C++ 标准库（§17.3、§20.3）。

语言借助半开序列的概念，为序列（例如 vector 的元素）全体元素提供了一种简单的循环写法。例如：

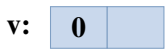
```
vector<int> v = {5, 7, 9, 4, 6, 8};
for (int x : v)      // 对 v 中的每个 x
        cout << x << '\n';
```

这称为范围 for 循环（range-for-loop），因为 `range` 这个词常被用来表示“元素的序列”。我们把 `for (int x : v)` 读作“对 v 中的每个 `int x`”，其含义与按下标 `[0:v.size())` 轮转的等价循环一模一样。对序列中全部元素逐个查看的简单循环，我们就用范围 for 循环来写。更复杂的循环——例如只看 vector 的每第三个元素、只看 vector 的后半段，或者比较两个 vector 的元素——通常还是用更复杂也更通用的传统 for 语句（§4.4.2.3）来写为好。

4.6.2 让 vector 增长

常见的情形是：先让 vector 保持为空，然后一边读入或计算出想要的数据，一边把它增长到所需的规模。这里的关键操作是 `push_back()`，它向 vector 追加一个新元素；新元素成为 vector 的最后一个元素。例如：

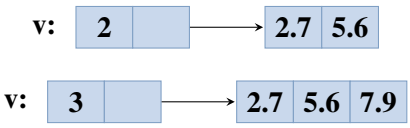
```
vector<double> v;    // 起初为空；也就是说，v 没有任何元素
```




```
v.push_back(2.7);    // 在 v 的末尾（“后端”）追加一个值为 2.7 的元素
                     // v 现在有一个元素，v[0]==2.7
```




```
v.push_back(5.6);    // 在 v 的末尾追加一个值为 5.6 的元素
                     // v 现在有两个元素，v[1]==5.6
```




```
v.push_back(7.9);     // 在 v 的末尾追加一个值为 7.9 的元素
                      // v 现在有三个元素，v[2]==7.9
```

注意 push_back() 调用的语法。它称为成员函数调用（member function call）；push_back() 是 vector 的成员函数（member function），必须用如下点记法来调用：

```
member-function-call:
    object_name.member-function-name ( argument-list )
```

vector 的大小可以通过调用 vector 的另一个成员函数 size() 得到。最初 v.size() 是 0，第三次调用 push_back() 之后，v.size() 变成了 3。

如果以前编过程，你会觉得 vector 和 C 等其他语言中的数组（array）相似。不过，你不必预先说明 vector 的大小（长度），想加多少个元素都可以。接着读下去，你会发现 C++ 标准 vector 还有其他一些有用性质。

4.6.3 一个数值实例

让我们看一个更贴近现实的例子。我们常常要把一串值读入程序，好对它们做些事情。所谓“做些事情”，可能是给这些值画一张图、计算平均值和中位数、找出最大的元素、给它们排序、把它们与其他数据组合起来、搜索“有趣”的值、把它们与其他数据作比较，等等。对数据能做的计算无穷无尽，但首先得把数据送进计算机内存。下面就是把一份事先未知——可能还相当大——的数据读入计算机的基本技术。作为具体例子，我们选择读入表示温度的浮点数：

```
// 把一些温度读入一个 vector
int main()
{
    vector<double> temps;               // 温度
    for (double temp; cin>>temp; )      // 读入 temp
            temps.push_back(temp);      // 把 temp 放进 vector
        // . . . 做些什么 . . .
}
```

这运行起来发生了什么？我们先声明一个 vector 来盛放数据：

```
vector<double> temps;          // 温度
```

在这一行，我们指明了预期的输入类型：读入并存储 double。

接下来才轮到真正的读取循环：

```
for (double temp; cin>>temp; )      // 读入 temp
        temps.push_back(temp);      // 把 temp 放进 vector
```

我们定义了一个 double 类型的变量 temp 来接收读入的数据。cin>>temp 读入一个 double，然后这个 double 被压入 vector（追加在末尾）。这些操作我们都已经分别见过。新东西在于：我们把输入操作 cin>>temp 用作 for 语句的条件。大体上说，cin>>temp 在值被读对了时为真，否则为假，于是这个 for 语句会读入我们输入的所有 double，直到输入的内容不再是数字才停下。比如，如果输入

```
1.2 3.4 5.6 7.8 9.0 |
```

那么 temps 就会得到五个元素 1.2、3.4、5.6、7.8、9.0（依此顺序，例如 temps[0]==1.2）。我们用字符 '|' 终止输入——任何不是 double 的字符都可以。在 §10.6 我们将讨论如何终止输入，以及如何处理输入中的错误。

为了把输入变量 temp 的作用域限制在循环之内，我们用的是 for 语句而不是 while 语句：

```
    double temp;
    while (cin>>temp)               // 读入
            temps.push_back(temp);      // 放进 vector
// …这里或许还会用到 temp …
```

和往常一样，for 循环把事情“摆在明面上”，代码因之更易理解，也更容易避免不计的失误。

把数据装进 vector 之后，操纵它就很容易了。作为例子，我们来计算平均温度和中位温度：

```
// 计算平均温度和中位温度
int main()
{
    vector<double> temps;             // 温度
    for (double temp; cin>>temp; )    // 读入 temp
            temps.push_back(temp);  // 把 temp 放进 vector

    // 计算平均温度:
    double sum = 0;
    for (int x : temps) sum += x;
    cout << "Average temperature: " << sum/temps.size() << '\n';

    // 计算中位温度:
    sort(temps);           // 给温度排序
    cout << "Median temperature: " << temps[temps.size()/2] << '\n';
}
```

求平均（mean），就是把所有元素累加进 sum，然后把总和除以元素个数（即 temps.size()）：

```
// 计算平均温度:
double sum = 0;
for (int x : temps) sum += x;
cout << "Average temperature: " << sum/temps.size() << '\n';
```

注意 += 运算符在这里多么顺手。

要计算中位数（median，这样一个值：一半的值小于它，另一半大于它），我们需要先把元素排序。为此，我们用标准库 sort 算法的一个变体（`sort()`）：

```
// 计算中位温度:
sort(temps);           // 给温度排序
cout << "Median temperature: " << temps[temps.size()/2] << '\n';
```

标准库算法我们要到很久之后才详细解释（第 20 章）。温度一排好序，找中位数就很容易了：直接取中间那个元素，也就是下标为 `temps.size()/2` 的那个。如果你愿意挑剔一下（如果你真的挑剔，说明你已经开始像程序员那样思考了），你会发现：按上面给出的定义，我们找到的这个值未必是真正的中位数。本章末尾的习题 2 正是专门用来解决这个小问题的。

4.6.4 一个文本实例

举温度的例子，并不是因为我们特别关心温度。气象学家、农学家和海洋学家这类人对温度数据以及由它得出的平均值、中位数确实兴趣盎然，但我们对此并不在意。从程序员的角度看，这个例子有趣之处在其通用性：vector 以及对它的简单操作可以用在多如繁星的应用中。可以这么说：无论你关心什么领域，只要需要分析数据，你都会用上 vector（或类似的数据结构，见第 21 章）。举个例子，让我们造一个简单的词典（dictionary）：

```
// 简单词典: 排好序的单词列表
int main()
{
    vector<string> words;
    for(string temp; cin>>temp; )    // 读入以空白分隔的单词
            words.push_back(temp); // 放进 vector
    cout << "Number of words: " << words.size() << '\n';

    sort(words);           // 给单词排序

    for (int i = 0; i<words.size(); ++i)
            if (i==0 || words[i-1]!=words[i]) // 这是一个新词吗？
                    cout << words[i] << "\n";
}
```

给这个程序输入几个单词，它就会按序写出它们，且不重复。例如，给定

```
a man a plan a canal panama
```

它将输出

```
a
canal
man
panama
plan
```

怎样让程序停止读取字符串输入？换句话说，怎么终止这个输入循环？

```
for (string temp; cin>>temp; )      // 读入
        words.push_back(temp);      // 放进 vector
```

读数字的时候（§4.6.2），我们只要随便输入一个不是数字的字符就行。这里可不能照搬，因为任何（寻常的）字符都可以读进 `string`。所幸，还有一些字符是“不寻常”的。如 §3.5.1 所述，在 Windows 下用 Ctrl+Z 终止输入流，Unix 下则用 Ctrl+D。

这个程序的大部分与温度那部分惊人地相似。事实上，这个“词典程序”就是从“温度程序”复制粘贴改写出来的。唯一新鲜的东西只有那个判断

```
if (i==0 || words[i-1]!=words[i])       // 这是一个新词吗？
```

如果删掉这个判断，输出将是

```
a
a
a
canal
man
panama
plan
```

我们不喜欢重复，就用那个判断把它消灭掉了。这个判断是干什么的？它查看上一次打印的词与这次即将打印的词（`words[i-1]!=words[i]`）是否不同：若不同，就打印这个词；否则不打印。显然，当要打印第一个词（`i==0`）的时候，谈不上“上一个词”，所以我们先做 `i==0` 的判断，再用 `||`（或）运算符把这两个判断合起来：

```
if (i==0 || words[i-1]!=words[i])       // 这是一个新词吗？
```

注意，string 是可以比较的。这里我们用 `!=`（不等于）；`==`（等于）、`<`（小于）、`<=`（小于等于）、`>`（大于）和 `>=`（大于等于）对 string 也都适用。`<`、`>` 等运算符采用通常的字典序（lexicographical ordering），因此”Ape” 排在”Apple” 和”Chimpanzee” 之前。

试一试

写一个程序，把你不喜欢的话“消音”（bleep）掉；也就是说，用 `cin` 读入单词，再逐一输出到 `cout`。如果某个词在你事先定义的少数几个词之中，就输出 BLEEP 来代替它。先从一个“不喜欢的词”开始，比如

```
string disliked = "Broccoli";
```

当这个版本能跑通之后，再加上几个。

4.7 语言特性

温度程序和词典程序用上了本章所介绍的绝大多数基本语言特性：迭代（for 语句和 while 语句）、选择（if 语句）、简单算术（++ 和 += 运算符）、比较与逻辑运算符（==、!= 和 || 运算符）、变量，以及函数（例如 `main()`、`sort()` 和 `size()`）。此外，我们还用到了标准库设施，例如 vector（元素的容器）、cout（输出流）和 `sort()`（一种算法）。

数一数你会发现，我们其实用相当少的特性干成了相当多的事。这就是理想所在！每一个语言特性都是为表达一个基本思想而存在的，把这些特性以近乎无限的方式组合，就能写出有用的程序。这是一个关键观念：计算机不是一件功能固定的小玩意儿。它是一台机器，我们可以为它编程，让它去做我们能想到的任何计算；而且，既然可以把计算机接到能与外部世界交互的设备上，原则上我们就能让它做任何事。

小试身手

这个测试要一步一步地做，不要指望跳步图快。每完成一步，都至少输入三对值来测试——多多益善。

1．写一个程序，它由一个 while 循环组成；循环每转一圈，读入两个 int 并把它们打印出来。当读入终止符'|' 时退出程序。

2．把程序改成：输出 the smaller value is:，后跟两数中较小者；再输出 the larger value is:，后跟较大者。

3．扩充程序：仅当两数相等时，输出一行 the numbers are equal（only）。

4．把程序改成用 double 而不是 int。

5．当两数之差小于 1.0/100 时，程序在写出较大者与较小者之后，还要写出 the numbers are almost equal。

6．现在把循环体改成每次只读入一个 double。定义两个变量，随时记录迄今见过的最小者和最大者。每转一圈，都把输入的值写出来：如果它是迄今最小，就在它后面写 the smallest so far；如果它是迄今最大，就在它后面写 the largest so far。

7．给输入的每个 double 加一个单位；也就是说，输入形如 10cm、2.5in、5ft、3.33m 的值。只接受 cm、m、in、ft 四种单位。假定换算率 1m == 100cm、1in == 2.54cm、1ft == 12in。把单位的符号读进一个字符串。你可以认为 12 m（数与单位之间有空格）等价于 12m（无空格）。

8．拒绝没有单位的值，或“非法”的单位写法，比如 y、yard、meter、km 和 gallons。

9．记录输入值的总和（还有最小值和最大值），以及输入值的个数。当循环结束，打印最小值、最大值、值的个数和值的总和。注意：要保留总和，你得为这个总和选定一种单位；就用米（m）。

10．把所有输入的值（换算成米）存进一个 vector。最后，把这些值写出来。

11．在写出 vector 中的值之前，先对它们排序（这样它们会以升序出现）。

要点回顾

1．什么是计算？

2．对一个计算而言，输入和输出指什么？举例说明。

3．程序员表达计算时应牢记哪三个要求？

4．表达式起什么作用？

5．按本章的讲法，语句与表达式有何区别？

6．什么是左值（lvalue）？列出需要左值的运算符。为什么需要左值的是这些运算符，而不是其他运算符？

7．什么是常量表达式（constant expression）？

8．什么是字面值（literal）?
9．什么是符号常量（symbolic constant）? 我们为什么要用符号常量?
10．什么是幻数（magic constant）? 举例说明。
11．哪些运算符既可用于整数、也可用于浮点值?
12．哪些运算符可用于整数、却不能用于浮点数?
13．哪些运算符可用于 string?
14．程序员在什么情况下会更愿意用 switch 语句而不是 if 语句?
15．switch 语句有哪些常见问题?
16．for 循环头部行的各个部分各起什么作用? 它们按什么次序执行?
17．什么时候该用 for 循环，什么时候该用 while 循环?
18．如何打印一个 char 的数值?
19．说明函数定义中 `char foo(int x)` 一行的意义。
20．什么时候应当为程序的一部分单独定义一个函数? 列出理由。
21．对 int 可以做、对 string 却不能做的有哪些?
22．对 string 可以做、对 int 却不能做的有哪些?
23．vector 中第三个元素的索引是多少?
24．如何写一个 for 循环，把一个 vector 的每个元素都打印出来?
25．`vector<char> alphabet(26);` 的作用是什么?
26．说明 `push_back()` 对 vector 做了什么。
27．vector 的成员函数 `begin()`、`end()` 和 `size()` 各做什么?
28．是什么让 vector 如此受欢迎、如此有用?
29．如何对 vector 的元素排序?

术语

抽象（abstraction） `begin()` 计算（computation）
条件语句（conditional statement） 声明（declaration） 定义（definition）
分而治之（divide and conquer） `else` `end()`
表达式（expression） for 语句（for-statement） 范围 for 语句（range-for-statement）
函数（function） if 语句（if-statement） 递增（increment）
输入（input） 迭代（iteration） 循环（loop）
左值（lvalue） 成员函数（member function） 输出（output）
`push_back()` 重复（repetition） 右值（rvalue）
选择（selection） `size()` `sort()`
语句（statement） switch 语句（switch-statement） vector
while 语句（while-statement）

习题

1．如果还没做，就先把本章的试一试练习做完。
2．如果把一个序列的中位数定义为“序列中排在它之前和之后的元素恰好一样多的那个数”，请修正 §4.6.3 的程序，使它总能打印出一个中位数。提示：中位数不必是序列中的元素。
3．把一串 double 值读进一个 vector。把每个值看做给定路线上相邻两个城市之间的距离。计算并打印总距离（所有距离之和）；找出并打印相邻两城市之间的最小距离与最大距离；找出并打印相邻两城市

之间的平均距离。

4．写一个玩猜数游戏的程序。用户心里想一个 1 到 100 之间的数，你的程序提出问题（如“你心里想的数小于 50 吗?”）来猜出它是多少。程序提问不超过七次就应该能确定这个数。提示：使用 < 和 <= 运算符以及 if-else 结构。

5．写一个程序，扮演一台非常简单的计算器。这台计算器要能对两个输入值执行四种基本运算——加、减、乘、除。程序提示用户输入三个实参：两个 double 值和一个表示运算的字符。如果输入实参为 35.6、24.1 和’+’，程序的输出应为 The sum of 35.6 and 24.1 is 59.7。第 6 章我们将考察一台高级得多的简单计算器。

6．建一个存放 10 个 string 值”zero”、”one”、…、”nine” 的 vector。在程序中使用它：把一个数字转换为对应的拼写形式，例如输入 7 输出 seven。用同一个程序、同一个输入循环，反过来把拼出的数字转回数字形式，例如输入 seven 输出 7。

7．修改习题 5 的“mini calculator”（迷你计算器），让它只接受以数字或拼写形式写出的个位数。

8．有个古老的故事：皇帝要感谢国际象棋的发明者，并请他开口要赏赐。发明者要求在棋盘第一格放一粒米，第二格 2 粒，第三格 4 粒，如此倍增，直到 64 格放满为止。这听着不起眼，可其实整个帝国都没有那么多米！写一个程序，算出让发明者至少得到 1000 粒米、至少 1,000,000 粒米、至少 1,000,000,000 粒米，各需要多少格。当然，你得用一个循环；多半还需要一个 int 来记录当前走到第几格，一个 int 记录当前格上的米粒数，一个 int 记录此前所有格子上的米粒总数。我们建议你每一圈循环都把所有变量的值写出来，好看清发生了什么。

9．试着算一算习题 8 里发明者要的米粒数。你会发现这个数大得连 int 或 double 都装不下。观察当数大到 int 无法事先预案精确表示时会发生什么；double 同理。用 int 能精确计算米粒数的最大格数是多少？用 double 能近似计算米粒数的最大格数又是多少？

10．写一个程序玩“石头、剪刀、布”。如果你不熟悉这个游戏，就先做点研究（比如上网用 Google 查查）。做研究是程序员的常见任务。这道题要用 switch 语句来做。另外，机器应当随机出招（也就是随机选出下一步的石头、剪刀或布）。真随机眼下还难以做到，可以先建一个 vector，装入一串值作为“下一个值”来用。如果这个 vector 固定写在程序里，它就会一直玩同一种招式，所以也许应该让用户输入一些值。多试几种变化，让用户更难猜中机器的下一招。

11．写一个程序，找出 1 到 100 之间的所有素数。一种做法是写一个函数判断某数是否为素数（即看它能否被比自己小的素数整除），判断时按顺序用素数 vector（设这个 vector 叫做 primes，则 primes[0]==2、primes[1]==3、primes[2]==5 等等）。然后写一个从 1 到 100 的循环，检查每个数是不是素数，每发现一个素数就存进一个 vector。再写一个循环，列出你找到的素数。你可以把得到的素数 vector 与 primes 相比，来检验结果。把 2 视作第一个素数。

12．修改习题 11 所描述的程序，使其先接受输入值 max，然后找出 1 到 max 的所有素数。

13．写一个程序，找出 1 到 100 之间的所有素数。有一种经典方法可以做到，叫“埃拉托色尼筛法”（Sieve of Eratosthenes）。如果不知道这个方法，就上网查一查。用这种方法写你的程序。

14．修改习题 13 所描述的程序，使其先接受输入值 max，然后找出 1 到 max 的所有素数。

15．写一个程序，先输入一个值 n，然后找出最前面的 n 个素数。

16．在小试身手里，你写过一个程序：给一列数，求其最大值和最小值。在一个序列中出现次数最多的数称为众数（mode）。写一个程序，求一组正整数的众数。

17．写一个程序，求一个 string 序列的最小值、最大值和众数。

18．写一个程序求解二次方程。二次方程的形式是

$$ax^2 + bx + c = 0$$

如果你不知道求解这种表达式的二次公式（quadratic formula），就做些研究。记住，研究怎么解一个问题，正是程序员教计算机怎么解它之前常常要做的事。a、b 和 c 的用户输入用 double。由于二次方程有两个解，请把 x1 和 x2 都输出。

19.写一个程序，先输入一组名字—值对，比如 Joe 17 和 Barbara 22。对每一对，把名字加进名为 names 的 vector，把数值加进名为 scores 的 vector（放进对应的位置：如果 names[7]==”Joe”，则 scores[7]==17）。输入以 NoName 0 终止。检查每个名字是否唯一；一旦有名字被重复输入，就输出错误信息并终止。把所有 (名字, 分数) 对一行一个地写出来。

20. 修改习题 19 的程序：输入一个名字时，程序输出对应的分数，或输出 name not found。

21. 修改习题 19 的程序：输入一个整数时，程序输出所有持有这个分数的名字，或输出 score not found。

后记

从哲学的角度看，你现在已经能做用计算机做的一切——剩下的只是细节！除此之外，这也看得出“细节”的价值，也可见实际技能的重要，因为显然，作为程序员你才刚刚起步。但我们是认真的。本章给出的工具确实能让你表达任何计算：你拥有想要多少就有多少的变量（包括 vector 和 string），拥有算术与比较，还拥有选择与迭代。每个计算都能用这些基本构件表达。你有文本和数值的输入输出，而任何输入或输出都能表达成文本（连图形也不在话下）。你甚至能把计算组织成若干有名有姓的函数。剩下的就“不过”是学会写出好的程序，也就是写出正确、易维护、而且效率还说得过去的程序。重要的是，你必须努力以不算太费劲的方式学会做到这一点。

5 错误

“我意识到，从今往后，我生命中的很大一部分都将花在寻找和改正我自己犯的错误上。”
——Maurice Wilkes，1949

在本章中，我们讨论程序的正确性、错误（error）与错误处理（error handling）。如果你是一个真正的新手，有时会觉得这些讨论有点抽象，有时又觉得它细致得让人痛苦。错误处理真的如此重要吗？是的。而且在你能写出别人愿意使用的程序之前，你迟早会以这种或那种方式认识到这一点。我们要向你展示“像程序员一样的思考”到底是怎么回事：它把相当抽象的策略，与对细节和各种备选方案穷追不舍的分析结合在一起。

5.1 引言

前几章里我们已经反复提到错误——做过小试身手和若干习题之后，你应该已经约略明白是为什么了。开发程序时，错误在所难免；但最终的程序必须没有错误，或者至少没有我们认为不可接受的错误。

把错误分类的方法有很多，例如：

- 编译时错误（*compile-time errors*）：由编译器发现的错误。还可以按违反了哪条语言规则进一步分类，例如：
  - 语法错误（syntax errors）
  - 类型错误（type errors）
- 链接时错误（*link-time errors*）：链接器在把目标文件（object file）组合成可执行程序时发现的错误。
- 运行时错误（*run-time errors*）：由对运行中的程序所做的检查发现的错误。还可以进一步细分为：
  - 由计算机（硬件和/或操作系统）检测到的错误
  - 由库（例如标准库）检测到的错误
  - 由用户代码检测到的错误
- 逻辑错误（*logic errors*）：程序员在查找错误结果的原因时发现的错误。

5.1 引言
5.2 错误的来源
5.3 编译时错误
5.3.1 语法错误
5.3.2 类型错误
5.3.3 非错误
5.4 链接时错误
5.5 运行时错误
5.5.1 由调用者处理错误
5.5.2 由被调函数处理错误
5.5.3 错误报告
5.6 异常
5.6.1 坏实参
5.6.2 范围错误
5.6.3 坏输入
5.6.4 窄化错误
5.7 逻辑错误
5.8 估算
5.9 调试
5.9.1 实用调试建议
5.10 前置条件与后置条件
5.10.1 后置条件
5.11 测试

有一种诱惑，是说我们做程序员的工作就是消灭所有错误。这当然是理想，但往往并不可行。事实上，对真实世界的程序来说，很难说“所有错误”到底指什么。如果你的程序正在你的计算机上运行，而我们把电源线拔了，那是你应当处理的错误吗？多数情况下，答案显然是“当然不是”；但如果我们说的是一个医疗监护程序，或者电话交换机的控制程序呢？在这些场合，用户完全有理由期望：即使你的计算机断电，即使宇宙射线损坏了存放你程序的内存，你的程序所在的系统中也总有某些部分能够合理应对。关键的问题于是变成：“我的程序应该检测这个错误吗？”除非特别声明，我们假定你的程序：

1. 对所有合法输入都应当产生期望的结果
2. 对所有非法输入都应当给出合理的错误消息
3. 不必担心硬件失灵
4. 不必担心系统软件失灵
5. 发现错误之后，允许就此终止程序

凡是假设 3、4 或 5 不成立的程序，基本上都可以视为高级程序，超出了本书的范围。但是，假设 1 和 2 属于基本职业素养的范畴，而职业素养正是我们的目标之一。即便我们并不能百分之百达到这个理想，它也必须是我们的理想。

写程序的时候，出现错误是自然的、不可避免的；问题在于：我们如何应对？据我们估计，在开发严肃软件的过程中，避免、发现和改正错误要占用的工作量在 90% 以上。对安全攸关的程序而言，投入还会更大。小程序你可以做得好得多；反过来，如果马虎了事，你也很容易做得更糟。

基本上，我们提供三种途径来实现可接受的软件：

- 通过组织软件来尽量减少错误。
- 通过调试和测试，消除我们所犯的大部分错误。
- 确保剩下的错误不严重。

这几种办法没有哪种能单独把错误完全消灭；三种都必须用上。

在产出可靠的程序——错误率可接受、可以信赖它会做分内之事的程序——方面，经验是极其重要的。请不要忘记，理想是我们的程序永远把事情做对。我们通常只能逼近这个理想，但这并非不尽全力的借口。

5.2 错误的来源

下面这些是错误的一些来源：

- 糟糕的规格说明（*poor specification*）：如果对程序应当做什么含糊其辞，就不太可能彻底检查那些“黑暗角角”，并确保所有情况都得到处理（即每个输入要么得到正确的答案，要么得到恰当的错误消息）。
- 未完成的程序（*incomplete programs*）：开发过程中，显然总有一些情况是我们尚未处理到的。这在所难免。我们要争取做到的，是知道什么时候所有情况都已经处理了。
- 意外的实参（*unexpected arguments*）：函数要接收实参。如果一个函数被传入了我们没有处理的实参，就有问题了。例如以 -1.2 调用标准库的平方根函数：`sqrt(-1.2)`。既然以 `double` 作参数的 `sqrt()` 返回一个 `double`，不可能存在正确的返回值。§5.5.3 讨论这类问题。
- 意外的输入（*unexpected input*）：程序通常要读入数据（来自键盘、文件、GUI、网络连接等）。程序对这些输入做了许多假设，例如假设用户会输入一个数。万一用户输入的是“aw, shut up!”而不是预期的整数呢？§5.6.3 和 §10.6 讨论这类问题。
- 意外的状态（*unexpected state*）：大多数程序都会保存大量数据（“状态”），供系统的不同部分使用。例子有地址簿、电话目录和温度读数的 `vector`。万一这样的数据不完整或者不对呢？程序的各个部分届时仍然必须设法应对。§26.3.5 讨论这类问题。
- 逻辑错误（*logical errors*）：也就是代码全然没有做到它本该做的事；这类问题我们只能自己去找自己修。§6.6 和 §6.9 给出了发现这类问题的例子。

这份清单有一个实际用途：考察一个程序的进度时可以用它当检查单。只有考虑过所有这些潜在的错误来源，程序才算完备。实际上，明智之举是从项目的一开始就牢记它们，因为对一个不假思索、随手拼凑出来的程序而言，不经过一次伤筋动骨的重写，它的错误几乎不可能被完全查明并清除。

5.3 编译时错误

编写程序时，编译器是你抵御错误的第一道防线。在生成代码之前，编译器会分析代码，检测语法错误和类型错误；只有当它查明程序完全符合语言规范，才会放你继续。编译器发现的许多错误，只是打字错误或源代码编辑没做完造成的“傻错误”，另一些则源自我们对程序各部分如何交互的理解中的问题。对初学者来说，编译器常常显得斤斤计较；但随着你学会用语言设施——尤其是类型系统——直接表达自己的想法，你会慢慢欣赏这种能力：它查出的问题，要不然就会害你花上好几个小时，枯燥地搜寻 bug。

作为示例，我们来看对这个简单函数的几次调用：

```
int area(int length, int width);     // 计算矩形面积
```

5.3.1 语法错误

如果我们像下面这样调用 area() 会怎样：

```
int s1 = area(7;      // 错误：缺少 )
int s2 = area(7)      // 错误：缺少 ;
Int s3 = area(7);     // 错误：Int 不是类型
int s4 = area('7);    // 错误：未终结的字符常量（缺少 '）
```

上面每一行都有一个语法错误；也就是说，按照 C++ 语法它们的构成不合格，因此编译器会拒绝它们。遗憾的是，语法错误的报告不一定总是能以你——程序员——容易理解的方式给出。这是因为编译器可能必须再往后多读一点，才能确信那里确实有错误。结果是，虽然语法错误往往完全小到家（一旦找到，你常常难以相信自己竟会犯这样的错误），但报错往往晦涩难懂，而且时不时指向程序更靠后面的某一行。所以，遇到语法错误时，如果编译器指的那一行看不出任何毛病，还要看看程序中之前的几行。

注意，编译器并不知道你试图做什么，所以它不可能以你的意图为措辞来报错，只能就你实际做的事而言。例如，对于上面 s3 声明中的错误，编译器不大可能说：

“你把 int 拼错了，别把 i 大写。”

更可能的是，它会说出类似这样的话：

“syntax error: missing ’;’ before identifier ’s3’”

“’s3’ missing storage-class or type identifiers”

“’Int’ missing storage-class or type identifiers”

这样的消息在你习惯之前往往难以读懂，而且用的词汇晦涩难解。不同的编译器对同一段代码给出的消息面貌可能差别很大。幸好，这类东西你很快就能读惯了。毕竟，扫一眼那些晦涩的行，可以理解为：

“s3 之前有语法错误，

而且它与 Int 的类型或 s3 有点关系。”

既然如此，找这个问题也用不着什么高深学问。

试一试

试着编译那些示例，看看编译器如何回应。

5.3.2 类型错误

一旦清除了语法错误，编译器就会开始报类型错误；也就是说，它会报告你为变量、函数等声明的类型（或忘了声明的类型），与你赋给它们的值的类型、作为函数实参传入的表达式的类型等之间的不匹配。例如：

```
int x0 = arena(7);           // 错误：未声明的函数
int x1 = area(7);            // 错误：实参数目不对
int x2 = area("seven",2);    // 错误：第 1 个实参类型不对
```

我们来逐个分析这些错误。

1. 对于 `arena(7)`，我们把 `area` 拼成了 `arena`，于是编译器认为我们想调用一个名叫 `arena` 的函数。（它还能“想”成什么呢？我们就是这么写的。）假如并没有一个名叫 `arena()` 的函数，你会收到一条抱怨“未声明函数”的消息。如果确实有一个叫 `arena` 的函数，而且那个函数也接受 7 为实参，可麻烦就大了：程序能通过编译，却会做出你没料到的事（这是逻辑错误，见 §5.7）。
2. 对于 `area(7)`，编译器检测出实参数目不对。在 C++ 中，每一次函数调用都必须提供预期数目的实参，类型正确、顺序无误。类型系统用得当时，这可以成为避免运行时错误的有力工具（见 §14.1）。
3. 对于 `area("seven",2)`，你也许希望计算机看一看 `"seven"`，然后琢磨出你想说的是整数 7。它不会。一个函数如果需要整数，你就不能给它 string。C++ 支持一些隐式类型转换（见 §3.9），但不包括从 string 到 `int`。编译器不会去猜你想说什么。要是换成 `area("Hovel lane",2)`、`area("7,2")` 和 `area("sieben","zwei")`，你会期望得到什么？

这些只是几例。编译器会替你发现的错误还有很多。

试一试

试着编译那些示例，看看编译器如何回应。再自己想出几个错误，也试一试。

5.3.3 非错误

和编译器打交道的过程中，你会希望它聪明到能弄清你的意图；也就是说，你希望它报告的某些错误并不是错误。这很自然。更出人意料的是，随着经验的增长，你会开始希望编译器拒绝更多的代码，而不是拒绝得更少。看下面：

```
int x4 = area(10,-7);        // OK: 但宽度为 -7 的矩形算什么？
int x5 = area(10.7,9.3);     // OK: 但实际调用的是 area(10,9)
char x6 = area(100,9999);    // OK: 但会发生结果截断
```

对 `x4`，我们从编译器那里得不到任何错误消息。站在编译器的立场，`area(10,-7)` 无可挑剔：`area()` 要求两个整数，你如数奉上了它们；谁也没说过这两个实参必须为正。

对 `x5`，好的编译器会对 `double` 值 10.7 和 9.3 被截断成 `int` 值 10 和 9 提出警告（见 §3.9.2）。不过，（古旧的）语言规则规定，你可以把 `double` 隐式转换成 `int`，因此编译器无权拒绝调用 `area(10.7,9.3)`。

`x6` 的初始化也遭遇了与调用 `area(10.7,9.3)` 同一问题的变体。`area(100,9999)` 返回的 `int`（很可能是 999900）将被赋给一个 `char`。最可能的结果是 `x6` 得到“截断后”的值 -36。同样，好的编译器会给你警告，尽管（古旧的）语言规则使它不能拒绝这段代码。

随着经验增长，你会学会怎样最大限度利用编译器检错的能力，同时规避它已知的弱点。但是别过度自信：“我的程序编译过了”并不等于它能运行。就算能运行，起初它一般也会给出错误的结果，直到你找出自己逻辑中的缺陷为止。

5.4 链接时错误

程序由若干分别编译的部分组成，它们称为翻译单元（translation unit）。程序中的每个函数，在其被用到的每一个翻译单元中，都必须以完全相同的类型加以声明；我们靠头文件来保证这一点，见

§8.3。每个函数在程序中还必须恰好定义一次。这两条规则违犯任何一条，链接器都会报错。§8.3 讨论如何避免链接时错误。现在先看一个可能引发典型链接器错误的程序示例：

```
int area(int length, int width);       // 计算矩形面积


int main()
{
    int x = area(2,3);
}
```

除非我们碰巧在另一个源文件里定义了 area()，并把由那个源文件生成的代码链接到了这段代码，否则链接器会抱怨它找不到 area() 的定义。

area() 的定义必须与我们文件里所用的类型（返回类型和实参类型都）完全一致，也就是：

```
int area(int x, int y) { /* . . . */ }       // " 我们的" area()
```

名字相同但类型不同的函数不会匹配，将被忽略：

```
double area(double x, double y) { /* . . . */ }       // 不是" 我们的" area()


int area(int x, int y, char unit) { /* . . . */ }      // 不是" 我们的" area()
```

注意，拼错的函数名一般不会招致链接器报错。情况恰恰相反：编译器一看到对未声明函数的调用，就会立刻报错。这是好事：编译时错误比链接时错误发现得早，通常也更容易修。

如上所述的函数链接规则，同样适用于程序中所有其他实体（entity），例如变量和类型：给定名字的实体必须有恰好一个定义，但可以有多个声明，而且所有声明必须在类型上严格一致。详见 §8.2-3。

5.5 运行时错误

如果你的程序既没有编译时错误也没有链接时错误，它就能运行了。真正好戏这才开场。编写程序时你能够检测错误，但在运行时一旦捕到了错误，接下来该怎么办却未必那么容易说清。看下面这段代码：

```
int area(int length, int width)      // 计算矩形面积
{
    return length*width;
}


int framed_area(int x, int y)        // 计算框架内的面积
{
    return area(x-2,y-2);
}
```

```
int main()
{
    int x = -1;
    int y = 2;
    int z = 4;
    // . . .
    int area1 = area(x,y);
    int area2 = framed_area(1,z);
    int area3 = framed_area(y,z);
    double ratio = double(area1)/area3;   // 转成 double 以便
                                          // 做浮点除法
}
```

我们用变量 x、y、z（而不是直接把值当作实参写进去），是为了让问题对人类读者不那么显眼，也让编译器更难发现。然而，这些调用导致负值——却表示面积——被赋给了 area1 和 area2。这样的错误结果违背了我们对数学和物理的几乎所有观念，我们应当容忍吗？如果不该，又该由谁来检测错误：是 area() 的调用者，还是 area() 自己？这类错误又该如何报告？

在回答这些问题之前，先看上面代码中 ratio 的计算。它看起来人畜无害。你有没有注意到它有什么不对？如果没有，再看一眼：area3 将是 0，于是 double(area1)/area3 就在除以零。这将引发一个由硬件检测出的错误，程序将带着一条与硬件相关的晦涩消息终止。如果你不检测并且妥善处理运行时错误，那么不得不去面对这种错误的，就轮到你了——或者你的用户。大多数人对这种“硬件违背”的容忍度都很低：对任何不熟悉程序内情的人，所能得到的全部信息不过是“某个地方出了问题!”，这不足以支撑任何建设性的行动，于是我们感到愤怒，恨不得痛骂那个提供程序的家伙。

那么，我们就来着手解决 area() 的实参错误问题。有两个显而易见的选择：

a. 让 area() 的调用者处理坏实参。

b. 让 area()（被调函数）处理坏实参。

5.5.1 由调用者处理错误

先试第一种选择（“让使用者当心!”）。如果 area() 是某个我们无权修改的库里的函数，就只能选这一种。不管是好是坏，这是最常见的方式。

在 main() 中保护对 area(x,y) 的调用相当容易：

```
if (x<=0) error("non-positive x");
if (y<=0) error("non-positive y");
int area1 = area(x,y);
```

其实，唯一要问的问题是发现了错误该怎么办。这里我们调用了函数 error()，假定它会做出一些合理的事来。事实上，我们在 std_lib_facilities.h 中提供了一个 error() 函数，默认行为是终止程序并输出一条系统错误消息，附上我们传递给 error() 的实参字符串。如果你更愿意自己写出错误消息或者采取其他行动，可以去捕获 runtime_error（见 §5.6.2、§7.3、§7.8、§B.2.1）。这一方式足以满足大多数学生程序，也是一种可用于更复杂错误处理的风格示例。

如果不需要对每个实参分别给出错误消息，可以简化成：

```
if (x<=0 || y<=0) error("non-positive area() argument");       // || 表示" 或"
int area1 = area(x,y);
```

为了把 area() 从坏实参那里完全保护起来，还得处理经由 framed_area() 而来的调用。可以写成：

```
if (z<=2)
    error("non-positive 2nd area() argument called by framed_area()");
int area2 = framed_area(1,z);
if (y<=2 || z<=2)
    error("non-positive area() argument called by framed_area()");
int area3 = framed_area(y,z);
```

这段代码乱糟糟，但其中还有更根本的毛病。只有确切知道 framed_area() 是如何使用 area() 的，我们才能写出它来。我们必须知道 framed_area() 对每个实参减了 2。这种细节本不该由我们来知道！万一有人把 framed_area() 改成减 1 而不是 2 呢？做这种改动的人就必须查看每一处对 framed_area() 的调用，并相应地修改错误检查代码。这样的代码被称为“易碎的（brittle）”，因为它一碰就碎。这也是“魔法常量（magic constant）”（§4.3.1）的一个例子。为 framed_area() 所减去的值起个名字，可以让这段代码不那么易碎：

```
constexpr int frame_width = 2;

int framed_area(int x, int y)   // 计算框架内的面积
{
    return area(x-frame_width,y-frame_width);
}
```

调用 framed_area() 的代码里也可以用上这个名字：

```
if (1-frame_width<=0 || z-frame_width<=0)
    error("non-positive argument for area() called by framed_area()");
int area2 = framed_area(1,z);
if (y-frame_width<=0 || z-frame_width<=0)
    error("non-positive argument for area() called by framed_area()");
int area3 = framed_area(y,z);
```

看看这段代码！你确定它是对的吗？你觉得它漂亮吗？容易读吗？说实话，我们觉得它丑（也因此易错）。代码膨胀到了原来的三倍还多，而且还暴露了 framed_area() 的一个实现细节。总该有更好的办法！

再看原始代码：

```
int area2 = framed_area(1,z);
int area3 = framed_area(y,z);
```

它可能是错的，但至少看得出它想做什么。如果把检查放进 framed_area() 内部，我们就可以保留这段代码。

5.5.2 由被调函数处理错误

在 framed_area() 内部检查实参的合法性不难，error() 照样可以用来报告问题：

```
int framed_area(int x, int y)   // 计算框架内的面积
{
    constexpr int frame_width = 2;
    if (x-frame_width<=0 || y-frame_width<=0)
        error("non-positive area() argument called by framed_area()");
    return area(x-frame_width,y-frame_width);
}
```

这样就恰到好处了，而且我们再也不必为 framed_area() 的每一次调用写一遍检查。对一个在大型程序中被调用 500 次的有用函数来说，这可是个巨大的优势。更进一步，哪怕错误处理有任何修改变动，也只需修改一处的代码。

这里有个值得注意的有趣现象：我们几乎是不知不觉地从“调用者必须检查实参”的方式滑向了“函数必须自己检查实参”的方式（也叫“由被调者检查”，因为被调用的函数常被称为 callee，即“被调函数”）。后一种方式的一个好处是实参检查代码集中在一处；我们不必在整个程序中搜索各个调用点。而且这“一处”恰恰就是实参将要被使用的地方，检查所需的全部信息都唾手可得。

把这个解法应用到 area() 上：

```
int area(int length, int width)     // 计算矩形面积
{
    if (length<=0 || width <=0) error("non-positive area() argument");
    return length*width;
}
```

这将会捕获对 area() 调用中的所有错误，所以我们不再需要在 framed_area() 中检查。不过，要想得到更好——更具体——的错误消息，我们也许仍然想在 framed_area() 中加检查。

在函数里检查实参看起来那么简单，为什么人们不总是这样做呢？对错误处理的疏忽是一个原因，马虎是另一个，但其实也有一些站得住脚的理由：

- *我们无权修改函数定义*：函数在一个库里，而由于某种原因这个库不能改。也许它正在被别人使用，而别人并不赞同你对“好的错误处理”的看法；也许它是别人的，你没有源码；也许它所在的库定期出新版本，这样一旦你做了修改，库每出一个新版本你就得重新改一遍。
- *被调函数不知道出错时该做什么*：对库函数来说，这一般就是实情。库的作者能够检测到错误，但只有你才知道错误发生时该做什么。
- *被调函数不知道自己是从哪里被调用的*：当你收到一条错误消息时，它告诉你的是出了问题，而不是正在执行的程序是如何走到这一步的。有时候，你会希望错误消息更具体一些。
- *性能*：对小函数来说，做一次检查的开销可能超过计算结果的开销。例如 area() 的情况就是这样，检查还会使整个函数的大小（也就是需要执行的机器指令的数目，而不仅仅是源代码的长度）增加一倍以上。对某些程序而言，这一点可能至关重要，尤其是当随着函数之间一层层调用，同样的信息被反复检查、并且在传递过程中基本保持原样不动的时候。

那么，你该怎么办？除非有好的理由不这样做，否则就在函数里检查自己的实参。

在考察了几个相关话题之后，§5.10 将回到如何处理坏实参的问题上来。

5.5.3 错误报告

我们考虑一个稍微不同的问题：查过一组实参、发现了错误之后，该怎么办？有时候，你可以返回一个“错误值”。例如：

```
// 问用户一个 yes-or-no 问题;
// 返回 'b' 以表示回答无效 (即既不是 yes 也不是 no)
char ask_user(string question)
{
    cout << question << "? (yes or no)\n";
    string answer = " ";
    cin >> answer;
    if (answer=="y" || answer=="yes") return 'y';
    if (answer=="n" || answer=="no") return 'n';
    return 'b';   // 'b' 表示" 错误回答"
}


// 计算矩形面积;
// 返回 -1 以表示实参无效
int area(int length, int width)
{
    if (length<=0 || width <=0) return -1;
    return length*width;
}
```

这样，我们就可以让被调函数负责细致的检查，同时让每个调用者随心所欲地处理错误。这一办法看起来行得通，但它有几个问题，使得它在很多情况下根本用不上：

- 现在被调函数和所有调用者都必须做检查。调用者要做的不过是一个简单的判定，但仍然得亲手写出来，并决定判定失败时该怎么办。
- 调用者可能忘记检查。这会在程序的后续运行中导致不可预知的行为。
- 许多函数没有可供指示错误的“多余”返回值。例如，一个从输入读取整数的函数（比如 `cin` 的 `operator >>`）显然可以返回任意 `int` 值，因此没有任何 `int` 可以拿来指示失败。

上面第二种情况——调用者忘记检查——很容易带来意外。例如：

```
int f(int x, int y, int z)
{
    int area1 = area(x,y);
    if (area1<=0) error("non-positive area");
    int area2 = framed_area(1,z);
    int area3 = framed_area(y,z);
    double ratio = double(area1)/area3;
    // . . .
}
```

你看出错误了吗？这类错误之所以难找，是因为没有明摆着的“错误代码”可查：错误恰恰在于一次检查的缺席。

试一试

用各种不同的值测试这个程序。打印出 area1、area2、area3 和 ratio 的值。多加一些检查进去，直到所有错误都被捕获。你怎么知道自己已捕获了所有错误？这不是刁难题：在这个具体例子中，你可以给出像样的论证，证明所有错误已被捕获。

还有一种方案可以解决那个问题：使用异常（exception）。

5.6 异常

和大多数现代编程语言一样，C++ 提供了一种帮助处理错误的机制：异常。其基本思想是把错误的检测（应当在被调函数中进行）与错误的处理（应当在调用函数中进行）分离开来，同时确保被检测到的错误不可能被无视；也就是说，异常提供了一种机制，让我们得以把前面探讨过的各种错误处理方法各自的优点结合起来。没有什么东西能让错误处理变得轻松，但异常能让它更容易些。

基本思想是这样的：如果一个函数发现了一个它自己无法处理的错误，它就不做正常返回；而是抛出（throw）一个异常，指明哪里出了问题。任何直接或间接的调用者都可以捕获（catch）异常，也就是说，指定如果被调代码用了 throw 时该怎么做。函数通过使用 try 块（try-block，见后续小节的说明）来表达对异常的关注，并在 try 块的 catch 部分列出它想处理的异常种类。如果没有调用者捕获某个异常，程序就会终止。

我们会在很后面（第 19 章）再回到异常这个话题，看看稍微高级一些的用法。

5.6.1 坏实参

下面是使用异常的 area() 版本：

```
class Bad_area { };    // 专门用来从 area() 报告错误的类型


// 计算矩形面积;
// 实参有问题时抛出 Bad_area 异常
int area(int length, int width)
{
    if (length<=0 || width<=0) throw Bad_area{};
    return length*width;
}
```

这就是说，实参没问题就照常返回面积；有问题就经由 throw 从 area() 中跳出，指望某个 catch 会给出恰当的响应。Bad_area 是我们新定义的一个类型，其唯一的目的就是提供一个独特的东西供 area() 抛出，那样某个 catch 就能把异常认出来，知道它是从 area() 抛出的那一类。我们将在第 9 章讨论用户自定义类型（类和枚举）。记号 Bad_area{} 的意思是“创建一个 Bad_area 类型的、取默认值的对象”，所以 throw Bad_area{} 的意思是“创建一个 Bad_area 类型的对象并把它抛出去”。

现在，我们可以这样写：

```
int main()
try {
    int x = -1;
    int y = 2;
    int z = 4;
    // . . .
    int area1 = area(x,y);
    int area2 = framed_area(1,z);
    int area3 = framed_area(y,z);
    double ratio = area1/area3;
}
catch (Bad_area) {
    cout << "Oops! bad arguments to area()\n";
}
```

首先注意，这套写法处理了对 `area()` 的所有调用：既有 `main()` 里那一次调用，也有经由 `framed_area()` 的那两次调用。其次注意，错误的处理同错误的检测被干干净净地分开了：`main()` 对哪个函数执行了 `throw Bad_area{}` 一无所知，`area()` 对哪个函数（如果有的话）愿意捕获它抛出的 `Bad_area` 异常也一无所知。在使用大量库编写的大型程序中，这种分离尤其重要。在这类程序中，谁也没法“需要在哪儿处理错误，就把一点儿代码就地放在哪儿”了事，因为没有人愿意既去修改应用程序，又去修改所有那些库。

5.6.2 范围错误

大多数现实世界的代码都要处理成批的数据；也就是说，它们要用各式各样的表格、列表等数据元素的集合来做事情。在 C++ 的语境下，我们常把“数据的集合”称为容器（container）。最常用、最有用的标准库容器，是我们在 §4.6 介绍的 vector。一个 vector 持有一定数目的元素，元素的数目可以通过调用该 vector 的 `size()` 成员函数来确定。如果我们试图用超出合法范围 `[0:v.size())` 的下标（subscript，即下标）来使用某个元素，会发生什么？记号 `[low:high)` 表示从 low 到 high-1 的下标，换句话说，包含 low 但不包含 high：



“你干嘛要那样做？”毕竟，你知道 v 的下标应当处于范围 `[0,v.size())` 内，那就确证它确实在嘛！

碰巧，说来容易做来难。考虑下面这个看似合理的小程序：

```
vector<int> v;                          // 一个 int 的 vector
for (int i; cin>>i; )
    v.push_back(i);                     // 读入值
for (int i = 0; i<=v.size(); ++i)       // 打印值
    cout << "v[" << i <<"] == " << v[i] << '\n';
```

你看出错误了吗？请在往下读之前先自己试着找出来。这并不是一种罕见的错误。我们自己就犯过这类错误——尤其是在深夜非常疲惫的时候。累的时候或赶时间的时候，出错总是更频繁。

我们用 0 和 `size()` 来尽力保证在执行 `v[i]` 时 `i` 总是在有效范围内。

不幸的是，我们犯了一个错。看看那个 for 循环：终止条件是 `i<=v.size()`，而非正确的 `i<v.size()`。这带来的不良后果是：如果读入了五个整数，我们就会试图写出六个。我们试图读 `v[5]`，超出了 vector 末尾一个位置。这种错误非常普遍、“出了名”地多，以致它有好几个名字：它是差一错误（off-by-one error）的一个例子；因为下标不在 vector 要求的范围内，它是一个范围错误（range error）；又因为下标不在 vector 的边缘（bounds）之内，它也是一个越界错误（bounds error）。

我们为什么不用 range-for 语句来表达那个循环呢？采用 range-for，我们就不可能把循环的结束边界写错。不过对这个循环来说，我们想要的不仅是每个元素的值，还有下标。而 range-for 不费额外的工夫是给不出这些的。

下面是一个更简单的版本，它产生的范围错误与刚才那个循环一样：

```
vector<int> v(5);
int x = v[5];
```

不过，我们怀疑你会觉得这个例子简直不真实，不值得严肃对待。

那么，犯了这种范围错误时，实际会发生什么？`vector` 的下标操作知道 vector 的大小，所以它能做检查（我们所用的这个 vector 的确做了检查；见 §4.6 和 §19.4）。如果检查失败，下标操作就会抛出一个类型为 `out_of_range` 的异常。所以，如果上面的差一代码本来就是一个捕获异常的程序的一部分，我们至少会得到一条像样的错误消息：

```
int main()
try {
    vector<int> v;                  // 一个 int 的 vector
    for (int x; cin>>x; )
        v.push_back(x);             // 设置值
    for (int i = 0; i<=v.size(); ++i)   // 打印值
        cout << "v[" << i <<"] == " << v[i] << '\n';
} catch (out_of_range) {
    cerr << "Oops! Range error\n";
    return 1;
} catch (...) {                     // 捕获所有其他异常
    cerr << "Exception: something went wrong\n";
    return 2;
}
```

注意，范围错误实际上就是 §5.5.2 讨论的实参错误的一种特例。我们不放心让自己始终如一地检查 vector 下标的范围，于是把这件事交代给 vector 的下标操作去做。基于前面给出的那些理由，vector

的下标函数（称为 vector::operator[]）发现错误时通过抛异常来报告。它还能怎么办呢？它根本不知道我们希望发生范围错误时发生什么；vector 的作者甚至无从知晓自己的代码会被放进哪些程序里。

5.6.3 坏输入

如何处理坏输入的详细讨论，我们推迟到 §10.6。不过，坏输入一旦检测到，处理它用的就是与实参错误和范围错误相同的技术和语言设施。这里我们只告诉你如何判断输入操作是否成功。考虑读入一个浮点数：

```
double d = 0;
cin >> d;
```

可以通过检测 cin 来判断上一次输入操作是否成功：

```
if (cin) {
    // 一切正常，我们可以继续读
}
else {
    // 上一次读取没有成功，于是我们采取别的行动
}
```

那次输入操作失败的原因可能有几种。眼下应当引起你注意的一种是：根本没有可供 » 读取的 double。

在开发的早期阶段，我们常常想表明自己发现了一个错误，但还没做好准备去做什么特别聪明的事；我们只想报告这个错误并终止程序。也许以后我们会回来，做点更恰当的处理。例如：

```
double some_function()
{
    double d = 0;
    cin >> d;
    if (!cin) error("couldn't read a double in 'some_function()'");
    // 做点有用的事
}
```

条件 !cin（“非 cin”，即 cin 不处于良好状态）意味着对 cin 的上一次操作失败了。

传给 error() 的字符串随后可以被打印出来，以帮助调试或作为给用户的消息。怎样编写 error()，才能让它在很多程序中都有用呢？它不能返回值，因为我们对那个值不知所措；error() 的任务是先写好自己的消息，然后再终止程序。而且，在退出之前，我们可能还想做一点小动作，例如让窗口多保持存活一段时间，好让我们来得及把消息读完。这显然是异常的一项拿手活儿（见 §7.3）。

标准库定义了一些种类的异常类型，例如 vector 抛出的 out_of_range。它还提供了 runtime_error，后者对我们目前的需要来说几乎称心如意，因为它随身携带一个字符串，能够被错误处理器用到。于是，我们可以这样写自己简单的 error()：

```
void error(string s)
{
    throw runtime_error(s);
}
```

当我们想处理 runtime_error 时，简单地捕获它即可。对简单程序来说，在 main() 中捕获 runtime_error 就很理想：

```
int main()
try {
    // . . . 我们的程序 . . .
    return 0;       // 0 表示成功
}
catch (runtime_error& e) {
    cerr << "runtime error: " << e.what() << '\n';
    keep_window_open();
    return 1;       // 1 表示失败
}
```

调用 e.what() 就能从 runtime_error 中取出错误消息。而

```
catch(runtime_error& e) {
```

中的 & 是一个标志，表明我们想把异常“按引用传递（by reference）”。眼下，请把它单纯当作一个无关紧要的技术细节。§8.5.4–6 会另行解释按引用传递某个东西是什么意思。

注意，我们的错误输出用的是 cerr 而不是 cout：cerr 与 cout 几乎一模一样，只是专门用于错误输出。默认情况下，cerr 和 cout 都写到屏幕上，但 cerr 没有经过优化，所以对错误更有韧性，而且在某些操作系统上可以把它改道到别处，比如一个文件。使用 cerr 还有一个简单的附带优点：它以注脚的方式表明我们所写的内容与错误有关。因此我们把错误消息都定向到 cerr。

碰巧，out_of_range 并不是 runtime_error，所以捕获 runtime_error 无法处理我们因误用 vector 或其他标准库容器类型而可能得到的 out_of_range 错误。不过 out_of_range 和 runtime_error 都是“异常”，因此我们可以通过捕获 exception 来一起处理它们：

```
int main()
try {
    // 我们的程序
    return 0;     // 0 表示成功
}
catch (exception& e) {
    cerr << "error: " << e.what() << '\n';
    keep_window_open();
    return 1;     // 1 表示失败
}
catch (...) {
    cerr << "Oops: unknown exception!\n";
    keep_window_open();
```

```
    return 2;     // 2 表示失败
}
```

我们添加了 `catch(...)` 来处理不管什么类型的异常。

通过一个 `exception` 类型——它被说是 `out_of_range` 和 `runtime_error` 二者的共同基类型（超类型 supertype）——来处理这两种异常，是一种极其有用且一般的技巧，我们将在第 13–16 章深入探讨。

再提醒一次，`main()` 的返回值会传给启动这个程序的“系统”。有的系统（比如 Unix）常常会利用这个值，而另一些系统（比如 Windows）一般忽略它。零值表示成功完成，`main()` 的非零返回值表示某种形式的失败。

使用 `error()` 时，你常常希望随它一起传两个信息，来把问题描述清楚。这种情况下，把描述这两条信息的字符串拼接起来即可。这种做法太常见了，所以我们提供了一个用于此目的的第二个版本的 `error()`：

```
void error(string s1, string s2)
{
    throw runtime_error(s1+s2);
}
```

这套简单的错误处理，眼下将够用一阵子，直到我们对错误处理的需求显著增长、而我们作为设计者和程序员的老练程度也相应提高为止。注意，我们使用 `error()` 时，并不关心在错误发生之前我们已经做过了多少次函数调用：`error()` 会自己找到距离最近的 `runtime_error` 的 catch，通常就是 `main()` 里的那一个。异常和 `error()` 用法的例子见 §7.3 和 §7.7。如果不捕获异常，你就会得到默认的系统错误（一条“未被捕获的异常”（uncaught exception）错误消息）。

试一试

想看看未被捕获的异常错误长什么样子，可以运行一个使用 `error()` 而不捕获任何异常的小程序。

5.6.4 窄化错误

在 §3.9.2 里我们见过一种挺讨厌的错误：当我们把一个“大到装不下”的值赋给变量时，这个值会被隐式截断。例如：

```
int x = 2.9;
char c = 1066;
```

这里 `x` 将得到值 2，而不是 2.9，因为 `x` 是一个 `int`，而 `int` 的值只能是完整的整数，没有整数的分数部分（这很显然）。类似地，如果我们使用常见的 ASCII 字符集，`c` 将得到值 42（表示字符 `*`），而不是 1066，因为那个字符集中没有值 1066 的 `char`。

在 §3.9.2 里我们见过怎样通过检测来保护自己免遭这种窄化（narrowing）之害。既然有了异常（还有模板，见 §19.3），我们可以编写一个执行这种检测的函数，一旦赋值或初始化会导致值的改变，就抛出一个 `runtime_error` 异常。例如：

```
int x1 = narrow_cast<int>(2.9);      // 抛出
int x2 = narrow_cast<int>(2.0);      // OK
char c1 = narrow_cast<char>(1066);   // 抛出
char c2 = narrow_cast<char>(85);     // OK
```

尖括号 < . . . > 与 vector<int> 所用的相同。当我们要表达的想法需要指定的是类型而非值时，就用它们。它们叫做模板实参（template argument）。当我们需要转换一个值、但不确定“它装不装得下”时，可以使用 narrow_cast；它定义在 std_lib_facilities.h 中，是用 error() 来实现的。cast 一词的意思是“类型转换”，暗指这一操作在处理“坏损之物”时所扮演的角色（就像骨折后打上的石膏）。注意，cast 并不改变它的操作数；它产生一个新值（类型就是在 < . . . > 中指定的类型），这个新值对应于操作数的值。

5.7 逻辑错误

清除了最初的编译期错误和链接期错误之后，程序就能运行了。接下来发生的事情，通常是没有输出，或者输出根本就是错的。原因可能有很多：也许你对程序底层逻辑的理解有缺陷；也许你写出来的并不是你以为自己写的东西；也许你在某个 if 语句之类的地方犯了“愚蠢的小错”。逻辑错误通常是最难发现和排除的，因为到了这个阶段，计算机做的正是你让它做的事。你现在的任务是弄清楚，为什么那不是你真正想要的。从本质上说，计算机是个跑得飞快的笨蛋：它严格按你说的做，而且这一点能把人难住到极点。

我们用一个简单的例子来说明。看看下面这段代码，它求一组数据中的最低温、最高温和平均温度：

```
int main()
{
    vector<double> temps;               // 气温
    for (double temp; cin>>temp; )      // 读入并放入 temps
        temps.push_back(temp);

    double sum = 0;
    double high_temp = 0;
    double low_temp = 0;

    for (int x : temps)
    {
        if(x > high_temp) high_temp = x;    // 找最高温
        if(x < low_temp) low_temp = x;      // 找最低温
        sum += x;                           // 求和
    }

    cout << "High temperature: " << high_temp << '\n';
    cout << "Low temperature: " << low_temp << '\n';
    cout << "Average temperature: " << sum/temps.size() << '\n';
}
```

我们用 2004 年 2 月 16 日得克萨斯州卢伯克（Lubbock）气象中心的逐小时气温数据测试了这个程序（得克萨斯州还在用华氏度）：

−16.5 −23.2 −24.0 −25.7 −26.1 −18.6 −9.7 −2.4
7.5 12.6 23.8 25.3 28.0 34.8 36.7 41.5
40.3 42.6 39.7 35.4 12.6 6.5 −3.7 −14.3

输出是

```
High temperature: 42.6
Low temperature: -26.1
Average temperature: 9.3
```

一个天真的程序员会据此断定程序工作得很好；一个不负责任的程序员会把它直接交付给客户。谨慎的做法是用另一组数据再测一遍，这次用 2004 年 7 月 23 日的气温：

76.5 73.5 71.0 73.6 70.1 73.5 77.6 85.3
88.5 91.7 95.9 99.2 98.2 100.6 106.3 112.4
110.2 103.6 94.9 91.7 88.4 85.2 85.4 87.7

而这一次，输出是

```
High temperature: 112.4
Low temperature: 0.0
Average temperature: 89.2
```

糟糕！有些地方不对劲。七月里在卢伯克出现硬霜（0.0°F 约为 −18°C）可就意味着世界末日了！你看出错误在哪儿了吗？由于 `low_temp` 的初始值是 `0.0`，除非数据中有某个温度低于零度，不然它就会一直保持 `0.0`。

试一试

让这个程序运行起来。检验我们的输入确实能产生那样的输出。试着“搞坏”这个程序（即给它别的输入，让它的结果出错）。为了搞坏它，最少需要给它多少输入？

遗憾的是，这个程序里还有更多的错误。如果所有温度都在零度以下会怎样？`high_temp` 的初始化有着与 `low_temp` 相同的问题：除非数据中出现更高的温度，否则 `high_temp` 将一直保持 `0.0`。这个程序对南极的冬天同样无能为力。

这些错误很有代表性：编译程序时它们不会引发任何错误，对“合理”的输入也不会给出错误结果。然而，我们忘了想一想，应当把什么算作“合理”。下面是改进后的程序：

```
int main()
{
    double sum = 0;
    double high_temp = -1000;        // 初始化为不可能的低值
    double low_temp = 1000;          // 初始化为“不可能的高值”
    int no_of_temps = 0;

    for (double temp; cin>>temp; ) {    // 读入温度
        ++no_of_temps;       // 统计温度个数
        sum += temp;         // 计算总和
        if (temp > high_temp) high_temp = temp;      // 找最大值
        if (temp < low_temp) low_temp = temp;        // 找最小值
    }

    cout << "High temperature: " << high_temp<< '\n';
    cout << "Low temperature: " << low_temp << '\n';
    cout << "Average temperature: " << sum/no_of_temps << '\n';
}
```

它能正确工作吗？你怎么才能确信这一点？你又该如何精确定义“工作”？1000 和 −1000 这两个值是从哪儿来的？记住，我们曾警告过“魔法常数”（§5.5.1）。把 1000 和 −1000 这样的字面值直接写在程序中间是不好的风格，但这两个值本身也不对吗？有些地方的气温会低于 −1000°F（−573°C）吗？有些地方的气温会高于 1000°F（538°C）吗？

试一试

查一查。查一些资料，为程序中的 **min_temp**（“最低温度”）和 **max_temp**（“最高温度”）这两个常数挑选合适的取值。这些值将决定我们程序的适用范围。

5.8 估算

设想你写了一个做简单计算的程序，比如计算六边形的面积。运行后它给出的面积是 −34.56。你马上就知道这不对。为什么？因为任何形状的面积都不可能是负数。于是你修好那个 bug（不管它是什么），得到 21.65685。这回对了吗？这就难说了，因为我们不会把六边形面积的公式总记在脑子里。在因为交付一个产生荒谬结果的程序而出丑之前，我们必须做的是先检查这个答案是否合理（plausible）。这件事在这里很容易：六边形和一个正方形差不多大。我们在纸上草草画一个正六边形，目测它大约相当于一个 3×3 的正方形，这样的正方形面积是 9。真糟糕，我们的 21.65685 不可能是对的！于是我们再来改进程序，得到 10.3923。这下，可能就对了！

这里的要点与六边形毫无关系。要点是：除非我们对一个正确答案大致是什么样有点概念——哪怕非常粗略——我们就完全无从判断自己的结果是否合理。要经常自问这个问题：

1. 这个问题在这个特定问题中的答案合理吗？

你还应该问一个更一般的（而且往往难得多）问题：

2. 我如何才能认出一个合理的结果？

在这里，我们不是在问“精确答案是什么？”或“正确答案是什么？”——那正是我们写程序要告诉我们的。我们只想要知道答案不荒谬。只有当我们知道自己得到了一个合理的答案，继续做后面的工作才有意义。

估算（estimation）是一种高尚的艺术：把常识和一些非常简单的算术运用到少数几个事实之上。有些人擅长心算估算，但我们更喜欢“在信封背面”打草稿，因为这能让我们少犯糊涂。这里所说的估算，是一套非正式的技巧，有时（开玩笑地）被称作“猜算”（guesstimation），因为它把一点猜测和一点计算结合了起来。

试一试

我们的六边形是边长 2 厘米的正六边形。我们的答案算对了吗？就用“信封背面”的方法算一下。拿一张纸，在上面随便画一画。别觉得这有失身份。许多著名科学家正是因为用铅笔和信封背面（甚至是一张餐巾纸）就能得出近似答案的能力而备受推崇。这是一种能力——其实只是一个简单的习惯——它能为我们省去大量的时间和困惑。

做估算时，常常要给正式计算所需的、但我们还没有的数据提出估值。设想你要测试一个估算城市间驾车时间的程序。从纽约市到丹佛，15 小时 33 分钟的驾车时间合理吗？从伦敦到尼斯呢？为什么合理或不合理？要回答这些问题，你需要“猜”些什么数据？很多时候，快速上网搜索一下会非常有帮助。例如，从纽约市到丹佛的公路里程，2000 英里不算一个很差的猜测；而要保持平均时速 130 英里（而且是合法的）就太难了，所以 15 小时不合理（15*130 只比 2000 略小一点）。你可以验证：我们对距离和平均速度都高估了，但对合理性检查而言，用不着完全精确，只要猜得足够好就行。

试一试

估算那些驾车时间。再估算相应的飞行时间（按普通商业航班计算）。然后，使用地图、时刻表等合适的资料来源验证你的估算。我们会使用网上资料。

5.9 调试

当你写完（打完草稿?）一个程序，它总会有错误。小程序偶尔也能在你第一次尝试时就正确地编译运行；但如果这种事发生在任何一个不是完全平凡的程序上，你首先就要非常、非常怀疑。如果它真的第一次就运行正确，去告诉朋友们庆祝一下吧——因为这不是年年都有的好事。

所以，写完一些代码后，你就得找出并消除这些错误。这一过程通常称为调试（debugging），这些错误则称为 bug。据说 bug 这个词源于计算机还是占满几个房间的真空管和继电器的年代，由电子装置中的昆虫引发的硬件故障而产生。有多人曾被归功为发现 bug 一词并将其用于软件错误的人，其中最有名的是 COBOL 程序设计语言的发明者 Grace Murray Hopper（§22.2.2.2）。无论 50 多年前真正创造这个词的是谁，bug 一词都生动而无处不在。有意地查找并消除错误的活动称为调试（debugging）。

调试大致是这样进行的：

1. 让程序通过编译。
2. 让程序通过链接。
3. 让程序实现它应该实现的功能。

基本上，我们会一遍又一遍地重复这个序列：几百次、几千次，对真正的大程序还要年复一年地反复进行。每当什么地方不能正常工作，我们就要找出问题的原因并修复它。我认为调试是编程中最

乏味、最浪费时间的一环，因此在设计和编程中会千方百计地减少追找 bug 所花的时间。也有人觉得这种追猎令人兴奋，是编程的精髓——它可能像任何电子游戏一样让人上瘾，让程序员一连几天几夜守在计算机旁（就我个人经验而言，这一点我也作证）。

下面是不该有的调试方式：

```
while (the program doesn't appear to work) {    // 伪代码
    在程序的代码中随意翻看，寻找“看起来有点怪”的地方
    把它改成看起来更好一些
}
```

我们为什么要费心提这一点？这显然是个糟糕的算法，几乎没有成功可言。遗憾的是，当许多人在深夜感到格外迷茫无助、觉得“其他办法”都试过之后，他们实际所做的大抵就是这个描述的一个略带夸张的漫画式写照。

调试中的关键问题是

我怎么才能知道程序确实正确地工作了？

如果你回答不了这个问题，那么等着你的将是一段漫长而乏味的调试时间，而且你的用户十有八九也会跟着受些挫折。我们之所以不断回到这一点，是因为任何有助于回答这个问题的事情都能减少调试，并有助于写出正确且可维护的程序。归根结底，我们希望把程序设计成让 bug 无处藏身。这个要求通常有点过高，但我们的目标是组织程序以尽量减少出错的可能，并尽量提高发现那些确实混进来的错误的可能。

5.9.1 实用调试建议

在写下第一行代码之前就开始考虑调试问题。等你已经写了大量代码，再想简化调试就太迟了。

决定如何报告错误：“使用 error() 并在 main() 中捕获异常”将是本书给你的默认答案。

让程序易读，这样你才有机会发现 bug：

- 给代码写好注释。这并不只是“多加注释”的意思。能用代码说清楚的事，就不要用英语（中文）再赘述。而是在注释中——尽可能清晰、简练地——说明那些代码本身说不清楚的内容：
  - 程序名
  - 程序的用途
  - 这段代码是谁、在什么时候写的
  - 版本号
  - 复杂的代码片段应当做什么
  - 总体设计思想
  - 源代码如何组织
  - 对输入作了哪些假定
  - 代码还缺哪些部分、哪些情况尚未处理
- 使用有意义的名字。
  - 这不等于“只要名字长就行”。
- 使用统一的代码排版。
  - 你的 IDE 会努力帮忙，但它做不了所有事情，责任要由你自负。
  - 本书使用的风格是一个合理的起点。

- 把代码拆成小函数，每个函数表达一个逻辑动作。
  - 尽量避免超过一两页的函数；大多数函数都远比这短。
- 避免复杂的代码序列。
  - 尽量避免嵌套循环、嵌套 `if` 语句、复杂的条件表达式等等。可惜有时你确实需要它们；但请记住，复杂的代码正是 bug 最容易藏身之处。
- 能用库设施（library facilities）时就用，别自己另写。
  - 一个库的设计与测试，多半要比你忙于解决主要问题时顺手写出的替代品更周全、更可靠。

这些目前听起来还很抽象，但随着学习的推进，我们会一个接一个地举例子。

让程序通过编译。显然，编译器是你在这方面最好的帮手。它的错误信息通常很有用——尽管我们总盼着能得到更好的——而且，除非你是真正的专家，否则就假定编译器总是对的；如果你是真正的专家，这本书就不是为你写的。有时你会觉得编译器强制执行的那些规则愚蠢而多余（其实极少如此），觉得事情本可以、也本应该更简单（确实如此，但现实并非如此）。不过，正如俗话所说，“蹩脚的工匠诅咒自己的工具。”好工匠了解自己工具的长处与短处，并据此调整自己的工作。下面是一些常见的编译期错误：

每个字符串字面值（string literal）都以右引号结尾了吗？

```
cout << "Hello, << name << '\n';    // 糟了！
```

每个字符字面值（character literal）都以右引号结尾了吗？

```
cout << "Hello, " << name << '\n;    // 糟了！
```

每个代码块都以右花括号结尾了吗？

```
int f(int a)
{
    if (a>0) { /* 做点什么 */ else { /* 做点别的事 */ }
}    // 糟了！
```

每一对圆括号都配对了吗？

```
if (a<=0        // 糟了！
    x = f(y);
```

编译器通常“较晚”才报告这类错误；它并不知道你本想在 0 之后键入一个右圆括号。

是否每个名字都已声明？

- 是否已包含必需的头文件（眼下用 `#include "std_lib_facilities.h"`）？
- 是否每个名字都在使用前声明了？
- 所有名字的拼写都正确吗？

```
int count;  /* ... */ ++Count;      // 糟了！
char ch;    /* ... */ Cin>>c;      // 错上加错！
```

每个表达式语句都以分号结尾了吗？

```
x = sqrt(y)+2      // 糟了!
z = x+3;
```

在本章的小试身手（drill）里我们还会给出更多此类例子。另外，请记住 §5.2 中给出的错误分类。

程序编译、链接通过之后，接下来通常是最难的部分：弄清程序为什么没有实现它应该实现的功能。你盯着输出，试图推断代码怎么会产生这样的结果。其实，通常你最先面对的是一块空白的屏幕（或窗口），纳闷程序怎么会连一点输出都没有。Windows 控制台模式下程序的一个常见首要问题是：还没来得及看清输出（如果有的话），控制台窗口就消失了。一种解决办法是让程序在 `main()` 的末尾调用我们的 `std_lib_facilities.h` 中的 `keep_window_open()`。这样，程序在退出前会请求一次输入，你就可以先查看已经产生的输出，再输入让它把窗口关掉。

找 bug 时，要沿着代码逐条语句地走：从最后一个你确信正确的点开始。假装你就是那台执行程序的计算机。输出与你的预期相符吗？当然不符，否则你就不必调试了。

通常，当你发现不了问题时，原因在于你“看到”的是你期望看到的东西，而不是你实际写下的东西。考虑下面的代码：

```
for (int i = 0; i<=max; ++j) {      // 糟了!（两处）
    for (int i=0; 0<max; ++i);      // 打印 v 的元素
                cout << "v[" << i << "]==" << v[i] << '\n';
    // ...
}
```

上面这个例子出自一个真实程序，由经验丰富的程序员编写（我们估计是深夜里写的）。

```
int my_fct(int a, double d)
{
    int res = 0;
    cerr << "my_fct(" << a << "," << d << ")\n";
    // ... 行为不当的代码在这里 ...
    cerr << "my_fct() returns " << res << '\n';
    return res;
}
```

- 在怀疑藏有 bug 的代码段中插入检查不变式（invariant，即应当永远成立的条件；见 §9.4.3）的语句。例如：

```
int my_complicated_function(int a, int b, int c)
// 参数均为正数，且 a < b < c
{
    if (!(0<a && a<b && b<c))     // ! 表示“非”，&& 表示“且”
        error("bad arguments for mcf");
    // ...
}
```

如果那样还没有任何效果，就在不怀疑藏有 bug 的代码段里插入不变式检查；如果你找不到 bug，那你几乎肯定是在错误的地方找。

陈述（断言）一个不变式的语句称为断言（assertion），或干脆叫 assert。

有趣的是，行之有效的编程方式有很多。不同的人运用截然不同的技巧，都能取得成功。调试技巧上的许多差异来自人们所做程序类型的不同；另一些差异似乎与思维方式的差异有关。据我们所知，不存在唯一的最佳调试方法。但有一点应当始终记住：凌乱的代码容易藏 bug。尽量保持代码简洁、有逻辑、格式良好，就能缩短你的调试时间。

5.10 前置条件与后置条件

现在，让我们回到如何处理传给函数的错误参数这一问题上来。函数调用基本上是思考代码正确性和捕获错误的最佳位置：一个逻辑上独立的计算从这里开始（并随 return 而结束）。看看我们在上面那条建议里做了什么：

```
int my_complicated_function(int a, int b, int c)
// 参数均为正数，且 a < b < c
{
    if (!(0<a && a<b && b<c))     // ! 表示“非”，&& 表示“且”
        error("bad arguments for mcf");
    // ...
}
```

首先，我们（在注释里）陈述了函数对其参数的要求，然后检查这一要求是否成立（如果没成立就抛出异常）。

这是一个很好的基本策略。函数对其参数的要求通常称为前置条件（pre-condition）：要使函数正确执行它的动作，前置条件必须成立。问题只在于：如果前置条件被违反（不成立），该怎么办。

1. 忽略它（希望/假定所有调用者都给出正确的参数）。2. 检查它（并以某种方式报告错误）。

这样看来，参数类型不过是让编译器替我们检查最简单的前置条件并在编译期报告的一种手段。例如：

```
int x = my_complicated_function(1, 2, "horsefeathers");
```

在这里，编译器会查出“第三个参数必须是整数”这一要求（“前置条件”）被违反了。基本上，我们这里所谈的，就是编译器无法检查的要求/前置条件该怎么处理。

我们的建议是：总是在注释中写明前置条件的文档（好让调用者能看到函数期望什么）。没有注释的函数将被假定能处理任何可能的参数值。可是，我们要相信调用者会读那些注释并遵守规则吗？有时不得不如此，但“由被调函数检查参数”这条规则也可以表述为“让函数检查它自己的前置条件”。只要没有不这样做的理由，我们就应该这样做。人们最常给出的不检查前置条件的理由有：

- 不会有人给错参数。
- 检查会拖慢我的代码。
- 检查太复杂了。

第一种理由只有在碰巧知道“是谁”调用该函数时才勉强说得过去——而在真实世界的代码中，这往往非常难知道。

第二种理由成立的频率远比人们以为的低，多半应当作为“过早优化”（premature optimization）的例子而被忽略。如果这些检查最终真成了负担，你随时可以去掉它们；但它们所保证的正确性却不容易再挣回来，你为寻找那些本可被这些检查捕获的 bug 而失去的安眠之夜也换不回来。

第三种理由才是严重的那一个。等你成为有经验的程序员，就不难找出检查某个前置条件所需的工作量远大于执行函数本身的例子。词典查找（lookup in a dictionary）就是一个例子：它的前置条件是词典条目已排序——而验证一本词典确已排序，代价可能远高于一次查找。有时，用代码表达一个前置条件、并确信表达得正确，也是困难的。然而，写函数时要一直考虑能否写一个快速的前置条件检查；除非有充分的理由不做，否则就做。

写前置条件（哪怕只把它写在注释里）对你程序的质量还有一个显著的好处：它迫使你去思考函数对其参数的要求。

如果你不能用几行注释把它说得简单而精确，那你多半还没有认真想过自己正在做的事情。经验表明，写前置条件注释和前置条件检查能帮你避免许多设计错误。我们确实提到过我们讨厌调试；显式陈述前置条件既有助于避免设计错误，也有助于尽早发现使用错误。写下

```
int my_complicated_function(int a, int b, int c)
// 参数均为正数，且 a < b < c
{
    if (!(0<a && a<b && b<c))     // ! 表示“非”，&& 表示“且”
        error("bad arguments for mcf");
    // ...
}
```

比起下面这个看似更简单的版本，能省下你的时间和烦恼：

```
int my_complicated_function(int a, int b, int c)
{
    // ...
}
```

5.10.1 后置条件

陈述前置条件能帮我们改进设计、尽早发现使用错误。这一“显式陈述要求”的思想还能用到别处吗？能——马上就能想到一处：返回值！毕竟，我们通常都得声明函数返回什么；也就是说，只要从函数返回一个值，我们就是在对返回值作出承诺（否则调用者还能凭什么知道可以期望什么？）。再看一下我们的 area 函数（来自 §5.6.1）：

```
// 计算矩形的面积；
// 参数不好时抛出 Bad_area 异常
int area(int length, int width)
{
    if (length<=0 || width <=0) throw Bad_area();
    return length*width;
}
```

它检查了自己的前置条件，但没有在注释中写明（对这么短的函数或许可以接受），并且假定计算本身是正确的（对这么简单的计算，大概也可以接受）。不过，我们可以再明确一些：

```
int area(int length, int width)
// 计算矩形的面积;
// 前置条件: length 和 width 为正数
// 后置条件: 返回一个正值, 即面积
{
    if (length<=0 || width <=0) error("area() pre-condition");
    int a =  length*width;
    if (a<=0) error("area() post-condition");
    return a;
}
```

我们没法检查完整的后置条件，但检查了其中“应当为正”的那部分。

试一试

找出一对值，使这一版 area 的前置条件成立，但后置条件不成立。

前置条件与后置条件为代码提供了基本的合理性检查。因此，它们与不变式（§9.4.3）、正确性（§4.2、§5.2）和测试（第 26 章）这些概念密切相关。

5.11 测试

我们怎么知道何时该停止调试？嗯，我们会一直调试下去，直到找齐所有的 bug——至少我们是努力这样做的。可我们又怎么知道自己找到了最后一个 bug？不知道。“最后一个 bug”是程序员之间的一个笑话：根本没有这样的物种；在一个大程序里，我们永远不会找到“最后一个 bug”。等到我们可能找到的时候，我们早已忙着为某个新用途修改这个程序了。

除了调试，我们还需要一种系统性地寻找错误的方法。这称为测试（testing），我们将在 §7.3、第 10 章的习题以及第 26 章再回到这个话题。基本上，测试就是用一个数量庞大、经过系统挑选的输入集合来执行程序，并将结果与预期进行比较。用一组给定输入进行的一次运行称为一个测试用例（test case）。真实的程序可能需要上百万个测试用例。基本上，系统化的测试无法靠人工一个接一个键入测试来完成，所以我们要等到几章之后，才具备好好处理测试所需的工具。不过与此同时，请记住：必须抱着“发现错误是好事”的态度来进行测试。考虑一下：

心态 1：　我比任何程序都聪明！我要干掉那 @#$%^ 代码！

心态 2：　我把这份代码润色了两周。它完美无缺！

你觉得谁会找出更多的错误？当然，最好是一位略带一点“心态 1”的有经验人士：冷静、沉着、耐心、系统地排查程序可能存在的缺陷。好的测试人员身价堪比同等重量的黄金。

我们努力做到在选择测试用例时保持系统性，并且总是把正确的输入和错误的输入都拿来试一试。§7.3 给出了这方面的第一个例子。

小试身手

下面是 25 个代码片段。每一个都打算插进下面这个“脚手架”（scaffolding）中：

```
#include "std_lib_facilities.h"

int main()
try {
    <<your code here>>
    keep_window_open();
    return 0;
}
catch (exception& e) {
    cerr << "error: " << e.what() << '\n';
    keep_window_open();
    return 1;
}
catch (...) {
    cerr << "Oops: unknown exception!\n";
    keep_window_open();
    return 2;
}
```

每个片段都含有零个或多个错误。你的任务是找出并消除每个程序中的所有错误。当你除去了这些 bug，得到的程序就能编译、运行并写出“Success!”。即使你认为已经看出了某个错误，你仍然需要把（原始的、未经改进的）程序片段录入并测试；因为你对错误所在的猜测可能不对，或片段里的错误比你发现的更多。此外，这个“小试身手”的一个目的，是让你体会一下你的编译器对各类错误的反应。不要把脚手架键入 25 遍——那是复制粘贴或类似“机械”手段的活儿。不要用随手删掉一条语句的方式来解决问题；要通过改动、增加或删除少数几个字符来修复。

```
1.  Cout << "Success!\n";
2.  cout << "Success!\n;
3.  cout << "Success" << !\n"
4.  cout << success << '\n';
5.  string res = 7; vector<int> v(10); v[5] = res; cout << "Success!\n";
6.  vector<int> v(10); v(5) = 7; if (v(5)!=7) cout << "Success!\n";
7.  if (cond) cout << "Success!\n"; else cout << "Fail!\n";
8.  bool c = false; if (c) cout << "Success!\n"; else cout << "Fail!\n";
9.  string s = "ape"; boo c = "fool"<s; if (c) cout << "Success!\n";
10. string s = "ape"; if (s=="fool") cout << "Success!\n";
11. string s = "ape"; if (s=="fool") cout < "Success!\n";
12. string s = "ape"; if (s+"fool") cout < "Success!\n";
13. vector<char> v(5); for (int i=0; 0<v.size(); ++i) ; cout << "Success!\n";
14. vector<char> v(5); for (int i=0; i<=v.size(); ++i) ; cout << "Success!\n";
15. string s = "Success!\n"; for (int i=0; i<6; ++i) cout << s[i];
16. if (true) then cout << "Success!\n"; else cout << "Fail!\n";
17. int x = 2000; char c = x; if (c==2000) cout << "Success!\n";
18. string s = "Success!\n"; for (int i=0; i<10; ++i) cout << s[i];
19. vector v(5); for (int i=0; i<=v.size(); ++i) ; cout << "Success!\n";
20. int i=0; int j = 9; while (i<10) ++j; if (j<i) cout << "Success!\n";
```

```
21. int x = 2; double d = 5/(x-2); if (d==2*x+0.5) cout << "Success!\n";
22. string<char> s = "Success!\n"; for (int i=0; i<=10; ++i) cout << s[i];
23. int i=0; while (i<10) ++j; if (j<i) cout << "Success!\n";
24. int x = 4; double d = 5/(x-2); if (d=2*x+0.5) cout << "Success!\n";
25. cin << "Success!\n";
```

要点回顾

1. 说出四种主要错误类型，并简要定义每一种。
2. 学生程序中，哪几类错误可以忽略？
3. 每一个完成的项目都应当提供哪些保证？
4. 列出我们可以采取的三种途径，以消除程序错误、产出可接受的软件。
5. 为什么我们讨厌调试？
6. 什么是语法错误？给出五个例子。
7. 什么是类型错误？给出五个例子。
8. 什么是链接错误（linker error）？给出三个例子。
9. 什么是逻辑错误？给出三个例子。
10. 列出正文中讨论过的四种潜在程序错误来源。
11. 你如何知道一个结果是否合理？回答这类问题你有哪些技术可用？
12. 比较并对比：由函数的调用者处理运行期错误，与由被调函数处理运行期错误。
13. 为什么使用异常（exception）比返回一个“错误值”更好？
14. 如何检测一次输入操作是否成功了？
15. 描述异常被抛出和捕获的过程。
16. 对于名为 v 的向量（vector）来说，为什么 `v[v.size()]` 是范围错误（range error）？调用它会产生什么结果？
17. 定义前置条件与后置条件；举一个（不是本章的 `area()` 函数）例子，最好是一个需要循环的计算。
18. 什么时候你会不检查前置条件？
19. 什么时候你会不检查后置条件？
20. 调试一个程序有哪些步骤？
21. 为什么写注释对调试有帮助？
22. 测试与调试有何区别？

术语

参数错误　异常　要求
断言（assertion）　不变式　运行期错误
`catch`　链接期错误　语法错误
编译期错误　逻辑错误　测试
容器　后置条件　**`throw`**
调试　前置条件　类型错误
error　范围错误

习题

1. 如果你还没有做，请完成本章的各个“试一试”练习。

2. 下面的程序读入一个摄氏温度（Celsius）值，将其转换成开尔文（Kelvin）温度。这段代码里有许多错误。找出这些错误，把它们列出来，并改正代码。

```
double ctok(double c)    // 将摄氏温度转换为开尔文温度
{
    int k = c + 273.15;
    return int
}


int main()
{
    double c = 0;          // 声明输入变量
    cin >> d;                   // 把温度读进输入变量
    double k = ctok("c");      // 转换温度
    Cout << k << '/n' ;          // 打印出温度
}
```

3. 绝对零度是所能达到的最低温度；它是 −273.15°C，即 0 K。上面的程序即便改正了，当给出的温度低于这一值时仍会产生错误的结果。请在主程序中加一个检查：当给出的温度低于 −273.15°C 时就报错误。
4. 再做一遍习题 3，但这回在 `ctok()` 内部处理这个错误。
5. 对该程序加以扩充，使它也能把开尔文温度转换为摄氏温度。
6. 写一个程序，把摄氏温度转换成华氏温度，以及把华氏温度转换成摄氏温度（公式见 §4.3.3）。用估算（§5.8）来检验你的结果是否合理。
7. 二次方程（quadratic equation）的形式如下

$$a\,x^2 + b\,x + c = 0$$

要解这类方程，使用的就是二次求根公式（quadratic formula）：

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

不过这里有个问题：如果 b^2-4ac 小于零，公式就会失效。

写一个能对二次方程计算出 x 的程序。创建一个函数，给定 a、b、c，它打印出该二次方程的根。当程序检测到方程没有实数根时，让它打印出一条消息。你怎么知道自己的结果是合理的？你能检验它们是否正确吗？

习题 8：写一个程序，读入并存储一系列整数，然后计算前 N 个整数的和。先询问 N，再把各个值读入一个向量（vector），接着计算前 N 个值的和。例如：

```
“请输入你要求和的数值个数:”
3
“请输入一些整数 (按 '|' 结束输入):”
12 23 13 24 15 |
“前 3 个数 (12 23 13) 的和是 48。”
```

9. 处理所有输入。例如，要确保当用户要求求和的数值个数多于向量中的数值个数时，给出错误信息。
10. 改写习题 8 的程序：当结果无法用 `int` 表示时，写出一条错误信息。
11. 改写习题 8 的程序：用 `double` 而不是 `int`。另外，构造一个 `vector`，其中存放相邻数值之间那 $N-1$ 个差值，并写出这个差值向量。
12. 写一个程序，写出斐波那契（Fibonacci）级数的前若干个值，也就是从 1 1 2 3 5 8 13 21 34 开始的级数。级数中下一个数是前两个数之和。找出能放进一个 `int` 的最大斐波那契数。
13. 实现一个（出于某种不明原因而）叫作“公牛与母牛”（Bulls and Cows）的小猜数游戏。程序持有一个由 0 到 9 范围内四个互不相同的整数组成的向量（如 1234，但不是 1122），用户的任务是通过反复猜测找出这些数字。设要猜的数是 1234、用户猜的是 1359；回答应当是“1 头公牛和 1 头母牛”，因为用户猜对了一个数字（1）且位置正确（一头公牛），还猜对了一个数字（3）但位置不对（一头母牛）。猜数一直进行下去，直到用户得到 4 头公牛，即四个数字都对且顺序正确为止。
14. 这个程序有点无趣，因为答案被硬编码进了程序。做一个版本，让用户可以反复玩（不必停下并重启程序），而且每一局都有一组新的四个数字。可以调用 `std_lib_facilities.h` 中的随机数生成器 `randint(10)` 四次来得到四个随机数字。你会注意到，如果反复运行该程序，每次启动它都会选出同样的四个数字的序列。为避免这一点，可请用户输入一个数（任意数），并在调用 `randint(10)` 之前调用 `srand(n)`，其中 n 是用户输入的那个数。这样的一个 n 称为种子（seed），不同的种子会给出不同的随机数序列。
15. 从标准输入读入（星期几，值）对。例如：`Tuesday 23 Friday 56 Tuesday -3 Thursday 99`

把一周之中属于每一天的所有值都收集到各自的 `vector<int>` 中。写出七个“星期几”向量中的值。打印出每个向量中各值的和。忽略非法的星期几（如 Funday），但接受 Mon、monday 等常见的同义写法。写出被拒绝的值的个数。

后记

你觉得我们是不是过分强调了错误？作为新手程序员，我们大概也会这么想。自然会想到的反应是“事情不可能有那么糟吧！”嗯，事情就是有那么糟。世界上许多最聪明的头脑都曾为写出正确程序的困难而震惊并束手无策。凭我们的经验，最可能低估 bug 问题的人是优秀的数学家；但我们每个人都会很快就超出自己天生“第一次就写对程序”的能力上限。好，你已经受到警告了！所幸在大约 50 年之后，我们在组织代码以尽量减少问题方面已经积累了大量经验，也掌握了一些用于找出 bug 的技术——尽管我们尽了最大努力，初写程序时仍不可避免地会把 bug 留在里面。本章介绍的技术和例子是一个很好的起点。

6 编写程序

“程序设计就是理解。”

——Kristen Nygaard

编写程序的过程，就是把你“想做什么”以及“想如何表达它”的想法逐步打磨清晰的过程。在本章和下一章中，我们将从一个最初模糊的想法出发，经历分析、设计、实现、测试、重新设计和重新实现等阶段来开发一个程序。我们的目的是让你体会一下，当你开发一段代码时进行的是怎样的思考。在这一过程中，我们还将讨论程序组织、用户自定义类型（user-defined types）和输入处理。

6.1 问题

编写程序要从一个问题（problem）出发；也就是说，你有一个希望由程序帮助解决的问题。理解这个问题是好程序的关键。毕竟，一个解决了错误问题的程序，无论多么优雅，多半对你没什么用处。的确有过这样的幸运偶然：某个程序碰巧对它从未打算的用途派上了用场；但我们不能指望这种罕见的好运。我们想要的是一个能简单而干脆地解决我们决定要解决的问题的程序。

到了这一步，考察一个什么样的程序比较合适呢？一个能

- 展示设计与编程技术；
- 给我们机会去探索程序员必须做出的各种决策，以及这类决策背后需要权衡的考量；
- 不需要太多新的程序语言构造；
- 复杂到足以促使我们对它的设计进行思考；
- 允许解法存在很多变体；
- 解决一个容易理解的问题；
- 解决一个值得解决的问题；
- 其解足够小，可以完整呈现并被完全理解。

我们选择的是“让计算机对我们输入的表达式做普通的算术运算”；也就是说，我们要写一个简单的计算器（calculator）。这类程序显然有用：每台桌面计算机都自带这样一个程序，你甚至能买到专门只运行这类程序的计算机——袖珍计算器。

6.1 问题
6.2 思考问题
6.2.1 开发阶段
6.2.2 策略
6.3 回到计算器！
6.3.1 首次尝试
6.3.2 记号
6.3.3 实现记号
6.3.4 使用记号
6.3.5 重回白板
6.4 文法
6.4.1 岔路：英语文法
6.4.2 编写文法
6.5 把文法变成代码
6.5.1 实现文法规则
6.5.2 表达式
6.5.2.1 表达式：第一次尝试
6.5.2.2 表达式：第二次尝试
6.5.2.3 表达式：第三次尝试
6.5.3 项
6.5.4 初等表达式
6.6 试运行第一版
6.7 试运行第二版
6.8 记号流
6.8.1 实现 Token_stream
6.8.2 读取记号
6.8.3 读取数
6.9 程序结构

例如，如果你输入

```
2+3.1*4
```

程序应当回应：

```
14.4
```

遗憾的是，这样的计算器程序并不能给我们计算机上原本没有的新东西；不过对第一个程序就提这种要求，未免太苛刻了。

6.2 思考问题

那么我们从哪里入手？基本上就是先想一想这个问题以及如何解决它。首先思考程序应该做什么、你希望怎样与它交互；稍后再考虑程序要如何编写才能做到这一点。试着把解决方案的初步想法简要写下来，看看这个最初的想法有什么毛病。也可以和朋友讨论这个问题及解决办法——向朋友解释一件事是发现想法毛病的好得惊人的办法，甚至比写下来更好；因为纸（或计算机）不会跟你顶嘴、挑战你的种种假设。理想情况下，设计并不是一种孤独的活动。

遗憾的是，并不存在一种对所有人和所有问题都有效的通用解题策略。市面上有整类书籍声称能帮你在解决问题上更进一步，文献中还另有专门的庞大分支讨论程序设计。我们不深入那上面去。作为替代，我们将用一页的篇幅，提出针对个人所能遇到的较小型问题的一般性策略建议，然后马上把这些建议拿去对付我们小小的计算器问题。

阅读我们关于计算器程序的讨论时，建议你抱着一种比平常更怀疑的态度。为了真实起见，我们让程序历经一系列版本逐步演化，把通向每个版本的推理也一并呈现出来。显然，这些推理有许多必然是不完整甚至错误的，否则这一章提前就结束了。我们会沿途给出设计者与程序员时时面对的种种考量和推理的例子。要到下一章的末尾，我们才得到一个我们自己满意的程序版本。

请记住：在本章和下一章中，我们走到程序最终版本的方式——这段穿过部分解法、想法和错误的旅程——与那个最终版本同等重要，而且比我们沿途遇到的语言技术细节更重要（那些以后再谈）。

6.2.1 开发阶段

这里是关于程序开发的一点术语。在研究一个问题的过程中，你会反复经历以下这些阶段：

- 分析：弄清楚应该做什么，并写下你对它（当前所理解）的描述。这样的描述称为一组需求（requirements）或规格说明（specification）。这些需求如何产生并写下来，我们没有篇幅细讲；那超出了本书的范围，但随着问题规模的增大，它会变得越来越重要。
- 设计：为系统创建一个总体结构，决定实现应由哪些部分组成，以及这些部分之间应当如何沟通。作为设计的一部分，还要考虑哪些工具（如库）能帮你组织程序。
- 实现：编写代码，调试它，并测试它确实做到了它应该做的事。

6.2.2 策略

下面这些建议——如被审慎而富有想象力地运用——能帮助许多编程项目：

- 要解决的 问题是什么？第一件要做的事是力求明确你要完成的目标。这一步通常意味着构造一个问题陈述；或者——如果别人已经给了你一份陈述——设法弄清它真正的含义。此时你应当站在用户的视角（而不是程序员/实现者的视角）；也就是说，你应当就“程序该做什么”提问，而不是问“它打算怎么做”。问问自己：“这个程序能为我做什么？”“我希望怎样与这个程序交互？”记住，我们大多数人都有作为计算机用户的丰富经验可以借鉴。
- 问题陈述清楚吗？对真实问题而言，从来都不清楚。即便是学生练习，也可能难以做到足够精密、具体。所以我们要设法澄清它。要是我们把错误的问题解掉了，那就糟了。另一个陷阱是要得太多。当我们试图弄清自己想什么时候，很容易变得太贪婪/有野心。要求少一些几乎总是更好：这使程序更容易描述、更容易理解、更容易使用，而（有望）也更容易实现。一旦它跑起来，我们总可以基于已有经验打造一个更炫的“2.0 版”。
- 以可用的时间、技能和工具而论，这个问题看来可解吗？启动一个你根本不可能完成的项目毫无意义。如果没有足够时间来实现（包括测试）一个满足全部需求的程序，那么通常明智的做法是先别开始：去争取更多资源（尤其是更多时间），或者（最好）修改需求以简化你的任务。
- 试着把程序分解成若干易于管理的部分。哪怕解决真实问题的最小程序，也大到足以再作细分。
- 你知道有哪些工具、库等资源可能帮上忙吗？答案几乎总是“知道”。哪怕处在学编程的最早期，你也已经掌握 C++ 标准库的一部分。之后你会熟悉标准库的很大部分，并懂得如何找到更多；你会有图形和 GUI 库、矩阵库等。有了一点经验之后，简单的网络搜索就能找到数以千计的库。记住：为真实使用而构建软件时，重新发明轮子没什么价值；而学编程时则是另一回事——为了弄清轮子是怎么造出来的而重新发明一次，往往是好主意。把好库省下来的时间可以花在问题的其他部分上，或者用来休息。你怎么知道某个库对你的任务合适、质量又足够好？这个问题很难。部分解决办法是请教同事、在讨论组里发问，并在决定采用一个库之前先用它试做些小例子。
- 寻找解法中可以单独描述的组成部分（它们潜在地可被程序中多处、甚至其他程序使用）。找到这样的部分需要经验，所以我们在本书中提供了大量例子。我们已经用过 `vector`、`string` 和 iostream（`cin` 和 `cout`）。本章给出第一批完整例子：以用户自定义类型（`Token` 和 `Token_stream`）形式提供的程序部件的设计、实现与使用。第 8 章和第 13–15 章会给出更多例子及其设计理由。现在打个比方：假如我们设计一辆汽车，会先确定诸如车轮、发动机、座椅、门把手等部件，在组装整车之前分别对它们展开工作。一辆现代汽车有几万个这样的部件。真实世界的程序在这方面并无不同——当然，它的部件是代码。我们不会试图用铁、塑料、木材这样的原材料直接造出

一辆汽车；同样，我们也不会试图（只）用语言提供的表达式、语句和类型直接构建一个大程序。设计与实现这样的部件是本书乃至整个软件开发的一大主题；参见用户自定义类型（第 9 章）、类层次（第 14 章）和泛型类型（第 20 章）的讨论。

- 先构建一个解决问题的关键部分、小型而受限的程序版本。起步时我们很少真正熟悉问题。我们常常以为自己熟悉（难道我们不知道计算器程序是什么吗?），其实并不熟悉。只有把对问题的思考（分析）与实验（设计和实现）结合起来，才能获得写好程序所需的扎实理解。因此，我们构建一个小型受限版本：
  - 以便暴露我们在理解、想法和工具上的问题；
  - 看看问题陈述的细节是否需要修改，以使问题变得可控。分析和初始设计之后，发现原本的一切都预料到了的情况极其罕见。编写代码和测试给予我们的反馈应当善加利用。

  有时，这种以实验为目的的小型初始版本被称为 *原型*。如果（这是很可能的）我们的第一版不能运行，或者丑陋笨拙得让我们不想再用它，就把它扔掉，基于已有经验再造一个受限版本。如此重复，直到找到一个满意的版本。不要带着一团糟继续前进；糟粕只会随时间滋长。
- 再构建一个完整规模的解法，理想情况下尽量复用初始版本的部件。理想做法是让程序从可工作的部件“生长”出来，而不是一次性写出全部代码。另一种选择则是指望奇迹：一个未经测试的想法会碰巧成立并做我们想做的事。

6.3 回到计算器！

我们希望如何与计算器交互？这很简单：我们知道怎么用 `cin` 和 `cout`，而图形用户界面（GUI）要到第 16 章才讲，所以我们只用键盘和控制台窗口。从键盘输入表达式，对它们求值，并把结果值写到屏幕上。例如：

```
Expression: 2+2
Result: 4
Expression: 2+2*3
Result: 8
Expression: 2+3-25/5
Result: 0
```

表达式（例如 `2+2` 和 `2+2*3`）由用户输入；其余内容都由程序产生。我们选择输出 `Expression:` 来提示用户输入。我们本可以选择 `Please enter an expression followed by a newline`（请输入一个表达式和一个换行符），但那样显得啰嗦而无意义。另一方面，像 `>` 这样讨人喜欢的短促提示又显得太晦涩。尽早勾画出这类使用示例很重要：它们对“程序至少要做什么”给出了非常实用的定义。在讨论设计与分析时，这类使用示例被称为用例（use case）。

第一次面对计算器问题时，大多数人会为程序的主逻辑想出类似这样的最初方案：

```
read_a_line
calculate       // 完成计算工作
write_result
```

这种“涂鸦”显然并非代码；它叫做伪代码（pseudo code）。我们在设计的早期阶段、尚未确定所用记法究竟意味着什么时，倾向使用它。例如，`calculate` 是一个函数调用吗？如果是，它的实参是什么？回答这类问题为时尚早。

6.3.1 首次尝试

到了这一步，我们并未真正准备好编写计算器程序；我们只是思考得还不够深，但思考是件苦差事，而——像大多数程序员一样——我们急于写点代码。那就碰碰运气，写个简单的计算器，看看它能将我们引向何方。最初的想法差不多是这样：

```
#include "std_lib_facilities.h"

int main()
{
    cout << "Please enter expression (we can handle + and -): ";
    int lval = 0;
    int rval;
    char op;
    int res;
    cin>>lval>>op>>rval;    // 读入形如 1 + 3 的内容

    if (op=='+')
        res = lval + rval;      // 加法
    else if (op=='-')
        res = lval - rval;      // 减法

    cout << "Result: " << res << '\n';
    keep_window_open();
    return 0;
}
```

也就是说，读入由一个运算符隔开的一对值（如 2+2），计算出结果（此处为 4），再打印结果值。我们把变量名选为 lval（left-hand value，左手值）和 rval（right-hand value，右手值）。

它（多少）能工作！就算这个程序尚不完备又如何？让某个东西跑起来的感觉真棒！也许编程和计算机科学确实比传言中容易。嗯，也许吧，但别被一次小小的成功冲昏头脑。让我们接着：

1. 稍微清理一下代码
2. 加入乘法和除法（例如 2*3）
3. 加入处理多于一个操作数的能力（例如 1+2+3）

特别是，我们知道应当始终检查输入是否合理（一时匆忙，我们“忘了”）；而且，把一个值与多个常量逐一比较这件事，用 switch 语句而不是 if 语句来做最好。

对 1+2+3+4 这类运算的“串联”，我们采用随读随加的办法来处理；也就是说，我们从 1 出发，看到 +2 就把 2 加到 1 上（得到中间结果 3），看到 +3 就把这个 3 加到中间结果（3）上，如此继续。在几次无功而返并改正了几个语法和逻辑错误之后，我们得到：

```
#include "std_lib_facilities.h"

int main()
{
    cout << "Please enter expression (we can handle +, -, *, and /)\n";
    cout << "add an x to end expression (e.g., 1+2*3x): ";
    int lval = 0;
    int rval;
    cin>>lval;                          // 读入最左操作数
    if (!cin) error("no first operand");
    for (char op; cin>>op; ) {          // 读入运算符与右操作数,
                                        // 反复进行
        if (op!='x') cin>>rval;
        if (!cin) error("no second operand");
        switch(op) {
        case '+':
            lval += rval;               // 加: lval = lval + rval
            break;
        case '-':
            lval -= rval;               // 减: lval = lval - rval
            break;
        case '*':
            lval *= rval;               // 乘: lval = lval * rval
            break;
        case '/':
            lval /= rval;               // 除: lval = lval / rval
            break;
        default:                        // 不是别的运算符: 输出结果
            cout << "Result: " << lval << '\n';
            keep_window_open();
            return 0;
        }
    }
    error("bad expression");
}
```

这还不算糟。但我们试一下 1+2*3，会发现结果是 9 而不是我们算术老师告诉我们应该得到的 7。类似地，1-2*3 给出 -3，而不是我们期望的 -5。我们的运算次序不对：1+2*3 被算成了 (1+2)*3 而不是惯常的 1+(2*3)；类似地，1-2*3 被算成了 (1-2)*3 而不是惯常的 1-(2*3)。真扫兴！我们或许可以把“乘法比加法结合得更紧”这条约定当成傻气的老规矩，但几百年的约定不会为了简化我们的编程而消失。

6.3.2 记号

所以，我们必须在输入行上“向前看”，看后面有没有 *（或 /）；如果有，就必须设法把求值次序从简单直观的从左到右次序中调整出来。遗憾的是，在这里贸然前进，我们立刻碰上几个绊脚石：

1. 我们实际上并不要求表达式写在一行里。例如：

```
1
+
2
```

（这样写完全可以。）

2. 我们如何在几行输入的数字、加号、减号和括号之中搜出一个 `*`（或 `/`）？
3. 我们如何记住某个 `*` 出现在哪儿？
4. 我们如何处理并非严格从左到右的求值（例如 `1+2*3`）？

既然我们决定当超级乐观主义者，那就先解决问题 1–3，问题 4 留待以后再操心。

此外，我们还会四处求援。肯定有人知道一种惯例做法：把输入中诸如数和运算符之类的“东西”读进来，并以某种让我们日后取用时很方便的方式加以存储。这个惯例且非常有用的答案就是“把输入词法化（tokenize）”：先把输入字符读进来并组装成记号（tokens），于是如果你输入

```
45+11.5/7
```

程序应当产生一个记号列表来表示

```
45
+
11.5
/
7
```

一个 记号（token）是一个字符序列，它表示我们认为是一个单位的某种东西，比如一个数或一个运算符。C++ 编译器就是这样处理它的源码的。实际上，某种形式的“词法化”是几乎所有文本分析的起点。仿照 C++ 表达式的先例，我们看到需要三种记号：

- 浮点字面值（floating-point-literals）：按 C++ 的定义，例如 `3.14`、`0.274e2` 和 `42`；
- 运算符：例如 `+`、`-`、`*`、`/`、`%`；
- 括号：`(`、`)`。

浮点字面值看上去可能成为问题所在：读入 `12` 显然比读入 `12.3e-3` 容易得多，但计算器又确实倾向于做浮点算术。类似地，我们料到还得接受括号，否则我们的计算器就算不上有用。

我们在程序中如何表示这样的记号？我们可以试图记住每个记号在何处开始（和结束），但那会弄得一团糟（尤其如果我们允许表达式跨行边界的话）。而且，如果把一个数保存为字符组成的字符串，那么之后我们就得弄清它的值是多大；也就是说，如果我们看到 `42` 并把字符 `4` 和 `2` 存到了什么地方，那么之后我们就得弄明白这些字符表示的是数值 `42`（即 `4*10+2`）。显而易见且惯例的解决办法是，把每个记号表示成一个 (*kind*,*value*) 对。*kind* 告诉我们这个记号是数、运算符还是括号。对于数——并且在本例中仅对数——我们用它的数值作为 *value*。

那么，如何在代码中表达 (*kind*,*value*) 对这一思想？我们定义一个类型 `Token` 来表示记号。为什么？想想我们为什么要用类型吧：它们保存我们所需的数据，并为这些数据提供有用的操作。例如，`int` 保存整数并提供加、减、乘、除和取余运算，

而 `string` 保存字符的序列并提供拼接（concatenation）和下标（subscripting）操作。C++ 语言及其标准库给了我们许多类型，例如 `char`、`int`、`double`、`string`、`vector` 和 `ostream`，但没有 `Token`

类型。事实上，存在海量的类型——数千乃至数万个——是我们希望拥有的，而语言及其标准库并不提供。在不受支持而我们钟爱的类型中，有 Matrix（见第 24 章）、Date（见第 9 章）和无限精度整数（试试在网络中搜索“Bignum”）。如果想上一两秒，你就会意识到一门语言不可能提供几万个类型：谁来定义它们？谁来实现它们？你怎么找到它们？而手册又得厚到什么程度？像大多数现代语言一样，C++ 通过让我们在需要时定义自己的类型（用户自定义类型（user-defined types））绕开了这个问题。

6.3.3 实现记号

一个记号在我们的程序中应当长什么样？换句话说，我们希望 Token 类型是什么样的？Token 必须既能表示 + 和 - 这样的运算符，也能表示 42 和 3.14 这样的数值。显而易见的实现是一种能表示记号的“什么种类”（kind），又能为那些有数值的记号保存其数值的东西：



这一思想在 C++ 代码中有许多表示方法。以下是我们发现的最简单且好用的写法:

```
class Token {        // 一种非常简单的用户自定义类型
public:
    char kind;
    double value;
};
```

Token 是一种类型（就像 int 和 char 一样），因此它可以用来定义变量并保存值。它有两个部分（称为成员（members））：kind 和 value。关键字 class 的意思是“用户自定义类型”；它表明我们正在定义一个具有零个或多个成员的类型。第一个成员 kind 是字符（char），这样它恰好能保存 '+' 和 '*' 来表示 + 和 *。我们可以像这样使用它来产生对象：

```
Token t;            // t is a Token
t.kind = '+';       // t 表示一个 +

Token t2;           // t2 是另一个 Token
t2.kind = '8';      // 我们用数字 8 作为一个数的 “kind”
t2.value = 3.14;
```

我们用成员访问记号来访问一个成员。可以把 t.kind 读作“t 的 kind”，把 t2.value 读作“t2 的 value”。可以像复制 int 一样复制 Token：

```
Token tt = t;          // copy initialization
if (tt.kind != t.kind) error("impossible!");
t = t2;                // assignment
cout << t.value;       // 将打印 3.14
```

有了 Token，我们能像下面这样用七个记号来表示表达式 (1.5+4)*11：

'('	'8''	'+'	'8''	')'	'*'	'8''
	1.5		4			11

注意，对于 + 这样的简单记号，我们不需要 *value*，因此不使用它的 *value* 成员。我们需要一个字符来表示“数”，于是挑了 '8'——这仅仅是因为 '8' 显然既不是运算符，也不是标点字符。用 '8' 表示“数”有点难懂，但眼下将就着用。

Token 是一个 C++ 用户自定义类型的例子。用户自定义类型除了数据成员外还可以有成员函数（operations）。定义成员函数可以出于多种原因；这里我们只提供两个成员函数，用来给我们提供一种初始化 Token 的便捷方法：

```
class Token {
public:
    char kind;         // 什么种类的记号
    double value;      // 数：保存其数值
};
```

我们现在可以初始化（“构造”）Token 对象了。例如：

```
Token t1 {'+'};          // 初始化 t1，使 t1.kind = '+'
Token t2 {'8',11.5};     // 初始化 t2，使 t2.kind = '8' 且 t2.value = 11.5
```

关于类对象初始化的 farther 细节，见 §9.4.2 和 §9.7。

6.3.4 使用记号

那么，现在我们也许就可以把计算器写完了吧！不过，稍微提前规划一下也许是值得的。Token 在计算器中会怎样使用呢？我们可以把输入读进一个 Token 的 vector 中：

```
Token get_token();       // 从 cin 读入一个记号的函数
                         // 使用 cin

vector<Token> tok;       // 我们将把记号放在这里

int main()
{
    while (cin) {
        Token t = get_token();
        tok.push_back(t);
    }
    // . . .
}
```

现在我们可以先读入一个表达式，之后再求值。例如，对 11*12，我们得到：

'8''	'*'	'8''
11		12

接下来我们可以查看它，从中找到乘法运算及其操作数。做到这一步之后，求出该乘法就不难了，因为 11 和 12 是作为数值存储的，而不是字符串。

现在来看看更复杂的表达式。给定 1+2*3，tok 将包含五个 Token：

'8''	'+'	'8''	'*'	'8''
1		2		3

此刻我们可以用一个简单的循环来找出乘法运算：

```
for (int i = 0; i<tok.size(); ++i) {
    if (tok[i].kind=='*') {              // 我们找到了一个乘法!
        double d = tok[i-1].value*tok[i+1].value;
                                         // 现在怎么办？
    }
}
```

是的，可接下来怎么办？我们怎样处置乘积 d？如何决定各个子表达式的求值次序？嗯，+ 先于 * 读到，所以我们不能从左到右求值。可以试试从右到左求值！那样对 1+2*3 行得通，但对 1*2+3 就不行了。更糟的是，考虑 1+2*3+4：这一例只能“由内向外”求值，即 1+(2*3)+4。而我们终归要面对的括号又怎样处理呢？看来我们走进了死胡同。我们必须退后一步，暂停编程，好好想一想我们是如何阅读并理解一个输入串、把作为一个算术表达式来求值的。

于是，这番热情洋溢的初次尝试（写一个计算器）失去了后劲。初次尝试走不到头并不少见；而且它担当着帮我们理解问题的重要角色。这一次，它甚至给了我们记号（token）这一有用的概念——而记号又是一对概念的一个例子，我们今后会一再遇到这一概念。不过，我们必须始终确保这类相对欠思考、无规划的“编码”不占去过多时间。在至少做了些许分析（理解问题）和设计（决定解决方案的总体结构）之前，我们只应做极少的编程。

试一试

反过来说，为什么我们就不能为这个问题找到一个简单的解法呢？它看起来并没有那么难。退一步说，试一试也能让我们对问题及其最终解法有更好的鉴赏力。考虑一下你此时此刻能立刻做什么。例如，看输入 12.5+2：我们可以对它做记号化，认定该表达式很简单，并算出答案。这可能有点繁琐，但直截了当，所以也许我们就能沿此方向推进，找到某种足够好的办法！考虑一下，如果在 2+3*4 这一行里同时发现了一个 + 和一个 *，该怎么办。这同样可以用“蛮力”处理。那么，对 1+2*3/4%5+(6-7*(8)) 这样的复杂表达式怎么办？还有 2+*3 和 2&3 这样的错误又该怎么处理？把这些问题考虑一会儿，最好在一张纸上草草勾勒出可能的解法，以及有趣或重要的输入表达式。

6.3.5 重回白板

现在，我们重新审视这个问题，这一次争取不再匆忙抛出又一个半成品解法。我们确实发现了一件事：让程序（计算器）只求值一个表达式太单调乏味了。我们希望

能在程序的一次运行中计算多个表达式；也就是说，我们的伪代码扩展为：

```
while (not_finished) {
    read_a_line
    calculate       // 完成计算工作
    write_result
}
```

显然这是一种复杂化；但一想想人们使用计算器的方式，我们意识到多次计算是非常常见的。可以让用户多次启动我们的程序来做多个计算吗？可以是可以，但程序的启动在许多现代操作系统上（既不合情理地又很）慢，所以我们最好不要指望那个。

审视这段伪代码、我们早期对解法的尝试以及我们的使用示例，浮现出若干问题——其中一些已有初步答案：

1. 如果我们输入 45+5/7，如何在输入中找出 45、+、5、/ 和 7 这些单个的部分？（记号化！）
2. 什么会结束一个输入表达式？自然是换行符！（对“当然”要永远保持怀疑：“当然”不是理由。）
3. 我们如何把 45+5/7 表示为数据以便求值？在做加法之前，我们必须想办法把字符 4 和 5 变成整数值 45（即 4*10+5）。（所以记号化是解决方案的一部分。）
4. 我们如何确保 45+5/7 被求值为 45+(5/7)，而不是 (45+5)/7？
5. 5/7 的值是多少？大约.71，但那不是一个整数。依据计算器的经验，我们知道人们会期望浮点结果。要不要也允许浮点输入？当然要！
6. 我们能不能使用变量？例如，能不能写

```
v=7
m=9
v*m
```

好主意，不过让我们等一等再说。首先要把基本功能做起来。

在这里，最重要的决策也许就是对问题 6 的回答。在 §7.8 中你会看到，如果当初答了“是”，初始项目的规模几乎会翻倍；使初始版本跑起来所需的时间也会不止翻倍。我们猜测，如果你真是新手，所需精力至少还会

再翻两番，而且极可能把项目推进到你耐心极限之外。项目早期最重要的是避免“功能蔓延（feature creep）”。相反，永远要先构建一个简单的版本，只实现那些必要功能。一旦有了能运行的东西，你就可以变得更有雄心。分阶段构建程序远比一夜之间全部写完容易。对问题 6 答“是”还会有另一个坏作用：它会让人很难抵抗此后继续添加“酷功能”的诱惑。加上常用的数学函数如何？加上循环如何？一旦开始添加“酷功能”，就很难停下来。

从程序员的视角看，问题 1、3 和 4 是最麻烦的。它们相互关联：因为一旦找到了 45 或 +，我们对它们做什么呢？也就是说，我们在程序中如何存储它们？显然，记号化是解决方案的一部分，但只是其中一部分。

有经验的程序员会怎么做？当我们面临一个棘手的技术问题时，常常存在一个标准答案。我们知道，人们写计算器程序的历史至少和计算机用键盘接收符号输入的历史一样长。那至少有 50 年了。一定会存在标准答案！在这种情况下，有经验的程序员会去请教同行和（或）文献。莽撞硬闯、指望用一上午的时间胜过 50 年的经验，是愚蠢的。

6.4 文法

对“如何理解表达式”这个问题，有一种标准答案：先把输入字符读进来，装配成记号（这是我们已经发现的）。所以如果你输入

```
45+11.5/7
```

程序应当产生一个记号列表来表示

```
45
+
11.5
/
7
```

记号是一个字符序列，它表示我们认为是一个单位的某种东西，比如一个数或一个运算符。

记号产生之后，程序还必须确保对完整表达式的理解是正确的。例如，我们知道 `45+11.5/7` 的意思是 `45+(11.5/7)` 而不是 `(45+11.5)/7`；但如何教会程序这条有用的规则（除法“结合得比加法更紧”）？标准答案是：我们写一个文法（grammar）来定义输入的语法，然后编写一个实现该文法规则的程序。例如：

```
// a simple expression grammar:
Expression:
        Term
        Expression "+" Term      // 加法
        Expression "-" Term      // 减法
Term:
        Primary
        Term "*" Primary         // 乘法
        Term "/" Primary         // 除法
        Term "%" Primary         // 余数（取模）
Primary:
        Number
        "(" Expression ")"       // 分组
Number:
        floating-point-literal
```

这是一组简单规则。最后一条规则读作“一个 Number 是一个浮点字面值（floating-point-literal）”。倒数第二条规则说的是：“一个 Primary 或者是一个 Number，或者是一个 (后跟着一个 Expression、再跟着一个)。”Expression 与 Term 的规则类似，每一条都由某条后继规则来定义。

如 §6.3.2 所示，我们的记号——从 C++ 定义中借来的那些——有：

- 浮点字面值（floating-point-literal）（按 C++ 定义，例如 `3.14`、`0.274e2` 或 `42`）；
- `+`、`-`、`*`、`/`、`%`（运算符）；
- `(`、`)`（括号）。

从我们最初试探的伪代码走到这一步——使用记号和文法的玩法——实际上是一次巨大的概念跃迁。这是我们所盼望却很少能凭一己之力完成的跃迁。经验、文献和导师的作用正在于此。

乍一看，文法多半显得完全是胡言乱语。技术记号往往如此。不过请记住：它是一种通用且优美的记号（这一点你终将体会到），描述的却是你自初中乃至更早就会做的事情。你毫无问题能算出 `1-2*3` 和 `1+2-3` 和 `3*2+4/2`。它似乎已经深植于你的大脑。可是你能解释你是怎么算的吗？你能把它解释到让一个从未见过传统算术的人理解的程度吗？对每一种运算符和操作数的组合也都能做到吗？

要想把解释陈述得足够细致、足够精确，让计算机都能明白，我们需要一门记号——而文法正是为此目的最有效、最常规的工具。

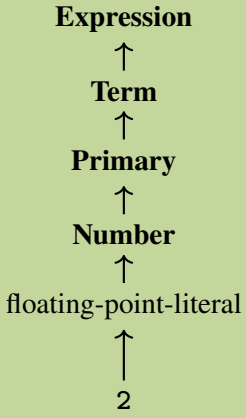
怎么读一个文法呢？基本上，给定某些输入后，你从“顶规则”Expression 出发，随着记号的读入，在规则之中寻找匹配。按照一个文法来读记号流称为解析（parsing），做这件事的程序通常被称为解析器（parser）或语法分析器（syntax analyzer）。我们的解析器同我们敲入它们和读出它们的方式一样，从左到右读记号。我们试一个非常简单的例子：2 是不是一个表达式？

1. 一个 Expression 必须是一个 Term，或以 Term 结尾。那个 Term 必须是一个 Primary，或以 Primary 结尾。那个 Primary 必须以 `(` 开头，或是一个 Number。显然，2 不是一个 `(`，它是一个浮点字面值（floating-point-literal），即一个 Number，也即一个 Primary。
2. 那个 Primary（即 Number 2）不在 `/`、`*` 或 `%` 之前，所以它是一个完整的 Term（而非一个 `/`、`*` 或 `%` 表达式的结尾部分）。
3. 那个 Term（即 Primary 2）不在 `+` 或 `-` 之前，所以它是一个完整的 Expression（而非一个 `+` 或 `-` 表达式的结尾部分）。

所以是的：按照我们的文法，2 是一个表达式。我们可以像下面这样图示这一在文法中行进的路径：

解析数 2

```
Expression:
    Term
    Expression "+" Term
    Expression "-" Term
Term:
    Primary
    Term "*" Primary
    Term "/" Primary
    Term "%" Primary
Primary:
    Number
    "(" Expression ")"
Number:
    floating-point-literal
```

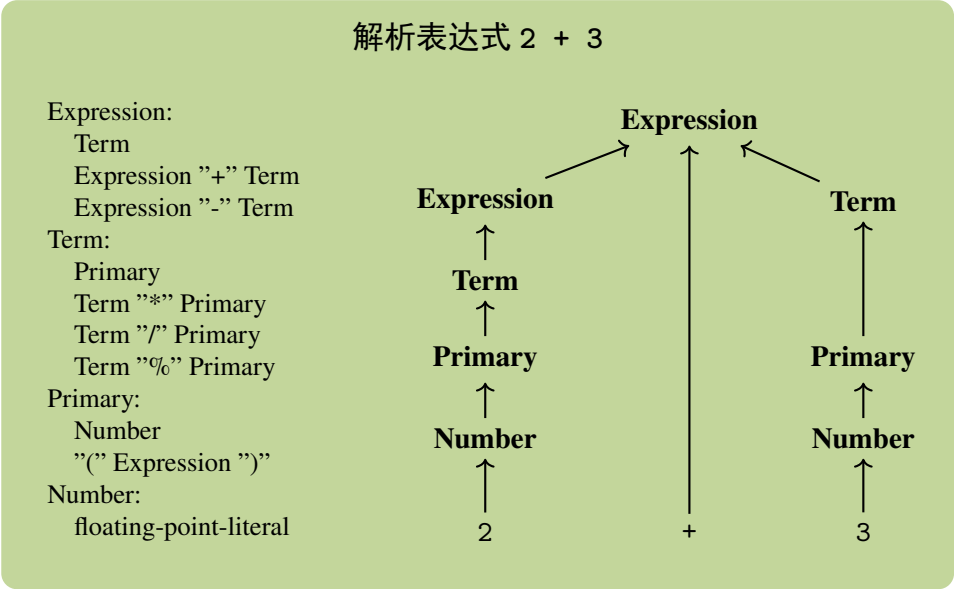


此图表示的就是我们穿越定义走过的路径。重返路径，我们可以说：2 之所以是一个 Expression，是因为 2 是一个浮点字面值（floating-point-literal），它是一个 Number，Number 是一个 Primary，Primary 是一个 Term，Term 是一个 Expression。

我们来试一个复杂一点的：`2+3` 是不是 Expression？自然，很多推理与 2 的情形相同：

1. 一个 Expression 必须是一个 Term 或以 Term 结尾，那个 Term 必须是一个 Primary 或以 Primary 结尾，而 Primary 必须以 (开头或是一个 Number。显然 2 不是一个 (，它是一个浮点字面值（floating-point-literal），即一个 Number，也即一个 Primary。
2. 那个 Primary（即 Number 2）不在 /、* 或 % 之前，所以它是一个完整的 Term（而非一个 /、* 或 % 表达式的结尾部分）。
3. 那个 Term（即 Primary 2）后面跟着一个 +，所以它是一个 Expression 的头一部分，我们必须在 + 之后寻找 Term。正如我们发现 2 是一个 Term，我们也发现 3 是一个 Term。既然 3 的后面不再有 + 或 -，它就是一个完整的 Term（而非一个 + 或 - Expression 的开头部分）。因此，2+3 符合 Expression+Term 规则，是一个 Expression。

同样，我们可以把这一推理用图示出来（为简单起见，仍然省略从浮点字面值到 Number 的规则）：



此图表示的又是我们穿越定义走过的路径。重返路径，我们可以说：2+3 之所以是一个 Expression，是因为 2 是一个 term，它是一个 Expression；3 是一个 Term；而一个 Expression 后跟一个 + 再跟一个 Term，这本身就是一个 Expression。

我们关心文法的真正理由是：它们能解决我们如何正确解析既带 + 又带 * 的表达式这一问题，所以我们来试 45+11.5*7 吧。然而，像上面那样按部就班“假扮计算机”按规则细跑是枯燥乏味的，所以我们略去对 2 和 2+3 已经历过的若干中间步骤。显然，45、11.5 和 7 都是浮点字面值（floating-point-literal），也即 Number，也即 Primary，所以 Primary 之下的所有规则都可忽略。于是我们得到：

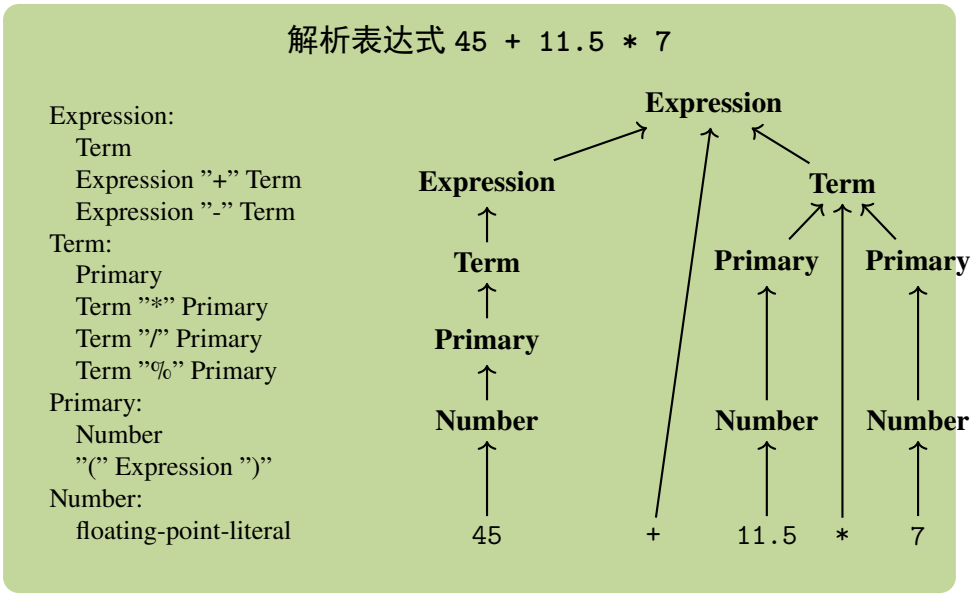
1. 45 是一个 Expression，其后跟着 +，所以我们寻找一个 Term 来完成 Expression+Term 规则。
2. 11.5 是一个 Term，其后跟着 *，所以我们寻找一个 Primary 来完成 Term*Primary 规则。
3. 7 是 Primary，所以按 Term*Primary 规则 11.5*7 是一个 Term。

现在可以看到，按 Expression+Term 规则，45+11.5*7 是一个 Expression。特别地，它先算乘法 11.5*7，再做 45+11.5*7 的加法，就仿佛我们写的已经是 45+(11.5*7) 一样。

同样，我们可以把这一推理图示出来（再次省略从浮点字面值到 Number 的规则以简化）：

同样，此图表示的就是我们穿越定义走过的路径。注意 Term*Primary 规则是如何保证 11.5 乘以 7 而不是加到 45 之上的。

这一套逻辑乍看起来艰涩难跟，但实际上许多人都读文法，而且简单文法并不难懂。然而，我们原本并不是在教你理解 2+2 或 45+11.5*7；显然你早就会了。我们是想找一种办法，让计算机“理解”



45+11.5*7 以及你可能给它求值的所有其它复杂表达式。实际上，复杂的文法并不适合人类阅读，可计算机擅长于此：它们驾轻就熟地又快又对地遵循这类文法规则。遵循精确的规则恰恰是计算机最拿手的事。

6.4.1 岔路：英语文法

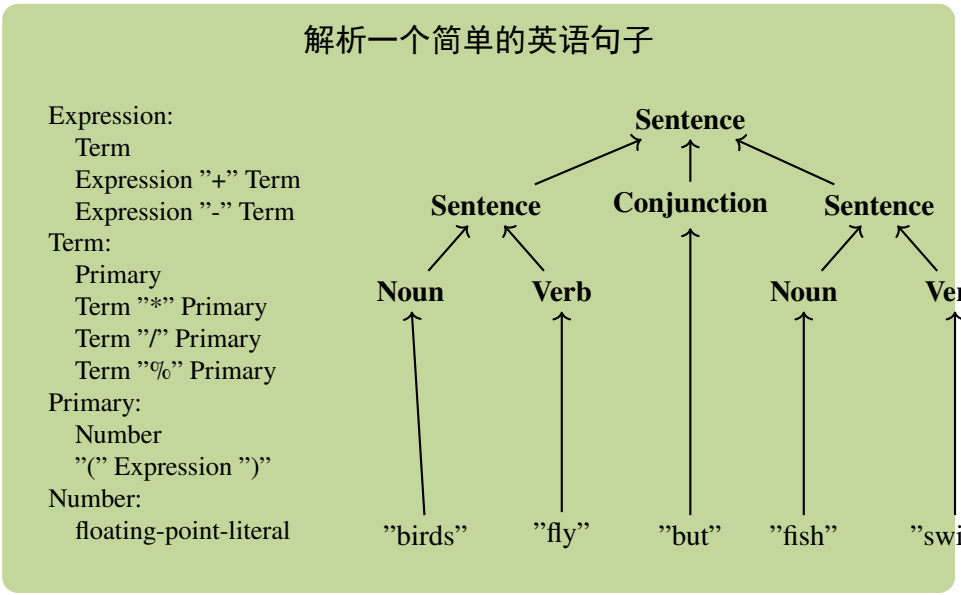
如果你以前从未与文法打过交道，我们料想你此刻正在晕头转向。不过，即使你以前见过文法，也可能仍然晕头转向；那不妨看看下面这个针对英语很小的一个子集的文法：

```
Sentence:
        Noun Verb                       // 例如: C++ rules
        Sentence Conjunction Sentence // 例如: Birds fly but fish swim
Conjunction:
        "and"
        "or"
        "but"
Noun:
        "birds"
        "fish"
        "C++"
Verb:
        "rules"
        "fly"
        "swim"
```

句子由词类的各部分（如名词、动词和连词）构成。一个句子可以按照这些规则来解析，以判定哪些词是名词、动词等。这个简单文法也包括语义上无意义的句子，例如“C++ fly and birds rules”（C++飞而鸟群懂行）；但修好这一点就属于另一码事了，得多高深的书里去谈。

许多人在中学或外语课上（例如英语课）都被教过或展示过这类规则。这些文法规则是非常基本的。事实上，确有神经科学的严肃论证支持这些规则是长在我们大脑里的！

现在来看看我们为表达式用过的那种解析树（parsing tree），不过这里将一例拿来开拓思路，用在简单英语上：



这并没有那么复杂。如果你在 §6.4 上栽了跟头，请回去把开头重读一遍；多读一遍也许就能更清楚！

6.4.2 编写文法

那些表达式文法规则我们是怎么挑选的？老实说，“凭经验”。我们的做法就是人们通常写表达式文法的那一路数。不过，写一个简单的文法相当直截了当：我们需要知道如何

1. 区分规则与记号；
2. 把一条规则安到另一条规则之后（排序（sequencing））；
3. 表达可选择的模式（备择（alternation））；
4. 表达重复出现的模式（重复（repetition））；
5. 识别出该从哪条文法规则开始。

不同的教科书和不同的解析器系统采用不同的记号约定和不同的术语。例如，有的把记号称为终结符（terminals），把规则称为非终结符（non-terminals）或产生式（productions）。我们的做法很朴素：把记号放进（双）引号中，从第一条规则开始。各可选项另起一行。例如：

```
List:
        "{" Sequence "}"
Sequence:
        Element
        Element " ," Sequence
Element:
        "A"
        "B"
```

所以，一个 Sequence 或者是一个 Element，或者是一个 Element 后跟着另一个 Sequence，中间用逗号隔开。一个 Element 或者是字母 `A`，或者是字母 `B`。一个 List 就是用“花括号”括起来的一个 Sequence。我们可以生成下面这些 List（怎么生成的？）：

```
{ A }
{ B }
{ A,B }
{A,A,A,A,B }
```

然而，下面这些就不是 List 了（为什么不是？）：

```
{ }
A
{ A,A,A,A,B
{A,A,C,A,B }
{ A B C }
{A,A,A,A,B, }
```

这条 Sequence 规则既不是你在幼儿园学过的，也不是天生刻在你脑子里的，但它也没尖端到火箭科学的程度。关于我们如何使用文法来表达语法思想，请看 §7.4 和 §7.8.1 的例子。

6.5 把文法变成代码

让计算机遵循一个文法的办法有很多。我们将采用最简单的一种：为每条文法规则简单地写一个函数，并用类型 `Token` 来表示记号。实现了一个文法的程序通常称为解析器（parser）。

6.5.1 实现文法规则

要实现我们的计算器，需要四个函数：一个读记号，再加上文法中每条规则各一个：

```
get_token()       // 读入字符并组装记号
                  // 使用 cin
expression()      // 处理 + 和 -
                  // 调用 term() 和 get_token()
term()            // 处理 *、/ 和 %
                  // 调用 primary() 和 get_token()
primary()         // 处理数和括号
                  // 调用 expression() 和 get_token()
```

注意：每个函数只和一个表达式的特定部分打交道，其余的一切都留给其它函数；这就从根本上简化了每个函数。这很像一群人分工解决问题：让每人处理其专长范围里的问题，把所有其它问题转交给同事。

这些函数实际上应该做什么呢？每个函数都应当按它正在实现的文法规则调用相应的文法函数，在规则中需要记号的地方调用 `get_token()`。例如，当 `primary()` 想遵循 `(Expression)` 规则时，它必须调用

```
get_token()        // 处理 ( 和 )
expression()       // 处理 Expression
```

这些解析函数应该返回什么？不如返回我们真正想要的答案，如何？例如，对 2+3，让 expression() 返回 5。毕竟信息全齐了。我们就这么干！这样做还让我们免去回答清单中最难的问题之一："我如何把 45+5/7 表示为数据，以便求值？"我们不再是先把 45+5/7 的一种表示存进内存，而是从输入把它读进来的同时就把它求值。这个小小的主意是一次重大突破！它把程序的规模维持在原本的四分之一——假如我们让 expression() 返回某种复杂的数据留待日后求值的话。这一下省掉了大约 80% 的工作。

这套函数中的"另类"是 get_token()：因为它处理的是记号而非表达式，所以不能返回子表达式的值。例如 + 和 (就不是表达式。于是它必须返回一个 Token。我们由此得出结论，需要这组函数：

```
// functions to match the grammar rules:
Token get_token()      // 读入字符并组装记号
double expression()    // 处理 + 和 -
double term()          // 处理 *、/ 和 %
double primary()       // 处理数和括号
```

6.5.2 表达式

我们先写 expression()。那段文法形如：

```
Expression:
        Term
        Expression '+' Term
        Expression '-' Term
```

由于这是我们第一次尝试把一组文法规则变成代码，会历经几次半途而废；新技术的上手大抵如此，而且我们在路上还能学到有用的东西。特别是，新手程序员能从观察相似代码截然不同的行为中学到很多。读代码是一门值得好好培养的技能。

6.5.2.1 表达式：第一次尝试

看 Expression '+' Term 这条规则，我们先试调用 expression()，然后寻找 +（和 -），然后再调用 term()::

```
double expression()
{
    double left = expression();     // 读入并求值一个 Expression
    Token t = get_token();          // 取下一个记号
    switch (t.kind) {               // 看看是什么种类的记号
    case '+':
        return left + term();       // 读入并求值一个 Term,
                                    // 然后做加法
    case '-':
        return left - term();       // 读入并求值一个 Term,
                                    // 然后做减法
    default:
        return left;                // 返回 Expression 的值
    }
}
```

看起来不错。它几乎是那段文法的逐句转抄。确实相当简单：先读一个 Expression，然后看它是否跟着 + 或者 -，若然，再读一个 Term。

遗憾的是，这其实讲不通。我们如何知道 Expression 从哪里结束，以便接着找 + 或 -？记住，我们的程序从左到右读入，不能向前窥探后面是否有个 +。事实上，这个 `expression()` 永远也走不出第一行：`expression()` 开头先调用 `expression()`，而那个 `expression()` 又开头先调用 `expression()`，如此“永远”递推下去。这叫做无限递归（infinite recursion），实际上它会在稍后终止，因为计算机为保存这串“没有尽头”的 `expression()` 调用序列而将内存耗尽。术语 *递归*（*recursion*）用来描述一个函数调用它自身时发生的事情。并非所有递归都是无限的；递归是一种非常有用的编程技术（见 §8.5.8）。

6.5.2.2 表达式：第二次尝试

那可怎么办？每个 Term 都是一个 Expression，但并非每个 Expression 都是 Term；也就是说，我们可以先寻找一个 Term，而只有在发现了 + 或 - 时才寻找一个完整的 Expression。例如：

```
double expression()
{
    double left = term();               // 读入并求值一个 Term
    Token t = get_token();              // 取下一个记号
    switch (t.kind) {                   // 看看是哪种记号
    case '+':
        return left + expression();     // 读入并求值一个 Expression,
                                        // 然后做加法
    case '-':
        return left - expression();     // 读入并求值一个 Expression,
                                        // 然后做减法
    default:
        return left;                    // 返回 Term 的值
    }
}
```

它确实（或多或少）能运行。我们已经在完成的程序里试过：它要么能把你丢给它的所有正确表达式完整解析（而不捕获任何非法的），要么正确地求出大多数表达式的值。例如，1+2 被读作一个 Term（值为 1），随后是 +，随后是一个 Expression（它恰好是一个值为 2 的 Term），并给出答案 3。类似地，1+2+3 给出 6。我们本可以就“什么能行”讲上很长一段，但长话短说：1-2-3 怎么办？这个 expression() 会把 1 读作一个 Term，然后接着把 2-3 读作一个 Expression（由 Term 2 后跟 Expression 3 组成）。然后它将把 2-3 的值从 1 里减除。换句话说，它将求值 1-(2-3)。而 1-(2-3) 的值是 2（正数 2）。然而，我们却被教导过（小学，甚至更早），1-2-3 意思是 (1-2)-3，所以其值为 -4（负四）。

于是我们得到了一个非常漂亮的程序，只是它做的事不对。这很危险。尤其危险的是，它在很多情况下给出的都是正确答案。例如，1+2+3 就能得到正确答案（6），因为 1+(2+3) 等于 (1+2)+3。从编程的角度看，我们根本性的错误在哪儿？发现错误时应当永远自问这个问题。这样我们才有可能避免一次又一次地重犯同样的错误。

从根本上说，我们只是看了看代码就凭猜测行事。这通常是不够的！我们必须弄清自己的代码在做什么，并且能够解释它为什么做的是正确的事。

分析错误往往也是找到正确解法的最佳途径。我们这里的做法是：把 expression() 定义为先查找一个 **Term**，如果这个 **Term** 后面跟有 + 或 -，再去查找一个 **Expression**。这实际上实现的是略有不同的一条文法：

```
Expression:
    Term
    Term '+' Expression     // 加法
    Term '-' Expression     // 减法
```

它与我们所期望的文法的差别恰恰在于：我们想让 1-2-3 读作 **Expression** 1-2，后跟 -，再后跟 **Term** 3；而实际得到的却是 **Term** 1，后跟 -，再后跟 **Expression** 2-3。也就是说，我们想让 1-2-3 表示 (1-2)-3，得到的却是 1-(2-3)。

没错，调试可能冗长、棘手而耗时；但在本例中，我们实际上是在梳理你在小学就学过的规则，而且当年学起来也不费什么劲。麻烦在于必须把这些规则教给计算机——而计算机可是一个比你慢得多的学习者。

注意，我们本可以把 1-2-3 定义为 1-(2-3) 而非 (1-2)-3，从而完全避开这番讨论。许多最棘手的编程问题，恰恰出现在必须遵从人类在计算机远远未出现之前就已定下的传统规则的时候。

6.5.2.3 表达式：第三次尝试

那么，现在该怎么办？再看看文法（正确文法见 §6.5.2）：任何 **Expression** 都以一个 **Term** 开头，而这样的 **Term** 之后可以跟一个 + 或 -。因此，我们必须查找一个 **Term**，看它后面是否为 + 或 -，并照此进行下去，直到不再有加号或减号。例如：

```
double expression()
{
    double left = term();           // 读入并求值一个项
    Token t = get_token();          // 取得下一个 token
    while (t.kind=='+' || t.kind=='-') {        // 寻找 + 或 -
        if (t.kind == '+')
            left += term();         // 求值项并相加
        else
            left -= term();         // 求值项并相减
        t = get_token();
    }
    return left;                    // 最后：不再有 + 或 -；返回答案
}
```

这版代码有点乱：为了不断寻找加号和减号，引入了一个循环。也有点重复：我们测试了两次 + 和 -，两次调用 `get_token()`。因为它妨碍了对代码逻辑的理解，就把 + 和 - 判断的重复去掉吧：

```
double expression()
{
    double left = term();       // 读入并求值一个项
    Token t = get_token();      // 取得下一个 token
    while (true) {
        switch (t.kind) {
        case '+':
            left += term();     // 求值项并相加
            t = get_token();
            break;
        case '-':
            left -= term();     // 求值项并相减
            t = get_token();
            break;
        default:
            return left;        // 最后：不再有 + 或 -；返回答案
        }
    }
}
```

注意——除了这个循环——它其实与第一次尝试（§6.5.2.1）相当相似。我们做的是把 `expression()` 内对 `expression()` 的提及去掉，换成了一个循环。换句话说，我们是把 **Expression** 文法规则中引用的 **Expression**，翻译成了一个寻找 **Term** 后跟 + 或 - 的循环。

6.5.3 项

Term 的文法规则与 **Expression** 非常相似：

```
Term:
    Primary
    Term '*' Primary
    Term '/' Primary
    Term '%' Primary
```

因此，代码也应非常相似。有此第一版尝试：

```
double term()
{
    double left = primary();
    Token t = get_token();
    while (true) {
        switch (t.kind) {
        case '*':
            left *= primary();
            t = get_token();
            break;
        case '/':
            left /= primary();
            t = get_token();
            break;
        case '%':
            left %= primary();
            t = get_token();
            break;
        default:
            return left;
        }
    }
}
```

遗憾的是，这段代码不能编译：取余运算（%）对浮点数没有定义，编译器会好心地告诉我们这一点。当我们在 §6.3.5 回答问题 5——“我们还应该允许浮点输入吗？”——并信心满满地说“当然！”时，其实并没有把这个问题想透，结果中了功能蔓延（feature creep）的招。这种事总会发生！那怎么办呢？可以在运行时检查% 的两个操作数是否都是整数，不是就报错；也可以干脆不让计算器支持%。眼下先取最简单的做法：以后总可以把% 加回来（见 §7.5）。

去掉% 的分支之后，函数就能正确工作了：各项都能正确地解析和求值。然而，有经验的程序员会发现一个不可接受的细节。如果你输入 2/0，会发生什么？

不能除以零。若你试图这样做，计算机硬件会检测到并终止你的程序，给出的错误信息不甚友好。没有经验的程序员很快就会痛苦地发现这一点。所以，我们最好检查一下，并给出像样的错误信息：

```
double term()
{
    double left = primary();
    Token t = get_token();
    while (true) {
        switch (t.kind) {
        case '*':
            left *= primary();
            t = get_token();
            break;
        case '/':
        {   double d = primary();
            if (d == 0) error("divide by zero");
            left /= d;
            t = get_token();
            break;
        }
        default:
            return left;
        }
    }
}
```

为什么我们把处理 / 的语句放进一个块里？这是编译器要求的：如果想在一个 switch 语句中定义并初始化变量，就必须把它们置于块内。

6.5.4 初等表达式

初等表达式（primary expression）的文法规则同样简单：

```
Primary:
    Number
    '(' Expression ')'
```

实现它的代码有点凌乱，因为这里出现语法错误的机会更多：

```
double primary()
{
    Token t = get_token();
    switch (t.kind) {
    case '(':       // 处理 '(' 表达式 ')'
        {   double d = expression();
            t = get_token();
            if (t.kind != ')') error("')' expected");
            return d;
        }
    case '8':
            return t.value;     // 我们用 '8' 表示一个数；返回该数的值
    default:
            error("primary expected");
    }
}
```

基本上，与 **expression()** 和 **term()** 相比没有什么新东西：用的是同样的语言原语，处理 **Token** 的方式相同，编程技术也一样。

6.6 试运行第一版

要运行这些计算器函数，需要实现 **get_token()** 并提供一个 **main()**。**main()** 很简单：我们只是不停地调用 **expression()** 并打印出其结果：

```
int main()
try {
    while (cin)
        cout << expression() << '\n';
    keep_window_open();
}
catch (exception& e) {
    cerr << e.what() << '\n';
    keep_window_open();
    return 1;
}
catch (...) {
    cerr << "exception\n";
    keep_window_open();
    return 2;
}
```

错误处理就是通常那些“样板代码”（§5.6.3）。让我们把 **get_token()** 的实现推迟到 §6.8 说明，先来测试计算器的这个第一版。

试一试

计算器程序的这一第一版（包括 `get_token()`）作为文件 `calculator00.cpp` 提供。让它跑起来，动手试一试。

不出所料，计算器的第一版没能像我们期望的那样工作。于是我们耸耸肩问："为什么不工作？"或者更确切地，"那么，它为什么像现在这样工作？""它究竟做了什么？"键入一个 **2**，接着一个换行。没有反应。再敲一个换行，看它是不是睡着了。还是没有反应。键入一个 **3**，接着一个换行。没有反应！键入一个 **4**，接着一个换行。它回答了一个 **2**！现在屏幕是这样的：

```
2
3
4
2
```

接着继续键入 **5+6**。程序回应一个 **5**，屏幕看起来是这样的：

```
2
3
4
2
5+6
5
```

除非你以前编过程序，否则你多半会非常困惑！其实，即便是有经验的程序员也可能会困惑。这里发生了什么？此时，你想退出这个程序。该怎么做呢？我们"忘了"编写退出命令，但出错会让程序退出，于是你键入一个 **x**，程序打印 **Bad token** 并退出。终于有一件事按计划进行了！

不过，我们忘了在屏幕上区分输入和输出。在着手解开主要的谜题之前，先修正输出，好让我们看清自己在做什么。加一个 = 来表示输出，眼下这就够了：

```
while (cin) cout << "="<< expression() << '\n';        // 版本 1
```

现在，输入与之前完全相同的字符序列，我们得到

```
2
3
4
= 2
5+6
= 5
x
Bad token
```

奇怪！试着弄清这个程序的行为。我们又试了另外几个例子，但就先看这个。这里有三道谜题：

为什么程序在最初的 **2** 和 **3** 以及换行之后没有回应？

为什么输入 **4** 之后，程序回应的是 **2** 而不是 **4**？

为什么输入 **5+6** 之后，程序回答的是 **5** 而不是 **11**？

面对如此神秘的结果，有很多种继续推进的办法。下一章会考察其中一些，但这里让我们就动动脑子。程序会不会算错了数？这最不可能：**4** 的值不是 **2**，而 **5+6** 的值是 **11** 而非 **5**。想想输入 **1 2 3 4+5 6+7 8+9 10 11 12** 并接一个换行会发生什么。我们得到

```
1 2 3 4+5 6+7 8+9 10 11 12
= 1
= 4
= 6
= 8
= 10
```

咦？没有 **2** 或 **3**。为什么是 **4** 而不是 **9**（即 **4+5**）？为什么是 **6** 而不是 **13**（即 **6+7**）？仔细看：程序每三个 token 才输出一个！也许程序“吞掉”了我们的部分输入而没有求值？确实如此。考虑一下 **`expression()`**：

```
double expression()
{
    double left = term();       // 读入并求值一个项
    Token t = get_token();      // 取得下一个 token
    while (true) {
        switch (t.kind) {
        case '+':
            left += term();     // 求值项并相加
            t = get_token();
            break;
        case '-':
            left -= term();     // 求值项并相减
            t = get_token();
            break;
        default:
            return left;        // 最后：不再有 + 或 -；返回答案
        }
    }
}
```

当 **`get_token()`** 返回的 **Token** 不是 **+** 或 **-** 时，我们只是返回。那个 token 我们没有使用，也没有把它存到任何地方供其他函数稍后使用。这可不明智：连它是什么都还没确定就把输入扔掉，绝不会是好主意。快速看一眼就会发现，**`term()`** 有一模一样的问题。这就解释了为什么计算器每用掉一个 token 就多“吃”掉两个。

让我们修改 **`expression()`**，使它不“吞食”token。当程序用不上下一个 token（`t`）时，应当把它放在哪儿？可以想出很多精巧的方案，但让我们直接跳到那个显而易见的答案（看到了才觉得“显而易见”）：这个 token 会被其他某个读 token 的函数用到，那就把它放回输入流，供其他函数再次读取！实际上，确实可以把字符放回一个 `istream`，但那并不是我们想要的。我们想处理的是 token，而不是摆弄字符。我们想要的是一个处理 token 的输入流，并且可以把已读的 token 放回去。

于是，假定我们有一个 token 流——一个“**`Token_stream`**”——名为 `ts`。进一步假定 **`Token_stream`** 有一个成员函数 **`get()`**，返回下一个 token；还有一个成员函数 **`putback(t)`**，把 token `t` 放回流中。我们

将在 §6.8 实现 **Token_stream**——先看看它需要怎样使用。有了 **Token_stream**，就可以重写 **expression()**，把不用的 token 放回 **Token_stream**：

```
double expression()
{
    double left = term();       // 读入并求值一个项
    Token t = ts.get();         // 从 token 流取得下一个 Token
    while (true) {
        switch (t.kind) {
        case '+':
            left += term();     // 求值项并相加
            t = ts.get();
            break;
        case '-':
            left -= term();     // 求值项并相减
            t = ts.get();
            break;
        default:
            ts.putback(t);      // 把 t 放回 token 流
            return left;        // 最后：不再有 + 或 -；返回答案
        }
    }
}
```

此外，还必须对 **term()** 作同样的修改：

```
double term()
{
    double left = primary();
    Token t = ts.get();         // 从 token 流取得下一个 Token
    while (true) {
        switch (t.kind) {
        case '*':
            left *= primary();
            t = ts.get();
            break;
        case '/':
        {   double d = primary();
            if (d == 0) error("divide by zero");
            left /= d;
            t = ts.get();
            break;
        }
        default:
            ts.putback(t);      // 把 t 放回 token 流
            return left;
        }
```

```
    }
}
```

最后一个解析器函数是 **primary()**，只需把 **get_token()** 改成 **ts.get()**；**primary()** 用掉它读到的每个 token。

6.7 试运行第二版

于是，我们准备好测试第二版了。计算器程序的第二版（包括 **Token_stream**）作为文件 **calculator01.cpp** 提供。让它跑起来，动手试一试。键入 **2** 接一个换行。没有反应。再敲一个换行，看它是不是睡着了。还是没有反应。键入 **3** 接一个换行，它回答了 **2**。试试 **2+2** 接一个换行，它回答了 **3**。现在你的屏幕是这样的：

```
2
3
=2
2+2
=3
```

嗯。也许我们引入的 **putback()** 及其在 **expression()** 和 **term()** 中的使用并没有解决问题。再做一次测试：

```
2 3 4 2+3 2*3
= 2
= 3
= 4
= 5
```

对！这些答案都正确！但最后一个答案（**6**）不见了。仍然存在 token 超前（look-ahead）问题。不过这一次，问题不在于代码“吃”字符，而是在我们输入下一个表达式之前，程序没有为一个表达式产生任何输出。表达式的结果不是立即打印的；输出被推迟到程序看到下一个表达式的第一个 token 时。遗憾的是，只有在我们键完下一个表达式并按下 Return 之后，程序才会看到那个 token。程序其实并不算错，只是反应慢了点。

这该怎么修？一个显而易见的方案是引入“打印命令”。因此，让我们接受在表达式之后跟一个分号来终止它并触发输出。既然改到这儿了，就再加一个“退出命令”，以便体面地退出。字符 **q**（quit）用作退出命令就很合适。在 **main()** 中，我们有

```
while (cin) cout << "=" << expression() << '\n';        // 版本 1
```

可以把它改成更凌乱但更有用的形式：

```
double val = 0;
while (cin) {
    Token t = ts.get();

    if (t.kind == 'q') break;       // 'q' 表示 “退出”
    if (t.kind == ';')              // ';' 表示 “现在打印”
        cout << "=" << val << '\n';
    else
        ts.putback(t);
    val = expression();
}
```

现在，计算器真正可用了。例如，我们得到

```
2;
= 2
2+3;
= 5
3+4*5;
= 23
q
```

至此，我们有了计算器的一个不错的初始版本。它还不完全是我们真正想要的，但我们已经有了一个程序，可以作为基础去做出一个更满意的版本。重要的是，现在我们可以一边保持程序始终可运行，一边一个一个地修问题、加功能。

6.8 记号流

在进一步改进计算器之前，先给出 `Token_stream` 的实现。毕竟，在获得正确的输入之前，什么都不行——完全不行。我们是先实现 `Token_stream` 的；但在展示最小解之前，不想在计算问题上岔得太远。

计算器的输入是一个 token 的序列，正如我们在 §6.3.3 中对 `(1.5+4)*11` 所展示的那样。我们需要一种东西，它从标准输入 `cin` 读入字符，并在程序索要时向程序提供下一个 token。此外我们已经看到，我们——也就是我们的计算器程序——经常会多读一个 token，因此必须能够把它放回去供以后使用。这是典型而根本的问题：严格地从左往右读到 `1.5+4` 时，如果不读到 `+`，你怎么能知道数 `1.5` 已经读完了呢？在看到 `+` 之前，我们可能正走在读 `1.55555` 的路上。所以，我们需要一种“流”：用 `get()` 向它索要时它产出一个 token，并且可以用 `putback()` 把一个 token 放回流中。C++ 中我们使用的任何东西都有类型，所以第一步就是定义类型 `Token_stream`。

你可能已经注意到 §6.3.3 中 **Token** 定义里的 `public:`。在那里它似乎毫无用处，但对 `Token_stream` 就需要它了，必须说明其功用。C++ 用户自定义类型（user-defined type）通常由两部分组成：公共接口（标注为 `public:`）和实现细节（标注为 `private:`）。其思想是：把类型的用户为了方便使用所需要的内容，与我们实现该类型所需要的细节分离开来——那些细节用户最好别乱碰：

```
class Token_stream {
public:
        // 用户接口
private:
        // 实现细节
        //  (Token_stream 的用户不能直接访问)
};
```

显然，用户和实现者常常是同一些人——不过是我们在“扮演不同的角色”。但把（面向用户的公共）接口与（只由实现者使用的私有）实现细节区分开，是组织代码结构的一种有力手段。公共接口应当包含（且仅包含）用户需要的东西，通常是一组函数。私有实现包含实现那些公共函数所必需的内容，通常是数据和处理杂乱细节的函数，用户既无需了解也不应直接使用。

让我们把 **Token_stream** 类型再细化一些。用户希望从它那里得到什么？显然是 **get()** 和 **putback()** 函数——这正是我们发明“token 流”这一概念的原因。**Token_stream** 要把为输入而读入的字符做成 **Token**，因此我们需要能构造一个 **Token_stream**，并把它定义为从 **cin** 读取。于是，最简单的 **Token_stream** 看上去是这样：

```
class Token_stream {
public:
        Token_stream();         // 构造一个从 cin 读取的 Token_stream
        Token get();            // 取得一个 Token
        void putback(Token t);  // 放回一个 Token
private:
        // implementation details
};
```

这就是使用 **Token_stream** 的用户所需要的全部。有经验的程序员会奇怪，为什么 **cin** 是唯一的字符来源；但我们决定从键盘获取输入。这个决定将在第 7 章的一道习题中重新审视。

为什么用冗长的名字 **putback()**，而不用逻辑上已经够用的 **put()**？我们想强调 **get()** 与 **putback()** 之间的不对称性：这是一个输入流，不是也能用于一般输出的东西。另外，**istream** 也有一个 **putback()** 函数：命名上的一致性是一个系统有用的属性，能帮助人们记忆，也帮助人们避免错误。

现在就可以构造一个 **Token_stream** 并使用它：

```
Token_stream ts;        // 一个名为 ts 的 Token_stream
Token t = ts.get();     // 从 ts 取得下一个 Token
// ...
ts.putback(t);          // 把 Token t 放回 ts
```

这就是编写计算器其余部分所需的全部。

6.8.1 实现 Token_stream

现在需要实现那三个 **Token_stream** 函数。我们如何表示一个 **Token_stream**？也就是说，要让 **Token_stream** 履行职责，需要在其中存储哪些数据？我们需要空间存放放回 **Token_stream** 的 token。为简单起见，假定一次最多放回一个 token。这对我们的程序（以及许多许多类似的程序）恰好够用。这样，我们只需要存放一个 **Token** 的空间，以及一个指示该空间是满还是空的标志：

```
class Token_stream {
public:
        Token get();            // 取得一个 Token (get() 的定义见 §6.8.2)
        void putback(Token t);  // 放回一个 Token
private:
        bool full {false};      // 缓冲区里有 Token 吗？
        Token buffer;           // 这里存放用 putback() 放回的 Token
};
```

现在可以定义（“编写”）这两个成员函数了。**putback()** 很简单，所以先定义它。**putback()** 成员函数把它的实参放回 **Token_stream** 的缓冲区：

```
void Token_stream::putback(Token t)
{
        buffer = t;             // 把 t 复制到缓冲区
        full = true;            // 缓冲区现在满了
}
```

关键字 **void**（意为“无”）用来表示 **putback()** 不返回值。

当我们在类定义本身之外定义类的成员时，必须指明所说的成员属于哪个类。为此使用这样的记法

```
class_name :: member_name
```

这里，我们定义的是 **Token_stream** 的成员 **putback**。

为什么要到类之外定义成员？最主要的答案是明晰：类定义（大致上）陈述类能做什么；成员函数定义是规定事情怎么做才能得以完成的实现。我们更愿意把它们放在“别处”，免得分心。我们的理想是让程序中每个逻辑实体都恰好占一屏。若成员函数定义放在别处，类定义通常能做到这一点；若放在类定义内部（“类内”，in-class），就做不到。

如果想确保不会在没有用 **get()** 读走先前放回的东西之前再次调用 **putback()**，可以加一个测试：

```
void Token_stream::putback(Token t)
{
        if (full) error("putback() into a full buffer");
        buffer = t;             // 把 t 复制到缓冲区
        full = true;            // 缓冲区现在满了
}
```

对 full 的测试检查的是前置条件（pre-condition）“缓冲区中没有 **Token**”。

显然，**Token_stream** 应当从空开始。也就是说，在第一次调用 **get()** 之前 full 应为 false。我们在 **Token_stream** 的定义中直接初始化 full 来做到这一点。

6.8.2 读取记号

真正的工作由 **get()** 完成。如果 **Token_stream::buffer** 中并没有现成的 **Token**，**get()** 就必须从 **cin** 读取字符，并把它们组装成 **Token**：

```
Token Token_stream::get()
{
    if (full) {                     // 是否已经有一个 Token 备好？
            full = false;           // 从缓冲区取出 Token
            return buffer;
    }
    char ch;
    cin >> ch;      // 注意 >> 会跳过空白（空格、换行、制表符等）

    switch (ch) {
    case ';':       // 用于 “打印”
    case 'q':       // 用于 “退出”
    case '(': case ')': case '+': case '-': case '*': case '/':
            return Token{ch};       // 让每个字符表示它自己
    case '.':
    case '0': case '1': case '2': case '3': case '4':
    case '5': case '6': case '7': case '8': case '9':
    {       cin.putback(ch);        // 把数字放回输入流
            double val;
            cin >> val;             // 读入一个浮点数
            return Token{'8',val};  // 让 '8' 表示 “一个数”
    }
    default:
            error("Bad token");
    }
}
```

让我们详细考察 `get()`。首先检查缓冲区中是否已有 **Token**；如果有，直接返回它：

```
if (full) {         // 是否已经有一个 Token 备好？
        full = false;       // 从缓冲区取出 Token
        return buffer;
    }
```

只有当 `full` 为 `false`（即缓冲区里没有 token）时，才需要处理字符。这时读入一个字符并恰当地处理它。我们寻找圆括号、运算符和数。其他任何字符都会引发终止程序的 `error()` 调用：

```
default:
        error("Bad token");
```

`error()` 函数在 §5.6.3 中有所描述，我们在 `std_lib_facilities.h` 中使其可用。

我们必须决定如何表示不同种类的 **Token**；也就是说，必须为成员 `kind` 选定一些值。为了简单和便于调试，我们决定让 **Token** 的 `kind` 就是圆括号和运算符本身。这使得处理圆括号和运算符非常简单：

```
case '(': case ')': case '+': case '-': case '*': case '/':
        return Token{ch};       // 让每个字符表示它自己
```

说实话，在第一版里我们忘了';'（“打印”）和'q'（“退出”），直到第二版解法需要它们时才把这两个字符加上。

6.8.3 读取数

现在只剩数要处理了。这实际上不那么容易。到底怎么求出 123 的值？嗯，它是 100+20+3；那 12.34 呢？要接受科学记数法，如 12.34e5 吗？把这件事彻底弄对也许要花上几个小时甚至几天，但幸好不必如此。输入流知道 C++ 字面值长什么样，也知道怎样把它们变成 double 类型的值。我们要做的只是想清楚如何在 get() 内部让 cin 替我们做这些：

```
case '.':
case '0': case '1': case '2': case '3': case '4':
case '5': case '6': case '7': case '8': case '9':
{   cin.putback(ch);        // 把数字放回输入流
    double val;
    cin >> val;             // 读入一个浮点数
    return Token{'8',val};  // 让 '8' 表示 “一个数”
}
```

我们（多少有些武断地）选定'**8**' 来表示 **Token** 中的“一个数”。

我们怎么知道马上要来的是一个数？凭经验猜测，或查一查 C++ 参考资料（例如附录 A），可以发现数值字面值必须以一个数字或 . （小数点）开头。因此我们检测这一点。接下来应当让 cin 来读这个数，但我们已经读进了第一个字符（一个数字或点），于是如果放任 cin 去处理剩余部分就会得出错误的结果。可以尝试把第一个字符的值与 cin 所读的“其余部分”的值合并；例如，若有人键入 123，我们会得到 1，cin 会读到 23，我们就得往 23 上加 100。真怪物！而那只是个最简单的情形呢。幸好（而这并非偶然），cin 的用法与 Token_stream 很像：你也可以把一个字符放回去。所以，不必做任何凌乱的算术，只需把最初的字符放回 cin，然后让 cin 读入整个数。

请注意，我们一次又一次地避免做复杂的工作，而是寻找更简单的解法——常常是依靠库设施。这正是编程的精髓：对简单性的持续追求。这种追求有时——带点戏谑地——被表述为“好程序员都是懒惰的”。在这个意义上（且只在这个意义上），我们的确应该“懒惰”：既然有办法写出少得多的代码，何苦写一大堆？

6.9 程序结构

有时俗话说的好，只见树木，不见森林。同样，盯着一个程序的所有函数、类等细节时，也很容易看不清整个程序。所以，让我们略去细节，看一看这个程序的整体：

```
#include "std_lib_facilities.h"

class Token { /* ... */ };
class Token_stream { /* ... */ };

void Token_stream::putback(Token t) { /* ... */ }
Token Token_stream::get() { /* ... */ }

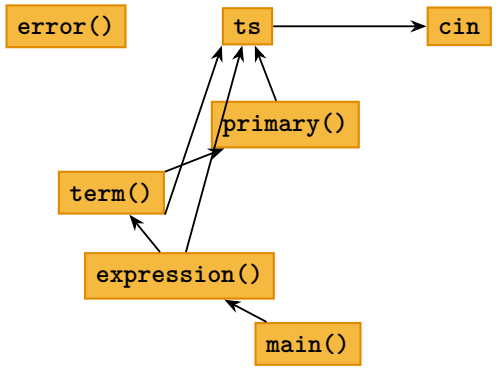
Token_stream ts;        // 提供 get() 和 putback()
double expression();    // 声明，使 primary() 可以调用 expression()

double primary() { /* ... */ }      // 处理数与圆括号
double term() { /* ... */ }         // 处理 * 与 /
double expression() { /* ... */ }   // 处理 + 与 -

int main() { /* ... */ }            // 主循环，处理错误
```

声明的顺序很重要。一个名字在被声明之前不能使用，因此 `ts` 必须在 **`ts.get()`** 使用它之前声明，而 **`error()`** 也必须在各个解析器函数之前声明，因为它们都要用到它。调用图中有一个有趣的环路：**`expression()`** 调用 **`term()`**，**`term()`** 调用 **`primary()`**，**`primary()`** 又调用 **`expression()`**。

可以用图形来表现这一点（省去对 **`error()`** 的调用——每个人都会调用 error()）：



这意味着我们不能只是按任意顺序定义那三个函数：不存在一种能让我们在每个函数被使用之前就定义好它的顺序。我们至少需要一个本身不是定义的声明。我们选择声明（“前向声明”，forward declare）**`expression()`**。

但这样能行吗？行，就“行”的某种定义来说。它能编译、能运行、能正确求值表达式，也能给出像样的错误信息。但它以我们喜欢的方式工作了吗？不出意料的答案是“并不真的如此”。我们在 §6.6 试过第一版并去掉了一个严重的 bug。第二版（§6.7）也没有好多少。不过这没关系（本来就该如此）。对它的主要目的来说，它已经够好了；这个主要目的，就是给我们一个可以检验基本想法并从其身上获得反馈的东西。就这一点而言它是成功的。但试试吧：它仍然会把你逼疯！

试一试

让上面给出的计算器运行起来，看看它做些什么，并试着弄清它为什么会那样工作。

小试身手

这个“小试身手”要求对同一个有 bug 的程序做一连串修改，把它从没什么用处的东西变成相当有用的东西。

1. 从文件 `calculator02buggy.cpp` 中取出该计算器。让它通过编译。你需要找出并修好少许 bug。书正文中没有讲这些 bug。找出狡猾地埋进 `calculator02buggy.cpp` 的三个逻辑错误并将其消除，使计算器产生正确的结果。
2. 把退出命令所用字符从 `q` 改为 `x`。
3. 把打印命令所用字符从 `;` 改为 `=`。
4. 在 `main()` 中加一句问候语：

```
"Welcome to our simple calculator.
Please enter expressions using floating-point numbers."
```

5. 改进这句问候：指出有哪些可用运算符，以及如何打印和退出。

要点回顾

1. 我们所说的“Programming is understanding（编程即理解）”是什么意思？
2. 本章详述了一个计算器程序的创建过程。写一小段分析，说明计算器应当能做哪些事。
3. 如何把一个问题拆成若干较易处理的小部分？
4. 为什么先创建一个小的、功能有限版本的程序是个好主意？
5. 为什么功能蔓延（feature creep）不是好主意？
6. 软件开发有哪三个主要阶段？
7. 什么是“用例”（use case）？
8. 测试的目的是什么？
9. 按照本章的提纲，描述 **Term**、**Expression**、**Number** 和 **Primary** 之间的差别。
10. 在本章中，输入被分解为其组成部分 **Term**、**Expression**、**Primary** 和 **Number**。请对 `(17+4)/(5-1)` 作同样的分解。
11. 程序里为什么没有一个叫 `number()` 的函数？
12. 什么是 token？
13. 什么是文法（grammar）？文法规则是什么？
14. 什么是类（class）？我们用类来做哪些事？
15. 我们如何为一个类的成员提供默认值？
16. 在 expression 函数中，为什么 switch 语句的 default 分支是把 token “放回去”？
17. 什么是“超前查看”（look-ahead）？
18. `putback()` 做些什么？它为什么有用？
19. 为什么取余（模）运算% 在 `term()` 中难以实现？
20. **Token** 类的那两个数据成员各用于什么？
21. 我们为什么（有时）要把一个类的成员分为 `private` 成员和 `public` 成员？
22. 当 `Token_stream` 类的缓冲区里有一个 token 而 `get()` 函数被调用时，会发生什么？
23. 为什么要把’;’ 和’**q**’ 字符加进 `Token_stream` 类 `get()` 函数的 switch 语句中？

24. 我们应当何时开始测试程序？
25. 什么是“用户自定义类型”（user-defined type）？我们为什么想要它？
26. C++“用户自定义类型”的接口是什么？
27. 为什么我们要依靠现成的代码库？

术语

分析　文法　原型
`class`　实现　伪代码
类成员　接口　`public`
数据成员　成员函数　语法分析程序（syntax analyzer）
设计　语法分析器（parser）　单词（token）
除以零　`private`　用例（use case）

习题

1. 如果你还没有做，请完成本章的各个“试一试”练习。
2. 让程序除了支持 `()` 还支持 `{}`，使 `{(4+5)*6} / (3+4)` 成为合法的表达式。
3. 增加一个阶乘运算符：使用一个后缀 `!` 运算符来表示“阶乘”。例如，表达式 `7!` 表示 `7*6*5*4*3*2*1`。让 `!` 的结合强度高于 `*` 和 `/`；也就是说，`7*8!` 应当意为 `7*(8!)` 而不是 `(7*8)!`。首先修改文法，让它顾及这种更高一级的运算符。为了符合阶乘的标准数学定义，令 `0!` 的值为 `1`。提示：计算器函数处理的是 double 值，但阶乘只对 int 定义，所以对于 `x!`，只需把 `x` 转成 int 并计算该 int 的阶乘。
4. 定义一个类 `Name_value`，其中存放一个 string 和一个值。改做第 4 章习题 19，用 `vector{Name_value}` 代替两个向量。
5. 把定冠词 **the** 加进 §6.4.1 的“英语”文法，使它能描述诸如“The birds fly but the fish swim.”这样的句子。
6. 写一个程序，按 §6.4.1 的“英语”文法检查一个句子是否正确。假定每个句子都以一个句点（.）结尾，且句点两侧有空白。例如，birds fly but the fish swim . 是一个句子，而 birds fly but the fish swim（缺句点）和 birds fly but the fish swim.（句点之前无空格）都不是。对输入的每个句子，程序只需回答“OK”或“not OK”。提示：别去管 token，直接用 » 读入一个 string 即可。
7. 为按位逻辑表达式写一个文法。按位逻辑表达式与算术表达式很相似，只是其运算符是 `!`（非，not）、`~`（取反，complement）、`&`（与，and）、`|`（或，or）以及 `^`（异或，exclusive or）。每个运算符对其整型操作数的每个位进行运算（见 §25.5）。`!` 和 `~` 为前缀单目运算符。`^` 的结合强度高于 `|`（正如 `*` 高于 `+`），因此 `x|y^z` 意为 `x|(y^z)` 而非 `(x|y)^z`。`&` 的结合强度高于 `^`，因此 `x^y&z` 意为 `x^(y&z)`。
8. 改做第 5 章习题 12 的“公牛与母牛”游戏，让它针对四个字母而不是四个数字。
9. 写一个程序，读入数字并把它们组成整数。例如，`123` 是读作 `1`、`2`、`3` 三个字符。程序应当输出 123 is 1 hundred and 2 tens and 3 ones。该数应作为 int 值输出。请处理一、二、三、四位的数字。提示：要从字符 `'5'` 得到整数值 `5`，用 `'5'` 减去 `'0'`，即 `'5'-'0'` 为 5。
10. 排列（permutation）是一个集合中元素的有序子集。例如，设想你要为保险柜挑选一组密码：可用的数字共 60 个，而你需要三个不同的数字。可供选择的组合共有 $P(60,3)$ 种排列，其中 P 由下式定义

$$P(a,b) = \frac{a!}{(a-b)!}$$

其中! 用作后缀的阶乘运算符。例如，4! 为 4×3×2×1。组合（combination）与排列相似，只是对象的顺序无关紧要。例如，你要做一份“香蕉船”圣代，想用拥有的五种冰激凌中的三种，那么先放一勺香草还是最后放一勺香草你都不会在乎；你反正用了香草。组合的计算公式为

$$C(a,b) = \frac{P(a,b)}{b!}$$

设计一个程序：询问用户两个数，再询问他们想计算排列还是组合，然后打印出结果。这个程序会有若干部分。请对上述需求做分析，准确地写出程序必须做什么；然后进入设计阶段，为程序写出伪代码，并把程序拆成子部件。这个程序应当带错误检查：确保所有错误的输入都会产生好的错误信息。

后记

读懂输入是编程的基本活动之一。每个程序都会以某种方式面对这个问题。而理解由人直接产生的东西，是其中最难的问题之一。例如，语音识别的许多方面至今仍是研究课题。这一问题的简单变体（比如我们的计算器）则通过用文法定义输入来应对。

7 完成程序

“女主角唱罢，歌剧才算落幕。”
——歌剧谚语

编写程序的过程，就是把你“想做什么”以及“想如何表达它”的想法逐步打磨清晰的过程。在第 6 章中，我们做出了一个计算器程序的最初可用版本。这里，我们将对它加以精炼。完成这个程序——也就是说，让它适于用户使用、适于维护者维护——包括改进用户界面、在错误处理上下真功夫、添加若干有用的特性，以及重新组织代码结构以便于理解和修改。

7.1 引言

当你的程序第一次“基本能跑”起来时，你可能才完成了一半；而对于一个大型程序，或者一个一旦行为失常就可能造成危害的程序来说，你离一半还远得很。一旦程序“基本可用”，真正有趣的工作才刚刚开始！这时候，我们才有了足够多的可运行代码去试验各种想法。

在本章中，我们将带你体会一个专业程序员在设法改进第 6 章的计算器时可能有的种种考量。注意，这里对程序提出的问题和考察的议题，远比计算器本身有趣得多。我们要做的是给出一个例子，展示真实的程序如何在需求与约束的压力下演化，以及程序员如何逐步改进代码。

7.2 输入与输出

如果回顾第 6 章的开头，你会发现我们当时决定用这样的方式提示（prompt）用户：

```
Expression:
```

并用这样的方式报告答案：

```
Result:
```

在让程序跑起来的紧张忙碌中，我们把这些忘得一干二净。这很典型。我们不可能时时刻刻想着所有事情，所以一旦停下来回顾，就会发现自己忘了些什么。

对某些编程任务来说，最初的需求是不能改变的。这种政策通常过于僵硬，会导致程序对它所要解决的问题给出不必要地拙劣的解。所以，让我们假定程序规格说明——即程序到底该做什么——是可以修改的，来考虑一下我们会怎么做。我们真的希望程序输出 **`Expression:`** 和 **`Result:`** 吗？我们怎么知道呢？光靠“想”很少有用。我们必须动手去试，看看怎样效果最好。

7.1 引言
7.2 输入与输出
7.3 错误处理
7.4 负数
7.5 余数：%
7.6 清理代码
7.6.1 符号常量
7.6.2 函数的使用
7.6.3 代码布局
7.6.4 注释
7.7 从错误中恢复
7.8 变量
7.8.1 变量与定义
7.8.2 引入名字
7.8.3 预定义名字
7.8.4 我们完成了吗？

```
2+3; 5*7; 2+9;
```

目前会得到

```
= 5
= 35
= 11
```

如果用上 **Expression:** 和 **Result:**，我们会得到

```
Expression: 2+3; 5*7; 2+9;
Result : 5
Expression: Result: 35
Expression: Result: 11
Expression:
```

可以肯定，有人会喜欢其中一种风格，也有人会喜欢另一种。遇到这种情况，我们可以考虑让人们自行选择，但对这个简单的计算器来说那就小题大做了，所以我们必须做出决定。我们认为输出 **Expression:** 和 **Result:** 有点太“重”了，让人分心。用了它们，屏幕上真正重要的表达式和结果反而只占次要位置；而表达式和结果才是关键，不该有任何东西喧宾夺主。另一方面，如果不以某种方式把用户的输入和计算机的输出区分开，结果会令人困惑。在最初的调试中，我们加入了 = 作为结果指示符。我们还想要一个简短的“提示符”，表明程序在等待输入。字符 > 常被用作提示符：

```
> 2+3;
= 5
> 5*7;
= 35
>
```

这样看起来好多了，而且只需对 `main()` 的主循环稍加修改就能实现：

```
double val = 0;
while (cin) {
    cout << "> " ;       // 打印提示
    Token t = ts.get();
    if (t.kind == 'q') break;
    if (t.kind == ';')
        cout << "= " << val << '\n';        // 打印结果
    else
        ts.putback(t);
    val = expression();
}
```

遗憾的是，把多个表达式放在一行里，结果仍然很乱：

```
> 2+3; 5*7; 2+9;
= 5
> = 35
> = 11
>
```

根本问题在于，我们一开始（至少我们假装如此）没有考虑到一行里放多个表达式的情况。我们想要的是

```
> 2+3; 5*7; 2+9;
= 5
= 35
= 11
>
```

这看起来对了，但遗憾的是，没有真正显而易见的办法来实现它。我们先看 `main()`：有没有办法只在不紧接着跟一个 = 时输出 >？我们无法知道！我们需要在调用 `get()` 之前输出 >，但 `get()` 究竟是读入了新字符，还是只是把从键盘上已读过的字符组成的 **Token** 交给我们，我们无从判断。换句话说，为了这个最终的改进，我们不得不去折腾 `Token_stream`。

目前，我们认为现状已经足够好了。如果将来不得不修改 `Token_stream`，我们会重新考虑这个决定。不过，为了一个次要的好处而大动干戈地做结构性改动是不明智的，何况我们还没有对计算器进行彻底的测试。

7.3 错误处理

一旦程序“基本可用”，要做的第一件事就是设法把它弄坏；也就是说，我们尝试喂给它各种输入，指望看到它行为失常。我们说“指望”，是因为这里的挑战是要找出尽可能多的错误，以便在别人发现之前把它们修好。如果你带着“我的程序没问题，我不会犯错误！”的态度去做这件事，你会发现不了几个 bug，而真发现一个时又会很沮丧。你这是在跟自己玩心理游戏！测试时应有的态度是：“我要弄坏它！我比任何程序都聪明——包括我自己的！”所以，我们给计算器喂一组正确与错误的表达式混合体。例如：

```
1+2+3+4+5+6+7+8
1-2-3-4
!+2
;;;
(1+3;
(1+);
1*2/3%4+5-6;
();
1+;
+1
1++;
1/0
1/0;
1++2;
-2;
-2;;;;
1234567890123456;
'a';
q
1+q
1+2; q
```

试一试

给计算器喂上几个这样的“问题”表达式，看看你能用多少种方式让它行为失常。你能让它崩溃吗——也就是说，让它突破我们的错误处理而报出机器级错误？我们认为你不能。你能让它在不给出有用错误信息的情况下退出吗？你能。

严格来说，这就叫作测试（testing）。有人就是以此为生——专门弄坏程序。测试是软件开发中非常重要的一环，而且实际上可以很有趣。第 26 章会较为详细地讨论测试。一个大问题是：“我们能否系统地测试程序，从而找出所有的错误?”这个问题没有一般性的答案；也就是说，不存在对所有程序都成立的答案。不过，只要认真对待测试，对很多程序你都能做得相当好。你会系统地构造测试用例；同时，以防你挑选测试的策略不够完备，你还要做一些“不讲道理”的测试，例如：

```
Mary had a little lamb
srtvrqtiewcbet7rewaewre-wqcntrretewru754389652743nvcqnwq;
!@#$%^&*()~:;
```

有一次测试编译器时，我养成了一个习惯：把报告编译错误的电子邮件原封不动地喂给编译器——邮件头、用户的解释，统统都给。这种做法并不“讲道理”，因为“没人会那样做”。然而，一个程序理想情况下应当捕获所有错误，而不只是那些“讲道理”的错误。不久之后，那个编译器对“怪异输入”的抵抗力就非常强了。

在测试计算器时，我们注意到的第一件真正恼人的事是：输入下面这些内容后，窗口会立即关闭：

```
+1;
()
!+2
```

稍加思考（或者跟踪一下程序的执行）就能发现问题所在：错误信息刚一写出，窗口就被关闭了。之所以会这样，是因为我们保持窗口存活的机制是等待你输入一个字符。然而，在上面三种情形中，程序在读入全部字符之前就检测到了错误，于是输入行上还残留着一个字符。程序无法区分这种“残留”字符与响应 **Enter a character to close window** 提示而输入的字符。于是，那个“残留”字符就把窗口关掉了。

我们可以通过修改 `main()` 来解决这个问题（参见 §5.6.3）：

```
catch (runtime_error& e) {
    cerr << e.what() << '\n';
    // keep_window_open():
    cout << "Please enter the character ~ to close the window\n";
    for (char ch; cin >> ch; )        // 一直读，直到找到 ~
        if (ch=='~') return 1;
    return 1;
}
```

基本上，我们用自己的代码取代了 **keep_window_open()**。注意，如果恰好有一个 是错误发生后要读入的字符，我们的问题依然存在，不过这种情况相当罕见。

遇到这个问题时，我们写了一个接受 `string` 作为实参的 **keep_window_open()** 版本，它只有在你在提示后输入该 `string` 时才关闭窗口。因此，更简单的解决办法是

```
catch (runtime_error& e) {
    cerr << e.what() << '\n';
    keep_window_open("~~");
    return 1;
}
```

现在，诸如

```
+1
!1~~
()
```

这样的例子会让计算器给出正确的错误信息，然后说

```
Please enter ~~ to exit
```

并且在你输入 之前不会退出。

计算器从键盘读取输入。这让测试变得枯燥：每做一次改进，我们都得（再次!）输入一大堆测试用例，以确保没有弄坏什么。如果我们能把测试用例存放在某处，并用一条命令来运行它们，那就好多了。某些操作系统（尤其是 Unix）可以轻而易举地让 **cin** 从文件读取输入而不必修改程序，同样也能把 `cout` 的输出转向到文件。如果这不便做到，我们就必须修改程序，改为使用文件（参见第 10 章）。

现在考虑：

```
1+2; q
```

和

```
1+2 q
```

我们希望两者都能打印出结果（3），然后退出程序。说来奇怪，

```
1+2 q


1+2; q
```

却能做到这一点，而看上去更规整的

```
double val = 0;
while (cin) {
    cout << "> ";
    Token t = ts.get();
    if (t.kind == 'q') break;
    if (t.kind == ';')
        cout << "= " << val << '\n';
    else
        ts.putback(t);
    val = expression();
}
```

却引发一个 **Primary expected** 错误。我们该到哪儿找这个错误呢？当然是在处理；和 q 的 main() 里。我们当初为了尽快让计算器跑起来，匆匆加了“打印”和“退出”这两条命令（§6.7），现在开始为那份仓促付出代价了。再看一次：

```
int main()
try
{
    while (cin) {
        cout << "> ";
        Token t = ts.get();
        while (t.kind == ';') t=ts.get();      // 吃掉 ';'
        if (t.kind == 'q') {
            keep_window_open();
            return 0;
        }
        ts.putback(t);
        cout << "= " << expression() << '\n';
    }
    keep_window_open();
    return 0;
}
```

```
catch (exception& e) {
    cerr << e.what() << '\n';
    keep_window_open("~~");
    return 1;
}
catch (...) {
    cerr << "exception \n";
    keep_window_open("~~");
    return 2;
}
```

这样就有了相当健全的错误处理。于是我们可以开始考虑，还能做些什么来改进计算器。

7.4 负数

如果你测试过这个计算器，就会发现它不能优雅地处理负数。例如，下面这样是一个错误：

```
-1/2
```

我们必须写成

```
(0-1)/2
```

这不能接受。

在后期调试和测试中才发现这类问题是很常见的。直到现在，我们才有机会看清我们的设计究竟效果如何，并获得让我们改进想法的反馈。在规划项目时，明智的做法是设法保留时间和灵活性，以便从我们在此学到的经验中受益。太多时候，“1.0 版”在缺乏必要改进的情况下就发布了，因为紧张的进度或僵化的项目管理策略阻止了对规格说明的“晚期”修改；而“晚期”添加“特性”尤其令人畏惧。实际上，当一个程序已经好到可以供其设计者简单使用、但尚未准备好发布时，它在开发序列中并不算“晚”；这恰恰是我们能从程序的扎实使用经验中获益的最早时机。切合实际的进度安排会把这一点考虑进去。

就本例而言，我们基本上需要修改文法以允许一元负号。最简单的改动似乎是在 **Primary** 上。我们有

```
Primary:
        Number
        "(" Expression ")"
```

而我们需要类似这样的东西：

```
Primary:
        Number
        "(" Expression ")"
        "-" Primary
        "+" Primary
```

我们加入一元正号，是因为 C++ 就是这么做的。一旦有了一元负号，总会有人去试一元正号，与其解释它为什么没用，不如直接实现它更省事。实现 **Primary** 的代码就变成了

```
double primary()
{
    Token t = ts.get();
    switch (t.kind) {
    case '(':           // 处理 '(' 表达式 ')'
        {
            double d = expression();
            t = ts.get();
            if (t.kind != ')') error("')' expected");
            return d;
        }
    case '8':                   // 我们用 '8' 表示一个数
        return t.value;         // 返回数的值
    case '-':
        return - primary();
    case '+':
        return primary();
    default:
        error("primary expected");
    }
}
```

改动简单得惊人，而且居然一次就通过了。

7.5 余数：%

当初分析对计算器的理想要求时，我们就想要余数（取模，modulo）运算符 %。然而，% 对浮点数没有定义，所以我们退让了。现在可以重新考虑它了。它应该很简单：

1. 我们把 % 加为一个 **Token**。
2. 我们为 % 定义一个含义。

我们知道 % 对整数操作数的含义。例如：

```
> 2%3;
= 2
> 3%2;
= 1
> 5%3;
= 2
```

但对于不是整数的操作数该怎么处理呢？考虑：

```
> 6.7%3.3;
```

结果值应该是什么？没有一个完美的技术答案。不过，取模常常也被定义为可用于浮点操作数。特别地，`x%y` 可以定义为 `x-y=x-y*int(x/y)`，这样 `6.7%3.3==6.7-3.3*int(6.7/3.3)`，即 `0.1`。用标准库函数 `fmod()`（浮点取模，来自 `<cmath>`，§24.8）可以轻而易举地做到。我们修改 `term()`，加入

```
case '%':
{       double d = primary();
    if (d == 0) error("divide by zero");
    left = fmod(left,d);
    t = ts.get();
    break;
}
```

`<cmath>` 库是我们找到所有标准数学函数的地方，例如 `sqrt(x)`（x 的平方根）、`abs(x)`（x 的绝对值）、`log(x)`（x 的自然对数）以及 `pow(x,e)`（x 的 y 次幂）。

另一种做法是禁止对浮点实参使用 `%`。我们检查浮点操作数是否带有小数部分，如果有就给出错误信息。确保 `%` 的 **int** 操作数的问题是窄化问题（§3.9.2 和 §5.6.4）的一个变种，因此我们可以用 **narrow_cast** 来解决：

```
case '%':
{       int i1 = narrow_cast<int>(left);
    int i2 = narrow_cast<int>(primary());
    if (i2 == 0) error("%: divide by zero");
    left = i1%i2;
    t = ts.get();
    break;
}
```

对一个简单的计算器来说，两种解法都行。

7.6 清理代码

我们已经对代码做了几处修改。我们认为这些都是改进，但代码开始显得有点乱了。现在是回顾代码的好时机，看看能否让它更清晰、更短，增添和改进注释，等等。换句话说，只有当程序达到适合交给别人维护的状态时，我们才算完成。除了几乎完全没有注释之外，计算器代码其实还不算太糟，但让我们来做一点清理。

7.6.1 符号常量

回顾之下，我们发现用 '8' 来指示含数值的 **Token** 有点怪。用什么值来指示数 **Token** 其实无关紧要，只要该值不同于所有其他指示不同种类 **Token** 的值即可。然而，这代码看起来确实有点怪，我们不得不在注释中一再提醒自己：

```
case '8':                // 我们用 '8' 表示一个数
    return t.value;      // 返回数的值
case '-':
    return - primary();
```

老实说，我们还犯过几个错误，把 '0' 敲成了 '8'——因为忘了自己当初选的是哪个值。换句话说，在操作 **Token** 的代码里直接使用 '8' 太草率、太难记、也容易出错；'8' 正是我们在 §4.3.1 中告诫过的“魔数”之一。我们本该为我们用来表示数的那个常量引入一个符号名：

```
const char number = '8';       // t.kind==number 表示 t 是一个数 Token
```

const 修饰符只是告诉编译器，我们定义的是一个不应被改变的对象：例如，赋值 **number='0'** 会导致编译器给出错误信息。有了 number 的这个定义，我们就再也不必显式使用 '8' 了。上面 primary 中的代码片段现在变成

```
case number:
    return t.value;          // 返回数的值
case '-':
    return - primary();
```

这就用不着注释了。凡是能用代码清楚而直接表达的，就不要写在注释里。用重复的注释来解释某件事，往往意味着代码本身该改进了。

类似地，**Token_stream::get()** 中识别数的代码变成

```
case '.':
case '0': case '1': case '2': case '3': case '4':
case '5': case '6': case '7': case '8': case '9':
    {    cin.putback(ch);      // 把数字放回输入流
         double val;
         cin >> val;           // 读入一个浮点数
         return Token(number,val);
    }
```

我们本可以给所有 token 都起符号名，但那似乎小题大做。毕竟，对 (和 + 来说，'(' 和 '+' 是谁都能想出的最直观的记法。检视各个 token，只有表示“打印”（或者说“终止表达式”）的 ';' 和表示“退出”的 'q' 显得随意。为什么不是 'p' 和 'e' 呢？在更大的程序里，这种晦涩随意的记法迟早会惹出麻烦，所以我们引入

```
const char quit = 'q';        // t.kind==quit 表示 t 是一个 quit Token
const char print = ';';       // t.kind==print 表示 t 是一个 print Token
```

现在我们可以把 main() 的循环写成这样：

```
while (cin) {
    cout << "> ";
    Token t = ts.get();
    while (t.kind == print) t=ts.get();
    if (t.kind == quit) {
        keep_window_open();
        return 0;
    }
    ts.putback(t);
    cout << "= " << expression() << '\n';
}
```

为“打印”和“退出”引入符号名让代码更易读。此外，它也不会诱使阅读 main() 的人对“打印”和“退出”在输入中如何表示做出假设。例如，如果我们决定把“退出”的表示改为 'e'（表示“exit”），那也不足为奇。而现在，这无需对 main() 做任何修改。

现在，字符串 "> " 和 "= " 显得突兀了。代码里为什么会有这些“神奇”的字面值？阅读 main() 的新程序员怎么能猜到它们的

用途？也许该加一条注释？加注释可能是个好主意，但引入一个符号名更有效：

```
const string prompt = "> ";
const string result = "= ";     // 用于指示其后是一个结果
```

如果将来想改变提示符或结果指示符，只需修改这几个 const 即可。循环现在变成

```
while (cin) {
    cout << prompt;
    Token t = ts.get();
    while (t.kind ==print) t=ts.get();
    if (t.kind == quit) {
        keep_window_open();
        return 0;
    }
    ts.putback(t);
    cout << result << expression() << '\n';
}
```

7.6.2 函数的使用

我们所用的函数应当反映程序的结构，函数名应当标识出代码中逻辑上相互分离的部分。总体而言，到目前为止我们的程序在这方面做得相当好：expression()、term() 和 primary() 直接反映了我们对表达式文法的理解，get() 则处理输入和记号识别。不过，看看 main()，我们注意到它做了两件逻辑上相互分离的事情：

1. main() 提供通用的“脚手架”：启动程序、结束程序，并处理“致命”错误。
2. main() 处理计算循环。

理想情况下，一个函数只执行一个逻辑动作（§4.5.1）。让 main() 同时执行这两个动作模糊了程序的结构。显而易见的解决办法是把计算循环变成一个单独的函数 calculate()：

```
void calculate()          // 表达式求值循环
{
    while (cin) {
        cout << prompt;
        Token t = ts.get();
        while (t.kind == print) t=ts.get();      // 首先丢弃所有“打印”
        if (t.kind == quit) return;
        ts.putback(t);
        cout << result << expression() << '\n';
    }
}


int main()
try {
    calculate();
    keep_window_open();      // 应付 Windows 控制台模式
    return 0;
}
catch (runtime_error& e) {
    cerr << e.what() << '\n';
    keep_window_open("~~");
    return 1;
}
catch (...) {
    cerr << "exception \n";
    keep_window_open("~~");
    return 2;
}
```

这更直接地反映了程序结构，因而更容易理解。

7.6.3　代码布局

在代码中搜罗难看的代码时，我们发现了

```
switch (ch) {
case 'q': case ';': case '%': case '(': case ')': case '+': case'-': case '*': case '/':
    return Token{ch};          // 让每个字符表示它自身
```

在加入 `'q'`、`';'` 和 `'%'` 之前，这还不算太糟，但现在开始变得晦涩了。难以阅读的代码正是 bug 更容易藏身之处。没错，这里就潜伏着一个潜在的 bug！每个 case 占一行，再加上一两条注释，会有帮助。于是，`Token_stream` 的 `get()` 变成

```
Token Token_stream::get()
    // 从 cin 读取字符并组装一个 Token
{
    if (full) {        // 检查是否已有一个就绪的 Token
        full = false;
        return buffer;
    }
    char ch;
    cin >> ch;         // 注意 >> 会跳过空白（空格、换行、制表符等）


    switch (ch) {
    case quit:
    case print:
    case '(':
    case ')':
    case '+':
    case '-':
    case '*':
    case '/':
    case '%':
        return Token{ch};       // 让每个字符表示它自身
    case '.':                   // 浮点字面值可以以点开始
    case '0': case '1': case '2': case '3': case '4':
    case '5': case '6': case '7': case '8': case '9':    // 数值字面值
    {    cin.putback(ch);       // 把数字放回输入流
        double val;
        cin >> val;             // 读入一个浮点数
        return Token{number,val};
    }
    default:
        error("Bad token");
    }
}
```

当然，我们也可以把每个数字 case 单独放一行，但那似乎并没有带来什么清晰性。而且，那样做会让 `get()` 无法在屏幕上完整地一览无余。我们的理想是每个函数都能在一屏内放下；bug 最明显的藏身之处，就是那些因超出屏幕（无论是横向还是纵向）而看不见的代码。代码布局很重要。

还要注意，我们把朴素的 `'q'` 改成了符号名 `quit`。这既提高了可读性，也保证了：如果我们误选了一个与其他 token 名冲突的 `quit` 值，就会得到一个编译时错误。

清理代码时，我们可能会不小心引入错误。清理之后务必重新测试程序。更好的做法是，每完成一小批改进就做一点测试，这样即使出了问题，你还能清楚记得自己刚做了什么。记住：尽早测试，勤于测试。

7.6.4 注释

我们在编写程序的过程中加了一些注释。好的注释是写代码的重要组成部分。编程正酣时，我们往往顾不上注释。而当你回过头来清理代码时，正是检查程序各部分的绝佳时机，看看你最初写的注释

1. 是否仍然成立（写注释后你可能已经改了代码）；
2. 对读者来说是否足够（通常并不够）；
3. 是否冗长到喧宾夺主、盖过了代码。

要强调最后一条：能用代码说清楚的，就应该用代码说。避免那种对懂这门编程语言的人来说一目了然的注释。例如：

```
x = b+c;        // 把 b 和 c 相加并把结果赋给 x
```

本书中你会见到这类注释，但那只出现在我们试图向你讲解某种你可能还不熟悉的语言特性的时候。

注释是留给代码难以表达的东西的。一个例子就是意图：代码说明它做什么，却说明不了它本意要做什么（§5.9.1）。看看计算器的代码，就缺了点什么：这些函数展示了我们如何处理表达式和 token，但对于我们本想把表达式和 token 定义成什么（除了代码本身之外）却毫无交代。文法正是值得写进注释或计算器文档的好材料。

```
/*
    简单计算器

    修订历史:
        Bjarne Stroustrup 于 2013 年 11 月修订
        Bjarne Stroustrup 于 2007 年 5 月修订
        Bjarne Stroustrup 于 2006 年 8 月修订
        Bjarne Stroustrup 于 2004 年 8 月修订
        最初由 Bjarne Stroustrup 编写
            (bs@cs.tamu.edu) 2004 年春。

    本程序实现一个基本的表达式计算器。
    输入来自 cin; 输出到 cout。
    输入文法如下:

        Statement:
                Expression
                Print
                Quit

        Print:
                ;

        Quit:
                q
```

```
        Expression:
                Term
                Expression + Term
                Expression - Term
        Term:
                Primary
                Term * Primary
                Term / Primary
                Term % Primary
        Primary:
                Number
                ( Expression )
                - Primary
                + Primary
        Number:
                floating-point-literal

    输入通过名为 ts 的 Token_stream 进入。
*/
```

这里我们用的是块注释（block comment），它以 `/*` 开始，直到 `*/` 结束。在真实的程序中，修订历史里还会记载做了哪些更正和改进。

注意，注释并不是代码。事实上，这条文法做了一点简化：把 **Statement** 规则与实际发生的情况对照一下（例如，看一眼下一节中的代码）。这条注释没有解释 `calculate()` 中的那个循环——正是它让我们能在程序的一次运行中完成多次计算。我们将在 §7.8.1 回到这个问题。

7.7 从错误中恢复

为什么我们一发现错误就退出？这在当时看起来既简单又显然，但究竟是为什么？难道我们不能写一条错误信息然后继续吗？毕竟，我们常常犯些小小的输入错误，而这样的错误并不意味着我们决定放弃计算。所以，让我们试着从错误中恢复。这基本上意味着，我们必须捕获异常，并在清理完残留的混乱之后继续执行。

到目前为止，所有错误都以异常的形式表示，并由 `main()` 处理。如果我们想从错误中恢复，`calculate()` 就必须捕获异常，并设法在求值下一个表达式之前清理混乱：

```
void calculate()
{
    while (cin)
    try {
        cout << prompt;
        Token t = ts.get();
        while (t.kind == print) t=ts.get();    // 先丢弃所有“打印”
        if (t.kind == quit) return;
        ts.putback(t);
        cout << result << expression() << '\n';
    }
    catch (exception& e) {
        cerr << e.what() << '\n';              // 写出错误信息
        clean_up_mess();
    }
}
```

我们只是把 while 循环的语句块变成了一个 **try** 语句块，它会写出错误信息并清理混乱。一旦做完这些，就照常继续。

“清理混乱”究竟意味着什么？基本上，在错误得到处理之后准备好再次计算，意味着要确保我们的所有数据都处于良好且可预测的状态。在计算器中，我们在各个函数之外唯一保存的数据就是 `Token_stream`。因此我们需要做的，是确保不会让与被中止计算相关的 token 残留在那里，干扰下一次计算。例如，

```
1++2*3; 4+5;
```

会导致一个错误，并且在第二个 + 触发异常之后，**2*3; 4+5** 会残留在 `Token_stream` 和 **cin** 的缓冲区中。我们有两个选择：

1. 清空 `Token_stream` 中的所有 token。
2. 只从 `Token_stream` 中清除属于当前计算的 token。

第一种选择会丢弃全部内容（包括 **4+5;**），而第二种选择只丢弃 **2*3;**，留下 **4+5** 待求值。两种都可能是合理的选择，也都可能让用户感到意外。碰巧，两者实现起来难度差不多。我们选择了第二种，因为它便于测试。

因此，我们需要读取输入直到遇见一个分号。这看起来很简单。我们有 `get()` 替我们做读取工作，所以可以这样写一个 `clean_up_mess()`：

```
void clean_up_mess()            // 天真版
{
    while (true) {              // 跳过，直到找到一个 print
        Token t = ts.get();
        if (t.kind == print) return;
    }
}
```

遗憾的是，它并不那么奏效。为什么？考虑这样的输入：

```
1@z; 1+3;
```

@ 使我们进入 while 循环的 catch 子句。接着，我们调用 clean_up_mess() 去寻找下一个分号。然后，clean_up_mess() 调用 get() 并读入 z。这又引发一个错误（因为 z 不是一个 token），于是我们发现自己落入了 main() 的 catch(...) 处理程序，程序就退出了。糟糕！我们没机会求值 1+3 了。回到绘图板重新设计吧！

我们可以尝试更精巧的 try 和 catch，但基本上我们是在走向更大的混乱。错误本来就难以处理，而错误处理过程中的错误比其他错误更糟。所以，让我们设法设计一种能从 Token_stream 中清除字符、却绝不可能抛出异常的方法。把输入送进计算器的唯一途径是 get()，而它——正如我们刚刚惨痛发现的——会抛出异常。因此我们需要一个新操作。显而易见的位置是把它放进 Token_stream：

```
class Token_stream {
public:
    Token get();                    // 读取一个 Token
    void putback(Token t);          // 放回一个 Token
    void ignore(char c);            // 丢弃字符，直到并包含 c
private:
    bool full {false};              // 缓冲区中是否有 Token？
    Token buffer;                   // 这里存放 putback() 放回的 Token
};
```

这个 ignore() 函数必须是 Token_stream 的成员，因为它需要查看 Token_stream 的缓冲区。我们选择把“要寻找的东西”作为 ignore() 的实参——毕竟，Token_stream 并不必知道计算器认为什么字符适合用于错误恢复。我们决定这个实参是一个字符，因为我们不想冒险拼合 Token——我们已经领教过那么做的后果。于是我们得到

```
void Token_stream::ignore(char c)
        // c 表示 Token 的种类
{
    // 先查看缓冲区：
    if (full && c==buffer.kind) {
        full = false;
        return;
    }
    full = false;

    // 再搜索输入：
    char ch = 0;
    while (cin>>ch)
        if (ch==c) return;
}
```

这段代码先查看缓冲区。如果那里有一个 c，丢弃这个 c 之后我们就完事了；否则，需要从 cin 读取字符，直到找到一个 c。

现在我们可以相当简单地写出 clean_up_mess()：

```
void clean_up_mess()
{
    ts.ignore(print);
}
```

处理错误总是很棘手。它需要大量的试验和测试，因为极难想象会发生什么错误。试图让程序万无一失从来都是一项非常技术性的工作；业余者通常并不在意。高质量的错误处理是专业水准的标志之一。

7.8 变量

在完成了风格和错误处理方面的工作之后，我们可以回过头来寻找改进计算器功能的途径。现在我们已经有了一个工作得相当不错的程序；怎样改进它呢？计算器的第一份愿望清单里就有变量。有了变量，我们就能用更好的方式表达较长的计算。类似地，对于科学计算，我们希望有内置的命名值，比如 **pi** 和 **e**，就像科学计算器上那样。

添加变量和常量是对计算器的一次重大扩展。它会触及代码的绝大部分。这种扩展，如果没有充分的理由和充足的时间，就不应该着手进行。在这里，我们添加变量和常量，是因为它给了我们一个重新审视代码、并试验更多编程技术的机会。

7.8.1 变量与定义

显然，变量和内置常量的关键都在于让计算器程序保存 *(name,value)* 对，从而可以由名字访问值。我们可以这样定义一个 **Variable**：

```
class Variable {
public:
    string name;
    double value;
};
```

我们将用 `name` 成员来标识一个 **Variable**，用 `value` 成员保存与该名字对应的值。

我们如何存储 **Variable**，才能按给定的 `name` 字符串搜索 **Variable**、找到它的值或赋予它新值？回顾一下到目前为止遇到的编程工具，我们发现只有一个好答案：一个由 **Variable** 组成的 `vector`：

```
vector<Variable> var_table;
```

我们可以把任意多个 **Variable** 放进 vector `var_table`，并逐个查看 vector 元素来搜索给定名字。我们可以写一个 `get_value()` 函数，查找给定的 `name` 字符串并返回其对应的 `value`：

```
double get_value(string s)
        // 返回名为 s 的 Variable 的值
{
    for (const Variable& v : var_table)
        if (v.name == s) return v.value;
    error("get: undefined variable ", s);
}
```

这段代码确实非常简单：遍历 var_table 中的每个 **Variable**（从第一个元素开始，直到最后一个），看它的 name 是否与实参字符串 **s** 匹配。如果匹配，就返回它的 value。

类似地，我们可以定义一个 **set_value()** 函数，给一个 **Variable** 赋予新的 value：

```
void set_value(string s, double d)
        // 把名为 s 的 Variable 设为 d
{
    for (Variable& v : var_table)
        if (v.name == s) {
            v.value = d;
            return;
        }
    error("set: undefined variable ", s);
}
```

现在我们可以读写 var_table 中以 **Variable** 表示的“变量”了。那么，怎样把一个新的 **Variable** 放入 var_table？我们计算器的用户必须写什么，才能定义一个新变量、之后又取到它的值？我们可以考虑 C++ 的记法

```
double var = 7.2;
```

这样可行，但这个计算器中所有变量保存的都是 double 值，所以说“double”就是多余的了。我们能否将就用

```
var = 7.2;
```

也许可以，但这样一来，我们就无法区分“声明一个新变量”和“拼写错误”了：

```
var1 = 7.2;      // 定义一个名为 var1 的新变量
var1 = 3.2;      // 定义一个名为 var2 的新变量
```

糟糕！显然，我们本意是 **var2 = 3.2;**，但我们没有说出来（除了在注释里）。我们本可以这样凑合，但我们要遵循 C++ 等语言的传统：把（带初始化的）声明与赋值区分开。我们可以用 double，但对计算器来说，我们想要个短一些的东西，所以——借鉴另一项古老传统——我们选择关键字 **let**：

```
let var = 7.2;
```

文法将是

```
Calculation:
    Statement
    Print
    Quit
    Calculation Statement

Statement:
    Declaration
    Expression

Declaration:
    "let" Name "=" Expression
```

Calculation 是文法新的顶层产生式（规则）。它表示一个循环（在 `calculate()` 中），使我们能在计算器程序的一次运行中进行多次计算。它依靠 **Statement** 产生式来处理表达式和声明。我们可以这样处理一条语句：

```
double statement()
{
    Token t = ts.get();
    switch (t.kind) {
    case let:
        return declaration();
    default:
        ts.putback(t);
        return expression();
    }
}
```

现在我们可以在 `calculate()` 中用 `statement()` 取代 `expression()` 了：

```
void calculate()
{
    while (cin)
    try {
        cout << prompt;
        Token t = ts.get();
        while (t.kind == print) t=ts.get();    // 先丢弃所有“打印”
        if (t.kind == quit) return;            // 退出
        ts.putback(t);
        cout << result << statement() << '\n';
    }
    catch (exception& e) {
        cerr << e.what() << '\n';              // 写出错误信息
        clean_up_mess();
    }
}
```

接下来我们必须编写 `declaration()`。它应该做什么？它应确保 **let** 之后是一个 **Name**，随后是一个 `=`，再后是一个 **Expression**。我们的文法就是这么说的。对 `name` 应该怎么处理？我们应该把一个带有该名字字符串、值为表达式值的 **Variable** 加入名为 `var_table` 的 `vector<Variable>`。做完之后，就可以用 `get_value()` 取值、用 `set_value()` 改值了。不过，在编写之前，必须先决定：如果一个变量被定义两次，应该发生什么？例如：

```
let v1 = 7;
let v1 = 8;
```

我们选择把这种重定义视为错误。通常，它只是个拼写错误。我们写的并不是想要的，想要的很可能是

```
let v1 = 7;
let v2 = 8;
```

定义一个名字为 **var**、值为 **val** 的 **Variable**，在逻辑上有两个部分：

1. 检查 `var_table` 中是否已有名为 **var** 的 **Variable**。
2. 把 *(var,val)* 加入 `var_table`。

我们用不上未初始化的变量。我们定义了函数 `is_declared()` 和 `define_name()` 来表示这两个逻辑上独立的操作：

```
bool is_declared(string var)
        // var 是否已在 var_table 中？
{
    for (const Variable& v : var_table)
        if (v.name == var) return true;
    return false;
}


double define_name(string var, double val)
        // 把 (var,val) 加入 var_table
{
    if (is_declared(var)) error(var," declared twice");
    var_table.push_back(Variable(var,val));
    return val;
}
```

向 `vector<Variable>` 添加一个新的 **Variable** 很容易；这正是 `vector` 的成员函数 `push_back()` 的职责：

```
var_table.push_back(Variable(var,val));
```

`Variable(var,val)` 构造出相应的 **Variable**，而 `push_back()` 随后把这个 **Variable** 加到 `var_table` 的末尾。有了这些，再假定我们能处理 **let** 和 `name` token，`declaration()` 就很好写了：

```
double declaration()
        // 假定我们已经见到了 "let"
        // 处理：name = expression
        // 声明一个名为 "name" 的变量，初始值为 "expression"
{
    Token t = ts.get();
    if (t.kind != name) error ("name expected in declaration");
    string var_name = t.name;

    Token t2 = ts.get();
    if (t2.kind != '=') error("= missing in declaration of ", var_name);

    double d = expression();
    define_name(var_name,d);
    return d;
}
```

注意，我们返回了保存在新变量中的值。当初始化表达式不平凡时，这很有用。例如：

```
let v = d/(t2-t1);
```

这条声明会定义 **v**，还会打印它的值。此外，打印被声明变量的值简化了 **calculate()** 中的代码，因为每条 **statement()** 都返回一个值。一般性的规则往往使代码保持简单，而特殊情况往往导致复杂化。

这种保存 **Variable** 的机制，就是常说的符号表（symbol table）；使用标准库的 **map** 可以把它大幅简化，参见 §21.6.1。

7.8.2　引入名字

这些都很好，但遗憾的是，它还跑不起来。到这个时候，这应该不足为奇了。我们的第一版从来就——嗯，几乎从来不——不能直接工作。这里，我们甚至还没把程序写完——它还编译不过。我们没有 '=' token，不过这好办，给 **Token_stream::get()**（§7.6.3）加一个 case 就行。但是，该如何把 **let** 和 name 表示为 token？显然，我们需要修改 **get()**，让它能识别这些 token。怎么改？下面是一种办法：

```
const char name = 'a';              // 名字 token
const char let = 'L';               // 声明 token
const string declkey = "let";       // 声明关键字
```

```
Token Token_stream::get()
{
    if (full) {
        full = false;
        return buffer;
    }
    char ch;
    cin >> ch;
    switch (ch) {
        // 同前
    default:
        if (isalpha(ch)) {
            cin.putback(ch);
            string s;
            cin>>s;
            if (s == declkey) return Token(let);    // 声明关键字
            return Token{name,s};
        }
        error("Bad token");
    }
}
```

首先注意 `isalpha(ch)` 调用。它回答“**ch** 是字母吗?”这个问题；`isalpha()` 是标准库的一部分，我们通过 `std_lib_facilities.h` 获得它。更多字符分类函数参见 §11.6。识别名字的逻辑与识别数的逻辑相同：先找到一个种类正确的字符（这里是字母），然后用 `putback()` 把它放回，再用 `>>` 读入整个名字。

遗憾的是，这编译不过；我们还没有能保存 `string` 的 **Token**，因此编译器拒绝 `Token{name,s}`。要解决这个问题，必须修改 **Token** 的定义，让它既能保存 `string` 又能保存 `double`，并处理三种初始化形式，例如

- 仅一个 `kind`；例如 `Token{'*'}`
- 一个 `kind` 和一个数；例如 `Token{number,4.321}`
- 一个 `kind` 和一个 `name`；例如 `Token{name,"pi"}`

为此我们引入三个初始化函数——之所以称为构造函数（constructor），是因为它们构造对象：

```
class Token {
public:
    char kind;
    double value;
    string name;
    Token(char ch) :kind{ch} { }                          // 用 ch 初始化 kind
    Token(char ch, double val) :kind{ch}, value{val} { }  // 用 ch 初始化
                                                          // kind 与 value
    Token(char ch, string n) :kind{ch}, name{n} { }       // 用 ch 初始化
                                                          // kind 与 name
};
```

构造函数为初始化增添了一层重要的控制和灵活性。我们将在第 9 章（§9.4.2、§9.7）详细考察构造函数。

我们选择 `'L'` 作为 **let** token 的表示，把字符串 **let** 作为关键字。显然，只要改一改用来与 **s** 比较的字符串 `declkey`，就能把这个关键字轻而易举地改成 `double`、**var**、**#** 或别的什么。

现在再试一次程序。如果键入这些，你会看到一切正常：

```
let x = 3.4;
let y = 2;
x + y * 2;
```

然而，这样就不管用了：

```
let x = 3.4;
let y = 2;
x+y*2;
```

这两个例子有什么区别？看一看会发生什么。

问题在于我们对 **Name** 的定义太草率了。我们甚至“忘了”在文法（§7.8.1）中定义 **Name** 产生式。哪些字符可以成为名字的一部分？字母？当然可以。数字？当然可以，只要不是开头字符。下划线呢？呃……+ 字符呢？嗯？呃……再看看代码。读到首字母之后，我们用 `>>` 读入一个 `string`。它会接受一切字符，直到遇见空白。于是，例如，`x+y*2;` 就是一个单独的名字——连末尾的分号也被当作名字的一部分读进来了。这绝非本意，也不能接受。

那该怎么办？首先必须精确规定我们想要的名字是什么样的，然后修改 `get()` 去实现它。下面是一份可行的名字规范：以字母开头的字母和数字序列。按这个定义，

```
a
ab
a1
Z12
asdsddsfdfdasfdsa434RTHTD12345dfdsa8fsd888fadsf
```

是名字，而

```
1a
as_s
#
as*
a car
```

不是。除了没有纳入下划线，这正是 C++ 的规则。我们可以在 `get()` 的 `default` 分支中实现它：

```
default:
        if (isalpha(ch)) {
            string s;
            s += ch;
            while (cin.get(ch) && (isalpha(ch) || isdigit(ch))) s+=ch;
            cin.putback(ch);
            if (s == declkey) return Token{let};    // 声明关键字
            return Token{name,s};
        }
        error("Bad token");
```

我们不再直接读入 `string` **s**，而是读取字符，只要它们是字母或数字就放进 **s**。语句 **s+=ch** 把字符 **ch** 追加（append）到字符串 **s** 的末尾。这条看起来有些古怪的语句

```
while (cin.get(ch) && (isalpha(ch) || isdigit(ch))) s+=ch;
```

把一个字符读入 **ch**（用 **cin** 的成员函数 **get()**），并检查它是不是字母或数字。如果是，就把 **ch** 加进 **s**，再继续读。成员函数 **get()** 的工作方式与 **>>** 一样，只是默认不跳过空白。

7.8.3　预定义名字

有了名字，我们就能轻松地预定义几个常用的名字。例如，假设计算器将用于科学计算，我们会希望有 **pi** 和 **e**。在代码的哪里定义它们呢？在 **main()** 中调用 **calculate()** 之前，或在 **calculate()** 中循环之前。我们把它们放在 **main()** 里，因为这些定义真的不属于任何计算：

```
int main()
try {
    // 预定义名字:
    define_name("pi",3.1415926535);
    define_name("e",2.7182818284);

    calculate();

    keep_window_open();          // 应付 Windows 控制台模式
    return 0;
}
catch (exception& e) {
    cerr << e.what() << '\n';
    keep_window_open("~~");
    return 1;
}
catch (...) {
    cerr << "exception \n";
    keep_window_open("~~");
    return 2;
}
```

7.8.4 我们完成了吗？

还没有。我们做了太多修改，需要把所有东西再测一遍，清理代码，并检查注释。此外，还可以做更多定义。例如，我们“忘了”提供赋值运算符（见习题 2），而且如果有赋值，我们可能还想区分变量和常量（习题 3）。

起初，我们放弃了在计算器中引入命名变量的想法。回看实现它们的代码，我们可能有两种反应：

1. 实现变量没那么糟；只花了大约三打行代码。
2. 实现变量是一次重大扩展。它几乎触及每一个函数，为计算器增加了一个全新的概念。它使计算器的规模增大了 45%，而我们甚至还没实现赋值！

以第一个具有相当复杂度的程序为背景来看，第二种反应才是正确的。更一般地说，对任何会使程序的规模和复杂度都增加约 50% 的建议，这种反应都是正确的。当不得不这样做时，这更像是基于一个旧程序编写一个新程序，也应当以这种方式对待。特别是，如果你能像我们对计算器所做的那样分阶段构建程序，并在每个阶段都进行测试，你就会比试图一次写出整个程序好得多。

小试身手

1. 从文件 `calculator08buggy.cpp` 出发，让计算器通过编译。
2. 通读整个程序，加上恰当的注释。
3. 在加注释的过程中，你会发现一些错误（特意狡猾地埋进去让你找的）。修好它们；这些错误并不在书正文里。
4. 测试：准备一组输入，用它们测试计算器。你的清单相当完整吗？应该找些什么？要包括负值、0、非常小、非常大以及“傻”输入。
5. 执行测试，修掉加注释时漏掉的 bug。
6. 添加一个预定义名字 **k**，表示 **1000**。
7. 给用户提供一个平方根函数 `sqrt()`，例如 `sqrt(2+6.7)`。当然，`sqrt(x)` 的值是 **x** 的平方根；例如 `sqrt(9)` 是 **3**。使用标准库中通过头文件 `std_lib_facilities.h` 提供的 `sqrt()` 函数。记得更新注释，包括文法。
8. 捕获对负数求平方根的企图，并打印恰当的错误信息。
9. 允许用户用 `pow(x,i)` 表示“将 **x** 自乘 **i** 次”；例如 `pow(2.5,3)` 是 `2.5*2.5*2.5`。要求 **i** 是整数，采用我们对 `%` 使用的技术。
10. 把“声明关键字”从 **let** 改为 **#**。
11. 把“退出关键字”从 `quit` 改为 `exit`。这需要像 §7.8.2 中为 **let** 定义字符串那样，为 `quit` 定义一个字符串。

要点回顾

1. 在第一版程序能工作之后还继续改进它，目的是什么？列出理由。
2. 为什么向计算器键入 **1+2; q** 后，程序在收到错误之后并不退出？
3. 为什么我们选择定义一个名为 `number` 的常量字符？
4. 我们把 `main()` 拆成了两个独立的函数。新函数做什么？我们为什么拆分 `main()`？
5. 我们为什么把代码拆成多个函数？请陈述原则。
6. 写注释的目的是什么？应该怎么写？
7. `narrow_cast` 是做什么的？
8. 符号常量有什么用？
9. 我们为什么在意代码排版？

10. 我们如何处理浮点数的 `%`（取余）？
11. `is_declared()` 做什么？它是如何工作的？
12. **let** 的输入表示不止一个字符。修改后的代码如何把它接受为单个 token？
13. 在计算器程序中，名字可以是什么、不可以是什么，规则有哪些？
14. 为什么渐进地构建程序是个好主意？
15. 你什么时候开始测试？
16. 你什么时候重新测试？
17. 你如何决定什么应当是独立的函数？
18. 你如何为变量和函数选择名字？列出可能的理由。
19. 你为什么添加注释？
20. 什么应当写进注释，什么不应当？
21. 我们何时认为一个程序完成了？

术语

代码排版　维护　脚手架
注释　恢复　符号常量
错误处理　修订历史　测试
功能蔓延

习题

1. 允许计算器的变量名中使用下划线。
2. 提供赋值运算符 `=`，使你可以先用 **let** 引入一个变量、之后再改变它的值。讨论一下这为什么有用，以及它为什么会成为问题之源。
3. 提供真正无法改变值的命名常量。提示：你必须给 **Variable** 添加一个成员，用以区分常量和变量，并在 `set_value()` 中检查它。如果你想让用户也能定义常量（而不只是把 **pi** 和 **e** 定义为常量），还得增加一种记法让用户表达这一点，例如 `const pi = 3.14;`。
4. `get_value()`、`set_value()`、`is_declared()` 和 `define_name()` 函数都在变量 `var_table` 上操作。定义一个名为 `Symbol_table` 的类，它有一个类型为 `vector<Variable>` 的成员 `var_table`，以及成员函数 `get()`、`set()`、`is_declared()` 和 `declare()`。重写计算器，改用一个 `Symbol_table` 类型的变量。
5. 修改 `Token_stream::get()`，使它在见到换行时返回 `Token(print)`。这意味着要查找空白字符，并对换行（`'\n'`）特殊处理。标准库函数 `isspace(ch)` 在 **ch** 是空白字符时返回 `true`，你可能会发现它很有用。
6. 每个程序都应当做的一件事，是为用户提供某种帮助。让计算器在用户按下 **H** 键（大小写均可）时打印一些如何使用计算器的说明。
7. 把 **q** 和 **h** 命令分别改为 `quit` 和 `help`。
8. §7.6.4 中的文法是不完整的（我们确实警告过你不要过分依赖注释）；它没有定义语句序列，例如 `4+4; 5-6;`，也没有纳入 §7.8 概述的文法改动。修正那个文法。另外，把你认为需要的内容加进那条注释，作为计算器程序的第一条注释和总体注释。
9. 为计算器提出三项改进（本章未提到的）。实现其中一项。
10. 修改计算器，使其只对 **int** 运算；对溢出和下溢给出错误。提示：使用 `narrow_cast`（§7.5）。

11. 重温你在第 4 章或第 5 章习题中写的两个程序。按照本章概述的规则清理那些代码。看看在此过程中是否发现什么 bug。

后记

碰巧，我们已经见识了编译器工作方式的一个简单例子。计算器把输入分析分解为 token，并依据文法来理解它。这正是编译器所做的。在分析输入之后，编译器会生成一种表示（目标代码），供我们以后执行。计算器则立即执行它已分析的表达式；这样做的程序称为解释器（interpreter），而不是编译器。

8 技术细节：函数等

“再多的天才也敌不过对细节的痴迷。”

——Traditional

在本章和下一章中，我们把焦点从编程转移到我们的编程主工具上：C++ 编程语言。我们将介绍语言的技术细节，以便让你对 C++ 的基本设施有一个稍微更宽的视野，并对这些设施给出一个更系统的概观。这两章还充当对迄今介绍的许多编程概念的复习，并提供一个在不引入新编程技术或概念的情况下探索我们工具的机会。

8.1 技术细节
8.2 声明与定义
8.2.1 声明的种类
8.2.2 变量与常量声明
8.2.3 默认初始化
8.3 头文件
8.4 作用域
8.5 函数调用与返回
8.5.1 声明实参与返回类型
8.5.2 返回一个值
8.5.3 传值
8.5.4 传 const 引用
8.5.5 传引用
8.5.6 传值与传引用之比较
8.5.7 实参检查与转换
8.5.8 函数调用的实现
8.5.9 constexpr 函数
8.6 求值顺序
8.6.1 表达式求值
8.6.2 全局初始化
8.7 命名空间
8.7.1 using 声明与 using 指令

8.1 技术细节

如果可以选择，我们更愿意谈论编程，而不是编程语言的特性；也就是说，我们认为“如何把思想表达为代码”远比“用来表达这些思想的编程语言的技术细节”有趣。打个自然语言中的比方：我们更愿意讨论一本好小说中的思想以及表达这些思想的方式，而不是去研究英语的语法和词汇。重要的是思想以及如何在代码中表达这些思想，而不是一条条的语言特性。

然而，我们并非总有得选。当你开始编程时，编程语言对你来说是一门外语，你需要去看看它的“语法和词汇”。这正是本章和下一章要做的事，但请不要忘记：

- 我们的首要研究对象是编程。
- 我们的产出是程序/系统。
- 编程语言（只是）一个工具。

把这一点记在心里似乎难得出奇。许多程序员会满怀热情地关注语言语法和语义中那些表面上很细微的细节。特别是，太多人错误地相信，他们的第一门编程语言中做事的方式就是“唯一正确的方式”。请不要掉进这个陷阱。C++ 在很多方面是一门非常好的语言，但它并不完美；其他任何编程语言也都一样。

大多数设计和编程概念都是通用的，而且其中许多概念都得到了主流编程语言的广泛支持。这意味着，我们在一门好的编程课程中学到的基本思想和技术可以在语言之间迁移，能够以不同的难易程度应用于所有语言。然而，语言的技术细节则是特定于某一门语言的。幸运的是，编程语言并非在真空

中发展，因此你在这里学到的大部分内容，在其他语言中都会有相当明显的对应物。特别是，C++ 属于一个语言家族，其中还包括 C（第 27 章）、Java 和 C#，所以有不少技术细节是与这些语言共通的。

请注意，在讨论语言技术问题时，我们有意使用不具描述性的名字，例如 **f**、**g**、**X** 和 **y**。我们这样做是为了强调这类示例的技术性质、让这些示例非常短，并尽量避免把语言技术细节和真正的程序逻辑混在一起使你困惑。当你看到不具描述性的名字（在实际代码中绝不应这样用）时，请把注意力放在代码的语言技术方面。技术性示例通常只包含单纯演示语言规则的代码。如果你编译并运行它们，会得到许多“变量未使用”的警告，而且这类技术性程序片段几乎不会做什么有意义的事。

请注意，我们在这里所写的并不是对 C++ 语法和语义的完整描述——甚至对我们所描述的那些设施来说也不是。ISO C++ 标准是 1300 多页密集的技术性文字，而 Stroustrup 的《The C++ Programming Language》则是 1300 多页面向有经验程序员的文字（两者都同时覆盖 C++ 语言及其标准库）。我们不试图在完整性和全面性上与它们竞争；我们竞争的是可理解性，以及阅读所花时间的价值。

8.2 声明与定义

声明（declaration）是一条把一个名字引入某个作用域（§8.4）的语句，它

- 为所命名的实体（例如一个变量或函数）指定一个类型；
- 可选地，指定一个初始化器（例如一个初始值或函数体）。

例如：

```
int a = 7;                      // 一个 int 变量
const double cd = 8.7;          // 一个双精度浮点常量
double sqrt(double);            // 一个接受 double 实参
                                // 并返回 double 结果的函数
vector<Token> v;                // 一个 Token 的 vector 变量
```

在 C++ 程序中，一个名字在使用之前必须先被声明。考虑：

```
int main()
{
    cout << f(i) << '\n';
}
```

编译器会为此给出至少三个“未声明的标识符”错误：`cout`、**f** 和 **i** 在这个程序片段中都没有被声明。我们可以通过包含头文件 **std_lib_facilities.h** 来让 `cout` 得到声明，它包含了 `cout` 的声明：

```
#include "std_lib_facilities.h"    // 我们在这里找到 cout 的声明

int main()
{
    cout << f(i) << '\n';
}
```

现在，我们只会得到两个“未定义”错误。当你编写真实世界的程序时，你会发现大多数声明都在头文件中。正是在头文件里，我们为“在别处”定义的实用设施定义接口（interface）。基本上，声明规定了某样东西可以如何被使用；它定义了函数、变量或类的接口。请注意这种声明用法一个明显

但看不见的优点：我们不必去查看 cout 和它的 << 运算符是如何定义的细节；我们只是 **#include** 了它们的声明。我们甚至不必看它们的声明；从教科书、手册、代码示例或其他来源，我们就知道 cout 应该怎么用。编译器读取头文件中理解我们代码所需的那些声明。

然而，我们仍然必须声明 **f** 和 **i**。我们可以这样做：

```
#include "std_lib_facilities.h"    // 我们在这里找到 cout 的声明

int f(int);                        // f 的声明

int main()
{
    int i = 7;                     // i 的声明
    cout << f(i) << '\n';
}
```

这可以编译通过，因为每个名字都已被声明，但它无法链接（§2.4），因为我们没有定义 f()；也就是说，我们没有在任何地方说明 f() 到底做什么。

一条（同时）完整指明了被声明实体的声明称为定义（definition）。例如：

```
int a = 7;
vector<double> v;
double sqrt(double d) { /* . . . */ }
```

每个定义（按定义 ☺）也都是声明，但只有一部分声明同时是定义。下面是一些不是定义的声明的例子；如果用到了它们所指称的实体，那么每一条都必须在代码的其他地方有一个与之匹配的定义：

```
double sqrt(double);    // 这里没有函数体
extern int a;           // “extern 加上没有初始化器”意味着“不是定义”
```

当我们对比定义和声明时，按照惯例，我们用“声明”（declarations）一词来表示“不是定义的声明”，尽管这个术语稍有不够严谨。

定义精确地规定了一个名字所指称的是什么。特别地，一个变量的定义为该变量留出了内存。因此，你不能把同一个东西定义两次。例如：

```
double sqrt(double d) { /* . . . */ }   // 定义
double sqrt(double d) { /* . . . */ }   // 错误：重复定义

int a;                                  // 定义
int a;                                  // 错误：重复定义
```

相反，一条不同时是定义的声明只是告诉你如何使用一个名字；它只是一个接口，不分配内存，也不指定函数体。因此，只要保持一致，你可以随意多次声明同一个东西：

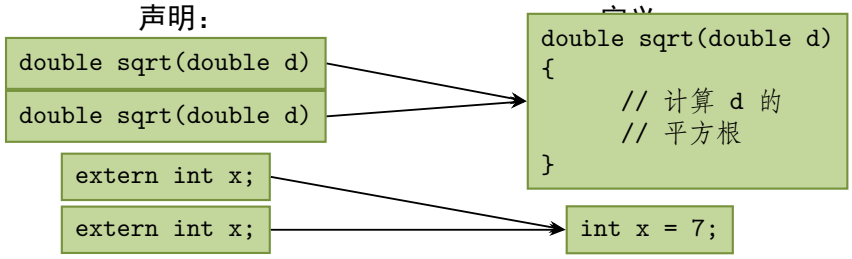
```
int x = 7;          // 定义
extern int x;       // 声明
extern int x;       // 另一条声明
```

```
double sqrt(double);                    // 声明
double sqrt(double d) { /* . . . */ }   // 定义
double sqrt(double);                    // sqrt 的另一条声明
double sqrt(double);                    // sqrt 的又一条声明

int sqrt(double);                       // 错误：sqrt 的声明不一致
```

为什么最后这条声明是错误的？因为不可能存在两个都叫 sqrt、都接受一个 double 类型实参、却返回不同类型（**int** 和 double）的函数。

第二条 **x** 的声明中使用的 extern 关键字只是表明：**x** 的这条声明不是定义。它很少有用。我们建议你不要使用它，但你会在别人的代码中见到它，特别是那些使用了太多全局变量的代码（见 §8.4 和 §8.6.2）。



为什么 C++ 要同时提供声明和定义？声明/定义的区分反映了两个更基本概念之间的区别：我们要使用某样东西所需要的信息（接口），以及让那样东西完成其分内之事所需要的信息（实现，implementation）。对变量而言，声明提供了类型，但只有定义才提供对象（内存）。对函数而言，声明同样提供了类型（实参类型加返回类型），但只有定义才提供函数体（可执行语句）。注意，函数体作为程序的一部分存储在内存中，所以可以公平地说，函数和变量的定义消耗内存，而声明不消耗。

声明/定义的区分使我们能够把一个程序分离成许多可以分别编译的部分。声明让程序的每个部分都能了解程序其余部分的情况，而不必理会其他部分中的定义。由于所有声明（包括那唯一的定义）必须保持一致，整个程序中名字的使用就会是一致的。我们将在 §8.3 进一步讨论这一点。这里，我们只想让你回忆起第 6 章的表达式分析器：**expression()** 调用 **term()**，**term()** 调用 **primary()**，**primary()** 又调用 **expression()**。由于 C++ 程序中每个名字都必须在使用之前声明，我们不可能直接去定义这三个函数：

```
double expression();    // 只是声明，不是定义
```

```
double primary()
{
    // . . .
    expression();
    // . . .
}

double term()
{
    // . . .
    primary();
    // . . .
}

double expression()
{
    // . . .
    term();
    // . . .
}
```

这四个函数我们可以按任意顺序排列；总会有一个调用指向定义在其下方的函数。某处需要一个“前向”声明。因此，我们在 **primary()** 的定义之前声明了 **expression()**，于是一切正常。这种循环调用模式非常常见。

为什么名字必须先声明后使用？难道我们不能让语言的实现去读整个程序（就像我们一样），找到定义来看看函数应该如何被调用吗？我们可以那样做，但这会带来“有趣的”技术难题，所以我们决定不那样做。C++ 的定义要求先声明后使用（类成员除外；见 §9.4.4）。毕竟，这已经是普通（非程序）写作的惯例：当你读一本教科书时，你期望作者在使用术语之前先定义它；否则，你就得不停地猜或者查索引。“先声明后使用”规则为人和编译器都简化了阅读。在程序中，还有第二个原因使“先声明后使用”很重要。在一个数千行（也许数十万行）的程序中，我们想调用的大多数函数都定义在“别处”。那个“别处”往往是我们在真正意义上不想了解的地方。只需知道我们所用之物的声明，就使我们（和编译器）免于在浩如烟海的程序文本中翻找。

8.2.1 声明的种类

程序员可以在 C++ 中定义很多种实体。最有趣的有

- 变量
- 常量
- 函数（见 §8.5）
- 命名空间（namespace，见 §8.7）
- 类型（类和枚举；见第 9 章）
- 模板（template，见第 19 章）

8.2.2 变量与常量声明

变量或常量的声明指定一个名字、一个类型，以及可选的一个初始化器。例如：

```
int a;                      // 没有初始化器
double d = 7;               // 使用 = 语法的初始化器
vector<int> vi(10);         // 使用 ( ) 语法的初始化器
vector<int> vi2 {1,2,3,4};  // 使用 { } 语法的初始化器
```

你可以在 ISO C++ 标准中找到完整的文法。

常量与变量具有相同的声明语法。它们的区别在于类型中带有 const，并且必须有初始化器：

```
const int x = 7;            // 使用 = 语法的初始化器
const int x2 {9};           // 使用 {} 语法的初始化器
const int y;                // 错误：没有初始化器
```

要求 const 必须有初始化器的原因是显而易见的：如果一个 const 没有值，它怎么能成为常量呢？对变量也进行初始化几乎总是个好主意；一个未初始化的变量是晦涩 bug 的祸根。例如：

```
void f(int z)
{
    int x;                  // 未初始化
    // . . . 这里没有对 x 的赋值 . . .
    x = 7;                  // 给 x 一个值
    // . . .
}
```

这看起来相当无害，但如果第一个. . . 中用到了 **x** 呢？例如：

```
void f(int z)
{
    int x;                  // 未初始化
    // . . . 这里没有对 x 的赋值 . . .
    if (z>x) {
        // . . .
    }
    // . . .
    x = 7;                  // 给 x 一个值
    // . . .
}
```

由于 **x** 未初始化，执行 **z>x** 将是未定义行为。比较 **z>x** 在不同机器上可能给出不同结果，在同一台机器上程序的不同运行中也可能给出不同结果。原则上，**z>x** 可能导致程序因硬件错误而终止，但大多数情况下不会发生。相反，我们得到的是不可预测的结果。

自然，我们不会故意做那样的事，但如果我们不能始终如一地初始化变量，它终究会因为失误而发生。记住，大多数“愚蠢的错误”（例如在赋值之前就使用未初始化的变量）都发生在你忙碌或疲惫的时候。编译器会试图发出警告，但在复杂的代码中——恰恰是这类错误最可能出现的地方——编译

器还不够聪明，抓不住所有这类错误。有些人没有初始化变量的习惯，往往是因为他们学的语言不允许或不鼓励一致的初始化；所以你会在别人的代码中见到这样的例子。请别再给这个问题添乱，忘记初始化你自己定义的变量。

我们偏好 {} 初始化器语法。它最通用，也最明确地表达了“初始化器”的含义。除了非常简单的初始化——出于老习惯我们有时会用 =——以及用 ( ) 指定 vector 的元素个数（见 §17.4.4）之外，我们倾向于使用它。

8.2.3 默认初始化

你可能已经注意到，我们常常不为 string、vector 等提供初始化器。例如：

```
vector<string> v;
string s;
while (cin>>s) v.push_back(s);
```

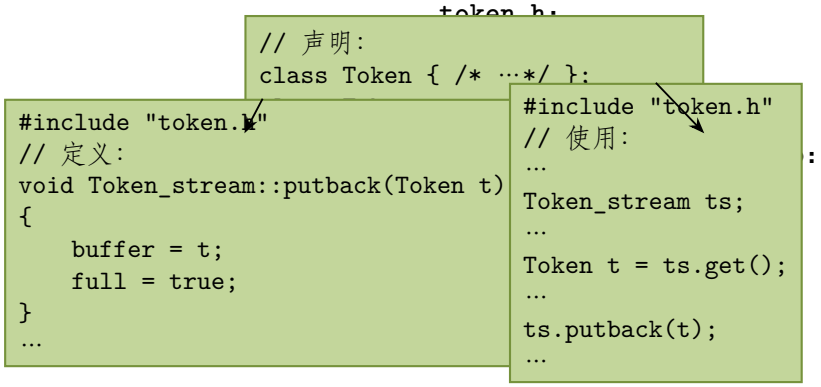
这并不是“变量必须先初始化后使用”规则的例外。这里的情况是：string 和 vector 被定义为：每当我们没有显式提供初始化器时，这些类型的变量就会被一个默认值初始化。于是，在到达循环之前，v 是空的（它没有元素），s 是空字符串 ""。保证默认初始化（default initialization）的机制称为默认构造函数（default constructor）；见 §9.7.3。

遗憾的是，语言不允许我们对内置类型做出这样的保证。全局变量（§8.4）会被默认初始化为 0，但你应该尽量少用全局值。最有用的那些变量——局部变量和类成员——除非你提供初始化器（或默认构造函数），否则是不会被初始化的。已经提醒过你了！

8.3 头文件

我们如何管理自己的声明和定义？毕竟，它们必须保持一致，而在现实世界的程序中，声明可能数以万计；拥有数十万条声明的程序也不罕见。通常，当我们写一个程序时，我们所用的大多数定义并不是我们自己写的。例如，cout 和 sqrt() 的实现是许多年前由别人写好的。我们只是使用它们。

在 C++ 中，管理“在别处”定义的设施的声明，关键在于头文件（header）。基本上，头文件（header file）是声明的集合，通常定义在一个文件中，因此头文件也被称为头文件。这样的头文件随后被 **#include** 到我们的源文件中。例如，我们可能决定改进计算器（第 6 章和第 7 章）源代码的组织，把 token 管理分离出去。我们可以定义一个头文件 token.h，其中包含使用 **Token** 和 Token_stream 所需的声明：



Token 和 Token_stream 的声明在头文件 **token.h** 中，它们的定义在 **token.cpp** 中。后缀 **.h** 是 C++ 头文件最常见的后缀，后缀 **.cpp** 是 C++ 源文件最常见的后缀。实际上，C++ 语言并不在乎文件后缀，但有些编译器和大多数程序开发环境却坚持要有后缀，所以请给你的源代码使用这个惯例。

原则上，**#include "file.h"** 只是把 **file.h** 中的声明复制到你文件中 **#include** 所在的位置。例如，我们可以写一个头文件 **f.h**：

```
// f.h
int f(int);
```

并在我们的文件 **user.cpp** 中包含它：

```
// user.cpp
#include "f.h"
int g(int i)
{
    return f(i);
}
```

编译 **user.cpp** 时，编译器会执行 **#include** 并编译

```
int f(int);
int g(int i)
{
    return f(i);
}
```

由于 **#include** 在逻辑上发生在编译器所做的任何事情之前，处理 **#include** 是所谓预处理（preprocessing，§A.17）的一部分。

为了便于一致性检查，我们既在使用其声明的源文件中 **#include** 一个头文件，也在为这些声明提供定义的源文件中 **#include** 它。这样，编译器就能尽早发现错误。例如，设想 **Token_stream::putback()** 的实现者犯了错误：

```
Token Token_stream::putback(Token t)
{
    buffer.push_back(t);
    return t;
}
```

这看起来相当无害。幸运的是，编译器抓住了这些错误，因为它看到了（#include 进来的）**Token_stream::putback()** 的声明。将那条声明与我们的定义相比较，编译器发现 **putback()** 不应返回 **Token**，而且 buffer 是一个 **Token**，而不是 **vector<Token>**，所以我们不能使用 **push_back()**。这类错误发生在我们改进代码的时候——做了改动，却没能让整个程序保持一致。

类似地，考虑这些错误：

```
Token t = ts.gett();     // 错误：没有成员 gett
// . . .
ts.putback();            // 错误：缺少实参
```

编译器会立即给出错误；头文件 **token.h** 为它提供了检查所需的全部信息。

我们的头文件 **std_lib_facilities.h** 包含我们所用标准库设施的声明，例如 cout、vector 和 **sqrt()**，还包含几个不属于标准库的简单实用函数，例如 **error()**。在 §12.8 中，我们将展示如何直接使用标准库头文件。

一个头文件通常会被许多源文件包含。这意味着，头文件应当只包含那些可以在多个文件中重复的声明（例如函数声明、类定义和数值常量的定义）。

8.4 作用域

作用域（scope）是程序文本中的一个区域。一个名字在一个作用域中被声明，并从其声明点起直到其被声明的作用域末尾一直有效（处于“作用域内”）。例如：

```
void f()
{
    g();                 // 错误：g() 尚不在作用域内
}

void g()
{
    f();                 // OK：f() 在作用域内
}

void h()
{
    int x = y;           // 错误：y 尚不在作用域内
    int y = x;           // OK：x 在作用域内
    g();                 // OK：g() 在作用域内
}
```

一个作用域中的名字可以在嵌套于其中的作用域内看到。例如，对 **f()** 的调用位于 **g()** 的作用域之内，而 **g()** 又“嵌套”在全局作用域中。全局作用域（global scope）是不嵌套在任何其他作用域中的作用域。“名字必须先声明后使用”的规则仍然成立，所以 **f()** 不能调用 **g()**。

我们用几种作用域来控制名字可以在哪里使用：

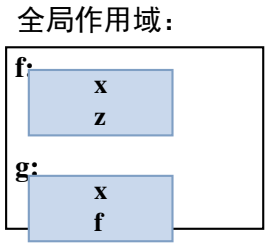
- 全局作用域（global scope）：任何其他作用域之外的文本区域
- 命名空间作用域（namespace scope）：嵌套在全局作用域或另一个命名空间中的命名作用域；见 §8.7
- 类作用域（class scope）：一个类之内的文本区域；见 §9.2
- 局部作用域（local scope）：块的 { . . . } 花括号之间，或函数实参列表中
- 语句作用域（statement scope）：例如在一条 **for** 语句中

作用域的主要目的是让名字保持局部性，使它们不会干扰在别处声明的名字。例如：

```
void f(int x)           // f 是全局的；x 是 f 的局部变量
{
    int z = x+7;        // z 是局部变量
}

int g(int x)            // g 是全局的；x 是 g 的局部变量
{
    int f = x+2;        // f 是局部变量
    return 2*f;
}
```

或者用图来表示：



这里 **f()** 的 **x** 不同于 **g()** 的 **x**。它们不会“冲突”，因为它们不在同一个作用域中：**f()** 的 **x** 是 **f** 的局部变量，而 **g()** 的 **x** 是 **g** 的局部变量。同一作用域中两条不兼容的声明通常被称为一次冲突（clash）。类似地，在 **g()** 中定义和使用的 **f**（显然）不是那个函数 **f()**。

下面是一个逻辑上等价、但更贴近实际的局部作用域用例：

```
int max(int a, int b)   // max 是全局的；a 和 b 是局部变量
{
    return (a>=b) ? a : b;
}


int abs(int a)          // 不是 max() 的 a
{
    return (a<0) ? -a : a;
}
```

在标准库中可以找到 **max()** 和 **abs()**，所以你不必自己写它们。**?:** 结构称为算术 if（arithmetic if）或条件表达式（conditional expression）。**(a>=b)?a:b** 的值在 **a>=b** 时是 **a**，否则是 **b**。条件表达式使我们免于写出这样冗长的代码：

```
int max(int a, int b)   // max 是全局的；a 和 b 是局部变量
{
    int m;              // m 是局部变量
    if (a>=b)
        m = a;
    else
        m = b;
    return m;
}
```

所以，除了值得注意的全局作用域这个例外，作用域让名字保持局部。对大多数目的而言，局部性是好事，所以请让名字尽量局部。当我在函数、类、命名空间等内部声明我的变量、函数等时，它们不会干扰你的。记住：真实的程序有数以千计的命名实体。要让这样的程序保持可管理，大多数名字都必须是局部的。

下面是一个更大的技术性示例，说明名字如何在语句和块（包括函数体）结束时离开作用域：

```
// 这里没有 r、i 或 v
class My_vector {
    vector<int> v;      // v 在类作用域中
public:
    int largest()
    {
        int r = 0;                      // r 是局部变量（最小的非负 int）
        for (int i = 0; i<v.size(); ++i)
            r = max(r,abs(v[i]));       // i 在 for 语句的语句作用域中
        // 这里没有 i
        return r;
    }
    // 这里没有 r
};
// 这里没有 v


int x;                  // 全局变量——尽可能避免
int y;
```

```
int f()
{
    int x;              // 局部变量，隐藏全局的 x
    x = 7;              // 局部的 x
    {
        int x = y;      // 局部 x 用全局 y 初始化，隐藏上一个局部 x
        ++x;            // 上一行的 x
    }
    ++x;                // f() 第一行的 x
    return x;
}
```

只要有可能，就避免这样复杂的嵌套和隐藏。记住：“保持简单!”

名字的作用域越大，它的名字就应该越长、越具描述性：**x**、**y** 和 **f** 作为全局名字是很糟糕的。你不希望在程序中使用全局变量的主要原因，是很难知道哪些函数会修改它们。在大型程序中，基本上不可能知道哪些函数修改了某个全局变量。设想你正在调试一个程序，发现某个全局变量的值出乎意料。是谁给了它那个值？为什么？哪些函数会向那个值写入？你怎么知道？向那个变量写入坏值的函数，可能位于你从未见过的源文件中！一个好的程序只会拥有极少数（比如一两个）全局变量，甚至一个都没有。例如，第 6 章和第 7 章的计算器有两个全局变量：token 流 **ts** 和符号表 `names`。

注意，大多数定义作用域的 C++ 构造都可以嵌套：

- 类中的函数：成员函数（member function，见 §9.4.2）

```
class C {
public:
    void f();
    void g()        // 成员函数可以在其类内定义
    {
        // . . .
    }
    // . . .
};


void C::f()         // 成员定义也可以在其类外
{
    // . . .
}
```

这是最常见、最有用的情况。

- 类中的类：成员类（member class，也称嵌套类，nested class）

```
class C {
public:
    struct M {
        // . . .
    };
    // . . .
};
```

这往往只在复杂的类中才有用；记住，理想的做法是让类保持小而简单。

- 函数中的类：局部类（local class）

```
void f()
{
    class L {
        // . . .
    };
    // . . .
}
```

避免这样做；如果你觉得需要一个局部类，那你的函数可能太长了。

- 函数中的函数：局部函数（local function，也称嵌套函数，nested function）

```
void f()
{
    void g()        // 非法
    {
        // . . .
    }
    // . . .
}
```

这在 C++ 中是不合法的；不要这样做。编译器会拒绝它。

- 函数及其他块中的块：嵌套块（nested block）

```
void f(int x, int y)
{
    if (x>y) {
        // . . .
    }
    else {
        // . . .
        {
            // . . .
        }
        // . . .
    }
}
```

嵌套块不可避免，但要对复杂的嵌套保持警惕：它很容易藏匿错误。

C++ 还专门提供了一个用来表达作用域的语言特性——namespace（命名空间）；见 §8.7。

请注意我们用一致的缩进来表示嵌套。没有一致的缩进，嵌套的构造就会变得不可读。例如：

```
// 危险的丑陋代码
struct X {
void f(int x) {
struct Y {
int f() { return 1; } int m; };
int m;
m=x; Y m2;
return f(m2.f()); }
int m; void g(int m) {
if (m) f(m+2); else {
g(m+2); }}
X() { } void m3() {
}

void main() {
X a; a.f(2);}
};
```

难以阅读的代码通常藏着 bug。当你使用 IDE 时，它会试图自动把你的代码缩进得“恰当”（按照对“恰当”的某种定义），而且还有一些“代码美化器”可以替你重新格式化源代码文件（往往会提供几种格式供你选择）。然而，让你的代码保持可读的最终责任在你自己。

8.5 函数调用与返回

函数是我们表示动作和计算的方式。每当有一件事值得起个名字，我们就写一个函数。C++ 语言给了我们运算符（例如 + 和 *），可以在表达式中由操作数产生新值；还给了我们语句（例如 **for** 和 **if**），可以控制执行的顺序。要把由这些基本构件组成的代码组织起来，我们靠的是函数。

为了完成工作，一个函数通常需要实参，而许多函数会返回一个结果。本节的重点是实参如何指定和传递。

8.5.1 声明实参与返回类型

在 C++ 中，我们用函数来命名和表示计算与动作。函数声明由一个返回类型、后接函数名、再后接圆括号中的形式实参（formal argument）列表组成。例如：

```
double fct(int a, double d);                    // fct 的声明（无函数体）
double fct(int a, double d) { return a*d; }     // fct 的定义
```

定义包含函数体（调用时要执行的语句），而不同时是定义的声明只有一个分号。形式实参常常被称为参数（parameter）。如果你不想让函数接受实参，略去形式实参即可。例如：

```
int current_power();    // current_power 不接受实参
```

如果你不想让函数返回一个值，就把它的返回类型给为 void。例如：

```
void increase_power(int level);    // increase_power 不返回值
```

这里，void 的意思是“不返回值”或“不返回任何东西”。

在声明和定义中，你可以按自己的需要给参数命名或不命名。例如：

```
// 在 vs 中查找 s;
// vs[hint] 可能是个合适的搜索起点
// 返回匹配处的下标; -1 表示“未找到”
int my_find(vector<string> vs, string s, int hint);  // 命名实参

int my_find(vector<string>, string, int);            // 不命名实参
```

在声明中，形式实参的名字在逻辑上并非必需，只是对写好注释非常有用。从编译器的角度看，**my_find()** 的第二条声明与第一条同样好：它拥有调用 **my_find()** 所需的全部信息。

通常，我们在定义中给所有实参都命名。例如：

```
int my_find(vector<string> vs, string s, int hint)
// 从 hint 开始在 vs 中查找 s
{
    if (hint<0 || vs.size()<=hint) hint = 0;
    for (int i = hint; i<vs.size(); ++i)    // 从 hint 开始查找
        if (vs[i]==s) return i;
    if (0<hint) {                           // 如果没找到 s，在 hint 之前查找
        for (int i = 0; i<hint; ++i)
            if (vs[i]==s) return i;
    }
    return -1;
}
```

hint 让代码复杂了不少，但提供 hint 的前提假设是：用户如果大致知道在 vector 的什么位置能找到 string，就可以利用它取得好的效果。然而，设想我们已经使用 **my_find()** 有一段时间了，然后发现调用者很少把 hint 用好，以至于它实际上损害了性能。现在我们不再需要 hint 了，但“外面”有大量代码用带 hint 的方式调用 **my_find()**。我们不想重写那些代码（或者无法重写，因为那是别人的代码），所以我们不想改变 **my_find()** 的声明。取而代之，我们只是不使用最后一个实参。既然不用它，我们就可以不给它命名：

```
int my_find(vector<string> vs, string s, int)   // 第 3 个实参未使用
{
    for (int i = 0; i<vs.size(); ++i)
        if (vs[i]==s) return i;
    return -1;
}
```

你可以在 ISO C++ 标准中找到函数定义的完整文法。

8.5.2 返回一个值

我们用 return 语句从函数返回一个值：

```
T f()       // f() 返回一个 T
{
    V v;
    // . . .
    return v;
}


T x = f();
```

这里，返回的值，恰好就是我们用一个 **V** 类型的值去初始化一个 **T** 类型的变量所会得到的值：

```
V v;
// . . .
T t(v);     // 用 v 初始化 t
```

也就是说，值返回（value return）是初始化的一种形式。

声明为返回一个值的函数就必须返回一个值。特别地，“从函数末尾掉出去”是一个错误：

```
double my_abs(int x)    // 警告：有 bug 的代码
{
    if (x < 0)
        return -x;
    else if (x > 0)
        return x;
}                       // 错误：x 为 0 时没有返回值
```

实际上，编译器多半不会注意到我们“忘了”**x==0** 这种情况。原则上它可以做到，但很少有编译器那么聪明。对于复杂的函数，编译器可能根本无法知道你是否返回了值，所以要小心。这里的“小心”是指：真正确保函数的每一条退出路径上都有 return 语句或者 **error()**。

出于历史原因，**main()** 是个特例。从 **main()** 的末尾掉出去等价于返回值 **0**，表示程序“成功完成”。

在一个不返回值的函数中，我们可以使用不带值的 return 来从函数返回。例如：

```
void print_until_s(vector<string> v, string quit)
{
    for(int s : v) {
        if (s==quit) return;
        cout << s << '\n';
    }
}
```

如你所见，从 void 函数的末尾“掉出去”是可以接受的。这等价于一条 **return;**。

8.5.3 传值

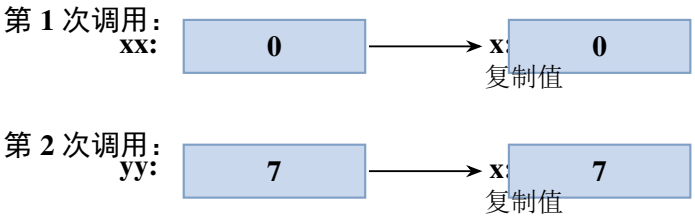
把实参传给函数的最简单方式，是把你所用的值的一份副本交给函数。函数 `f()` 的实参是 `f()` 中的一个局部变量，每次调用 `f()` 时它都会被初始化。例如：

```
// 传值（把所传值的一份副本交给函数）
int f(int x)
{
    x = x+1;            // 给局部变量 x 一个新值
    return x;
}


int main()
{
    int xx = 0;
    cout << f(xx) << '\n';      // 输出：1
    cout << xx << '\n';         // 输出：0；f() 不会改变 xx

    int yy = 7;
    cout << f(yy) << '\n';      // 输出：8
    cout << yy << '\n';         // 输出：7；f() 不会改变 yy
}
```

由于传递的是副本，`f()` 中的 `x=x+1` 不会改变两次调用所传入的 **xx** 和 **yy** 的值。我们可以像这样图示传值的实参传递：



传值相当简单直接，它的开销就是复制值的开销。

8.5.4 传 const 引用

当我们传递小值时——例如一个 **int**、`double` 或 **Token**（§6.3.2）——传值既简单又直接且高效。但如果值很大呢？比如一幅图像（往往有几百万个比特）、一张大的数值表（比如几千个整数），或者一个长字符串（比如几百个字符）？那时，复制就可能代价高昂。我们不应痴迷于开销，但做不必要的工作可能令人难堪，因为这表明我们没有直接表达出自己对本意的想法。例如，我们可以这样写一个打印浮点数 `vector` 的函数：

```
void print(vector<double> v)        // 传值；合适吗？
{
    cout << "{ ";
    for (int i = 0; i<v.size(); ++i) {
        cout << v[i];
        if (i!=v.size()-1) cout << ", ";
    }
    cout << " }\n";
}
```

我们可以把这个 **print()** 用于各种大小的 vector。例如：

```
void f(int x)
{
    vector<double> vd1(10);         // 小 vector
    vector<double> vd2(1000000);    // 大 vector
    vector<double> vd3(x);          // 某个未知大小的 vector
    // . . . 用值填充 vd1、vd2、vd3 . . .
    print(vd1);
    print(vd2);
    print(vd3);
}
```

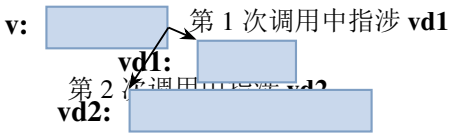
这段代码能工作，但 **print()** 的第一次调用必须复制十个 double（大概 80 字节），第二次调用必须复制一百万个 double（大概 8 MB），而第三次调用要复制多少我们无从知晓。这里我们必须问自己的问题是：“我们到底为什么非要复制？”我们只是想打印这些 vector，并不想复制它们的元素。显然，必须有一种办法，让我们把一个变量传给函数而不复制它。打个比方：如果交给你一项任务，要为图书馆的藏书编个清单，图书管理员不会把图书馆大楼连同全部馆藏给你运一份副本过来；他们会把图书馆的地址发给你，让你自己去看书。所以，我们需要一种办法，把 vector 的“地址”交给我们的 **print()** 函数，而不是 vector 的副本。这样一个“地址”称为引用（reference），用法如下：

```
void print(const vector<double>& v)   // 传 const 引用
{
    cout << "{ ";
    for (int i = 0; i<v.size(); ++i) {
        cout << v[i];
        if (i!=v.size()-1) cout << ", ";
    }
    cout << " }\n";
}
```

& 的意思是“引用”，而 const 在那里是为了防止 **print()** 意外地修改它的实参。除了实参声明的变化，一切都和之前一样；唯一的变化是，**print()** 现在不是在副本上操作，而是通过引用回指到实参。注意“回指”（refer back）这个说法；这类实参之所以称为引用（reference），是因为它们“指涉”（refer）到定义在别处的对象。我们可以完全像以前一样调用这个 **print()**：

```
void f(int x)
{
    vector<double> vd1(10);          // 小 vector
    vector<double> vd2(1000000);     // 大 vector
    vector<double> vd3(x);           // 某个未知大小的 vector
    // . . . 用值填充 vd1、vd2、vd3 . . .
    print(vd1);
    print(vd2);
    print(vd3);
}
```

我们可以用图形来说明：



const 引用有一个有用的性质：我们不会意外修改被传入的对象。例如，如果我们犯了个愚蠢的错误，试图在 **print()** 内部给一个元素赋值，编译器就会抓住它：

```
void print(const vector<double>& v)     // 传 const 引用
{
    // . . .
    v[i] = 7;                           // 错误：v 是一个 const（不可修改）
    // . . .
}
```

传 const 引用是一种有用且广受欢迎的机制。再考虑一下 **my_find()** 函数（§8.5.1），它在一个 string 的 vector 中查找一个 string。传值可能带来不必要的开销：

```
int my_find(vector<string> vs, string s);   // 传值：复制
```

如果这个 vector 中有几千个 string，即使在一台很快的计算机上，你可能也会察觉到所花费的时间。所以，我们可以改进 **my_find()**，让它通过 const 引用接受实参：

```
// 传 const 引用：不复制，只读访问
int my_find(const vector<string>& vs, const string& s);
```

8.5.5　传引用

但如果我们确实想让函数修改它的实参呢？有时候，这完全是合情合理的需求。例如，我们可能想要一个 init() 函数，为 vector 的元素赋值：

```
void init(vector<double>& v)        // 传引用
{
    for (int i = 0; i<v.size(); ++i) v[i] = i;
}
```

```
void g(int x)
{
    vector<double> vd1(10);          // 小 vector
    vector<double> vd2(1000000);     // 大 vector
    vector<double> vd3(x);           // 某个未知大小的 vector

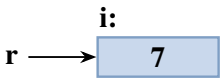
    init(vd1);
    init(vd2);
    init(vd3);
}
```

这里，我们想让 init() 修改作为实参的 vector，所以既没有拷贝（没有用传值），也没有把引用声明为 const（没有用传 const 引用），而是直接向这个 vector 传递了一个“普通引用”。

我们从更技术的角度来考虑引用。引用（reference）是这样一种语言构造：它允许用户为一个对象声明一个新名字。例如，int& 是指向 **int** 的引用，因此我们可以写

```
int i = 7;


int& r = i;      // r 是 i 的引用
r = 9;           // i 变成 9
i = 10;
cout << r << ' ' << i << '\n';     // 写出：10 10
```



也就是说，对 **r** 的任何使用，实际上都是对 **i** 的使用。

引用作为简写形式很有用。例如，假设我们有一个

```
vector< vector<double> > v;  // 由 double 的 vector 组成的 vector
```

并且需要多次引用某个元素 v[f(x)][g(y)]。显然，v[f(x)][g(y)] 是一个相当复杂的表达式，我们不想不必要地反复写它。如果只需要它的值，可以写

```
double val = v[f(x)][g(y)];      // val 是 v[f(x)][g(y)] 的值
```

然后反复使用 **val**。但如果我们既要从 v[f(x)][g(y)] 读，又要向 v[f(x)][g(y)] 写呢？这时引用就派上用场了：

```
double& var = v[f(x)][g(y)];     // var 是 v[f(x)][g(y)] 的引用
```

现在，我们可以通过 **var** 读写 v[f(x)][g(y)]。例如：

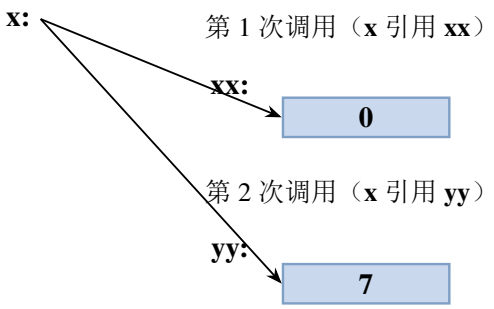
```
var = var/2+sqrt(var);
```

引用的这一关键性质——引用可以作为某个对象的便捷简写——正是它作为实参很有用的原因。例如：

```
// 传引用（让函数指回传递给它的变量）
int f(int& x)
{
    x = x+1;
    return x;
}
int main()
{
    int xx = 0;
    cout << f(xx) << '\n';    // 写出：1
    cout << xx << '\n';       // 写出：1；f() 改变了 xx 的值

    int yy = 7;
    cout << f(yy) << '\n';    // 写出：8
    cout << yy << '\n';       // 写出：8；f() 改变了 yy 的值
}
```

我们可以像下面这样图示传引用的实参传递：



请将它与 §8.5.3 中的类似例子比较。

传引用显然是一种非常强大的机制：我们可以让一个函数直接操作我们传递给它的引用所指向的任何对象。例如，交换两个值在许多算法（比如排序）中都是极其重要的操作。使用引用，我们可以这样编写一个交换 double 的函数：

```
void swap(double& d1, double& d2)
{
    double temp = d1;     // 把 d1 的值拷贝给 temp
    d1 = d2;              // 把 d2 的值拷贝给 d1
    d2 = temp;            // 把 d1 的旧值拷贝给 d2
}
```

```
int main()
{
    double x = 1;
    double y = 2;
    cout << "x == " << x << " y== " << y << '\n';    // 写出：x==1 y==2
    swap(x,y);
    cout << "x == " << x << " y== " << y << '\n';    // 写出：x==2 y==1
}
```

标准库为每种可拷贝的类型都提供了一个 swap()，因此你不必为每种类型自己去写 swap()。

8.5.6 传值与传引用的对比

什么时候该用传值、传引用和传 const 引用呢？先看一个技术性的例子：

```
void f(int a, int& r, const int& cr)
{
    ++a;      // 改变局部的 a
    ++r;      // 改变 r 所引用的对象
    ++cr;     // 错误：cr 是 const
}
```

如果想改变被传递对象的值，就必须使用非 const 引用：传值得到的是副本，而传 const 引用则禁止你改变被传递对象的值。因此我们可以试试

```
void g(int a, int& r, const int& cr)
{
    ++a;        // 改变局部的 a
    ++r;        // 改变 r 所引用的对象
    int x = cr; // 读取 cr 所引用的对象
}

int main()
{
    int x = 0;
    int y = 0;
    int z = 0;

    g(x,y,z);     // x==0; y==1; z==0
    g(1,2,3);     // 错误：引用实参 r 需要一个变量来引用
    g(1,y,3);     // OK：由于 cr 是 const，可以传递字面值
}
```

所以，如果想改变以引用传递的对象的值，就必须传递一个对象。严格来说，整数字面值 **2** 只是一个值（右值，rvalue），而不是一个持有值的对象。g() 的实参 **r** 需要的是一个左值（lvalue），即可以出现在赋值号左边的东西。

注意，const 引用并不需要左值。它可以像初始化或传值一样进行类型转换。从根本上说，最后一处调用 g(1,y,3) 中发生的事情是：编译器为 g() 的实参 **cr** 设置了一个 **int** 供它引用：

```
g(1,y,3);       // 意思是：int __compiler_generated = 3; g(1,y,__compiler_generated)
```

这样一个由编译器生成的对象称为临时对象（temporary object），或简称临时量（temporary）。

我们的经验法则是：

1. 用传值传递非常小的对象。
2. 用传 const 引用传递不需要修改的大对象。
3. 返回结果，而不是通过引用实参修改对象。
4. 只有在不得已时才用传引用。

这些规则能带来最简单、最不容易出错、也最高效的代码。所谓“非常小”，是指一两个 **int**、一两个 double 之类的东西。如果看到某个实参是以非 const 引用传递的，我们必须假定被调用的函数会修改那个实参。

第三条规则反映出：当你想用一个函数改变某个变量的值时，你是有选择的余地的。考虑：

```
int incr1(int a) { return a+1; }    // 把新值作为结果返回
void incr2(int& a) { ++a; }         // 修改以引用传递的对象

int x = 7;
x = incr1(x);      // 相当显而易见
incr2(x);          // 相当晦涩
```

那我们到底为什么还要用非 const 引用实参呢？偶尔，它们是必不可少的

- 用于操作容器（如 vector）和其他大对象
- 用于要改变若干个对象的函数（返回值只能有一个）

例如：

```
void larger(vector<int>& v1, vector<int>& v2)
        // 使 v1 中每个元素成为 v1 与 v2 中对应元素中较大者；
        // 类似地，使 v2 中每个元素成为较小者
{
    if (v1.size()!=v2.size()) error("larger(): different sizes");
    for (int i=0; i<v1.size(); ++i)
        if (v1[i]<v2[i])
            swap(v1[i],v2[i]);
}
void f()
{
    vector<int> vx;
    vector<int> vy;
    // 从输入读入 vx 和 vy
    larger(vx,vy);
    // ...
}
```

对于像 larger() 这样的函数，使用传引用实参是唯一合理的选择。

通常最好避免使用会修改若干个对象的函数。理论上，总是有替代方案，例如返回一个保存了多个值的类对象。然而，市面上已有大量程序是用修改一个或多个实参的函数来表达的，你很可能会遇到

它们。例如，在大约 50 年里用于数值计算的主要语言 Fortran 中，所有实参按传统都是以引用传递的。许多数值程序员照搬 Fortran 的设计，并调用 Fortran 写的函数。这类代码常常使用传引用或传 `const` 引用。

如果使用引用只是为了避免拷贝，我们就用 `const` 引用。因此，当我们看到一个非 `const` 引用实参时，就假定函数会改变其实参的值；也就是说，看到传非 `const` 引用时，我们假定那个函数不但能够修改传递给它的实参，而且一定会修改，所以我们必须格外仔细地检查调用，确保它做的正是我们期望的事。

8.5.7 实参检查与转换

传递实参，就是用调用处给定的实参来初始化函数的形式参数。考虑：

```
void f(T x);
f(y);
T x = y;      // 用 y 初始化 x（参见 §8.2.2）
```

只要初始化 `T x=y;` 是合法的，调用 `f(y)` 就是合法的，而且当它合法时，两个 `x` 得到相同的值。例如：

```
void f(double x);

void g(int y)
{
    f(y);
    double x = y;    // 用 y 初始化 x（参见 §8.2.2）
}
```

注意，要用 **x** 接收 **y**，必须把一个 **int** 转换为 `double`。调用 `f()` 时发生的也是同样的转换。`f()` 收到的 `double` 值与存储在 **x** 中的完全相同。

类型转换通常很有用，但偶尔也会给出令人意外的结果（参见 §3.9.2）。因此我们必须小心使用它们。把一个 `double` 作为实参传给一个需要 **int** 的函数，很少是个好主意：

```
void ff(int x);

void gg(double y)
{
    ff(y);       // 你怎么知道这是否合理？
    int x = y;   // 你怎么知道这是否合理？
}
```

如果你确实打算把一个 `double` 值截断为 **int**，请明确地说出来：

```
void ggg(double x)
{
    int x1 = x;                     // 截断 x
    int x2 = int(x);
    int x3 = static_cast<int>(x);    // 非常显式的转换 (§17.8)

    ff(x1);
    ff(x2);
    ff(x3);

    ff(x);                          // 截断 x
    ff(int(x));
    ff(static_cast<int>(x));        // 非常显式的转换 (§17.8)
}
```

这样，下一个阅读这段代码的程序员就能看出你考虑过这个问题。

8.5.8 函数调用的实现

可是，计算机究竟是如何完成一次函数调用的？第 6 章和第 7 章的 `expression()`、`term()` 和 `primary()` 函数非常适合用来说明这一点，只有一个细节不合适：它们不接受任何实参，所以我们没法用它们解释实参是如何传递的。但是且慢！它们总得接受些输入；如果不接受，它们就做不了任何有用的事。它们确实接受一个隐式实参：它们使用一个名为 **ts** 的 `Token_stream` 来获得输入；**ts** 是一个全局变量。这有点偷偷摸摸。我们可以改进这些函数，让它们接受一个 `Token_stream&` 实参。下面就是加入了 `Token_stream&` 形参、并去掉了所有与函数调用实现无关的内容之后的版本。

首先，`expression()` 完全直截了当；它有一个实参（**ts**）和两个局部变量（`left` 和 **t**）：

```
double expression(Token_stream& ts)
{
    double left = term(ts);
    Token t = ts.get();
    // ...
}
```

其次，`term()` 与 `expression()` 非常相似，只是多了一个局部变量（**d**），用于保存'/' 的除数结果：

```
double term(Token_stream& ts)
{
    double left = primary(ts);
    Token t = ts.get();
    // ...
        case '/':
        {
            double d = primary(ts);
            // ...
        }
    // ...
}
```

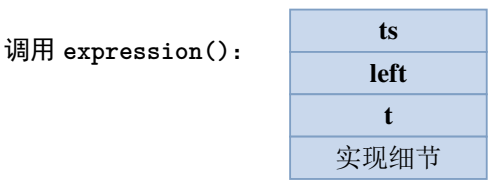
第三，primary() 与 term() 很像，只是它没有局部变量 left：

```
double primary(Token_stream& ts)
{
    Token t = ts.get();
    switch (t.kind) {
    case '(':
        {    double d = expression(ts);
            // ...
        }
        // ...
    }
}
```

现在，它们不再使用任何“偷偷摸摸的全局变量”，对我们的说明来说堪称完美：它们有实参、有局部变量，而且彼此调用。

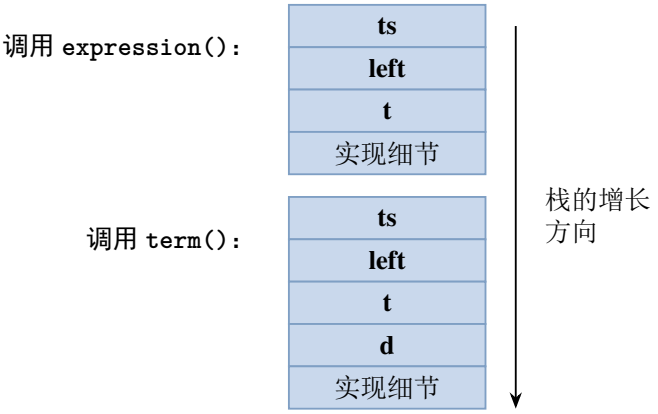
你可以借此机会回忆一下完整的 expression()、term() 和 primary() 是什么样子，但就函数调用而言，这里呈现的正是它们的要点。

当一个函数被调用时，语言的实现会设置一个数据结构，其中包含它所有形参和局部变量的副本。例如，expression() 第一次被调用时，编译器确保创建一个类似这样的结构：



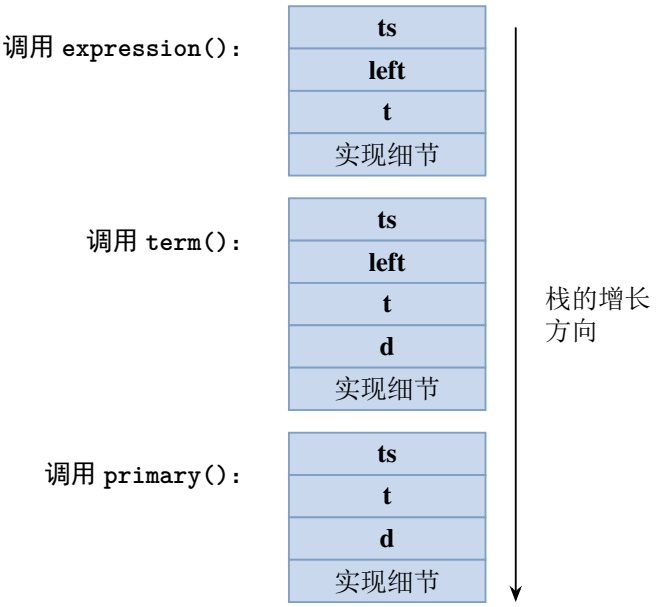
“实现细节”因实现而异，但它基本上就是函数返回到其调用者所需的、以及向调用者返回一个值所需的信息。这样的数据结构称为函数活动记录（function activation record），每个函数都有自己活动记录的详细布局。注意，从实现的角度看，形参不过是又一个局部变量。

到目前为止一切正常。现在 expression() 调用 term()，于是编译器确保为 term() 的这次调用生成一个活动记录：



注意，term() 多了一个需要存储的变量 **d**，因此即使代码可能永远不会用到它，我们也在调用中为它留出了空间。这没问题。对于合理的函数（例如本书中我们直接或间接用到的每个函数），建立函数活动记录的运行时开销与其大小无关。局部变量 **d** 只有在我们执行到它的 **case '/'** 时才会被初始化。

现在 term() 调用 primary()，于是得到



这开始有点重复了，但现在 primary() 又调用 expression()：

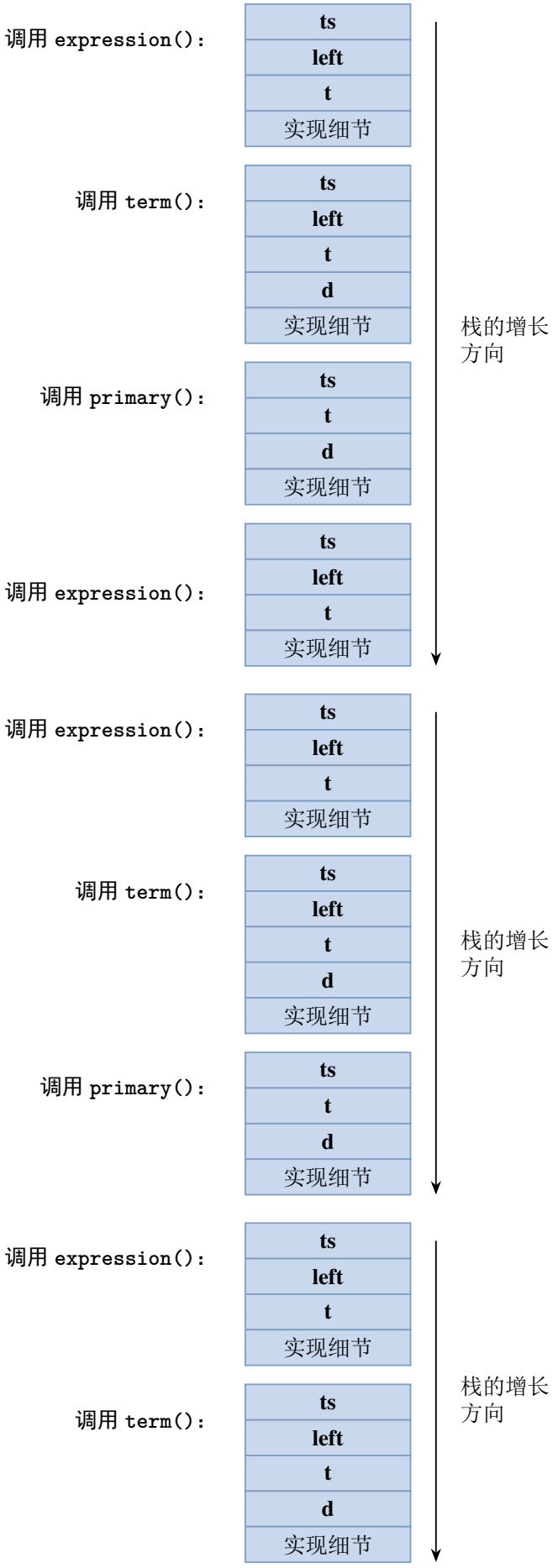
于是，expression() 的这次调用得到了它自己的活动记录，与 expression() 的第一次调用不同。这很好，否则我们就乱套了，因为两次调用中的 left 和 **t** 是不同的。直接或（如本例）间接调用自身的函数称为递归的（recursive）。如你所见，递归函数很自然地源于我们实现函数调用与返回所用的技术（反之亦然）。

所以，每调用一个函数，活动记录的栈（stack of activation records，通常简称栈，stack）就增长一条记录。反过来，当函数返回时，它的记录就不再使用了。例如，当 expression() 的最后一次调用返回到 primary() 时，栈会回退成这样：

而当那次 primary() 调用返回到 term() 时，我们就回到

如此继续。这个栈也称为调用栈（call stack），是一种在一端按“后进先出”规则增长和收缩的数据结构。

请记住，调用栈如何实现和使用的细节因 C++ 实现而异，但基本原理如上所述。要使用函数调用，你需要知道它们是如何实现的吗？当然不需要；在这个讲实现的小节之前，你一直做得挺好。但许多


调用 expression():
ts
left
t
实现细节
调用 term():
ts
left
t
d
实现细节
调用 primary():
ts
t
d
实现细节
调用 expression():
ts
left
t
实现细节
栈的增长方向
调用 expression():
ts
left
t
实现细节
调用 term():
ts
left
t
d
实现细节
调用 primary():
ts
t
d
实现细节
栈的增长方向
调用 expression():
ts
left
t
实现细节
调用 term():
ts
left
t
d
实现细节
栈的增长方向

程序员喜欢了解这些，而且很多人会使用“活动记录”和“调用栈”这样的说法，所以最好知道它们是什么意思。

8.5.9 constexpr 函数

函数表示一次计算，而有时我们想在编译时（compile time）完成一次计算。希望计算由编译器来求值，通常是为了避免同样的计算在运行时做上百万次。我们使用函数是为了让计算易于理解，所以自然有时会想在常量表达式中使用函数。我们通过把函数声明为 `constexpr` 来表明希望它由编译器求值的意图。如果给 `constexpr` 函数的实参是常量表达式，它就可以由编译器求值。例如：

```
constexpr double xscale = 10;     // 缩放因子
constexpr double yscale = 0.8;

constexpr Point scale(Point p) { return {xscale*p.x,yscale*p.y}; };
```

假定 **Point** 是一个简单的 `struct`，其成员 **x** 和 **y** 表示二维坐标。现在，给 `scale()` 一个 **Point** 实参，它会返回一个按因子 `xscale` 和 `yscale` 缩放了坐标的 **Point**。例如：

```
void user(Point p1)
{
    Point p2 {10,10};

    Point p3 = scale(p1);    // OK: p3 == {100,8}; 运行时求值也没问题
    Point p4 = scale(p2);    // p4 == {100,8}

    constexpr Point p5 = scale(p1);    // 错误：scale(p1) 不是常量
                                       // 表达式
    constexpr Point p6 = scale(p2);    // p6 == {100,8}

    // ...
}
```

在用到需要常量的地方之前，`constexpr` 函数的行为与普通函数完全一样。在那里，只要它的实参是常量表达式（例如 **p2**），它就会在编译时计算；如果不是（例如 **p1**），则给出错误。为此，`constexpr` 函数必须简单到编译器（每一个符合标准的编译器）都能对它求值。在 C++11 中，这意味着 `constexpr` 函数的函数体必须只由一条 `return` 语句组成（像 `scale()` 那样）；在 C++14 中，我们还可以写简单的循环。`constexpr` 函数不能有副作用；也就是说，它不能改变自己函数体之外的变量的值，除了它被赋值给或用来初始化的那些对象。

下面是一个违反了这些简单性规则的函数：

```
int glob = 9;

constexpr void bad(int & arg)     // 错误：没有返回值
{
    ++arg;      // 错误：通过实参修改了调用者的对象
    glob = 7;   // 错误：修改了非局部变量
}
```

如果编译器无法断定一个 constexpr 函数“足够简单”（依据标准中的详细规则），这个函数就被认定为错误。

8.6 求值顺序

程序的求值——也称为程序的执行——按照语言规则逐条语句推进。当这条“执行线索”到达一个变量的定义时，该变量被构造（construct）；也就是说，为对象留出内存并对其进行初始化。当变量离开作用域时，该变量被销毁（destroy）；也就是说，它所指的对象原则上被移除，编译器可以把它的内存挪作他用。例如：

```
string program_name = "silly";
vector<string> v;                 // v 是全局的

void f()
{
    string s;                     // s 局限于 f
    while (cin>>s && s!="quit") {
        string stripped;          // stripped 局限于循环
        string not_letters;
        for (int i=0; i<s.size(); ++i)     // i 具有语句作用域
            if (isalpha(s[i]))
                stripped += s[i];
            else
                not_letters += s[i];
        v.push_back(stripped);
        // ...
    }
    // ...
}
```

全局变量（例如 program_name 和 **v**）在 main() 的第一条语句执行之前初始化。它们“存活”到程序终止，然后被销毁。它们按定义的顺序构造（即 program_name 在 **v** 之前），并按相反的顺序销毁（即 **v** 在 program_name 之前）。

当有人调用 f() 时，首先构造 **s**；也就是说，**s** 被初始化为空字符串。它会存活到我们从 f() 返回。

每次进入作为 while 语句体的那个块，stripped 和 not_letters 都被构造。由于 stripped 定义在 not_letters 之前，stripped 先于 not_letters 构造。它们存活到循环结束，在条件重新求值之

前，按构造的相反顺序销毁（即 not_letters 先于 stripped）。因此，如果在遇到字符串 quit 之前读到了十个字符串，stripped 和 not_letters 就各被构造和销毁十次。

每次到达 **for** 语句，**i** 被构造。每次退出 **for** 语句，**i** 在到达 v.push_back(stripped); 语句之前被销毁。

请注意，编译器（和链接器）是很聪明的野兽，只要结果与我们这里描述的等价，它们被允许——而且确实会——优化代码。特别是，编译器很擅长不在真正必要之外更多地分配和释放内存。

8.6.1 表达式的求值

子表达式的求值顺序由这样一些规则支配：这些规则旨在取悦优化器，而不是让程序员的日子好过。这很遗憾，但无论如何你都应避免复杂的表达式，而且有一个简单的规则可以让你免于麻烦：如果在一个表达式中改变了某个变量的值，就不要在同一表达式中再读写它两次。例如：

```
v[i] = ++i;     // 不要：未定义的求值顺序
v[++i] = i;     // 不要：未定义的求值顺序
int x = ++i + ++i;              // 不要：未定义的求值顺序
cout << ++i << ' ' << i << '\n';     // 不要：未定义的求值顺序
f(++i,++i);                     // 不要：未定义的求值顺序
```

遗憾的是，并非所有编译器都会对这样的坏代码发出警告；它之所以坏，是因为如果把代码搬到另一台计算机、换一种编译器或换一种优化器设置，你就不能指望结果仍然相同。对这种代码，编译器之间的差别真的很大；千万别这么写。

特别要注意，=（赋值）在表达式中只是又一个运算符，因此无法保证赋值号左边先于右边求值。这正是 v[++i] = i 未定义的原因。

8.6.2 全局初始化

在单个翻译单元（translation unit）中，全局变量（以及命名空间变量；参见 §8.7）按它们出现的顺序初始化。例如：

```
// 文件 f1.cpp
int x1 = 1;
int y1 = x1+2;     // y1 变为 3
```

这种初始化在逻辑上发生在“main() 中的代码执行之前”。

在绝大多数场合使用全局变量通常都不是好主意。我们已经提到过这个问题：程序员没有真正有效的办法知道一个大程序的哪些部分读和/或写某个全局变量（§8.4）。另一个问题是，不同翻译单元中全局变量的初始化顺序是没有定义的。例如：

```
// 文件 f2.cpp
extern int y1;
int y2 = y1+2;    // y2 变为 2 或 5
```

这种代码有几条理由必须避免：它使用了全局变量，它给全局变量起了短名字，而且它对全局变量使用了复杂的初始化。如果 `f1.cpp` 中的全局变量先于 `f2.cpp` 的全局变量初始化，**y2** 将被初始化为 **5**（程序员可能天真而合理地期望如此）。然而，如果 `f2.cpp` 中的全局变量先于 `f1.cpp` 的初始化，**y2** 将被初始化为 **2**（因为用于全局变量的内存在尝试复杂初始化之前被初始化为 **0**）。避免这样的代码；看到带有非平凡初始化器的全局变量时要高度警惕；任何不是常量表达式的初始化器都应视为复杂的。

可如果你确实需要一个带复杂初始化器的全局变量（或常量），怎么办？一个说得过去的例子是：我们想为某个支持商业事务的库所提供的 **Date** 类型设置一个默认值：

```
const Date default_date(1970,1,1);    // 默认日期是 1970 年 1 月 1 日
```

我们怎么知道 `default_date` 在初始化之前从未被使用过？基本上无法知道，所以就不应该那样定义。我们最常用的技术是调用一个返回该值的函数。例如：

```
const Date default_date()             // 返回默认日期
{
    return Date(1970,1,1);
}
```

这样，每次调用 `default_date()` 都会构造这个 **Date**。这通常没问题，但如果 `default_date()` 被频繁调用而构造 **Date** 的代价又高，我们就希望只构造一次 **Date**。做法如下：

```
const Date& default_date()
{
    static const Date dd(1970,1,1);    // 第一次到达这里时初始化 dd
    return dd;
}
```

`static` 局部变量只在第一次调用其函数时被初始化（构造）。注意，我们返回一个引用以消除不必要的拷贝；而且特别地，我们返回一个 `const` 引用，以防调用函数意外地改变这个值。关于如何传递实参的论证（§8.5.6）同样适用于返回值。

8.7 命名空间

我们用块来组织函数内部的代码（§8.4），用类把函数、数据和类型组织成一个类型（第 9 章）。函数和类都为我们做了两件事：

- 它们允许我们定义许多“实体”，而不必担心它们的名字与程序中的其他名字冲突。
- 它们给我们一个名字，用来指称我们所定义的东西。

到目前为止我们还缺少一种东西：把类、函数、数据和类型组织成程序中一个可识别、有名字的部分，而又不定义一个类型。实现这种声明分组的语言机制就是命名空间（namespace）。例如，我们可能想提供一个图形库，其中包含名为 **Color**、**Shape**、**Line**、**Function** 和 **Text** 的类（参见第 13 章）：

```
namespace Graph_lib {
    struct Color { /* ... */ };
    struct Shape { /* ... */ };
    struct Line : Shape { /* ... */ };
    struct Function : Shape { /* ... */ };
    struct Text : Shape { /* ... */ };
    // ...
    int gui_main() { /* ... */ }
}
```

很可能世界上已有别人用过这些名字，但现在这不要紧了。你可能也定义了一个叫 **Text** 的东西，但我们的 **Text** 不会与之干扰。`Graph_lib::Text` 是我们的类之一，而你的 **Text** 不是。只有当你也有一个名为 `Graph_lib` 的类或命名空间、且 **Text** 是其成员时，才会出问题。`Graph_lib` 这个名字有点丑；我们选它是因为“漂亮而显然”的名字 **Graphics** 更有可能已经在某处被用过。

假设你的 **Text** 是某个文本处理库的一部分。促使我们把图形设施放进命名空间 `Graph_lib` 的同一逻辑，也应当促使你把你的文本处理设施放进一个类似 **TextLib** 的命名空间：

```
namespace TextLib {
    class Text { /* ... */ };
    class Glyph { /* ... */ };
    class Line { /* ... */ };
    // ...
}
```

假如我们都用了全局命名空间，那可就麻烦大了。想同时使用我们两个库的人，会在 **Text** 和 **Line** 上遇到严重得可怕的名字冲突。更糟的是，如果我们俩的库都有用户，我们就无法通过改名字（比如 **Line** 和 **Text**）来避免冲突了。我们用命名空间避免了这个问题：我们的 **Text** 是 `Graph_lib::Text`，你的 **Text** 是 `TextLib::Text`。由命名空间名（或类名）与成员名用 `::` 组合而成的名字称为完全限定名（fully qualified name）。

8.7.1 using 声明与 using 指令

完全限定名写起来可能很繁琐。例如，C++ 标准库的设施都定义在命名空间 **std** 中，可以这样使用：

```
#include<string>       // 获得 string 库
#include<iostream>     // 获得 iostream 库

int main()
{
    std::string name;
    std::cout << "Please enter your first name\n";
    std::cin >> name;
    std::cout << "Hello, " << name << '\n';
}
```

标准库的 string 和 cout 我们已经见过成千上万次了，实在不想每次都用它们“正式的”完全限定名 std::string 和 std::cout。一种解决办法是声明“我写 string，指的就是 std::string”“我写 cout，指的就是 std::cout”，等等：

```
using std::string;      // string 表示 std::string
using std::cout;        // cout 表示 std::cout
// ...
```

这种构造称为 using 声明（using declaration）；它就好比：当房间里没有其他叫 Greg 的人时，直接用“Greg”来称呼 Greg Hansen。

有时，我们更喜欢对命名空间中名字的用法采用一种更强的“简写”：“如果在当前作用域中找不到某个名字的声明，就到 **std** 里去找。”表达这一点的方式是使用 using 指令（using directive）：

```
using namespace std;    // 使 std 中的名字可直接使用
```

于是就有了这种常见风格：

```
#include<string>        // 获得 string 库
#include<iostream>      // 获得 iostream 库
using namespace std;    // 使 std 中的名字可直接使用

int main()
{
    string name;
    cout << "Please enter your first name\n";
    cin >> name;
    cout << "Hello, " << name << '\n';
}
```

其中的 **cin** 是 std::cin，string 是 std::string，等等。只要你使用 std_lib_facilities.h，就不必为标准头文件和 **std** 命名空间操心。

对任何命名空间，除了像 **std** 这样在某个应用领域内人尽皆知的命名空间之外，通常都应避免使用 using 指令。过度使用 using 指令的问题在于，你会搞不清哪些名字来自哪里，于是又开始出现名字冲突。用命名空间名显式限定和使用 using 声明则不会有这个问题。因此，把一条 using 指令放进头文件（使用户无法避开它）是一个非常坏的习惯。不过，为了简化最初的代码，我们确实在 std_lib_facilities.h 中为 **std** 放了一条 using 指令。这让我们得以写

```
#include "std_lib_facilities.h"

int main()
{
    string name;
    cout << "Please enter your first name\n";
    cin >> name;
    cout << "Hello, " << name << '\n';
}
```

我们保证，除了 **std** 之外，绝不对任何命名空间这样做。

小试身手

1. 创建三个文件：my.h、my.cpp 和 use.cpp。头文件 my.h 包含

```
extern int foo;
void print_foo();
void print(int);
```

源代码文件 my.cpp #include 了 my.h 和 std_lib_facilities.h，定义 **print_foo()** 用 cout 打印 **foo** 的值，并定义 **print(int i)** 用 cout 打印 **i** 的值。

源代码文件 use.cpp #include 了 my.h，定义 **main()**：把 **foo** 的值设为 **7** 并用 **print_foo()** 打印它，再用 **print()** 打印 **99** 的值。注意，use.cpp 不 #include std_lib_facilities.h，因为它不直接使用其中的任何设施。

让这些文件通过编译并运行。在 Windows 上，需要把 use.cpp 和 my.cpp 都放进一个项目，并在 use.cpp 中使用 **{ char cc; cin>>cc; }** 才能看到输出。提示：需要 #include **<iostream>** 才能使用 **cin**。

2. 编写三个函数 **swap_v(int,int)**、**swap_r(int&,int&)** 和 **swap_cr(const int&, const int&)**。每个函数的函数体都应为

```
{ int temp; temp = a, a=b; b=temp; }
```

其中 **a** 和 **b** 是实参的名字。

试着像下面这样调用每个 swap：

```
int x = 7;
int y =9;
swap_?(x,y);     // 将 ? 换成 v、r 或 cr
swap_?(7,9);
const int cx = 7;
const int cy = 9;
swap_?(cx,cy);
swap_?(7.7,9.9);
double dx = 7.7;
double dy = 9.9;
swap_?(dx,dy);
swap_?(7.7,9.9);
```

哪些函数和调用能通过编译，为什么？在每个通过了编译的 swap 调用之后，打印调用后实参的值，看看它们是否真的被交换了。如果你对某个结果感到意外，请参阅 §8.6。

3. 编写一个程序，用一个文件包含三个命名空间 **X**、**Y** 和 **Z**，使下面的 `main()` 能正确工作：

```
int main()
{
    X::var = 7;
    X::print();          // 打印 X 的 var
    using namespace Y;
    var = 9;
    print();             // 打印 Y 的 var
    {    using Z::var;
        using Z::print;
        var = 11;
        print();         // 打印 Z 的 var
    }
    print();             // 打印 Y 的 var
    X::print();          // 打印 X 的 var
}
```

每个命名空间都需要定义一个名为 **var** 的变量和一个名为 `print()` 的函数，后者用 `cout` 输出相应的 **var**。

要点回顾

1. 声明和定义有什么区别？
2. 从语法上如何区分函数声明和函数定义？
3. 从语法上如何区分变量声明和变量定义？
4. 为什么不先声明，就不能使用第 6 章计算器程序中的函数？
5. `int a;` 是定义还是仅仅是声明？
6. 为什么在声明变量的同时初始化它们是个好主意？
7. 函数声明可以由哪些部分组成？
8. 缩进有什么好处？
9. 头文件有什么用途？
10. 声明的作用域是什么？
11. 有哪些种作用域？各举一例。
12. 类作用域和局部作用域有什么区别？
13. 程序员为什么应尽量少用全局变量？
14. 传值和传引用有什么区别？
15. 传引用和传 const 引用有什么区别？
16. 什么是 `swap()`？
17. 你会用 `vector<double>` 传值形参来定义函数吗？
18. 举一个求值顺序未定义的例子。为什么求值顺序未定义会成问题？
19. `x&&y` 和 **x||y** 分别是什么意思？
20. 下面哪些是符合标准的 C++：函数内的函数、类内的函数、类内的类、函数内的类？

21. 活动记录里有什么？
22. 什么是调用栈？我们为什么需要它？
23. 命名空间的目的是什么？
24. 命名空间与类有什么不同？
25. 什么是 `using` 声明？
26. 为什么应避免在头文件中使用 `using` 指令？
27. 什么是命名空间 **std**？

术语

活动记录　函数　传引用
实参　函数定义　传值
实参传递　全局作用域　递归
调用栈　头文件　return
类作用域　初始化器　返回值
const　局部作用域　作用域
constexpr　命名空间　语句作用域
声明　命名空间作用域　技术细节
定义　嵌套块　未声明的标识符
extern　形参　using 声明
前向声明　传 const 引用　using 指令

习题

1. 修改第 7 章的计算器程序，把输入流变成一个显式形参（如 §8.5.8 所示），而不是简单地使用 **cin**。同时给 `Token_stream` 构造函数（§7.8.2）一个 **`istream&`** 形参，这样等我们弄清如何创建自己的 **`istream`**（例如附加到文件的流）时，就能把计算器用于它们。提示：不要试图拷贝 **`istream`**。
2. 编写一个函数 **`print()`**，把一个 int 的 `vector` 打印到 `cout`。给它两个实参：一个用于“标注”输出的 `string` 和一个 `vector`。
3. 创建一个由斐波那契数组成的 `vector`，并用习题 2 的函数打印它们。为了创建这个 `vector`，编写一个函数 **`fibonacci(x,y,v,n)`**，其中整数 **x** 和 **y** 是 int，**v** 是一个空的 **`vector<int>`**，**n** 是要放入 **v** 的元素个数；**`v[0]`** 将是 **x**，**`v[1]`** 将是 **y**。斐波那契数是这样一个序列中的数：序列中每个元素是前两个元素之和。例如，从 1 和 2 开始，我们得到 1、2、3、5、8、13、21、……。你的 **`fibonacci()`** 函数应该以它的 **x** 和 **y** 实参为起点生成这样的序列。
4. 一个 int 只能保存到某个最大数为止的整数。用 **`fibonacci()`** 求这个最大数的一个近似值。
5. 编写两个函数，反转一个 **`vector<int>`** 中元素的顺序。例如，1、3、5、7、9 变成 9、7、5、3、1。第一个 reverse 函数应产生一个包含反转后序列的新 `vector`，原来的 `vector` 保持不变。另一个 reverse 函数应原地反转其 `vector` 的元素，不借助任何其他 `vector`（提示：swap）。
6. 为习题 5 的函数编写处理 **`vector<string>`** 的版本。
7. 将五个名字读入一个 **`vector<string>`** `name`，然后提示用户输入这些人的年龄，并将年龄保存在一个 **`vector<double>`** **`age`** 中。接着打印出五对 **`(name[i],age[i])`**。对名字排序（**`sort(name.begin(),name.end())`**）后再打印出 **`(name[i],age[i])`** 对。这里的难点是让 **age**

vector 的顺序与排序后的 name vector 相对应。提示：在排序 name 之前先留一份副本，并用它在排序 name 之后按正确顺序复制出 **age**。然后，再做一遍这道题，但允许任意数量的名字。

9. 编写一个函数，给定两个 `vector<double>` price 和 weight，计算一个值（一个“指数”），即所有 `price[i]*weight[i]` 的和。确保 `weight.size()==price.size()`。
10. 编写一个函数 `maxv()`，返回其 vector 实参中最大的元素。
11. 编写一个函数，找出其 vector 实参中最小和最大的元素，并计算均值和中位数。不要使用全局变量。要么返回一个包含结果的 struct，要么通过引用实参把它们带回来。这两种返回多个结果值的方式，你更喜欢哪一种？为什么？
12. 改进 §8.5.2 的 `print_until_s()`。对它进行测试。什么样的测试用例集合才算好？给出理由。然后，编写一个 `print_until_ss()`，它一直打印到第二次看到其 quit 实参为止。
13. 编写一个函数，接受一个 `vector<string>` 实参，返回一个 `vector<int>`，其中包含每个 string 的字符数。同时找出最长和最短的 string，以及按字典序排最前和最后的 string。你会为这些任务使用多少个独立的函数？为什么？
14. 我们能把非引用的函数实参声明为 const 吗（例如 `void f(const int);`）？这可能意味着什么？我们为什么会想这么做？为什么人们不常这样做？试一试；写几个小程序，看看什么能行。

后记

我们本可以把本章的许多内容（以及下一章的许多内容）放进附录。然而，本书第二部分会用到这里介绍的绝大部分设施。这些设施被发明出来所要帮助解决的问题，你很快也会遇到大多数。你可能着手的大多数简单编程项目，都会要求你解决这类问题。因此，为了节省时间、减少困惑，需要一种多少有些系统性的方法，而不是一次次“随机地”翻阅手册和附录。

9 技术细节：类等

“记住，事情需要时间。”

——Piet Hein

在本章中，我们继续关注我们的主要编程工具——C++ 编程语言。我们将介绍语言的技术细节（technicalities），其中大部分与用户自定义类型（user-defined type）有关，也就是与类（class）和枚举（enumeration）有关。大部分语言特性的讲解，都是以逐步改进一个 **Date** 类型的形式展开的。这样，我们也有机会展示一些实用的类设计技术。

9.1 用户自定义类型
9.2 类与成员
9.3 接口与实现
9.4 演化一个类
9.4.1 `struct` 与函数
9.4.2 成员函数与构造函数
9.4.3 将细节保持私有
9.4.4 定义成员函数
9.4.5 引用当前对象
9.4.6 报告错误
9.5 枚举
9.5.1 “普通”枚举
9.6 运算符重载
9.7 类接口
9.7.1 实参类型
9.7.2 拷贝
9.7.3 默认构造函数
9.7.4 `const` 成员函数
9.7.5 成员与“辅助函数”
9.8 **Date** 类

9.1 用户自定义类型

C++ 语言为你提供了一些内置类型（built-in type），例如 `char`、**int** 和 `double`（§A.8）。如果编译器无需程序员在源代码中通过声明来告知，就知道如何表示该类型的对象、以及可以对其进行哪些运算（例如 `+` 和 `*`），那么这个类型就称为内置的。

非内置的类型称为用户自定义类型（user-defined type，UDT）。它们可以是标准库类型——作为每个 ISO 标准 C++ 实现的一部分，对所有 C++ 程序员可用——例如 `string`、`vector` 和 `ostream`（第 10 章）；也可以是我们自己构造的类型，例如 **Token** 和 `Token_stream`（§6.5 和 §6.6）。一旦掌握了必要的技术细节，我们还会构造诸如图形类型 **Shape**、**Line** 和 **Text**（第 13 章）。标准库类型与内置类型一样，都是语言的一部分，但我们仍将其视为用户自定义的，因为它们与你自己构造类型所用的基本构件和技术完全相同；标准库的实现者并没有任何你所没有的特殊特权或设施。与内置类型一样，大多数用户自定义类型都提供运算。例如，`vector` 有 `[]` 和 `size()`（§4.6.1、§B.4.8），`ostream` 有 `<<`，`Token_stream` 有 `get()`（§6.8），**Shape** 有 `add(Point)` 和 `set_color()`（§14.2）。

我们为什么要构造类型？编译器并不了解我们可能想在程序中使用的所有类型。它也不可能了解，因为有用的类型实在太多——任何语言设计者或编译器实现者都不可能全都知晓。我们每天都在发明新类型。为什么？类型究竟有什么用？类型的价值在于能直接地在代码中表达思想。编写代码时，理想情况是把我们的思想直接呈现在代码中，使得我们自己、我们的同事以及编译器都能理解我们所写的内容。当想做整数算术时，**int** 帮助很大；当想处理文本时，`string` 帮助很大；当想处理计算器的输入时，**Token** 和 `Token_stream` 帮助很大。这种帮助体现在两个方面：

- 表示（representation）：类型“知道”如何在对象中表示所需的数据。
- 运算（operations）：类型“知道”可以对对象施加哪些运算。

许多概念都遵循这一模式：“某个东西”拥有表示其当前值——有时称为当前状态（current state）——的数据，以及一组可施加的运算。想想计算机文件、网页、烤面包机、音乐播放器、咖啡杯、汽车发动机、手机、电话簿：它们都可以由一些数据来刻画，并且都有一组或多或少固定的标准操作可供你执行。在每种情形下，运算的结果都取决于对象的“当前状态”。

于是，我们希望在代码中把这样的“想法”或“概念”表示为一个数据结构加上一组函数。问题是：“究竟怎么做？”本章就介绍在 C++ 中实现这一点的基本方法的技术细节。

C++ 提供两类用户自定义类型：类和枚举。类是迄今为止最通用、最重要的，所以我们首先聚焦于类。类直接地在程序中表达一个概念。类（class）是这样一种（用户自定义）类型：它规定了该类型的对象如何表示、如何创建、如何使用以及如何销毁（参见 §17.5）。如果你把某个事物看作一个独立的实体，那么你可能就应该定义一个类，在程序中表示这个“东西”。例如 `vector`、矩阵、输入流、`string`、FFT（快速傅里叶变换）、阀门控制器、机械臂、设备驱动程序、屏幕上的图片、对话框、图、窗口、温度读数和时钟。

在 C++ 中（大多数现代语言也是如此），类是大型程序的关键构件——正如我们在计算器（第 6 章和第 7 章）中所见，它对小型程序也非常有用。

9.2 类与成员

类是一种用户自定义类型。它由内置类型、其他用户自定义类型以及函数组成。用于定义类的这些组成部分称为成员（member）。一个类可以有零个或多个成员。例如：

```
class X {
public:
    int m;                                          // 数据成员
    int mf(int v) { int old = m; m=v; return old; }     // 函数成员
};
```

成员可以是各种类型。大多数成员要么是数据成员（data member）——定义类的对象的表示；要么是函数成员（function member）——提供对这类对象的运算。我们使用对象.成员（object.member）记法来访问成员。例如：

```
X var;                  // var 是一个 X 类型的变量
var.m = 7;              // 对 var 的数据成员 m 赋值
int x = var.mf(9);      // 调用 var 的成员函数 mf()
```

你可以把 `var.m` 读作“var 的 m”。大多数人把它念作“var 点 m”或“var 的 m”。成员的类型决定了我们能对它做什么运算。我们可以读写一个 **int** 成员、调用一个函数成员，等等。

成员函数（例如 X 的 `mf()`）不需要使用 `var.m` 这种记法。它可以直接使用成员名（本例中是 **m**）。在成员函数内，成员名指的是为其调用该成员函数的那个对象中同名的成员。因此，在调用 `var.mf(9)` 中，`mf()` 定义里的 **m** 指的是 `var.m`。

9.3 接口与实现

通常，我们认为一个类由接口（interface）加实现（implementation）构成。接口是类的声明中其用户直接访问的那部分；实现是类的声明中用户只能通过接口间接访问的那部分。公共接口用标签 `public:` 标识，实现用标签 `private:` 标识。你可以这样想象一个类的声明：

```
class X {       // 这个类的名字是 X
public:
    // 公共成员：
    //     -- 对用户的接口（所有人可访问）
    // 函数
    // 类型
    // 数据（通常最好保持私有）
private:
    // 私有成员：
    //     -- 实现细节（仅本类的成员可使用）
    // 函数
    // 类型
    // 数据
};
```

类的成员默认是私有的；也就是说，

```
class X {
    int mf(int);
    // . . .
};
```

等价于

```
class X {
private:
    int mf(int);
    // . . .
};
```

于是，

```
X x;                // X 类型的变量 x
int y = x.mf();     // 错误：mf 是私有的（即不可访问）
```

用户不能直接引用私有成员。相反，我们必须经由一个能使用它的公共函数。例如：

```
class X {
    int m;
    int mf(int);
public:
    int f(int i) { m=i; return mf(i); }
};

X x;
int y = x.f(2);
```

我们用 private 和 public 来表达接口（类的使用者眼中的类）与实现细节（类的实现者眼中的类）之间的重要区别。随着讲解的推进，我们会解释这一点并给出大量例子。这里只想提一句：对于纯粹只是数据的东西，这种区分没有意义。因此，对于没有私有实现细节的类，有一种有用的简化记法。`struct` 就是一种成员默认为公有的类：

```
struct X {
    int m;
    // . . .
};
```

等价于

```
class X {
public:
    int m;
    // . . .
};
```

`struct` 主要用于那些成员可以取任意值的数据结构；也就是说，我们无法为它定义任何有意义的不变量（§9.4.3）。

9.4 演化一个类

下面我们通过展示如何——以及为什么——把一个简单的数据结构演化成一个带有私有实现细节和配套运算的类，来说明支持类的语言设施和使用类的基本技术。我们用一个看起来微不足道的问题作为例子：如何在程序中表示一个日期（例如 1954 年 8 月 14 日）。很多程序都需要日期（商业交易、气象数据、日历程序、工作记录、库存管理等），这一点显而易见。唯一的问题是如何表示它们。

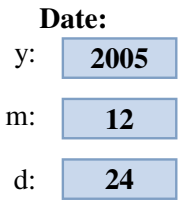
9.4.1 `struct` 与函数

我们会如何表示一个日期？如果被问到，多数人会回答：“嗯，用年份、月份，再加上月份中的第几天，怎么样？”这不是唯一的答案，也不总是最好的答案，但对我们的用途而言已经足够好了，我们就这么做。我们的第一次尝试是一个简单的 `struct`：

```
// 简单的 Date（过于简单？）
struct Date {
    int y;      // 年
    int m;      // 年中的月
    int d;      // 月中的日
};

Date today;     // 一个 Date 变量（一个有名字的对象）
```

一个 **Date** 对象（例如 today）其实就只是三个 **int**：



任何地方都不存在依赖于与 **Date** 相关的隐藏数据结构的“魔法”——对本章中每一个版本的 **Date** 而言，情况都将如此。

那么，我们现在有了 **Date**，能对它们做什么呢？从“我们可以访问 today（以及任何其他 **Date**）的成员并随意读写”这个意义上说，什么都能做。问题在于，没有什么真正便利的用法。我们想对 **Date** 做的几乎所有事情，都不得不用对这些成员的读写来表达。例如：

```
// 把 today 置为 2005 年 12 月 24 日
today.y = 2005;
today.m = 24;
today.d = 12;
```

这既繁琐又容易出错。你发现错误了吗？凡是繁琐的东西都容易出错！例如，下面这些有道理吗？

```
Date x;
x.y = -3;
x.m = 13;
x.d = 32;
```

大概没有，也没有人会这么写——真的吗？那么这个呢：

```
Date y;
y.y = 2000;
y.m = 2;
y.d = 29;
```

2000 年是闰年吗？你确定吗？

我们要做的，是提供一些辅助函数（helper function）来替我们完成最常见的运算。这样，我们就不必一遍又一遍地重复同样的代码，也不会一遍又一遍地犯、发现并修复同样的错误。对几乎每一个类型来说，初始化和赋值都是最常见的运算。对 **Date** 而言，递增 **Date** 的值是另一种常见运算，所以我们写出

```
// 辅助函数:

void init_day(Date& dd, int y, int m, int d)
{
    // 检查 (y,m,d) 是否是一个有效日期
    // 如果是，就用它初始化 dd
}

void add_day(Date& dd, int n)
{
    // 将 dd 增加 n 天
}
```

现在我们可以试着使用 **Date** 了：

```
void f()
{
    Date today;
    init_day(today, 12, 24, 2005);    // 糟糕！(12 年中没有 2005 日)
    add_day(today,1);
}
```

首先我们注意到这类“运算”——这里以辅助函数的形式实现——的用处。检查一个日期是否有效是相当困难且繁琐的事情，如果我们不一次性写好一个检查函数，就会偶尔跳过检查，得到充满 bug 的程序。无论何时定义一个类型，我们都会需要它的一些运算。我们需要多少运算、需要哪种运算会各有不同；我们提供它们的方式（作为函数、成员函数或运算符）也会各有不同。但无论何时，当我们决定提供一个类型时，都应自问：“对这种类型，我们希望有哪些运算？”

9.4.2 成员函数与构造函数

我们为 **Date** 提供了一个初始化函数，它对 **Date** 的有效性进行了一项重要检查。然而，如果我们不去使用检查函数，它们就没什么用。例如，假设我们已经为 **Date** 定义了输出运算符 <<（§9.8）：

```
void f()
{
    Date today;
    // . . .
    cout << today << '\n';        // 使用 today
    // . . .
    init_day(today,2008,3,30);
    // . . .
    Date tomorrow;
    tomorrow.y = today.y;
    tomorrow.m = today.m;
    tomorrow.d = today.d+1;        // 给 today 加 1
    cout << tomorrow << '\n';      // 使用 tomorrow
}
```

这里，我们“忘了”立即初始化 today，而在我们还没来得及调用 init_day() 之前，“某人”就使用了它。“另一个人”认为调用 add_day() 是浪费时间——或者根本没听说过它——于是手工构造出了 tomorrow。碰巧，这是糟糕的代码——非常糟糕的代码。有时（很可能多数时候）它能正常工作，但微小的改动就会导致严重的错误。例如，写出一个未初始化的 **Date** 会产生乱码输出；而简单地给成员 **d** 加 1 来递增一天，则是一枚定时炸弹：当 today 是当月最后一天时，递增会得到一个无效日期。这段“非常糟糕的代码”最糟糕的一点在于：它看起来并不糟。

这样的思考引出了对一种不会被遗忘的初始化函数，以及对一些更不容易被忽视的运算的需求。实现这一点的基本工具就是成员函数（member function），也就是在类体中声明为类的成员的函数。例如：

```
// 简单的 Date
// 用构造函数保证初始化
// 提供一些记法上的便利
struct Date {
    int y, m, d;                    // 年、月、日
    Date(int y, int m, int d);      // 检查日期有效性并初始化
    void add_day(int n);            // 将 Date 增加 n 天
};
```

与其类同名的成员函数很特殊。它被称为构造函数（constructor），将用于该类对象初始化（“构造”）。对于一个拥有需要实参的构造函数的类，忘记初始化它的对象是一个错误——编译器会捕获这个错误——而且语言为这种初始化提供了一种特别的便利语法：

```
Date my_birthday;                   // 错误：my_birthday 未初始化
Date today {12,24,2007};            // 糟糕！运行时错误
Date last {2000,12,31};             // 正确（口语化风格）
Date next = {2014,2,14};            // 也正确（略显繁琐）
Date christmas = Date{1976,12,24};  // 也正确（繁琐风格）
```

声明 my_birthday 的尝试之所以失败，是因为我们没有指定必需的初始值。声明 today 的尝试能通过编译器，但构造函数中的检查代码会在运行时捕获这个非法日期（{12,24,2007}——12 年的第 24 个月没有 2007 日）。

last 的定义把初始值——即 **Date** 的构造函数所要求的实参——作为紧随变量名之后的 {} 列表提供。对于拥有需要实参的构造函数的类，这是变量最常用的初始化风格。我们也可以使用更啰嗦的风格：先显式创建一个对象（这里是 **Date{1976,12,24}**），再用 = 初始化语法以它初始化变量。除非你真的喜欢打字，否则你很快就会厌倦这种写法。

现在我们可以试着使用新定义的变量了：

```
last.add_day(1);
add_day(2);                 // 错误：哪个日期？
```

注意，成员函数 add_day() 要用点成员访问记法针对某个特定的 **Date** 调用。我们将在 §9.4.4 展示如何定义成员函数。

在 C++98 中，人们用圆括号来定界初始化列表，所以你会见到很多这样的代码：

```
Date last(2000,12,31);          // 正确（旧式口语化风格）
```

我们更推荐用 {} 作为初始化列表，因为它清楚地表明初始化（构造）在何时进行，也因为这种记法的适用面更广。这种记法还可用于内置类型。例如：

```
int x {7};                      // 正确（现代初始化列表风格）
```

还可以选择在 {} 列表之前加一个 =：

```
Date next = {2014,2,14};        // 也正确（略显繁琐）
```

有些人觉得这种新旧风格结合的写法可读性更好。

9.4.3 将细节保持私有

我们仍然有一个问题：如果有人忘了使用成员函数 add_day() 怎么办？如果有人决定直接修改月份怎么办？毕竟，我们“忘了”为此提供设施：

```
Date birthday {1960,12,31};     // 1960 年 12 月 31 日
++birthday.d;                   // 哎哟！无效日期
                                //  （birthday.d==32 使 today 无效）

Date today {1970,2,3};
today.m = 14;                   // 哎哟！无效日期
                                //  （today.m==14 使 today 无效）
```

只要我们让 **Date** 的表示对所有人都可访问，就会有人——出于意外或故意——把它弄乱；也就是说，有人会做出产生无效值的操作。在本例中，我们创建了一个 **Date**，其值并不对应日历上的任何一天。这样的无效对象就是定时炸弹；用不了多久，某个不知情的人就会用到这个无效值，从而得到运行时错误，或者——往往更糟——产生错误的结果。

这样的顾虑让我们得出结论：**Date** 的表示应当对用户不可访问，除非通过我们提供的公共成员函数。下面是第一步：

```
// 简单的 Date（控制访问）
class Date {
    int y, m, d;                    // 年、月、日
public:
    Date(int y, int m, int d);      // 检查日期有效性并初始化
    void add_day(int n);            // 将 Date 增加 n 天
    int month() { return m; }
    int day() { return d; }
    int year() { return y; }
};
```

我们可以这样使用它：

```
Date birthday {1970, 12, 30};          // 正确
birthday.m = 14;                       // 错误：Date::m 是私有的
cout << birthday.month() << '\n';      // 我们提供了一种读取 m 的途径
```

“有效的 **Date**”这一概念，是有效值思想的一个重要特例。我们力求这样设计类型：值保证有效；也就是说，我们隐藏表示，提供一个只创建有效对象的构造函数，并设计所有成员函数——它们假定接收有效值，并且返回时只留下有效值。对象的值常被称为它的状态（state），因此有效值的思想也常被称为对象的有效状态（valid state）。

另一种做法是我们每次使用对象时都检查有效性，或者干脆指望没人留下无效值。经验表明，“指望”能带来“还不错”的程序。然而，产出“还不错”、但偶尔给出错误结果、偶尔崩溃的程序，并不是作为一个专业人士赢得朋友和尊重的方式。我们更愿意编写能被证明正确的代码。

规定什么构成有效值的规则称为不变量（invariant）。**Date** 的不变量（“一个 **Date** 必须表示过去、现在或未来的某一天”）出奇地难以精确陈述：要考虑闰年、公历（格里高利历）、时区等。不过，对于简单而实际的 **Date** 用途，我们能够做到。例如，如果我们分析的是互联网日志，就不必为公历、儒略历或玛雅历操心。如果我们想不出一个好的不变量，那么我们面对的多半就是纯粹的数据。如果是这样，就用 `struct`。

9.4.4 定义成员函数

到目前为止，我们都是从接口设计者和用户的角度考察 **Date**。但迟早，我们必须实现那些成员函数。首先，下面是 **Date** 类的一个子集，按“公共接口在前”这一常见风格重新组织：

```
// 简单的 Date（有些人喜欢把实现细节放在后面）
class Date {
public:
    Date(int y, int m, int d);  // 构造函数：检查日期有效性并初始化
    void add_day(int n);        // 将 Date 增加 n 天
    int month();
    // . . .
private:
    int y, m, d;                // 年、月、日
};
```

人们把公共接口放在前面，因为接口才是大多数人感兴趣的部分。原则上，用户无需查看实现细节。而实际上，我们通常都会好奇地快速瞄一眼，看看实现是否合理，实现者是否用了什么值得我们学习的技巧。不过，除非我们就是实现者，否则我们花在公共接口上的时间往往多得多。编译器并不在乎类的函数成员和数据成员的顺序；它按你给出的任意顺序接受这些声明。

当我们在类外定义一个成员时，需要说明它是哪个类的成员。我们使用 *类名::* *成员名*（class_name::member_name）记法：

```
Date::Date(int yy, int mm, int dd)    // 构造函数
    :y{yy}, m{mm}, d{dd}              // 注意：成员初始化器
{
}

void Date::add_day(int n)
{
    // . . .
}


int month()               // 糟糕：我们忘了 Date::
{
    return m;             // 不是成员函数，无法访问 m
}
```

记法 `:y{yy}, m{mm}, d{dd}` 就是我们初始化成员的方式。它称为（成员）初始化列表（(member) initializer list）。我们本可以写成

```
Date::Date(int yy, int mm, int dd)    // 构造函数
{
    y = yy;
    m = mm;
    d = dd;
}
```

但那样的话，原则上我们会先对成员进行默认初始化，然后再为它们赋值。这还会带来在成员初始化之前意外使用它的可能。记法 `:y{yy}, m{mm}, d{dd}` 更直接地表达了我们的意图。这个区别与下面的区别完全一样：

```
int x;        // 先定义变量 x
// . . .
x = 2;        // 之后再对 x 赋值
```

与

```
int x {2};    // 定义并立即用 2 初始化
```

我们也可以直接在类定义中定义成员函数：

```
// 简单的 Date（有些人喜欢把实现细节放在后面）
class Date {
public:
    Date(int yy, int mm, int dd)
            :y{yy}, m{mm}, d{dd}
    {
        // . . .
    }

    void add_day(int n)
    {
        // . . .
    }

    int month() { return m; }

    // . . .
private:
    int y, m, d;    // 年、月、日
};
```

我们首先注意到，类的声明变大了，也变得“凌乱”了。在本例中，构造函数和 **add_day()** 的代码各自都可能达到十几行甚至更多。这会使类声明大出好几倍，也使人们更难从实现细节中找出接口。因此，我们不在类声明中定义大型函数。

不过，再看看 **month()** 的定义。它很直白，而且比把 Date::month() 放到类声明之外的版本更短。对于这样短小、简单的函数，我们可以考虑直接把定义写在类声明中。

注意，**month()** 可以引用 **m**，即使 **m** 是在 **month()** 之后（下方）定义的。成员可以引用其类的函数或数据成员，而与该成员在类中声明的位置无关。“名字必须先声明后使用”这条规则，在类的有限作用域内是放宽的。

把成员函数的定义写在类定义中有三个效应：

- 函数将成为内联的（inline）；也就是说，编译器会尽量在每次调用点生成该函数的代码，而不是用函数调用指令去使用公共代码。对于像 **month()** 这样几乎不做什么事却被大量使用的函数，这可能是显著的性能优势。
- 每当我们修改了某个内联函数的函数体，该类的所有使用处都必须重新编译。如果函数体在类声明之外，则只有当类声明本身改变时才需要重新编译用户代码。在大型程序中，函数体改变时不重新编译可能是巨大的优势。
- 类定义会变大。因此，可能更难从成员函数定义中找出类的成员。

显而易见的经验法则是：不要把成员函数体放进类声明，除非你确知需要从内联微型函数中获得性能提升。大型函数（比如说五行或更多行代码）无法从内联中受益，还会让类声明更难读。我们很少内联由多于一个或两个表达式构成的函数。

9.4.5 引用当前对象

考虑一下对目前为止的 **Date** 类的一个简单使用：

```
class Date {
    // . . .
    int month() { return m; }
    // . . .
private:
    int y, m, d;    // 年、月、日
};

void f(Date d1, Date d2)
{
    cout << d1.month() << ' ' << d2.month() << '\n';
}
```

`Date::month()` 如何知道第一次调用要返回 `d1.m` 的值、第二次调用要返回 `d2.m` 的值？再看一眼 `Date::month()`：它的声明没有指定任何函数实参！`Date::month()` 是怎么知道它是为哪个对象而被调用的？类成员函数（如 `Date::month()`）有一个隐式实参，它用这个实参来标识为其调用的对象。因此，在第一次调用中，**m** 正确地引用 `d1.m`；在第二次调用中，它引用 `d2.m`。关于这个隐式实参的更多用法，参见 §17.10。

9.4.6 报告错误

当我们发现一个无效日期时该怎么办？我们应该在代码中的哪里寻找无效日期？从 §5.6 我们知道，第一个问题的答案是“抛出异常”，而显而易见的查找位置就是我们最初构造 **Date** 的地方。如果我们不创建无效的 **Date**，并且正确编写成员函数，就永远不会拥有带无效值的 **Date**。所以，我们要防止用户创建处于无效状态的 **Date**：

```
// 简单的 Date（防止无效日期）
class Date {
public:
    class Invalid { };                  // 用作异常
    Date(int y, int m, int d);          // 检查日期有效性并初始化
    // . . .
private:
    int y, m, d;                        // 年、月、日
    bool is_valid();                    // 如果日期有效则返回 true
};
```

我们把有效性检查放进一个单独的 `is_valid()` 函数，因为检查有效性与初始化在逻辑上是两回事，而且我们可能想要多个构造函数。如你所见，我们既可以有私有数据，也可以有私有函数：

```
Date::Date(int yy, int mm, int dd)
    : y{yy}, m{mm}, d{dd}              // 初始化数据成员
{
    if (!is_valid()) throw Invalid{}; // 检查有效性
}

bool Date::is_valid()                  // 如果日期有效则返回 true
{
    if (m<1 || 12<m) return false;
    // . . .
}
```

有了 **Date** 的这个定义，我们可以写出

```
void f(int x, int y)
try {
    Date dxy {2004,x,y};
    cout << dxy << '\n';               // << 的声明见 §9.8
    dxy.add_day(2);
}
catch(Date::Invalid) {
    error("invalid date");             // error() 定义于 §5.6.3
}
```

现在我们知道，`<<` 和 `add_day()` 都会有一个有效的 **Date** 可供操作。

在 §9.7 中完成 **Date** 类的演化之前，我们先绕点路，介绍两个我们为此需要的通用语言设施：枚举和运算符重载。

9.5 枚举

枚举（enum，enumeration）是一种非常简单的用户自定义类型，它把自己的值集合（其枚举符，enumerator）规定为符号常量。例如：

```
enum class Month {
    jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};
```

枚举的“体”就是其枚举符的列表。**enum class** 中的 `class` 表示这些枚举符位于该枚举的作用域内。也就是说，要引用 **jan**，我们必须写 `Month::jan`。

你可以像这里对 **jan** 所做的那样，为某个枚举符指定一个特定的表示值，也可以让编译器去挑选合适的值。如果你让编译器挑选，它会为每个枚举符赋予前一个枚举符的值加一。因此，我们对 **Month** 的定义让各个月份从 **1** 开始取连续的值。我们本可以等价地写成

```
enum class Month {
    jan=1, feb=2, mar=3, apr=4, may=5, jun=6,
    jul=7, aug=8, sep=9, oct=10, nov=11, dec=12
};
```

然而，那样很繁琐，而且给了出错的机会。事实上，在得到现在这个正确版本之前，我们就打错了两次字；最好让编译器去做简单、重复的“机械性”工作。编译器比我们更擅长这类任务，而且它不会觉得无聊。

如果我们不初始化第一个枚举符，计数从 **0** 开始。例如：

```
enum class Day {
    monday, tuesday, wednesday, thursday, friday, saturday, sunday
};
```

这里 monday 表示为 **0**，sunday 表示为 **6**。实践中，从 **0** 开始常常是个好选择。

我们可以这样使用我们的 **Month**：

```
Month m = Month::feb;

Month m2 = feb;              // 错误：feb 不在作用域内
m = 7;                       // 错误：不能把 int 赋给 Month
int n = m;                   // 错误：不能把 Month 赋给 int
Month mm = Month(7);         // 把 int 转换为 Month（未检查）
```

Month 与其“底层类型”**int** 是不同的类型。每个 **Month** 都有一个等价的整数值，但大多数 **int** 并没有等价的 **Month**。例如，我们确实希望下面这个初始化失败：

```
Month bad = 9999;            // 错误：不能把 int 转换为 Month
```

如果你坚持使用 **Month(9999)** 这种记法，后果自负！在很多情况下，当程序员明确坚持时，C++ 并不会阻止他做潜在愚蠢的事；毕竟，程序员也许真的更清楚。注意，你不能使用 **Month{9999}** 这种记法，因为它只允许那些能用于 **Month** 初始化的值，而 **int** 不行。

遗憾的是，我们不能为枚举定义构造函数来检查初始化值，但写一个简单的检查函数是轻而易举的：

```
Month int_to_month(int x)
{
    if (x<int(Month::jan) || int(Month::dec)<x) error("bad month");
    return Month(x);
}
```

我们用 int(Month::jan) 记法来获得 Month::jan 的 **int** 表示。有了它，我们可以写

```
void f(int m)
{
    Month mm = int_to_month(m);
    // . . .
}
```

我们用枚举来做什么？从根本上说，每当我们需要一组相关的具名整数常量时，枚举就有用。当我们试图表示一组候选项（上、下；是、否、也许；开、关；北、东北、东、东南、南、西南、西、西北）或一组有区别的值（红、蓝、绿、黄、栗色、绯红、黑）时，这种情况随处可见。

9.5.1 “普通”枚举

除了 **enum class**（也称为作用域枚举，scoped enumeration）之外，还有“普通”（plain）枚举。它们与作用域枚举的差别在于：会把枚举符隐式“导出”到枚举所在的作用域，并允许向 **int** 的隐式转换。例如：

```
enum Month {                    // 注意：没有 class
    jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};

Month m = feb;                  // 正确：feb 在作用域内
Month m2 = Month::feb;          // 也正确
m = 7;                          // 错误：不能把 int 赋给 Month

int n = m;                      // 正确：我们可以把 Month 赋给 int
Month mm = Month(7);            // 把 int 转换为 Month（未检查）
```

显然，“普通”enum 不如 **enum class** 严格。它们的枚举符会“污染”定义枚举符的作用域。这可以是一种便利，但偶尔也会带来意外。例如，如果你试图把这个 **Month** 与 iostream 的格式化机制（§11.2.1）一起使用，你会发现表示十二月的 **dec** 与表示十进制的 **dec** 冲突了。

类似地，让枚举值能转换为 **int** 可以是一种便利（当我们想要转换到 **int** 时，它省去了显式写出转换的麻烦），但偶尔也会带来意外。例如：

```
void my_code(Month m)
{
    If (m==17) do_something();              // 嗯：第 17 个月？
    If (m==monday) do_something_else();     // 嗯：拿月份和
                                            // 星期一比较？
}
```

如果 **Month** 是一个 **enum class**，两个条件都无法通过编译。如果 monday 是“普通”enum 的枚举符（而非 **enum class** 的），把月份与星期一比较就能通过编译，其结果多半不合人意。

请优先使用更简单、更安全的 **enum class**，而不是“普通”enum，但要有心理准备会在旧代码中见到“普通”enum：**enum class** 是 C++11 中新出现的。

9.6 运算符重载

对于类或枚举类型的运算对象，你可以定义几乎所有的 C++ 运算符。这常被称为运算符重载（operator overloading）。当我们想为所设计的类型提供惯用记法时，就会用到它。例如：

```
enum class Month {
    Jan=1, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec
};

Month operator++(Month& m)                  // 前置递增运算符
{
    m = (m==Dec) ? Jan : Month(int(m)+1);   // 循环回绕
    return m;
}
```

? : 结构是一个“算术 if”：若 (m==Dec) 为真，**m** 变成 **Jan**，否则变成 **Month(int(m)+1)**。这是表达“月份在十二月之后循环回绕”这一事实的一种相当优雅的方式。现在可以像这样使用 **Month** 类型了：

```
Month m = Sep;
++m;          // m 变成 Oct
++m;          // m 变成 Nov
++m;          // m 变成 Dec
++m;          // m 变成 Jan (循环回绕)
```

你可能觉得递增一个 **Month** 还不至于常见到需要一个专门的运算符。也许是这样，那么输出运算符如何？我们可以这样定义一个：

```
vector<string> month_tbl;

ostream& operator<<(ostream& os, Month m)
{
    return os << month_tbl[int(m)];
}
```

这里假定 month_tbl 已在某处被初始化，使得（例如）month_tbl[int(Month::mar)] 是 "March" 或该月份的某个其他合适名字；参见 §10.11.3。

对于你自己的类型，C++ 提供的运算符你几乎都能定义，但只能定义已有的运算符，例如 +、-、*、/、%、[]、()、^、!、&、<、<=、> 和 >=。你不能定义自己的运算符；你也许希望你的程序里有 ** 或 $= 这样的运算符，但 C++ 不会允许。你只能按运算符惯常的操作数个数来定义它们；例如，你可以定义一元 -，但不能定义一元 <=（小于等于），可以定义二元 +，但不能定义二元 !（非）。基本上，语言允许你把已有的语法用于你自己定义的类型，但不允许你扩展语法本身。

重载的运算符必须至少有一个运算对象是用户自定义类型：

```
int operator+(int,int);     // 错误：你不能重载内置的 +
Vector operator+(const Vector&, const Vector &);     // 正确
Vector operator+=(const Vector&, int);               // 正确
```

一般而言，除非你确实确信为某个类型定义运算符会给代码带来很大的积极改变，否则最好不要定义。而且，只按运算符的惯常含义来定义它们：+ 应该表示加法，二元 *

乘法、[] 下标访问、() 调用等。这只是建议，不是语言规则，但它是好建议：按惯例使用运算符，例如用 + 表示加法，能显著帮助我们理解程序。毕竟，这种用法是数百年数学记法经验的结晶。反过来，晦涩的运算符和不合惯例的运算符用法会带来严重的干扰，也是错误之源。这一点我们不再展开。在接下来的章节中，我们只会在少数我们认为合适的地方使用运算符重载。

注意，最值得重载的运算符并不像人们常以为的那样是 +、-、* 和 /，而是 =、==、!=、<、[]（下标）和 ()（调用）。

9.7 类接口

我们已经论证过，一个类的公共接口部分和实现部分应当分开。只要保留对“纯旧数据”（plain old data）类型使用 struct 的可能性，很少有专业人士会反对。然而，怎样设计一个好的接口？好的公共接口与一团糟的接口的区别何在？这部分答案只能通过例子给出，但也有一些一般性的原则可以列举出来，而且 C++ 对其中若干条提供了支持：

- 保持接口完整。
- 保持接口最小。
- 提供构造函数。
- 支持拷贝（或禁止拷贝）（见 §14.2.4）。
- 使用类型进行良好的实参检查。
- 标识出不修改对象的成员函数（见 §9.7.4）。
- 在析构函数中释放所有资源（见 §17.5）。

另见 §5.5（如何检测和报告运行时错误）。

前两条原则可以概括为“让接口尽可能小，但不能更小”。我们希望接口小，因为小的接口易学易记，实现者也不会把大量时间浪费在实现不必要的、很少使用的设施上。小的接口还意味着，当出现问题时，只需检查少数几个函数就能找到问题所在。平均而言，公共成员函数越多，查找 bug 就越困难——关于调试带有公有数据的类的复杂性，就请别让我们多说了。但当然，我们希望接口是完整的；否则它就毫无用处。如果一个接口不允许我们做真正需要做的一切，我们就无法使用它。

接下来看看另外一些不那么抽象、而且得到更直接支持的理想。

9.7.1 实参类型

在 §9.4.3 中定义 Date 的构造函数时，我们用了三个 int 作为实参。这带来了一些问题：

```
Date d1 {4,5,2005};      // 糟糕：年 4，日 2005
Date d2 {2005,4,5};      // 4 月 5 日还是 5 月 4 日？
```

第一个问题（日号非法）很容易在构造函数中用一次检查解决。然而，第二个问题（月份与日号的混淆）靠用户写的代码是抓不住的。第二个问题的根源只在于月份和日号的书写惯例不同：例如，4/5

在美国是 4 月 5 日，在英国则是 5 月 4 日。既然我们无法算出答案，就必须另想办法。显而易见的办法是使用一个 Month 类型：

```
enum class Month {
    jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};


// 简单的 Date (使用 Month 类型)
class Date {
public:
    Date(int y, Month m, int d);    // 检查日期有效性并初始化
    // . . .
private:
    int y;                          // 年
    Month m;
    int d;                          // 日
};
```

当我们使用 Month 类型时，如果我们把月份和日对调，编译器就会抓住我们；而且用枚举作为 Month 类型还为我们提供了符号名。读写符号名通常比摆弄数字更容易，因此也更不容易出错：

```
Date dx1 {1998, 4, 3};              // 错误：第 2 个实参不是 Month
Date dx2 {1998, 4, Month::mar};     // 错误：第 2 个实参不是 Month
Date dx2 {4, Month::mar, 1998};     // 糟糕：运行时错误：日为 1998
Date dx2 {Month::mar, 4, 1998};     // 错误：第 2 个实参不是 Month
Date dx3 {1998, Month::mar, 30};    // 正确
```

这就处理了大多数“意外”。注意枚举符 mar 用枚举名限定的写法：Month::mar。我们不写 Month.mar，因为 Month 不是对象（它是一个类型），而 mar 也不是数据成员（它是一个枚举符——符号常量）。在类名、枚举名或命名空间名（§8.7）之后用 ::，在对象名之后用 .（点）。

有选择余地时，我们宁可在编译时捕获错误，而不是在运行时。我们更希望编译器替我们找出错误，而不是靠我们自己费劲地弄清问题究竟发生在代码中的哪个位置。而且，编译时捕获的错误不需要编写和执行检查代码。

照这个思路，我们能不能也把“日号与年份对调”的错误抓出来呢？可以，但解决办法不像对 Month 那样简单优雅；毕竟历史上确实有过 4 年，你也许会想表示它。即使把自己限制在现代，相关的年份也可能太多，无法在一个枚举中全部列出。

（在对 Date 的预期用途知之不多的情形下）我们大概能做的最好的事情是这样的：

```
class Year {                                        // [min:max) 范围内的年
    static const int min = 1800;
    static const int max = 2200;
public:
    class Invalid { };
    Year(int x) : y{x} { if (x<min || max<=x) throw Invalid{}; }
    int year() { return y; }
private:
    int y;
};


class Date {
public:
    Date(Year y, Month m, int d);   // 检查日期有效性并初始化
    // . . .
private:
    Year y;
    Month m;
    int d;                          // 日
};
```

现在我们得到

```
Date dx1 {Year{1998}, 4, 3};            // 错误：第 2 个实参不是 Month
Date dx2 {Year{1998}, 4, Month::mar};   // 错误：第 2 个实参不是 Month
Date dx2 {4, Month::mar, Year{1998}};   // 错误：第 1 个实参不是 Year
Date dx2 {Month::mar, 4, Year{1998}};   // 错误：第 2 个实参不是 Month
Date dx3 {Year{1998}, Month::mar, 30};  // 正确
```

下面这个怪异且不太可能发生的错误，仍然要等到运行时才会被抓住：

```
Date dx2 {Year{4}, Month::mar, 1998};   // 运行时错误：Year::Invalid
```

为检查年份而付出的额外工作和记法值得吗？自然，这取决于你用 Date 所解问题受到的约束，但在本例中我们持怀疑态度，因此后面不再使用 Year 类。

编程时，我们总是要自问：对一个给定的应用来说，怎样才算足够好？在已经找到一个足够好的方案之后，我们通常没有“永远”搜下去以寻求完美方案的奢侈。再搜下去，我们甚至可能想出某个精巧过头、反而不如最初简单方案的东西。这正是“完美是好的敌人”（伏尔泰）这句名言的一种含义。

注意 min 和 max 定义中对 static const 的使用。这就是我们在类内定义整数类型符号常量的方式。对于类成员，我们用 static 来确保该值在整个程序中只有一份副本，而不是类的每个对象各有一份。在本例中，由于初始化器是常量表达式，我们本可以改用 constexpr 而不是 const。

9.7.2 拷贝

我们总是要创建对象；也就是说，必须始终考虑初始化和构造函数。可以说它们是一个类最重要的成员：要写好它们，你必须决定初始化一个对象需要什么、一个值怎样才算有效（不变量是什么?）。仅仅思考初始化这件事，就能帮你避免错误。

接下来要考虑的问题往往是：我们能拷贝自己的对象吗？如果能，怎么拷贝？

对 Date 或 Month 来说，答案是我们显然想拷贝该类型的对象，而且拷贝的含义很平常：把所有成员逐一拷贝即可。实际上，这正是默认情形。只要你不另行说明，编译器就会照此办理。例如，当你把一个 Date 作为初始化器或赋值的右值来拷贝时，它的所有成员都会被拷贝：

```
Date holiday {1978, Month::jul, 4};      // 初始化
Date d2 = holiday;
Date d3 = Date{1978, Month::jul, 4};
holiday = Date{1978, Month::dec, 24};    // 赋值
d3 = holiday;
```

这一切都会如预期般工作。Date{1978, Month::dec, 24} 这种记法会创建一个合适的无名 Date 对象，随后你可以恰当地使用它。例如：

```
cout << Date{1978, Month::dec, 24};
```

这是构造函数的一种用法，其作用很像类类型的字面值。它常常是“先定义一个变量或 const、然后只用一次”这一做法的便捷替代。

如果我们不想要拷贝的默认含义怎么办？我们可以自己定义（见 §18.3），或者删除（delete）拷贝构造函数和拷贝赋值运算符（见 §14.2.4）。

9.7.3 默认构造函数

未初始化的变量是错误的严重来源。为了对抗这一问题，我们有了构造函数的概念，以保证类的每个对象都被初始化。例如，我们声明了构造函数 Date::Date(int,Month,int)，以确保每个 Date 都被正确初始化。对 Date 来说，这意味着程序员必须提供三个类型正确的实参。例如：

```
Date d0;                    // 错误：没有初始化器
Date d1 {};                 // 错误：空的初始化器
Date d2 {1998};             // 错误：实参太少
Date d3 {1,2,3,4};          // 错误：实参太多
Date d4 {1,"jan",2};        // 错误：实参类型不对
Date d5 {1,Month::jan,2};   // 正确：使用三实参构造函数
Date d6 {d5};               // 正确：使用拷贝构造函数
```

注意，即使我们为 Date 定义了构造函数，我们仍然可以拷贝 Date。

许多类型都有“默认值”这一良好概念；也就是说，对于“如果我不给它初始化器，它应该取什么值?”这个问题，存在一个显而易见的答案。例如：

```
string s1;                  // 默认值：空字符串 " "
vector<string> v1;          // 默认值：空 vector；没有元素
```

这看起来很合理，而且它也确实按注释所说的那样工作。这是通过给 `vector` 和 `string` 提供隐式完成所需初始化的默认构造函数（default constructor）来实现的。

对一个类型 `T`，`T{}` 表示由默认构造函数定义的默认值，因此我们可以写

```
string s1 = string{};                   // 默认值：空字符串 " "
vector<string> v1 = vector<string>{};   // 默认值：空 vector；
                                        // 没有元素
```

不过，我们更喜欢与之等价、也更口语化的写法：

```
string s1;                  // 默认值：空字符串 " "
vector<string> v1;          // 默认值：空 vector；没有元素
```

对内置类型（如 `int` 和 `double`），默认构造函数记法表示 0，因此 `int{}` 是说 0 的一种繁琐方式，`double{}` 则是说 0.0 的一种绕弯方式。

使用默认构造函数可不只是好看不好看的问题。设想我们可以拥有一个未初始化的 `string` 或 `vector`：

```
string s;
for (int i=0; i<s.size(); ++i)  // 糟糕：循环次数不确定
    s[i] = toupper(s[i]);       // 糟糕：读写随机的内存位置


vector<string> v;
v.push_back("bad");             // 糟糕：写向随机地址
```

如果 `s` 和 `v` 的值确实未定义，`s` 和 `v` 就既不知道自己含有多少个元素，也不知道（按常见的实现技术；见 §17.5）这些元素应存储在哪里。其结果就是使用随机地址——而这可能导致最糟糕的一类错误。从根本上说，没有构造函数，我们就无法建立不变量——无法确保这些变量中的值是有效的（§9.4.3）。我们必须坚持这类变量被初始化。我们可以坚持要求提供初始化器，然后写

```
string s1 = "";
vector<string> v1 {};
```

但我们并不觉得这样特别好看。对 `string` 来说，`""` 表示“空字符串”相当直白；对 `vector` 来说，`{}` 表示没有元素的 `vector` 也还算漂亮。然而，对许多类型来说，很难为默认值找到一种合理的记法。对许多类型而言，更好的办法是定义一个构造函数，让“不经显式初始化器而创建对象”这件事有意义。这种构造函数不接受任何实参，称为默认构造函数（default constructor）。

日期没有一个显而易见的默认值。这就是为什么我们至今没有为 `Date` 定义默认构造函数；不过，让我们提供一个（只是为了证明可以做到）：

```
class Date {
public:
    // . . .
    Date();             // 默认构造函数
    // . . .
private:
    int y;
    Month m;
    int d;
};
```

我们必须挑选一个默认日期。21 世纪的第一天也许是个合理的选择：

```
Date::Date()
    :y{2001}, m{Month::jan}, d{1}
{
}
```

除了把成员的默认值放在构造函数里，我们还可以把它们放在成员自身上：

```
class Date {
public:
    // . . .
    Date();                             // 默认构造函数
            Date(year, Month, day);
            Date(int y);                // y 年 1 月 1 日
    // . . .
private:
    int y {2001};
    Month m {Month::jan};
    int d {1};
};
```

这样，默认值对每个构造函数都可用。例如：

```
Date::Date(int y)                       // y 年 1 月 1 日
    :y{yy}
{
    if (!is_valid()) throw Invalid{};   // 检查有效性
}
```

由于 `Date(int)` 没有显式初始化月份（`m`）和日期（`d`），指定的初始化器（`Month::jan` 和 `1`）就被隐式使用。作为类成员声明的一部分而指定的初始化器称为类内初始化器（in-class initializer）。

如果我们不喜欢把默认值直接写进构造函数代码，也可以用一个常量（或变量）。为了避免全局变量及其相关的初始化问题，我们使用 §8.6.2 的技术：

```
const Date& default_date()
{
    static Date dd {2001,Month::jan,1};
    return dd;
}
```

我们用 static 得到一个只创建一次的变量（dd），而不是每次调用 default_date() 都创建，并且它会在第一次调用 default_date() 时被初始化。有了 default_date()，为 Date 定义一个默认构造函数就是轻而易举的事了：

```
Date::Date()
    :y{default_date().year()},
    m{default_date().month()},
    d{default_date().day()}
{
}
```

注意，这个默认构造函数不需要检查它的值；default_date 的构造函数已经检查过了。有了这个 Date 默认构造函数，我们现在不必逐一列出元素值，就能定义非空的 Date vector 了：

```
vector<Date> birthdays(10);   // 十个元素，取默认的 Date 值，
                              // 即 Date{}
```

如果没有默认构造函数，我们就不得不显式写出：

```
vector<Date> birthdays(10,default_date());   // 十个默认的 Date


vector<Date> birthdays2 = {                  // 十个默认的 Date
        default_date(), default_date(), default_date(), default_date(), default_
                date(),
        default_date(), default_date(), default_date(), default_date(), default_
                date()
};
```

在为 vector 指定元素个数时，我们使用圆括号 ()，而不是 {} 初始化列表记法，以避免在 vector<int> 的情形下产生混淆（§18.2）。

9.7.4 const 成员函数

有些变量就是用来被修改的——这正是我们称它们为“变量”的原因——但也有一些不是；也就是说，我们有表示不可变值的“变量”。这些我们通常称为常量（constant），或干脆叫 const。考虑：

```
void some_function(Date& d, const Date& start_of_term)
{
    int a = d.day();                // 正确
    int b = start_of_term.day();    // 应该正确（为什么？）

    d.add_day(3);                   // 没问题
    start_of_term.add_day(3);       // 错误
}
```

这里我们的意图是 d 可变，而 start_of_term 不可变；some_function() 去修改 start_of_term 是不可接受的。编译器怎么会知道这一点？它知道，是因为我们通过把 start_of_term 声明为 const 而告诉了它。到目前为止一切顺利，可是为什么用 day() 读取 start_of_term 的日号又是可行的呢？按照 Date 目前的定义，start_of_term.day() 是一个错误，因为编译器并不知道 day() 不会修改它的 Date。我们从没告诉过它，所以编译器假定 day() 可能修改它的 Date，于是报错。

我们可以把对类的操作分为修改的（modifying）和不修改的（nonmodifying）来解决这一问题。这是一个相当根本的区别，有助于我们理解一个类，同时也有非常实际的重要性：不修改对象的操作可以被 const 对象调用。例如：

```
class Date {
public:
    // . . .
    int day() const;            // const 成员：不能修改对象
    Month month() const;        // const 成员：不能修改对象
    int year() const;           // const 成员：不能修改对象

    void add_day(int n);        // 非 const 成员：可以修改对象
    void add_month(int n);      // 非 const 成员：可以修改对象
    void add_year(int n);       // 非 const 成员：可以修改对象
private:
    int y;                      // 年
    Month m;
    int d;                      // 月中的日
};


Date d {2000, Month::jan, 20};
const Date cd {2001, Month::feb, 21};


cout << d.day() << " - " << cd.day() << '\n';   // 正确
d.add_day(1);                                    // 正确
cd.add_day(1);                                   // 错误：cd 是一个 const
```

我们在成员函数声明的实参列表之后使用 const，表明该成员函数可以为 const 对象调用。一旦我们声明了一个成员函数为 const，编译器就会督促我们信守不修改对象的承诺。例如：

```
int Date::day() const
{
    ++d;    // 错误：试图从 const 成员函数修改对象
    return d;
}
```

当然，我们通常不会这样“作弊”。编译器为类实现者提供的主要是对意外的防护，这在更复杂的代码中尤其有用。

9.7.5 成员与“辅助函数”

当我们把接口设计得最小（虽然完整）时，就不得不省去许多只是有用的运算。凡是可以简单、优雅且高效地实现为独立函数（即非成员函数）的函数，都应该放在类外实现。这样，该函数中的 bug 就无法直接破坏类对象中的数据。不去访问表示这一点很重要，因为常用的调试技术是“把通常的嫌疑犯都圈起来”；也就是说，当某个类出了问题时，我们首先查看直接访问其表示的那些函数：几乎可以肯定就是其中之一干的。这样的函数如果有十来个，我们会比有 50 个时愉快得多。

一个 Date 类居然要 50 个函数！你一定觉得我们在开玩笑。我们没有：几年前我调查过若干商用的 Date 库，发现其中满是 next_Sunday()、next_workday() 之类的函数。对于一个为用户便利而非为易于理解、实现和维护而设计的类来说，50 个并不算过分。

还要注意，如果表示改变了，只有直接访问表示的那些函数需要重写。这是保持接口最小化的另一个有力的实际理由。在我们的 Date 例子中，我们可能会认为，用“自 1900 年 1 月 1 日以来的天数”这一整数来表示，对我们的用途来说比 (year,month,day) 好得多。届时只需修改成员函数。

下面是一些辅助函数（helper function）的例子：

```
Date next_Sunday(const Date& d)
{
    // 用 d.day()、d.month() 和 d.year() 访问 d
    // 构造要返回的新 Date
}

Date next_weekday(const Date& d) { /* . . . */ }

bool leapyear(int y) { /* . . . */ }

bool operator==(const Date& a, const Date& b)
{
    return a.year()==b.year()
            && a.month()==b.month()
            && a.day()==b.day();
}

bool operator!=(const Date& a, const Date& b)
{
    return !(a==b);
}
```

辅助函数也被称为便利函数（convenience function）、辅助性函数（auxiliary function）等许多其他名字。这些函数与其他非成员函数之间的区分是逻辑上的；也就是说，“辅助函数”是一个设计概念，而不是程序设计语言概念。辅助函数通常以其所辅助的类的对象为实参；但也有例外，例如 `leapyear()`。我们常用命名空间来标识一组辅助函数；见 §8.7：

```
namespace Chrono {
    enum class Month { /* ... */ };
    class Date { /* . . . */ };
    bool is_date(int y, Month m, int d);      // 对有效日期为 true
    Date next_Sunday(const Date& d) { /* . . . */ }
    Date next_weekday(const Date& d) { /* . . . */ }
    bool leapyear(int y) { /* . . . */ }      // 见习题 10
    bool operator==(const Date& a, const Date& b) { /* . . . */ }
// . . .
}
```

注意 == 和!= 这两个函数。它们是典型的辅助函数。对许多类来说，== 和!= 的含义显而易见；但由于并非对所有类都有意义，编译器无法像替你写出拷贝构造函数和拷贝赋值那样替你写出它们。

还要注意，我们引入了一个辅助函数 `is_date()`。这个函数取代了 `Date::is_valid()`，因为检查一个日期是否有效在很大程度上与 `Date` 的表示无关。例如，我们不需要知道 `Date` 对象是如何表示的，就知道“2008 年 1 月 30 日”是有效日期，而“2008 年 2 月 30 日”不是。当然，日期的某些方面仍可能依赖于表示（例如，我们能表示“1066 年 1 月 30 日”吗?），但（如有必要）`Date` 的构造函数可以处理这些。

9.8 Date 类

现在，让我们把所有这些想法与考量合在一起，看看 `Date` 类会是什么样子。凡是函数体只有一个. . . 注释的地方，其实际实现都是有难度的（请先不要急着把它们写出来）。首先，我们把声明放在头文件 `Chrono.h` 中：

```
// 文件 Chrono.h

namespace Chrono {

enum class Month {
    jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};

class Date {
public:
    class Invalid { };                  // 用作异常

    Date(int y, Month m, int d);        // 检查日期有效性并初始化
    Date();                             // 默认构造函数
    // 默认的拷贝操作没有问题
```

```
    // 不修改对象的运算:
    int day() const { return d; }
    Month month() const { return m; }
    int year() const { return y; }

    // 修改对象的运算:
    void add_day(int n);
    void add_month(int n);
    void add_year(int n);
private:
    int y;
    Month m;
    int d;
};

bool is_date(int y, Month m, int d);    // 对有效日期为 true

bool leapyear(int y);                   // 若 y 是闰年则为 true

bool operator==(const Date& a, const Date& b);
bool operator!=(const Date& a, const Date& b);

ostream& operator<<(ostream& os, const Date& d);

istream& operator>>(istream& is, Date& dd);

}                                       // Chrono
```

定义则放入 Chrono.cpp:

```
// Chrono.cpp
#include "Chrono.h"

namespace Chrono {
// 成员函数定义:

Date::Date(int yy, Month mm, int dd)
    : y{yy}, m{mm}, d{dd}
{
    if (!is_date(yy,mm,dd)) throw Invalid{};
}

const Date& default_date()
{
    static Date dd {2001,Month::jan,1};   // 21 世纪之始
```

```
    return dd;
}

Date::Date()
    :y{default_date().year()},
    m{default_date().month()},
    d{default_date().day()}
{
}

void Date::add_day(int n)
{
    // . . .
}

void Date::add_month(int n)
{
    // . . .
}

void Date::add_year(int n)
{
    if (m==feb && d==29 && !leapyear(y+n)) {   // 当心闰年!
        m = mar;                               // 用 3 月 1 日代替 2 月 29 日
        d = 1;
    }
    y+=n;
}
// 辅助函数:

bool is_date(int y, Month m, int d)
{
    // 假定 y 是有效的

    if (d<=0) return false;                    // d 必须为正
    if (m<Month::jan || Month::dec<m) return false;

    int days_in_month = 31;                    // 大多数月份有 31 天

    switch (m) {
    case Month::feb:                           // 二月的长度有变化
        days_in_month = (leapyear(y))?29:28;
        break;
    case Month::apr: case Month::jun: case Month::sep: case Month::nov:
        days_in_month = 30;                    // 其余月份有 30 天
        break;
```

```
    }

    if (days_in_month<d) return false;

    return true;
}

bool leapyear(int y)
{
    // 见习题 10
}

bool operator==(const Date& a, const Date& b)
{
    return a.year()==b.year()
            && a.month()==b.month()
            && a.day()==b.day();
}

bool operator!=(const Date& a, const Date& b)
{
    return !(a==b);
}

ostream& operator<<(ostream& os, const Date& d)
{
    return os << '(' << d.year()
                << ',' << d.month()
                << ',' << d.day() << ')';
}

istream& operator>>(istream& is, Date& dd)
{
    int y, m, d;
    char ch1, ch2, ch3, ch4;
    is >> ch1 >> y >> ch2 >> m >> ch3 >> d >> ch4;
    if (!is) return is;
    if (ch1!='(' || ch2!=',' || ch3!=',' || ch4!=')') {   // 糟糕：格式错误
        is.clear(ios_base::failbit);                      // 设置 fail 位
        return is;
    }

    dd = Date(y, Month(m),d);                             // 更新 dd

    return is;
}
```

```
enum class Day {
    sunday, monday, tuesday, wednesday, thursday, friday, saturday
};

Day day_of_week(const Date& d)
{
    // . . .
}

Date next_Sunday(const Date& d)
{
    // ...
}

Date next_weekday(const Date& d)
{
    // . . .
}

}                                       // Chrono
```

为 Date 实现 » 和 « 的函数将在 §10.8 和 §10.9 中详细解释。

小试身手

小试身手

这个小试身手只是让 Date 的这一系列版本都能工作。对每个版本，定义一个名为 today 的 Date，初始化为 1978 年 6 月 25 日；然后定义一个名为 tomorrow 的 Date，把 today 拷贝进去，并用 add_day() 把它的日号加 1。最后，用 §9.8 中定义的 « 输出 today 和 tomorrow。

你对有效日期的检查可以非常简单，尽可忽略闰年；但不要接受不在 [1,12] 范围内的月份，也不要接受不在 [1,31] 范围内的日号。用至少一个无效日期（例如 2004, 13, -5）测试每个版本。

1. §9.4.1 的版本
2. §9.4.2 的版本
3. §9.4.3 的版本
4. §9.7.1 的版本
5. §9.7.4 的版本

要点回顾

1. 如本章所述，一个类有哪两个部分？
2. 在一个类中，接口与实现有什么区别？
3. 本章最初创建的那个 Date struct 有哪些局限和问题？
4. 对 Date 类型为什么用构造函数，而不用一个 init_day() 函数？
5. 什么是不变量？举例说明。
6. 什么时候应把函数放进类定义，什么时候应在类外定义？为什么？
7. 程序中什么时候应使用运算符重载？列出你可能想重载的运算符（每个都给出理由）。

8. 为什么类的公共接口应尽可能小？
9. 给成员函数加上 const 有什么作用？
10. 为什么“辅助函数”最好放在类定义之外？

术语

内置类型　枚举符　表示
class　辅助函数　struct
const　实现　结构
构造函数　类内初始化器　用户自定义类型
析构函数　内联　有效状态
enum　接口
枚举　不变量

习题

1. 为 §9.1 中那些现实世界对象的例子（例如烤面包机）列出一组合理的运算。
2. 设计并实现一个 Name_pairs 类，保存 (name,age) 对，其中 name 是一个 string，age 是一个 double。将它表示为一个 vector<string>（名为 name）和一个 vector<double>（名为 age）成员。提供一个输入运算 read_names()，读取一系列名字；提供一个 read_ages() 运算，为每个名字提示用户输入年龄；提供一个 print() 运算，按 name 向量确定的顺序打印出 (name[i],age[i]) 对（每行一对）；提供一个 sort() 运算，将 name 向量按字母序排序，并相应重排 age 向量。把所有“运算”都实现为成员函数。测试这个类（当然：尽早并经常测试）。
3. 将 Name_pair::print() 替换为一个（全局）运算符 «，并为 Name_pairs 定义 == 和!=。
4. 看看 §8.4 那个令人头疼的最后一个例子。将它正确缩进，并解释每个结构的含义。注意，该例子什么都不做、毫无意义；它是纯粹的混淆代码。
5. 本题以及接下来几题要求你设计并实现一个 Book 类，你可以把它想象为图书馆软件的一部分。Book 类应有 ISBN、书名、作者和版权日期的成员，还应保存该书是否被借出的数据。创建返回这些数据值的函数，以及办理借出和归还的函数。对录入 Book 的数据做简单的校验；例如，只接受形如 n-n-n-x 的 ISBN，其中 n 是整数，x 是数字或字母。将 ISBN 保存为一个 string。
6. 为 Book 类添加运算符。让 == 运算符检查两本书的 ISBN 是否相同；让!= 也比较 ISBN；让 « 把书名、作者和 ISBN 分行打印出来。
7. 为 Book 类创建一个名为 Genre 的枚举类型，类型取值有 fiction（小说）、nonfiction（非小说）、periodical（期刊）、biography（传记）和 children（儿童读物）。给每本书一个 Genre，并对 Book 的构造函数和成员函数做适当的修改。
8. 为图书馆创建一个 Patron 类。该类应包含用户姓名、图书证号和图书馆欠费（如有）。提供访问这些数据的函数，以及设置用户费用的函数；再提供一个辅助函数，根据用户是否欠费返回一个布尔值（bool）。
9. 创建一个 Library 类。其中包含 Book 和 Patron 的 vector；包含一个名为 Transaction 的 struct，其中有本章的一个 Book、一个 Patron 和一个 Date。建立一个 Transaction 的 vector。创建向图书馆添加书、添加读者以及办理借书的函数。每当有用户借书时，图书馆都要确认该用户和该书都已在馆中；若不在，报错。然后确认该用户没有欠费；若欠费，报错。若都没问题，就

创建一个 Transaction 并放入该 vector。再编写一个函数，返回一个包含所有欠费 Patron 名字的 vector。

10. 实现 §9.8 中的 leapyear()。
11. 为 Date 类设计并实现一组有用的辅助函数，例如 next_workday()（假定凡不是星期六或星期日的日子都是工作日）和 week_of_year()（假定含 1 月 1 日的那一周是第 1 周，且一周的第一天是星期日）。
12. 把 Date 的表示改为“自 1970 年 1 月 1 日（称为第 0 天）以来的天数”，用一个 long int 表示，并重新实现 §9.8 的函数。务必拒绝超出可表示范围的日期（尽可拒绝第 0 天之前的日期，即不允许负数天数）。
13. 设计并实现一个有理数类 Rational。一个有理数有两个部分：分子和分母，例如 5/6（六分之五，也约等于.83333）。如有需要可查阅定义。提供赋值、加法、减法、乘法、除法和相等运算符；还要提供一个到 double 的转换。人们为什么会想使用 Rational 类？
14. 设计并实现一个 Money 类，用于涉及美元和美分的计算，其中算术必须精确到最后一美分，采用“四舍五入”规则（.5 美分向上舍入；小于.5 的向下舍入）。将金额表示为一个 long int 的美分数，但按美元和美分输入输出，例如 $123.45。不必操心放不进 long int 的金额。
15. 通过添加一种货币（作为构造函数实参给出）来改进 Money 类。只要浮点初始化器能被精确表示为 long int，就接受它。不接受非法运算。例如，Money*Money 没有意义；USD1.23+DKK5.00 只有在你提供了一张定义美元（USD）与丹麦克朗（DKK）之间换算因子的换算表时才有意义。
16. 定义一个输入运算符（»），将带货币单位的金额（如 USD1.23 和 DKK5.00）读入一个 Money 变量；并定义一个相应的输出运算符（»）。（原文如此；按上下文此处应为 <<，疑为原书排印笔误。）
17. 举一个计算的例子，其中 Rational 比 Money 给出数学上更好的结果。
18. 举一个计算的例子，其中 Rational 比 double 给出数学上更好的结果。

后记

用户自定义类型的内容远比我们这里介绍的要多。用户自定义类型——尤其是类——是 C++ 的核心，也是许多最有效的设计技术的关键。本书余下的大部分内容都关乎类的设计与使用。类——或一组类——是我们在代码中表达概念的手段。本章主要介绍类的语言技术层面；在别处，我们聚焦于如何把有用的思想优雅地表达为类。

第二部分

输入与输出

10 输入与输出流

“科学就是我们所学到的、关于如何不欺骗自己的学问。”

——Richard P. Feynman

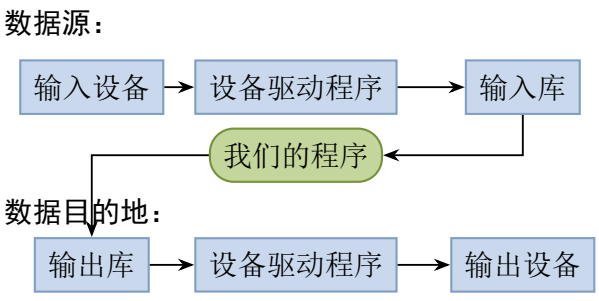
在本章和下一章中，我们将介绍 C++ 标准库中处理来自各种来源的输入与输出的设施：I/O 流。我们将展示如何读写文件、如何处理错误、如何处理格式化的输入，以及如何为用户自定义类型提供和使用 I/O 运算符。本章聚焦于基本模型：如何读写单个值，以及如何打开、读取和写入整个文件。最后一个例子说明了编写一段较大规模的代码所涉及的种种考量。下一章再讨论细节。

10.1 输入与输出
10.2 I/O 流模型
10.3 文件
10.4 打开文件
10.5 读写文件
10.6 I/O 错误处理
10.7 读取单个值
10.7.1 把问题分解成可管理的部分
10.7.2 将对话与函数分离
10.8 用户自定义输出运算符
10.9 用户自定义输入运算符
10.10 一个标准输入循环
10.11 读取结构化文件
10.11.1 内存中的表示
10.11.2 读取结构化值
10.11.3 改变表示

10.1 输入与输出

没有数据，计算就毫无意义。我们需要把数据送入程序去做有意义的计算，还需要把结果再输出出来。在 §4.1 中，我们提到过数据来源和输出目标的五花八门、令人眼花缭乱。如果不加留心，我们最后写出的程序就只能从特定的来源接收输入、只能把结果送到特定的输出设备。对于专用的应用（例如数码相机或发动机燃油喷射器的传感器）来说，这或许可以接受（有时甚至是必需的）；但对于更常见的任务，我们需要一种方法，把程序读写数据的方式与实际使用的输入输出设备分离开来。如果我们必须直接面对每一种设备，那么每当市场上出现新的屏幕或磁盘，我们就得修改程序，要么就只能把用户限制在我们恰好喜欢的那些屏幕和磁盘上。那简直是荒唐的。

大多数现代操作系统都把 I/O 设备的细节处理分离到设备驱动程序（device driver）中，程序则通过一个 I/O 库来访问这些设备驱动程序，这个库使得从/向不同来源的 I/O 看起来尽可能地相似。一般而言，设备驱动程序深藏在操作系统内部，大多数用户看不到它们；而 I/O 库则提供 I/O 的一个抽象，使程序员不必去考虑设备和设备驱动程序：



10.2 I/O 流模型

当使用这样的模型时，输入和输出就可以看作是由输入/输出库处理的字节（字符）流。更复杂形式的 I/O 需要专门的知识，超出了本书的范围。于是，作为应用程序员，我们的工作就变成

1. 建立通向相应数据源和数据目的地的 I/O 流。
2. 从/向这些流读取/写入数据。

至于我们的字符实际上如何传送到/来自设备，这些细节由 I/O 库和设备驱动程序处理。在本章和下一章中，我们将看到如何使用 C++ 标准库来完成由格式化数据流组成的 I/O。

从程序员的角度看，输入和输出有很多不同的种类。一种分类方式是

- （许多）数据项组成的流（通常与文件、网络连接、记录设备或显示设备打交道）
- 与键盘前的用户交互
- 通过图形界面与用户交互（输出对象、接收鼠标点击等）

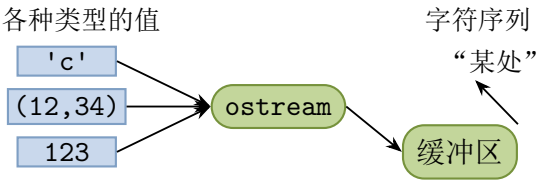
这种分类并非唯一可能的分类，而且这三种 I/O 之间的区别也不像表面上那么清晰。例如，如果一串输出字符恰好是发给浏览器的 HTTP 文档，其结果看起来就非常像用户交互，还可能包含图形元素。反过来，与 GUI（图形用户界面，graphical user interface）交互的结果也可能以字符序列的形式呈现给程序。不过，这种分类与我们的工具相吻合：前两种 I/O 由 C++ 标准库的 I/O 流提供，并得到大多数操作系统相当直接的支持。从第 1 章起我们一直在使用 iostream 库，本章和下一章也将聚焦于它。图形输出和图形用户交互由各种不同的库提供，我们将在第 12 至 16 章聚焦于那类 I/O。

C++ 标准库提供了类型 `istream` 来处理输入流，类型 `ostream` 来处理输出流。我们已经使用过名为 **cin** 的标准 istream 和名为 `cout` 的标准 ostream，所以我们已经了解如何使用标准库这一部分（通常称为 iostream 库）的基础知识。

一个 ostream

- 把各种类型的值转换为字符序列
- 把这些字符发送到“某处”（例如控制台、文件、主存或另一台计算机）

我们可以这样用图形来表示一个 ostream：



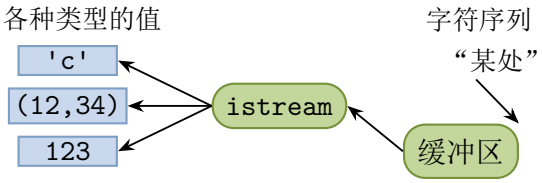
缓冲区（buffer）是 ostream 在与操作系统通信时，内部用来存放你交给它的数据的一种数据结构。如果你注意到向 ostream 写入与字符出现在目的地之间有一段“延迟”，通常是因为它们还在缓冲区里。缓冲对性能很重要，而如果你要处理大量数据，性能就很重要。

一个 istream

- 把字符序列转换为各种类型的值
- 从某处获取这些字符（例如控制台、文件、主存或另一台计算机）

我们可以这样用图形来表示一个 istream：

与 ostream 一样，istream 也使用缓冲区与操作系统通信。对 istream 来说，缓冲对用户可能是相当显而易见的。当你使用连接到键盘的 istream 时，你键入的内容会留在缓冲区中，直到你按下回车键（return/换行符）；而且你还可以用擦除键（Backspace）“改变主意”（直到按下回车键为止）。



输出的一个主要用途，是产生供人阅读的数据。想想电子邮件、学术文章、网页、账单记录、商业报告、联系人列表、目录、设备状态读出等等。因此，ostream 提供了许多按不同口味格式化文本的特性。类似地，许多输入是由人写成的，或者被格式化得让人容易阅读。因此，istream 提供了读取 ostream 所产生的那类输出的特性。我们将在 §11.2 讨论格式化，在 §11.3.2 讨论如何读取非字符输入。与输入相关的大部分复杂性都与如何处理错误有关。为了给出更贴近实际的例子，我们先来讨论 iostream 模型与数据文件的关系。

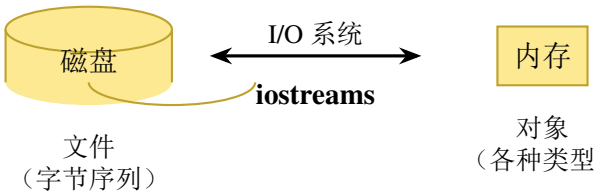
10.3 文件

我们通常拥有的数据远多于计算机主存所能容纳的量，所以我们把大部分数据存储在磁盘或其他大容量存储设备上。这类设备还有一个可贵的性质：断电后数据不会消失——数据是持久的（persistent）。在最基本的层面上，文件（file）不过是编号从 0 向上递增的字节序列：



文件有格式（format）：也就是说，它有一组规则，决定这些字节的含义。例如，如果我们有一个文本文件，那么前 4 个字节就是前四个字符。反过来，如果我们有一个使用整数二进制表示的文件，那么同样这前 4 个字节就会被当作第一个整数的（二进制）表示（见 §11.3.2）。格式之于磁盘上的文件，正如类型之于主存中的对象。只有（且只要）知道文件的格式，我们才能理解文件中的那些位（见 §11.2-3）。

对于文件，ostream 把主存中的对象转换为字节流并写到磁盘上；istream 则相反，它从磁盘取来字节流，并用它们组装出对象：



大多数时候，我们假定这些“磁盘上的字节”实际上就是我们常用字符集中的字符。情况并非总是如此，但在这一假设下我们已经可以走得非常远，而且其他表示方式也不难处理。我们也说得好像所有文件都在磁盘上（即旋转磁存储器上）似的。这同样并不总是成立（想想闪存），但在这个编程层面上，实际的存储介质没有什么影响。这正是文件和流抽象的美妙之处之一。

要读一个文件，我们必须

1. 知道它的名字
2. 打开它（供读取）
3. 读入字符
4. 关闭它（尽管这通常是隐式完成的）

要写一个文件，我们必须

1. 给它起个名字
2. 打开它（供写入），或创建一个该名字的新文件
3. 写出我们的对象
4. 关闭它（尽管这通常是隐式完成的）

我们已经了解读写的基础知识，因为就我们迄今为止所做的事情而言，连接到文件的 ostream 的行为与 `cout` 完全一样，连接到文件的 istream 的行为与 **cin** 完全一样。我们会在后面（§11.3.3）介绍只能对文件进行的操作，但现在我们只看看如何打开文件，然后把注意力集中于适用于所有 ostream 和所有 istream 的运算与技术上。

10.4 打开文件

如果你想从一个文件读取或向一个文件写入，就必须专门为该文件打开一个流。`ifstream` 是用于从文件读取的 istream，`ofstream` 是用于向文件写入的 ostream，`fstream` 则是既可读又可写的 iostream。文件流在被使用之前必须连接到一个文件。例如：

```
cout << "Please enter input file name: ";
string iname;
cin >> iname;
ifstream ist {iname};    // ist 是读取名为 name 的文件的输入流
if (!ist) error("can't open input file ",iname);
```

用一个名字字符串定义一个 `ifstream`，就会打开该名字的文件供读取。对 **!ist** 的测试检查文件是否被正确打开。此后，我们就可以像从任何其他 istream 读取那样从这个文件读取。例如，假定输入运算符 >> 已为类型 Point 定义，我们就可以写

```
vector<Point> points;
for (Point p; ist>>p; )
    points.push_back(p);
```

向文件输出由 ofstream 以类似的方式处理。例如：

```
cout << "Please enter name of output file: ";
string oname;
cin >> oname;
ofstream ost {oname};      // ost 是写入名为 oname 的文件的输出流
if (!ost) error("can't open output file ",oname);
```

用一个名字字符串定义一个 `ofstream`，就会打开该名字的文件供写入。对 **!ost** 的测试检查文件是否被正确打开。此后，我们就可以像向任何其他 ostream 写入那样向这个文件写入。例如：

```
for (Point p : points)
    ost << '(' << p.x << ',' << p.y << ")\n";
```

当一个文件流离开其作用域时，与之关联的文件就会被关闭。文件被关闭时，其关联的缓冲区会被“冲刷”（flush）；也就是说，缓冲区中的字符会被写到文件里。

通常最好在程序中尽早打开文件，在任何 serious 的计算发生之前。毕竟，辛辛苦苦做了大量工作，最后却发现无处可写结果，那纯粹是浪费。

在创建 ostream 或 istream 之时隐式地打开文件，并依靠流的作用域来负责关闭文件，这是理想的做法。例如：

```
void fill_from_file(vector<Point>& points, string& name)
{
    ifstream ist {name};      // 打开文件供读取
    if (!ist) error("can't open input file ",name);
    // . . . 使用 ist . . .
    // 当我们离开函数时，文件会被隐式关闭
}
```

你也可以显式地执行 `open()` 和 `close()` 操作（§B.7.1）。不过，依靠作用域能把有人在文件流尚未连接到流之前、或在它关闭之后试图使用它的机会降到最低。例如：

```
ifstream ifs;
// . . .
ifs >> foo;                    // 不会成功：尚未为 ifs 打开文件
// . . .
ifs.open(name,ios_base::in);   // 打开名为 name 的文件供读取
// . . .
ifs.close();                   // 关闭文件
// . . .
ifs >> bar;                    // 不会成功：ifs 的文件已关闭
// . . .
```

在真实世界的代码中，这类问题通常要难发现得多。所幸，你无法在第一次尚未关闭的情况下把一个文件流再次打开。例如：

```
fstream fs;
fs.open("foo", ios_base::in) ;   // 打开供输入
// 缺少 close()
fs.open("foo", ios_base::out);   // 不会成功：fs 已经打开
if (!fs) error("impossible");
```

打开流之后，别忘了测试它。

那你为什么还要显式地使用 `open()` 或 `close()` 呢？呃，偶尔会出现文件连接的生命周期不方便由作用域限定的情况，这时你就不得不这么做。但这种情况足够罕见，我们在此不必为此操心。更要紧的是，你会在一些人写的代码里见到这种用法，他们使用的是来自某些语言和库的风格，那些语言和库没有 iostream（以及 C++ 标准库的其余部分）所用的作用域惯用法。

正如我们将在第 11 章看到的，关于文件还有很多内容，但就目前而言，我们已有的知识足以把文件用作数据源和数据目的地。这将让我们能够写出一些程序，如果我们假定用户必须直接键入全部输入，这些程序就会不切实际。从程序员的角度看，文件的一大优点是，在调试期间你可以反复读取它，直到你的程序正确为止。

10.5 读写文件

考虑你可能如何从一个文件读入一组测量结果，并在内存中表示它们。这些可能是一家气象站的温度读数：

```
0 60.7
1 60.6
2 60.3
3 59.22
...
```

这个数据文件包含一串（hour,temperature）对。小时编号为 0 到 23，温度为华氏度。这里不假定任何进一步的格式；也就是说，文件不含任何特殊的头部信息（例如读数是在哪里测得的）、值的单位、标点（例如每一对值外面的圆括号）或终止指示符。这是最简单的情况。

我们可以用一个 Reading 类型来表示一条温度读数：

```
struct Reading {                // 一条温度读数
    int hour;                   // 午夜后的钟点 [0:23]
    double temperature;         // 华氏温度
};
```

有了这个类型，我们就可以这样读取：

```
vector<Reading> temps;          // 在这里存放读数
int hour;
double temperature;
while (ist >> hour >> temperature) {
    if (hour < 0 || 23 <hour) error("hour out of range");
    temps.push_back(Reading{hour,temperature});
}
```

这是一个典型的输入循环。名为 **ist** 的 istream 可以是上一节所示的输入文件流（`ifstream`）、标准输入流（**cin**）（或其别名），或任何其他种类的 istream。对于这样的代码，istream 究竟从哪里取得数据并不重要。我们的程序所关心的，只是 ist 是一个 istream，而且数据具有预期的格式。下一节将讨论一个有趣的问题：如何检测输入数据中的错误，以及检测到格式错误之后我们能做什么。

向文件写入通常比从文件读取简单。同样，一旦流被初始化，我们就不必确切知道它是哪种流。特别地，我们可以像使用任何其他 ostream 一样使用上一节的输出文件流（`ofstream`）。例如，我们可能想把读数输出为每一对值都带圆括号的形式：

```
for (int i=0; i<temps.size(); ++i)
    ost << '(' << temps[i].hour << ',' << temps[i].temperature << ")\n";
```

这样得到的程序将读取原始的温度读数文件，并产生一个数据为（hour,temperature）格式的新文件。

由于文件流在离开作用域时会自动关闭其文件，完整的程序就变成了

```
#include "std_lib_facilities.h"

struct Reading {                // 一条温度读数
    int hour;                   // 午夜后的钟点 [0:23]
    double temperature;         // 华氏温度
};

int main()
{
    cout << "Please enter input file name: ";
    string iname;
    cin >> iname;
    ifstream ist {iname};       // ist 从名为 iname 的文件读取
    if (!ist) error("can't open input file ",iname);

    string oname;
    cout << "Please enter name of output file: ";
    cin >> oname;
    ofstream ost {oname};       // ost 写入名为 oname 的文件
    if (!ost) error("can't open output file ",oname);

    vector<Reading> temps;      // 在这里存放读数
    int hour;
    double temperature;
    while (ist >> hour >> temperature) {
        if (hour < 0 || 23 <hour) error("hour out of range");
        temps.push_back(Reading{hour,temperature});
    }

    for (int i=0; i<temps.size(); ++i)
        ost << '(' << temps[i].hour << ','
            << temps[i].temperature << ")\n";
}
```

10.6 I/O 错误处理

处理输入时，我们必须预期错误并处理它们。什么样的错误？又如何处理？错误发生的原因包括：人会犯错误（误解指令、打错字、让猫在键盘上乱踩等）、文件不符合规范、我们（作为程序员）抱有

错误的预期，等等。输入错误的可能性是无穷无尽的！不过，istream 把它们统统归约为四种可能的情况，称为流状态（stream state）：

流状态	
good()	操作成功。
eof()	我们遇到了输入结束（“文件尾”）。
fail()	发生了意料之外的事情（例如，我们想找一个数字，却遇到了 'x'）。
bad()	发生了意料之外且严重的事情（例如，磁盘读取错误）。

遗憾的是，fail() 与 bad() 之间的区别并没有精确定义，对于为新类型定义 I/O 操作的程序员们来说，这也是个见仁见智的问题。不过，基本思想很简单：如果输入操作遇到一个简单的格式错误，它会让流 fail()，假定你（我们这个输入操作的使用者）或许能从中恢复。反过来，如果发生了真正糟糕的事情，例如磁盘读取错误，输入操作就让流 bad()，假定除了放弃从该流获取数据的尝试之外，你基本上无计可施。处于 bad() 状态的流同时也处于 fail() 状态。这就给我们留下了如下的一般逻辑：

```
int i = 0;
cin >> i;
if (!cin) {    // 只有当某个输入操作失败时才会执行到这里
    if (cin.bad()) error("cin is bad");   // 流已损坏：赶紧离开!
    if (cin.eof()) {
        // 没有更多输入
        // 这常常正是我们希望一系列输入操作结束的方式
    }
    if (cin.fail()) {       // 流遇到了意料之外的东西
        cin.clear();        // 为更多输入做好准备
        // 以某种方式恢复
    }
}
```

!cin 可以读作“**cin** 不处于 good 状态”或“**cin** 出了问题”或“**cin** 的状态不是 good()”。它与“操作成功”相反。注意处理 fail() 时的 cin.clear()：当流已经 fail 时，我们或许能从中恢复。为了尝试恢复，我们显式地把流从 fail() 状态中取出来，以便能再次从它读取字符；clear() 做的就是这件事——在 cin.clear() 之后，**cin** 的状态就是 good() 了。

下面是一个我们可能如何使用流状态的例子。考虑如何把一串整数读入一个 vector，这串整数可能以字符 * 或“文件尾”（Windows 上是 Ctrl+Z，Unix 上是 Ctrl+D）结束。例如：

```
1 2 3 4 5 *
```

这可以用下面这样的函数来完成：

```
void fill_vector(istream& ist, vector<int>& v, char terminator)
    // 从 ist 读取整数存入 v，直到遇到 eof() 或 terminator
{
    for (int i; ist >> i; ) v.push_back(i);
    if (ist.eof()) return;                // 好：我们遇到了文件尾

    if (ist.bad()) error("ist is bad");  // 流已损坏；赶紧离开！
    if (ist.fail()) {    // 尽我们所能清理残局，并报告问题
        ist.clear();     // 清除流状态，
                         // 以便我们能寻找终止符
        char c;
        ist>>c;          // 读入一个字符，希望是终止符
        if (c != terminator) {          // 意料之外的字符
            ist.unget();                // 把那个字符放回去
            ist.clear(ios_base::failbit);  // 把状态置为 fail()
        }
    }
}
```

注意，当我们没有找到终止符时，仍然会返回。毕竟，我们可能已经收集到了一些数据，而 `fill_vector()` 的调用方或许能从 `fail()` 状态中恢复。既然我们为了检查那个字符而清除了状态，就必须把流状态重新置回 `fail()`。我们用 `ist.clear(ios_base::failbit)` 来做这件事。请注意 `clear()` 这种可能令人困惑的用法：带参数的 `clear()` 实际上是把提到的那些 iostream 状态标志（位）置位，并且（只）清除未提到的标志。通过把状态置为 `fail()`，我们表明遇到的是格式错误，而不是更严重的问题。我们用 `unget()` 把那个字符放回 **ist**；`fill_vector()` 的调用方可能用得上它。`unget()` 函数是 `putback()`（§6.8.2、§B.7.3）的简短版本，它依靠流记住自己最后产生的是哪个字符，因而你不必提及那个字符。

如果你调用了 `fill_vector()`，并想知道读取是被什么终止的，可以测试 `fail()` 和 `eof()`。你也可以捕获由 `error()` 抛出的 `runtime_error` 异常，但人们普遍认为，从处于 `bad()` 状态的 istream 获取更多数据是不太可能的。大多数调用方不会去费这个心。这意味着，在几乎所有的情况下，遇到 `bad()` 时我们唯一想做的事就是抛出一个异常。为了让生活轻松些，我们可以让一个 istream 替我们这样做：

```
// 让 ist 在变 bad 时抛出异常
ist.exceptions(ist.exceptions()|ios_base::badbit);
```

这个记法看起来可能有点怪，但其效果很简单：从这条语句开始，一旦变 `bad()`，**ist** 就会抛出标准库异常 `ios_base::failure`。在一个程序中，这条 `exceptions()` 调用只需执行一次。这样，我们就可以忽略 `bad()`，从而简化对 **ist** 的所有输入循环：

```
void fill_vector(istream& ist, vector<int>& v, char terminator)
    // 从 ist 读取整数存入 v，直到遇到 eof() 或 terminator
{
    for (int i; ist >> i; ) v.push_back(i);
    if (ist.eof()) return;    // 好：我们遇到了文件尾

    // 不是 good()、不是 bad() 也不是 eof()，ist 必定是 fail()
    ist.clear();              // 清除流状态

    char c;
    ist>>c;                   // 读入一个字符，希望是终止符

    if (c != terminator) {    // 糟糕：不是终止符，所以我们必须 fail
        ist.unget();          // 也许调用方能用上那个字符
        ist.clear(ios_base::failbit);   // 把状态置为 fail()
    }
}
```

这里那里出现的 `ios_base` 是 iostream 的一部分，它保存着诸如 `badbit` 这样的常量、诸如 `failure` 这样的异常，以及其他有用的东西。你用 `::` 运算符来引用它们，例如 `ios_base::badbit`（§B.7.2）。我们不打算深入到 iostream 库那么细的层面；要把 iostream 的所有内容都解释清楚，可能得花上一整门课的时间。例如，iostream 能处理不同的字符集、实现不同的缓冲策略，还包含按各种语言的习惯格式化货币金额的设施；我们曾经收到过一个与乌克兰货币格式化有关的 bug 报告。你需要了解哪些内容，就可以去阅读哪些内容；参见 Stroustrup 的 The C++ Programming Language 和 Langer 的 Standard C++ IOStreams and Locales。

你可以用与 istream 完全相同的状态来测试一个 ostream：good()、`fail()`、`eof()` 和 `bad()`。不过，对我们这里写的这类程序而言，输出出错的情形远比输入少见，所以我们不那么经常去测试它。对于那些输出设备更可能不可用、被填满或损坏的程序，我们就会像每次输入操作之后那样，在每次输出操作之后进行测试。

10.7 读取单个值

现在，我们已经知道如何读取一串以文件尾或终止符结束的值了。随着讲解的推进，我们还会展示更多例子，但这里先来看一个非常流行的想法：反复要求输入一个值，直到得到一个可接受的值为止。这个例子将让我们考察好几种常见的设计抉择。我们将通过针对一个简单问题——“如何从用户那里得到一个可接受的值”——的一系列备选解法，来讨论这些备选方案。我们从一个不太好看的、明显“粗糙的第一版”开始，经历一系列改进的版本。我们的基本假定是：我们面对的是交互式输入，有一个真人在键入输入，并阅读程序发出的消息。我们来要求一个 1 到 10（含）范围内的整数：

```
cout << "Please enter an integer in the range 1 to 10 (inclusive):\n";
int n = 0;
while (cin>>n) {                    // 读取
    if (1<=n && n<=10) break;       // 检查范围
    cout << "Sorry "
         << n << " is not in the [1:10] range; please try again\n";
}
// ... 在这里使用 n ...
```

这相当难看，但它“多少能工作”。如果你不喜欢使用 `break`（§A.6），可以把读取和范围检查合在一起：

```
cout << "Please enter an integer in the range 1 to 10 (inclusive):\n";
int n = 0;
while (cin>>n && !(1<=n && n<=10))   // 读取并检查范围
    cout << "Sorry "
         << n << " is not in the [1:10] range; please try again\n";
// ... 在这里使用 n ...
```

然而，这只是一个表面上的改动。为什么它只是“多少能工作”？如果用户小心地键入整数，它是能工作的。如果用户打字不熟练，想敲 6 却敲成了 t（在大多数键盘上 t 就在 6 的下方），程序就会退出循环而不改变 **n** 的值，于是 **n** 将带有一个超出范围的值。我们不会把这称作有质量的代码。一个恶作剧者（或一位勤勉的测试员）还可能从键盘发送一个“文件尾”（Windows 机器上是 Ctrl+Z，Unix 机器上是 Ctrl+D）。同样，我们会带着超出范围的 **n** 退出循环。换句话说，要得到一次健壮的读取，我们必须处理三个问题：

1. 用户键入了超出范围的值
2. 没有得到值（文件尾）
3. 用户键入了类型不对的东西（此处，不是一个整数）

在这三种情况下，我们想做什么？这是编写程序时经常要问的问题：我们究竟想要什么？这里，对于这三种错误中的每一种，我们都有三种备选方案：

1. 在执行读取的代码中处理这个问题。
2. 抛出一个异常，让别人来处理这个问题（可能会终止程序）。
3. 无视这个问题。

碰巧，这三种是处理错误状况时非常常见的备选方案。因此，这是一个很好的例子，展示了我们对错误必须做的那种思考。

人们很容易说第三种备选方案——无视问题——总是不可接受的，但那样说就有点居高临下了。如果我是在为自己写一个无关紧要的小程序，我可以随心所欲，包括忘掉错误检查、任其带来糟糕的后果。然而，对于一个我可能想在写完之后的几个小时之外还要使用的程序，留下这样的错误恐怕是愚蠢的；而如果我想把这个程序分享给任何人，我就不应该在代码的错误检查中留下这样的漏洞。请注意，我们在这里刻意使用第一人称单数；用“我们”会产生误导。即便只涉及两个人，我们也不认为备选方案 3 是可以接受的。

备选方案 1 和 2 之间的选择则是真实存在的；也就是说，在一个给定的程序中，选择其中任何一种都可能有充分的理由。首先我们注意到，在大多数程序中，对于坐在键盘前的用户不再提供输入这

种情况，并没有什么局部而优雅的处理办法：输入流已经关闭，再让用户输入一个数字已经没什么意义了。我们可以重新打开 **cin**（使用 `cin.clear()`），但用户不太可能是意外关闭那个流的（你怎么会不小心按到 Ctrl+Z 呢？）。如果程序想要一个整数却遇到了“文件尾”，试图读取该整数的程序部分通常必须放弃，并指望程序的其他部分能够应对；也就是说，我们向用户请求输入的代码必须抛出一个异常。这意味着，选择并不在于“抛出异常”与“局部处理问题”之间，而在于我们应当局部处理哪些问题（如果有的话）。

10.7.1 把问题分解成可管理的部分

我们来试着在局部同时处理超出范围的输入和类型不对的输入：

```
cout << "Please enter an integer in the range 1 to 10 (inclusive):\n";
int n = 0;
while (true) {
    cin >> n;
    if (cin) {          // 我们得到一个整数；现在检查它
        if (1<=n && n<=10) break;
        cout << "Sorry "
             << n << " is not in the [1:10] range; please try again\n";
    }
    else if (cin.fail()) {   // 我们遇到了不是整数的东西
        cin.clear();         // 把状态置回 good();
                             // 我们想看看那些字符
        cout << "Sorry, that was not a number; please try again\n";
        for (char ch; cin>>ch && !isdigit(ch); )    // 丢弃非数字字符
            /* nothing */ ;
        if (!cin) error("no input");    // 我们没找到数字：放弃
        cin.unget();        // 把数字放回去，以便我们能读取该数
    }
    else {
        error("no input"); // eof 或 bad：放弃
    }
}
// 如果执行到这里，n 就在 [1:10] 内
```

这很乱，而且相当啰嗦。事实上，它太乱了，我们不会建议人们每次需要从用户那里得到一个整数时就写一遍这样的代码。另一方面，我们的确需要处理这些潜在的错误，因为人们确实会犯，那我们能做什么呢？代码之所以乱，是因为处理几个不同关注点的代码全都混在了一起：

- 读取值
- 提示用户输入
- 写出错误消息
- 跳过“坏的”输入字符
- 对照范围检查输入

让代码更清晰的办法，往往是把逻辑上互不相同的关注点分离到不同的函数中。例如，我们可以把看到“坏的”（即意料之外的）字符之后进行恢复的那部分代码分离出来：

```
void skip_to_int()
{
    if (cin.fail()) {           // 我们遇到了不是整数的东西
        cin.clear();            // 我们想看看那些字符
        for (char ch; cin>>ch; ) {    // 丢弃非数字字符
            if (isdigit(ch) || ch=='-') {
                cin.unget();    // 把数字放回去，
                                // 以便我们能读取该数
                return;
            }
        }
    }
    error("no input");          // eof 或 bad：放弃
}
```

有了这个“工具函数”skip_to_int()，我们就可以写

```
cout << "Please enter an integer in the range 1 to 10 (inclusive):\n";
int n = 0;
while (true) {
    if (cin>>n) {       // 我们得到一个整数；现在检查它
        if (1<=n && n<=10) break;
        cout << "Sorry " << n
             << " is not in the [1:10] range; please try again\n";
    }
    else {
        cout << "Sorry, that was not a number; please try again\n";
        skip_to_int();
    }
}
// 如果执行到这里，n 就在 [1:10] 内
```

这段代码好一些了，但它仍然太长、太乱，不适合在一个程序里用上很多次。除非经过（过多的）大量测试，我们绝不可能每次都把它写对。

我们真正想要的会是什么操作呢？一个合理的答案是“一个读取 int（任意 int）的函数，和另一个读取给定范围内 int 的函数”：

```
int get_int();                  // 从 cin 读取一个 int
int get_int(int low, int high); // 从 cin 读取一个 [low:high] 范围内的 int
```

如果有了它们，我们至少能够简单而正确地使用它们。它们并不难写：

```
int get_int()
{
    int n = 0;
    while (true) {
        if (cin >> n) return n;
        cout << "Sorry, that was not a number; please try again\n";
        skip_to_int();
    }
}
```

基本上，get_int() 会固执地一直读下去，直到找到一些它能解释为整数的数字。如果我们想从 get_int() 中脱身，就必须提供一个整数或文件尾（而文件尾会导致 get_int() 抛出异常）。

利用这个通用的 get_int()，我们就可以写出带范围检查的 get_int()：

```
int get_int(int low, int high)
{
    cout << "Please enter an integer in the range "
         << low << " to " << high << " (inclusive):\n";

    while (true) {
        int n = get_int();
        if (low<=n && n<=high) return n;
        cout << "Sorry "
             << n << " is not in the [" << low << ':' << high
             << "] range; please try again\n";
    }
}
```

这个 get_int() 和另一个一样固执。它会一直从不带范围检查的 get_int() 获取 int，直到得到的 int 落在预期的范围内。

现在，我们可以像这样可靠地读取整数了：

```
int n = get_int(1,10);
cout << "n: " << n << '\n';


int m = get_int(2,300);
cout << "m: " << m << '\n';
```

不过，别忘了要在某处捕获异常，以便在（可能很少见的）get_int() 实在无法为我们读到一个数的情况下，也能有像样的错误消息。

10.7.2 将对话与函数分离

get_int() 这两个函数仍然把读取与向用户写消息混在一起。对于一个简单的程序，这大概已经足够好了；但在一个大型程序中，我们可能想改变写给用户的消息。我们可能想这样调用 get_int()：

```
int strength = get_int(1,10, "enter strength", "Not in range, try again");
cout << "strength: " << strength << '\n';

int altitude = get_int(0,50000,
                       "Please enter altitude in feet",
                       "Not in range, please try again");
cout << "altitude: " << altitude << "f above sea level\n";
```

我们可以这样实现：

```
int get_int(int low, int high, const string& greeting, const string& sorry)
{
    cout << greeting << ": [" << low << ':' << high << "]\n";

    while (true) {
        int n = get_int();
        if (low<=n && n<=high) return n;
        cout << sorry << ": [" << low << ':' << high << "]\n";
    }
}
```

要拼出任意的消息很难，所以我们把消息“格式化”了。这通常是可以接受的；而要组织真正灵活的消息——例如为了支持多种自然语言（如阿拉伯语、孟加拉语、汉语、丹麦语、英语和法语）——则不是新手能胜任的任务。

注意，我们的解决方案仍然不完整：不带范围检查的 `get_int()` 还是会“喋喋不休”。更深层的要点是：在程序许多部分都要使用的“工具函数”不应该把消息“硬编码”在里边。此外，打算供许多程序使用的库函数根本就不应该向用户写东西——毕竟，库的编写者甚至可能不知道使用该库的程序是否运行在一台有人在看着屏幕的机器上。这也是我们的 `error()` 函数不直接写出错误消息的原因之一（§5.6.3）；一般而言，我们并不知道该写到哪儿去。

10.8 用户自定义输出运算符

为给定的类型定义输出运算符 << 通常是很简单的事。主要的设计问题在于，不同的人可能偏好不同的输出样式，因此很难就单一的格式达成一致。然而，即使没有哪种单一的输出格式能适合所有用途，为用户自定义类型定义 << 往往仍是个好主意。这样一来，我们至少可以在调试和早期开发期间轻而易举地写出该类型的对象。之后，我们还可以提供一个更复杂的 <<，允许用户提供格式化信息。另外，如果我们想要的输出样式与 << 所提供的不一样，我们也可以干脆绕开 <<，按我们应用碰巧喜欢的方式写出用户自定义类型的各个部分。

下面是 §9.8 中 `Date` 的一个简单输出运算符，它只是把年、月、日以逗号分隔放在圆括号里打印出来：

```
ostream& operator<<(ostream& os, const Date& d)
{
    return os << '(' << d.year()
              << ',' << d.month()
              << ',' << d.day() << ')';
}
```

这会把 2004 年 8 月 30 日打印成 (2004,8,30)。这种简单的元素列表表示，是我们对成员不多的类型通常采用的方式——除非我们有更好的想法或更具体的需求。

在 §9.6 中我们提到过，用户自定义的运算符是通过调用它的函数来处理的。这里我们就能看到一个具体的例子。给定 Date 的 << 定义，

```
cout << d1;
```

其中 d1 是一个 Date，其含义就是调用

```
operator<<(cout,d1);
```

注意 operator<<() 的第一个实参是 ostream&，并把它原样作为返回值返回。输出流就是这样一路传递下去的，因此你可以把输出操作“串接”起来。例如，我们可以这样输出两个日期：

```
cout << d1 << d2;
```

处理时会先解析第一个 <<，然后再解析第二个 <<：

```
cout << d1 << d2;   // 意即 operator<<(cout,d1) << d2;
                    // 意即 operator<<(operator<<(cout,d1),d2);
```

也就是说，先把 d1 输出到 cout，再把 d2 输出到作为第一个输出操作之结果的那个输出流。事实上，这三种写法中的任何一种都可以用来写出 d1 和 d2。不过我们知道哪一种更易读。

10.9 用户自定义输入运算符

为给定的类型和输入格式定义输入运算符 >>，基本上是一次错误处理的练习，因此可能相当棘手。

下面是 §9.8 中 Date 的一个简单输入运算符，它读取的格式正是上面定义的 << 所写出的格式：

```
istream& operator>>(istream& is, Date& dd)
{
    int y, m, d;
    char ch1, ch2, ch3, ch4;
    is >> ch1 >> y >> ch2 >> m >> ch3 >> d >> ch4;
    if (!is) return is;
    if (ch1!='(' || ch2!=',' || ch3!=',' || ch4!=')') {   // 糟糕：格式错误
        is.clear(ios_base::failbit);
        return is;
    }
    dd = Date{y,Date::Month(m),d};                        // 更新 dd
    return is;
}
```

这个 >> 会读取形如 (2004,8,20) 的条目，并试着用这三个整数构造一个 Date。一如既往，输入比输出更难对付。输入可能出——而且经常出——的问题，就是比输出多。

如果这个 >> 没有找到符合 (integer, integer, integer) 格式的内容，它就会把流置于非 good 的状态（fail、eof 或 bad），并保持目标 Date 不变。成员函数 clear() 用来设置 istream 的状态。显然，ios_base::failbit 会把流置入 fail() 状态。读取失败时保持目标 Date 不变是理想的做法；这往往能使代码更整洁。理想情况下，operator>>() 不应消费（丢弃）任何它没有用到的字符，但在本例中这太难做到了：在捕获到格式错误之前，我们可能已经读入了大量字符。举个例子，考虑 (2004, 8, 30}。只有当我们看到最后的 } 时，才知道碰上了格式错误，而一般而言我们不能指望把许多字符都退回流中。普遍保证的只有回退一个字符的 unget()。如果这个 operator>>() 读到了一个无效的 Date（例如 (2004,8,32)），Date 的构造函数就会抛出异常，从而把我们带出这个 operator>>()。

10.10 一个标准输入循环

在 §10.5 中，我们见过如何读写文件。不过那是在我们仔细考察错误（§10.6）之前，所以当时的输入循环只是简单假定可以从文件头一直读到文件尾。这个假定还算合理，因为我们常常会另行检查以确保文件是有效的。然而，我们经常希望边读边检查。下面是一个通用策略，其中假定 ist 是一个 istream：

```
for (My_type var; ist>>var; ) {   // 一直读到文件尾
    // 也许检查一下 var 是否有效
    // 对 var 做点什么
}
// 我们几乎无法从 bad 中恢复；除非万不得已，不要尝试：
if (ist.bad()) error("bad input stream");
if (ist.fail()) {
    // 它是可接受的终止符吗？
}
// 继续下去：我们遇到了文件尾
```

也就是说，我们把一串值读进变量，当再也读不到值时，就检查流的状态，看看原因何在。如§10.6所述，我们可以再改进一点：让 `istream` 在变 `bad` 时抛出 `failure` 类型的异常。这省得我们老是去检查它：

```
// 在某处：让 ist 在变 bad 时抛出异常：
ist.exceptions(ist.exceptions()|ios_base::badbit);
```

我们也可以决定指定一个字符作为终止符：

```
for (My_type var; ist>>var; ) {   // 一直读到文件尾
    // 也许检查一下 var 是否有效
    // 对 var 做点什么
}
if (ist.fail()) {                 // 用 '|' 作为终止符和/或分隔符
    ist.clear();
    char ch;
    if (!(ist>>ch && ch=='|')) error("bad termination of input");
}
// 继续下去：我们遇到了文件尾或终止符
```

如果我们不想接受终止符——也就是只接受文件尾作为结束——那么只需删掉调用 `error()` 之前的那个测试。不过，当你读取带有嵌套结构的文件时，终止符非常有用，例如一个存月度读数的文件，其中包含每日读数，每日读数又包含每小时读数，等等。所以我们会继续考虑终止字符这个可能性。

遗憾的是，那段代码还是有点乱。特别是，如果我们要读很多文件，反复写终止符测试就很烦人。我们可以写一个函数来处理这件事：

```
// 在某处：让 ist 在变 bad 时抛出异常：
ist.exceptions(ist.exceptions()|ios_base::badbit);

void end_of_loop(istream& ist, char term, const string& message)
{
    if (ist.fail()) {                          // 用 term 作为终止符和/或分隔符
        ist.clear();
        char ch;
        if (ist>>ch && ch==term) return;       // 一切正常
        error(message);
    }
}
```

这就把输入循环简化为

```
for (My_type var; ist>>var; ) {                // 一直读到文件尾
    // 也许检查一下 var 是否有效

    // ... 对 var 做点什么 ...
}
end_of_loop(ist,'|',"bad termination of file");   // 测试是否可以继续

// 继续下去：我们遇到了文件尾或终止符
```

`end_of_loop()` 在流不处于 `fail()` 状态时什么也不做。我们认为它足够简单，也足够通用，可以满足很多用途。

10.11 读取结构化文件

让我们用一个具体的例子来试试这个“标准循环”。一如既往，我们将用这个例子来说明广泛适用的设计和编程技术。假设你有一个温度读数文件，它的结构如下：

- 一个文件保存若干年（由若干月的读数组成）。
- 一个年以 `{ year` 开始，后跟一个表示年份的整数（如 1900），并以 `}` 结束。
- 一个年保存若干月（由若干天的读数组成）。
- 一个月以 `{ month` 开始，后跟一个三字母的月份名（如 `jan`），并以 `}` 结束。
- 一条读数保存一个时刻和一个温度。
- 一条读数以 `(` 开始，后跟日、时和温度，并以 `)` 结束。

例如：

```
{ year 1990 }
{year 1991 { month jun }}
{ year 1992 { month jan ( 1 0 61.5) }  {month feb (1 1 64) (2 2 65.2) } }
{year 2000
        { month feb (1 1 68 ) (2 3 66.66 ) ( 1 0 67.2)}
        {month dec (15 15 -9.2 ) (15 14 -8.8) (14 0 -2) }
}
```

这个格式有点怪。文件格式常常如此。业界有一种走向更规整、更具层次结构的文件（如 HTML 和 XML 文件）的趋势，但现实仍然是：我们很少能控制我们所需文件的输入格式。文件是什么样就是什么样，我们只能照读不误。如果格式实在太糟糕，或者文件里错误太多，我们可以写一个格式转换程序，把它转成更适合主程序的格式。另一方面，我们通常可以自行选择数据在内存中的表示来满足需要，也常常可以按需求与品味挑选输出格式。

那么，就让我们假定上面这个温度读数格式是别人给定的，我们只能接受。幸好，它带有自我标识的成分，例如年和月（有点像 HTML 或 XML）。另一方面，单条读数的格式就不太有帮助了。例如，要是有人把“某月第几日”的值和“一天中第几小时”的值弄颠倒了，或者有人生成的文件里温度是摄氏而程序却预期是华氏（或反过来），没有任何信息能帮上我们。我们只能将就着办。

10.11.1 内存中的表示

我们应该如何在内存中表示这些数据？显而易见的第一选择是三个类 `Year`、`Month` 和 `Reading`，与输入完全对应。在操作数据时，`Year` 和 `Month` 显然很有用：我们想比较不同年份的温度、计算月均值、比较一年中不同的月份、比较不同年份的同一个月份、把温度读数与日照记录和湿度读数配对，等等。基本上，`Year` 和 `Month` 与我们思考温度和天气的方式相吻合：`Month` 保存一个月的信息量，`Year` 保存一年的信息量。可是 `Reading` 呢？它是一个与某件硬件（传感器）相对应的低层概念。`Reading` 的数据（日、时、温度）很“怪”，只有在 `Month` 内部才有意义。它还是无结构的：谁也没有向我们承诺读数会按日或按时的顺序到来。基本上，每当想对读数做点有趣的事情，我们都必须先排序。

为了在内存中表示温度数据，我们做如下假设：

- 如果我们有一个月的任何读数，那么这个月的读数往往很多。
- 如果我们有某一天的任何读数，那么这一天的读数往往很多。

当情况如此时，把一个 `Year` 表示为 12 个 `Month` 组成的 `vector`，把一个 `Month` 表示为大约 30 个 `Day` 组成的 `vector`，把一个 `Day` 表示为 24 个温度（每小时一个），就是合理的。这样简单，而且易于为各种用途操作。于是，`Day`、`Month` 和 `Year` 就是简单的数据结构，各自带一个构造函数。由于我们打算在还不知道有哪些温度读数时就先创建作为 `Year` 一部分的 `Month` 和 `Day`，我们需要一个“不是读数”的概念，用来表示某个小时我们还没有（尚）读到数据。

```
const int not_a_reading = -7777;   // 比绝对零度还低
```

类似地，我们注意到常常会遇到没有数据的月份，于是引入了“不是一个月”的概念来直接表示这种情况，而不必翻遍所有天数去确认没有数据藏在哪里：

```
const int not_a_month = -1;
```

于是，三个关键的类就是

```
struct Day {
    vector<double> hour {vector<double>(24,not_a_reading)};
};
```

也就是说，一个 Day 有 24 个小时，每个都初始化为 not_a_reading。

```
struct Month {                      // 一个月的温度读数
    int month {not_a_month};        // [0:11] 一月是 0
    vector<Day> day {32};           // [1:31] 每天一个读数 vector
};
```

我们“浪费”掉 day[0]，以让代码保持简单。

```
struct Year {                       // 一年的温度读数，按月组织
    int year;                       // 正数 == 公元
    vector<Month> month {12};       // [0:11] 一月是 0
};
```

每个类基本上都是一个由“部件”组成的简单 vector，而且 Month 和 Year 分别有一个起标识作用的成员 month 和 year。

这里出现了几个“幻数”（例如 24、32 和 12）。我们力求避免在代码中使用这样的字面常量。这几个还算是相当基本的（一年中的月份数很少变化），而且不会在其余代码中用到。不过，我们之所以把它们留在代码里，主要是为了提醒你“幻数”的问题；符号常量几乎总是更好的选择（§7.6.1）。用 32 表示一个月的天数肯定需要解释一下；32 在这里显然是“幻”的。

我们为什么不写成

```
struct Day {
    vector<double> hour {24,not_a_reading};
};
```

这样本来更简单，但遗憾的是，我们会得到一个有两个元素（24 和 -1）的 vector。当我们想为一个 vector 指定元素个数、而整数又可以转换为它的元素类型时，很遗憾就必须使用（ ）初始化器语法（§18.2）。

10.11.2 读取结构化值

Reading 类只用来读取输入，因此还要简单：

```
struct Reading {
    int day;
    int hour;
    double temperature;
};
```

```
istream& operator>>(istream& is, Reading& r)
// 从 is 读取一条温度读数存入 r
// 格式: ( 3 4 9.7 )
// 检查格式, 但不操心数据的有效性
{
    char ch1;
    if (is>>ch1 && ch1!='(') {                    // 会不会是一条 Reading?
        is.unget();
        is.clear(ios_base::failbit);
        return is;
    }

    char ch2;
    int d;
    int h;
    double t;
    is >> d >> h >> t >> ch2;
    if (!is || ch2!=')') error("bad reading");  // 乱七八糟的读数
    r.day = d;
    r.hour = h;
    r.temperature = t;
    return is;
}
```

基本上，我们检查格式开头是否像样；如果不像样，就把流状态置为 `fail()` 并返回。这样我们还可以换个方式再去试着读取这些信息。另一方面，如果我们在读了些数据之后才发现格式不对，已经没有真正恢复的可能，就用 `error()` 退出。

`Month` 的输入操作与此基本相同，只是它必须读取任意数目的 `Reading`，而不是固定的一组值（如 `Reading` 的 `>>` 那样）：

```
istream& operator>>(istream& is, Month& m)
// 从 is 读取一个月存入 m
// 格式: { month feb ... }
{
    char ch = 0;
    if (is >> ch && ch!='{') {
        is.unget();
        is.clear(ios_base::failbit);              // 我们没能读到一个 Month
        return is;
    }

    string month_marker;
    string mm;
    is >> month_marker >> mm;
    if (!is || month_marker!="month") error("bad start of month");
    m.month = month_to_int(mm);
```

```
    int duplicates = 0;
    int invalids = 0;
    for (Reading r; is >> r; ) {
        if (is_valid(r)) {
            if (m.day[r.day].hour[r.hour] != not_a_reading)
                ++duplicates;
            m.day[r.day].hour[r.hour] = r.temperature;
        }
        else
            ++invalids;
    }
    if (invalids) error("invalid readings in month",invalids);
    if (duplicates) error("duplicate readings in month", duplicates);
    end_of_loop(is,'}',"bad end of month");
    return is;
}
```

我们稍后再回到 `month_to_int()`；它把月份的符号记法（如 `jun`）转换为 `[0:11]` 范围内的一个数。注意这里使用了 §10.10 的 `end_of_loop()` 来检查终止符。我们对无效和重复的 `Reading` 进行计数；说不定有人对此感兴趣。

`Month` 的 `>>` 在存入一条 `Reading` 之前会对它做一次粗略检查：

```
constexpr int implausible_min = -200;
constexpr int implausible_max = 200;


bool is_valid(const Reading& r)
// 一个粗略的测试
{
    if (r.day<1 || 31<r.day) return false;
    if (r.hour<0 || 23<r.hour) return false;
    if (r.temperature<implausible_min|| implausible_max<r.temperature)
        return false;
    return true;
}
```

最后，我们就可以读 `Year` 了。`Year` 的 `>>` 与 `Month` 的 `>>` 类似：

```
istream& operator>>(istream& is, Year& y)
// 从 is 读取一个年存入 y
// 格式: { year 1972 ... }
{
    char ch;
    is >> ch;
    if (ch!='{') {
        is.unget();
        is.clear(ios::failbit);
        return is;
    }

    string year_marker;
    int yy;
    is >> year_marker >> yy;
    if (!is || year_marker!="year") error("bad start of year");
    y.year = yy;

    while(true) {
        Month m;                                // 每轮都取一个干净的 m
        if(!(is >> m)) break;
        y.month[m.month] = m;
    }

    end_of_loop(is,'}',"bad end of year");
    return is;
}
```

我们本来更希望它“乏味地相似”，而不只是“相似”，但这里有一个重要的差别。请看一看那个读取循环。你是不是曾期待它像下面这样？

```
for (Month m; is >> m; )
    y.month[m.month] = m;
```

你大概应该期待过，因为我们迄今写的所有读取循环都是这个样子。实际上我们最初写的就是它——而它是错的。问题在于 `operator>>(istream& is, Month& m)` 并不会给 `m` 赋一个全新的值；它只是把来自 `Reading` 的数据加到 `m` 上。于是，反复执行 `is>>m` 就会不断往我们唯一的那个 `m` 里添加数据。糟糕！每一个新月份都会拿到这一年里之前所有月份的读数。每次执行 `is>>m` 时，我们都需要一个全新的、干净的 `Month` 来读入。最简单的做法就是把 `m` 的定义放进循环体内，使它每转一圈都被初始化一次。另一种做法是让 `operator>>(istream& is, Month& m)` 在读入之前先把一个空月份赋给 `m`，或者由循环来做这件事：

```
for (Month m; is >> m; ) {
    y.month[m.month] = m;
    m = Month{}; // “重新初始化” m
}
```

让我们试着使用它：

```
// 打开一个输入文件:
cout << "Please enter input file name\n";
string iname;
cin >> iname;
ifstream ist {iname};
if (!ifs) error("can't open input file",iname);

ifs.exceptions(ifs.exceptions()|ios_base::badbit);   // bad() 时抛出异常

// 打开一个输出文件:
cout << "Please enter output file name\n";
string oname;
cin >> oname;
ofstream ost {oname};
if (!ofs) error("can't open output file",oname);

// 读入任意数目的年:
vector<Year> ys;
while(true) {
    Year y;                     // 每轮都取一个全新初始化的 Year
    if (!(ifs>>y)) break;
    ys.push_back(y);
}
cout << "read " << ys.size() << " years of readings\n";

for (Year& y : ys) print_year(ofs,y);
```

我们把 `print_year()` 留作习题。

10.11.3 改变表示

为了让 `Month` 的 `>>` 能工作，我们需要提供一种读取月份符号表示的方法。为了对称，我们也将提供一种使用符号表示的配套写出。麻烦的写法是用一串 `if` 语句做转换：

```
if (s=="jan")
    m = 1;
else if (s=="feb")
    m = 2;
...
```

这不但麻烦，还把月份的名字硬编进了代码。更好的做法是把它们放在某个表中，这样即使我们必须改变符号表示，主程序也可以保持不变。我们决定用一个 vector<string> 来表示输入表示，外加一个初始化函数和一个查找函数：

```
vector<string> month_input_tbl = {
    "jan", "feb", "mar", "apr", "may", "jun", "jul",
    "aug", "sep", "oct", "nov", "dec"
};


int month_to_int(string s)
// s 是月份名吗？是则返回其下标 [0:11]，否则返回 -1
{
    for (int i=0; i<12; ++i) if (month_input_tbl[i]==s) return i;
    return -1;
}
```

顺带一提：C++ 标准库确实提供了一种更简单的做法。参见 §21.6.1 的 map<string,int>。

当我们要产生输出时，就遇到了相反的问题。我们手里有一个表示月份的 int，想要一个可以打印的符号表示。我们的解法在根本上类似，只是不用从 string 到 int 的表，而是用从 int 到 string 的表：

```
vector<string> month_print_tbl = {
    "January", "February", "March", "April", "May", "June", "July",
    "August", "September", "October", "November", "December"
};


string int_to_month(int i)
// 月份 [0:11]
{
    if (i<0 || 12<=i) error("bad month index");
    return month_print_tbl[i];
}
```

那么，你真的把那些代码和解释都读过了吗？还是说你的目光已经呆滞，直接跳到了末尾？记住，学会写好代码的最容易的办法就是大量阅读代码。信不信由你，我们在这个例子中使用的技术都很简单，但若无人指点，要自己发现它们却并不容易。读取数据是基础。正确地写循环（把你用到的每个变量都正确地初始化）是基础。在不同表示之间转换是基础。也就是说，你终将学会做这些事。唯一的问题在于：你能否把它们学好，以及你能否在牺牲太多睡眠之前学会这些基本技术。

小试身手

1. 开始编写 §10.4 中讨论的处理点的程序。先定义数据类型 `Point`，它有两个坐标成员 **x** 和 **y**。
2. 使用 §10.4 的代码和讨论，提示用户输入七个 `(x,y)` 对。在数据录入时，把它存入一个名为 `original_points` 的 `Point` 的 `vector`。
3. 打印 `original_points` 中的数据，看看它是什么样子。
4. 打开一个 `ofstream`，把每个点输出到名为 `mydata.txt` 的文件。在 Windows 上，我们建议用 `.txt` 后缀，以便用普通文本编辑器（如“写字板”）更容易查看数据。
5. 关闭那个 `ofstream`，然后为 `mydata.txt` 打开一个 `ifstream`。从 `mydata.txt` 读取数据，存入一个名为 `processed_points` 的新 `vector`。
6. 打印两个 `vector` 的数据元素。
7. 比较这两个 `vector`，如果元素个数或元素的值不同，就打印 **Something's wrong!**。

要点回顾

1. 处理输入和输出时，大多数现代计算机是如何应对设备多样性的？
2. 从根本上说，`istream` 做什么？
3. 从根本上说，`ostream` 做什么？
4. 从根本上说，文件是什么？
5. 什么是文件格式？
6. 说出四种可能需要程序为之做 I/O 的不同类型的设备。
7. 读取文件有哪四个步骤？
8. 写文件有哪四个步骤？
9. 说出并定义四种流状态。
10. 讨论以下输入问题各可以如何解决：
    - 用户键入了超出范围的值
    - 没有得到值（文件尾）
    - 用户键入了类型不对的东西
11. 输入通常在哪方面比输出难？
12. 输出通常在哪方面比输入难？
13. 我们为什么（常常）想把输入输出与计算分开？
14. `istream` 的成员函数 `clear()` 最常见的两种用途是什么？
15. 对用户自定义类型 `X`，`<<` 和 `>>` 通常的函数声明是什么？

术语

bad()　设备驱动程序　istream
缓冲区　ifstream　ofstream
clear()　输入设备　open()
close()　输入运算符　ostream
eof()　iostream　流状态
fail()　文件　终止符
文件　输出设备　unget()
good()　输出运算符

习题

1. 编写一个程序，对一个由空白分隔的整数组成的文件，求其中所有数之和。
2. 编写一个程序，创建一个数据文件，其形式为 §10.5 定义的温度 `Reading` 类型。为了测试，向文件中填入至少 50 条“温度读数”。把这个程序命名为 **`store_temps.cpp`**，把它创建的文件命名为 **`raw_temps.txt`**。
3. 编写一个程序，把习题 2 创建的 **`raw_temps.txt`** 中的数据读入一个 `vector`，然后计算你数据集中温度的均值和中位数。把这个程序命名为 **`temp_stats.cpp`**。
4. 修改习题 2 的 **`store_temps.cpp`** 程序，为温度加上后缀 **c**（摄氏）或 **f**（华氏）。然后修改 **`temp_stats.cpp`** 程序，测试每条温度，并在放入 `vector` 之前把摄氏读数转换为华氏。
5. 编写 §10.11.2 提到的函数 **`print_year()`**。
6. 定义一个 **`Roman_int`** 类，用于保存罗马数字（作为 `int`），并带 `<<` 和 `>>`。为 **`Roman_int`** 提供一个返回其 `int` 值的成员 **`as_int()`**，使得若 **r** 是一个 **`Roman_int`**，我们就能写 `cout << "Roman" << r << " equals " << r.as_int() << 'n';`。
7. 改造第 7 章的计算器，让它接受罗马数字而不是通常的阿拉伯数字，例如 **XXI + CIV == CXXV**。
8. 编写一个程序，接受两个文件名，产生一个新文件，其内容是第一个文件的内容后跟第二个文件的内容；也就是说，该程序把两个文件串接起来。
9. 编写一个程序，取两个包含已排序的空白分隔单词的文件，把它们按序合并。
10. 为第 7 章的计算器添加命令 **from x**，使它从文件 **x** 获取输入。为计算器添加命令 **to y**，使它把输出（标准输出和错误输出都算）写入文件 **y**。依据 §7.3 的想法编写一组测试用例，并用它们测试计算器。讨论你会如何把这些命令用于测试。
11. 编写一个程序，对一个文本文件中所有由空白分隔的整数求和。例如，`bears: 17 elephants 9 end` 应当输出 **26**。

后记

计算工作的很大一部分，是把大量数据从一处搬到另一处，例如把文本从文件复制到屏幕，或者把音乐从计算机传到 MP3 播放器上。通常，数据在途中还需要做某种变换。iostream 库就是处理这类任务的一种方式，其中数据可以被看作一个由值组成的序列（一个流）。输入和输出在常见编程任务中所占的比例大得出人意料。这部分是因为我们（或我们的程序）需要大量数据，部分是因为数据进入系统的地方正是大量错误可能发生的地方。所以，我们必须力求让 I/O 保持简单，并尽量减少坏数据“溜进”我们系统的机会。

11 定制输入与输出

“尽可能简单：
但不要过于简单。”
——阿尔伯特·爱因斯坦

在本章中，我们将集中讨论如何把第 10 章介绍的通用 iostream 框架调整为符合特定需求与品味的形式。这会涉及大量杂乱的细节——它们或源于人们对所读内容的人类感受，或源于文件使用上的实际限制。最后一个例子展示了一个输入流的设计，对它你可以指定分隔符的集合。

11.1 规则性与不规则性
11.2 输出格式化
11.2.1 整数输出
11.2.2 整数输入
11.2.3 浮点数输出
11.2.4 精度
11.2.5 域
11.3 文件打开与定位
11.3.1 文件打开模式
11.3.2 二进制文件
11.3.3 文件中的定位
11.4 字符串流
11.5 行导向输入
11.6 字符分类
11.7 使用非标准分隔符
11.8 还有太多太多的内容

11.1 规则性与不规则性

iostream 库——ISO C++ 标准库中负责输入/输出的部分——为文本的输入与输出提供了一个统一且可扩展的框架。所谓“文本”，我们指的是任何能够表示为字符序列的东西。因此，在谈论输入与输出时，我们可以把整数 1234 也当作文本，因为它可以用 1、2、3、4 这四个字符写出来。

到目前为止，我们把所有输入源都当作等价的对象来处理。有时这还不够。例如，文件与其他输入源（如通信连接）的不同之处在于，我们可以对单个字节进行寻址。类似地，我们一直假定对象的类型完全决定了其输入和输出的布局。这并不完全正确，而且也不够充分。例如，我们常常想指定浮点数输出时使用的位数（即它的精度，precision）。本章将介绍许多按我们的需求裁剪输入与输出的方法。

作为程序员，我们偏爱规则性：统一对待所有内存中的对象，等同对待所有输入源，对进入和离开系统的对象的表示方式施加单一标准，这样得到的代码最简洁、最简单、最易维护，而且往往最高效。然而，程序的存在是为了服务于人，而人有着强烈的偏好。因此，作为程序员，我们必须在程序复杂性与迁就用户个人品味之间努力寻求平衡。

11.2 输出格式化

人们非常在意输出中那些看似细枝末节的地方。例如，对一位物理学家来说，1.25（保留小数点后两位）可能与 1.24670477 大不相同；对一位会计师来说，(1.25) 在法律意义上可以不同于 (1.2467)，并

且与 1.25 完全不同（在财务文件中，圆括号有时用来表示亏损，即负值）。作为程序员，我们的目标是让输出尽可能清晰、尽可能接近程序“消费者”的预期。输出流（ostream）为内置类型的输出格式化提供了多种方式。对于用户自定义类型，定义合适的 « 运算符就是程序员自己的责任了。

输出的细节、精化和选项看起来数不胜数，输入也有相当多。例如：表示小数点所用的字符（通常是点或逗号）、输出货币值的方式、把 true 表示成单词 true（或法语 vrai、德语 sandt）而不是数字 1 的方式、处理非 ASCII 字符集（如 Unicode）的方式，以及限制读入 string 的字符数的方式。这些设施在你需要它们之前往往显得乏味，所以我们将它们的描述留给手册和专门著作，如 Langer 的 *Standard C++ IOStreams and Locales*、Stroustrup 的 *The C++ Programming Language* 第 38 章和第 39 章，以及 ISO C++ 标准的 §22 和 §27。这里我们将介绍最常用的特性和一些一般概念。

11.2.1　整数输出

整数值可以按八进制（octal，以 8 为基的数制）、十进制（decimal，我们常用的以 10 为基的数制）和十六进制（hexadecimal，以 16 为基的数制）输出。如果你不了解这些数制，请在继续之前先阅读 §A.2.1.1。大多数输出使用十进制。十六进制在输出与硬件相关的信息时很流行，原因是一位十六进制数字恰好表示一个 4 位（bit）的值。于是，两位十六进制数字可以表示一个 8 位字节的值，四位十六进制数字给出 2 个字节（这常是一个半字，half word）的值，八位十六进制数字可以表示 4 个字节（这常常是一个字（word）或寄存器的大小）。当 C++ 的前身 C 在 20 世纪 70 年代首次被设计出来时，八进制曾流行用于表示位模式，但如今已很少使用。

我们可以指定输出（十进制）值 1234 为十进制、十六进制（常简称为“hex”）和八进制：

```
cout << 1234 << "\t(decimal)\n"
          << hex << 1234 << "\t(hexadecimal)\n"
          << oct << 1234 << "\t(octal)\n";
```

字符 '\t' 是“制表符”（tab，tabulation character 的简称）。这会打印

```
1234  (decimal)
4d2   (hexadecimal)
2322  (octal)
```

记号 << hex 和 << oct 并不输出值。<< hex 是告知流：此后所有整数值都应以十六进制显示；<< oct 则是告知流：此后所有整数值都应以八进制显示。例如：

```
cout << 1234 << '\t' << hex << 1234 << '\t' << oct << 1234 << '\n';
cout << 1234 << '\n';  // 八进制仍然保持有效
```

它产生

```
1234  4d2   2322
2322  // 整数将继续以八进制显示，直至改变
```

注意，最后一行输出是八进制的；也就是说，oct、hex 和 dec（十进制）会“保持有效”（stick）——它们对输出的每个整数值都起作用，直到我们另行告知流。像 hex 和 oct 这样用于改变流行为的术语称为操纵器（manipulator）。

试一试

以十进制、十六进制和八进制形式输出你的出生年份。为每个值加上标签。使用制表符将你的输出按列对齐。然后输出你的年龄。

看到非 10 基的值常常会令人困惑。例如，除非我们另有说明，你会假定 11 表示（十进制）数 11，而不是 9（八进制的 11）或 17（十六进制的 11）。为缓解这类问题，我们可以要求 ostream 显示所打印的每个整数的数制。例如：

```
cout << 1234 << '\t' << hex << 1234 << '\t' << oct << 1234 << '\n';
cout << showbase << dec;          // 显示数制
 cout << 1234 << '\t' << hex << 1234 << '\t' << oct << 1234 << '\n';
```

它打印

```
1234   4d2     2322
1234   0x4d2   02322
```

因此，十进制数没有前缀，八进制数有前缀 0，十六进制值有前缀 0x（或 0X）。这恰好是 C++ 源代码中整数字面值的记法。例如：

```
cout << 1234 << '\t' << 0x4d2 << '\t' << 02322 << '\n';
```

按十进制形式，它将打印

```
1234   1234   1234
```

你可能已经注意到，showbase 与 oct 和 hex 一样会保持有效。操纵器 noshowbase 逆转 showbase 的作用，恢复为默认行为，即显示每个数而不带其数制。

总结起来，整数输出操纵器有：

整数输出操纵器	
oct	使用以 8 为基（八进制）的记法
dec	使用以 10 为基（十进制）的记法
hex	使用以 16 为基（十六进制）的记法
showbase	八进制加前缀 0，十六进制加前缀 0x
noshowbase	不使用前缀

11.2.2　整数输入

默认情况下，» 假定数使用十进制记法，但你可以让它读取十六进制或八进制整数：

```
int a;
int b;
int c;
int d;
cin >> a >> hex >> b >> oct >> c >> d;
cout << a << '\t' << b << '\t' << c << '\t' << d << '\n';
```

如果你键入

```
1234  4d2  2322  2322
```

它将打印

```
1234  1234  1234  1234
```

注意，这意味着 oct、dec 和 hex 对输入也会“保持有效”，就像它们对输出那样。

试一试

把上面的代码片段补全为一个完整的程序。尝试建议的输入；然后键入

```
1234  1234  1234  1234
```

解释其结果。尝试其他输入，看看会发生什么。

你可以让 » 接受并正确解释 0 和 0x 前缀。为此，你需要“取消设置”所有默认项。例如：

```
cin.unsetf(ios::dec);  // 不假定十进制（这样 0x 可以表示十六进制）
cin.unsetf(ios::oct);   // 不假定八进制（这样 12 可以表示十二）
cin.unsetf(ios::hex);  // 不假定十六进制（这样 12 可以表示十二）
```

流的成员函数 unsetf() 清除作为实参给出的标志（一个或多个）。现在，如果你写

```
cin >>a >> b >> c >> d;
```

并输入

```
1234   0x4d2   02322   02322
```

你会得到

```
1234   1234
1234
1234
```

11.2.3 浮点数输出

如果你直接与硬件打交道，就需要十六进制（或可能八进制）记法。类似地，如果你从事科学计算，就必须处理浮点值的格式化。浮点值用 iostream 操纵器处理的方式与整数值非常相似。例如：

```
cout << 1234.56789 << "\t\t(defaultfloat)\n"    // \t\t 使列对齐
          << fixed << 1234.56789 << "\t(fixed)\n"
          << scientific << 1234.56789 << "\t(scientific)\n";
```

它打印

```
1234.57             (general)
1234.567890         (fixed)
1.234568e+003       (scientific)
```

操纵器 `fixed`、`scientific` 和 `defaultfloat` 用于选择浮点格式；`defaultfloat` 是默认格式（也称为 general 格式）。现在，我们可以写

```
cout << 1234.56789 << '\t'
          << fixed << 1234.56789 << '\t'
          << scientific << 1234.56789 << '\n';
cout << 1234.56789 << '\n';                          // 浮点格式“保持有效”
cout << defaultfloat << 1234.56789 << '\t'   // 浮点数输出的默认格式
          << fixed << 1234.56789 << '\t'
          << scientific << 1234.56789 << '\n';
```

它打印

```
1234.57    1234.567890    1.234568e+003
1.234568e+003                          // scientific 操纵器“保持有效”
1234.57    1234.567890    1.234568e+003
```

总结起来，基本的浮点数输出格式化操纵器有：

浮点数格式	
`fixed`	使用定点记法
`scientific`	使用尾数加指数的记法；尾数总是处于 `[1:10)` 范围内，也就是说，小数点前只有单个非零数字
`defaultfloat`	在 `fixed` 和 `scientific` 之间选择，在 `defaultfloat` 的精度范围内给出数值上最精确的表示

11.2.4 精度

默认情况下，浮点值以 `defaultfloat` 格式打印，共使用六位数字。系统会选择最合适的格式，并对数进行舍入，以给出仅用六位数字就能打印的最佳近似（这是 `defaultfloat` 格式的默认精度）。例如：

```
1234.567 prints as 1234.57
1.2345678 prints as 1.23457
```

舍入规则是通常的四舍五入规则：0 到 4 舍去（向零的方向），5 到 9 进位（远离零的方向）。注意，浮点格式化只作用于浮点数，因此

```
1234567 prints as 1234567 (because it's an integer)
1234567.0 prints as 1.23457e+006
```

在后一种情况下，ostream 判定 1234567.0 无法用 `fixed` 格式只用六位数字打印，于是切换到 `scientific` 格式以保留最精确的表示。基本上，`defaultfloat` 格式在 `scientific` 与 `fixed` 格式之间进行选择，以便在 general 格式（默认共六位数字）的精度范围内，为用户呈现浮点值最精确的表示。

试一试

编写一些代码，把数 1234567.89 打印三次：先用 `defaultfloat`，再用 `fixed`，然后用 `scientific`。哪种输出形式为用户呈现了最精确的表示？解释原因。

程序员可以用操纵器 `setprecision()` 设置精度。例如：

```
cout << 1234.56789 << '\t'
          << fixed << 1234.56789 << '\t'
          << scientific << 1234.56789 << '\n';
cout << defaultfloat << setprecision(5)
          << 1234.56789 << '\t'
          << fixed << 1234.56789 << '\t'
          << scientific << 1234.56789 << '\n';
cout << defaultfloat << setprecision(8)
          << 1234.56789 << '\t'
          << fixed << 1234.56789 << '\t'
          << scientific << 1234.56789 << '\n';
```

它打印（注意舍入）

```
1234.57    1234.567890    1.234568e+003
1234.6     1234.56789     1.23457e+003
1234.5679  1234.56789000  1.23456789e+003
```

精度的定义如下：

浮点数精度	
`defaultfloat`	精度是数字的总个数
`scientific`	精度是小数点后的数字个数
`fixed`	精度是小数点后的数字个数

除非有理由不这样做，请使用默认设置（`defaultfloat` 格式、精度 6）。通常不这样做的理由是“因为我们需要更高的输出精度”。

11.2.5　域

使用 scientific 和 fixed 格式，程序员可以精确控制一个值在输出时占用多少空间。这对打印表格等显然很有用。整数值的等价机制称为域（field）。使用“设置域宽”操纵器 setw()，你可以精确指定一个整数值或字符串值将占据多少个字符位置。例如：

```
cout << 123456                                  // 未使用域
          <<'|'<< setw(4) << 123456 << '|'      // 123456 放不进 4 字符的域
          << setw(8) << 123456 << '|'           // 域宽设为 8
          << 123456 << "|\n";                   // 域宽不保持有效
```

它打印

```
123456|123456|  123456|123456|
```

注意首先看第三次出现的 123456 之前的两个空格。对一个八字符域中的六位数来说，这正是我们所期望的。然而，123456 并没有被截断以放入四字符的域。为什么没有？|1234| 或 |3456| 或许会被认为是对四字符域说得过去的输出。然而，那会在没有任何警告提示可怜的读者出了问题的情况下，完全改变打印的值。ostream 不这样做；它宁可破坏输出格式。“坏的格式”几乎总是比“坏的数据”更可取。在域最常见的用途（例如打印表格）中，“溢出”在视觉上非常显眼，因而能够被纠正。

域也可用于浮点数和字符串。例如：

```
cout << 12345 <<'|'<< setw(4) << 12345 << '|'
          << setw(8) << 12345 << '|' << 12345 << "|\n";
cout << 1234.5 <<'|'<< setw(4) << 1234.5 << '|'
          << setw(8) << 1234.5 << '|' << 1234.5 << "|\n";
cout << "asdfg" <<'|'<< setw(4) << "asdfg" << '|'
          << setw(8) << "asdfg" << '|' << "asdfg" << "|\n";
```

它打印

```
12345|12345|    12345|12345|
1234.5|1234.5|  1234.5|1234.5|
asdfg|asdfg| asdfg|asdfg|
```

注意，域宽“不保持有效”。在这三种情况中，第一个和最后一个值都以默认的“需要多少字符就用多少字符”格式打印。换句话说，除非你在输出操作之前立即设置域宽，否则“域”的概念不会被使用。

试一试

制作一个简单的表格，包含你自己和至少五位朋友的姓（last name）、名（first name）、电话号码和电子邮件地址。试验不同的域宽，直到你满意地认为表格呈现得不错为止。



11.3 文件打开与定位

从 C++ 的视角看，文件是对操作系统所提供内容的一种抽象。如 §10.3 所述，文件就是一个从 0 开始编号的字节序列：

问题在于我们如何访问这些字节。使用 iostream 时，这在很大程度上取决于我们打开文件并将流与之关联时所作的选择。流的属性决定了我们在打开文件之后能执行哪些操作，以及这些操作的含义。最简单的例子是：如果我们为一个文件打开 istream，就可以从该文件读取；而如果我们用 ostream 打开一个文件，就可以向它写入。

11.3.1 文件打开模式

你可以用几种模式之一打开文件。默认情况下，ifstream 以读方式打开其文件，ofstream 以写方式打开其文件。这满足了大多数常见需求。不过，你还可以在若干选项中进行选择：

文件流打开模式	
`ios_base::app`	追加（即添加到文件末尾）
`ios_base::ate`	“at end”（打开并定位到文件末尾）
`ios_base::binary`	二进制模式——当心系统相关的行为
`ios_base::in`	用于读取
`ios_base::out`	用于写入
`ios_base::trunc`	将文件截断为 0 长度

文件模式可以选择性地在文件名之后指定。例如：

```
ofstream of1 {name1};                    // 默认为 ios_base::out
ifstream if1 {name2};                    // 默认为 ios_base::in

ofstream ofs {name, ios_base::app};      // ofstream 默认已包含
                                         // ios_base::out
fstream fs {"myfile", ios_base::in|ios_base::out};   // 读和写
```

最后一个例子中的 | 是“按位或”运算符（§A.5.5），如上所示，它可用来组合模式。app 选项在写日志文件时很常用，因为写日志总是添加到文件末尾。

在每种情况下，打开文件的确切效果都可能取决于操作系统；如果操作系统无法按某种特定方式满足打开文件的请求，结果将是一个不处于 good() 状态的流：

```
if (!fs)     // 唉：我们不能那样打开那个文件
```

为读取而打开文件失败最常见的原因是文件不存在（至少不存在于我们使用的那个名字之下）：

```
ifstream ifs {"redungs"};
if (!ifs)   // 错误：不能打开 "readings" 进行读取
```

在这个例子里，我们猜测问题可能出在拼写错误上。

注意，通常情况下，如果你试图为输出打开一个不存在的文件，操作系统会创建一个新文件；但（幸好）如果你试图为输入打开一个不存在的文件，则不会创建：

```
ofstream ofs {"no-such-file"};          // 创建名为 no-such-file 的新文件
ifstream ifs {"no-file-of-this-name"};  // 错误：ifs 将不会是 good()
```

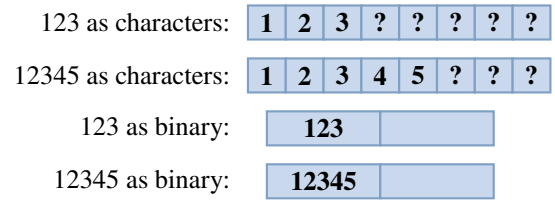
不要在文件打开模式上自作聪明。操作系统对“不常见”模式的处理并不一致。只要有可能，就坚持从以 istream 方式打开的文件读取、向以 ostream 方式打开的文件写入。

11.3.2　二进制文件

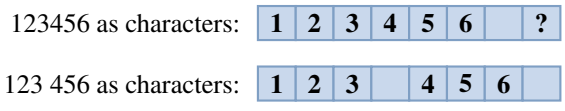
在内存中，我们可以把数 123 表示为整数值，也可以表示为字符串值。例如：

```
int n = 123;
string s = "123";
```

在第一种情况下，123 作为一个（二进制）数存储，所占内存量与其他所有 `int` 相同（在 PC 上是 4 个字节，即 32 位）。假如我们选择的是值 12345，同样会使用这 4 个字节。在第二种情况下，123 作为一个由三个字符组成的字符串存储。假如我们选择的是字符串值 `"12345"`，它就会占用五个字符（再加上管理一个 `string` 的固定开销）。我们可以像下面这样图示这一点（使用普通的十进制和字符表示，而不是计算机内部实际使用的二进制表示）：



当我们使用字符表示时，就必须用某个字符来表示内存中一个数的结束，正如我们在纸面上所做的那样：123456 是一个数，而 123 456 是两个数。在“纸面上”，我们用空格字符表示数的结束。在内存中，我们也可以这样做：



存储固定大小的二进制表示（例如一个 `int`）与可变长度的字符串表示（例如一个 `string`）之间的区别，在文件中同样存在。默认情况下，iostream 处理的是字符表示；也就是说，istream 读取一个字符序列，并将它转换为一个所需类型的对象；ostream 则取得一个指定类型的对象，将它转换为字符序列并写出。不过，也可以请求 istream 和 ostream

简单地在文件之间来回拷贝字节。这称为二进制 I/O（binary I/O），通过以 `ios_base::binary` 模式打开文件来请求。下面是一个读写整数二进制文件的例子。其中专门处理“二进制”的关键行在下面解释：

```
int main()
{
        // 打开一个 istream，从文件进行二进制输入：
        cout << "Please enter input file name\n";
        string iname;
        cin >> iname;
        ifstream ifs {iname,ios_base::binary};   // 注意：流模式
                // binary 告知流不要对这些字节自作聪明
        if (!ifs) error("can't open input file ",iname);

        // 打开一个 ostream，向文件进行二进制输出：
        cout << "Please enter output file name\n";
        string oname;
        cin >> oname;
        ofstream ofs {oname,ios_base::binary};   // 注意：流模式
                // binary 告知流不要对这些字节自作聪明
        if (!ofs) error("can't open output file ",oname);

        vector<int> v;

        // 从二进制文件读取：
        for(int x; ifs.read(as_bytes(x),sizeof(int)); )   // 注意：读取字节
                v.push_back(x);

        // . . . 对 v 做些处理 . . .

        // 写入二进制文件：
        for(int x : v)
                ofs.write(as_bytes(x),sizeof(int));   // 注意：写入字节
        return 0;
}
```

我们以 ios_base::binary 作为流模式打开文件：

```
ifstream ifs {iname, ios_base::binary};

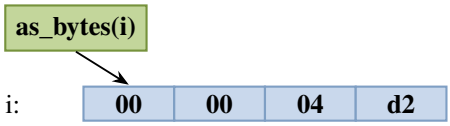
ofstream ofs {oname, ios_base::binary};
```

在这两种情况中，我们都选择了更麻烦、但往往更紧凑的二进制表示。当我们从面向字符的 I/O 转向二进制 I/O 时，我们放弃了常用的 » 和 « 运算符。这两个运算符专门按照默认约定把值转换为字符序列（例如，字符串 "asdf" 转换为字符 a、s、d、f，整数 123 转换为字符 1、2、3）。如果我们想要的正是这种效果，就不必声明 binary——默认方式就足够了。只有当我们（或其他人）认为自己总有可能比默认方式做得更好时，才使用 binary。我们用 binary 告知流：不要对这些字节自作聪明。

对一个 int 我们可能做什么“聪明事”呢？显而易见的就是把一个 4 字节的 int 存进 4 个字节；也就是说，我们可以查看该 int 在内存中的表示（4 个字节的序列），把这些字节传送到文件中。之后，我们可以用同样的方式把那些字节读回来，重新组装成这个 int：

```
ifs.read(as_bytes(i),sizeof(int))          // 注意：读取字节
ofs.write(as_bytes(v[i]),sizeof(int))      // 注意：写入字节
```

ostream 的 `write()` 和 istream 的 `read()` 都接受一个地址（这里由 `as_bytes()` 提供）和一个字节数（字符数），后者是我们用 `sizeof` 运算符得到的。该地址应当指向存放我们要读写的值的内存的首字节。例如，如果我们有一个值为 1234 的 `int`，就会得到 4 个字节（用十六进制记法）00、00、04、d2：



`as_bytes()` 函数用于获取对象表示的首字节地址。它可以——使用一些尚未讲解的语言设施（§17.8 和 §19.3）——定义如下：

```
template<class T>
char* as_bytes(T& i)                       // 把一个 T 当作字节序列对待
{
        void* addr = &i;                   // 获取存储该对象的内存
                                           // 的首字节地址
        return static_cast<char*>(addr); // 把那块内存当作字节对待
}
```

为了获取一个变量的“原始字节”，就必须使用 `static_cast` 进行（不安全的）类型转换。地址的概念将在第 17 章和第 18 章中详细探讨。这里我们只是展示如何把内存中的任何对象当作字节序列对待，以供 `read()` 和 `write()` 使用。

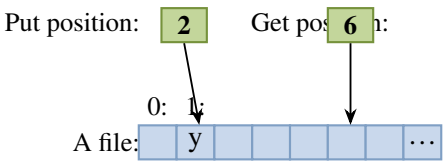
这种二进制 I/O 杂乱、有点复杂而且容易出错。然而，作为程序员，我们并不总有挑选文件格式的自由，所以偶尔就必须使用二进制 I/O，只因为那就是别人为我们要读写的文件选定的格式。另一种可能是，选择非字符表示确实有充分的逻辑理由。典型的例子是图像或声音文件：对它们来说不存在合理的字符表示——一张照片或一段音乐基本上就是一堆比特。

iostream 库默认提供的字符 I/O 是可移植的、人类可读的，并且得到类型系统的合理支持。只要能有所选择就使用它，除非真的不得已，否则不要碰二进制 I/O。

11.3.3　文件中的定位

只要可以，就从文件头到文件尾顺序读写文件。这是最简单、最不容易出错的方式。很多时候，当你觉得必须修改文件时，更好的解决办法是生成一个包含改动的新文件。

然而，如果实在不得已，你可以使用定位（positioning）在文件中选取读写操作的特定位置。基本上，每个为读取而打开的文件都有一个“读/get 位置”，每个为写入而打开的文件都有一个“写/put 位置”：



可以这样使用它：

```
fstream fs {name};    // 为输入和输出打开
if (!fs) error("can't open ",name);

fs.seekg(5);          // 移动读取位置（g 表示 “get”）到 5（第 6 个字符）
char ch;
fs>>ch;               // 读取并递增读取位置
cout << "character[5] is " << ch << ' (' << int(ch) << ")\n";

fs.seekp(1);          // 移动写入位置（p 表示 “put”）到 1
fs<<'y';              // 写入并递增写入位置
```

注意，`seekg()` 和 `seekp()` 会递增各自的位置，所以上图表示的是程序执行之后的状态。

请务必小心：使用定位时几乎没有任何运行时错误检查。特别是，如果你试图定位（使用 `seekg()` 或 `seekp()`）到文件末尾之外，其行为是未定义的，而且操作系统在这类情况下的表现确实各不相同。

11.4　字符串流

你可以把一个 `string` 用作 istream 的数据来源，或用作 ostream 的输出目标。从一个 `string` 读取的 istream 称为 istringstream；把写入其中的字符存储到一个 `string` 中的 ostream 称为 ostringstream。例如，istringstream 可用于从 `string` 中提取数值：

```
double str_to_double(string s)
          // 如果可能，将 s 中的字符转换为浮点值
{
          istringstream is {s};     // 创建一个流，以便我们能从 s 读取
          double d;
          is >> d;
          if (!is) error("double format error: ",s);
          return d;
}

double d1 = str_to_double("12.4");                 // 测试
double d2 = str_to_double("1.34e-3");
double d3 = str_to_double("twelve point three");   // 将调用 error()
```

如果我们试图读取超出 istringstream 的字符串末尾的内容，istringstream 就会进入 `eof()` 状态。这意味着我们可以对 istringstream 使用 “通常的输入循环”；istringstream 确实是一种 istream。

反过来，对于一个要求简单 `string` 实参的系统（例如 GUI 系统，参见 §16.5），ostringstream 可以用来为其格式化输出。例如：

```
void my_code(string label, Temperature temp)
{
          // . . .
          ostringstream os;    // 用于组装消息的流
          os << setw(8) << label << ": "
                    << fixed << setprecision(5) << temp.temp << temp.unit;
          someobject.display(Point(100,100), os.str().c_str());
          // . . .
}
```

ostringstream 的成员函数 str() 返回由向 ostringstream 输出操作所构成的 string。c_str() 是 string 的成员函数，它返回 C 风格字符串，许多系统接口都要求这种形式。

当我们想把实际的 I/O 与处理过程分开时，通常就会用到 stringstream。例如，str_to_double() 的 string 实参通常来自文件（如 web 日志）或键盘。类似地，我们在 my_code() 中构造的消息最终会写到屏幕的某个区域上。例如，在 §11.7 中，我们将用一个 stringstream 从输入中过滤掉不良字符。因此，可以把 stringstream 看作一种按特殊需要与偏好裁剪 I/O 的机制。

ostringstream 的一个简单用途是通过串接来构造字符串。例如：

```
int seq_no = get_next_number();            // 获取日志文件的编号
ostringstream name;
name << "myfile" << seq_no << ".log";      // 如 myfile17.log
ofstream logfile{name.str()};              // 如打开 myfile17.log
```

通常，我们用一个 string 来初始化 istringstream，然后用输入操作从该字符串读取字符。反过来，我们一般把 ostringstream 初始化为空 string，再用输出操作填充它。还有一种有时会派上用场的更直接的方式访问 stringstream 中的字符：ss.str() 返回 ss 的字符串的一份副本，而 ss.str(s) 将 ss 的字符串设置为 s 的一份副本。§11.7 给出了一个 ss.str(s) 不可或缺的例子。

11.5　行导向输入

» 运算符按给定类型的标准格式将数据读入该类型的对象。例如，读入 int 时，» 会一直读取直到遇到不是数字的字符；读入 string 时，» 会一直读取直到遇到空白。标准库 istream 库还提供了读取单个字符和整行的功能。考察下面的例子：

```
string name;
cin >> name;                 // 输入：Dennis Ritchie
cout << name << '\n';        // 输出：Dennis
```

如果我们想一次读入一整行内容，之后再决定如何处理它的格式，该怎么办？这可以用函数 getline() 实现。例如：

```
string name;
getline(cin,name);          // 输入：Dennis Ritchie
cout << name << '\n';       // 输出：Dennis Ritchie
```

现在我们得到了整行内容。为什么会想要这样做呢？一个好的回答是“因为我们想做一件 » 做不到的事情”。而常见的回答则不太高明：“因为用户键入了一整行。”如果你能想到的只有这个理由，那就坚持用 » 好了，因为一旦你拿到了输入的整行内容，通常就不得不以某种方式解析它。例如：

```
string first_name;
string second_name;
stringstream ss {name};
ss>>first_name;             // 输入 Dennis
ss>>second_name;            // 输入 Ritchie
```

直接读入 `first_name` 和 `second_name` 本来会更简单。

想读入一整行的一个常见原因在于，空白的定义并不总是合适的。有时，我们希望把换行与其他空白字符区别对待。例如，与游戏的文本通信可能把一行当作一个句子，而不依赖常规的标点符号：

```
go left until you see a picture on the wall to your right
remove the picture and open the door behind it. take the bag from there
```

在这种情况下，我们会先读入一整行，然后再从中提取出单个单词。

```
string command;
getline(cin,command);       // 读入该行


stringstream ss {command};
vector<string> words;
for (string s; ss>>s; )
    words.push_back(s);     // 提取各个单词
```

另一方面，如果可以有所选择，我们多半更愿意依赖恰当的标点符号，而不是换行。

11.6 字符分类

通常，我们按格式约定读取整数、浮点数、单词等。然而，我们可以——有时也必须——向下降一层抽象，去读取单个字符。这要做更多工作，但当我们读取单个字符时，就对自己的所作所为有了完全的控制。考察对一个表达式进行词法记号化（§7.8.2）的情形。例如，我们希望把 `1+4*x<=y/z*5` 分隔成十一个记号

```
1 + 4 * x <= y / z * 5
```

我们可以用 » 读取数字，但若试图把标识符作为 `string` 读取，就会导致 `x<=y` 被读成一个字符串（因为 `<` 和 `=` 不是空白字符），`z*` 也被读成一个字符串（因为 `*` 也不是空白字符）。作为替代，我们可以这样写

```
for (char ch; cin.get(ch); ) {
    if (isspace(ch)) {                              // 若 ch 是空白
                        // 什么也不做（即跳过空白）
    }
    if (isdigit(ch)) {
            // 读取一个数
    }
    else if (isalpha(ch)) {
            // 读取一个标识符
    }
    else {
            // 处理运算符
    }
}
```

`istream::get()` 函数读取单个字符存入其实参。它不跳过空白。与 » 一样，`get()` 返回其 istream 的引用，因此我们可以检测它的状态。

当我们读取单个字符时，通常希望对它们进行分类（character classification）：这个字符是数字吗？是大写字母吗？等等。标准库为此提供了一组函数：

字符分类	
`isspace(c)`	c 是空白（`' '`、`'\t'`、`'\n'` 等）吗？
`isalpha(c)`	c 是字母（`'a'..'z'`、`'A'..'Z'`）（注意：不是 `'_'`）吗？
`isdigit(c)`	c 是十进制数字（`'0'..'9'`）吗？
`isxdigit(c)`	c 是十六进制数字（十进制数字或 `'a'..'f'` 或 `'A'..'F'`）吗？
`isupper(c)`	c 是大写字母吗？
`islower(c)`	c 是小写字母吗？
`isalnum(c)`	c 是字母或十进制数字吗？
`iscntrl(c)`	c 是控制字符（ASCII 0..31 和 127）吗？
`ispunct(c)`	c 不是字母、数字、空白或不可见控制字符吗？
`isprint(c)`	c 是可打印字符（ASCII `' '..'~'`）吗？
`isgraph(c)`	`isalpha(c)` 或 `isdigit(c)` 或 `ispunct(c)`（注意：不含空格）？

注意，这些分类可以用“或”运算符（||）组合起来。例如，`isalnum(c)` 的意思就是 `isalpha(c)||isdigit(c)`，即“c 是字母或数字之一吗？”

此外，标准库还提供了两个消除大小写差异的有用函数：

字符大小写	
`toupper(c)`	c 或 c 的大写等价字符
`tolower(c)`	c 或 c 的小写等价字符

当你想忽略大小写差异时，这些函数很有用。例如，用户输入的 Right、right 和 rigHT 很可能意思相同（rigHT 很可能是误碰了大写锁定键 Caps Lock 的结果）。对这几个字符串中的每个字符应用 tolower() 后，我们都得到 right。我们可以对一个任意字符串这样做：

```
void tolower(string& s)      // 把 s 变为小写
{
    for (char& x : s) x = tolower(x);
}
```

我们使用按引用传递（§8.5.5）来真正修改这个 string。如果想保留原字符串，我们也可以写一个生成小写副本的函数。与 toupper() 相比，更推荐 tolower()，因为它对某些自然语言（如德语）的文本效果更好——在这些语言中，并非每个小写字符都有大写等价形式。

11.7 使用非标准分隔符

本节给出一个半真实性的例子，展示如何用 iostream 解决一个真实问题。当我们读取 string 时，单词默认由空白分隔。遗憾的是，istream 没有提供一种设施，让我们定义哪些字符构成空白，或以其他方式直接改变 » 读取 string 的方式。那么，如果我们需要另一种空白定义，该怎么办呢？考察 §4.6.3 中读入“单词”并比较它们的例子。那些单词是由空白分隔的，所以如果我们读入

```
As planned, the guests arrived; then,
```

我们就会得到这些“单词”

```
As
planned,
the
guests
arrived;
then,
```

这不是我们在词典里能找到的东西：planned, 和 arrived; 不是单词。它们是单词加上干扰性的、无关的标点字符。对大多数用途而言，我们必须像对待空白那样对待标点。怎样才能去掉这些标点呢？我们可以读取字符，去掉标点字符——或者把它们变成空白——然后再读一遍“清理过”的输入：

```
string line;
getline(cin,line);           // 读入 line
for (char& ch : line)        // 把每个标点字符替换为空格
    switch(ch) {
    case ';': case '.': case ',': case '?': case '!':
        ch = ' ';
    }
```

```
stringstream ss(line);       // 令 istream ss 从 line 读取
vector<string> vs;
for (string word; ss>>word; )    // 读取不含标点字符的单词
    vs.push_back(word);
```

用它来读取这一行，我们就得到了想要的结果

```
As
planned
the
guests
arrived
then
```

遗憾的是，上面的代码很凌乱，而且相当特殊化。如果我们有另一种标点定义，该怎么办呢？让我们提供一种更通用、更有用的方式，从一个输入流中去除不想要的字符。那会是什么呢？我们希望用户代码是什么样子？像下面这样如何：

```
ps.whitespace(";:,.");       // 把分号、冒号、逗号和点当作空白
for (string word; ps>>word; )
    vs.push_back(word);
```

我们会如何定义一个像 ps 那样工作的流呢？基本思想是：从普通输入流中读取单词，然后把用户指定的“空白”字符当作空白对待；也就是说，我们并不把“空白”字符交给用户，只是用它们来分隔单词。例如，

```
as.not
```

应该成为两个单词

```
as
not
```

我们可以定义一个类来做这件事。它必须从一个 istream 获取字符，并有一个 » 运算符，其工作方式与 istream 的 » 完全一样，只是我们可以告诉它哪些字符应被看作空白。为简单起见，我们不提供把已有空白字符（空格、换行等）当作非空白对待的途径；我们只允许用户指定额外的“空白”字符。我们也不提供把指定字符从流中彻底去除的途径；与前面一样，我们只是把它们变成空白。让我们把这个类称为 Punct_stream：

```
class Punct_stream {          // 类似 istream，但用户可以添加
                              // 空白字符集合
public:
    Punct_stream(istream& is)
        : source{is}, sensitive{true} { }

    void whitespace(const string& s)      // 令 s 成为空白集合
        { white = s; }
    void add_white(char c) { white += c; }    // 加入空白集合
    bool is_whitespace(char c);           // c 在空白集合中吗？

    void case_sensitive(bool b) { sensitive = b; }
    bool is_case_sensitive() { return sensitive; }

    Punct_stream& operator>>(string& s);
    operator bool();
private:
    istream& source;          // 字符来源
    istringstream buffer;     // 我们让 buffer 完成格式化
    string white;             // 被视为“空白”的字符
    bool sensitive;           // 流区分大小写吗？
};
```

其基本思想——正如上面的例子那样——是从 istream 一次读入一行，把“空白”字符转换为空格，然后用 istringstream 完成格式化。除了处理用户定义的空白之外，我们还给 `Punct_stream` 提供了一项相关功能：如果我们通过 `case_sensitive()` 提出要求，它就可以把区分大小写的输入转换为不区分大小写的输入。例如，只要我们提出要求，就能让一个 `Punct_stream` 把

```
Man bites dog!
```

读成

```
man
bites
dog
```

`Punct_stream` 的构造函数接受一个 istream 作为字符来源，并赋予它局部名字 `source`。构造函数还将流默认设置为通常的区分大小写行为。我们可以创建一个从 `cin` 读取、把分号、冒号和点看作空白、并把所有字符转换为小写的 `Punct_stream`：

```
Punct_stream ps {cin};        // ps 从 cin 读取
ps.whitespace(";:.");         // 分号、冒号和点也是空白
ps.case_sensitive(false);     // 不区分大小写
```

显然，最有趣的操作是输入运算符 »。它也是迄今为止最难定义的。我们的总体策略是：从 istream 读一整行到称为 `line` 的字符串中。然后我们把所有“我们的”空白字符都转换为空格字符（`' '`）。完

成之后，我们把这一行放入称为 buffer 的 istringstream 中。现在我们就可以用通常的按空白分隔的 » 从 buffer 读取了。代码看起来比这要复杂一点，因为我们只是先尝试从 buffer 读取，只有当发现它为空时才去填充它：

```
Punct_stream& Punct_stream::operator>>(string& s)
{
    while (!(buffer>>s)) {                    // 尝试从 buffer 读取
        if (buffer.bad() || !source.good()) return *this;
        buffer.clear();

        string line;
        getline(source,line);                 // 从 source 获取一行

        // 按需进行字符替换:
        for (char& ch : line)
            if (is_whitespace(ch))
                ch = ' ';                     // 变为空格
            else if (!sensitive)
                ch = tolower(ch);             // 变为小写

        buffer.str(line);                     // 把字符串放入流
    }
    return *this;
}
```

让我们逐一考察这段代码。首先看这个有点不寻常的

```
while (!(buffer>>s)) {
```

如果称为 buffer 的 istringstream 中还有字符，读取 buffer>>s 就会成功，s 将收到一个以“空白”分隔的单词；这样就没有别的事要做了。只要 buffer 中还有可供读取的字符，这种情况就会持续发生。然而，当 buffer>>s 失败时——即 !(buffer>>s)——我们必须从 source 补充 buffer。注意，buffer>>s 这个读取位于循环中；在我们尝试补充 buffer 之后，还需要再尝试一次读取，因此就有了

```
while (!(buffer>>s)) {            // 尝试从 buffer 读取
    if (buffer.bad() || !source.good()) return *this;
    buffer.clear();

    // 补充 buffer
}
```

如果 buffer 处于 bad() 状态，或者 source 出了问题，我们就放弃；否则，我们清除 buffer 并再次尝试。我们需要清除 buffer，因为只有在读取失败时才会进入这个“补充循环”，而失败的原因通常是我们碰到了 buffer 的 eof()，即 buffer 中已没有更多字符可供读取。处理流状态总是很麻烦，而且它往往是那些需要烦琐调试的微妙错误的根源。幸运的是，补充循环的其余部分相当直截了当：

```
string line;
getline(source,line);           // 从 source 获取一行

// 按需进行字符替换:
for (char& ch : line)
    if (is_whitespace(ch))
        ch = ' ';               // 变为空格
    else if (!sensitive)
        ch = tolower(ch);       // 变为小写

buffer.str(line);               // 把字符串放入流
```

我们读入一行到 line 中。然后我们查看这一行的每个字符，判断是否需要修改它。is_whitespace() 函数是 Punct_stream 的成员，我们稍后定义它。tolower() 函数是一个标准库函数，它做的事情显而易见，比如把 A 变成 a（参见 §11.6）。

一旦有了一行经过正确处理的内容，我们就需要把它放入我们的 istringstream。这正是 buffer.str(line) 所做的事；可以把它读作“将 istringstream buffer 的字符串设置为 line。”

注意，我们“忘了”在用 getline() 从 source 读取后检测它的状态。我们不需要检测，因为最终总会到达循环顶部的 !source.good() 检测。

一如既往，我们返回对流本身的引用 *this 作为 » 的结果；参见 §17.10。

检测空白很容易；我们只需把一个字符与保存我们空白集合的字符串中的每个字符逐一比较：

```
bool Punct_stream::is_whitespace(char c)
{
    for (char w : white)
        if (c==w) return true;
    return false;
}
```

记住，我们把通常的空白字符（如换行和空格）留给 istringstream 按通常方式处理，因此对它们不必做任何特殊的事情。

还剩下一个神秘的函数：

```
Punct_stream::operator bool()
{
    return !(source.fail() || source.bad()) && source.good();
}
```

istream 的惯常用法是检测 » 的结果。例如：

```
while (ps>>s) { /* ... */ }
```

这意味着我们需要一种把 ps>>s 的结果当作布尔值来看待的方法。ps>>s 的结果是一个 Punct_stream，因此我们需要一种将 Punct_stream 隐式转换为 bool 的方法。这正是 Punct_stream 的 operator bool() 所做的。名为 operator bool() 的成员函数定义了一个到 bool 的转换。特别是，当对 Punct_stream 的操作成功时，它返回 true。

现在我们可以编写程序了：

```
int main()
    // 对给定的文本输入，产生其中所有单词的排序列表
    // 忽略标点和大小写差异
    // 从输出中消除重复单词
{
    Punct_stream ps {cin};
    ps.whitespace(";:,.?!()\"{}<>/&$@#%^*|~");  // 注意 \" 在字符串中表示 "
    ps.case_sensitive(false);

    cout << "please enter words\n";
    vector<string> vs;
    for (string word; ps>>word; )
        vs.push_back(word);              // 读取单词

    sort(vs.begin(),vs.end());           // 按字典序排序
    for (int i=0; i<vs.size(); ++i)      // 输出词典
        if (i==0 || vs[i]!=vs[i-1]) cout << vs[i] << '\n';
}
```

这将从输入产生一个正确排序的单词列表。检测

```
if (i==0 || vs[i]!=vs[i-1])
```

将抑制重复单词。给这个程序输入

```
There are only two kinds of languages: languages that people complain
about, and languages that people don't use.
```

它将输出

```
about
and
are
complain
don't
kind
languages
of
only
people
that
there
two
use
```

为什么我们得到的是 `don't` 而不是 `dont`？因为我们在 `whitespace()` 调用中把单引号排除在外了。

注意： `Punct_stream` 在许多重要而有用的方面表现得像 istream，但它并不是真正的 istream。例如，我们不能用 `rdstate()` 查询它的状态，`eof()` 没有定义，我们也没有费心提供读取整数的 »。重

要的是，我们不能把 `Punct_stream` 传递给期望 istream 的函数。我们能定义一个真正是 istream 的 `Punct_istream` 吗？可以，但我们目前还不具备实现这一绝技所需的编程经验、设计概念和语言设施（如果你——在很久之后——想重新研究这个问题，就必须在一本专家级指南或手册中查找流缓冲区（stream buffer））。

你发觉 `Punct_stream` 容易读懂吗？你发觉这些解释容易理解吗？你认为你自己能写出它吗？如果你几天前还是一名真正的新手，诚实的回答很可能是“不，不，不!”甚至是“不，不！不——你疯了吗?”我们能理解——而对最后一个问题（或咆哮）的回答是“不，至少我们认为不疯狂。”这个例子的目的在于

- 展示一个较为真实的问题及其解决方案
- 展示用相对朴素的手段能取得什么
- 为一个表面容易的问题提供一个易用的解决方案
- 说明接口与实现之间的区别

要成为程序员，你需要阅读代码，而不仅仅是那些精心打磨过的教学问题解答。这个例子就是如此。再过几天或几周，读懂它对你来说就会变得轻松，而且你会开始琢磨改进这个解决方案的途径。

看待这个例子的一种方式是，把它想象成一位教师在英语入门课程里加入了一些真实的英语俚语，以增添一点色彩、活跃气氛。

11.8 还有太多太多的内容

I/O 的细节似乎无穷无尽。很可能的确如此，因为它们只受人类创造力和善变的限制。例如，我们还没有考虑自然语言带来的复杂性。英语中写作 `12.35` 的数，在大多数其他欧洲语言中按惯例写作 `12,35`。自然而然地，C++ 标准库提供了处理这一以及许多其他与自然语言相关的 I/O 方面的设施。如何书写汉字？如何比较用马拉雅拉姆文字符书写的字符串？这些问题都有答案，但远远超出了本书的范围。如果你需要了解，请查阅更专门或更高级的书籍（如 Langer 的 *Standard C++ IOStreams and Locales* 以及 Stroustrup 的 *The C++ Programming Language*）以及库和系统文档。查找“locale”（区域设置）一词；这是通常用来指称处理自然语言差异的设施的术语。

复杂性的另一个来源是缓冲：标准库 iostream 依赖于一个称为 streambuf 的概念。对于 iostream 的高级应用——无论出于性能还是功能考虑——这些 streambuf 都是不可避免的。如果你觉得需要定义自己的 iostream，或者需要把 iostream 调优到新的数据源/数据汇，请参阅 Stroustrup 的 *The C++ Programming Language* 第 38 章或你的系统文档。

使用 C++ 时，你还可能遇到 C 标准 `printf()/scanf()` 这一族的 I/O 函数。如果遇到了，请在 §27.6、§B.10.2、Kernighan 和 Ritchie 的优秀 C 教材（*The C Programming Language*）或网络上数不清的资料中查阅它们。每种语言都有自己的 I/O 设施；它们各不相同，大多数都有些古怪，但大多数都以各种奇特的方式反映了我们在第 10 章和第 11 章中介绍的相同基本概念。

标准库 I/O 设施总结于附录 B。图形用户界面（GUI）这一相关主题在第 12–16 章中介绍。

小试身手

1. 建立一个名为 **`Test_output.cpp`** 的程序。声明一个整数 **`birth_year`**，并将它赋值为你出生的年份。
2. 以十进制、十六进制和八进制形式输出你的 **`birth_year`**。
3. 用所使用的数制的名字为每个值加标签。

4. 你是否使用制表符将输出按列对齐了？如果没有，请照做。
5. 现在输出你的年龄。
6. 出现问题了吗？发生了什么？将你的输出修正为十进制。
7. 回到第 2 步，令你的输出为每个输出值显示其数制。
8. 尝试按八进制、十六进制等方式读取：

```
cin >> a >>oct >> b >> hex >> c >> d;
cout << a << '\t'<< b << '\t'<< c << '\t'<< d << '\n' ;
```

用输入

```
1234 1234 1234 1234
```

运行这段代码。解释其结果。

9. 编写一些代码，把数 1234567.89 打印三次，分别使用 defaultfloat、fixed 和 scientific 形式。哪种输出形式为用户呈现了最精确的表示？解释原因。
10. 制作一个简单的表格，包含你自己和至少五位朋友的名（last name）、姓（first name）、电话号码和电子邮件地址。试验不同的域宽，直到你对表格的呈现效果感到满意为止。

要点回顾

1. 对程序员来说，为什么 I/O 很棘手？
2. 记号 « hex 做什么？
3. 计算机科学中十六进制数有什么用途？为什么？
4. 说出你可能想为整数输出实现的一些选项。
5. 什么是操纵器（manipulator）？
6. 十进制的前缀是什么？八进制呢？十六进制呢？
7. 浮点值默认的输出格式是什么？
8. 什么是域（field）？
9. 解释 setprecision() 和 setw() 做什么。
10. 文件打开模式的用途是什么？
11. 下列操纵器中哪个不“保持有效”（stick）：hex、scientific、setprecision()、showbase、setw？
12. 字符 I/O 与二进制 I/O 的区别是什么？
13. 举一个例子，说明在什么情况下使用二进制文件而不是文本文件可能更有益。
14. 举两个例子，说明 stringstream 在什么情况下有用。
15. 什么是文件位置？
16. 如果你把文件位置定位到文件末尾之后，会发生什么？
17. 在什么情况下你会更倾向行导向输入而非按类型读取？
18. isalnum(c) 做什么？

术语

二进制　十六进制　八进制
字符分类　不规则性　输出格式化
十进制　行导向输入　规则性
defaultfloat　操纵器　scientific

文件定位 非标准分隔符 setprecision()
fixed noshowbase showbase

习题

1. 编写一个程序，读取一个文本文件并将输入全部转换为小写，产生一个新文件。
2. 编写一个程序，给定文件名和一个单词，输出包含该单词的每一行及其行号。提示：`getline()`。
3. 编写一个程序，从文件中去除所有元音字母（“去元音”）。例如，`Once upon a time!` 变为 `nc pn tm!`。出人意料的是，结果往往仍然可读；在你的朋友身上试试。
4. 编写一个名为 `multi_input.cpp` 的程序，提示用户输入若干整数，它们可以是以 0 和 0x 为进制后缀的八进制、十进制或十六进制的任意组合；正确解释这些数；并将它们转换为十进制形式。然后程序应像下面这样按间距规整的列输出这些值：

```
0x43   hexadecimal   converts to    67  decimal
0123  octal         converts to    83  decimal

65   decimal       converts to    65  decimal
```

5. 编写一个程序，读取若干字符串，并对每个字符串输出其每个字符的字符分类，分类按 §11.6 介绍的字符分类函数定义。注意，一个字符可以同时具有多个分类（例如，x 既是字母又是字母数字字符）。
6. 编写一个程序，用空白替换标点符号。考虑.（点）、;（分号）、,（逗号）、?（问号）、-（连字符）、’（单引号）这些标点字符。不要修改一对双引号（”）内的字符。例如，“- don’t use the as-if rule.”变为“don t use the as if rule ”。
7. 修改上一题的程序，使它将 don’t 替换为 do not，将 can’t 替换为 cannot，等等；保持单词内部的连字符不变（这样我们得到“do not use the as-if rule ”）；并将所有字符转换为小写。
8. 使用上一题的程序制作一个词典（作为 §11.7 中方法的替代方案）。用一个多页的文本文件运行它，查看结果，看看你能否改进程序，以做出更好的词典。
9. 将 §11.3.2 的二进制 I/O 程序拆分为两个：一个程序把普通文本文件转换为二进制，另一个程序读取二进制并将其转换回文本。通过将一个文本文件与先转换为二进制再转换回来的结果进行比较，测试这两个程序。
10. 编写一个函数 `vector<string> split(const string& s)`，返回从实参 s 中按空白分隔的子串组成的 `vector`。
11. 编写一个函数 `vector<string> split(const string& s, const string& w)`，返回从实参 s 中按空白分隔的子串组成的 `vector`，其中空白定义为“普通空白”再加上 w 中的字符。
12. 反转一个文本文件中字符的顺序。例如，`asdfghjkl` 变为 `lkjhgfdsa`。警告：没有真正好的、可移植且高效的反向读取文件的方法。
13. 反转一个文件中单词（定义为以空白分隔的字符串）的顺序。例如，Norwegian Blue parrot 变为 parrot Blue Norwegian。你可以假定文件中的所有字符串能一次全部装入内存。
14. 编写一个程序，读取一个文本文件，并写出文件中属于每个字符分类（§11.6）的字符各有多少个。
15. 编写一个程序，读取一个由空白分隔的数组成的文件，以 scientific 格式和精度 8 输出一个文件，每行四个域，每个域 20 个字符。

16. 编写一个程序，读取一个由空白分隔的数组成的文件，按顺序（最小值在前）输出它们，每行一个值。每个值只写一次，如果一个值出现多次，则在其行上写出它出现的次数。例如，7 5 5 7 3 117 5 应该产生

```
3
5     3
7     2
117
```

后记

输入和输出是杂乱的，因为我们人类的品味和惯例并不遵循易于陈述的规则和直截了当的数学定律。作为程序员，我们很少有资格要求用户背离他们的偏好；即使有，我们也通常应该少一点傲慢，不要以为我们能为经年累月形成的惯例提供一个简单的替代品。因此，我们必须预料到、接受并适应输入和输出的一定程度的杂乱，同时仍努力让我们的程序尽可能简单——但不能过于简单。

第三部分

数据与算法

17 vector 与自由存储

"Use vector as the default!"
("把 *vector* 作为默认选择!")
——Alex Stepanov

本章及随后四章介绍 C++ 标准库中容器与算法那部分内容，传统上称为 STL。我们介绍 STL 中的关键设施以及它们的一些用法。此外，我们还介绍实现 STL 所用的关键设计与编程技术，以及为此用到的一些底层语言特性——其中包括指针、数组和自由存储。本章与随后两章的重点，是使用最广、也最有用的 STL 容器：`vector`。

17.1 引言
17.2 vector 基础
17.3 内存、地址与指针
  17.3.1 `sizeof` 运算符
17.4 自由存储与指针
  17.4.1 自由存储的分配
  17.4.2 通过指针访问
  17.4.3 范围
  17.4.4 初始化
  17.4.5 空指针
  17.4.6 自由存储的释放
17.5 析构函数
  17.5.1 生成的析构函数
  17.5.2 析构函数与自由存储
17.6 元素访问
17.7 指向类对象的指针
17.8 摆弄类型：`void*` 与类型转换
17.9 指针与引用
  17.9.1 指针参数与引用参数
  17.9.2 指针、引用与继承
  17.9.3 一个例子：链表
  17.9.4 链表操作
  17.9.5 链表的使用
17.10 `this` 指针
  17.10.1 更多链表用法

17.1 引言

C++ 标准库中最有用的容器就是 `vector`。一个 `vector` 提供某一给定类型的元素序列。你可以通过下标（索引）引用其中的元素；可以用 `push_back()` 向 `vector` 追加元素；可以用 `size()` 询问它含有多少个元素；访问元素时还会检查是否越界。标准库的 `vector` 是一个方便、灵活、高效（在时间与空间上都高效）、静态类型安全的元素容器。标准 `string` 具有类似的性质；其他有用的标准容器

类型（如 list 和 map，我们将在第 20 章介绍）也是如此。然而，计算机的内存并不直接支持这样好用的类型。硬件直接支持的只是字节序列。例如，对于 vector<double>，操作 v.push_back(2.3) 会把 2.3 追加到一段 double 序列的末尾，并使 v 的元素个数（v.size()）增加 1。在最底层，计算机对 push_back() 这样精巧的操作一无所知；它只知道如何一次读写几个字节。

在本章及随后两章中，我们演示如何用每个程序员都能获得的基本语言设施，构造出 vector。这样做既能说明一些有用的概念与编程技术，也能看到它们如何用 C++ 语言特性表达出来。我们在 vector 实现中遇到的语言设施与编程技术，普遍有用，而且应用极广。

一旦看清 vector 是如何设计、实现和使用的，我们就可以接着考察其他标准库容器，例如 map，并研究 C++ 标准库为使用它们所提供的优雅而高效的设施（第 20、21 章）。这些设施称为算法，它们使我们不必亲自去编写处理数据的常见任务，而可以直接使用每个 C++ 实现都提供的现成东西，让库的编写与测试变得轻松。我们已经见过并使用过标准库中最有用的算法之一：sort()。

我们通过一系列逐步完善的 vector 实现来接近标准库的 vector。首先，构造一个非常简单的 vector；然后看看它有什么不足，并加以修正。这样反复几次之后，我们就得到一个与标准库 vector 大致相当的实现——也就是随你的 C++ 编译器一起提供、你在前面几章一直在用的那个。这种逐步细化的过程，与我们通常面对一个新编程任务时的做法非常相似。在此过程中，我们会遇到并探讨许多与内存使用和数据结构有关的经典问题。基本计划如下：

- 第 17 章（本章）：如何处理数量可变的内存？具体来说，不同的 vector 怎样才能拥有不同数目的元素，同一个 vector 又怎样才能在不同时刻拥有不同数目的元素？这促使我们考察自由存储（堆存储）、指针、类型转换（显式类型转换）和引用。
- 第 18 章：如何复制 vector？如何为它提供下标操作？我们还会介绍数组，并探讨数组与指针的关系。
- 第 19 章：如何让 vector 的元素类型可以不同？如何处理越界错误？为回答这些问题，我们考察 C++ 的模板与异常设施。

除了为支持灵活、高效且类型安全的 vector 实现而引入的新语言设施与技术之外，我们还会（重复）使用许多已经见过的语言设施与编程技术。偶尔，我们也会借机给它们一个稍微正式、技术化的定义。

所以，到这里我们终于要直接面对内存了。为什么非得这样？我们的 vector 和 string 已经极其有用、极其方便，直接用它们就好了。毕竟，vector、string 这类容器本来就是为了让我们避开真实内存中一些令人不快的方面而设计的。然而，除非我们甘愿相信魔法，否则就必须考察内存管理的最底层。为什么不能“就相信魔法”呢？或者换个积极的说法：为什么不能“就相信 vector 的实现者知道自己在做什么”呢？毕竟，我们并没有建议你去研究让计算机内存得以工作的器件物理。

嗯，我们是程序员（计算机科学家、软件开发人员，或随便什么称呼），而不是物理学家。假如我们研究的是器件物理，那就必须深究计算机内存设计的细节。然而，既然我们研究的是编程，就必须深究程序的设计细节。理论上，我们大可以把底层的内存访问与管理设施当作“实现细节”，就像对待器件物理那样。可是真若如此，你就不只是“相信魔法”了：你将无法实现新的容器（万一你需要一个——这并不罕见）。而且，你也读不懂大量直接使用内存的 C 和 C++ 代码。接下来的几章里我们将会看到，指针（一种低层、直接引用对象的方式）还因种种与内存管理无关的理由而有用。若从不使用指针，很难把 C++ 用好。

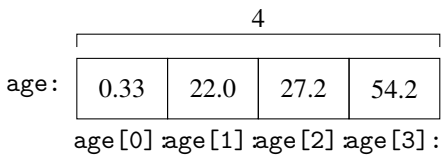
从更哲学的角度说，我属于计算机专业人士中的大多数：我认为，如果你对程序如何映射到计算机的内存与操作缺乏基本而实际的理解，那么在牢固掌握数据结构、算法和操作系统这类更高层主题时就会遇到困难。

17.2 vector 基础

我们先从一个非常简单的用法入手，开始 vector 的渐进设计：

```
vector<double> age(4);  // 一个含 4 个 double 型元素的 vector
age[0]=0.33;
age[1]=22.0;
age[2]=27.2;
age[3]=54.2;
```

显然，这就创建了一个含四个 double 型元素的 vector，并把这四个元素的值分别设为 0.33、22.0、27.2 和 54.2。四个元素的编号是 0、1、2、3。C++ 标准库容器中元素的编号总是从 0 开始。从 0 开始编号非常普遍，也是 C++ 程序员普遍遵循的约定。一个 vector 的元素个数称为它的大小（size）。所以，age 的大小是 4。vector 的元素从 0 编号（索引）到 size-1。例如，age 的元素编号从 0 到 age.size()-1。我们可以把 age 用图形表示如下：

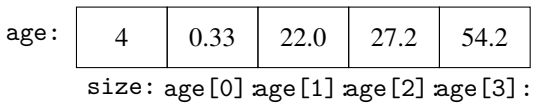


怎样才能把这个“图形设计”真正落实到计算机内存中？怎样才能让这些值按那种方式存放和访问？显然，我们必须定义一个类，并且希望把这个类称做 vector。此外，它需要一个数据成员保存元素个数，还需要一个数据成员保存元素本身。可是，当元素个数可以变化时，我们该如何表示这样一组元素呢？我们本可以使用标准库的 vector，但在目前这个语境下那无异于作弊：我们正在自己动手造一个 vector。

那么，上面图中那个箭头该怎么表示？考虑不要它。我们可以定义一个固定大小的数据结构：

```
class vector {
    int size, age0, age1, age2, age3;
    // . . .
};
```

先忽略一些记法上的细节，我们会得到大致如下的东西：



这简单又漂亮，可一旦我们想用 push_back() 添加元素，就傻眼了：我们没有办法添加元素；元素个数在程序文本中就固定为四。我们需要的不只是一个保存固定数量元素的数据结构。如果我们把 vector 定义成元素个数固定，那么像 push_back() 这样改变元素个数的操作根本无法实现。基本上，

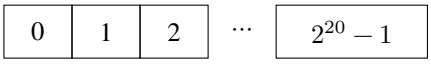
我们需要一个指向这组元素的数据成员，以便在需要更多空间时让它指向另一组元素。我们需要类似“第一个元素的内存地址”这样的东西。在 C++ 中，能保存地址的数据类型称为指针，语法上以 * 后缀区分，因此 double* 表示“指向 double 的指针”。有了这些，就可以定义 vector 类的第一个版本：

```
// 一个非常简化的 double 向量（类似 vector<double>）
class vector {
    int sz;                         // 元素个数
    double* elem;                   // 指向第一个元素（类型 double）
public:
    vector(int s);                  // 构造函数：分配 s 个 double，
                                    // 让 elem 指向它们，
                                    // 并把 s 存入 sz
    int size() const { return sz; } // 当前大小
};
```

在继续 vector 的设计之前，让我们稍微详细地研究一下“指针”这个概念。“指针”这个概念——以及与之密切相关的“数组”概念——是理解 C++ 中“内存”概念的钥匙。

17.3 内存、地址与指针

计算机的内存是一个字节序列。我们可以把这些字节从 0 一直编号到最后一个。我们把这种“表示内存中某个位置的编号”称为地址。你可以把地址看成一种整数值。内存的第一个字节地址为 0，下一个字节地址为 1，依此类推。我们可以把一兆字节的内存画成这个样子：



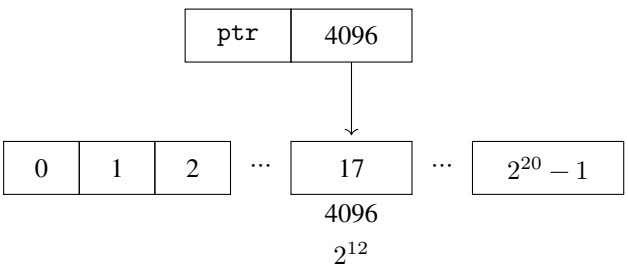
我们放进内存的每一样东西都有一个地址。例如：

```
int var = 17;
```

这会在某处为 var 留出一块“int 大小”的内存，并把值 17 存入其中。我们也可以保存和操作地址。保存地址值的对象称为指针。例如，保存一个 int 的地址所需的类型称为“指向 int 的指针”或“int 指针”，记法是 int*：

```
int* ptr = &var;            // ptr 保存 var 的地址
```

“取地址”运算符，即一元 &，用于取得对象的地址。因此，如果 var 恰好从地址 4096（也就是 2^{12}）开始，那么 ptr 中将保存值 4096：



基本上，我们把计算机的内存看成从 0 到“内存大小减 1”依次编号的字节序列。在某些机器上这只是一种简化，但作为内存的初始编程模型，它已经够用了。

每种类型都有一个对应的指针类型。例如：

```
int x = 17;
int* pi = &x;           // 指向 int 的指针

double e = 2.71828;
double* pd = &e;        // 指向 double 的指针
```

如果想查看所指对象的值，可以使用“取内容”运算符，即一元 *。例如：

```
cout << "pi==" << pi << "; contents of pi==" << *pi << "\n";
cout << "pd==" << pd << "; contents of pd==" << *pd << "\n";
```

*pi 的输出将是整数 17，*pd 的输出将是 double 值 2.71828。而 pi 和 pd 的输出，则取决于编译器把变量 x 和 e 分配在内存中的什么位置。指针值（地址）所用的记法也可能因系统惯例而异；十六进制记法（§A.2.1.1）在表示指针值时很常用。

“取内容”运算符（常称为解引用运算符）也可以出现在赋值的左侧：

```
*pi = 27;          // 正确：可以把 27 赋给 pi 所指的 int
*pd = 3.14159;     // 正确：可以把 3.14159 赋给 pd 所指的 double
*pd = *pi;         // 正确：可以把一个 int (*pi) 赋给一个 double (*pd)
```

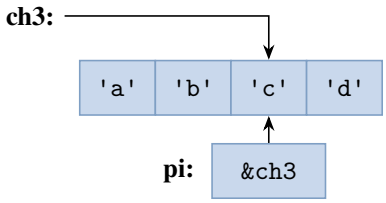
注意，尽管指针值可以按整数形式打印出来，指针却并不是整数。“int 指向什么？”不是一个合乎语法的问题；int 不指向任何东西，指针才指向。指针类型提供适合地址的运算，而 int 提供适合整数的（算术与逻辑）运算。因此，指针与整数不会隐式混用：

```
int i = pi;      // 错误：不能把 int* 赋给 int
pi = 7;          // 错误：不能把 int 赋给 int*
```

同样，指向 char 的指针（char*）也不是指向 int 的指针（int*）。例如：

```
char* pc = pi;     // 错误：不能把 int* 赋给 char*
pi = pc;           // 错误：不能把 char* 赋给 int*
```

为什么把 pc 赋给 pi 是错误？考虑一种回答：char 通常比 int 小得多，所以请看下面这段代码：




```
char ch1 = 'a';
char ch2 = 'b';
char ch3 = 'c';
char ch4 = 'd';
int* pi = &ch3;     // 指向 ch3，一块 char 大小的内存
                    // 错误：我们不能把 char* 赋给 int*
                    // 但姑且假设我们可以
*pi = 12345;        // 向一块 int 大小的内存写入
*pi = 67890;
```

编译器究竟如何在内存中分配变量，是由实现决定的，但我们很可能得到类似下面的结果：

现在，假如编译器允许那段代码，我们就会从 &ch3 开始的内存处写入 12345。这肯定会改变附近某些内存的值，比如 ch2 或 ch4。要是我们特别倒霉（这很可能），甚至会把 pi 本身的一部分覆盖掉！那样的话，接下来的赋值 *pi=67890 就会把 67890 放到内存中某个完全不同的地方。庆幸这种赋值被禁止了吧——不过这也是编译器在如此低层次的编程中提供的、为数不多的保护之一。

如果你确实需要把一个 int 转换成指针，或者把一种指针类型转换成另一种（这种情形很少见），可以使用 reinterpret_cast；见 §17.8。

我们现在非常接近硬件了。对程序员来说，这并不是一个特别舒服的位置。可用的原始操作寥寥无几，语言和标准库也几乎给不了什么支持。然而，我们必须走到这一层，才知道 vector 之类的高层设施是如何实现的。我们需要懂得如何在这一层写代码，因为并非所有代码都能“高层化”（见第 25 章）。另外，在体验过高层设施的缺席之后，我们也许更能体会到软件高层带来的便利与相对的安全。我们的目标始终是：在给定问题及其求解约束的前提下，工作在尽可能高的抽象层次上。本章以及第 18–19 章会演示如何通过实现一个 vector，回到更舒适的抽象层次。

17.3.1 sizeof 运算符

那么，一个 int 到底占用多少内存？一个指针呢？sizeof 运算符能回答这类问题：

```
void sizes(char ch, int i, int* p)
{
    cout << "the size of char is " << sizeof(char) << ' ' << sizeof (ch) << '\n';
    cout << "the size of int is " << sizeof(int) << ' ' << sizeof (i) << '\n';
    cout << "the size of int* is " << sizeof(int*) << ' ' << sizeof (p) << '\n';
}
```

如你所见，sizeof 既可以用于类型名，也可以用于表达式：用于类型时，sizeof 给出该类型对象的大小；用于表达式时，它给出表达式结果类型的大小。sizeof 的结果是一个正整数，单位是 sizeof(char)，其值被定义为 1。通常一个 char 存放在一个字节里，因此 sizeof 报告的是字节数。

试一试

运行上面的例子，看看结果是什么。然后扩充这个例子，确定 bool、double 以及其他类型的大小。

一个类型的大小并不保证在所有 C++ 实现上都相同。如今在笔记本电脑或台式机上，sizeof(int) 通常是 4。若一个字节为 8 位，那就意味着一个 int 是 32 位。不过，使用 16 位 int 的嵌入式系统处理器，以及采用 64 位 int 的高性能体系结构，也很常见。

那么一个 vector 要占用多少内存呢？我们可以试试：

```
vector<int> v(1000);          // 含 1000 个 int 元素的 vector
cout << "the size of vector<int>(1000) is " << sizeof (v) << '\n';
```

输出大致如下：

```
the size of vector<int>(1000) is 20
```

解释会在本章和下一章逐渐明朗（另见 §19.2.1），但显然，sizeof 并没有把元素计算在内。

17.4 自由存储与指针

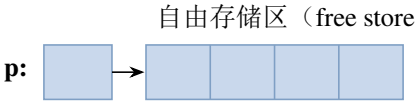
回顾 §17.2 末尾给出的 vector 实现。vector 从哪里得到存放元素的空间？我们怎样让指针 elem 指向这些元素？启动一个 C++ 程序时，编译器会为你的代码（有时称为代码存储区或文本存储区）以及你定义的全局变量（称为静态存储区）留出内存。它还会留出一些内存，供调用函数时使用——函数需要空间存放实参和局部变量（这称为栈存储区或自动存储区）。计算机其余的内存则有可能用于其他用途；它是“空闲的”。我们可以用图形来说明这一点：



C++ 语言通过一个名为 new 的运算符，让程序员可以使用这块“自由存储区”（也称为堆）。例如：

```
double* p = new double[4];     // 在自由存储区上分配 4 个 double
```

这段代码请求 C++ 运行时系统在自由存储区上分配 4 个 double，并把指向第一个 double 的指针返回给我们。我们用这个指针来初始化指针变量 p。这一点可以用图形表示如下：



new 运算符返回一个指向它所创建对象的指针。如果它创建了多个对象（一个数组），就返回指向其中第一个对象的指针。如果那个对象的类型是 X，那么 new 返回的指针类型就是 X*。例如：

```
char* q = new double[4];    // 错误：double* 不能赋给 char*
```

这里的 new 返回的是指向 double 的指针，而 double 并不是 char，所以我们不应该（也不能）把它赋给指向 char 的指针变量 q。

17.4.1 自由存储区的分配

我们通过 new 运算符请求在自由存储区上分配内存：

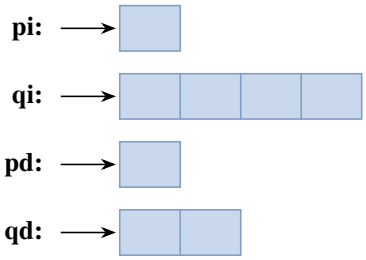
- new 运算符返回指向所分配内存的指针。
- 指针值就是这块内存第一个字节的地址。
- 指针指向一个指定类型的对象。
- 指针并不知道自己指向多少个元素。

new 运算符既可以分配单个元素，也可以分配元素序列（数组）。例如：

```
int* pi = new int;              // 分配一个 int
int* qi = new int[4];           // 分配 4 个 int（一个含 4 个 int 的数组）

double* pd = new double;        // 分配一个 double
double* qd = new double[n];     // 分配 n 个 double（一个含 n 个 double 的数组）
```

注意，所分配对象的数目可以是一个变量。这一点很重要，因为它使我们能够在运行时决定分配多少个对象。如果 n 是 2，我们就得到：



指向不同类型对象的指针，本身就是不同的类型。例如：

```
pi = pd;    // 错误：不能把 double* 赋给 int*
pd = pi;    // 错误：不能把 int* 赋给 double*
```

为什么不行呢？毕竟，我们可以把一个 int 赋给一个 double，也可以把一个 double 赋给一个 int。原因在于 [ ] 运算符：它要靠元素类型的大小来确定到哪里去找某个元素。例如，qi[2] 在内存中比 qi[0] 靠后两个 int 的大小，而 qd[2] 比 qd[0] 靠后两个 double 的大小。如果 int 与 double 的大小不同（在很多计算机上正是如此），那么我们一旦允许 qi 指向为 qd 分配的内存，就可能得到相当奇怪的结果。

这是“实用层面”的解释。理论上的解释则很简单：“允许把一种指针赋给另一种类型的指针，就等于允许类型错误。”

17.4.2 通过指针访问

除了对指针使用解引用运算符 * 之外，我们还可以使用下标运算符 [ ]。例如：

```
double* p = new double[4];    // 在自由存储区上分配 4 个 double
double x = *p;                // 读取 p 所指的（第一个）对象
double y = p[2];              // 读取 p 所指的第 3 个对象
```

不出所料，下标运算符和 vector 的下标运算符一样从 0 开始计数，所以 p[2] 指的是第三个元素；p[0] 是第一个元素，因此 p[0] 与 *p 完全同义。[ ] 和 * 运算符也可用于写入：

```
*p = 7.7;                     // 写入 p 所指的（第一个）对象
p[2] = 9.9;                   // 写入 p 所指的第 3 个对象
```

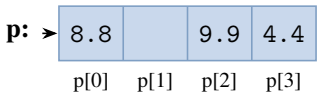
指针指向内存中的某个对象。“取内容”运算符（也叫解引用运算符）让我们可以读写指针 p 所指的那个对象：

```
double x = *p;                // 读取 p 所指的对象
*p = 8.8;                     // 写入 p 所指的对象
```

当作用于指针时，[ ] 运算符把内存看作一系列对象（类型由指针的声明指定），其中第一个对象由指针 p 指向：

```
double x = p[3];              // 读取 p 所指的第 4 个对象
p[3] = 4.4;                   // 写入 p 所指的第 4 个对象
double y = p[0];              // p[0] 与 *p 相同
```

就是这样。没有检查，也没有实现上的巧妙花招，只是对计算机内存的简单访问：



这正是我们实现 vector 时所需要的、简单而又效率最优的内存访问机制。

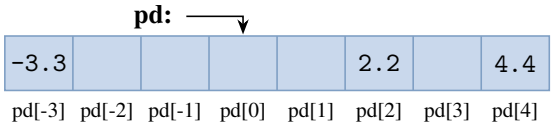
17.4.3 范围

指针最主要的问题在于：指针“不知道”自己指向多少个元素。考虑：

```
double* pd = new double[3];
pd[2] = 2.2;
pd[4] = 4.4;
pd[-3] = -3.3;
```

pd 有第三个元素 pd[2] 吗？有第五个元素 pd[4] 吗？看看 pd 的定义就会发现，答案分别是“有”和“没有”。然而编译器并不知道这一点；它不跟踪指针的值。我们的代码会照常访问内存，就好像我们分配了足够的内存一样。它甚至会照常访问 pd[-3]，就好像 pd 所指位置往前三个 double 的地方也属于我们分配的范围：

我们完全不知道标记为 pd[-3] 和 pd[4] 的内存位置是做什么用的。不过我们确知：它们本来并不是作为 pd 所指的那三个 double 的数组的一部分来使用的。它们很可能是其他对象的一部分，而我们刚刚在上面乱涂乱画了一通。这可不是个好主意。事实上，这通常是个灾难性的坏主意：“灾难性”的意思是“我的程序莫名其妙地崩溃了”，或者“我的程序输出了错误的结果”。试着把这话大声说出来，



听起来一点也不美妙。我们会花很大力气去避免它。越界访问之所以特别讨厌，是因为程序中看起来毫不相关的部分也会受到影响。越界读会给我们一个“随机”的值，它可能取决于某个完全无关的计算。越界写则可能让某个对象进入“不可能”的状态，或者干脆给它一个完全出乎意料的错误值。这类写入通常要过很久才被发现，因此特别难查。更糟的是：把带有越界错误的程序用略有不同的输入跑两遍，结果可能就不一样。这类 bug（“瞬态 bug”）是最难查找的 bug 之一。

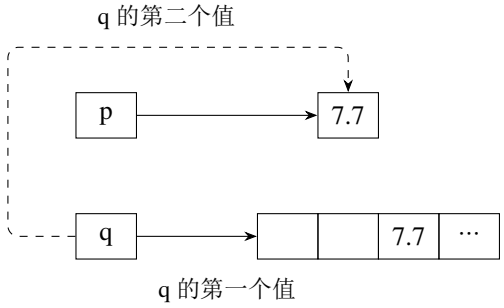
我们必须确保这类越界访问不会发生。我们使用 vector 而不直接使用 `new` 分配的内存，原因之一就是：vector 知道自己的大小，因此它（或我们）可以很容易地防止越界访问。

有一件事会让越界访问难以防止：我们可以把一个 `double*` 赋给另一个 `double*`，而不管它们各自指向多少个对象。

指针其实并不知道自己指向多少个对象。例如：

```
double* p = new double;                 // 分配一个 double
double* q = new double[1000];           // 分配 1000 个 double
q[700] = 7.7;                           // 没问题
q = p;                                  // 让 q 指向与 p 相同的位置
double d = q[700];                      // 越界访问!
```

在这里，仅仅三行代码，`q[700]` 就先后指代了两个不同的内存位置，而最后一次使用是越界访问，很可能酿成大祸。



读到这里，我们希望你会问：“可指针为什么不能记住大小呢？”显然，我们完全可以设计一种“指针”让它确实记住大小——vector 就几乎是这样的东西；如果你翻一翻 C++ 的文献和库，还会发现许多“智能指针”（smart pointer），它们弥补了内建低层指针的这些缺陷。然而，我们总得在某个地方深入到硬件层面，理解对象是怎样被寻址的——而机器地址并不“知道”它指向的是什么。此外，理解指针是理解大量真实代码的必要前提。

17.4.4 初始化

和以往一样，我们希望确保对象在被使用之前已经得到一个值；也就是说，我们要确保指针已经初始化，并且它们所指向的对象也已经初始化。考虑：

```
double* p0;                         // 未初始化：很可能出问题
double* p1 = new double;            // 得到一个（分配到的）未初始化的 double
double* p2 = new double{5.5};       // 得到一个初始化为 5.5 的 double
double* p3 = new double[5];         // 得到（分配）5 个未初始化的 double
```

显然，声明 p0 却不初始化它，就是在自找麻烦。考虑：

```
*p0 = 7.0;
```

这会把 7.0 赋给内存中的某个位置。至于那是内存的哪一部分，我们毫无概念。它可能无害，但千万、千万不要指望这一点。迟早我们会得到和越界访问一样的结果：“我的程序莫名其妙地崩溃了”，或者“我的程序输出不对”。在旧式 C++ 程序（“C 风格程序”）的严重问题里，相当可怕的一部分是由通过未初始化指针的访问和越界访问造成的。我们必须尽一切努力避免这类访问：一方面我们追求专业水准，另一方面我们也不想把时间浪费在查找这类错误上。很少有哪种活动比追查这类 bug 更令人沮丧、更枯燥。预防 bug 要比追捕 bug 愉快得多，也高效得多。

由 new 分配的内存，对内置类型来说并不会初始化。如果单个对象这样不合你的意，你可以指定一个值，就像我们对 p2 所做的那样：*p2 是 5.5。注意这里用 ( ) 表示初始化，与用 [ ] 表示“数组”形成对比。

我们还可以为由 new 分配的对象数组指定初始化列表。例如：

```
double* p4 = new double[5] {0,1,2,3,4};
double* p5 = new double[] {0,1,2,3,4};
```

现在 p4 指向 5 个 double 类型的对象，其中的值分别是 0.0、1.0、2.0、3.0 和 4.0。p5 也一样；当提供了一组元素时，元素个数可以省略。

和往常一样，我们应当留心未初始化的对象，确保在读它们之前先给它们一个值。要注意，编译器常常有一个“调试模式”，在这种模式下它们默认把每个变量初始化为一个可预测的值（通常是 0）。这意味着，当你关掉调试特性去发布程序、运行优化器，或者只是换一台机器编译时，带有未初始化变量的程序可能会突然表现出不同的行为。不要栽在未初始化的变量上。

当我们定义自己的类型时，对初始化就有更好的控制。如果类型 X 有一个默认构造函数，我们会得到：

```
X* px1 = new X;                 // 一个默认初始化的 X
X* px2 = new X[17];             // 17 个默认初始化的 X
```

如果类型 Y 有构造函数，但没有默认构造函数，我们就必须显式地初始化：

```
Y* py1 = new Y;                 // 错误：没有默认构造函数
Y* py2 = new Y{13};             // 正确：初始化为 Y{13}
Y* py3 = new Y[17];             // 错误：没有默认构造函数
Y* py4 = new Y[17] {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
```

为 new 写很长的初始化列表可能不切实际，但当我们只需要少数几个元素时，它们却非常方便——而后者通常是最常见的情形。

17.4.5 空指针

如果你没有别的指针可用来初始化一个指针，那就用空指针 nullptr：

```
double* p0 = nullptr;      // 空指针
```

赋给指针时，值零被称为空指针；我们常常通过检查一个指针是不是 nullptr 来判断它是否有效（即它是否指向某个东西）。例如：

```
if (p0 != nullptr)         // 认为 p0 有效
```

这并不是一个完美的检查，因为 p0 里可能是一个恰好非零的“随机”值（例如我们忘了初始化），也可能是某个已经被 delete 的对象的地址（见 §17.4.6）。不过，这往往已经是我们能做到的最好的了。其实我们并不一定要显式写出 nullptr，因为 if 语句真正检查的就是它的条件是否不等于 nullptr：

```
if (p0)                    // 认为 p0 有效；等价于 p0!=nullptr
```

我们更喜欢这种较短的写法，认为它更直接地表达了“p0 有效”这个意思，不过各人的看法不尽相同。

当我们有一个指针，它有时指向某个对象、有时什么都不指时，才需要用空指针。这种情况比很多人想象的更少见。想想看：

> 如果你根本没有对象让指针去指，那你为什么要定义那个指针？为什么不等有了对象再定义？

空指针的名字 nullptr 是 C++11 新增的，所以在较老的代码里，人们常用 0（零）或 NULL 代替 nullptr。这两种老写法都可能引起混淆，也可能引起错误，所以请优先使用更明确的 nullptr。

17.4.6 释放自由存储

new 运算符从自由存储中分配（“取得”）内存。由于计算机的内存是有限的，当我们用完一块内存之后，通常最好把它归还给自由存储。这样，自由存储就能把这块内存重新用于新的分配。对于大型程序和长时间运行的程序来说，把内存这样释放出来以便重用是必不可少的。例如：

```
double* calc(int res_size, int max)          // 会泄漏内存
{
    double* p = new double[max];
    double* res = new double[res_size];
    // 用 p 计算要放进 res 的结果
    return res;
}

double* r = calc(100,1000);
```

就现在这样写，每次调用 calc() 都会“泄漏”为 p 分配的那些 double。例如，调用 calc(100,1000) 会让 1000 个 double 所需的空间在程序余下的运行中再也无法使用。

把内存归还给自由存储的运算符叫 delete。我们对由 new 返回的指针施用 delete，从而让这块内存可供自由存储将来分配使用。现在这个例子变成：

```
double* calc(int res_size, int max)
    // 调用者负责为 res 分配的内存
{
    double* p = new double[max];
    double* res = new double[res_size];
    // 用 p 计算要放进 res 的结果
    delete[] p;    // 我们不再需要那块内存了：释放它
    return res;
}

double* r = calc(100,1000);
// 使用 r
delete[] r;      // 我们不再需要那块内存了：释放它
```

顺便说一句，这个例子展示了使用自由存储的主要原因之一：我们可以在函数里创建对象，再把它们传回给调用者。

delete 有两种形式：

- delete p 释放由 new 分配的单个对象的内存。
- delete[ ] p 释放由 new 分配的对象数组的内存。

选对版本，是程序员要做的一件枯燥的活儿。

把同一个对象删除两次是个严重的错误。例如：

```
int* p = new int{5};
delete p;              // 正确：p 指向由 new 创建的对象
// . . . 这里没有再使用 p . . .
delete p;              // 错误：p 指向的内存已归自由存储管理器所有
```

第二个 delete p 有两个问题：

- 你已经不再拥有那个被指向的对象，因此自由存储管理器可能已经改动了它的内部数据结构，以至于它无法再次正确地执行 delete p。
- 自由存储管理器可能已经把 p 所指的内存“回收”掉了，于是 p 现在指向另一个对象；删除那个对象（它属于程序的其他部分）会在你的程序里造成错误。

这两个问题在真实程序里都会出现，并非只是理论上的可能。

删除空指针什么也不做（因为空指针不指向任何对象），所以删除空指针是无害的。例如：

```
int* p = nullptr;
delete p;              // 正确：无需任何操作
delete p;              // 同样正确（仍然无需任何操作）
```

我们为什么非得操心释放内存不可？编译器难道不能自己判断出一块内存什么时候不再需要，然后不经人工干预就把它回收掉吗？它能。这被称为自动垃圾回收（automatic garbage collection），或简称垃圾回收。不幸的是，自动垃圾回收并非没有代价，也不是对所有类型的应用都理想。如果你确实需要自动垃圾回收，可以在你的 C++ 程序里接入一个垃圾回收器。好的垃圾回收器是有的（见 www.stroustrup.com/C++.html）。不过在本书里，我们假设你必须自己处理自己的“垃圾”，并且会告诉你怎样方便而高效地做到这一点。

什么时候不泄漏内存很重要？一个需要“永远”运行的程序负担不起任何内存泄漏。操作系统就是“永远运行”的程序的例子，大多数嵌入式系统（见第 25 章）也是如此。库不应该泄漏内存，因为别人可能把它用作某个不该泄漏内存的系统的一部分。总的来说，不泄漏就是一个好主意。很多程序员把泄漏看作马虎的证据。不过这话说得有点过头了。当你在某个操作系统（Unix、Windows，什么都行）下运行程序时，程序结束时所有内存都会自动归还给系统。由此可知，如果你知道自己的程序不会用到比可用内存更多的内存，你完全可以合理地决定“泄漏”下去，让操作系统替你做释放。但是，如果你决定这么做，就一定要确保你对内存消耗的估计是正确的，否则人们就有充分的理由认为你马虎。

17.5 析构函数

现在我们知道怎样为 vector 存放元素了。我们只要在自由存储上为元素分配足够的空间，然后通过一个指针访问它们：

```
// 一个非常简化的 double 的 vector
class vector {
    int sz;                         // 元素个数
    double* elem;                   // 指向元素的指针
public:
    vector(int s)                   // 构造函数
        :sz{s},                     // 初始化 sz
        elem{new double[s]}         // 初始化 elem
    {
        for (int i=0; i<s; ++i) elem[i]=0;  // 初始化元素
    }
    int size() const { return sz; }  // 当前大小
    // . . .
};
```

所以，sz 是元素的个数。我们在构造函数里初始化它，vector 的用户可以调用 size() 得到元素个数。元素的空间是在构造函数里用 new 分配的，自由存储返回的指针存放在成员指针 elem 里。

注意我们把元素初始化成了它们的默认值（0.0）。标准库的 vector 就是这么做的，所以我们觉得最好从一开始就照做。

不幸的是，我们这个最初版本的 vector 会泄漏内存。构造函数里，它用 new 为元素分配了内存。按照 §17.4 中陈述的规则，我们必须确保这块内存用 delete 释放掉。考虑：

```
void f(int n)
{
    vector v(n);     // 分配 n 个 double
    // . . .
}
```

当我们离开 f() 时，v 在自由存储上创建的那些元素并没有被释放。我们可以为 vector 定义一个 clean_up() 操作，然后调用它：

```
void f2(int n)
{
    vector v(n);     // 定义一个 vector (它又分配了 n 个 int)
    // . . . 使用 v . . .
    v.clean_up();    // clean_up() 删除 elem
}
```

这样能行。然而，自由存储最常见的问题之一，就是人们忘了 delete。同样的问题也会出现在 clean_up() 上：人们会忘了调用它。我们可以做得更好。基本想法是让编译器知道一个与构造函数作用相反的函数，就像它知道构造函数一样。这样的函数理所当然地被称为析构函数（destructor）。正如创建类的对象时会隐式调用构造函数一样，对象离开作用域时也会隐式调用析构函数。构造函数确保对象被正确地创建和初始化；反过来，析构函数确保对象在被销毁之前被正确地清理。例如：

```
// 一个非常简化的 double 的 vector
class vector {
    int sz;                     // 元素个数
    double* elem;               // 指向元素的指针
public:
    vector(int s)               // 构造函数
        :sz{s}, elem{new double[s]}   // 分配内存
    {
        for (int i=0; i<s; ++i) elem[i]=0;  // 初始化元素
    }
    ~vector()                   // 析构函数
        { delete[] elem; }      // 释放内存
    // . . .
};
```

有了它，我们就可以写：

```
void f3(int n)
{
    double* p = new double[n];      // 分配 n 个 double
    vector v(n);                    // vector 分配 n 个 double
    // . . . 使用 p 和 v . . .
    delete[ ] p;                    // 释放 p 的那些 double
}    // vector 自动完成 v 之后的清理
```

突然之间，那个 delete[ ] 显得相当繁琐，而且容易出错！既然有了 vector，就没有理由仅为在函数末尾用 delete[ ] 释放而用 new 去分配内存。那正是 vector 做的事，而且做得更好。特别是，vector 不会忘记调用它的析构函数来释放元素所用的内存。

这里我们不会详谈析构函数的种种用法，但它们非常适合处理那些我们需要先（从某处）获取、后来又归还的资源：文件、线程、锁，等等。还记得 iostream 是怎样自己清理的吗？它们刷新缓冲区、关闭文件、释放缓冲区空间，等等。这些都是由它们的析构函数完成的。每个“拥有”资源的类都需要一个析构函数。

17.5.1 自动生成的析构函数

如果一个类的某个成员有析构函数，那么当包含该成员的对象被销毁时，这个析构函数就会被调用。例如：

```
struct Customer {
    string name;
    vector<string> addresses;
    // . . .
};

void some_fct()
{
    Customer fred;
    // 初始化 fred
    // 使用 fred
}
```

当我们退出 some_fct()、fred 因此离开作用域时，fred 就被销毁了；也就是说，name 和 addresses 的析构函数被调用。要让析构函数有用，这显然是必需的；有时这种说法被表述为“编译器为 Customer 生成了一个析构函数，它会调用各成员的析构函数”。析构函数一定会被调用这一显然而必需的保证，确实常常就是这样实现的。

成员的析构函数——以及基类的析构函数——是从派生类的析构函数中隐式调用的（无论该析构函数是用户定义的还是自动生成的）。基本上，所有规则归结起来就是一句话：“对象被销毁时（因离开作用域、因 delete 等），析构函数就会被调用。”

17.5.2 析构函数与自由存储

析构函数在概念上很简单，却是许多最有效的 C++ 编程技术的基础。基本想法很简单：

- 类对象要工作，需要什么资源，就在构造函数里获取。
- 在对象的生命期内，它可能释放资源，也可能获取新的资源。
- 在对象生命期结束时，析构函数释放该对象仍然拥有的一切资源。

vector 中这对配合默契的构造函数与析构函数负责处理自由存储的内存，是这一模式的典型例子。§19.5 会用更多例子回到这个想法。这里，我们要考察一个重要的应用，它来自自由存储与类层次结构的结合使用。考虑：

```
Shape* fct()
{
    Text tt {Point{200,200},"Annemarie"};
    // . . .
    Shape* p = new Text{Point{100,100},"Nicholas"};
    return p;
}

void f()
{
    Shape* q = fct();
    // . . .
    delete q;
}
```

这看起来相当合理——事实上也确实如此。它完全能工作，但让我们看看它是怎么工作的，因为这会揭示出一种优雅、重要而又简单的技术。在 `fct()` 内部，`Text`（§13.11）对象 `tt` 在退出 `fct()` 时被正确地销毁。`Text` 有一个 `string` 成员，显然需要调用它的析构函数——`string` 获取和释放内存的方式与 `vector` 完全一样。对 `tt` 来说这很容易；编译器只是按照 §17.5.1 所述调用 `Text` 的生成析构函数。可是，从 `fct()` 返回的那个 `Text` 对象呢？调用函数 `f()` 根本不知道 `q` 指向一个 `Text`；它只知道 `q` 指向一个 `Shape`。那么 `delete q` 怎么能调用到 `Text` 的析构函数呢？

在 §14.2.1 中，我们一带而过地提到 `Shape` 有析构函数。事实上，`Shape` 有一个虚析构函数。这就是关键。当我们写 `delete q` 时，`delete` 会查看 `q` 的类型，看是否需要调用析构函数，如果需要就调用它。所以，`delete q`

调用 Shape 的析构函数 `extasciitilde Shape()`。由于 `extasciitilde Shape()` 是 virtual（虚）的，该调用——通过虚调用机制（§14.3.1）——会转而调用 Shape 的某个派生类的析构函数，在这里就是 `extasciitilde Text()`。假如 `Shape::~Shape()` 不是 virtual 的，`Text::~Text()` 就不会被调用，Text 的 `string` 成员也就无法被正确销毁。

根据经验法则：如果一个类带有 virtual 函数，它就需要一个 virtual 析构函数（virtual destructor）。理由是：

1. 如果一个类有 virtual 函数，它很可能被用作基类；
2. 如果它是基类，其派生类对象很可能会用 `new` 分配；
3. 如果派生类对象用 `new` 分配、又通过指向其基类的指针来操纵，那么
4. 它很可能会通过指向其基类的指针被 `delete`。

请注意，析构函数是被隐式地或通过 `delete` 间接调用的，而不是被直接调用。这省去了大量棘手的工作。

试一试

编写一个小程序，使用基类和成员，并定义构造函数和析构函数，使它们在被调用时输出一行信息。然后创建几个对象，观察它们的构造函数和析构函数是如何被调用的。

17.6 访问元素

要让 vector 可用，我们需要一种读写元素的方法。作为起点，我们可以提供简单的 get() 和 set() 成员函数：

```
// 一个非常简化的 double 版 vector
class vector {
    int sz;                         // 元素数目
    double* elem;                   // 指向元素的指针
public:
    vector(int s) :sz{s}, elem{new double[s]} { /* ... */ }  // 构造函数
    ~vector() { delete[] elem; }                             // 析构函数

    int size() const { return sz; }                 // 当前元素数目

    double get(int n) const { return elem[n]; }     // 访问：读
    void set(int n, double v) { elem[n]=v; }        // 访问：写
};
```

get() 和 set() 都是通过对 elem 指针使用 [extasciitilde] 运算符来访问元素的：elem[n]。

现在我们可以创建一个 double 版的 vector 并使用它了：

```
vector v(5);
for (int i=0; i<v.size(); ++i) {
    v.set(i,1.1*i);
    cout << "v[" << i << "]==" << v.get(i) << '\n';
}
```

它将输出

```
v[0]==0
v[1]==1.1
v[2]==2.2
v[3]==3.3
v[4]==4.4
```

这仍然是一个过于简单的 vector，而且与惯用的下标记法相比，使用 get() 和 set() 的代码显得相当笨拙。不过，我们的目标是从小而简单的程序起步，然后一步步扩展程序，并在过程中不断测试。一如既往，这种渐进增长、反复测试的策略能把错误和调试工作降到最少。

17.7 指向类对象的指针

“指针”这个概念是通用的，因此我们可以指向几乎所有能放进内存的东西。例如，我们可以像使用指向 char 的指针那样使用指向 vector 的指针：

```
vector* f(int s)
{
    vector* p = new vector(s);      // 在自由存储上分配一个 vector
    // 填充 *p
    return p;
}

void ff()
{
    vector* q = f(4);
    // 使用 *q
    delete q;                       // 释放自由存储上的 vector
}
```

注意，当我们 delete 一个 vector 时，它的析构函数会被调用。例如：

```
vector* p = new vector(s);          // 在自由存储上分配一个 vector
delete p;                           // 释放
```

在自由存储上创建 vector 时，`new` 运算符

- 首先为一个 vector 分配内存；
- 然后调用该 vector 的构造函数来初始化它；构造函数为 vector 的元素分配内存并初始化这些元素。

删除 vector 时，`delete` 运算符

- 首先调用 vector 的析构函数；析构函数为元素调用各自的析构函数（如果它们有析构函数），然后释放 vector 的元素所用的内存；
- 然后释放 vector 本身所用的内存。

注意，这一切配合得多么精巧，它是递归工作的（参见 §8.5.8）。使用真正的（标准库）vector，我们还可以这样做：

```
vector<vector<double>>* p = new vector<vector<double>>(10);
delete p;
```

这里，`delete p` 调用 `vector<vector<double>>` 的析构函数；这个析构函数进而为其 `vector<double>` 元素调用析构函数，于是一切都被干干净净地清理完毕，不留下任何未销毁的对象，也不泄漏任何内存。

由于 `delete` 会调用析构函数（对于 vector 这类有析构函数的类型），人们常说 `delete` 是销毁（destroy）对象，而不只是释放（deallocate）它们。

一如既往，请记住：在构造函数之外“裸用”`new`，就有可能忘记 delete `new` 所创建的对象。除非你有一个删除对象的良好策略（也就是说，真正简单的策略，例如 §13.10 和 §E.4 的 `Vector_ref`），否则请尽量把 `new` 放在构造函数中、把 `delete` 放在析构函数中。

到目前为止一切顺利，但如果只给定一个指针，我们如何访问 vector 的成员呢？注意，所有类都支持 .（点）运算符，在给定对象名字的情况下访问其成员：

```
vector v(4);
int x = v.size();
double d = v.get(3);
```

类似地，所有类都支持 `->`（箭头）运算符，在给定指向对象的指针的情况下访问其成员：

```
vector* p = new vector(4);
int x = p->size();
double d = p->get(3);
```

与 `.`（点）一样，`->`（箭头）既可用于数据成员，也可用于函数成员。由于 `int` 和 `double` 这类内置类型没有成员，`->` 不适用于内置类型。点和箭头通常被称为成员访问运算符（member access operator）。

17.8 摆弄类型：void* 与类型转换

使用指针和自由存储分配的数组，我们已非常接近硬件。从根本上说，我们对指针的操作（初始化、赋值、`*` 和 `[ extasciitilde ]`）直接映射为机器指令。在这一层次上，语言只提供了一点点记法上的便利，以及类型系统所提供的编译期一致性。偶尔，我们甚至不得不放弃这最后一层保护。

自然，我们不愿意失去类型系统的保护，但有时并没有合乎逻辑的其他选择（例如，我们需要与另一种不了解 C++ 类型的语言交互）。此外，还有为数不少的情况需要对接那些设计时未考虑静态类型安全的老代码。为此，我们需要两样东西：

- 一种指针类型，它指向某块内存，但不关心内存中存放的是哪类对象；
- 一种操作，告诉编译器对这类指针所指向的内存应该（不加证明地）假定为什么类型。

类型 `void*` 的含义是“指向某块内存的指针，但编译器不知道其类型”。当我们想在彼此并不了解对方类型的代码之间传递地址时，就使用 `void*`。例子包括回调函数（callback function，§16.3.1）的“地址”实参，以及最低层次的内存分配器（例如 `new` 运算符的实现）。

不存在 void 类型的对象，但正如我们所见，我们用 void 表示“无返回值”：

```
void v;        // 错误：不存在 void 类型的对象
void f();      // f() 不返回任何值——f() 不返回 void 类型的对象
```

任何对象类型的指针都可以赋给 `void*`。例如：

```
void* pv1 = new int;                // 正确：int* 可转换为 void*
void* pv2 = new double[10];         // 正确：double* 可转换为 void*
```

由于编译器不知道 `void*` 指向什么，我们必须告诉它：

```
void f(void* pv)
{
    void* pv2 = pv;                     // 拷贝没问题（拷贝正是 void* 的用途）
    double* pd = pv;                    // 错误：不能把 void* 转换为 double*
    *pv = 7;                            // 错误：不能解引用 void*
                                        //  （我们不知道它指向的对象是什么类型）
    pv[2] = 9;                          // 错误：不能对 void* 下标
    int* pi = static_cast<int*>(pv);    // 正确：显式转换
    // ...
}
```

static_cast 可用于在相关指针类型（如 void* 和 double*）之间进行显式转换（§A.5.7）。static_cast 这个名字是刻意起得很丑的，对应一种丑陋（且危险）的操作——只在绝对必要时才使用它。你应该很少会需要用到它——甚至根本用不到。诸如 static_cast 这样的操作被称为显式类型转换（explicit type conversion，因为它干的就是这件事），口语中也叫强制类型转换（cast，因为它被用来支撑某种已经坏掉的东西）。

C++ 还提供了两种可能比 static_cast 更加要命的类型转换：

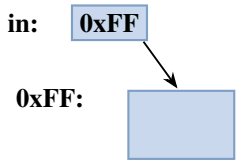
- reinterpret_cast 可以在不相关的类型之间转换，例如 int 和 double*。
- const_cast 可以“把 const 转换掉”。

例如：

```
Register* in = reinterpret_cast<Register*>(0xff);

void f(const Buffer* p)
{
    Buffer* b = const_cast<Buffer*>(p);
    // ...
}
```

第一个例子是 reinterpret_cast 经典的、必要且正当的用法。我们告诉编译器：内存的某个特定部分（从地址 0xFF 开始的内存）将被当作一个 Register（大概带有特殊的语义）。当你编写设备驱动程序之类的软件时，这样的代码是必要的。



在第二个例子中，const_cast 把名为 p 的 const Buffer* 中的 const 去掉了。假定我们知道自己在做什么。

至少 static_cast 不会搅乱指针与整数之间的区别，也不会搅乱“const 性”，因此，如果你觉得需要一个类型转换，请优先使用 static_cast。当你认为自己需要一个类型转换时，请三思：有没有办法不写这个转换？有没有办法重新设计程序的这部分，从而根本不需要它？除非你要对接别人的代码或硬件，通常都是有办法的。如果没有，就等着面对微妙而恶劣的 bug 吧。不要指望使用 reinterpret_cast 的代码具有可移植性。

17.9 指针与引用

你可以把引用（reference）看作一个自动解引用的不可变指针，或者看作对象的另一个名字。指针和引用有如下区别：

- 对指针赋值改变的是指针本身的值（而不是被指向的值）。
- 要得到一个指针，通常需要使用 `new` 或 `&`。
- 要访问指针所指向的对象，需要使用 `*` 或 `[ extasciitilde ]`。
- 对引用赋值改变的是被引用对象的值（而不是引用本身）。
- 初始化之后，无法让引用改指另一个对象。
- 引用的赋值执行深拷贝（赋给被引用的对象）；指针的赋值不是（赋给指针对象本身）。
- 当心空指针。

例如：

```
int x = 10;
int* p = &x;            // 需要用 & 取得指针
*p = 7;                 // 通过 p 用 * 给 x 赋值
int x2 = *p;            // 通过 p 读取 x
int* p2 = &x2;          // 得到指向另一个 int 的指针
p2 = p;                 // p2 和 p 都指向 x
p = &x2;                // 让 p 指向另一个对象
```

引用的对应例子是

```
int y = 10;
int& r = y;             // & 在类型之中，而不是在初始化器之中
r = 7;                  // 通过 r 给 y 赋值（不需要 *）
int y2 = r;             // 通过 r 读取 y（不需要 *）
int& r2 = y2;           // 得到对另一个 int 的引用
r2 = r;                 // y 的值被赋给 y2
r = &y2;                // 错误：不能改变引用的值
                        //  （不能把 int* 赋给 int&）
```

请注意最后一个例子：不只是这种写法会失败——初始化之后，根本没有任何办法让引用改指另一个对象。如果你需要指向不同的东西，就用指针。关于如何使用指针的一些思路，参见 §17.9.3。

引用和指针都是通过内存地址实现的。它们只是以不同的方式使用这个地址，从而为你——程序员——提供略有不同的功能。

17.9.1 指针与引用参数

当你想把一个变量的值改为由函数计算出的值时，你有三种选择。例如：

```
int incr_v(int x) { return x+1; }      // 计算新值并返回它
void incr_p(int* p) { ++*p; }          // 传递指针
                                       //  (解引用它并递增结果)
void incr_r(int& r) { ++r; }           // 传递引用
```

你会怎么选？我们认为返回值往往能得到最直截了当（因而最不容易出错）的代码，即：

```
int x = 2;
x = incr_v(x);          // 把 x 拷贝给 incr_v(); 再把结果拷贝出来并赋值
```

对于小的对象（例如一个 int），我们更喜欢这种风格。此外，如果“大对象”有移动构造函数（move constructor，§18.3.4），我们也能高效地将其来回传递。

那么，在引用实参和指针实参之间如何选择？遗憾的是，两种方式都各有吸引力和问题，所以答案同样不那么明确。你必须根据具体函数及其可能的用法来做出决定。

使用指针实参会提醒程序员：某些东西可能会被改变。例如：

```
int x = 7;
incr_p(&x)              // 需要写 &
incr_r(x);
```

在 incr_p(&x) 中必须使用 &，这就提醒了使用者 x 可能会被改变。相比之下，incr_r(x) “看起来人畜无害”。这使我们对指针版本略加偏爱。

另一方面，如果你把指针用作函数实参，函数就必须提防有人用空指针（即 nullptr）来调用它。例如：

```
incr_p(nullptr);          // 崩溃: incr_p() 将试图解引用空指针
int* p = nullptr;
incr_p(p);                // 崩溃: incr_p() 将试图解引用空指针
```

这显然很糟糕。编写 incr_p() 的人可以对此加以防护：

```
void incr_p(int* p)
{
    if (p==nullptr) error("null pointer argument to incr_p()");
    ++*p;                 // 解引用指针并递增所指向的对象
}
```

但现在 incr_p() 突然不像之前那么简单漂亮了。第 5 章讨论过如何应对不良实参。相比之下，引用的使用者（例如 incr_r()）尽可以假定引用一定指着某个对象。

如果从函数语义的角度看“什么都不传”（不传任何对象）是可接受的，那么就必须使用指针实参。注意：对递增操作来说并非如此——因此才需要针对 p==nullptr 抛出异常。

所以，真正的答案是：“选择取决于函数的性质”：

- 对于很小的对象，优先按值传递。
- 对于“无对象”（用 nullptr 表示）是合法实参的函数，使用指针参数（并记得检验 nullptr）。
- 其他情况下，使用引用参数。

另见 §8.5.6。

17.9.2　指针、引用与继承

在 §14.3 中，我们看到派生类（例如 Circle）如何能被用在需要其公有基类 Shape 对象的地方。我们可以用指针或引用来表述这一思想：因为 Shape 是 Circle 的公有基类，`Circle*` 可以隐式转换为 `Shape*`。例如：

```
void rotate(Shape* s, int n);            // 旋转 *s, n 度

Shape* p = new Circle{Point{100,100},40};
Circle c {Point{200,200},50};

rotate(p,35);
rotate(&c,45);
```

对引用也类似：

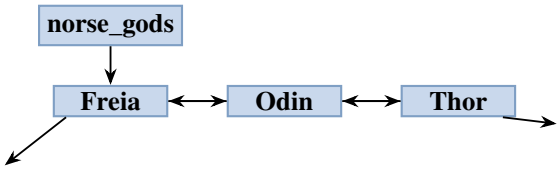
```
void rotate(Shape& s, int n);            // 旋转 s, n 度

Shape& r = c;
rotate(r,55);
rotate(*p,65);
rotate(c,75);
```

这对大多数面向对象编程技术（§14.3–4）至关重要。

17.9.3　一个例子：链表

链表（list）是最常见、最有用的数据结构之一。通常，链表由“链接”（link）构成，每个链接保存一些信息以及指向其他链接的指针。这是指针的经典用途之一。例如，我们可以这样表示一个简短的北欧诸神链表：



这样的链表称为双向链表（doubly-linked list），因为给定一个链接，我们既能找到前驱也能找到后继。只能找到后继的链表则称为单链表（singly-linked list）。当我们希望方便地删除元素时，就使用双向链表。我们可以这样定义这些链接：

```
struct Link {
    string value;
    Link* prev;
    Link* succ;
    Link(const string& v, Link* p = nullptr, Link* s = nullptr)
            : value{v}, prev{p}, succ{s} { }
};
```

也就是说，给定一个 Link，我们可以用 succ 指针到达它的后继，用 prev 指针到达它的前驱。我们用空指针表示某个 Link

没有后继或前驱的情况。我们可以像这样构建北欧诸神链表：

```
Link* norse_gods = new Link{"Thor",nullptr,nullptr};
norse_gods = new Link{"Odin",nullptr,norse_gods};
norse_gods->succ->prev = norse_gods;
norse_gods = new Link{"Freia",nullptr,norse_gods};
norse_gods->succ->prev = norse_gods;
```

我们是这样构建链表的：创建 Link 并把它们按图中的方式连接起来——先是 Thor，然后是作为 Thor 前驱的 Odin，最后是作为 Odin 前驱的 Freia。你可以顺着指针检查一下，确认我们做对了，每个 succ 和 prev 都指向正确的神。然而，这段代码很晦涩，因为我们没有显式地定义并命名一个 insert（插入）操作：

```
Link* insert(Link* p, Link* n)   // 在 p 之前插入 n（不完整）
{
    n->succ = p;                 // p 排在 n 之后
    p->prev->succ = n;           // n 排在 p 原来的前驱之后
    n->prev = p->prev;           // p 的前驱成为 n 的前驱
    p->prev = n;                 // n 成为 p 的前驱
    return n;
}
```

这段代码能工作的前提是：p 确实指向一个 Link，而且 p 所指向的 Link 确实有前驱。请你自己确信这一点确实成立。在思考指针和链式结构（例如由 Link 构成的链表）时，我们总是会在纸上画一些小盒子加箭头的图，来验证代码对小的例子是正确的。请不要自视清高而不屑于依靠这种有效的低技术设计手段。

上面那个版本的 insert() 是不完整的，因为它没有处理 n、p 或 p->prev 为 nullptr 的情况。我们补上对空指针的适当检验，得到更繁琐但正确的版本：

```
Link* insert(Link* p, Link* n)   // 在 p 之前插入 n; 返回 n
{
    if (n==nullptr) return p;
    if (p==nullptr) return n;
    n->succ = p;                 // p 排在 n 之后
    if (p->prev) p->prev->succ = n;
    n->prev = p->prev;           // p 的前驱成为 n 的前驱
    p->prev = n;                 // n 成为 p 的前驱
    return n;
}
```

有了它，我们就可以这样写：

```
Link* norse_gods = new Link{"Thor"};
norse_gods = insert(norse_gods,new Link{"Odin"});
norse_gods = insert(norse_gods,new Link{"Freia"});
```

现在，所有容易出错的 prev 和 succ 指针摆弄都从眼前消失了。指针摆弄是乏味且易错的，应该被隐藏在编写良好、测试充分的函数之中。尤其是，常规代码中的许多错误都源于人们忘记对照 nullptr 检验指针——正如我们（有意地）在第一个版本的 insert() 中所做的那样。

注意，我们使用了默认实参（default argument，§15.3.1、§A.9.2），使用户不必在每次使用构造函数时都提及前驱和后继。

17.9.4 链表操作

标准库提供了一个 list 类，我们将在 §20.4 中介绍。它隐藏了所有链接操纵，但在这里，我们将基于 Link 类细化我们自己的链表概念，以体验 list 类“表层之下”发生了什么，并看到更多使用指针的例子。

我们的 Link 类需要哪些操作，才能让它的用户免于“摆弄指针”？这在一定程度上是个品味问题，但下面是一组有用的操作：

- 构造函数
- insert：在某个元素之前插入
- add：在某个元素之后插入
- erase：删除一个元素
- find：查找具有给定值的 Link
- advance：取得第 n 个后继

我们可以这样编写这些操作：

```
Link* add(Link* p, Link* n)      // 在 p 之后插入 n; 返回 n
{
    // 与 insert 非常相似 (见习题 11)
}

Link* erase(Link* p)             // 从链表中删除 *p; 返回 p 的后继
{
    if (p==nullptr) return nullptr;
    if (p->succ) p->succ->prev = p->prev;
    if (p->prev) p->prev->succ = p->succ;
    return p->succ;
}


Link* find(Link* p, const string& s)   // 在链表中查找 s;
                                       // “未找到”则返回 nullptr
{
    while (p) {
        if (p->value == s) return p;
        p = p->succ;
    }
    return nullptr;
}

Link* advance(Link* p, int n)          // 在链表中移动 n 个位置
                                       // “未找到”则返回 nullptr
                                       // n 为正向后移动, 为负向反方向移动
{
    if (p==nullptr) return nullptr;
    if (0<n) {
        while (n--) {
            if (p->succ == nullptr) return nullptr;
            p = p->succ;
        }
    }
    else if (n<0) {
        while (n++) {
            if (p->prev == nullptr) return nullptr;
            p = p->prev;
        }
    }
    return p;
}
```

注意这里使用的后置 n++。这种递增形式（“后递增”）以递增之前的值作为其值。

17.9.5 链表的使用

作为一个小练习，我们来构建两个链表：

```
Link* norse_gods = new Link("Thor");
norse_gods = insert(norse_gods,new Link{"Odin"});
norse_gods = insert(norse_gods,new Link{"Zeus"});
norse_gods = insert(norse_gods,new Link{"Freia"});

Link* greek_gods = new Link("Hera");
greek_gods = insert(greek_gods,new Link{"Athena"});
greek_gods = insert(greek_gods,new Link{"Mars"});
greek_gods = insert(greek_gods,new Link{"Poseidon"});
```

“不巧”，我们犯了几个错误：Zeus 是希腊神，而不是北欧神；希腊的战神是 Ares，不是 Mars（Mars 是他的拉丁/罗马名字）。我们可以修正：

```
Link* p = find(greek_gods, "Mars");
if (p) p->value = "Ares";
```

注意我们对 `find()` 返回 `nullptr` 保持了谨慎。我们认为在这个例子中不会发生这种情况（毕竟我们刚把 Mars 插入了 `greek_gods`），但在真正的程序中，有人可能会修改那段代码。

类似地，我们可以把 Zeus 移到它正确的万神殿：

```
Link* p = find(norse_gods,"Zeus");
if (p) {
    erase(p);
    insert(greek_gods,p);
}
```

你注意到 bug 了吗？它相当隐蔽（除非你习惯于直接操作链接）。如果我们 `erase()` 的那个 Link 恰好是 `norse_gods` 所指向的那个，会怎么样？同样，在这里这种情况实际上并没有发生，但要编写良好、可维护的代码，我们必须把这种可能性考虑进去：

```
Link* p = find(norse_gods, "Zeus");
if (p) {
    if (p==norse_gods) norse_gods = p->succ;
    erase(p);
    greek_gods = insert(greek_gods,p);
}
```

顺带，我们还修正了第二个 bug：当我们把 Zeus 插到第一个希腊神之前时，需要让 `greek_gods` 指向 Zeus 的 Link。指针极为有用且灵活，但也很微妙。

最后，让我们把这两个链表打印出来：

```
void print_all(Link* p)
{
    cout << "{ ";
    while (p) {
        cout << p->value;
        if (p=p->succ) cout << ", ";
    }
    cout << " }";
}

print_all(norse_gods);
cout<<"\n";

print_all(greek_gods);
cout<<"\n";
```

这应该给出

```
{ Freia, Odin, Thor }
{ Zeus, Poseidon, Ares, Athena, Hera }
```

17.10 this 指针

注意，我们的每个链表函数都以一个 Link* 作为第一个实参，并访问该对象中的数据。这种函数正是我们常常做成成员函数的那一类。能不能通过把操作变成成员来简化 Link（或链接的使用）？能不能把这些指针设为私有的，使只有成员函数才能访问它们？我们可以：

```
class Link {
public:
    string value;

    Link(const string& v, Link* p = nullptr, Link* s = nullptr)
            : value{v}, prev{p}, succ{s} { }

    Link* insert(Link* n);                      // 在本对象之前插入 n
    Link* add(Link* n);                         // 在本对象之后插入 n
    Link* erase();                              // 将本对象从链表中删除
    Link* find(const string& s);                // 在链表中查找 s
    const Link* find(const string& s) const;    // 在 const 链表中查找 s（见 §18.5.1）

    Link* advance(int n) const;                 // 在链表中移动 n 个位置

    Link* next() const { return succ; }
    Link* previous() const { return prev; }
```

```
private:
    Link* prev;
    Link* succ;
};
```

这看起来大有希望。我们把不改变 Link 状态的操作定义成了 const 成员函数。我们添加了（不修改状态的）next() 和 previous() 函数，以便用户可以在链表（由 Link 构成）上迭代——如今对 succ 和 prev 的直接访问已被禁止，这些函数是必需的。我们把 value 留作公有成员，因为（迄今为止）我们没有任何理由不这样做；它“只是数据”。

现在，让我们尝试通过复制之前全局版本的 insert() 并适当修改，来实现 Link::insert()：

```
Link* Link::insert(Link* n)          // 在 p 之前插入 n; 返回 n
{
    Link* p = this;                  // 指向本对象的指针
    if (n==nullptr) return p;        // 没有要插入的东西
    if (p==nullptr) return n;        // 没有可插入的对象
    n->succ = p;                     // p 排在 n 之后
    if (p->prev) p->prev->succ = n;
    n->prev = p->prev;               // p 的前驱成为 n 的前驱
    p->prev = n;                     // n 成为 p 的前驱
    return n;
}
```

但是，我们如何得到指向调用 Link::insert() 的那个对象的指针呢？没有语言的帮助我们做不到。然而，在每个成员函数中，标识符 this 都是一个指针，它指向调用该成员函数所针对的对象。或者，我们也可以直接用 this 代替 p：

```
Link* Link::insert(Link* n)          // 在本对象之前插入 n; 返回 n
{
    if (n==nullptr) return this;
    if (this==nullptr) return n;
    n->succ = this;                  // 本对象排在 n 之后
    if (this->prev) this->prev->succ = n;
    n->prev = this->prev;            // 本对象的前驱
                                     // 成为 n 的前驱
    this->prev = n;                  // n 成为本对象的前驱
    return n;
}
```

这有点啰嗦，但我们访问成员时并不需要提及 this，因此可以简写为：

```
Link* Link::insert(Link* n)          // 在本对象之前插入 n; 返回 n
{
    if (n==nullptr) return this;
    if (this==nullptr) return n;
    n->succ = this;                  // 本对象排在 n 之后
    if (prev) prev->succ = n;
    n->prev = prev;                  // 本对象的前驱成为 n 的前驱
    prev = n;                        // n 成为本对象的前驱
    return n;
}
```

换言之，我们一直在隐式地使用 this 指针（this pointer）——指向当前对象的指针——每次访问成员时都在用它。只有当我们需要引用整个对象时，才需要显式地提及它。

注意，this 有特定的含义：它指向调用成员函数所针对的那个对象，而不指向任意旧对象。编译器会确保我们不能在成员函数中改变 this 的值。例如：

```
struct S {
    // ...
    void mutate(S* p)
    {
        this = p;                    // 错误: this 是不可变的
        // ...
    }
};
```

17.10.1 更多链接使用

处理完实现问题之后，我们来看看现在的使用方式：

```
Link* norse_gods = new Link{"Thor"};
norse_gods = norse_gods->insert(new Link{"Odin"});
norse_gods = norse_gods->insert(new Link{"Zeus"});
norse_gods = norse_gods->insert(new Link{"Freia"});

Link* greek_gods = new Link{"Hera"};
greek_gods = greek_gods->insert(new Link{"Athena"});
greek_gods = greek_gods->insert(new Link{"Mars"});
greek_gods = greek_gods->insert(new Link{"Poseidon"});
```

这与之前非常相似。像之前一样，我们修正“错误”。修正战神的名字：

```
Link* p = greek_gods->find("Mars");
if (p) p->value = "Ares";
```

把 Zeus 移到它正确的万神殿：

```
Link* p2 = norse_gods->find("Zeus");
if (p2) {
    if (p2==norse_gods) norse_gods = p2->next();
    p2->erase();
    greek_gods = greek_gods->insert(p2);
}
```

最后，让我们把这两个链表打印出来：

```
void print_all(Link* p)
{
    cout << "{ ";
    while (p) {
        cout << p->value;
        if (p=p->next()) cout << ", ";
    }
    cout << " }";
}

print_all(norse_gods);
cout<<"\n";

print_all(greek_gods);
cout<<"\n";
```

这次应该再次给出

```
{ Freia, Odin, Thor }
{ Zeus, Poseidon, Ares, Athena, Hera }
```

那么，你更喜欢哪个版本：`insert()` 等是成员函数的版本，还是它们是独立函数的版本？在这个例子中，差异无关紧要，但请参见 §9.7.5。

这里值得注意的一点是：我们仍然没有一个 list（链表）类，只有一个 link（链接）类。这迫使我们一直操心哪个指针是指向第一个元素的指针。我们可以做得更好——通过定义一个 List 类——但沿着这里所呈现的思路进行的设计非常常见。标准库的 list 将在 §20.4 中介绍。

小试身手

本小试身手分为两部分。第一部分练习并巩固你对自由存储分配的数组的理解，并将数组与 vector 进行对比：

1. 用 `new` 在自由存储上分配一个包含十个 `int` 的数组。
2. 将这十个 `int` 的值打印到 `cout`。
3. 释放该数组（使用 `delete[]`）。
4. 编写函数 `print_array10(ostream& os, int* a)`，将 `a` 的值（假定有十个元素）打印到 `os`。
5. 在自由存储上分配一个包含十个 `int` 的数组；用 100、101、102 等值初始化它；并打印出它的值。
6. 在自由存储上分配一个包含 11 个 `int` 的数组；用 100、101、102 等值初始化它；并打印出它的值。

7. 编写函数 `print_array(ostream& os, int* a, int n)`，将 a 的值（假定有 n 个元素）打印到 os。
8. 在自由存储上分配一个包含 20 个 int 的数组；用 100、101、102 等值初始化它；并打印出它的值。
9. 你记得释放那些数组了吗？（如果没有，请补上。）
10. 用 vector 代替数组、用 `print_vector()` 代替 `print_array()`，重做第 5、6 和 8 题。

第二部分聚焦指针及其与数组的关系。使用上一小试身手中的 `print_array()`：

1. 分配一个 int，将它初始化为 7，并把它的地址赋给变量 p1。
2. 打印 p1 的值以及它所指向的 int 的值。
3. 分配一个包含七个 int 的数组；用 1、2、4、8 等值初始化它；并把它的地址赋给变量 p2。
4. 打印 p2 的值以及它所指向的数组的值。
5. 声明一个名为 p3 的 int*，并用 p2 初始化它。
6. 把 p1 赋给 p2。
7. 把 p3 赋给 p2。
8. 打印 p1 和 p2 的值以及它们所指向的内容。
9. 释放你从自由存储分配的所有内存。
10. 分配一个包含十个 int 的数组；用 1、2、4、8 等值初始化它；并把它的地址赋给变量 p1。
11. 分配一个包含十个 int 的数组，并把它的地址赋给变量 p2。
12. 把 p1 所指数组中的值拷贝到 p2 所指向的数组中。
13. 用 vector 而不是数组重做第 10–12 题。

要点回顾

1. 为什么我们需要元素数目可变的数据结构？
2. 对一个典型的程序来说，我们有哪四种存储种类？
3. 什么是自由存储？它还有什么常用的名字？哪些运算符支持它？
4. 什么是解引用运算符？为什么需要它？
5. 什么是地址？在 C++ 中如何操纵内存地址？
6. 对于被指向的对象，指针掌握哪些信息？缺少哪些有用的信息？
7. 指针可以指向什么？
8. 什么是泄漏？
9. 什么是资源？
10. 我们如何初始化指针？
11. 什么是空指针？什么时候需要使用它？
12. 什么时候我们需要指针（而不是引用或具名对象）？
13. 什么是析构函数？什么时候需要析构函数？
14. 什么时候需要 virtual 析构函数？
15. 成员的析构函数是如何被调用的？
16. 什么是类型转换（cast）？什么时候需要使用它？
17. 我们如何通过指针访问类的成员？
18. 什么是双向链表？
19. 什么是 this？什么时候需要使用它？

术语

address（地址） address of: `&` allocation（分配）
cast（类型转换） container（容器） contents of: `*`
deallocation（释放） `delete` `delete[]`
dereference（解引用） destructor（析构函数） free store（自由存储）
link（链接） list（链表） member access: `-->`
member destructor（成员析构函数） memory（内存） memory leak（内存泄漏）
`new` null pointer（空指针） `nullptr`
pointer（指针） range（范围） resource leak（资源泄漏）
subscripting（下标） subscript: `[ extasciitilde ]` `this`
type conversion（类型转换） virtual destructor（虚析构函数） `void*`

习题

1. 在你的实现上，指针值的输出格式是什么？提示：不要去读文档。
2. 一个 `int` 有多少个字节？`double` 呢？`bool` 呢？除了用于验证答案之外，不要使用 `sizeof`。
3. 编写函数 `void to_lower(char* s)`，将 C 风格字符串 `s` 中的所有大写字符替换为对应的小写字符。例如，`Hello, World!` 变为 `hello, world!`。不要使用任何标准库函数。C 风格字符串是一个以 0 结尾的字符数组，因此如果你找到一个值为 0 的 `char`，就到了末尾。
4. 编写函数 `char* strdup(const char*)`，将一个 C 风格字符串拷贝到它在自由存储上分配的内存中。不要使用任何标准库函数。
5. 编写函数 `char* findx(const char* s, const char* x)`，查找 C 风格字符串 `x` 在 `s` 中首次出现的位置。
6. 本章没有说明用 `new` 耗尽内存时会发生什么。这称为内存耗尽（memory exhaustion）。弄清楚会发生什么。你有两个显而易见的选择：查找文档，或编写一个无限循环、只分配从不释放的程序。两种都试试。在失败之前，你大约成功分配了多少内存？
7. 编写一个程序，从 `cin` 读取字符存入你在自由存储上分配的数组。逐个读取字符，直到输入感叹号（!）为止。不要使用 `std::string`。不必担心内存耗尽。
8. 再做一遍习题 7，但这次读入到 `std::string` 中，而不是读入到你放在自由存储上的内存（string 知道如何替你使用自由存储）。
9. 栈向哪个方向增长：向上（朝更高的地址）还是向下（朝更低的地址）？自由存储最初向哪个方向增长（即在你使用 `delete` 之前）？编写一个程序来确定答案。
10. 看看你习题 7 的解。输入是否有可能使数组溢出；也就是说，你是否有可能输入超过所分配空间的字符（一个严重的错误）？如果你试图输入超过所分配空间的字符，会发生什么合理的事情吗？
11. 完成 §17.10.1 的“诸神链表”例子并运行它。
12. 我们为什么要定义两个版本的 `find()`？
13. 修改 §17.10.1 的 Link 类，使它保存一个 struct God 类型的值。struct God 应该有类型为 `string` 的成员：name、mythology、vehicle 和 weapon。例如，`God{"Zeus", "Greek", "", "lightning"}` 和 `God{"Odin", "Norse", "Eight-legged flying horse called Sleipner", "Spear called Gungnir"}`。编写一个 `print_all()` 函数，逐行列出诸神及其属性。添加一个成员函数 `add_ordered()`，把新元素放在其正确的字典序位置上。使用值为 God 类型的 Link，用三种神话

体系的神各建一个链表；然后把元素（神）从那个链表移到三个按字典序排列的链表中——每种神话体系一个。

14. §17.10.1 的“诸神链表”例子能否用单链表来编写；也就是说，我们能否把 `prev` 成员从 Link 中去掉？我们为什么会想这样做？对哪类例子来说使用单链表是合理的？只用单链表重新实现那个例子。

后记

既然我们可以直接使用 vector，为什么还要去纠缠指针和自由存储这类杂乱的底层东西？一个回答是：总得有人去设计和实现 vector 及类似的抽象，而我们想知道这是怎么做到的。有些编程语言不提供与指针对等的设施，从而回避了底层编程的问题。从根本上说，这类语言的使用者把涉及直接访问硬件的任务交给了 C++ 程序员（以及其他适合底层编程语言的使用者）。不过，我们最喜欢的理由很简单：在你亲眼见到软件如何与硬件相遇之前，你还不能真正声称自己理解计算机和编程。不了解指针、内存地址等概念的人，常常对自己编程语言的各个设施如何工作抱有最古怪的想法；这些错误的想法可能导致“有趣地糟糕”的代码。

18 向量与数组

"Caveat emptor!"（买者自慎）

——Good advice（忠告）

本章介绍 vector 如何被拷贝以及如何通过下标访问。为此，我们将一般性地讨论拷贝，考察 vector 与更底层的数组概念之间的关系。我们将介绍数组与指针的关系，并讨论使用它们所带来的问题。我们还将介绍每种类型都必须考虑的五种基本操作：构造、默认构造、拷贝构造、拷贝赋值和析构。此外，容器还需要移动构造函数和移动赋值运算符。

18.1 引言
18.2 初始化
18.3 拷贝
18.3.1 拷贝构造函数
18.3.2 拷贝赋值
18.3.3 拷贝术语
18.3.4 移动
18.4 基本操作
18.4.1 explicit 构造函数
18.4.2 调试构造函数与析构函数
18.5 Access to `vector` elements
18.6 数组
18.6.1 指向数组元素的指针
18.6.2 指针与数组
18.6.3 数组初始化
18.6.4 指针的问题
18.7 示例：回文
18.7.1 Palindromes using `string`
18.7.2 用数组求回文
18.7.3 用指针求回文
18.5.1 Overloading on `const`

18.1 引言

飞机要上天，就得沿跑道加速，直到快得足以"跳"入空中。当飞机在跑道上笨重滑行时，它不过是一辆特别笨重蹩脚的卡车；而一旦升空，它便翱翔天际，成为一种完全不同的、优雅而高效的运载工具——这才是它的用武之地。

在本章中，我们正处于一次"助跑"的中途：要积累足够的语言特性和技术，以摆脱普通计算机内存的种种限制与困难。我们的目标是达到这样一个境界：可以根据逻辑需要，使用恰好具备我们所要性质（property）的类型来编程。为了"到达那里"，我们必须克服与访问裸机（bare machine）相关的一系列基本限制，例如：

- 内存中的对象具有固定的大小。
- 内存中的对象位于某个特定位置。
- 计算机只对此类对象提供少数几种基本操作（例如拷贝一个字、把两个字中的值相加等）。

从根本上说，这些就是 C++ 内置类型与操作所受的限制（C++ 是通过 C 从硬件继承这些的；参见 §22.2.5 和第 27 章）。在第 17 章中，我们看到了一种 vector 类型的雏形，它控制着对其元素的一切访问，并从用户（而非硬件）的角度提供一些看起来"自然"的操作。

本章聚焦于拷贝这一概念。这是一个重要但相当技术性的问题：拷贝一个不那么简单的对象究竟意味着什么？拷贝操作完成后，两个副本在多大程度上是相互独立的？拷贝操作有哪些？我们如何指定它们？它们又与初始化和清理等其他基本操作如何关联？

不可避免地，我们将讨论在没有 vector 和 string 这类高层类型时如何操纵内存。我们将考察数组与指针、二者的关系、它们的用法以及使用中的陷阱与隐患。对于任何要接触 C++ 或 C 的底层用法的人来说，这些都是必不可少的知识。

请注意，vector 的种种细节是 vector 特有的，也是 C++ 从低层类型构建新的高层类型的特有方式。不过，任何语言中的每一种“高层”类型（string、vector、list、map 等）在某种程度上都是由同样的机器原语构建的，都是对这里所描述的基本问题的各种不同解决之道。

18.2　初始化

先来看看第 17 章末尾我们的 vector：

```
class vector {
    int sz;                         // 大小
    double* elem;                   // 指向元素的指针
public:
    vector(int s)                                           // 构造函数
        :sz{s}, elem{new double[s]} { /* ... */ }           // 分配内存
    ~vector()                                               // 析构函数
        { delete[] elem; }                                  // 释放内存
    // ...
};
```

这没问题，但如果我们想把 vector 初始化为一组并非默认值的值，该怎么办呢？例如：

```
vector v1 = {1.2, 7.89, 12.34};
```

我们可以这样做，而且这比先初始化为默认值、再赋上我们真正想要的值要好得多：

```
vector v2(2);           // 繁琐且易错
v2[0] = 1.2;
v2[1] = 7.89;
v2[2] = 12.34;
```

与 v1 相比，v2 的“初始化”既繁琐又易错（我们在那个代码片段中故意把元素数目弄错了）。使用 push_back() 可以让我们免于提及元素数目：

```
vector v3;                      // 繁琐且重复
v2.push_back(1.2);
v2.push_back(7.89);
v2.push_back(12.34);
```

但这仍然是重复的。那么，我们如何编写一个接受初始化器列表（initializer list）作为实参的构造函数呢？一个由类型为 T 的元素构成的、以 { } 界定的列表，会以标准库类型 initializer_list<T> 的对象的形式呈现给程序员——也就是一个 T 的列表。因此，我们可以这样写：

```
class vector {
    int sz;                                 // 大小
    double* elem;                           // 指向元素的指针
public:
    vector(int s)                           // 构造函数（s 是元素数目）
        :sz{s}, elem{new double[sz]}        // 为元素分配未初始化的内存
    {
        for (int i = 0; i<sz; ++i) elem[i] = 0.0;   // 初始化
    }

    vector(initializer_list<double> lst)    // 初始化器列表构造函数
        :sz{lst.size()}, elem{new double[sz]}   // 为元素分配未初始化的内存
    {
        copy(lst.begin(),lst.end(),elem);   // 初始化（使用 std::copy(); 参见 §B.5.2）
    }
    // ...
};
```

我们使用了标准库的 copy 算法（§B.5.2）。它把由前两个实参指定的元素序列（这里是 initializer_list 的开头和结尾）拷贝到从第三个实参开始的元素序列中（这里是从 elem 开始的 vector 元素）。

现在我们可以这样写：

```
vector v1 = {1,2,3};        // 三个元素 1.0, 2.0, 3.0
vector v2(3);               // 三个元素，每个的（默认）值为 0.0
```

注意，我们用 ( ) 表示元素数目，用 { } 表示元素列表。我们需要一种记法来区分二者。例如：

```
vector v1 {3};          // 一个元素，值为 3.0
vector v2(3);           // 三个元素，每个的（默认）值为 0.0
```

这不算优雅，但行之有效。如果存在选择，编译器会把 { } 列表中的值解释为元素值，并把它作为 initializer_list 的一个元素传递给初始化器列表构造函数。

在大多数情况下——包括本书将遇到的所有情况——{ } 初始化器列表之前的 = 是可选的，因此我们可以这样写：

```
vector v11 = {1,2,3};       // 三个元素 1.0, 2.0, 3.0
vector v12 {1,2,3};         // 三个元素 1.0, 2.0, 3.0
```

二者的差别纯粹是风格问题。

注意，我们是按值传递 initializer_list<double> 的。这是有意为之，也是语言规则所要求的：initializer_list 只不过是一个指向“别处”分配的元素的句柄（handle）（参见 §B.6.4）。

18.3 拷贝

再来看看我们那个不完整的 vector：

```
class vector {
    int sz;                             // 大小
    double* elem;                       // 指向元素的指针
public:
    vector(int s)                       // 构造函数
        :sz{s}, elem{new double[s]} { /* ... */ }   // 分配内存
    ~vector()                           // 析构函数
        { delete[] elem; }              // 释放内存
    // ...
};
```

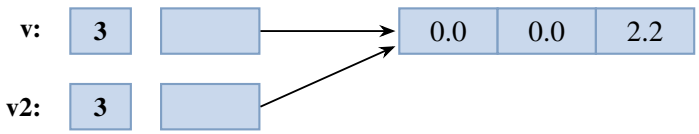
让我们试着拷贝一个这样的 vector：

```
void f(int n)
{
    vector v(3);            // 定义一个有 3 个元素的 vector
    v.set(2,2.2);           // 将 v[2] 设置为 2.2

    vector v2 = v;          // 这里会发生什么？
    // ...
}
```

理想情况下，v2 会成为 v 的一个副本（也就是说，= 实施拷贝）；也就是说，v2.size()==v.size()，并且对所有位于 [0:v.size()) 范围内的 i 都有 v2[i]==v[i]。此外，从 f() 退出时，所有内存都被归还给自由存储。标准库 vector（当然）正是这么做的，但对那个仍然过于简单的 vector 来说，事情并非如此。我们的任务是改进我们的 vector，使其能正确处理这类例子；但首先，让我们弄清楚现在的版本实际上做了什么：它到底错在哪里？怎么错的？为什么错？一旦弄清了这些，我们大概就能修复这些问题。更重要的是，当在其他场合遇到类似问题时，我们就有机会识别并避免它们。

对一个类来说，拷贝的默认含义是“拷贝所有数据成员”。这常常是完全合理的。例如，拷贝一个 Point 就是拷贝它的坐标。但对指针成员来说，仅仅拷贝成员就会带来问题。具体到我们例子中的两个 vector，这意味着拷贝之后，我们有 v.sz==v2.sz 和 v.elem==v2.elem，于是这两个 vector 看起来是这样：



也就是说，v2 并没有获得 v 的元素的副本，它共享了 v 的元素。我们可以这样写：

```
v.set(1,99);            // 将 v[1] 设置为 99
v2.set(0,88);           // 将 v2[0] 设置为 88
cout << v.get(0) << ' ' << v2.get(1);
```

结果会输出 88 99。这不是我们想要的。假如 v 与 v2 之间没有这种“隐藏”的关联，我们会得到输出 0 0，因为我们从未写过 v[0] 或 v2[1]。你可能会说，我们观察到的这种行为“有趣”、“妙极了”或者“有时挺有用”，但这既非我们的本意，也不是标准库 vector 提供的行为。而且，当我们从 f() 返回时，会发生一场不折不扣的灾难。此时，v 和 v2 的析构函数被隐式调用；v 的析构函数用

```
delete[] elem;
```

释放用于存放元素的存储空间，v2 的析构函数也会如此。由于 elem 在 v 和 v2 中指向同一块内存，这块内存将被释放两次，很可能带来灾难性后果（§17.4.6）。

18.3.1　拷贝构造函数

那么，我们该怎么办？答案很直白：提供一个拷贝元素的拷贝操作，并确保当我们用一个 vector 初始化另一个 vector 时，调用的是这个拷贝操作。

类对象的初始化由构造函数完成。因此，我们需要一个会拷贝的构造函数。不出所料，这样的构造函数称为拷贝构造函数（copy constructor）。它被定义为以被拷贝对象（即拷贝来源）的引用作为实参。于是，对于 vector 类，我们需要

```
vector(const vector&);
```

当我们试图用一个 vector 初始化另一个 vector 时，这个构造函数就会被调用。我们按引用传递，是因为（显然）我们不想拷贝定义拷贝的那个构造函数的实参；我们按 const 引用传递，是因为我们不想修改实参（§8.5.6）。于是，我们这样细化 vector：

```
class vector {
    int sz;
    double* elem;
public:
    vector(const vector&);              // 拷贝构造函数：定义拷贝
    // ...
};
```

拷贝构造函数先设定元素数目（sz）并为元素分配内存（初始化 elem），然后再从实参 vector 拷贝元素的值：

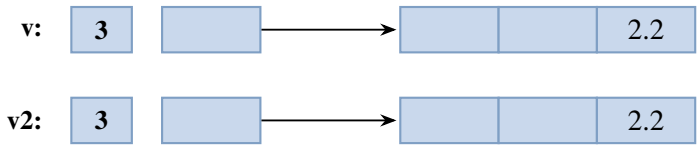
```
vector::vector(const vector& arg)
        // 分配元素，然后通过拷贝来初始化它们
    :sz{arg.sz}, elem{new double[arg.sz]}
{
    copy(arg,arg+sz,elem);              // std::copy(); 参见 §B.5.2
}
```

有了这个拷贝构造函数，再来看看我们的例子：

```
vector v2 = v;
```

这个定义会以 v 为实参调用 vector 的拷贝构造函数来初始化 v2。再给定一个有三个元素的 vector，现在我们得到

这样一来，析构函数就能做正确的事了：每一组元素都会被正确释放。显然，此时两个 vector 是相互独立的，因此我们可以修改 v 中元素的值而不影响 v2，反之亦然。例如：




```
v.set(1,99);                        // 将 v[1] 设置为 99
v2.set(0,88);                       // 将 v2[0] 设置为 88
cout << v.get(0) << ' ' << v2.get(1);
```

这将输出 0 0。

除了写

```
vector v2 = v;
```

我们同样可以写

```
vector v2 {v};
```

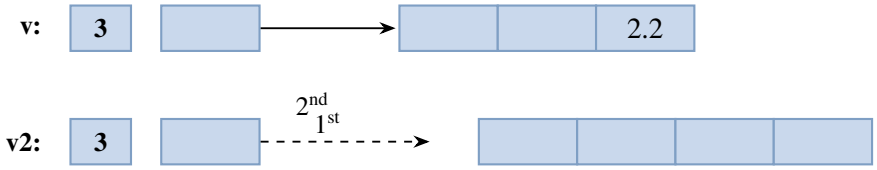
当 v（初始化器）与 v2（被初始化的变量）类型相同，且该类型按惯例定义了拷贝时，这两种记法的含义完全相同，你可以选用自己更喜欢的那种。

18.3.2　拷贝赋值

我们已经处理了拷贝构造（初始化），但我们还可以通过赋值来拷贝 vector。与拷贝初始化一样，拷贝赋值的默认含义是逐成员拷贝（memberwise copy），因此就目前为止所定义的 vector 而言，赋值会导致双重释放（与 §18.3.1 中针对拷贝构造函数所示的情形完全相同），外加一次内存泄漏。例如：

```
void f2(int n)
{
    vector v(3);            // 定义一个 vector
    v.set(2,2.2);
    vector v2(4);
    v2 = v;                 // 赋值：这里会发生什么？
    // ...
}
```

我们希望 v2 成为 v 的一个副本（标准库 vector 正是这么做的），但由于我们从未说明我们的 vector 的赋值含义，就会使用默认的赋值；也就是说，赋值是逐成员拷贝，v2 的 sz 和 elem 分别变成与 v 的 sz 和 elem 相同。我们可以这样图示：



当我们离开 f2() 时，就会遇到与 §18.3 中加入拷贝构造函数之前离开 f() 时同样的灾难：v 和 v2 所指向的元素都被释放了两次（用 delete[]）。此外，我们还泄漏了最初为 v2 的四个元素分配的内存——我们“忘了”释放它们。这个拷贝赋值问题的补救办法与拷贝初始化（§18.3.1）在本质上是相同的：我们定义一个能正确拷贝的赋值：

```
class vector {
    int sz;
    double* elem;
public:
    vector& operator=(const vector&);       // 拷贝赋值
    // ...
};

vector& vector::operator=(const vector& a)
        // 让本 vector 成为 a 的副本
{
    double* p = new double[a.sz];           // 分配新空间
    copy(a.elem,a.elem+a.sz,elem);          // 拷贝元素
    delete[] elem;                          // 释放旧空间
    elem = p;                               // 现在可以重置 elem 了
    sz = a.sz;
    return *this;                           // 返回自身引用（参见 §17.10）
}
```

赋值比构造要复杂一些，因为我们必须处理旧元素。我们的基本策略是先从源 vector 拷贝一份元素：

```
double* p = new double[a.sz];           // 分配新空间
copy(a.elem,a.elem+a.sz,elem);          // 拷贝元素
```

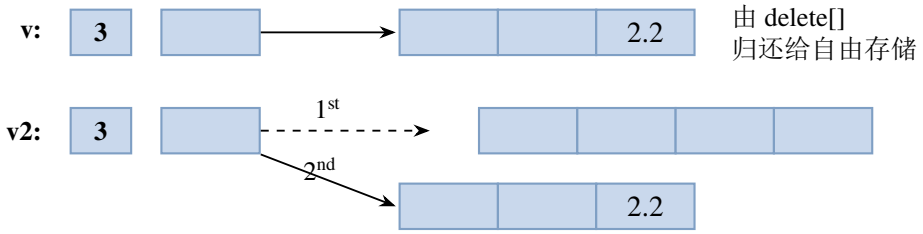
然后释放目标 vector 的旧元素：

```
delete[] elem;                          // 释放旧空间
```

最后，我们让 elem 指向新的元素：

```
elem = p;                               // 现在可以重置 elem 了
sz = a.sz;
```

我们可以用图形把结果表示为：



现在我们有了一个既不泄漏内存、也不会对任何内存释放（delete[]）两次的 vector。

在实现赋值时，你可能会考虑简化代码：在创建副本之前先释放旧元素的内存。但通常，在还不知道自己能否替换它之前就把它扔掉，绝不是一个好主意。而且，如果那样做，当你把一个 vector 赋给它自己时，就会发生怪事：

```
vector v(10);
    v = v;                  // 自赋值
```

请检查我们的实现能正确处理这种情况（即便效率不是最优）。

18.3.3　拷贝术语

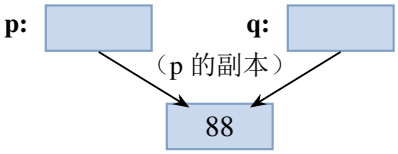
拷贝是大多数程序和大多数程序设计语言中的问题。基本问题在于：你拷贝的是指针（或引用）本身，还是拷贝被指向（被引用）的信息：

- 浅拷贝（shallow copy）只拷贝指针，于是两个指针现在指向同一个对象。指针和引用做的就是这种事。
- 深拷贝（deep copy）拷贝指针所指向的内容，于是两个指针现在指向不同的对象。vector、string 等做的就是这种事。当我们希望类的对象进行深拷贝时，就定义拷贝构造函数和拷贝赋值。

下面是一个浅拷贝的例子：

```
int* p = new int{77};
int* q = p;             // 拷贝指针 p
*p = 88;                // 修改 p 和 q 所指向的 int 的值
```

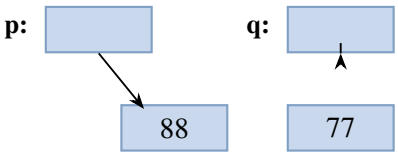
我们可以这样图示：



作为对比，我们可以进行深拷贝：

```
int* p = new int{77};
int* q = new int{*p};   // 分配一个新的 int，然后拷贝 p 所指向的值
*p = 88;                // 修改 p 所指向的 int 的值
```

我们可以这样图示：



用这套术语，我们可以说，原来的 vector 的问题在于它做的是浅拷贝，而不是拷贝 elem 指针所指向的元素。我们改进后的 vector 与标准库 vector 一样，通过为新元素分配空间并拷贝其值来进行深拷贝。提供浅拷贝的类型（如指针和引用）被称为具有指针语义（pointer semantics）或引用语义（reference semantics）（它们拷贝地址）。提供深拷贝的类型（如 string 和 vector）被称为具有值语义（value semantics）（它们拷贝被指向的值）。从用户的角度看，具有值语义的类型表现得就好像没有任何指针一样——只有可以拷贝的值。理解具有值语义的类型的一种方式是：就拷贝而言，它们“工作起来就像 int 一样”。

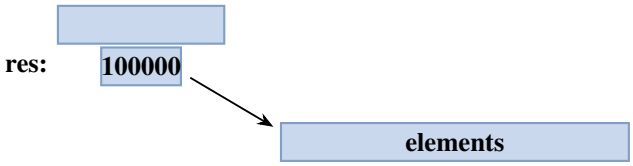
18.3.4 移动

如果一个 vector 有很多元素，拷贝它的代价可能很高。因此，我们应当只在必要时才拷贝 vector。看一个例子：

```
vector fill(istream& is)
{
    vector res;
    for (double x; is>>x; ) res.push_back(x);
    return res;
}

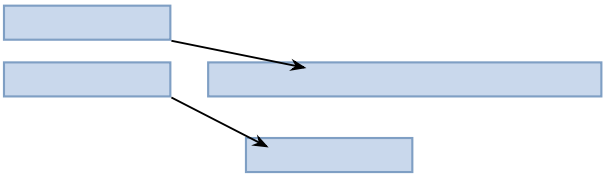
void use()
{
    vector vec = fill(cin);
    // ... 使用 vec ...
}
```

这里，我们从输入流填充局部 vector res，并把它返回给 use()。把 res 从 fill() 拷贝到 vec 中的代价可能很高。可为什么要拷贝呢？我们要的并不是副本！返回之后，我们绝不可能再使用原来的对象（res）。事实上，res 会作为从 fill() 返回的一部分被销毁。那么，如何避免这次拷贝呢？再看一下 vector 在内存中是如何表示的：



我们希望“窃取”res 的表示，拿给 vec 用。换句话说，我们希望 vec 不经任何拷贝就直接引用 res 的元素。

把 res 的元素指针和元素计数移动到 vec 之后，res 就不再持有任何元素。我们已成功把值从 fill() 内的 res 移动（move）到了 vec。现在，res 可以被销毁（简单而高效地），而不带来任何不良副作用：



我们以四次单字赋值的代价，成功地把 100 000 个 double 从 fill() 移到了它的调用者中。

如何在 C++ 代码中表达这样的移动呢？我们定义移动操作，作为拷贝操作的补充：

```
class vector {
    int sz;
    double* elem;
public:
    vector(vector&& a);                 // 移动构造函数
    vector& operator=(vector&&);        // 移动赋值
    // ...
};
```

这个奇怪的 && 记法称为“右值引用”（rvalue reference）。我们用它来定义移动操作。注意，移动操作不接受 const 实参；也就是说，我们写 (vector&&)，而不是 (const vector&&)。移动操作的目的之一就是修改源对象，使之“变空”。移动操作的定义往往很简单，通常比对应的拷贝操作更简单、更高效。对于 vector，我们得到

```
vector::vector(vector&& a)
    :sz{a.sz}, elem{a.elem}             // 拷贝 a 的 elem 和 sz
{
    a.sz = 0;                           // 让 a 成为空 vector
    a.elem = nullptr;
}

vector& vector::operator=(vector&& a)   // 把 a 移动到本 vector
{
    delete[] elem;                      // 释放旧空间
    elem = a.elem;                      // 拷贝 a 的 elem 和 sz
    sz = a.sz;
    a.elem = nullptr;                   // 让 a 成为空 vector
    a.sz = 0;
    return *this;                       // 返回自身引用（参见 §17.10）
}
```

通过定义移动构造函数，我们就能方便而廉价地移动大量信息，例如一个有很多元素的 vector。再看一遍：

```
vector fill(istream& is)
{
    vector res;
    for (double x; is>>x; ) res.push_back(x);
    return res;
}
```

移动构造函数被隐式地用来实现返回。编译器知道被返回的局部值（res）即将离开作用域，因此它可以从中移动，而不是拷贝。

移动构造函数的重要性在于：要把大量信息从函数中取出来，我们不必再跟指针或引用打交道。考虑下面这个有缺陷的（但传统的）替代方案：

```
vector* fill2(istream& is)
{
    vector* res = new vector;
    for (double x; is>>x; ) res->push_back(x);
    return res;
}

void use2()
{
    vector* vec = fill(cin);
    // ... 使用 vec ...
    delete vec;
}
```

现在我们必须记得去 delete 这个 vector。如 §17.4.6 所述，删除放在自由存储上的对象并不像看上去那样容易做到始终如一且正确。

18.4　基本操作

现在我们来到了这样一个节点：可以讨论如何决定一个类应该有哪些构造函数、是否应该有析构函数，以及是否需要提供拷贝和移动操作。有七种基本操作需要考虑：

- 带一个或多个实参的构造函数
- 默认构造函数
- 拷贝构造函数（拷贝同类型对象）
- 拷贝赋值（拷贝同类型对象）
- 移动构造函数（移动同类型对象）
- 移动赋值（移动同类型对象）
- 析构函数

通常，我们需要一个或多个构造函数，它们接受初始化对象所需的实参。例如：

```
string s {"cat.jpg"};                  // 将 s 初始化为字符串 "cat.jpg"
Image ii {Point{200,300},"cat.jpg"};   // 用坐标 {200,300} 初始化一个 Point，
                                       // 然后在该 Point 处显示文件 cat.jpg 的内容
```

初始化器的含义和用途完全由构造函数决定。标准 string 的构造函数把字符串当作初始值，而 Image 的构造函数把字符串当作要打开的文件名。我们通常用构造函数来建立不变式（invariant，§9.4.3）。如果我们无法为某个类定义一个其构造函数能够建立的良好不变式，那这个类多半设计得很糟，或者它只是一个普通的数据结构。

带实参的构造函数与其服务的类一样多种多样。其余操作则呈现出更有规律的模式。

我们如何知道一个类是否需要默认构造函数？如果我们希望能够在不指定初始化器的情况下创建该类的对象，就需要默认构造函数。最常见的例子是想把类的对象放进标准库 vector 的时候。下面的代码之所以能工作，只是因为 int、string 和 vector<int> 都有默认值：

```
vector<double> vi(10);          // 10 个 double 的 vector，每个初始化为 0.0
vector<string> vs(10);          // 10 个 string 的 vector，每个初始化为 ""
vector<vector<int>> vvi(10);    // 10 个 vector 的 vector，每个初始化为 vector{}
```

所以，拥有默认构造函数通常是很有用的。那么问题就变成：“什么时候拥有默认构造函数才是合理的？”一个答案是：“当我们能用一个有意义且显而易见的默认值建立类的不变式时。”对于 int 和 double 这类值类型，显而易见的值是 0（对 double 来说就是 0.0）。对 string 来说，空字符串”” 是显而易见的选择。对 vector 来说，空 vector 就很好。对任何类型 T 来说，T{} 就是默认值（如果默认值存在的话）。例如，double{} 是 0.0，string{} 是””，而 vector<int>{} 是空的 int 向量。

如果一个类获取了资源，它就需要析构函数。资源（resource）是你“从某处获得”的东西，用完之后必须归还。最明显的例子是从自由存储获得的内存（用 new 获得，必须用 delete 或 delete[] 归还）。我们的 vector 获取内存来存放元素，所以它必须把内存归还回去；因此它需要析构函数。随着程序雄心和复杂程度的增长，你可能遇到的其他资源还有文件（打开一个文件，就得关闭它）、锁、线程句柄和套接字（用于与进程及远程计算机通信）。

另一个表明类需要析构函数的迹象是：它拥有指针或引用类型的成员。如果一个类有指针或引用成员，它通常需要析构函数和拷贝操作。

需要析构函数的类几乎总是还需要拷贝构造函数和拷贝赋值。原因很简单：如果一个对象已获取了资源（并有指针成员指向它），拷贝的默认含义（浅层的逐成员拷贝）几乎肯定是错的。vector 又一次成为经典例子。

类似地，需要析构函数的类几乎总是还需要移动构造函数和移动赋值。原因也很简单：如果一个对象已获取了资源（并有指针成员指向它），拷贝的默认含义（浅层的逐成员拷贝）几乎肯定是错的，而通常的补救办法（通过拷贝操作复制完整的对象状态）可能代价高昂。vector 还是经典例子。

此外，对于派生类可能拥有析构函数的基类来说，它需要虚析构函数（virtual destructor，§17.5.2）。

18.4.1 explicit 构造函数

接受单个实参的构造函数定义了从其实参类型到其类的一个转换。这可能非常有用。例如：

```
class complex {
public:
    complex(double);            // 定义 double 到 complex 的转换
    complex(double,double);
    // ...
};

complex z1 = 3.14;              // 正确：把 3.14 转换为 (3.14,0)
complex z2 = complex{1.2, 3.4};
```

然而，隐式转换应当少用、慎用，因为它们可能造成意外的、不合需要的效果。例如，就目前为止所定义的 vector 有一个接受 int 的构造函数。这意味着它定义了从 int 到 vector 的转换。例如：

```
class vector {
     // ...
    vector(int);
    // ...
};

vector v = 10;                  // 奇怪：创建了一个有 10 个 double 的 vector
v = 20;                         // 嗯？把一个新的有 20 个 double 的 vector 赋给 v

void f(const vector&);
f(10);                          // 嗯？用一个新的有 10 个 double 的 vector 调用 f
```

看来我们得到的比预期 bargain 来的要多。幸运的是，禁止把构造函数用作隐式转换很简单。定义为 explicit 的构造函数只提供通常的构造语义，而不提供隐式转换。例如：

```
class vector {
    // ...
    explicit vector(int);
    // ...
};

vector v = 10;                  // 错误：没有 int 到 vector 的转换
v = 20;                         // 错误：没有 int 到 vector 的转换
vector v0(10);                  // 正确

void f(const vector&);
f(10);                          // 错误：没有 int 到 vector<double> 的转换
f(vector(10));                  // 正确
```

为了避免令人意外的转换，我们——以及标准——把 vector 的单实参构造函数定义为 explicit。遗憾的是构造函数默认不是 explicit 的；拿不准的时候，就把任何可以用单个实参调用的构造函数声明为 explicit。

18.4.2 调试构造函数与析构函数

构造函数和析构函数在程序执行中定义良好、可预测的时点被调用。然而，我们并不总是写显式调用，例如 vector(2)；我们做的是另外一些事，例如声明一个 vector、按值传递一个 vector 实参，或者用 new 在自由存储上创建一个 vector。这对那些从语法角度思考的人来说容易引起混淆。触发构造函数的并非只有一种语法。用下面的方式来思考构造函数和析构函数更简单：

- 每当创建一个类型为 X 的对象，就会调用 X 的某个构造函数。
- 每当销毁一个类型为 X 的对象，就会调用 X 的析构函数。

只要一个类的对象被销毁，就会调用析构函数；这发生在名字离开作用域、程序终止，或者对指向对象的指针使用 delete 之时。只要一个类的对象被创建，就会调用（某个适当的）构造函数；这发生在变量被初始化、用 new 创建对象（内置类型除外），以及对象被拷贝之时。

但这一切何时发生呢？找到感觉的一个好办法是：在构造函数、赋值操作和析构函数中加入打印语句，然后试一试。例如：

```
struct X {                          // 简单的测试类
    int val;

    void out(const string& s, int nv)
        { cerr << this << "->" << s << ": " << val << " (" << nv << ")\n"; }

    X() { out("X()",0); val=0; }                                    // 默认构造函数
    X(int v) { val=v; out("X(int)",v); }
    X(const X& x) { val=x.val; out("X(X&)",x.val); }                // 拷贝构造函数
    X& operator=(const X& a)                                        // 拷贝赋值
        { out("X::operator=()",a.val); val=a.val; return *this; }
    ~X() { out("~X()",0); }                                         // 析构函数
};
```

我们对这个 X 做的任何事情都会留下可供研究的踪迹。例如：

```
X glob(2);                  // 一个全局变量

X copy(X a) { return a; }

X copy2(X a) { X aa = a; return aa; }

X& ref_to(X& a) { return a; }

X* make(int i) { X a(i); return new X(a); }

struct XX { X a; X b; };

int main()
{
    X loc {4};              // 局部变量
    X loc2 {loc};           // 拷贝构造

    loc = X{5};             // 拷贝赋值
    loc2 = copy(loc);       // 按值调用并返回
    loc2 = copy2(loc);
    X loc3 {6};
    X& r = ref_to(loc);     // 按引用调用并返回
    delete make(7);
    delete make(8);
    vector<X> v(4);         // 默认值
    XX loc4;
    X* p = new X{9};        // 自由存储上的一个 X
    delete p;
    X* pp = new X[5];       // 自由存储上的 X 数组
    delete[] pp;
}
```

试一试，运行它。

试一试

我们是认真的：请务必运行这个例子，并确保你理解其结果。如果能做到这一点，你就已理解了对象的构造与 destruction 的绝大部分知识。

根据编译器的质量，你可能会注意到一些与 copy() 和 copy2() 调用相关的“缺失的拷贝”。我们（人类）能看出这两个函数什么都没做：它们只是把一个值原封不动地从输入拷贝到输出。如果编译器聪明到足以注意到这一点，它就获准消除对拷贝构造函数的调用。换句话说，编译器可以假定拷贝构造函数确实在拷贝、且只做拷贝。有些编译器聪明到足以消除大量无谓的拷贝。但是，编译器并不保证都那么聪明，所以如果你想要可移植的性能，请考虑移动操作（§18.3.4）。

现在考虑：我们为什么要费心搞这个“傻乎乎的类 X”？这有点像音乐家必做的指法练习：练过之后，别的事情——那些真正要紧的事情——就变得容易了。此外，如果你在构造函数和析构函数上遇到了问题，可以在自己的真实类的构造函数中插入这样的打印语句，看看它们是否按预期工作。对更大的程序来说，这种确切的跟踪方式会变得繁琐，但类似的技术依然适用。例如，你可以通过检查构造次数减去析构次数是否为零，来判断是否存在内存泄漏。对那些分配内存或持有指向对象指针的类，忘记定义拷贝构造函数和拷贝赋值是一个常见的——而且很容易避免的——问题来源。

如果你的问题大到用这些简单手段难以应付，那你就已经学到足够多的东西，可以开始使用查找此类问题的专业工具了；它们通常被称为“泄漏检测器”（leak detector）。当然，理想情况是借助能避免泄漏的技术从一开始就不泄漏内存。

18.5　vector 元素的访问

到目前为止（§17.6），我们一直用 set() 和 get() 成员函数来访问元素。这样的用法冗长而丑陋。我们想要惯常的下标记法：v[i]。实现它的方法是定义一个名为 operator[] 的成员函数。下面是我们的第一次（天真的）尝试：

```
class vector {
    int sz;                     // 大小
    double* elem;               // 指向元素的指针
public:
    // ...
    double operator[](int n) { return elem[n]; }    // 返回元素
};
```

这看起来不错，特别是看起来很简单，但遗憾的是它过于简单了。让下标运算符（operator[]()）返回一个值，只能读取元素而不能写入元素：

```
vector v(10);
double x = v[2];            // 没问题
v[3] = x;                   // 错误：v[3] 不是左值
```

这里，v[i] 被解释为调用 v.operator[](i)，而该调用返回 v 的第 i 个元素的值。对这个过于天真的 vector 来说，v[3] 是一个浮点值，而不是一个浮点变量。

试一试

做一个足够完整的版本让程序能编译，看看编译器对 v[3]=x; 给出什么错误信息。

我们的下一次尝试是让 operator[] 返回指向相应元素的指针：

```
public:
    // ...
    double* operator[](int n) { return &elem[n]; }    // 返回指针
};
```

有了这个定义，我们就可以写出

```
vector v(10);
for (int i=0; i<v.size(); ++i) {        // 能用，但仍然太丑
    *v[i] = i;
    cout << *v[i];
}
```

这里，v[i] 被解释为调用 v.operator[](i)，该调用返回一个指向 v 的编号为 i 的元素的指针。问题在于我们必须写 * 来解引用这个指针才能访问到元素。这几乎和必须写 set() 与 get() 一样糟糕。让下标运算符返回一个引用（reference）即可解决这个问题：

```
class vector {
    // ...
    double& operator[](int n) { return elem[n]; }    // 返回引用
};
```

现在我们可以写出

```
vector v(10);
for (int i=0; i<v.size(); ++i) {        // 能用!
    v[i] = i;                           // v[i] 返回元素 i 的一个引用
    cout << v[i];
}
```

我们得到了惯用的记法：v[i] 被解释为调用 v.operator[](i)，它返回对 v 的编号为 i 的元素的引用。

18.5.1 针对 const 的重载

迄今为止定义的 operator[]() 存在一个问题：不能对 const 的 vector 调用它。例如：

```
void f(const vector& cv)
{
    double d = cv[1];       // 错误，但本应没问题
    cv[1] = 2.0;            // 错误（理应如此）
}
```

原因在于我们的 vector::operator[]() 有可能修改 vector。它实际上并不会修改，但编译器并不知道这一点，因为我们“忘记”告诉它了。解决办法是提供一个版本，将其作为 const 成员函数（参见 §9.7.4）。这很容易做到：

```
class vector {
    // ...
    double& operator[](int n);          // 用于非 const 的 vector
    double operator[](int n) const;     // 用于 const 的 vector
};
```

显然，我们不能从 const 版本返回一个 double&，所以我们返回了一个 double 值。我们同样可以返回一个 const double&，但由于 double 是个小对象，返回引用毫无意义（参见 §8.5.6），因此我们决定按值传回。现在我们可以写出

```
void ff(const vector& cv, vector& v)
{
    double d = cv[1];       // 没问题（使用 const 版本的 []）
    cv[1] = 2.0;            // 错误（使用 const 版本的 []）
    double d = v[1];        // 没问题（使用非 const 版本的 []）
    v[1] = 2.0;             // 没问题（使用非 const 版本的 []）
}
```

由于 vector 常常以 const 引用的方式传递，这个 const 版本的 operator[] 是一个不可或缺的补充。

18.6 数组

一段时间以来，我们一直用数组（array）一词指代在自由存储上分配的对象序列。我们也可以在其他地方将数组作为命名变量来分配。实际上，数组很常见：

- 作为全局变量（但全局变量多半是个坏主意）
- 作为局部变量（但数组在这里有严重的局限）
- 作为函数实参（但数组不知道自身的大小）
- 作为类成员（但成员数组可能很难初始化）

现在，你可能已经察觉到，我们对 vector 的偏爱胜过数组，这种倾向毫不掩饰。只要你有得选，就用 std::vector——而且在大多数情况下你都有得选。然而，数组早在 vector 出现之前就存在了，并且大致上等价于其他语言（特别是 C）所提供的功能，因此你必须了解数组，而且要了解透彻，才能应付旧代码，以及那些不认同 vector 优势的人所写的代码。

那么，数组究竟是什么？我们如何定义一个数组？如何使用一个数组？数组是分配在连续内存中的同类型对象序列；也就是说，数组的所有元素都具有相同的类型，并且序列中的对象之间没有空隙。数组的元素从 0 开始编号。在声明中，数组用“方括号”表示：

```
const int max = 100;
int gai[max];                   // 一个全局数组（含 100 个 int）: " 永远存活"

void f(int n)
{
    char lac[20];               // 局部数组: " 存活" 到作用域结束
    int lai[60];
    double lad[n];              // 错误: 数组大小不是常量
    // ...
}
```

注意这个限制：命名数组的元素数目必须在编译时已知。如果希望元素数目是一个变量，就必须把它放到自由存储上，并通过指针来访问。vector 对其元素数组正是这样做的。

就像自由存储上的数组一样，我们使用下标运算符和解引用运算符（[ ] 和 *）来访问命名数组。例如：

```
void f2()
{
    char lac[20];               // 局部数组: " 存活" 到作用域结束

    lac[7] = 'a';
    *lac = 'b';                 // 等价于 lac[0]='b'

    lac[-2] = 'b';              // 咦？
    lac[200] = 'c';             // 咦？
}
```

这个函数能通过编译，但我们知道“能编译”并不意味着“能正确工作”。[ ] 的用法很直白，但没有范围检查，所以 f2() 能通过编译，而向 lac[-2] 和 lac[200] 写入的结果（与所有越界访问一样）通常是灾难性的。别这么做。数组不做范围检查。再说一遍，我们在这里是直接与物理内存打交道；不要指望有“系统支持”。

可是，编译器难道看不出 lac 只有 20 个元素，从而 lac[200] 是一个错误吗？编译器是可以看出来的，但据我们所知，还没有哪个产品级编译器这么做。问题在于，在编译时追踪数组边界通常是不可能的，而只捕获最简单情形（如上例）中的错误又没有多大帮助。

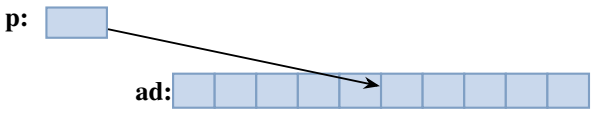
18.6.1 指向数组元素的指针

指针可以指向数组的一个元素。考虑：

```
double ad[10];
double* p = &ad[5];             // 指向 ad[5]
```

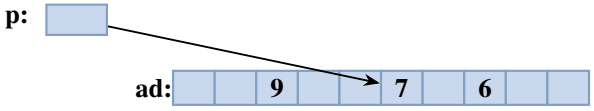
现在我们有了一个指针 p，指向那个被称为 ad[5] 的 double：

我们可以对这个指针进行下标操作和解引用：




```
*p = 7;
p[2] = 6;
p[-3] = 9;
```

我们得到

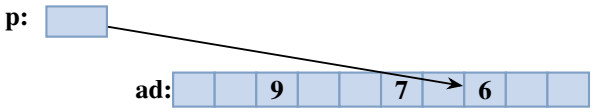


也就是说，我们既可以用正数也可以用负数对指针进行下标操作。只要结果元素仍在范围内，一切安好。然而，访问所指向数组范围之外的元素是非法的（与自由存储上分配的数组一样；参见 §17.4.3）。通常，越界访问不会被编译器检测到，而且（迟早）是灾难性的。

一旦指针指向了数组中的某个元素，就可以用加法和下标操作让它指向该数组的另一个元素。例如：

```
p += 2;                 // 将 p 向右移动 2 个元素
```

我们得到



以及

```
p -= 5;                 // 将 p 向左移动 5 个元素
```

我们得到

用 +、-、+= 和 -= 来移动指针称为指针算术（pointer arithmetic）。显然，如果这样做，我们必须非常小心，确保结果不是指向数组之外内存的指针：

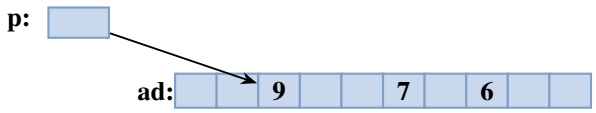
```
p += 1000;              // 疯狂：p 指向一个只有 10 个元素的数组
double d = *p;          // 非法：多半是个坏值
                        //  （必然是一个不可预测的值）
*p = 12.34;             // 非法：多半会破坏某些未知的数据
```

遗憾的是，并非所有涉及指针算术的糟糕缺陷都这么容易发现。最好的策略通常就是干脆避免指针算术。

指针算术最常见的用途，是用 ++ 递增指针使其指向下一个元素，以及用 -- 递减指针使其指向上一个元素。例如，我们可以这样打印 ad 的各个元素的值：

```
for (double* p = &ad[0]; p<&ad[10]; ++p) cout << *p << '\n';
```

或者反向进行：




```
for (double* p = &ad[9]; p>=&ad[0]; --p) cout << *p << '\n';
```

这种指针算术的用法并不罕见。然而，我们发现最后一个（“反向”）例子非常容易写错。为什么是 `&ad[9]` 而不是 `&ad[10]`？为什么用 `>=` 而不是 `>`？这些例子同样可以（而且同样高效地）用下标操作完成。这样的例子也可以通过对 vector 进行下标操作来做到，而且 vector 更容易做范围检查。

请注意，指针算术在现实世界中的大多数用法，都涉及一个作为函数实参传递的指针。在这种情况下，编译器完全不知道所指向的数组有多少个元素：你只能靠自己。这种情况只要能避免，我们就尽量避免。

C++ 为什么会（允许）有指针算术呢？它可能很麻烦，而且一旦有了下标操作，它也提供不了什么新东西。例如：

```
double* p1 = &ad[0];
double* p2 = p1+7;
double* p3 = &p1[7];
if (p2 != p3) cout << "impossible!\n";
```

主要原因是历史性的。这些规则是几十年前为 C 制定的，若不破坏大量代码就无法将其移除。部分原因是，在某些重要的低层应用（如内存管理器）中，使用指针算术能带来一些便利。

18.6.2　指针与数组

数组的名字指代数组的全部元素。考虑：

```
char ch[100];
```

`ch` 的大小 `sizeof(ch)` 是 100。然而，只要有最轻微的借口，数组名就会转化为（“衰变为”，decay）一个指针。例如：

```
char* p = ch;
```

这里 `p` 被初始化为 `&ch[0]`，而 `sizeof(p)` 大约是 4（而不是 100）。

这可能有用。例如，考虑一个统计以零结尾的字符数组中字符数目的函数 `strlen()`：

```
int strlen(const char* p)        // 类似于标准库的 strlen()
{
    int count = 0;
    while (*p) { ++count; ++p; }
    return count;
}
```

现在我们既可以用 `strlen(ch)` 调用它，也可以用 `strlen(&ch[0])` 调用它。你可能会指出，这只是个微不足道的记法上的便利，这一点我们不得不同意。

让数组名转化为指针的一个理由，是避免意外地按值传递大量数据。考虑：

```
int strlen(const char a[])       // 类似于标准库的 strlen()
{
    int count = 0;
    while (a[count]) { ++count; }
    return count;
}

char lots[100000];

void f()
{
    int nchar = strlen(lots);
    // ...
}
```

想当然地（而且也相当合理地），你可能以为这个调用会把作为实参指定的那 100 000 个字符拷贝给 `strlen()`，但实际并非如此。实参声明 `char p[]` 被视为等价于 `char* p`，调用 `strlen(lots)` 被视为等价于 `strlen(&lots[0])`。这为你省去了一次开销高昂的拷贝操作，但它应该让你感到意外。为什么它应该让你感到意外？因为在其他所有情况下，当你传递一个对象而未显式声明实参按引用传递（参见 §8.5.3-6）时，该对象都会被拷贝。

注意，从把数组名当作指向其首元素的指针得到的这个指针是一个值而不是变量，因此你不能对它赋值：

```
char ac[10];
ac = new char[20];               // 错误：不能对数组名赋值
&ac[0] = new char[20];           // 错误：不能对指针值赋值
```

终于有一个编译器能抓住的问题了！

由于这种数组名到指针的隐式转换，你甚至不能通过赋值来拷贝数组：

```
int x[100];
int y[100];
// ...
x = y;                           // 错误
int z[100] = y;                  // 错误
```

这是前后一致的，但常常很麻烦。如果需要拷贝一个数组，就必须编写更繁琐一些的代码。例如：

```
for (int i=0; i<100; ++i) x[i]=y[i];   // 拷贝 100 个 int
memcpy(x,y,100*sizeof(int));           // 拷贝 100*sizeof(int) 字节
copy(y,y+100, x);                      // 拷贝 100 个 int
```

注意，C 语言不支持任何类似 vector 的东西，因此在 C 中必须大量使用数组。这也意味着大量 C++ 代码会使用数组（参见 §27.1.2）。特别是，C 风格字符串（以零结尾的字符数组；参见 §27.5）非常常见。

如果想要赋值，我们就必须使用类似标准库 vector 的东西。与上面拷贝代码等价的 vector 版本是

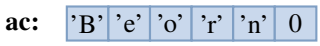
```
vector<int> x(100);
vector<int> y(100);
// ...
x = y;                          // 拷贝 100 个 int
```

18.6.3　数组初始化

字符数组可以用字符串字面值（string literal）来初始化。例如：

```
char ac[] = "Beorn";            // 含 6 个字符的数组
```

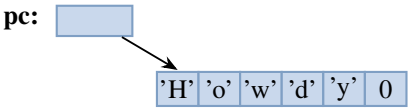
数一数那些字符。有五个，但 ac 变成了一个包含六个字符的数组，因为编译器会在字符串字面值的末尾添加一个终止零字符：



以零结尾的字符串在 C 语言和许多系统中都是常态。我们把这种以零结尾的字符数组称为 C 风格字符串（C-style string）。所有字符串字面值都是 C 风格字符串。例如：

```
char* pc = "Howdy";             // pc 指向一个含 6 个字符的数组
```

图示如下：



注意，数值为 0 的 char 不是字符 '0'，也不是任何其他字母或数字。这个终止零的用途是让函数能够找到字符串的末尾。切记：数组不知道自身的大小。依靠终止零这一约定，我们可以写出

```
int strlen(const char* p)               // 类似于标准库的 strlen()
{
    int n = 0;
    while (p[n]) ++n;
    return n;
}
```

实际上，我们不必自己定义 strlen()，因为它是标准库函数，定义在 <string.h> 头文件中（参见 §27.5、§B.11.3）。注意，strlen() 统计的是字符数，但不包括终止的 0；也就是说，要在 C 风格字符串中存储 n 个字符，你需要 n+1 个字符。

只有字符数组能用字符串字面值初始化，但所有数组都可以用其元素类型的值列表来初始化。例如：

```
int ai[] = { 1, 2, 3, 4, 5, 6 };                // 含 6 个 int 的数组
int ai2[100] = {0,1,2,3,4,5,6,7,8,9};           // 最后 90 个元素被初始化为 0
double ad[100] = { };                           // 所有元素都被初始化为 0.0
char chars[] = {'a', 'b', 'c'};                 // 没有终止 0!
```

注意，ai 的元素数目是六（而不是七），chars 的元素数目是三（而不是四）——“末尾加一个 0”的规则只适用于字符串字面值。如果数组没有给定大小，其大小就从初始化器列表推断得出。这是一项相当有用的特性。如果初始化器的值比数组元素少（如 ai2 和 ad 的定义那样），余下的元素将用元素类型的默认值初始化。

18.6.4 指针的问题

与数组一样，指针也常常被过度使用和滥用。人们陷入的麻烦往往同时涉及指针和数组，所以我们在这里对这些问题做一个小结。特别是，指针的所有严重问题都源于试图访问某个并非预期类型对象的东西，而这些问题中的许多都涉及对数组边界之外的访问。这里我们将考虑：

- 通过空指针访问
- 通过未初始化的指针访问
- 访问数组末端之后的元素
- 访问已释放的对象
- 访问已离开作用域的对象

在所有这些情况下，对程序员来说，实际的问题在于那次访问看上去完全无辜；只不过是指针尚未被赋予一个使其用法有效的值。更糟的是（当通过指针进行写操作时），问题可能要到很久之后才会显现，此时某个表面上毫不相关的对象已被破坏。让我们看一些例子：

不要通过空指针访问：

```
int* p = nullptr;
*p = 7;             // 哎哟!
```

显然，在现实世界的程序中，这种情况通常发生在初始化和使用之间还隔着一些代码的时候。特别是，把 p 传给某个函数、或从某个函数接收它作为结果，都是常见的例子。我们宁愿不传递空指针，但如果不得不传递，请在使用前先检查空指针：

```
int* p = fct_that_can_return_a_nullptr();

if (p == nullptr) {
    // 做点别的
}
else {
    // 使用 p
    *p = 7;
}
```

以及

```
void fct_that_can_receive_a_nullptr(int* p)
{
    if (p == nullptr) {
        // 做点别的
    }
    else {
        // 使用 p
        *p = 7;
    }
}
```

使用引用（参见 §17.9.1）以及用异常来报告错误（参见 §5.6 和 §19.5），是避免空指针的主要手段。

务必初始化你的指针：

```
int* p;
*p = 9;             // 哎哟!
```

尤其不要忘记初始化那些作为类成员的指针。

不要访问不存在的数组元素：

```
int a[10];
int* p = &a[10];
*p = 11;            // 哎哟!
a[10] = 12;         // 哎哟!
```

对循环的第一个和最后一个元素要格外小心，并且尽量不要把数组当作指向其首元素的指针到处传递。改用 vector 吧。如果确实要在多个函数中使用同一个数组（将其作为实参传递），那么要格外小心，并把它的大小一并传过去。

不要通过已被 delete 的指针访问：

```
int* p = new int{7};
// ...
delete p;
// ...
*p = 13;            // 哎哟!
```

`delete p` 或其后的代码可能已经在 `*p` 上胡乱写过东西，或把它用作别的用途了。在所有这些问题中，我们认为这是最难

系统性避免的一个。对付这个问题最有效的防御手段，是不要使用需要“裸”`delete` 的“裸”`new`：在构造函数和析构函数中使用 `new` 和 `delete`，或者使用诸如 `Vector_ref`（参见 §E.4）这样的容器来处理删除。

不要返回指向局部变量的指针：

```
int* f()
{
    int x = 7;
    // ...
    return &x;
}

// ...

int* p = f();
// ...
*p = 15;            // 哎哟!
```

f() 的返回动作或其后的代码可能已经在 *p 上胡乱写过东西，或把它用作别的用途了。原因在于，函数的局部变量在进入函数时（在栈上）分配，在退出函数时再次释放。特别是，对于带析构函数的类的局部对象，会调用其析构函数（参见 §17.5.1）。编译器本可以抓住大多数与返回指向局部变量的指针相关的问题，但很少有编译器这么做。

考虑一个逻辑上等价的例子：

```
vector& ff()
{
    vector x(7);        // 7 个元素
    // ...
    return x;
}                       // vector x 在此被销毁

// ...

vector& p = ff();
// ...
p[4] = 15;              // 哎哟!
```

相当多的编译器能抓住返回问题的这个变体。

程序员常常低估这些问题。然而，许多经验丰富的程序员都曾被这些简单的数组与指针问题数不清的组合变体击败过。解决办法是不要让代码里到处都是指针、数组、new 和 delete。如果这样做，在现实规模的程序中，单靠“小心”是远远不够的。相反，要依靠 vector、RAII（“资源获取即初始化”，Resource Acquisition Is Initialization；参见 §19.5）以及其他系统化的内存与其他资源管理方法。

18.7 示例：回文

技术性的例子看得够多了！我们来试一个小谜题。回文（palindrome）是指从两端拼读都一样的单词。例如，anna、petep 和 malayalam 是回文，而 ida 和 homesick 则不是。判定一个单词是否为回文有两种基本方法：

- 将字母按相反顺序复制一份，再将这份副本与原词比较。

- 检查第一个字母是否与最后一个相同，再检查第二个字母是否与倒数第二个相同，如此继续，直到抵达中间。

这里我们采用第二种方法。根据我们如何表示单词，以及如何记录字符比较进行到了哪一步，把这个思路写成代码有很多种方式。我们将编写一个小程序，用几种不同的方法测试单词是否为回文，只是为了看看不同的语言特性会如何影响代码的样子和行为。

18.7.1 用 string 求回文

首先，我们尝试使用标准库 `string`、用 `int` 索引来记录比较进度的版本：

```
bool is_palindrome(const string& s)
{
    int first = 0;                      // 第一个字母的索引
    int last = s.length()-1;            // 最后一个字母的索引
    while (first < last) {              // 我们尚未抵达中间
        if (s[first]!=s[last]) return false;
        ++first;                        // 向前移动
        --last;                         // 向后移动
    }
    return true;
}
```

如果我们抵达中间时没有发现差异，就返回 `true`。建议你仔细阅读这段代码，确信它在字符串中没有字母、只有一个字母、有偶数个字母和有奇数个字母等情况下都是正确的。当然，我们不应只靠逻辑来确认代码正确，还应当测试。我们可以这样来演练 `is_palindrome()`：

```
int main()
{
    for (string s; cin>>s; ) {
        cout << s << " is";
        if (!is_palindrome(s)) cout << " not";
        cout << " a palindrome\n";
    }
}
```

我们使用 `string` 的基本理由是“string 适合处理单词”。把一个以空白分隔的单词读入 string 很简单，而且 string 知道自身的大小。假如我们想用含有空白的字符串来测试 `is_palindrome()`，可以改用 `getline()` 读取（参见 §11.5）。那样会发现 ah ha 和 as df fd sa 也是回文。

18.7.2 用数组求回文

如果我们没有 string（也没有 vector），不得不用数组来存放字符，那会怎样？让我们看看：

```
bool is_palindrome(const char s[], int n)
        // s 指向含 n 个字符的数组的第一个字符
{
    int first = 0;                      // 第一个字母的索引
    int last = n-1;                     // 最后一个字母的索引
    while (first < last) {              // 我们尚未抵达中间
        if (s[first]!=s[last]) return false;
        ++first;                        // 向前移动
        --last;                         // 向后移动
    }
    return true;
}
```

要演练 `is_palindrome()`，我们首先得把字符读入数组。一种安全的做法（即没有溢出数组的风险）如下：

```
istream& read_word(istream& is, char* buffer, int max)
        // 从 is 向 buffer 最多读入 max-1 个字符
{
    is.width(max);          // 在下一次 >> 中最多读入 max-1 个字符
    is >> buffer;           // 读取以空白结尾的单词，
                            // 并在读入 buffer 的最后一个字符之后添加零
    return is;
}
```

适当地设置 `istream` 的 width 可以防止下一次 >> 操作造成缓冲区溢出。遗憾的是，这也意味着我们无法知道读取是因空白而终止，还是因缓冲区已满而终止（此时我们需要继续读取更多字符）。此外，有谁记得 `width()` 对输入行为的细节？作为输入缓冲区，标准库的 string 和 vector 实在要好得多，因为它们会扩展以适应输入量。终止 0 字符是必需的，因为字符数组（C 风格字符串）上最流行的操作都假定以 0 结尾。使用 `read_word()`，我们可以写出

```
int main()
{
    constexpr int max = 128;
    for (char s[max]; read_word(cin,s,max); ) {
        cout << s << " is";
        if (!is_palindrome(s,strlen(s))) cout << " not";
        cout << " a palindrome\n";
    }
}
```

`strlen(s)` 调用返回 `read_word()` 调用之后数组中的字符数目，而 `cout<<s` 输出数组中直到终止 0 之前的字符。

我们认为这个“数组解法”明显比“string 解法”凌乱得多，而如果我们认真对待长字符串的可能性，它还会糟糕得多。参见习题 10。

18.7.3 用指针求回文

我们可以不用索引来标识字符，而改用指针：

```
bool is_palindrome(const char* first, const char* last)
        // first 指向第一个字母，last 指向最后一个字母
{
    while (first < last) {              // 我们尚未抵达中间
        if (*first!=*last) return false;
        ++first;                        // 向前移动
        --last;                         // 向后移动
    }
    return true;
}
```

注意，我们实际上可以对指针进行递增和递减。递增使指针指向数组的下一个元素，递减使指针指向上一个元素。如果数组没有这样的下一个或上一个元素，你就遇到了一个严重的、未被捕获的越界错误。这是指针的又一个问题。

我们这样调用这个 is_palindrome()：

```
int main()
{
    const int max = 128;
    for (char s[max]; read_word(cin,s,max); ) {
        cout << s << " is";
        if (!is_palindrome(&s[0],&s[strlen(s)-1])) cout << " not";
        cout << " a palindrome\n";
    }
}
```

纯粹为了好玩，我们把 is_palindrome() 重写为这样：

```
bool is_palindrome(const char* first, const char* last)
        // first 指向第一个字母，last 指向最后一个字母
{
    if (first<last) {
        if (*first!=*last) return false;
        return is_palindrome(first+1,last-1);
    }
    return true;
}
```

当我们重新表述回文的定义时，这段代码就变得显而易见了：如果一个单词的第一个字符和最后一个字符相同，并且去掉首尾字符后得到的子串也是回文，那么它就是回文。

小试身手

本章有两个小试身手：一个练习数组，另一个以大致相同的方式练习 vector。请两者都做，并比较各自耗费的工夫。

数组小试身手：

1. 定义一个全局 int 数组 ga，包含十个 int，初始化为 1、2、4、8、16、……。
2. 定义一个函数 f()，接受一个 int 数组实参和一个指明数组元素数目的 int 实参。
3. 在 f() 中：
   (a). 定义一个含十个 int 的局部 int 数组 la。
   (b). 把 ga 中的值拷贝到 la。
   (c). 打印 la 的元素。
   (d). 定义一个指向 int 的指针 p，用一个在自由存储上分配的、元素数目与实参数组相同的数组将其初始化。
   (e). 把实参数组中的值拷贝到自由存储上的数组。
   (f). 打印自由存储上数组的元素。
   (g). 释放自由存储上的数组。
4. 在 main() 中：
   (a). 以 ga 为实参调用 f()。
   (b). 定义一个含十个元素的数组 aa，并用前十个体乘积值（1、2*1、3*2*1、4*3*2*1、……）将其初始化。
   (c). 以 aa 为实参调用 f()。

标准库 vector 小试身手：

1. 定义一个全局 vector<int> gv;，用十个 int 将其初始化：1、2、4、8、16、……。
2. 定义一个函数 f()，接受一个 vector<int> 实参。
3. 在 f() 中：
   (a). 定义一个局部 vector<int> lv，元素数目与实参 vector 相同。
   (b). 把 gv 中的值拷贝到 lv。
   (c). 打印 lv 的元素。
   (d). 定义一个局部 vector<int> lv2；将其初始化为实参 vector 的副本。
   (e). 打印 lv2 的元素。
4. 在 main() 中：
   (a). 以 gv 为实参调用 f()。
   (b). 定义一个 vector<int> vv，并用前十个体乘积值（1、2*1、3*2*1、4*3*2*1、……）将其初始化。
   (c). 以 vv 为实参调用 f()。

要点回顾

1. “Caveat emptor!”（买者自负）是什么意思？
2. 类对象的拷贝的默认含义是什么？
3. 类对象拷贝的默认含义什么时候是合适的？什么时候是不合适的？
4. 什么是拷贝构造函数（copy constructor）？
5. 什么是拷贝赋值（copy assignment）？
6. 拷贝赋值与拷贝初始化有什么区别？
7. 什么是浅拷贝（shallow copy）？什么是深拷贝（deep copy）？
8. vector 的副本与其源相比如何？
9. 一个类的五种“基本操作”是什么？

10. 什么是 `explicit` 构造函数？你会在什么时候更愿意用它而非（默认的）另一种选择？
11. 对一个类对象，哪些操作可能被隐式调用？
12. 什么是数组？
13. 如何拷贝一个数组？
14. 如何初始化一个数组？
15. 什么时候你更愿意用指针实参而非引用实参？为什么？
16. 什么是 C 风格字符串？
17. 什么是回文？

术语

array（数组） deep copy（深拷贝） move assignment（移动赋值）
array initialization（数组初始化） default constructor（默认构造函数） move construction（移动构造）
copy assignment（拷贝赋值） essential operations（基本操作） palindrome（回文）
copy constructor（拷贝构造函数） `explicit` constructor（`explicit` 构造函数） shallow copy（浅拷贝）

习题

1. 编写一个函数 `char* strdup(const char*)`，把一个 C 风格字符串拷贝到它在自由存储上分配的内存中。不要使用任何标准库函数。不要使用下标操作，改用解引用运算符 `*`。
2. 编写一个函数 `char* findx(const char* s, const char* x)`，在 `s` 中查找 C 风格字符串 `x` 的首次出现。不要使用任何标准库函数。不要使用下标操作，改用解引用运算符 `*`。
3. 编写一个函数 `int strcmp(const char* s1, const char* s2)`，比较两个 C 风格字符串。如果 `s1` 按字典序排在 `s2` 之前，返回一个负数；如果 `s1` 等于 `s2`，返回零；如果 `s1` 按字典序排在 `s2` 之后，返回一个正数。不要使用任何标准库函数。不要使用下标操作，改用解引用运算符 `*`。
4. 考虑一下，如果给 `strdup()`、`findx()` 和 `strcmp()` 传入的不是 C 风格字符串的实参，会发生什么。试一试！先弄清楚如何得到一个不指向以零结尾的字符数组的 `char*`，然后使用它（在真实的——非实验性的——代码中千万别这么做；它会造成严重破坏）。分别用自由存储上分配的和栈上分配的“伪 C 风格字符串”试一试。如果结果看上去仍然合理，就关闭调试模式。重新设计并重新实现这三个函数，让它们接受另一个实参，指明实参字符串所允许的最大元素数目。然后用正确的 C 风格字符串和“坏”字符串测试它们。
5. 编写一个函数 `string cat_dot(const string& s1, const string& s2)`，将两个 string 用一个点连接起来。例如，`cat_dot("Niels", "Bohr")` 将返回一个内容为 Niels.Bohr 的 string。
6. 修改上一习题中的 `cat_dot()`，让它接受一个 string 作为第三个实参，用作分隔符（而不是点）。
7. 为前面习题中的各个 `cat_dot()` 编写接受 C 风格字符串作为实参、并返回在自由存储上分配的 C 风格字符串作为结果的版本。实现中不要使用标准库函数或类型。用几个字符串测试这些函数。务必释放（用 `delete`）所有你从自由存储（用 `new`）分配的内存。比较本题与习题 5 和习题 6 所耗费的工夫。
8. 重写 §18.7 中的所有函数，改用先做出字符串的反向副本再进行比较的思路；例如，取”home”，生成”emoh”，然后比较这两个字符串，发现它们不同，于是 home 不是回文。
9. 考虑 §17.4 中的内存布局。编写一个程序，指出静态存储、栈和自由存储在内存中的布局顺序。栈向哪个方向生长：是向上朝更高的地址，还是向下朝更低的地址？在自由存储上的数组中，索引较大的元素分配在较高还是较低的地址？

10. 看看 §18.7.2 中回文问题的“数组解法”。修改它以处理长字符串：（a）当输入字符串过长时进行报告；（b）允许任意长的字符串。评论这两个版本的复杂度。
11. 查阅（例如在网上）跳表（skip list），并实现这种列表。这不是一道容易的习题。
12. 实现游戏“猎杀 Wumpus”的一个版本。“猎杀 Wumpus”（或简称“Wump”）是一个简单的（非图形界面的）电脑游戏，最初由 Gregory Yob 发明。基本设定是：一只相当臭的

怪物住在一个由连通房间组成的黑暗洞穴里。你的任务是弓箭射杀 wumpus。除了 wumpus，洞里还有两种危险：无底坑和巨蝠。如果你走进一个有无底坑的房间，游戏就结束了。如果你走进一个有巨蝠的房间，蝙蝠会把你抓起来，扔进另一个房间。如果你走进 wumpus 所在的房间，或它走进你的房间，它会吃掉你。当你进入一个房间时，你会被告知附近是否有危险：

- “我闻到了 wumpus 的气味”：它在一个相邻的房间里。
- “我感到一阵微风”：相邻的房间里有一个无底坑。
- “我听到了蝙蝠声”：一只巨蝠在一个相邻的房间里。

为方便起见，房间都有编号。每个房间都有隧道与另外三个房间连通。进入一个房间时，你会得到类似这样的提示：“你在 12 号房间；有隧道通向 1、13、4 号房间；移动还是射击？”可能的回答有 `m13`（“移动到 13 号房间”）和 `s13-4-3`（“射一支箭，穿过房间 13、4、3”）。箭的射程是三个房间。游戏开始时，你有五支箭。射击的麻烦在于，它会把 wumpus 惊醒，而它会移动到与它原来所在房间相邻的某个房间——那可能正是你的房间。

本题最棘手的部分很可能是构建洞穴，即挑选哪些房间与哪些房间连通。你可能想用随机数生成器（例如 `std_lib_facilities.h` 中的 `randint()`）来让程序的每次运行使用不同的洞穴，并让蝙蝠和 wumpus 到处移动。提示：务必留一种办法，能输出洞穴状态的调试信息。

后记

标准库的 vector 是用更低层的内存管理设施（如指针和数组）构建的，它的主要作用就是帮助我们避开这些设施的复杂性。无论何时设计一个类，都必须考虑初始化、拷贝和销毁。

19 向量、模板与异常

"Success is never final."（成功永无止境）

——Winston Churchill（温斯顿·丘吉尔）

本章完成最常见也最有用的 STL 容器——vector——的设计与实现。这里我们将展示怎样实现元素个数可以变化的容器，怎样指定以元素类型为参数的容器，以及怎样处理范围错误。和往常一样，所用的技术具有普遍适用性，而不只限于 vector 的实现，甚至也不限于容器的实现。归根到底，我们展示的是怎样安全地处理数量可变、类型多样的数据。此外，我们还会加入一些"现实感"的内容，作为设计上的经验教训。这些技术依赖于模板和异常，因此我们还会展示怎样定义模板，并给出资源管理的基本技术——这些技术正是用好异常的关键。

19.1 问题
19.2 改变大小
19.2.1 表示
19.2.2 reserve 与 capacity
19.2.3 resize
19.2.4 push_back
19.2.5 赋值
19.2.6 目前的 vector
19.3 模板
19.3.1 类型作为模板参数
19.3.2 泛型编程
19.3.3 概念
19.3.4 容器与继承
19.3.5 整数作为模板参数
19.3.6 模板实参推导
19.3.7 泛化 vector
19.4 范围检查与异常
19.4.1 题外话：设计考量
19.4.2 坦白：宏
19.5 资源与异常
19.5.1 潜在的资源管理问题
19.5.2 资源获取即初始化
19.5.3 保证
19.5.4 unique_ptr
19.5.5 通过移动返回
19.5.6 vector 的 RAII

19.1 问题

在第 18 章末尾，我们的 vector 已经达到了这样的程度：我们可以

- 创建由双精度浮点元素（类 `vector` 的对象）构成的 vector，元素个数任意
- 用赋值和初始化来拷贝我们的 vector
- 依靠 vector 在离开作用域时正确地释放其内存
- 用常规的下标记法访问 vector 的元素（赋值号的左侧和右侧都行）

这些都很好、很有用，但要达到我们所期望的精细程度（基于使用标准库 vector 的经验），还需要再解决三个问题：

- 怎样改变 vector 的大小（改变元素个数）?
- 怎样捕获并报告越界的 vector 元素访问？
- 怎样把 vector 的元素类型指定为一个实参？

例如，我们怎样定义 vector，才能使下面这段代码合法：

```
vector<double> vd;              // 元素类型为 double
for (double d; cin>>d; )
    vd.push_back(d);            // 让 vd 增长，以容纳所有元素
```

```
vector<char> vc(100);           // 元素类型为 char
int n;
cin>>n;
vc.resize(n);                   // 让 vc 拥有 n 个元素
```

显然，能有这样的 vector 既好又实用，但从编程的角度看，它为什么重要？对某个为将来使用而收集有用编程技术的人来说，它为什么有意思？因为我们用到了两种灵活性。我们有一个单一的实体 vector，对它我们可以改变两件事：

- 元素的个数
- 元素的类型

这些可变性以相当根本的方式发挥作用。我们总是在收集数据。环顾我的书桌，我看到一摞摞银行对账单、信用卡账单和电话账单。这些东西基本上都是一行行信息组成的列表，而信息的类型各不相同：字母组成的字符串和数值。在我面前放着一部电话，它保存着电话号码和姓名的列表。房间另一头的书柜里，一层又一层地摆满了书。我们的程序也往往是类似的：我们有一些容器，里面装着各种类型的元素。我们有许多种不同的容器（vector 只是其中用途最广的一种），它们装着电话号码、姓名、交易金额和文档之类的东西。基本上，我书桌上和房间里的所有例子都源自某个计算机程序。明显的例外是那部电话：它本身就是一台计算机，当我看着它上面的号码时，我看到的正是一个程序的输出，和我们在写的程序没什么两样。事实上，那些号码很可能就存在一个 `vector<Number>` 里。

显然，并非所有容器的元素个数都相同。能不能凑合着用一个在初始定义时就固定了大小的 vector 呢？也就是说，我们能不能不用 push_back()、resize() 以及类似的操作来写代码？当然可以，但那样会给程序员带来不必要的负担：与固定大小的容器打交道的诀窍是，当元素个数增长到超出初始大小时，把元素搬到一个更大的容器里。例如，我们可以像下面这样，在从不改变 vector 大小的前提下把数据读入一个 vector：

```
// 不使用 push_back，把元素读入一个 vector：
vector<double>* p = new vector<double>(10);
int n = 0;                      // 元素个数
for (double d; cin>>d; ) {
    if (n==p->size()) {
        vector<double>* q = new vector<double>(p->size()*2);
        copy(p->begin(), p->end(), q->begin());
        delete p;
        p = q;
    }
    (*p)[n] = d;
    ++n;
}
```

这并不好看。你能确信我们做对了吗？你怎么能确定？注意我们突然开始使用指针和显式的内存管理。我们所做的，是模仿“贴近机器”时不得不采用的那种编程风格，只使用处理固定大小对象的基本内存管理技术（数组；参见 §18.6）。使用 vector 这类容器的理由之一，就是要做得比这更好；也就是说，我们希望 vector 在内部处理这种大小变化，从而替我们——它的用户——省去麻烦，也省去犯错的机会。换句话说，我们更希望容器能够增长到恰好容纳我们所需要的元素个数。例如：

```
vector<double> vd;
for (double d; cin>>d; ) vd.push_back(d);
```

这种大小变化常见吗？如果不常见，那么改变大小的设施就不过是些小小的便利。然而，这种大小变化非常常见。最明显的例子是从输入中读取个数未知的值。其他例子还有：收集搜索得到的一组结果（我们事先并不知道会有多少个结果），以及从集合中逐个删除元素。因此，问题不是我们是否应当处理容器的大小变化，而是怎样处理。

我们到底为什么要费事改变大小？为什么不“直接分配足够的空间，然后一劳永逸”？那看起来是最简单、最高效的策略。然而，只有在我们能够可靠地分配足够的空间、又不会分配得过多时，它才如此——而我们做不到。尝试这种做法的人，往往要么不得不重写代码（如果他们细致而系统地检查了溢出），要么不得不应付灾难（如果他们对检查漫不经心）。

显然，并非所有 vector 的元素类型都相同。我们需要 double 的 vector、温度读数的 vector、记录（各种记录）的 vector、string 的 vector、操作的 vector、图形用户界面按钮的 vector、形状的 vector、日期的 vector、窗口指针的 vector，等等。可能性无穷无尽。

容器有很多种。这是一点重要的认识，而由于它具有重要的含义，不应当不加思考就接受。为什么不能所有容器都是 vector 呢？如果我们能用一种容器（例如 vector）应付过去，就可以不必操心怎样为它编程，直接把它做成语言的一部分。如果我们能用一种容器应付过去，也就不必费心去学习不同种类的容器，一直用 vector 就行了。

嗯，数据结构是大多数重要应用的关键。有许多又厚又有用的书讲怎样组织数据，其中很多内容都可以描述为对“我怎样才能最好地存储数据？”这一问题的回答。所以答案是：我们需要许多不同种类的容器，但这个题目太大，在这里无法充分讨论。不过，我们已经大量使用了 vector 和 string（string 是字符的容器）。在接下来的几章里，我们将会看到 list、map（map 是值对的树）和矩阵。正因为我们需要许多不同的容器，用来构建和使用容器的语言特性与编程技术才具有广泛的用途。事实上，我们用来存储和访问数据的技术，是所有非平凡的计算机应用中最基本也最有用的技术之一。

在最基本的内存层面上，所有对象的大小都是固定的，也不存在类型。我们在这里要做的，是引入一些语言设施和编程技术，使我们能够提供由各种类型的对象构成的容器，并且可以改变其元素个数。这就给我们带来了一种从根本上就有用的灵活性和便利。

19.2　改变大小

标准库 vector 提供了哪些改变大小的设施？它提供三个简单的操作。给定

```
vector<double> v(n);            // v.size()==n
```

我们可以用三种方式改变它的大小：

```
v.resize(10);                   // v 现在有 10 个元素
v.push_back(7);                 // 在 v 末尾添加一个值为 7 的元素
                                // v.size() 增加 1
v = v2;                         // 赋值另一个 vector；v 现在是 v2 的副本
                                // v.size() 现在等于 v2.size()
```

标准库 vector 还提供了更多可以改变 vector 大小的操作，例如 erase() 和 insert()（§B.4.7），但这里我们只看看怎样为我们的 vector 实现上面这三个操作。

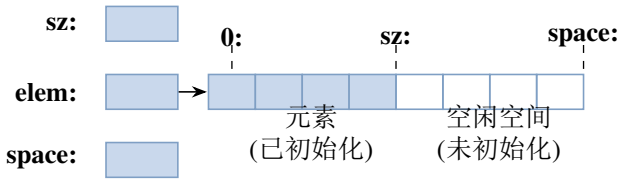
19.2.1　表示

在 §19.1 中，我们展示了改变大小的最简单策略：为新元素个数分配空间，并把旧元素拷贝到新空间中。但是，如果频繁改变大小，这样做效率很低。实践中，一旦我们改变一次大小，通常就会改变很多次。特别是，我们很少只见到一次 push_back()。

因此，我们可以通过预先考虑这种大小变化来优化程序。事实上，所有 vector 实现都同时记录元素个数，以及为“将来扩展”预留的“空闲空间”数量。例如：

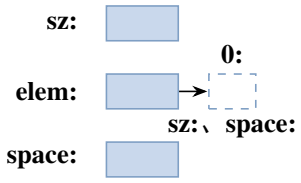
```
class vector {
    int sz;                    // 元素个数
    double* elem;              // 第一个元素的地址
    int space;                 // 元素个数加上“空闲空间”/“槽位”
                               //  (即“当前分配”中供新元素使用的部分)
public:
    // . . .
};
```

我们可以用图形这样表示：



由于我们从 0 开始计数元素，因此我们把 sz（元素个数）表示为指向最后一个元素之后的那一格，把 space 表示为指向最后一个已分配槽位之后的那一格。图中显示的指针实际上就是 elem+sz 和 elem+space。

当 vector 刚被构造出来时，space==sz，也就是说，没有“空闲空间”：

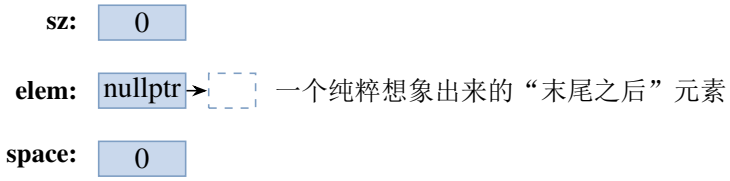


我们只有在开始改变元素个数之后，才会分配额外的槽位。通常 space==sz，所以除非使用 push_back()，否则没有内存开销。

默认构造函数（创建一个没有元素的 vector）把整数成员置为 0，把指针成员置为 nullptr：

```
vector::vector() :sz{0}, elem{nullptr}, space{0} { }
```

这就得到：



那个“末尾之后”的元素完全是想象出来的。默认构造函数不进行任何自由存储分配，占用最小的存储空间（但请参见习题 16）。

请注意，我们的 vector 展示的是可以用来实现标准 vector（以及其他数据结构）的技术，但标准库实现享有相当大的自由度，因此你系统上的 `std::vector` 可能采用不同的技术。

19.2.2 reserve 与 capacity

改变大小（也就是改变元素个数）时最基本的操作是 `vector::reserve()`。我们用它为新元素增加空间：

```
void vector::reserve(int newalloc)
{
    if (newalloc<=space) return;                // 决不减少分配
    double* p = new double[newalloc];           // 分配新空间
    for (int i=0; i<sz; ++i) p[i] = elem[i];    // 拷贝旧元素
    delete[] elem;                              // 释放旧空间
    elem = p;
    space = newalloc;
}
```

注意，我们不初始化预留空间中的元素。毕竟我们只是预留空间；把这些空间用于元素，是 push_back() 和 resize() 的工作。

显然，vector 中可用的空闲空间数量对用户可能很有意义，所以我们（和标准库一样）提供一个成员函数来获取这一信息：

```
int vector::capacity() const { return space; }
```

也就是说，对于名为 v 的 vector，v.capacity()-v.size() 就是我们不必引起重新分配就能 push_back() 到 v 的元素个数。

19.2.3 resize

有了 reserve()，为我们的 vector 实现 resize() 就相当简单了。我们必须处理几种情况：

- 新大小大于旧的分配量。
- 新大小大于旧的大小，但小于或等于旧的分配量。
- 新大小等于旧的大小。
- 新大小小于旧的大小。

来看看我们得到了什么：

```
void vector::resize(int newsize)
    // 让 vector 拥有 newsize 个元素
    // 把每个新元素初始化为默认值 0.0
{
    reserve(newsize);
    for (int i=sz; i<newsize; ++i) elem[i] = 0;    // 初始化新元素
    sz = newsize;
}
```

我们让 reserve() 去做处理内存这件难事。循环负责初始化新元素（如果有的话）。

这里我们并没有显式地处理任何情况，但你可以验证，所有情况还是都得到了正确处理。

试一试

要说服自己这个 resize() 是正确的，我们需要考虑（并测试）哪些情况？newsize == 0 呢？newsize == -77 呢？

19.2.4 push_back

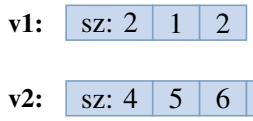
乍一想，push_back() 实现起来可能显得复杂，但有 reserve() 在手，它其实相当简单：

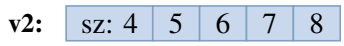
```
void vector::push_back(double d)
    // 把 vector 的大小增加 1；用 d 初始化新元素
{
    if (space==0)
        reserve(8);                 // 一开始分配 8 个元素的空间
    else if (sz==space)
        reserve(2*space);           // 获取更多空间
    elem[sz] = d;                   // 在末尾添加 d
    ++sz;                           // 增加大小（sz 是元素个数）
}
```

换句话说，如果没有空闲空间，我们就把分配量加倍。实践表明，对绝大多数 vector 用途来说，这是一个非常好的选择，而且这也是标准库 vector 大多数实现所采用的策略。

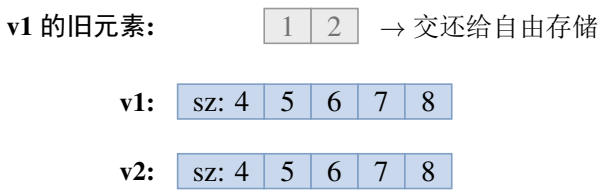
19.2.5 赋值

我们本可以用好几种不同的方式定义 vector 的赋值。例如，我们本可以决定：只有当参与赋值的两个 vector 元素个数相同时，赋值才合法。然而，在 §18.3.2 中我们决定，vector 的赋值应当具有最一般、也可以说是最显然的含义：执行 v1=v2 之后，vector v1 是 v2 的副本。考虑：





显然，我们需要拷贝元素，但空闲空间怎么办？我们要把末尾的“空闲空间”也“拷贝”过去吗？不：新的 vector 会得到元素的一份副本，但既然我们不知道这个新 vector 将如何使用，我们就不必费心在末尾留出额外空间：



这件事最简单的实现是：

- 为副本分配内存。
- 拷贝元素。
- 删除旧的分配。
- 把 sz、elem 和 space 设为新值。

就像这样：

```
vector& vector::operator=(const vector& a)
    // 类似拷贝构造函数，但必须处理旧元素
{
    double* p = new double[a.sz];                   // 分配新空间
    for (int i = 0; i<a.sz; ++i) p[i] = a.elem[i];  // 拷贝元素
    delete[] elem;                                  // 释放旧空间
    space = sz = a.sz;                              // 设置新的大小
    elem = p;                                       // 设置新元素
    return *this;                                   // 返回自引用
}
```

按照惯例，赋值运算符返回被赋值对象的一个引用。其记法是 *this，§17.10 将对此作出解释。

这个实现是正确的，但稍加审视就会发现，我们做了大量多余的分配和释放。如果我们赋值的目标 vector 比源 vector 元素更多呢？如果目标 vector 与源 vector 元素个数相同呢？在许多应用中，后一种情况非常常见。无论哪种情况，我们都可以直接把元素拷贝到目标 vector 已有的空间中：

```
vector& vector::operator=(const vector& a)
{
    if (this==&a) return *this;     // 自赋值，无需任何工作
    if (a.sz<=space) {              // 空间足够，无需重新分配
        for (int i = 0; i<a.sz; ++i) elem[i] = a.elem[i];    // 拷贝元素
        sz = a.sz;
        return *this;
    }
    double* p = new double[a.sz];                           // 分配新空间
    for (int i = 0; i<a.sz; ++i) p[i] = a.elem[i];           // 拷贝元素
    delete[] elem;                                          // 释放旧空间
    space = sz = a.sz;                                      // 设置新的大小
    elem = p;                                               // 设置新元素
    return *this;                                           // 返回自引用
}
```

这里我们先检测自赋值（例如 v=v）；这种情况下我们什么也不做。这个检测在逻辑上是多余的，但有时是一项显著的优化。不过，它确实展示了 this 指针的一种常见用法：检查实参 a 是否就是调用

成员函数（这里是 operator=()）的那个对象。请说服自己：如果去掉 this==&a 这一行，这段代码实际上仍然能工作。a.sz<=space 同样只是一项优化。请说服自己：如果去掉 a.sz<=space 这一分支，这段代码实际上仍然能工作。

19.2.6　目前的 vector

现在我们已经有了一个几乎真正的 double 的 vector：

```
// 一个几乎真正的 double 的 vector:
class vector {
/*
    不变式 (invariant):
    如果 0<=n<sz, elem[n] 是第 n 个元素
    sz<=space;
    如果 sz<space, 则在 elem[sz-1] 之后还有 (space-sz) 个 double 的空间
*/
    int sz;                 // 大小
    double* elem;           // 指向元素的指针 (或 0)
    int space;              // 元素个数加上空闲槽位的个数
public:
    vector() : sz{0}, elem{nullptr}, space{0} { }
    explicit vector(int s) :sz{s}, elem{new double[s]}, space{s}
    {
        for (int i=0; i<sz; ++i) elem[i]=0;      // 元素被初始化
    }
    vector(const vector&);                      // 拷贝构造函数
    vector& operator=(const vector&);           // 拷贝赋值
    vector(vector&&);                           // 移动构造函数
    vector& operator=(vector&&);                // 移动赋值


    ~vector() { delete[] elem; }                          // 析构函数
    double& operator[](int n) { return elem[n]; }         // 访问: 返回引用
    const double& operator[](int n) const { return elem[n]; }
    int size() const { return sz; }
    int capacity() const { return space; }
    void resize(int newsize);                             // 增长
    void push_back(double d);
    void reserve(int newalloc);
};
```

注意，它具备那些必不可少的操作（§18.4）：构造函数、默认构造函数、拷贝操作和析构函数。它还有访问数据的操作（下标 []），有提供数据信息的操作（size() 与 capacity()），以及控制增长的操作（resize()、push_back() 与 reserve()）。

19.3　模板

但我们并不只想要 double 的 vector，我们希望能自由指定 vector 的元素类型。例如：

```
vector<double>
vector<int>
vector<Month>
vector<Window*>                 // 指向 Window 的指针组成的 vector
vector<vector<Record>>          // Record 的 vector 组成的 vector
vector<char>
```

要做到这一点，我们必须弄清如何定义模板。从第一天起我们就一直在使用模板，但直到现在才第一次需要自己定义一个。标准库提供了我们迄今所需要的一切，不过我们不能相信魔法，因此需要考察标准库的设计者与实现者是怎样提供 vector 类型和 `sort()` 函数（§21.1、§B.5.4）这类设施的。这并不只是理论上的兴趣，因为——一如往常——标准库所用的工具与技术，也正是我们自己写代码时最有用的那些。例如，第 21、22 章会展示如何用模板实现标准库的容器和算法，第 24 章会展示如何为科学计算设计矩阵。

基本上，模板是一种机制，它允许程序员把类型用作类或函数的参数。当我们随后提供具体的类型作为实参时，编译器就会生成一个特定的类或函数。

19.3.1　类型作为模板参数

我们想让元素类型成为 vector 的参数。于是我们拿过自己的 vector，把 double 换成 T，这里的 T 是一个参数，可以取 double、int、string、vector<Record>、Window* 这类“值”。引入类型参数 T 的 C++ 记法是前缀 `template<typename T>`，意思是“对于所有类型 T”。例如：

```
// 一个几乎真实的 T 的 vector:
template<typename T>
class vector {                  // 读作“对于所有类型 T”(就像数学里那样)
    int sz;                     // 元素个数
    T* elem;                    // 指向元素的指针
    int space;                  // 元素个数 + 空闲空间
public:
    vector() : sz{0}, elem{nullptr}, space{0} { }
    explicit vector(int s) :sz{s}, elem{new T[s]}, space{s}
    {
        for (int i=0; i<sz; ++i) elem[i]=0;          // 元素被初始化
    }

    vector(const vector&);                           // 拷贝构造函数
    vector& operator=(const vector&);                // 拷贝赋值

    vector(vector&&);                                // 移动构造函数
    vector& operator=(vector&&);                     // 移动赋值
```

```
    ~vector() { delete[] elem; }                        // 析构函数

    T& operator[](int n) { return elem[n]; }            // 访问：返回引用
    const T& operator[](int n) const { return elem[n]; }

    int size() const { return sz; }                     // 当前元素个数
    int capacity() const { return space; }

    void resize(int newsize);                           // 增长
    void push_back(const T& d);
    void reserve(int newalloc);
};
```

这不过就是 §19.2.6 中那个 double 的 vector，只是把 double 换成了模板参数 T。我们可以像这样使用这个类模板 vector：

```
vector<double> vd;              // T 是 double
vector<int> vi;                 // T 是 int
vector<double*> vpd;            // T 是 double*
vector<vector<int>> vvi;        // T 是 vector<int>，其中 T 是 int
```

关于编译器在我们使用模板时做了什么，一种理解方式是：它用实际的类型（即模板实参）替换模板参数，生成对应的类。例如，当编译器在代码中看到 `vector<char>` 时，它会在某处生成类似下面这样的东西：

```
class vector_char {
    int sz;                         // 元素个数
    char* elem;                     // 指向元素的指针
    int space;                      // 元素个数 + 空闲空间
public:
    vector() : sz{0}, elem{nullptr}, space{0} { }
    explicit vector_char(int s) :sz{s}, elem{new char[s]}, space{s}
    {
        for (int i=0; i<sz; ++i) elem[i]=0;             // 元素被初始化
    }

    vector_char(const vector_char&);                    // 拷贝构造函数
    vector_char& operator=(const vector_char&);         // 拷贝赋值

    vector_char(vector_char&&);                         // 移动构造函数
    vector_char& operator=(vector_char&&);              // 移动赋值

    ~vector_char();                                     // 析构函数

    char& operator[](int n) { return elem[n]; }         // 访问：返回引用
    const char& operator[](int n) const { return elem[n]; }
```

```
    int size() const;                                   // 当前元素个数
    int capacity() const;

    void resize(int newsize);                           // 增长
    void push_back(const char& d);
    void reserve(int newalloc);
};
```

对于 vector<double>，编译器生成的大致就是 §19.2.6 中那个（double 的）vector（内部使用一个合适的名字来表示 vector<double>）。

有时我们把类模板称为类型生成器（type generator）。给定模板实参、从类模板生成类型（类）的过程叫做特化（specialization）或模板实例化（template instantiation）。例如，vector<char> 和 vector<Poly_line*> 被称为 vector 的特化。在像我们的 vector 这样简单的情形下，实例化是相当简单的过程；而在最一般、最高级的情形下，模板实例化会复杂得可怕。对模板的用户来说幸运的是，这种复杂性属于编译器作者的地盘，而不属于模板用户。模板实例化（生成模板特化）发生在编译期或链接期，而不是运行期。

自然，我们可以使用这种类模板的成员函数。例如：

```
void fct(vector<string>& v)
{
    int n = v.size();
    v.push_back("Norah");
    // . . .
}
```

当这种类模板的成员函数被使用时，编译器会生成相应的函数。例如，当编译器看到 v.push_back("Norah") 时，它会生成一个函数

```
void vector<string>::push_back(const string& d) { /* . . . */ }
```

它来自模板定义

```
template<typename T> void vector<T>::push_back(const T& d) { /* . . . */ };
```

这样，就有一个函数可供 v.push_back("Norah") 调用了。换句话说，当你需要针对给定的对象类型和实参类型的函数时，编译器会依据模板替你写出来。

除了写 template<typename T>，你也可以写 template<class T>。这两种写法意思完全相同，但有些人偏爱 typename，“因为它更清楚”，“因为不会有人被 typename 弄糊涂，以为不能把 int 这样的内置类型用作模板实参”。我们认为 class 本来就有类型的意思，所以两者没有区别；何况 class 还更短。

19.3.2　泛型编程

模板是 C++ 中泛型编程的基础。事实上，C++ 中“泛型编程”最简单的定义就是“使用模板”。不过这个定义有点过于简单了。我们不应该用编程语言的特性来定义基本的编程概念。编程语言特性存在的目的是支持编程技术，而不是相反。和大多数流行的说法一样，“泛型编程”有许多种定义。我们认为最有用、也最简单的定义是

泛型编程：编写能处理以实参形式提供的多种类型的代码，只要这些实参类型满足特定的语法与语义要求。

例如，vector 的元素必须是可拷贝的类型（通过拷贝构造和拷贝赋值）；在第 20、21 章我们还会看到要求对其实参进行算术运算的模板。当我们参数化的是一个类时，得到的是类模板，通常称为参数化类型（parameterized type）或参数化类；当我们参数化的是一个函数时，得到的是函数模板，通常称为参数化函数，有时也叫算法（algorithm）。因此，泛型编程有时被称为“面向算法的编程”：设计的焦点更多在算法上，而不是算法所用的数据类型上。

既然参数化类型的概念在编程中如此核心，我们不妨再考察一下这套多少令人眼花缭乱的术语。这样，当我们在别的场合遇到这些概念时，就不至于太糊涂。

这种依赖显式模板参数的泛型编程形式，常被称为参数化多态（parametric polymorphism）。与之相对，通过类层次和虚函数得到的多态被称为特定多态（ad hoc polymorphism），那种编程风格称为面向对象编程（§14.3–4）。两种风格都叫多态，是因为它们都要求程序员用单一接口呈现一个概念的多个版本。polymorphism 在希腊语中意为“多种形状”，指的是你可以通过一个公共接口操纵的多种不同类型。在第 16–19 章的 Shape 例子里，我们正是通过 Shape 定义的接口访问了许多形状（如 Text、Circle 和 Polygon）；当我们使用 vector 时，我们通过 vector 模板定义的接口使用了许多种 vector（如 `vector<int>`、`vector<double>` 和 `vector<Shape*>`）。

面向对象编程（使用类层次和虚函数）与泛型编程（使用模板）之间有几处差别。最明显的一点是：使用泛型编程时，调用哪个函数由编译器在编译期确定；而面向对象编程要到运行期才确定。例如：

```
v.push_back(x);      // 把 x 放入 vector v
s.draw();            // 画出形状 s
```

对 `v.push_back(x)`，编译器会确定 v 的元素类型，并调用相应的 `push_back()`；而对 `s.draw()`，编译器会间接地调用某个 `draw()` 函数（通过 s 的 vtbl，见 §14.3.1）。这使面向对象编程获得了泛型编程所缺少的一点自由度，却也让普通的泛型编程更规整、更易理解、性能更好（这正是“特定”与“参数化”这两个标签的由来）。

总结一下：

- 泛型编程：由模板支持，依赖编译期解析
- 面向对象编程：由类层次和虚函数支持，依赖运行期解析

两者结合也是可能的，而且很有用。例如：

```
void draw_all(vector<Shape*>& v)
{
    for (int i = 0; i<v.size(); ++i) v[i]->draw();
}
```

这里我们通过基类（Shape）调用了一个虚函数（draw()）——这当然是面向对象编程。不过我们也把 Shape* 放在一个 vector 里，而 vector 是参数化类型，所以我们同时也用了（简单的）泛型编程。

那么——假定你现在已经受够了哲学——人们实际上用模板做什么？为了无可比拟的灵活性与性能：

- 在对性能要求关键的地方使用模板（例如数值计算和硬实时；见第 24、25 章）。
- 在需要灵活地组合来自多种类型的信息的地方使用模板（例如 C++ 标准库；见第 20–21 章）。

19.3.3 概念

模板有许多有用的性质，比如极大的灵活性和近乎最优的性能，但遗憾的是它们并不完美。一如往常，好处总伴随着相应的弱点。对模板而言，主要问题在于：这种灵活性和性能是以模板的“内部”（其定义）与它的接口（其声明）之间分离不佳为代价的。这表现为糟糕的错误诊断——往往是极其糟糕的错误信息；有时这些错误信息出现在编译过程中远晚于我们所希望的位置。

编译模板的使用时，编译器会“查看”模板内部，也会查看模板实参。它这样做是为了获得生成最优代码所需的信息。为了让所有这些信息都可用，目前的编译器往往要求：模板在被使用的任何地方都必须完整定义。这包括它的所有成员函数，以及从这些成员函数中调用的所有模板函数。因此，模板的作者往往把模板定义放在头文件中。标准其实并不要求这样做，但在大幅改进的实现广泛可用之前，我们建议你对自己的模板也这样做：把要在多个翻译单元中使用的任何模板的定义放在头文件中。

最初，你自己只写非常简单的模板，然后谨慎地积累经验。一种有用的开发技术是像我们对 vector 所做的那样：先用具体的类型开发和测试一个类；一旦它能工作，就把具体类型替换为模板参数，并用多种模板实参进行测试。为了通用性、类型安全和性能，应使用基于模板的库，例如 C++ 标准库。第 20、21 章专门讲标准库的容器和算法，会给你提供使用模板的例子。

C++14 提供了一种机制，可以大幅改进对模板接口的检查。例如，在 C++11 中我们这样写

```
template<typename T>        // 对于所有类型 T
class vector {
    // . . .
};
```

我们无法精确陈述对一个实参类型 T 有什么要求。标准说明了这些要求是什么，但只是用英语说明，而不是用编译器能理解的代码。我们把对模板实参的一组要求称为概念（concept）。模板实参必须满足它所应用的那个模板的要求，也就是该模板的概念。例如，vector 要求其元素可以被拷贝或移动、可以取地址，并且在需要时可以默认构造。换句话说，元素必须满足一组要求，我们可以把这一组要求称为 Element。在 C++14 中，我们可以把这一点明确写出来：

```
template<typename T>            // 对于所有类型 T
    requires Element<T>()       // 使得 T 是一个 Element
class vector {
    // . . .
};
```

这表明，概念实际上是一个类型谓词（type predicate），即一个在编译期求值（constexpr）的函数：如果类型实参（这里是 T）具备该概念（这里是 Element）所要求的性质，它就返回 true，否则返回 false。这样写有点啰嗦，而一种简写记法把我们带到

```
template<Element T>      // 对于所有类型 T，使得 Element<T>() 为真
class vector {
    // . . .
};
```

如果我们没有支持概念的 C++14 编译器，可以用命名和注释来表明自己的要求：

```
template<typename Elem>      // 要求 Element<Elem>()
class vector {
    // . . .
};
```

编译器既不懂我们的命名，也不读我们的注释，但把概念明确写出来有助于我们思考代码、改进泛型代码的设计，也有助于别的程序员理解我们的代码。一路走来，我们会用到一些常见而有用的概念：

- `Element<E>()`：E 可以作为容器中的元素。
- `Container<C>()`：C 可以持有 Element，并能作为一个 [begin():end()) 序列被访问。
- `Forward_iterator<For>()`：For 可用于遍历序列 [b:e)（如链表、vector 或数组）。
- `Input_iterator<In>()`：In 可用于只读一遍序列 [b:e)（如输入流）。
- `Output_iterator<Out>()`：可以用 Out 输出一个序列。
- `Random_access_iterator<Ran>()`：Ran 可用于反复读写序列 [b:e)，并支持用 [] 下标访问。
- `Allocator<A>()`：A 可用于获取和释放内存（如自由存储）。
- `Equal_comparable<T>()`：我们可以用 == 比较两个 T 是否相等，得到布尔结果。
- `Equal_comparable<T,U>()`：我们可以用 == 比较一个 T 与一个 U 是否相等，得到布尔结果。
- `Predicate<P,T>()`：我们可以用类型为 T 的实参调用 P，得到布尔结果。
- `Binary_predicate<P,T>()`：我们可以用两个类型为 T 的实参调用 P，得到布尔结果。
- `Binary_predicate<P,T,U>()`：我们可以用类型分别为 T 和 U 的实参调用 P，得到布尔结果。
- `Less_comparable<L,T>()`：我们可以用 L 和 < 比较两个 T 的大小，得到布尔结果。
- `Less_comparable<L,T,U>()`：我们可以用 L 和 < 比较一个 T 与一个 U 的大小，得到布尔结果。
- `Binary_operation<B,T,U>()`：我们可以用 B 对两个 T 做运算。
- `Binary_operation<B,T,U>()`：我们可以用 B 对一个 T 和一个 U 做运算。
- `Number<N>()`：N 的行为像数，支持 +、-、* 和 /。

对标准库的容器和算法来说，这些概念（以及更多概念）都有极其详尽的规定。在本书中，尤其是在第 20、21 章，我们会非正式地用它们来记录我们的容器和算法。

容器类型和迭代器类型 T 都有一个值类型（写作 `Value_type<T>`），也就是元素类型。通常，这个 `Value_type<T>` 就是成员类型 `T::value_type`；参见 vector 和 list（§20.5）。

19.3.4　容器与继承

有一类把面向对象编程与泛型编程结合起来的尝试，人们总想去做，但它行不通：试图把“派生类对象的容器”当作“基类对象的容器”来用。例如：

```
vector<Shape> vs;
vector<Circle> vc;
vs = vc;                        // 错误：这里要求 vector<Shape>
void f(vector<Shape>&);
f(vc);                          // 错误：这里要求 vector<Shape>
```

但为什么不行？你也许会说，毕竟我可以把一个 Circle 转换成一个 Shape 啊！其实不行。你可以把 `Circle*` 转换成 `Shape*`，把 `Circle&` 转换成 `Shape&`，但我们有意禁止了 Shape 的赋值，免得你去琢磨“把一个带半径的 Circle 放进一个没有半径的 Shape 变量里会发生什么”（§14.2.4）。如果真的允许那样做，发生的就是所谓的“切片”（slicing），它是类对象版的整数截断（§3.9.2）。

于是我们改用指针再试一次：

```
vector<Shape*> vps;
vector<Circle*> vpc;
vps = vpc;                      // 错误：这里要求 vector<Shape*>
void f(vector<Shape*>&);
f(vpc);                         // 错误：这里要求 vector<Shape*>
```

类型系统又一次拒绝了；为什么？想一想 `f()` 可能做什么：

```
void f(vector<Shape*>& v)
{
    v.push_back(new Rectangle{Point{0,0},Point{100,100}});
}
```

显然，我们可以把一个 `Rectangle*` 放进 `vector<Shape*>` 里。可是，如果那个 `vector<Shape*>` 在别处被当作 `vector<Circle*>`，就会有人大吃一惊。特别是，假如编译器接受了上面的例子，那么 vpc 里怎么会出现一个 `Rectangle*` 呢？继承是一种强大而微妙的机制，模板并不会隐含地扩展它的作用范围。有一些用模板表达继承的办法，但超出了本书的范围。只要记住：对任意的模板 C，“D 是 B”并不意味着“C<D> 是 C<B>”——我们应当珍视这一点，因为它保护我们免遭意外的类型违规。另见 §25.4.4。

19.3.5　整数作为模板参数

显然，用类型来参数化类是有用的。那么用“别的东西”，比如整数值和字符串值来参数化类，又如何呢？基本上，任何种类的实参都可能有用，但我们只讨论类型参数和整数参数。其他种类的参数不那么常用，而且 C++ 对它们的支持方式要求使用者对语言特性有相当细致的了解。

考虑整数作为模板实参最常见的用法：一个元素个数在编译期就已知的容器。

```
template<typename T, int N> struct array {
    T elem[N];                  // 用成员数组保存元素

    // 依靠默认构造函数、析构函数和赋值运算
    T& operator[](int n);                   // 访问：返回引用
    const T& operator[](int n) const;

    T* data() { return elem; }              // 转换为 T*
    const T* data() const { return elem; }

    int size() const { return N; }
};
```

我们可以像这样使用 array（另见 §20.7）：

```
array<int,256> gb;                      // 256 个整数
array<double,6> ad = { 0.0, 1.1, 2.2, 3.3, 4.4, 5.5 };
const int max = 1024;


void some_fct(int n)
{
    array<char,max> loc;
    array<char,n> oops;         // 错误：编译器不知道 n 的值
    // . . .
    array<char,max> loc2 = loc; // 制作一份备份副本
    // . . .
    loc = loc2;                 // 恢复
    // . . .
}
```

显然，array 非常简单——比 vector 简单得多，也不如 vector 强大——那为什么还有人想用 array 而不用 vector 呢？一个答案是“效率”。我们在编译期就知道 array 的大小，因此编译器可以分配静态内存（用于 gb 这样的全局对象）和栈内存（用于 loc 这样的局部对象），而不必使用自由存储。当我们做范围检查时，检查可以与常量（大小参数 N）比较。对大多数程序来说，效率上的改善微不足道；但如果你在编写网络驱动这类关键的系统组件，即使很小的差别也可能有影响。更重要的是，有些程序根本不允许使用自由存储。这类程序通常是嵌入式系统程序和/或安全关键程序（见第 25 章）。在这样的程序里，array 给了我们 vector 的许多优点，又不违反那条关键限制（不使用自由存储）。

让我们问一个相反的问题：不是“为什么不能干脆用 vector?”，而是“为什么不干脆用内置数组?”。正如我们在 §18.6 中看到的，数组的脾气可能很坏：它们不知道自己的大小，一有机会就转换成指针，也不能正确地拷贝；与 vector 一样，array 没有这些问题。例如：

```
double* p = ad;                     // 错误：不能隐式转换为指针
double* q = ad.data();              // 正确：显式转换
```

```
template<typename C> void printout(const C& c)        // 函数模板
{
    for (int i = 0; i<c.size(); ++i) cout << c[i] <<'\n';
}
```

既可以用 array 来调用这个 `printout()`，也可以用 vector 来调用：

```
printout(ad);                         // 用 array 调用
vector<int> vi;
// ...
printout(vi);                         // 用 vector 调用
```

这是应用于数据访问的泛型编程（generic programming）的一个简单例子。它能工作，是因为 array 和 vector 所用的接口（`size()` 和下标访问）是相同的。第 20 章和第 21 章将更详细地探讨这种编程风格。

19.3.6　模板实参推导

对于类模板，我们在创建某个具体类的对象时指定模板实参（template argument）。例如：

```
array<char,1024> buf;           // 对 buf 而言，T 是 char，N 是 1024
array<double,10> b2;            // 对 b2 而言，T 是 double，N 是 10
```

对于函数模板，编译器通常能从函数实参推导出模板实参。例如：

```
template<class T, int N> void fill(array<T,N>& b, const T& val)
{
    for (int i = 0; i<N; ++i) b[i] = val;
}


void f()
{
    fill(buf,'x');              // 对 fill() 而言，T 是 char，N 是 1024
                                // 因为 buf 就是如此
    fill(b2,0.0);               // 对 fill() 而言，T 是 double，N 是 10
                                // 因为 b2 就是如此
}
```

严格说来，`fill(buf,'x')` 是 `fill<char,1024>(buf,'x')` 的简写，`fill(b2,0)` 是 `fill<double,10>(b2,0)` 的简写，但幸运的是，我们通常不必写得这么具体——编译器会替我们搞定。

19.3.7　泛化 vector

当我们将 vector 从一个“double 的 vector”类泛化为“T 的 vector”模板时，我们并没有重新审视 `push_back()`、`resize()` 和 `reserve()` 的定义。现在必须回过头来做这件事，因为按照它们在 §19.2.2 和 §19.2.3 中的定义，它们所做出的假设对 double 成立，但并非对我们想用作 vector 元素类型的所有类型都成立：

- 当 X 没有默认值时，如何处理 `vector<X>`？
- 我们如何确保元素在使用完毕后被销毁？

我们必须解决这些问题吗？我们本可以说“不要试图创建没有默认值的类型的 vector”以及“对带析构函数的类型，不要以会引发问题的方式使用 vector”。对于一个以“通用”为目标的设施而言，这类限制令用户恼火，并让人感觉设计者没有把问题想透彻，或者根本不在乎用户。这种怀疑常常是对的，但标准库的设计者没有把这些瑕疵留在原地。为了与标准库 vector 看齐，我们必须解决这两个问题。

对于没有默认值的类型，我们可以让用户选择指定“需要默认值时使用什么值”，以此作为应对手段：

```
template<typename T> void vector<T>::resize(int newsize, T def = T());
```

也就是说，除非用户另有指定，就用 `T()` 作为默认值。例如：

```
vector<double> v1;
v1.resize(100);                 // 追加 100 个 double(), 即 0.0
v1.resize(200, 0.0);            // 追加 100 个 0.0 ——提及 0.0 是多余的
v1.resize(300, 1.0);            // 追加 100 个 1.0


struct No_default {
    No_default(int);            // No_default 仅有的构造函数
    // ...
};


vector<No_default> v2(10);      // 错误：试图创建 10 个 No_default()
vector<No_default> v3;
v3.resize(100, No_default(2));  // 追加 100 个 No_default(2)
v3.resize(200);                 // 错误：试图追加 100 个 No_default()
```

析构函数问题则更难对付。从根本上说，我们需要处理一种非常棘手的情况：一个数据结构中既有一些已初始化的数据，又有一些未初始化的数据。迄今为止，我们一直在尽力避开未初始化数据及其通常伴随的编程错误。而现在——作为 vector 的实现者——我们必须直面这个问题，这样我们——作为 vector 的用户——在应用程序中就不必面对它了。

首先，我们需要找到获取和操纵未初始化存储的方法。幸运的是，标准库提供了一个类 allocator（分配器），它提供未初始化的内存。一个略微简化的版本如下所示：

```
template<typename T> class allocator {
public:
    // ...
    T* allocate(int n);              // 为 n 个 T 类型的对象分配空间
    void deallocate(T* p, int n);    // 释放从 p 开始的 n 个 T 类型的对象

    void construct(T* p, const T& v);    // 在 p 中用值 v 构造一个 T
    void destroy(T* p);                  // 销毁 p 中的 T
};
```

如果需要完整的内容，请查阅 The C++ Programming Language、<memory>（§B.1.1）或标准。不过，这里给出的内容已经展示了四个基本操作，它们使我们能够：

- 分配大小适合存放一个 T 类型对象的内存，但不初始化它；
- 在未初始化空间中构造一个 T 类型的对象；
- 销毁一个 T 类型的对象，从而将其空间归还至未初始化状态；
- 释放大小适合存放一个 T 类型对象的未初始化空间。

不出所料，allocator 恰恰是实现 vector<T>::reserve() 所需要的。我们首先给 vector 一个 allocator 参数：

```
template<typename T, typename A = allocator<T>> class vector {
    A alloc;        // 用 allocate 处理元素的内存
    // ...
};
```

除了要提供一个 allocator——并且默认使用标准的 allocator 而不是 new——其他一切照旧。作为 vector 的用户，我们可以忽略 allocator，直到发现自己需要以某种特殊方式管理元素内存的 vector。而作为 vector 的实现者，以及试图理解基本问题、学习基本技术的学生，我们必须弄清 vector 如何处理未初始化内存，并向其用户呈现已正确构造的对象。唯一受影响的代码是直接操作内存的 vector 成员函数，例如 vector<T>::reserve()：

```
template<typename T, typename A>
void vector<T,A>::reserve(int newalloc)
{
    if (newalloc<=space) return;              // 绝不减少分配量
    T* p = alloc.allocate(newalloc);          // 分配新空间
    for (int i=0; i<sz; ++i) alloc.construct(&p[i],elem[i]);    // 拷贝
    for (int i=0; i<sz; ++i) alloc.destroy(&elem[i]);           // 销毁
    alloc.deallocate(elem,space);             // 释放旧空间
    elem = p;
    space = newalloc;
}
```

我们把元素搬到新空间的方法是：先在未初始化空间中构造一个副本，然后销毁原对象。我们不能使用赋值，因为对 string 这类类型来说，赋值假定目标区域已被初始化。

有了 reserve()，vector<T,A>::push_back() 就很好写了：

```
template<typename T, typename A>
void vector<T,A>::push_back(const T& val)
{
    if (space==0) reserve(8);                // 先分配容纳 8 个元素的空间
    else if (sz==space) reserve(2*space);    // 获取更多空间
    alloc.construct(&elem[sz],val);          // 在末尾添加 val
    ++sz;                                    // 增加元素数目
}
```

类似地，vector<T,A>::resize() 也不难：

```
template<typename T, typename A>
void vector<T,A>::resize(int newsize, T val = T())
{
    reserve(newsize);
    for (int i=sz; i<newsize; ++i) alloc.construct(&elem[i],val);   // 构造
    for (int i = newsize; i<sz; ++i) alloc.destroy(&elem[i]);       // 销毁
    sz = newsize;
}
```

注意，由于某些类型没有默认构造函数，我们再次提供了选项，允许用户提供一个值，作为新元素的初始值。

这里另一个新东西是：当我们把 vector 调整为更小的时候，会销毁“多余的元素”。可以把析构函数理解为把一个有类型的对象还原为“原始内存”。

“摆弄 allocator”是相当高级且棘手的东西。在你准备好成为专家之前，先别碰它。

19.4　范围检查与异常

回顾一下我们迄今完成的 vector，我们会（惊恐地?）发现访问并未做范围检查。operator[] 的实现非常简单：

```
template<typename T, typename A> T& vector<T,A>::operator[](int n)
{
    return elem[n];
}
```

那么，请看：

```
vector<int> v(100);
v[-200] = v[200];           // 糟糕!
int i;
cin>>i;
v[i] = 999;                 // 肆意践踏某个任意的内存位置
```

这段代码能编译、能运行，访问的却是不属于我们 vector 的内存。这可能意味着大麻烦！在真正的程序中，这样的代码是不可接受的。让我们改进 vector 来解决这个问题。最简单的办法是添加一个带检查的访问操作，称为 at()：

```
struct out_of_range { /* ... */ };  // 用于报告越界访问错误的类


template<typename T, typename A = allocator<T>> class vector {
    // ...
    T& at(int n);                   // 带检查的访问
    const T& at(int n) const;       // 带检查的访问

    T& operator[](int n);           // 不带检查的访问
    const T& operator[](int n) const;   // 不带检查的访问
    // ...
};


template<typename T, typename A > T& vector<T,A>::at(int n)
{
    if (n<0 || sz<=n) throw out_of_range();
    return elem[n];
}


template<typename T, typename A > T& vector<T,A>::operator[](int n)
    // 同前
{
    return elem[n];
}
```

有了这些，我们就可以这样写：

```
void print_some(vector<int>& v)
{
    int i = -1;
    while(cin>>i && i!=-1)
    try {
        cout << "v[" << i << "]==" << v.at(i) << "\n";
    }
    catch(out_of_range) {
        cout << "bad index: " << i << "\n";
    }
}
```

这里，我们用 `at()` 获得带范围检查的访问，并在发生非法访问时捕获 out_of_range。

总体思路是：当我们确定下标有效时用 `[ ]` 下标访问；当索引可能越界时用 `at()`。

19.4.1 题外话：设计考量

到目前为止一切顺利，但我们为什么不直接把范围检查加进 `operator[]()` 呢？好吧，标准库 vector 正如这里所示，提供了带检查的 `at()` 和不带检查的 `operator[]()`。让我们试着解释一下这为何有其道理。基本上有四条论据：

1. 兼容性（compatibility）：早在 C++ 有异常之前，人们就已在使用不带检查的下标访问了。
2. 效率（efficiency）：你可以在一个速度最优的不带检查访问运算符之上构建带检查的访问运算符，但无法在带检查的访问运算符之上构建速度最优的访问运算符。
3. 约束（constraints）：在某些环境中，异常是不可接受的。
4. 可选检查（optional checking）：标准实际上并没有说不能对 vector 做范围检查，所以如果你想要检查，就选用一个会做检查的实现。

19.4.1.1 兼容性

人们非常非常不希望自己的旧代码被弄坏。例如，如果你有一百万行代码，把它们全部改写为正确使用异常，可能代价高昂。我们可以争辩说这些额外工作会让代码变得更好，但买单的（在时间或金钱上）并不是我们。此外，现有代码的维护者通常会辩解：不带检查的代码原则上可能不安全，但他们那份特定代码已经过多年测试和使用，所有 bug 都已被找出来了。我们可以对这种论调持怀疑态度，但话又说回来，没有对真实代码做过此类决定的人，不宜过于苛责。自然，在标准库 vector 被引入 C++ 标准之前，没有代码使用它，但有千百万行代码使用的 vector 与之非常相似（由于是标准之前的），它们不使用异常。这些代码很多后来都被改造成了使用标准库的版本。

19.4.1.2 效率

是的，在极端情况下范围检查可能成为负担，例如网络接口的缓冲区和高性能科学计算中的矩阵。然而，对大多数人大部分时间所从事的“普通计算”而言，范围检查的代价很少成为问题。因此，只要可行，我们就推荐并使用带范围检查的 vector 实现。

19.4.1.3 约束

同样，这条论据对某些程序员和某些应用成立。事实上，它对相当多的程序员都成立，不应轻易忽视。不过，如果你要在一个不涉及硬实时（hard real time，参见 §25.2.1）的环境中开始一个新程序，请优先使用基于异常的错误处理和带范围检查的 vector。

19.4.1.4 可选检查

ISO C++ 标准只是规定：对 vector 的越界访问不保证具有任何特定语义，应避免这类访问。当程序试图越界访问时抛出异常，是完全符合标准的做法。因此，如果你希望 vector 抛出异常，而某个特定应用又不受前三条理由的制约，就使用带范围检查的 vector 实现。本书正是这样做的。

长话短说：现实世界的设计可能比我们希望的更混乱，但总有应对之道。

19.4.2　坦白：宏

与我们的 vector 一样，标准库 vector 的大多数实现并不保证对下标运算符（[ ]）做范围检查，但提供会做检查的 at()。那么，我们程序中出现的那些 std::out_of_range 异常是从哪儿来的呢？基本上，我们选择了 §19.4.1 的“选项 4”：vector 实现没有义务对 [ ] 做范围检查，但也没有被禁止这么做，于是我们安排了检查。你可能一直在用的是我们的调试版本，称为 Vector，它确实对 [ ] 做检查。这就是我们开发代码时使用的版本。它能减少错误、缩短调试时间，而对性能的代价很小：

```
struct Range_error : out_of_range {     // 增强的 vector 范围错误报告
    int index;
    Range_error(int i) :out_of_range("Range error"), index(i) { }
};


template<typename T> struct Vector : public std::vector<T> {
    using size_type = typename std::vector<T>::size_type;
    using vector<T>::vector;            // 使用 vector<T> 的构造函数 (§20.5)

    T& operator[](size_type i)          // 而不是 return at(i);
    {
        if (i<0||this->size()<=i) throw Range_error(i);
        return std::vector<T>::operator[](i);
    }

    const T& operator[](size_type i) const
    {
        if (i<0||this->size()<=i) throw Range_error(i);
        return std::vector<T>::operator[](i);
    }
};
```

我们使用 Range_error 是为了让出错的索引在调试时可用。从 std::vector 派生，Vector 就获得了 vector 的全部成员函数。第一个 using 为 std::vector 的 size_type 引入了一个方便的同义词，见 §20.5。第二个 using 让 Vector 获得了 vector 的全部构造函数。

这个 Vector 在调试非平凡程序时一直很有用。另一种选择是使用对完整标准库 vector 做了系统性检查的实现——事实上，这很可能正是你一直在用的；除了标准所保证的之外，我们无从知道你的编译器和库到底提供了何种程度的检查。

在 std_lib_facilities.h 中，我们用一个不怎么光彩的伎俩（宏替换）把 vector 重定义为 Vector：

```
// 为得到带范围检查的 vector 而采用的丑陋宏手段:
#define vector Vector
```

这意味着无论你何时写下 vector，编译器看到的都是 Vector。这个伎俩很糟糕，因为你在代码中看到的并不是编译器所看到的。在现实世界的代码中，宏是晦涩错误的重要来源（§27.8、§A.17.2）。

我们对 string 做了同样的事，以提供带范围检查的访问。遗憾的是，要从一个 vector 实现的 [ ] 获得范围检查，并没有标准、可移植且干净的办法。不过，把带范围检查的 vector（和 string）做得比我

们更干净、更完整是可能的。只是那通常需要替换厂商的标准库实现、调整安装选项，或者改动标准库源码。这些选项没有一个是适合初学者编程第一周的——而我们在第 2 章就用上了 string。

19.5　资源与异常

好了，vector 可以抛出异常，而且我们推荐：当函数无法完成其应完成的动作时，就抛出一个异常告知其调用者（第 5 章）。现在是时候考虑：当我们编写的代码必须应对 vector 操作和我们调用的其他函数抛出的异常时，该怎么做。那个天真的答案——“用 try 块捕获异常，输出一条错误消息，然后终止程序”——对大多数非平凡系统来说都太粗糙了。

编程的一条基本准则是：如果我们获取了一个资源（resource），就必须——以某种方式，直接或间接地——把它归还给系统中管理该资源的那个部分。资源的例子有：

- 内存
- 锁
- 文件句柄
- 线程句柄
- 套接字
- 窗口

基本上，我们把资源定义为：被获取之后必须归还（释放）或由某个“资源管理器”回收的东西。最简单的例子就是自由存储内存：我们用 `new` 获取它，用 `delete` 把它还给自由存储。例如：

```
void suspicious(int s, int x)
{
    int* p = new int[s];        // 获取内存
    // ...
    delete[] p;                 // 释放内存
}
```

正如 §17.4.6 所看到的，我们必须记得释放内存，而这并不总是容易做到的。一旦把异常引入图景，资源泄漏就可能变得很常见；只需无知或稍欠小心就够了。特别地，我们对诸如 `suspicious()` 这样显式使用 `new` 并把得到的指针赋给局部变量的代码怀有极大的怀疑。

我们把像 vector 这样负责释放资源的对象称为其所负责资源的所有者（owner）或句柄（handle）。

19.5.1　潜在的资源管理问题

对于像这样貌似无害的指针赋值，令人起疑的一个理由是：

```
int* p = new int[s];            // 获取内存
```

很难验证这个 `new` 有与之对应的 `delete`。至少 `suspicious()` 中有一条 `delete[] p;` 语句可能释放内存，但让我们设想几种可能导致该释放不会发生的情形。在省略号部分放些什么会引起内存泄漏呢？我们找到的这些有问题的例子应当引起你的思考，让你对这类代码保持警惕；它们也应当让你欣赏到这类代码那种简单而强大的替代方案。

也许在执行到 `delete` 时，p 已经不再指向原来那个对象了：

```
void suspicious(int s, int x)
{
    int* p = new int[s];        // 获取内存
    // ...
    if (x) p = q;               // 让 p 指向另一个对象
    // ...
    delete[] p;                 // 释放内存
}
```

我们放那个 `if (x)` 是为了让你无法知道我们是否改变了 `p` 的值。也许我们根本执行不到 `delete`：

```
void suspicious(int s, int x)
{
    int* p = new int[s];        // 获取内存
    // ...
    if (x) return;
    // ...
    delete[] p;                 // 释放内存
}
```

也许我们根本执行不到 `delete`，因为我们抛出了异常：

```
void suspicious(int s, int x)
{
    int* p = new int[s];        // 获取内存
    vector<int> v;
    // ...
    if (x) p[x] = v.at(x);
    // ...
    delete[] p;                 // 释放内存
}
```

这里最令我们关心的就是最后这种可能性。人们初次遇到这个问题时，往往把它当成异常的问题，而不是资源管理问题。由于错误地归类了根源，他们想出的解决办法是捕获这个异常：

```
void suspicious(int s, int x)   // 代码杂乱
{
    int* p = new int[s];        // 获取内存
    vector<int> v;
    // ...
    try {
        if (x) p[x] = v.at(x);
        // ...
    } catch (...) {             // 捕获所有异常
        delete[] p;             // 释放内存
        throw;                  // 重新抛出异常
    }
    // ...
    delete[] p;                 // 释放内存
}
```

这解决了问题，代价是增加了一些代码，并且资源释放代码（这里是 `delete[] p;`）出现了重复。换言之，这个解法很丑陋；更糟的是，它的泛化性不好。考虑获取更多资源的情形：

```
void suspicious(vector<int>& v, int s)
{
    int* p = new int[s];
    vector<int>v1;
    // ...
    int* q = new int[s];
    vector<double> v2;
    // ...
    delete[] p;
    delete[] q;
}
```

注意，如果 `new` 找不到可分配的自由存储内存，它将抛出标准库异常 `bad_alloc`。对上面这个例子，try … catch 技术同样可行，但你需要好几个 try 块，而且代码重复且丑陋。我们不喜欢重复而丑陋的代码，因为“重复”意味着代码将成为维护隐患，而“丑陋”意味着代码难以写对、难以阅读，也是维护隐患。

试一试

给上面最后一个例子加上 try 块，确保在所有可能抛出异常的情况下，所有资源都被正确释放。

19.5.2　资源获取即初始化

幸运的是，我们不需要把复杂的 try … catch 语句涂满代码来对付潜在的资源泄漏。考虑：

```
void f(vector<int>& v, int s)
{
    vector<int> p(s);
    vector<int> q(s);
    // ...
}
```

这个版本更好。更重要的是，它显然更好。资源（这里是自由存储内存）由构造函数获取，由匹配的析构函数释放。其实，在解决 vector 的内存泄漏问题时，我们就已经解决了这个特定的“异常问题”。这个解法是通用的，适用于所有类型的资源：在某个管理该资源的对象的构造函数中获取资源，并在匹配的析构函数中释放它。通常最好用这种方式处理的资源包括数据库锁、套接字和 I/O 缓冲区（iostream 替你做好了）。这种技术通常用一个拗口的短语来称呼——“资源获取即初始化”（Resource Acquisition Is Initialization），缩写为 RAII。

考虑上面的例子。无论我们以何种方式离开 `f()`，p 和 q 的析构函数都会被恰当地调用：由于 p 和 q 不是指针，我们不能对它们赋值；return 语句不会阻止析构函数的调用，抛出异常也不会。这条通用规则成立：当执行线程离开一个作用域时，每个已完全构造的对象和子对象的析构函数都会被调用。当一个对象的构造函数执行完毕，它就被认为是已构造的。深究这两句话的细节含义可能会令人头疼，但它们的意思其实很简单：构造函数和析构函数会在需要时被调用。

特别地，当你在作用域内需要非常量的存储量时，使用 vector，而不要显式地用 `new` 和 `delete`。

19.5.3　保证

当我们无法把 vector 保留在单个作用域（及其子作用域）内时，能做什么呢？例如：

```
vector<int>* make_vec()             // 创建一个填充好的 vector
{
    vector<int>* p = new vector<int>;   // 我们在自由存储上分配
    // ... 用数据填充 vector; 这可能抛出异常 ...
    return p;
}
```

这是常见的一类代码的例子：我们调用一个函数来构造一个复杂的数据结构，并把该数据结构作为结果返回。问题在于：如果在“填充”vector 的过程中抛出了异常，`make_vec()` 就会泄漏那个 vector。另一个与之无关的问题是：如果函数成功了，总得有人去 delete `make_vec()` 返回的对象（参见 §17.4.6）。

我们可以加一个 try 块来应对 throw 的可能性：

```
vector<int>* make_vec()              // 创建一个填充好的 vector
{
    vector<int>* p = new vector<int>;   // 我们在自由存储上分配
    try {
        // 用数据填充 vector; 这可能抛出异常
        return p;
    }
    catch (...) {
        delete p;          // 做我们局部的清理工作
        throw;             // 重新抛出，让调用者去面对
                           // make_vec() 未能完成
                           // 交给它的任务这一事实
    }
}
```

这个 `make_vec()` 函数展示了一种非常常见的错误处理风格：它尽力完成自己的工作，如果做不到，就清理所有局部资源（这里是自由存储上的那个 vector），并通过抛出异常来指示失败。这里抛出的异常是某个其他函数（如 `vector::at()`）抛出的；`make_vec()` 只是用 `throw;` 把它重新抛出。这是一种简单有效的错误处理方式，可以系统地使用。

- 基本保证（basic guarantee）：try … catch 代码的目的是确保 `make_vec()` 要么成功，要么在不泄漏任何资源的前提下抛出异常。这常被称为基本保证。凡是程序中我们期望能从异常 throw 中恢复的代码，都应提供基本保证。所有标准库代码都提供基本保证。
- 强保证（strong guarantee）：如果在提供基本保证之外，一个函数还确保所有可观察的值（所有不属于函数局部范围的值）在失败后与调用该函数时相同，那么就说该函数提供强保证。强保证是我们编写函数时的理想：函数要么完成了它被要求做的全部事情，要么除了抛出一个异常指示失败之外什么也没发生。
- 不抛出保证（no-throw guarantee）：如果连简单操作都无法在没有任何失败和抛出异常风险的情况下完成，我们就不可能写出满足基本保证和强保证的代码。幸运的是，C++ 中本质上所有内置设施都提供不抛出保证：它们根本不会抛出。要避免抛出，只需避开 throw 本身、`new` 和引用类型的 `dynamic_cast`（§A.5.7）。

基本保证和强保证主要用于思考程序的正确性。而 RAII 对于简单、高效地实现按照这些理想编写的代码至关重要。

自然，我们应始终避免未定义（且通常后果惨重）的操作，例如解引用 0、除以 0、以及越界访问数组。捕获异常救不了那些违反语言基本规则的错误。

19.5.4　unique_ptr

于是，`make_vec()` 是一类很有用的函数：在存在异常的情况下，它遵守了良好资源管理的基本规则。当我们想从异常 throw 中恢复时，它提供了基本保证——正如所有良好的函数都应做到的那样。如果在“用数据填充 vector”那部分没有对非局部数据做什么恶劣操作，它甚至提供了强保证。然而，那段 try … catch 代码依然丑陋。解决办法显而易见：无论如何我们必须使用 RAII；也就是说，我们需要提供一个对象来持有那个 `vector<int>`，以便在异常发生时由它 delete 那个 vector。在 `<memory>` 中，标准库为此提供了 unique_ptr：

```
vector<int>* make_vec()          // 创建一个填充好的 vector
{
    unique_ptr<vector<int>> p {new vector<int>};   // 在自由存储上分配
    // ... 用数据填充 vector; 这可能抛出异常 ...
    return p.release();          // 返回 p 所持有的指针
}
```

unique_ptr 是一个持有指针的对象。我们立即用它从 new 得到的指针来初始化它。你可以像使用内置指针一样在 unique_ptr 上使用 --> 和 *（例如 p-->at(2) 或 (*p).at(2)），因此我们把 unique_ptr 视为一种指针。不过，unique_ptr 拥有（own）它所指向的对象：当 unique_ptr 被销毁时，它会 delete 它所指向的对象。这意味着，如果在填充 vector<int> 的过程中抛出异常，或者我们从 make_vec 提前返回，vector<int> 都会被正确销毁。p.release() 从 p 中抽出所包含的指针（指向 vector<int>）以便我们返回它，同时它还让 p 持有 nullptr，这样销毁 p（return 时会发生）就不会销毁任何东西。

使用 unique_ptr 极大地简化了 make_vec()。基本上，它让 make_vec() 简单得就像那个天真但不安全的版本。重要的是，有了 unique_ptr，我们就可以重申那条建议：对显式 try 块要保持怀疑；大多数 try 块——正如 make_vec() 中的那样——都可以用某种“资源获取即初始化”技术的变体来替代。

使用 unique_ptr 的 make_vec() 版本已经不错了，只是它仍然返回一个指针，因此仍然得有人记得去 delete 那个指针。返回一个 unique_ptr 就能解决这个问题：

```
unique_ptr<vector<int>> make_vec()       // 创建一个填充好的 vector
{
    unique_ptr<vector<int>> p {new vector<int>};   // 在自由存储上分配
    // ... 用数据填充 vector; 这可能抛出异常 ...
    return p;
}
```

unique_ptr 非常像普通指针，但它有一条重要限制：你不能把一个 unique_ptr 赋给另一个，从而得到指向同一对象的两个 unique_ptr。必须如此，否则就可能产生混乱：究竟哪个 unique_ptr 拥有被指向的对象、必须由它来 delete。例如：

```
void no_good()
{
    unique_ptr<X> p { new X };
    unique_ptr<X> q {p};         // 错误: 幸好
    // ...
} // 此处 p 和 q 都会 delete 那个 X
```

如果你想要一个既保证删除、又可拷贝的“智能”指针，请使用 shared_ptr（§B.6.5）。不过，那是一个更重量级的解决方案，它需要一个使用计数（use count）来确保最后一个被销毁的副本销毁所指的对象。

unique_ptr 有一个有趣的性质：与普通指针相比它没有任何额外开销。

19.5.5 通过移动返回

先把大量信息放到自由存储上、再返回指向它的指针，这种返回技术非常常见。它也是大量复杂性的来源，并且是内存管理错误的主要来源之一：谁负责 delete 一个从函数返回的、指向自由存储的指针？我们能确定指向自由存储上对象的指针在发生异常的情况下会被正确地 delete 吗？除非我们系统地进行指针管理（或使用 unique_ptr 和 shared_ptr 这类“智能”指针），否则答案将是“呃，我们认为应该没问题”，而这不够好。

幸运的是，当我们为 vector 添加移动操作时，就为 vector 解决了这个问题：只需用一个移动构造函数（move constructor）把元素的所有权移出函数即可。例如：

```
vector<int> make_vec()      // 创建一个填充好的 vector
{
    vector<int> res;
    // ... 用数据填充 vector；这可能抛出异常 ...
    return res;             // 移动构造函数高效地转移所有权
}
```

这个（最终）版本的 make_vec() 是最简单的，也是我们推荐的版本。移动方案可以泛化到所有容器，进一步泛化到所有资源句柄。例如，fstream 就用这种技术来跟踪文件句柄。移动方案简单而通用。使用资源句柄可以简化代码、消除一个主要错误来源。与直接使用指针相比，使用这类句柄的运行时开销为零，或非常轻微且可预测。

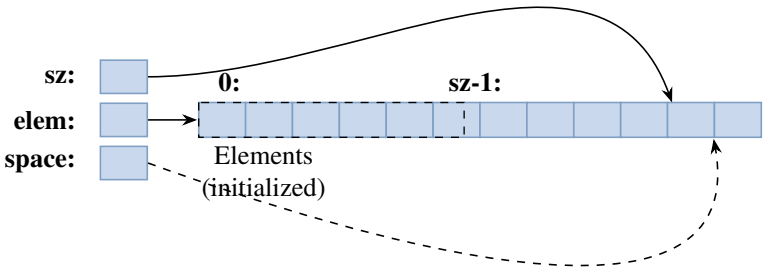
19.5.6 vector 的 RAII

即便使用 unique_ptr 这样的智能指针，可能仍显得有些临时拼凑。我们如何确定已找出了所有需要保护的指针？我们如何确定已释放了所有指向那些不该在作用域结尾销毁的对象的指针？考虑 §19.3.7 的 reserve()：

```
template<typename T, typename A>
void vector<T,A>::reserve(int newalloc)
{
    if (newalloc<=space) return;        // 绝不减少分配量
    T* p = alloc.allocate(newalloc);    // 分配新空间
    for (int i=0; i<sz; ++i) alloc.construct(&p[i],elem[i]);    // 拷贝
    for (int i=0; i<sz; ++i) alloc.destroy(&elem[i]);           // 销毁
    alloc.deallocate(elem,space);       // 释放旧空间
    elem = p;
    space = newalloc;
}
```

注意，对旧元素的拷贝操作 alloc.construct(&p[i],elem[i]) 可能抛出异常。因此，p 正是我们在 §19.5.1 中警告过的那种问题的例子。糟糕！我们可以套用 unique_ptr 方案。更好的办法是退一步，意识到“vector 的内存”本身就是一个资源；也就是说，我们可以定义一个类 vector_base，来表示我们一直在使用的基本概念——那张用三个元素刻画 vector 内存使用状况的图：

用代码写出来就是（为完整起见，加上分配器之后）：




```
template<typename T, typename A>
struct vector_base {
    A alloc;        // 分配器
    T* elem;        // 分配空间的起始处
    int sz;         // 元素数目
    int space;      // 已分配空间的数量

    vector_base(const A& a, int n)
        : alloc{a}, elem{alloc.allocate(n)}, sz{n}, space{n}{ }
    ~vector_base() { alloc.deallocate(elem,space); }
};
```

注意，vector_base 处理的是内存，而不是（有类型的）对象。我们的 vector 实现可以用它来保存所需元素类型的对象。基本上，vector 只是 vector_base 的一个方便的接口：

```
template<typename T, typename A = allocator<T>>
class vector : private vector_base<T,A> {
public:
    // ...
};
```

这样我们就可以把 `reserve()` 重写为更简单也更正确的版本：

```
template<typename T, typename A>
void vector<T,A>::reserve(int newalloc)
{
    if (newalloc<=this->space) return;          // 绝不减少分配量
    vector_base<T,A> b(this->alloc,newalloc);   // 分配新空间
    uninitialized_copy(b.elem,&b.elem[this->sz],this->elem);    // 拷贝
    for (int i=0; i<this->sz; ++i)
        this->alloc.destroy(&this->elem[i]);    // 销毁旧的
    swap<vector_base<T,A>>(*this,b);            // 交换表示
}
```

我们使用标准库函数 uninitialized_copy 来从 b 构造元素的副本，因为它能正确处理元素拷贝构造函数抛出的异常，而且调用一个函数比手写一个循环更简单。当我们退出 `reserve()` 时，如果拷贝操作成功了，旧的分配空间会由 vector_base 的析构函数自动释放；如果退出是由拷贝操作抛出异常引起的，则释放的是新分配的空间。`swap()` 函数是一个标准库算法（来自 `<algorithm>`），它交换两个对象的值。我们用 `swap<vector_base<T,A>>(*this,b)` 而不是更简单的 `swap(*this,b)`，是因为 `*this` 和 b 类型不同（分别是 vector 和 vector_base），所以必须明确指明我们想要的是哪个 swap 特化版本。类似地，当我们从派生类 `vector<T,A>` 的成员（例如 `vector<T,A>::reserve()`）中引用基类 `vector_base<T,A>` 的成员时，必须显式地使用 `this->`。

试一试

修改 `reserve`，改用 unique_ptr。记得在返回之前先 release。将这种方案与 vector_base 方案比较：考虑哪个更容易编写，哪个更容易写对。

小试身手

1. 定义 `template<typename T> struct S { T val; };`。
2. 添加一个构造函数，使你可以用一个 T 来初始化。
3. 定义类型为 `S<int>`、`S<char>`、`S<double>`、`S<string>` 和 `S<vector<int>>` 的变量；用你选择的值初始化它们。
4. 读取这些值并打印它们。
5. 添加一个函数模板 `get()`，返回对 val 的引用。
6. 把 `get()` 的定义移到类外。
7. 把 val 设为私有的。
8. 用 `get()` 再做一遍第 4 题。
9. 添加一个 `set()` 函数模板，使你可以改变 val。
10. 用 `S<T>::operator=(const T&)` 替代 `set()`。提示：比 §19.2.5 简单得多。
11. 提供 `get()` 的 const 和非 const 两个版本。
12. 定义函数模板 `template<typename T> read_val(T& v)`，从 `cin` 读入 v。
13. 用 `read_val()` 把第 3 题中除 `S<vector<int>>` 变量之外的每个变量都读入。
14. 附加题：为 `vector<T>` 定义输入和输出运算符（`>>` 和 `<<`）。输入和输出都使用 `{ val, val, val }` 格式。这将使 `read_val()` 也能处理 `S<vector<int>>` 变量。

记住每一步之后都要测试。

要点回顾

1. 我们为什么想改变 vector 的大小？
2. 为什么我们想让不同的 vector 有不同的元素类型？
3. 为什么我们不干脆总是把 vector 定义得足够大，以应付所有情况？
4. 我们为新 vector 分配多少备用空间？
5. 什么时候必须把 vector 元素拷贝到新位置？
6. 哪些 vector 操作能在构造之后改变 vector 的大小？
7. 拷贝之后 vector 的值是什么？
8. 哪两个操作定义了 vector 的拷贝？
9. 对类对象来说，拷贝的默认含义是什么？
10. 什么是模板？
11. 最有用的两类模板实参是什么？
12. 什么是泛型编程？
13. 泛型编程与面向对象编程有何不同？
14. array 与 vector 有何不同？
15. array 与内置数组有何不同？
16. `resize()` 与 `reserve()` 有何不同？
17. 什么是资源？请定义并举例。
18. 什么是资源泄漏？
19. 什么是 RAII？它解决什么问题？
20. unique_ptr 有什么用？

术语

`#define` at()（带检查访问） 基本保证（basic guarantee）
异常（exception） 保证（guarantees） 句柄（handle）
实例化（instantiation） 宏（macro） 所有者（owner）
`push_back()` RAII `resize()`
资源（resource） 重抛（re-throw） 自赋值（self-assignment）
`shared_ptr` 特化（specialization） 强保证（strong guarantee）
模板（template） 模板参数（template parameter） `this`
`throw;` `unique_ptr`

习题

对每道习题，请创建并测试（带输出）所定义类的几个对象，以证明你的设计和实现确实做到了你以为做到的事。涉及异常的地方，这可能需要仔细思考错误可能出现在哪里。

1. 编写一个模板函数 `f()`，把一个 `vector<T>` 的元素加到另一个的元素上；例如，`f(v1,v2)` 应对 v1 的每个元素执行 `v1[i]+=v2[i]`。
2. 编写一个模板函数，接受 `vector<T>` 类型的 vt 和 `vector<U>` 类型的 vu 作为实参，返回所有 `vt[i]*vu[i]` 的和。
3. 编写一个模板类 Pair，可以保存任何类型的一对值。用它实现一个简单的符号表，类似于我们在计算器（§7.8）中用的那种。

4. 把 §17.9.3 的 Link 类修改为模板，以值的类型作为模板实参。然后用 `Link<God>` 重做第 17 章的习题 13。
5. 定义一个类 Int，它有一个类型为 `int` 的成员。为它定义构造函数、赋值和运算符 `+`、`-`、`*`、`/`。测试它，并按需改进其设计（例如，为方便 I/O 定义运算符 `<<` 和 `>>`）。
6. 重复上一题，但用一个类 `Number<T>`，其中 T 可以是任何数值类型。试着给 Number 添加 `%`，看看对 `Number<double>` 和 `Number<int>` 使用 `%` 会发生什么。
7. 用一些 Number 试试你对习题 2 的解。
8. 使用基本分配函数 `malloc()` 和 `free()`（§B.11.4）实现一个 allocator（§19.3.7）。让 §19.4 末尾所定义的 vector 在几个简单测试用例下工作。提示：在完整的 C++ 参考手册中查阅“placement new（定位 new）”和“显式调用析构函数”。
9. 使用 allocator（§19.3.7）管理内存，重新实现 `vector::operator=()`（§19.2.5）。
10. 实现一个简单的 unique_ptr，只支持构造函数、析构函数、`-->`、`*` 和 `release()`。特别地，不要试图实现赋值或拷贝构造函数。
11. 设计并实现一个 `counted_ptr<T>`，它是一种持有指向 T 类型对象的指针和一个指向“使用计数”（一个 `int`）的指针的类型，该使用计数由所有指向同一 T 对象的 counted 指针共享。使用计数应保存指向某个给定 T 的 counted 指针的数目。让 `counted_ptr` 的构造函数在自由存储上分配一个 T 对象和一个使用计数；并让 `counted_ptr` 的构造函数接受一个实参，用作 T 元素的初始值。当最后一个针对某个 T 的 `counted_ptr` 被销毁时，`counted_ptr` 的析构函数应 delete 那个 T。为 `counted_ptr` 提供使我们能将其用作指针的操作。这是一个用来确保对象在其最后一个使用者停止使用之前不会被销毁的“智能指针”例子。为 `counted_ptr` 编写一组测试用例，把它用作调用的实参、容器元素等。
12. 定义一个 File_handle 类，其构造函数接受一个 string 实参（文件名），在构造函数中打开文件，在析构函数中关闭文件。
13. 编写一个 Tracer 类，其构造函数打印一个 string，其析构函数也打印一个 string。把这两个 string 作为构造函数实参给出。用它来观察 RAII 管理对象会在何处发挥作用（即，用局部对象、成员对象、全局对象、`new` 分配的对象等 Tracer 做实验）。然后为其添加拷贝构造函数和拷贝赋值，以便你能用 Tracer 对象观察拷贝何时发生。
14. 为第 18 章习题中的“捉拿 Wumpus”（Hunt the Wumpus）游戏提供一个 GUI 界面和一点图形输出。在一个输入框中获取输入，并在窗口中显示玩家当前已知的那部分洞穴的地图。
15. 修改上一题的程序，允许用户根据知识和猜测标记房间，例如“可能有蝙蝠”和“无底洞”。
16. 有时，我们希望空 vector 尽可能小。例如，有人可能大量使用 `vector<vector<vector<int>>>`，但大多数元素 vector 都是空的。请定义一个 vector，使 `sizeof(vector<int>)==sizeof(int*)`，也就是说，使 vector 本身只由一个指向表示（representation）的指针构成，该表示由元素、元素数目和 space 指针组成。

后记

模板和异常是无比强大的语言特性。它们支持极为灵活的编程技术——主要是允许人们分离关注点（separate concerns），即一次只处理一个问题。例如，使用模板，我们可以把 vector 这类容器的定义与元素类型的定义分开。类似地，使用异常，我们可以把检测并报告错误的代码与处理错误的代码分开。本章的第三个主题——改变 vector 的大小——也可以类似地看待：`push_back()`、`resize()` 和 `reserve()` 使我们能把 vector 的定义与其大小的规定分开。

第五部分

附录

附录 A　语言概要

“Be careful what you wish for;
you might get it.
（许愿须谨慎，因为你可能真的会如愿以偿。）”
——传统谚语

本附录概述 C++ 的关键语言要素。这份概述经过高度筛选，专门面向那些希望在本章所讲主题序列之外稍作探索的初学者。它的目标是简明，而不是完备。

A.1　总则

本附录是一份参考资料，并不打算像普通章节那样从头读到尾。它（或多或少）系统地描述 C++ 语言的关键要素。不过，它并不是完整的参考手册，而只是一份概要。它的重点与侧重由学生提出的问题决定。很多时候，你需要回到各章去寻找更完整的解释。本概要并不试图达到标准文档那样的精确程度，也不追求标准文档那样的术语体系；它追求的是易读。更多信息请参见 Stroustrup 的 *The C++ Programming Language*。

A.1 总则
- **A.1.1** 术语
- **A.1.2** 程序的启动与终止
- **A.1.3** 注释

A.2 字面量
- **A.2.1** 整数字面量
- **A.2.2** 浮点字面量
- **A.2.3** 布尔字面量
- **A.2.4** 字符字面量
- **A.2.5** 字符串字面量
- **A.2.6** 指针字面量

A.3 标识符
- **A.3.1** 关键字

A.4 作用域、存储类别与生存期
- **A.4.1** 作用域
- **A.4.2** 存储类别
- **A.4.3** 生存期

A.5 表达式
- **A.5.1** 用户定义运算符
- **A.5.2** 隐式类型转换
- **A.5.3** 常量表达式
- **A.5.4** `sizeof`
- **A.5.5** 逻辑表达式
- **A.5.6** `new` 与 `delete`
- **A.5.7** 类型转换

A.6 语句

A.7 声明
- **A.7.1** 定义

A.8 内置类型
- **A.8.1** 指针
- **A.8.2** 数组
- **A.8.3** 引用

A.9 函数
- **A.9.1** 重载解析
- **A.9.2** 默认实参
- **A.9.3** 未指定实参
- **A.9.4** 链接规范

A.10 用户定义类型
- **A.10.1** 运算符重载

A.11 枚举

A.12 类
- **A.12.1** 成员访问
- **A.12.2** 类成员定义
- **A.12.3** 构造、析构与复制
- **A.12.4** 派生类
- **A.12.5** 位域
- **A.12.6** 联合体

A.13 模板
- **A.13.1** 模板实参
- **A.13.2** 模板实例化
- **A.13.3** 模板成员类型

A.14 异常

A.15 命名空间

A.16 别名

A.17 预处理指令
- **A.17.1** #include
- **A.17.2** #define

C++ 的定义就是 ISO C++ 标准，但那份文档既不是为初学者准备的，也不适合初学者阅读。别忘了使用你的在线文档。如果你在前面的章节时就翻看本附录，那么其中很多内容会显得“神秘”，也就是说，它们要到后面的章节才会解释。

关于标准库设施，见附录 B。C++ 标准由一个委员会定义，该委员会在 ISO（国际标准化组织）主持下、与国家标准化机构合作工作，这些机构包括 INCITS（美国）、BSI（英国）和 AFNOR（法国）。当前的标准是 ISO/IEC 14882:2011《编程语言 C++ 标准》。

A.1.1　术语

C++ 标准定义了什么是 C++ 程序，也定义了各种构造的含义：

- 符合标准（conforming）：按标准属于 C++ 的程序称为符合标准的程序（口语中也叫合法或有效程序）。
- 实现定义（implementation defined）：程序可以（而且通常确实会）依赖这样一些特性（例如 `int` 的大小、`'a'` 的数值），它们只在特定的编译器、操作系统、机器架构等上才有明确定义。实现定

义特性在标准中列出，并且必须在实现文档中记录下来，其中许多还体现在标准头文件里，例如 <limits>（见 §B.1.1）。因此，符合标准并不等于可以移植到所有 C++ 实现。

- 未指定（unspecified）：有些构造的含义是未指定的、未定义的，或者虽不符合标准但也不要求给出诊断。显然，这类特性最好避开。本书避开了它们。要避免的未指定特性包括
- 不同源文件中不一致的定义（要一致地使用头文件；见 §8.3）
- 在一个表达式里反复读写同一个变量（主要例子是 a[i]=++i;）
- 大量使用显式类型转换（cast），尤其是 reinterpret_cast

A.1.2　程序的启动与终止

一个 C++ 程序必须有一个名为 main() 的全局函数。程序从执行 main() 开始。main() 的返回类型是 int（void 不是符合标准的替代写法）。main() 返回的值就是程序返回给“系统”的值。有些系统会忽略这个值，但成功终止是用返回 0 来表示的，失败则用返回非零值或抛出未捕获异常来表示（不过未捕获异常被认为是不良风格）。

传给 main() 的实参可以由实现定义，但每个实现都必须接受两个版本（不过每个程序只能使用其中一个）：

```
int main();                          // 无实参
int main(int argc, char* argv[]);    // argv[] 保存 argc 个 C 风格字符串
```

main() 的定义不必显式返回值。如果没有返回值，也就是所谓“从底部掉落”，它会返回 0。下面是最小的 C++ 程序：

```
int main() { }
```

如果你定义了一个带有构造函数和析构函数的全局（命名空间）作用域对象，那么构造函数在逻辑上会“在 main() 之前”执行，析构函数在逻辑上“在 main() 之后”执行（严格说来，执行这些构造函数是调用 main() 的一部分，执行析构函数是从 main() 返回的一部分）。只要有可能，就应避免全局对象，尤其是那些需要非平凡构造与析构的全局对象。

A.1.3　注释

凡是能在代码中说清楚的，就应该在代码中说清楚。不过，C++ 提供了两种注释风格，让程序员可以表达那些用代码不好表达的内容：

```
// 这是一条行注释

/*
    这是一段
    块注释
*/
```

显然，块注释主要用于多行注释，不过也有人即使写多行也偏好用单行注释：

```
// 这是一条
// 用三条行注释
// 表达的多行注释

/* 而这一条则是用块注释表达的单行注释 */
```

注释对于记录代码的意图至关重要；另见 §7.6.4。

A.2　字面量

字面量表示各种类型的值。例如，字面量 12 表示整数值“十二”，"Morning" 表示字符串值 Morning，而 true 表示布尔值 true。

A.2.1　整数字面量

整数字面量有三种变体：

- 十进制：一串十进制数字
  十进制数字：0、1、2、3、4、5、6、7、8 和 9
- 八进制：一串以 0 开头的八进制数字
  八进制数字：0、1、2、3、4、5、6 和 7
- 十六进制：一串以 0x 或 0X 开头的十六进制数字
  十六进制数字：0、1、2、3、4、5、6、7、8、9、a、b、c、d、e、f、A、B、C、D、E 和 F
- 二进制：一串以 0b 或 0B 开头的二进制数字（C++14）
  二进制数字：0、1

后缀 u 或 U 使整数字面量成为无符号类型（§25.5.3），后缀 l 或 L 使其成为 long 类型，例如 10u 和 123456UL。

C++14 还允许在数值字面量中用单引号作数字分隔符。例如 0b0000'0001'0010'0011 表示 0b0000000100100011，而 1'000'000 表示 1000000。

A.2.1.1　数制

我们通常用十进制记法写出数字。123 表示 1 个百加 2 个十加 3 个一，即 $1\times100+2\times10+3\times1$，或者（用上标表示“的幂”）$1\times10^2+2\times10^1+3\times10^0$。十进制的另一种说法是基 10。10 在这里其实并没有什么特别之处。我们真正有的是 $1\times\text{base}^2+2\times\text{base}^1+3\times\text{base}^0$，其中 base==10。关于我们为什么采用基 10，有很多理论。有一种理论已经“内置”到某些自然语言中：我们有十根手指，而位置记数法中每个直接代表一个值的符号（例如 0、1、2）都称为数字（digit）。digit 在拉丁语中就是“手指”的意思。

偶尔也会用到其他基数。通常，计算机内存中的正整数以基 2 表示（把 0 和 1 作为材料中的物理状态来可靠表示相对容易），而处理底层硬件问题的人有时用基 8、更常用基 16 来指代内存的内容。

考虑十六进制。我们需要为 0 到 15 这 16 个值命名。通常我们用 0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F，其中 A 的十进制值是 10，B 的十进制值是 11，依此类推：

```
A==10, B==11, C==12, D==13, E==14, F==15
```

这样我们就可以把十进制值 123 写成十六进制的 7B。要看清这一点，只需注意在十六进制中 7B 表示 $7 \times 16 + 11$，即（十进制）123。反过来，十六进制的 123 表示 $1 \times 16^2 + 2 \times 16 + 3$，也就是 $1 \times 256 + 2 \times 16 + 3$，即（十进制）291。如果你从未接触过非十进制的整数表示，我们强烈建议你试着把几个数在十进制与十六进制之间来回转换。注意，一个十六进制数字与一个二进制值之间有非常简单的对应关系：

十六进制与二进制

十六进制	0	1	2	3	4	5	6	7
二进制	0000	0001	0010	0011	0100	0101	0110	0111
十六进制	8	9	A	B	C	D	E	F
二进制	1000	1001	1010	1011	1100	1101	1110	1111

这就充分说明了十六进制记法的流行程度。尤其是，一个字节的值可以简单地用两个十六进制数字表示。

在 C++ 中（幸好如此），除非另有说明，数字都是十进制的。要说明一个数是十六进制的，我们加上前缀 `0X`（“X 代表 hex”），于是 `123==0X7B`、`0X123==291`。我们也可以等价地用小写 x，所以同样有 `123==0x7B` 和 `0x123==291`。类似地，十六进制数字也可以用小写 a、b、c、d、e、f。例如 `123==0x7b`。

八进制是基 8。我们只需要八个八进制数字：0、1、2、3、4、5、6、7。在 C++ 中，基 8 的数以 0 开头表示，因此 0123 不是十进制数 123，而是 $1 \times 8^2 + 2 \times 8 + 3$，也就是 $1 \times 64 + 2 \times 8 + 3$，即（十进制）83。反过来，八进制的 83，也就是 083，是 $8 \times 8 + 3$，即（十进制）67。用 C++ 记法表示，我们得到 `0123==83` 和 `083==67`。二进制是基 2。我们只需要两个数字：0 和 1。我们无法在 C++ 中把基 2 的数直接表示成字面量。只有基 8（八进制）、基 10（十进制）和基 16（十六进制）被直接支持为字面量以及整数的输入输出格式。不过，即使无法在 C++ 文本中直接表示二进制数，了解它们仍然有用。例如，（十进制）123 等于

$$1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2 + 1$$

也就是 $1 \times 64 + 1 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1$，即（二进制）1111011。

A.2.2　浮点字面量

浮点字面量（floating-point-literal）包含一个小数点（.）、一个指数（例如 e3）或一个浮点后缀（d 或 f）。例如：

```
123             // int（没有小数点、后缀或指数）
123.            // double: 123.0
123.0           // double
.123            // double: 0.123
0.123           // double
1.23e3          // double: 1230.0
1.23e-3         // double: 0.00123
1.23e+3         // double: 1230.0
```

浮点字面量的类型是 double，除非有后缀另行说明。例如：

```
1.23        // double
1.23f       // float
1.23L       // long double
```

A.2.3　布尔字面量

类型 bool 的字面量是 true 和 false。true 的整数值是 1，false 的整数值是 0。

A.2.4　字符字面量

字符字面量是用单引号括起来的一个字符，例如 'a' 和 '@'。此外还有一些“特殊字符”：

名称	ASCII 名	C++ 名
换行	NL	\n
水平制表	HT	\t
垂直制表	VT	\v
退格	BS	\b
回车	CR	\r
换页	FF	\f
响铃	BEL	\a
反斜杠	\	\\
问号	?	\?
单引号	'	\'
双引号	"	\"
八进制数	ooo	\ooo
十六进制数	hhh	\xhhh

特殊字符表示为用单引号括起来的“C++ 名”，例如 '\n'（换行）和 '\t'（制表）。

字符集包含以下可见字符：

```
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789
!@#$%^&*()_+|~`{}[]:";'<>?,./
```

在可移植的代码中，你不能依赖更多的可见字符。某个字符（例如字母 a 的 'a'）的值由实现决定（但很容易查出来，例如 cout << int('a')）。

A.2.5　字符串字面量

字符串字面量是用双引号括起来的一串字符，例如 "Knuth" 和 "King Canute"。换行不能成为字符串的一部分；要在字符串里表示换行，请改用特殊字符 \n：

```
"King
Canute "                    // 错误：字符串字面量中出现换行
"King\nCanute"      // 正确：把换行写进字符串字面量的正确方式
```

两个仅由空白分隔的字符串字面量会被当作一个字符串字面量。例如：

```
"King" "Canute"  // 等价于 "KingCanute"（没有空格）
```

注意，特殊字符（例如 \n）可以出现在字符串字面量中。

A.2.6　指针字面量

指针字面量只有一个：空指针 nullptr。为了兼容，任何求值为 0 的常量表达式也可以用作空指针。例如：

```
int* p1 = 0;            // 正确：空指针
int* p2 = 2-2;          // 正确：空指针
int* p3 = 1;            // 错误：1 是 int，不是指针
int z = 0;
int* p4 = z;            // 错误：z 不是常量
```

值 0 会被隐式转换为空指针。

在 C++ 中（C 中则不然，所以要留心 C 的头文件），NULL 被定义为 0，因此你可以写

```
int* p4 = NULL;   // （在 NULL 定义正确的前提下）空指针
```

A.3　标识符

标识符（identifier）是一个字符序列：它以字母或下划线开头，后面跟零个或多个（大写或小写）字母、数字或下划线：

```
int foo_bar;            // 正确
int FooBar;             // 正确
int foo bar;            // 错误：标识符中不能有空格
int foo$bar;            // 错误：标识符中不能有 $
```

以下划线开头或包含双下划线的标识符保留给实现使用；不要用它们。例如：

```
int _foo;               // 不要这样写
int foo_bar;            // 正确
int foo__bar;           // 不要这样写
int foo_;               // 正确
```

A.3.1　关键字

关键字是语言本身用来表达语言构造的标识符。

关键字（保留标识符）

alignas	class	explicit	noexcept	signed	typename
alignof	compl	export	not	sizeof	union
and	concept	extern	not_eq	static	unsigned
and_eq	const	false	nullptr	static_assert	using
asm	const_cast	float	operator	static_cast	virtual
auto	constexpr	for	or	struct	void
bitand	continue	friend	or_eq	switch	volatile
bitor	decltype	goto	private	template	wchar_t
bool	default	if	protected	this	while
break	delete	inline	public	thread_local	xor
case	do	int	register	throw	xor_eq
catch	double	long	reinterpret_cast	true	
char	dynamic_cast	mutable	requires	try	
char16_t	else	namespace	return	typedef	
char32_t	enum	new	short	typeid	

A.4　作用域、存储类别与生存期

C++ 中的每个名字（可惜预处理器名字是个例外；见 §A.17）都存在于某个作用域中；也就是说，这个名字属于一段可以使用它的文本区域。数据（对象）存储在内存的某个地方；用来存放一个对象的那类内存称为它的存储类别（storage class）。对象的生存期从它首次被初始化那一刻起，直到最终被销毁为止。

A.4.1　作用域

共有五种作用域（§8.4）：

- 全局作用域（global scope）：除非名字在某个语言构造（例如类或函数）内部声明，否则它就处于全局作用域。
- 命名空间作用域（namespace scope）：如果名字定义在一个命名空间内，而不在某个语言构造（例如类或函数）内部，它就处于命名空间作用域。严格说来，全局作用域就是一个“名字为空”的命名空间作用域。
- 局部作用域（local scope）：如果名字在函数内部声明（包括函数参数），它就处于局部作用域。
- 类作用域（class scope）：如果名字是某个类的成员的名字，它就处于类作用域。
- 语句作用域（statement scope）：如果名字在 for、while、switch 或 if 语句的（. . .）部分中声明，它就处于语句作用域。

变量的作用域（只）延伸到定义它的那条语句的末尾。例如：

```
for (int i = 0; i<v.size(); ++i) {
    // 这里可以使用 i
}
if (i < 27)          // for 语句里的 i 在这里不在作用域内
```

类和命名空间作用域是有名字的，因此我们可以从“别处”引用某个成员。例如：

```
void f();            // 在全局作用域中

namespace N {
    void f()         // 在命名空间作用域 N 中
    {
        int v;       // 在局部作用域中
        ::f();       // 调用全局的 f()
    }
}

void f()
{
    N::f();          // 调用 N 的 f()
}
```

如果你调用 `N::f()` 或 `::f()`，会发生什么？另见 §A.15。

A.4.2　存储类别

共有三种存储类别（§17.4）：

- 自动存储（automatic storage）：在函数中定义的变量（包括函数参数）被放在自动存储中（即“在栈上”），除非显式声明为 `static`。自动存储在一个函数被调用时分配，在调用返回时释放；因此，如果一个函数（直接或间接地）调用了自己，就可能存在多份自动数据：每次调用各一份（§8.5.8）。
- 静态存储（static storage）：在全局作用域和命名空间作用域中声明的变量存放在静态存储中，在函数和类中显式声明为 `static` 的变量也是如此。链接器“在程序开始运行之前”分配静态存储。

- 自由存储（free store，堆（heap））：由 new 创建的对象分配在自由存储中。

例如：

```
vector<int> vg(10);          // 在程序开始时构造一次（“在 main() 之前”）

vector<int>* f(int x)
{
    static vector<int> vs(x);   // 只在 f() 第一次调用时构造
    vector<int> vf(x+x);        // 每次调用 f() 时构造

    for (int i=1; i<10; ++i) {
        vector<int> vl(i);      // 每次迭代时构造
        // . . .
    }                           // vl 在此销毁（每次迭代）

    return new vector<int>(vf); // 在自由存储区构造 vf 的一个副本
}                               // vf 在此销毁

void ff()
{
    vector<int>* p = f(10);     // 从 f() 得到一个 vector
    // . . .
    delete p;                   // 删除来自 f 的 vector
}
```

静态分配的变量 vg 和 vs 在程序终止时（“在 main() 之后”）被销毁，前提是它们已被构造。

类成员并不按这种方式分配。当你在某处分配一个对象时，它的非静态成员也放在那里（其存储类别与所属类对象的存储类别相同）。

代码与数据分开存放。例如，成员函数并不会存放在其类的每个对象中；程序里只保存一份，与其余代码放在一起。

另见 §14.3 和 §17.4。

A.4.3　生存期

一个对象在被（合法地）使用之前，必须先被初始化。这种初始化可以是显式的，即使用初始化器；也可以是隐式的，即通过构造函数或内置类型的默认初始化规则完成。对象的生存期在由它的作用域和存储类别决定的位置结束（例如见 §17.4 和 §B.4.2）：

- 局部（自动）对象在执行线程到达它们时被构造（如果确实到达的话），并在作用域结束时销毁。
- 临时对象由某个特定的子表达式创建，并在其完整表达式结束时销毁。完整表达式是指不是其他表达式的子表达式的表达式。
- 命名空间对象和静态类成员在程序开始时（“在 main() 之前”）构造，在程序结束时（“在 main() 之后”）销毁。
- 局部静态对象在执行线程到达它们时被构造（如果确实到达的话），并（如果已构造）在程序结束时销毁。

- 自由存储对象由 `new` 构造，并可由 `delete` 选择性地销毁。

绑定到局部引用或命名空间引用的临时变量，“寿命”与那个引用一样长。例如：

```
const char* string_tbl[] = { "Mozart", "Grieg", "Haydn", "Chopin" };
const char* f(int i) { return string_tbl[i]; }
void g(string s){}

void h()
{
    const string& r = f(0);      // 把临时 string 绑定到 r
    g(f(1));                     // 生成一个临时 string 并传递出去
    string s = f(2);             // 用临时 string 初始化 s
    cout << "f(3): " << f(3)     // 生成一个临时 string 并传递出去
         << " s: " << s
         << " r: " << r << '\n';
}
```

结果是

```
f(3): Chopin s: Haydn r: Mozart
```

为调用 `f(1)`、`f(2)` 和 `f(3)` 而生成的临时 string 在创建它们的那个表达式结束时被销毁。可是，为 `f(0)` 生成的临时对象绑定到了 `r` 上，会“活”到 `h()` 结束。

A.5　表达式

本节概述 C++ 的运算符。我们使用一些自认为便于记忆的缩写，例如用 m 表示成员名、T 表示类型名、p 表示求值为指针的表达式、x 表示表达式、v 表示左值表达式、lst 表示实参列表。算术运算的结果类型由“通常的算术转换”决定（§A.5.2.2）。本节描述的是内置运算符，而不是你自己可能定义的运算符；不过，当你定义自己的运算符时，我们鼓励你遵循为内置运算描述的语义规则（§9.6）。

作用域解析

`N::m`　m 在命名空间 N 中；N 是命名空间或类的名字

`::m`　m 在全局命名空间中

注意，成员本身还可以嵌套，因此你可以得到 `N::C::m`；另见 §8.7。

后缀表达式

`x.m`	成员访问；x 必须是类对象
`p->m`	成员访问；p 必须指向类对象；等价于 `(*p).m`
`p[x]`	下标；等价于 `*(p+x)`
`f(lst)`	函数调用：以实参列表 lst 调用 f
`T(lst)`	构造：以实参列表 lst 构造一个 T
`v++`	（后置）递增；`v++` 的值是 v 递增之前的值
`v--`	（后置）递减；`v--` 的值是 v 递减之前的值
`typeid(x)`	对 x 做运行时类型识别
`typeid(T)`	对 T 做运行时类型识别
`dynamic_cast<T>(x)`	把 x 转换为 T，运行时检查
`static_cast<T>(x)`	把 x 转换为 T，编译时检查
`const_cast<T>(x)`	不检查的转换：为 x 的类型添加或去除 const，以得到 T
`reinterpret_cast<T>(x)`	不检查的转换：通过重新解释 x 的位模式，把 x 转换为 T

本书没有涉及 `typeid` 运算符及其用法；请查阅专家级的参考书。注意，类型转换并不修改其操作数；它们产生一个具有该类型的结果，而这个结果在某种程度上对应于操作数的值；见 §A.5.7。

一元表达式

`sizeof(T)`	T 的大小，以字节计
`sizeof(x)`	x 的类型的对象的大小，以字节计
`++v`	（前置）递增；等价于 `v+=1`
`--v`	（前置）递减；等价于 `v-=1`
`~x`	x 的按位取补；~ 是位运算
`!x`	非 x；返回 `true` 或 `false`
`&v`	v 的地址
`*p`	p 所指向对象的内容
`new T`	在自由存储上创建一个 T
`new T(lst)`	在自由存储上创建一个 T，并用 lst 初始化它
`new(lst) T`	在 lst 确定的位置构造一个 T
`new(lst) T(lst2)`	在 lst 确定的位置构造一个 T，并用 lst2 初始化它
`delete p`	释放 p 所指向的对象
`delete[] p`	释放 p 所指向的对象数组
`(T)x`	C 风格的类型转换；把 x 转换为 T

注意，`delete p` 和 `delete[] p` 中的 p 所指向的对象必须是用 `new` 分配的；见 §A.5.6。还要注意，(T)x 远不如那些更具体的类型转换运算符明确——因此也更容易出错；见 §A.5.7。

成员选择

`x.*ptm`　x 中由成员指针 ptm 标识的成员

`p->*ptm`　*p 中由成员指针 ptm 标识的成员

本书不涉及；请查阅专家级的参考书。

乘法运算符

`x*y`　用 y 乘 x。

`x/y`　用 y 除 x。

`x%y`　x 除以 y 的模（余数）（不适用于浮点类型）。

如果 `y==0`，`x/y` 和 `x%y` 的效果是未定义的。如果 x 或 y 为负，`x%y` 的效果由实现定义。

加法运算符

`x+y`　把 x 与 y 相加。

`x-y`　从 x 中减去 y。

移位运算符

`x<<y`　把 x 左移 y 个二进制位。

`x>>y`　把 x 右移 y 个二进制位。

关于 `>>` 和 `<<` 用于移动二进制位时的（内置）用法，见 §25.5.4。当它们最左边的运算对象是 iostream 时，这两个运算符用于输入输出；见第 10 章和第 11 章。

关系运算符

`x<y`　x 小于 y；返回 `bool`

`x<=y`　x 小于或等于 y

`x>y`　x 大于 y

`x>=y`　x 大于或等于 y

关系运算符的结果是 `bool`。

相等运算符

`x==y`　x 等于 y；返回 `bool`

`x!=y`　x 不等于 y

注意，`x!=y` 就是 `!(x==y)`。相等运算符的结果是 `bool`。

`x&y`　x 与 y 的按位与（bitwise and）

注意：`&`（与 `^`、`|`、`~`、`>>`、`<<` 一样）给出的是一个位集合。例如，如果 a 和 b 是 `unsigned char`，那么 `a&b` 是一个 `unsigned char`，它的每一位都是对 a 和 b 中对应位施加 `&` 的结果；见 §A.5.5。

`x^y`　x 与 y 的按位异或（bitwise exclusive or）

`x|y`　x 与 y 的按位或（bitwise or）

`x&&y`　逻辑与（logical and）：返回 true 或 false；仅当 x 为 true 时才求值 y

`x||y`　逻辑或（logical or）：返回 true 或 false；仅当 x 为 false 时才求值 y

另见 §A.5.5。

`x?y:z`　条件表达式：若 x 为真，结果为 y；否则结果为 z

例如：

```
template<class T> T& max(T& a, T& b) { return (a>b)?a:b; }
```

所谓“问号冒号运算符”在 §8.4 中解释。

赋值

v=x	把 x 赋给 v；结果就是赋值之后的 v
v*=x	大致相当于 v=v*(x)
v/=x	大致相当于 v=v/(x)
v%=x	大致相当于 v=v%(x)
v+=x	大致相当于 v=v+(x)
v-=x	大致相当于 v=v-(x)
v>>=x	大致相当于 v=v>>(x)
v<<=x	大致相当于 v=v<<(x)
v&=x	大致相当于 v=v&(x)
v^=x	大致相当于 v=v^(x)
v|=x	大致相当于 v=v|(x)

所谓“大致相当于 v=v*(x)”，意思是 v*=x 的值就是这个，只是 v 只被求值一次。例如，v[++i]*=7+3 表示的是 (++i, v[i]=v[i]*(7+3))，而不是 (v[++i]=v[++i]*(7+3))（后者是未定义行为；见 §8.6.1）。

抛出表达式　throw x：抛出 x 的值。

throw 表达式的类型是 void。

逗号表达式　x,y：先执行 x，再执行 y；结果是 y。

同一个框里的运算符具有相同的优先级。位置较高的框中的运算符，优先级高于位置较低的框中的运算符。例如，a+b*c 表示 a+(b*c) 而不是 (a+b)*c，因为 * 的优先级高于 +。类似地，*p++ 表示 *(p++)，而不是 (*p)++。一元运算符和赋值运算符是右结合的，其他所有运算符都是左结合的。例如，a=b=c 表示 a=(b=c)，而 a+b+c 表示 (a+b)+c。

左值（lvalue）是这样一个表达式：它标识一个原则上可以被修改（不过，具有 const 类型的左值显然受类型系统保护，不能被修改）而且可以取其地址的对象。与左值相对的是右值（rvalue），即标识某种不可修改、也不能取其地址的东西的表达式，例如函数返回的值（&f(x) 是错误的，因为 f(x) 是右值）。

A.5.1　用户自定义运算符

这里给出的规则是针对内置类型的。如果使用了某个用户自定义运算符，表达式就直接被变换成对相应运算符函数的调用，此时由函数调用的规则决定会发生什么。例如：

```
class Mine { /* . . . */ };
bool operator==(Mine, Mine);

void f(Mine a, Mine b)
{
    if (a==b) {            // a==b 表示调用 operator==(a,b)
        // . . .
    }
}
```

用户自定义类型（user-defined type）就是类（§A.12、第 9 章）或枚举（§A.11、§9.5）。

A.5.2　隐式类型转换

整型和浮点类型（§A.8）可以在赋值和表达式中自由混用。只要可能，值就会被转换，以免丢失信息。不幸的是，破坏值的转换也会隐式进行。

A.5.2.1　提升

那些保持值不变的隐式转换通常称为提升（promotion）。在执行算术运算之前，会先用整型提升把较短的整数类型转成 `int`。这正反映了这些提升当初的目的：把操作数带到算术运算的“自然”大小。此外，`float` 到 `double` 的转换也算作提升。

提升是常规算术转换的一部分（见 §A.5.2.2）。

A.5.2.2　转换

基本类型之间可以互相转换，方式多得让人眼花缭乱。写代码时，你应当始终致力于避免未定义行为，以及那些悄悄丢掉信息的转换（见 §3.9 和 §25.5.3）。对许多可疑的转换，编译器都能给出警告。

- 整型转换：一个整数可以转换成另一种整数类型。一个枚举值也可以转换成整数类型。如果目标类型是无符号的，结果值就是从源值中取出的、能放进目标类型的那么多位（必要时丢弃高位）。如果目标类型是有符号的，那么当该值能在目标类型中表示时，值保持不变；否则，值由实现定义。注意，`bool` 和 `char` 都是整数类型。
- 浮点转换：一个浮点值可以转换成另一种浮点类型。如果源值能在目标类型中精确表示，结果就是原来的数值。如果源值位于两个相邻的目标值之间，结果就是这两个值之一。否则，行为未定义。注意，`float` 到 `double` 的转换算作提升。
- 指针和引用转换：任何指向对象类型的指针都可以隐式转换成 `void*`（§17.8、§27.3.5）。指向派生类的指针（引用）可以隐式转换成指向可访问且无歧义的基类的指针（引用）（§14.3）。求值为 0 的常量表达式（§A.5、§4.3.1）可以隐式转换成任何指针类型。`T*` 可以隐式转换成 `const T*`。类似地，`T&` 可以隐式转换成 `const T&`。
- 布尔转换：指针、整数和浮点值都可以隐式转换成 `bool`。非零值转换成 `true`，零值转换成 `false`。
- 浮点到整数的转换：把浮点值转换成整数值时，小数部分被丢弃。换言之，从浮点类型到整数类型的转换会截断。如果截断后的值不能在目标类型中表示，行为就是未定义的。从整数到浮点类型的转换则尽可能在硬件允许的范围内做到数学上正确：如果某个整数值不能精确表示成该浮点类型的值，就会发生精度损失。

- 常规算术转换：这些转换施加于二元运算符的操作数，把它们变成共同的类型，这个类型随后用作结果的类型：

1. 如果任一操作数的类型是 long double，另一个就转换成 long double。否则，如果任一操作数是 double，另一个就转换成 double。否则，如果任一操作数是 float，另一个就转换成 float。否则，对两个操作数进行整型提升。
2. 然后，如果任一操作数是 unsigned long，另一个就转换成 unsigned long。否则，如果一个操作数是 long int、另一个是 unsigned int，那么当 long int 能表示 unsigned int 的所有值时，就把 unsigned int 转换成 long int；否则两个操作数都转换成 unsigned long int。否则，如果任一操作数是 long，另一个就转换成 long。否则，如果任一操作数是 unsigned，另一个就转换成 unsigned。否则，两个操作数都是 int。

显然，最好不要太依赖复杂的类型混合，从而把隐式转换的需要降到最低。

A.5.2.3　用户自定义转换

除了标准的提升和转换之外，程序员还可以为用户自定义类型定义转换。只接受一个实参的构造函数定义了一种从其实参类型到其自身类型的转换。如果该构造函数是显式的（见 §18.4.1），那么只有程序员明确要求转换时才会发生转换；否则，转换可以是隐式的。

A.5.3　常量表达式

常量表达式是可以在编译时求值的表达式。例如：

```
const int a = 2.7*3;
const int b = a+3;

constexpr int a = 2.7*3;
constexpr int b = a+3;
```

const 可以用涉及变量的表达式来初始化，而 constexpr 必须用常量表达式来初始化。有些地方要求使用常量表达式，例如数组边界、case 标号、枚举量初始值以及 int 模板实参。例如：

```
int var = 7;
switch (x) {
case 77:            // 正确
case a+2:           // 正确
case var:           // 错误（var 不是常量表达式）
    // . . .
};
```

声明为 constexpr 的函数可以用于常量表达式。

A.5.4　sizeof

在 sizeof(x) 中，x 可以是类型，也可以是表达式。如果 x 是表达式，sizeof(x) 的值就是所得对象的大小；如果 x 是类型，sizeof(x) 就是该类型的对象的大小。大小以字节为单位。按照定义，sizeof(char)==1。

A.5.5　逻辑表达式

C++ 为整数类型提供了逻辑运算符：

位逻辑运算

`x&y`　x 与 y 的按位与

`x|y`　x 与 y 的按位或

`x^y`　x 与 y 的按位异或

逻辑运算

`x&&y`　逻辑与：返回 true 或 false；仅当 x 为 true 时才计算 y

`x||y`　逻辑或：返回 true 或 false；仅当 x 为 false 时才计算 y

位运算符对其操作数的每一位分别进行运算，而逻辑运算符（`&&` 和 `||`）把 0 当作 false，把其他任何值都当作 true。这些运算的定义是：

&	0	1	|	0	1	∧	0	1
0	0	0	0	0	1	0	0	1
1	0	1	1	1	1	1	1	0

A.5.6　new 与 delete

自由存储（free store，也叫动态存储、堆）上的内存用 `new` 分配，用 `delete`（针对单个对象）或 `delete[]`（针对数组）释放。如果内存耗尽，`new` 会抛出 `bad_alloc` 异常。一次成功的 `new` 操作至少分配 1 个字节，并返回指向所分配对象的指针。所分配对象的类型在 `new` 之后指定。例如：

```
int* p1 = new int;              // 分配一个（未初始化的）int
int* p2 = new int(7);           // 分配一个 int，并初始化为 7
int* p3 = new int[100];         // 分配 100 个（未初始化的）int
// . . .
delete p1;                      // 释放单个对象
delete p2;
delete[] p3;                    // 释放数组
```

如果你用 `new` 分配内置类型的对象，那么除非你指定初始化器，否则这些对象不会被初始化。如果你用 `new` 分配带有构造函数的类的对象，则会调用构造函数；除非你指定初始化器，否则调用的是默认构造函数（§17.4.4）。

`delete` 会为其操作数调用析构函数（如果有的话）。注意，析构函数可以是虚函数（§A.12.3.1）。

A.5.7　类型转换

有四种类型转换运算符：

类型转换运算符	
`x=dynamic_cast<D*>(p)`	尝试把 p 转换成 `D*`（可能返回 0）
`x=dynamic_cast<D&>(*p)`	尝试把 `*p` 转换成 `D&`（可能抛出 `bad_cast`）
`x=static_cast<T>(v)`	如果 T 可以转换成 v 的类型，就把 v 转换成 T
`x=reinterpret_cast<T>(v)`	把 v 转换成用相同位模式表示的 T
`x=const_cast<T>(v)`	通过加上或去掉 const 把 v 转换成 T
`x=(T)v`	C 风格的类型转换：想怎么转就怎么转
`x=T(v)`	函数式类型转换：想怎么转就怎么转
`X=T{v}`	用 v 构造一个 T（不做窄化）

（译注：原书该行印作 `X=T{v}`，上一行、下一行均作小写 `x`，此处的 `X` 疑为排印之误；为保持代码逐字保真，未作改动。）

动态类型转换（dynamic cast）通常用于类层次结构中的导航，此时 p 是指向基类的指针，而 D 派生自该基类。如果 v 不是 `D*`，它就返回 0。如果你希望 `dynamic_cast` 抛出异常（`bad_cast`）而不是返回 0，那就转换到引用而不是转换到指针。动态类型转换是唯一依赖运行时检查的类型转换。

静态类型转换（static cast）用于“行为相当良好的转换”，也就是说，v 本来就可能是一个 T 隐式转换的结果；见 §17.8。

重新解释类型转换（reinterpret cast）用于重新解释一个位模式。它不保证可移植。事实上，最好假定每一次使用 `reinterpret_cast` 都是不可移植的。一个典型的例子是把 `int` 转换成指针，以便把机器地址取到程序中；见 §17.8 和 §25.4.1。

C 风格的类型转换和函数式类型转换可以执行 `static_cast` 或 `reinterpret_cast` 所能完成的任何转换，并且可以和 `const_cast` 组合使用。

类型转换最好避免。在大多数情况下，应把使用类型转换看作编程拙劣的迹象。这条规则的例外见 §17.8 和 §25.4.1。C 风格的类型转换和函数式类型转换有一个糟糕的性质：你不必确切理解这个转换究竟在做什么（§27.3.4）。当你无法避免显式类型转换时，请优先使用具名类型转换。

A.6　语句

下面是 C++ 语句的文法（`opt` 表示“可选的”）：

```
statement:
    declaration
    { statement-listopt }
    try { statement-listopt } handler-list
    expressionopt ;
    selection-statement
    iteration-statement
    labeled-statement
    control-statement

selection-statement:
    if ( condition ) statement
    if ( condition ) statement else statement
    switch ( condition ) statement

iteration-statement:
    while ( condition ) statement
    do statement while ( expression ) ;
    for ( for-init-statement conditionopt ; expressionopt ) statement
    for ( declaration : expression ) statement

labeled-statement:
    case constant-expression : statement
    default : statement
    identifier : statement

control-statement:
    break ;
    continue ;
    return expressionopt ;
    goto identifier ;

statement-list:
    statement statement-listopt

condition:
    expression
    type-specifier declarator = expression

for-init-statement:
    expressionopt ;
    type-specifier declarator = expression ;

handler-list:
    catch ( exception-declaration ) { statement-listopt }
    handler-list handler-listopt
```

注意，声明也是语句；而且并不存在赋值语句或过程调用语句——赋值和函数调用都是表达式。更多信息见：

- 迭代（for 和 while）；见 §4.4.2。
- 选择（if、switch、case 和 break）；见 §4.4.1。break 会“跳出”最内层的 switch、while、do 或 for 语句；也就是说，接下来执行的语句是那条外层语句之后的语句。
- 表达式；见 §A.5、§4.3。
- 声明；见 §A.6、§8.2。
- 异常（try 和 catch）；见 §5.6、§19.4。

下面这个例子只是为了演示各种语句而拼凑出来的（它做什么？）：

```
int* f(int p[], int n)
{
    if (p==0) throw Bad_p(n);
    vector<int> v;
    int x;
    while (cin>>x) {
        if (x==terminator) break;    // 退出 while 循环
        v.push_back(x);
    }
    for (int i = 0; i<v.size() && i<n; ++i) {
        if (v[i]==*p)
            return p;
        else
            ++p;
    }
    return 0;
}
```

A.7　声明

一个声明由三部分组成：

- 被声明实体的名字
- 被声明实体的类型
- 被声明实体的初始值（在大多数情况下是可选的）

我们可以声明

- 内置类型和用户自定义类型的对象（§A.8）
- 用户自定义类型（类和枚举）（§A.10-11、第 9 章）
- 模板（类模板和函数模板）（§A.13）
- 别名（§A.16）
- 命名空间（§A.15、§8.7）
- 函数（包括成员函数和运算符）（§A.9、第 8 章）
- 枚举量（枚举的值）（§A.11、§9.5）

- 宏（§A.17.2、§27.8）

初始化器可以是用花括号 { } 括起来的表达式列表，其中可以有零个或多个元素（§3.9.2、§9.4.2、§18.2）。例如：

```
vector<int> v {a,b,c,d};
int x {y*z};
```

如果定义中对象的类型是 auto，那么该对象必须初始化，而它的类型就是初始化器的类型（§13.3、§21.2）。例如：

```
auto x = 7;                          // x 是 int
const auto pi = 3.14;                // pi 是 double
for (const auto& x : v)              // x 是 v 中元素的引用
```

A.7.1　定义

一个声明，如果它进行初始化、留出内存，或者以其他方式提供了在程序中使用某个名字所需的全部信息，就称为定义（definition）。程序中的每个类型、对象和函数都必须恰好有一个定义。例如：

```
double f();                          // 一个声明
double f() { /* . . . */ };          // （也是）一个定义
extern const int x;                  // 一个声明

int y;                               // 也是定义
int z = 10;                          // 带显式初始化器的定义
```

const 必须初始化。为此，语言要求 const 变量必须带有初始化器，除非它的声明中显式出现 extern（这样初始化器就必须落在别处的定义中），或者它的类型带有默认构造函数（§A.12.3）。类中作为成员的 const 必须在每个构造函数中用成员初始化器初始化（§A.12.3）。

A.8　内建类型

C++ 拥有大量基本类型，以及用修饰符由基本类型构造出的类型：

内建类型

`bool x`	x 是布尔值（取值 true 和 false）。
`char x`	x 是字符（通常 8 位）。
`short x`	x 是短整数（`short int`，通常 16 位）。
`int x`	x 是默认的整数类型。
`float x`	x 是浮点数（“短 `double`”）。
`double x`	x 是（“双精度”）浮点数。
`void* p`	p 是指向原始内存（类型未知的内存）的指针。
`T* p`	p 是指向 `T` 的指针。
`T *const p`	p 是指向 `T` 的常量（不可变）指针。
`T a[n]`	a 是 n 个 `T` 组成的数组。
`T& r`	r 是 `T` 的引用。
`T f(arguments)`	f 是接受一组参数并返回 `T` 的函数。
`const T x`	x 是 `T` 的常量（不可变）版本。
`long T x`	x 是 `long T`。
`unsigned T x`	x 是 `unsigned T`。
`signed T x`	x 是 `signed T`。

这里，`T` 表示“某种类型”，因此你可以有 `long unsigned int`、`long double`、`unsigned char` 和 `const char*`（指向常量 `char` 的指针）。不过这套体系并不完全通用：例如没有 `short double`（那本来会是 `float`），没有 `signed bool`（那本来没有意义），没有 `short long int`（那本来会重复），也没有 `long long long long int`。`long long` 保证至少能容纳 64 位。

`float`、`double` 和 `long double` 是浮点类型。它们是 C++ 对实数的近似。

整数类型（有时称为整型）有 `bool`、`char`、`short`、`int`、`long` 和 `long long`，以及它们的无符号变体。注意，在需要整数类型或整数值的地方，往往也可以使用枚举类型或枚举值。

内建类型的大小在 §3.8、§17.3.1 和 §25.5.1 中讨论。指针和数组在第 17、18 章讨论，引用在 §8.5.4-6 中讨论。

A.8.1　指针

指针是对象或函数的地址。指针存放在指针类型的变量中。有效的对象指针保存着某个对象的地址：

```
int x = 7;
int* pi = &x;              // pi 指向 x
int xx = *pi;              // *pi 是 pi 所指对象的值，即 7
```

无效指针是不保存任何对象的值的指针：

```
int* pi2;                   // 未初始化
*pi2 = 7;                   // 未定义的行为
pi2 = nullptr;              // 空指针（pi2 仍然无效）
*pi2 = 7;                   // 未定义的行为
pi2 = new int(7);           // 现在 pi2 有效
int xxx = *pi2;             // 没问题：xxx 变成 7
```

我们尽量让无效指针取空指针值（nullptr），这样就可以测试它：

```
if (p2 == nullptr) {  // “无效”
    // 不要使用 *p2
}
```

或者干脆写成

```
if (p2) {             // “有效”
    // 使用 *p2
}
```

参见 §17.4 和 §18.6.4。

（非 void 的）对象指针上的运算有：

指针运算

*p	解引用 / 间接访问
p[i]	解引用 / 下标
p=q	赋值与初始化
p==q	相等
p!=q	不等
p+i	加整数
p-i	减整数
p-q	距离：两个指针相减
++p	前自增（前移）
p++	后自增（前移）
--p	前自减（后移）
p--	后自减（后移）
p+=i	前移 i 个元素
p-=i	后移 i 个元素

注意，任何形式的指针算术（例如 ++p 和 p+=7）只允许用于指向数组中元素的指针；对指向数组之外的指针解引用，其效果是未定义的（而且编译器或语言运行时系统很可能不会检查）。同一类型、指向同一对象或同一数组的指针之间，还可以使用 <、<=、> 和 >= 比较大小。

void* 指针上唯一的运算是复制（赋值或初始化）、类型转换（强制转换）和比较（==、!=、<、<=、> 和 >=）。

函数指针（§27.2.5）只能复制和调用。例如：

```
using Handle_type = void (*)(int);
void my_handler(int);
Handle_type handle = my_handler;
handle(10);             // 等价于 my_handler(10)
```

A.8.2　数组

数组是一段长度固定、连续存放的、给定类型的对象（元素）序列：

```
int a[10];              // 10 个 int
```

如果数组是全局的，它的元素会被初始化为该类型相应的默认值。例如 a[7] 的值会是 0。如果数组是局部的（在函数中声明的变量），或者是用 new 分配的，那么内建类型的元素不会被初始化，而类类型的元素会按该类的构造函数的要求初始化。

数组名会隐式转换为指向其首元素的指针。例如：

```
int* p = a;       // p 指向 a[0]
```

数组或指向数组元素的指针都可以用 [ ] 运算符下标访问。例如：

```
a[7] = 9;
int xx = p[6];
```

数组元素从 0 开始编号；见 §18.6。

数组不做范围检查；而且由于它们常常以指针的形式传递，做范围检查所需的信息无法可靠地提供给用户。优先使用 vector。

数组的大小是其各元素大小之和。例如：

```
int a[max];       // sizeof(a), 即 sizeof(int)*max
```

你可以定义并使用数组的数组（二维数组）、数组的数组的数组（多维数组），等等。例如：

```
double da[100][200][300];   // 300 个元素，元素类型为
                            // 200 个元素，元素类型为
                            // 100 个 double 类型的元素
da[7][9][11] = 0;
```

多维数组的非平凡用法很微妙，也容易出错；见 §24.4。如果有选择余地，优先使用矩阵库（例如第 24 章中的那个）。

A.8.3　引用

引用是对象的别名（另一个名字）：

```
int a = 7;
int& r = a;
r = 8;    // a 变成 8
```

引用最常见的用途是函数参数，用来避免复制：

```
void f(const string& s);
// . . .
f("this string could be somewhat costly to copy, so we use a reference");
```

参见 §8.5.4-6。

A.9　函数

函数是一段有名字的代码，它接受一组（可能为空的）实参，并可选地返回一个值。声明函数时，先给出返回类型，接着是函数名，再接着是形参列表：

```
char f(string, int);
```

因此，f 是一个接受 string 和 int、返回 char 的函数。如果只是声明函数，声明以分号结束。如果是在定义函数，实参声明后面还要跟函数体：

```
char f(string s, int i) { return s[i]; }
```

函数体必须是一个块（§8.2）或一个 try 块（§5.6.3）。

声明为返回值类型的函数必须返回一个值（用 return 语句）：

```
char f(string s, int i) { char c = s[i]; }   // 错误：没有返回值
```

main() 函数是这条规则的唯一例外（§A.1.2）。除 main() 之外，如果你不想返回值，就把函数声明为 void；也就是说，用 void 作为“返回类型”：

```
void increment(int& x) { ++x; }          // 正确：不需要返回值
```

调用函数要使用调用运算符（应用运算符）( )，并给出一组合法的实参：

```
char x1 = f(1,2);                        // 错误：f() 的第一个参数必须是 string
string s = "Battle of Hastings";
char x2 = f(s);                          // 错误：f() 需要两个参数
char x3 = f(s,2);                        // 正确
```

关于函数的更多信息，见第 8 章。

函数定义前面可以加 constexpr。这时，函数必须足够简单，使编译器在以常量表达式实参调用它时能够求值。constexpr 函数可以用在常量表达式中（§8.5.9）。

A.9.1　重载决议

重载决议是根据一组实参选择要调用哪个函数的过程。例如：

```
void print(int);
void print(double);
void print(const std::string&);

print(123);      // 调用 print(int)
print(1.23);     // 调用 print(double)
print("123");    // 调用 print(const string&)
```

按语言规则挑出正确的函数，是编译器的工作。遗憾的是，为了应付复杂的例子，这套语言规则相当复杂。这里我们给出一个简化版本。

从一组重载函数中找出应当调用的那个版本，做法是在实参表达式的类型与函数的形参（formal arguments）之间寻找最佳匹配。为了贴近我们对“合理”的直觉，语言按顺序尝试一系列判据：

1. 精确匹配，即不做转换或只做平凡转换的匹配（例如数组名到指针、函数名到函数指针，以及 `T` 到 `const T`）
2. 使用提升的匹配，即整型提升（`bool` 到 `int`、`char` 到 `int`、`short` 到 `int` 及其无符号对应类型；见 §A.8）以及 `float` 到 `double`
3. 使用标准转换的匹配，例如 `int` 到 `double`、`double` 到 `int`、`double` 到 `long double`、`Derived*` 到 `Base*`（§14.3）、`T*` 到 `void*`（§17.8）、`int` 到 `unsigned int`（§25.5.3）
4. 使用用户自定义转换的匹配（§A.5.2.3）
5. 使用函数声明中的省略号 `...` 的匹配（§A.9.3）

如果在找到匹配的最高层级上出现了两个匹配，这次调用就因有歧义而被拒绝。这些规则之所以如此繁复，主要是为了把内建数值类型的繁复规则（§A.5.3）也考虑进来。

对于基于多个实参的重载决议，我们先为每个实参找出最佳匹配。如果某个函数对每个实参都不比其它函数差，并且至少对一个实参比其它函数更好，那就选中这个函数；否则这次调用有歧义。例如：

```
void f(int, const string&, double);
void f(int, const char*, int);

f(1,"hello",1);              // 正确：调用 f(int, const char*, int)
f(1,string("hello"),1.0);    // 正确：调用 f(int, const string&, double)
f(1, "hello",1.0);           // 错误：有歧义
```

在最后一次调用中，`"hello"` 不加转换就匹配 `const char*`，而要匹配 `const string&` 必须转换。另一方面，`1.0` 不加转换就匹配 `double`，而要匹配 `int` 必须转换，因此两个 `f()` 谁也不比另一个更好。

如果这些简化规则与你的编译器所说的、以及你认为合理的结论不一致，请先想想你的代码是不是比需要的更复杂。如果是，就简化代码；如果不是，就查阅专家级的参考书。

A.9.2　默认实参

一般函数有时需要的实参比最常见情况下需要的更多。为此，程序员可以提供默认实参，在调用者没有指定某个实参时使用。例如：

```
void f(int, int=0, int=0);
f(1,2,3);
f(1,2);       // 调用 f(1,2,0)
f(1);         // 调用 f(1,0,0)
```

只有尾部的实参才能有默认值，也才能在调用中省略。例如：

```
void g(int, int =7, int);      // 错误：为非尾部参数指定了默认值
f(1,,1);                       // 错误：缺少第二个参数
```

重载可以代替默认实参（反之亦然）。

A.9.3　未指定实参

可以声明一个函数，而不指定其实参的个数或类型。这用省略号（...）表示，意思是“可能还有更多实参”。例如，下面是大概最有名的 C 函数 printf()（§27.6.1、§B.11.2）的声明和一些调用：

```
void printf(const char* format ...);  // 接受一个格式串，可能还有更多实参


int x = 'x';
printf("hello, world!");
printf("print a char '%c'\n",x);          // 把 int x 当作 char 打印
printf("print a string \"%s\"",x);        // 自找麻烦
```

格式串中的“格式说明符”，例如 %c 和 %s，决定是否以及如何使用后面的实参。如示例所示，这会导致讨厌的类型错误。在 C++ 中，最好避免使用未指定实参。

A.9.4　链接规范

C++ 代码常常和 C 代码用在同一个程序里，也就是说，程序的一部分用 C++ 编写（由 C++ 编译器编译），另一部分用 C 编写（由 C 编译器编译）。为了方便这一点，C++ 提供了链接规范，让程序员声明某个函数遵循 C 的链接约定。C 链接规范可以放在函数声明前面：

```
extern "C" void callable_from_C(int);
```

也可以作用于块中的所有声明：

```
extern "C" {
    void callable_from_C(int);
    int and_this_one_also(double, int*);
    /* . . . */
}
```

使用的细节见 §27.2.3。

C 不支持函数重载，因此对 C++ 的一个重载函数，你最多只能给其中一个版本加 C 链接规范。

A.10　用户自定义类型

程序员有两种方式定义新（用户自定义）类型：作为类（class、struct 或 union；见 §A.12），以及作为枚举（enum；见 §A.11）。

A.10.1　运算符重载

程序员可以为大多数运算符定义含义，使它们接受一个或多个用户自定义类型的运算对象。但不能改变运算符对内建类型的标准含义，也不能引入新的运算符。用户自定义运算符（“重载运算符”）的名字，是在运算符前面加关键字 operator；例如，定义 + 的函数名为 operator +：

```
Matrix operator+(const Matrix&, const Matrix&);
```

例子见 std::ostream（第 10–11 章）、std::vector（第 17–19 章、§B.4）、std::complex（§B.9.3）和 Matrix（第 24 章）。

除下列运算符外，其它运算符都可以由用户定义：

```
?:  .  .*  ::  sizeof  typeid  alignas  noexcept
```

定义下列运算符的函数必须是类的成员：

```
=  [ ]  ( )  ->
```

其它所有运算符既可以定义为成员函数，也可以定义为自由函数。

注意，每个用户自定义类型都默认定义了 =（赋值与初始化）、&（取地址）和 ,（逗号）。

使用运算符重载要克制，并遵循惯例。

A.11　枚举

枚举定义一种带有若干具名值（枚举量）的类型：

```
enum Color { green, yellow, red };                  // “普通”枚举
enum class Traffic_light { yellow, red, green };    // 作用域枚举
```

enum class 的枚举量位于该枚举的作用域内，而“普通”enum 的枚举量被导出到 enum 所在的作用域。例如：

```
Color col = red;            // 正确
Traffic_light tl = red;     // 错误：不能把整数值
                            // (即 Color::red) 转换为 Traffic_light
```

默认情况下，第一个枚举量的值为 0，因此 Color::green==0，之后的值依次加一，所以 Color 的 yellow==1、red==2。也可以显式定义枚举量的值：

```
enum Day { Monday=1, Tuesday, Wednesday };
```

这里得到 Monday==1、Tuesday==2、Wednesday==3。

“普通”enum 的枚举量和枚举值会隐式转换为整数，但整数不会隐式转换为枚举类型：

```
int x = green;          // 正确：Color 到 int 的隐式转换
Color c = green;        // 正确
c = 2;                  // 错误：没有 int 到 Color 的隐式转换
c = Color(2);           // 正确：(未检查的) 显式转换
int y = c;              // 正确：Color 到 int 的隐式转换
```

enum class 的枚举量和枚举值不会转换为整数，整数也不会隐式转换为枚举类型：

```
int x = Traffic_light::green;    // 错误：没有 Traffic_light 到 int 的隐式转换
Traffic_light c = green;         // 错误：没有 int 到 Traffic_light 的隐式转换
```

关于枚举用法的讨论，见 §9.5。

A.12　类

类是一种类型，用户为它的对象定义表示方式，以及允许对这些对象执行的操作：

```
class X {
public:
    // 用户接口
private:
    // 实现
};
```

在类声明中定义的变量、函数或类型称为该类的成员。类的技术细节见第 9 章。

A.12.1　成员访问

公有成员可以被用户访问；私有成员只能被该类自己的成员访问：

```
class Date {
public:
    // . . .
    int next_day();
private:
    int y, m, d;
};

void Date::next_day() { return d+1; }          // 正确

void f(Date d)
{
    int nd = d.d+1;            // 错误：Date::d 是私有的
    // . . .
}
```

struct 是一种成员默认为 public 的类：

```
struct S {
    // 成员（除非显式声明为 private，否则都是 public）
};
```

关于成员访问的更多细节，包括对 protected 的讨论，见 §14.3.4。

对象的成员既可以通过变量访问，也可以用 .（点）运算符经对象引用，或者通过指针用 ->（箭头）运算符访问：

```
struct Date {
    int d, m, y;
    int day() const { return d; }        // 在类内定义
    int month() const;                   // 仅声明；在别处定义
    int year() const;                    // 仅声明；在别处定义
};


Date x;
x.d = 15;                                // 通过变量访问
int y = x.day();                         // 通过变量调用
Date* p = &x;
p->m = 7;                                // 通过指针访问
int z = p->month();                      // 通过指针调用
```

类的成员可以用 ::（作用域解析）运算符来引用：

```
int Date::year() const { return y; }    // 类外定义
```

在成员函数内部，我们可以用非限定名引用其他成员：

```
struct Date {
    int d, m, y;
    int day() const { return d; }
    // . . .
};
```

这样的非限定名指的是调用该成员函数时所针对的那个对象的成员：

```
void f(Date d1, Date d2)
{
    d1.day();           // 将访问 d1.d
    d2.day();           // 将访问 d2.d
    // . . .
}
```

A.12.1.1　this 指针

如果想在引用“调用成员函数时所针对的那个对象”时写得更明确，可以使用预定义指针 this：

```
struct Date {
    int d, m, y;
    int month() const { return this->m; }
    // . . .
};
```

声明为 const 的成员函数（const 成员函数）不能修改“调用它所针对的那个对象”的成员的值：

```
struct Date {
    int d, m, y;
    int month() const { ++m; }  // 错误：month() 是 const 的
    // . . .
};
```

关于 const 成员函数的更多信息，见 §9.7.4。

A.12.1.2　友元

不是类成员的函数可以通过 friend 声明获得访问全部成员的权限。例如：

```
// 需要访问 Matrix 和 Vector 的成员:
Vector operator*(const Matrix&, const Vector&);

class Vector {
    friend
    Vector operator*(const Matrix&, const Vector&);  // 授予访问权限
    // . . .
};

class Matrix {
    friend
    Vector operator*(const Matrix&, const Vector&);  // 授予访问权限
    // . . .
};
```

如上所示，这通常是为那些需要访问两个类的函数而做的。`friend` 的另一种用途，是提供一个不应当用成员访问语法来调用的访问函数。例如：

```
class Iter {
public:
    int distance_to(const Iter& a) const;
    friend int difference(const Iter& a, const Iter& b);
    // . . .
};

void f(Iter& p, Iter& q)
{
    int x = p.distance_to(q);      // 用成员语法调用
    int y = difference(p,q);       // 用“数学语法”调用
    // . . .
}
```

请注意，声明为 `friend` 的函数不能再声明为 `virtual`。

A.12.2　类成员的定义

作为整数常量、函数或类型的类成员，既可以在类内定义/初始化（§9.7.3），也可以在类外定义（§9.4.4）：

```
struct S {
    int c = 1;
    int c2;
    void f() { }
    void f2();
    struct SS { int a; };
    struct SS2;
};
```

没有在类内定义的成员，必须在“别处”定义：

```
int S::c2 = 7;
void S::f2() { }
struct S::SS2 { int m; };
```

如果你想用一个由对象的创建者指定的值来初始化某个数据成员，就要在构造函数里做。

函数成员不占用对象中的空间：

```
struct S {
    int m;
    void f();
};
```

这里 `sizeof(S)==sizeof(int)`。标准其实并不保证这一点，但就我们所知的所有实现而言它都成立。不过要注意，带虚函数的类会有一个“隐藏”成员，用来支持虚调用（§14.3.1）。

A.12.3　构造、析构与拷贝

可以通过定义一个或多个构造函数来定义类对象初始化的含义。构造函数是一种成员函数，它的名字与类名相同，并且没有返回类型：

```
class Date {
public:
    Date(int yy, int mm, int dd) :y{yy}, m{mm}, d{dd} { }
    // . . .
private:
    int y,m,d;
};

Date d1 {2006,11,15};       // 正确：初始化由构造函数完成
Date d2;                    // 错误：没有初始化器
Date d3 {11,15};            // 错误：初始化器不对（需要三个初始化器）
```

注意，数据成员可以用构造函数中的初始化列表（基类与成员初始化列表）来初始化。成员的初始化顺序与它们在类中声明的顺序一致。

构造函数通常用来建立类的不变式，并获取资源（§9.4.2-3）。

类对象是“自底向上”构造的：先按声明顺序构造基类对象（§14.3.1），再按声明顺序构造成员，最后执行构造函数本身的代码。除非程序员做了非常奇怪的事，这就能保证每个对象在使用之前都已完成构造。

单参数构造函数除非声明为 `explicit`，否则就定义了一种从其参数类型到其类类型的隐式转换：

```
class Date {
public:
    Date(const char*);
    explicit Date(long);          // 使用 Date 的整数编码
    // . . .
};

void f(Date);

Date d1 = "June 5, 1848";          // 正确
f("June 5, 1848");                 // 正确

Date d2 = 2007*12*31+6*31+5;       // 错误：Date(long) 是 explicit 的
f(2007*12*31+6*31+5);              // 错误：Date(long) 是 explicit 的

Date d3(2007*12*31+6*31+5);        // 正确
Date d4 = Date{2007*12*31+6*31+5}; // 正确
f(Date{2007*12*31+6*31+5});        // 正确
```

除非一个类含有需要显式实参的基类或成员，并且除非该类还有其他构造函数，否则会自动生成一个默认构造函数。这个默认构造函数会初始化每个带有默认构造函数的基类或成员（没有默认构造函数的成员则保持未初始化）。例如：

```
struct S {
    string name, address;
    int x;
};
```

这个 `S` 有一个隐式构造函数 `S{}`，它会初始化 `name` 和 `address`，但不初始化 `x`。此外，没有构造函数的类还可以用初始化列表来初始化：

```
S s1 {"Hello!"};            // s1 变成 { "Hello! ",0}
S s2 {"Howdy!", 3};
S* p = new S{"G'day!"};     // *p 变成 {"G'day",0};
```

如上所示，末尾没有指定的值会取默认值（这里是 `int` 的 0）。

A.12.3.1　析构函数

可以通过定义析构函数来定义对象被销毁（例如离开作用域）的含义。析构函数的名字是 `~`（取补运算符）后跟类名：

```
class Vector { // double 的向量
public:
    explicit Vector(int s) : sz{s}, p{new double[s]} { }  // 构造函数
    ~Vector() { delete[] p; }                              // 析构函数
    // . . .
private:
    int sz;
    double* p;
};

void f(int ss)
{
    Vector v(s);
    // . . .
}  // 退出 f() 时 v 将被销毁；将对 v 调用 Vector 的析构函数
```

调用类成员的析构函数的析构函数可以由编译器生成；如果一个类要用作基类，它通常需要一个虚析构函数；见 §17.5.2。

析构函数通常用来“清理”并释放资源。

类对象是“自顶向下”析构的：先执行析构函数本身的代码，然后按声明顺序析构成员，再按声明顺序析构基类对象，也就是与构造相反的顺序。

A.12.3.2　拷贝

可以定义类的对象被拷贝的含义：

```
class Vector { // double 的向量
public:
    explicit Vector(int s) : sz{s}, p{new double[s]} { }  // 构造函数
    ~Vector() { delete[] p; }                              // 析构函数
    Vector(const Vector&);                                 // 拷贝构造函数
    Vector& operator=(const Vector&);                      // 拷贝赋值
    // . . .
private:
    int sz;
    double* p;
};

void f(int ss)
{
    Vector v(ss);
    Vector v2 = v;      // 使用拷贝构造函数
    // . . .
    v = v2;             // 使用拷贝赋值
    // . . .
}
```

默认情况下（也就是说，除非你定义了拷贝构造函数和拷贝赋值运算符），编译器会为你生成拷贝操作。拷贝的默认含义是逐成员拷贝；另见 §14.2.4 和 §18.3。

A.12.3.3　移动

可以定义类对象被移动的含义：

```
class Vector { // double 的向量
public:
    explicit Vector(int s) : sz{s}, p{new double[s]} { }    // 构造函数
    ~Vector() { delete[] p; }                               // 析构函数
    Vector(Vector&&);                                       // 移动构造函数
    Vector& operator=(Vector&&);                            // 移动赋值
    // . . .
private:
    int sz;
    double* p;
};

Vector f(int ss)
{
    Vector v(ss);
    // . . .
    return v;               // 使用移动构造函数
}
```

默认情况下（也就是说，除非你定义了拷贝构造函数和拷贝赋值运算符），编译器会为你生成移动操作。移动的默认含义是逐成员移动；另见 §18.3.4。

A.12.4　派生类

一个类可以定义为从其他类派生而来，此时它继承它所派生自的那些类（也就是它的基类）的成员：

```
struct B {
    int mb;
    void fb() { };
};

class D : B {
    int md;
    void fd();
};
```

这里 B 有两个成员 mb 和 fb()，而 D 有四个成员 mb、fb()、md 和 fd()。

与成员一样，基类也可以是 public 的或 private 的：

```
class DD : public B1, private B2 {
    // . . .
};
```

因此，B1 的公有成员成为 DD 的公有成员，而 B2 的公有成员成为 DD 的私有成员。派生类对其基类的成员并没有特殊访问权限，所以 DD 不能访问 B1 或 B2 的私有成员。

具有一个以上直接基类（例如 DD）的类，称为使用多继承。

指向派生类 D 的指针可以隐式转换为指向其基类 B 的指针，前提是 B 在 D 中可访问且没有歧义。例如：

```
struct B { };
struct B1: B { };    // B 是 B1 的 public 基类
struct B2: B { };    // B 是 B2 的 public 基类
struct C { };
struct DD : B1, B2, private C { };

DD* p = new DD;
B1* pb1 = p;         // 正确
B* pb = p;           // 错误：有歧义：B1::B 还是 B2::B
C* pc = p;           // 错误：DD::C 是 private 的
```

类似地，对派生类的引用可以隐式转换为一个没有歧义且可访问的基类。

关于派生类的更多信息，见 §14.3。关于 protected 的更多信息，见专家级教材或参考手册。

A.12.4.1　虚函数

虚函数是这样一种成员函数：它为派生类中同名、参数类型相同的函数定义了一个调用接口。调用虚函数时，调用实际触发的函数是为最派生类定义的那个函数。这时我们说，派生类覆盖（override）了基类中的虚函数。

```
class Shape {
public:
    virtual void draw();       // virtual 表示“可以被覆盖”
    virtual ~Shape() { }       // 虚析构函数
    // . . .
};

class Circle : public Shape {
public:
    void draw();               // 覆盖 Shape::draw
    ~Circle();                 // 覆盖 Shape::~Shape()
    // . . .
};
```

基本上，基类（这里是 Shape）的虚函数为派生类（这里是 Circle）定义了一个调用接口：

```
void f(Shape& s)
{
    // . . .
    s.draw();
}

void g()
{
    Circle c{Point{0,0}, 4};
    f(c);          // 将调用 Circle 的 draw
}
```

注意，f() 对 Circle 一无所知，它只知道 Shape。含虚函数的类的对象里会多包含一个指针，以便找到那组覆盖函数；见 §14.3。

注意，含有虚函数的类通常需要一个虚析构函数（Shape 就有一个）；见 §17.5.2。

想覆盖基类的虚函数时，可以用 override 后缀把这一意图明确写出来。例如：

```
class Square : public Shape {
public:
    void draw() override;      // 覆盖 Shape::draw
    ~Circle() override;        // 覆盖 Shape::~Shape()
    void silly() override;     // 错误：Shape 没有虚函数 Shape::silly()
    // . . .
};
```

A.12.4.2　抽象类

抽象类是一种只能用作基类的类。不能创建抽象类的对象：

```
Shape s;            // 错误：Shape 是抽象的

class Circle : public Shape {
public:
    void draw();    // 覆盖 Shape::draw
    // . . .
};

Circle c{p,20};             // 正确：Circle 不是抽象的
```

让一个类成为抽象类，最常见的做法是至少定义一个纯虚函数。纯虚函数是一种要求必须被覆盖的虚函数：

```
class Shape {
public:
    virtual void draw() = 0;     // =0 表示“纯”
    // . . .
};
```

见 §14.3.5。

另一种做法较少见，但同样有效：把类的所有构造函数都声明为 protected（§14.2.1），从而使该类成为抽象类。

A.12.4.3　生成的操作

定义一个类时，默认情况下它的对象上就已经定义好了若干操作：

- 默认构造函数
- 拷贝操作（拷贝赋值与拷贝初始化）
- 移动操作（移动赋值与移动初始化）
- 析构函数

其中每一个（同样是默认）都定义为递归地作用于它的每个基类和成员。构造是“自底向上”进行的，即基类先于成员。析构是“自顶向下”进行的，即成员先于基类。成员和基类按出现顺序构造，按相反顺序销毁。这样一来，构造函数和析构函数的代码就总是依赖定义良好的基类对象和成员对象。例如：

```
struct D : B1, B2 {
    M1 m1;
    M2 m2;
};
```

假设 B1、B2、M1 和 M2 都已定义，我们现在可以写

```
D f()
{
    D d;            // 默认初始化
    D d2 = d;       // 拷贝初始化
    d = D{};        // 默认初始化，随后拷贝赋值
    return d;       // d 被移出 f()
}   // 这里 d 和 d2 被销毁
```

例如，d 的默认初始化会（按顺序）调用四个默认构造函数：B1::B1()、B2::B2()、M1::M1() 和 M2::M2()。如果其中某个不存在或者无法调用，d 的构造就会失败。返回时会（按顺序）调用四个移动构造函数：B1::B1()、B2::B2()、M1::M1() 和 M2::M2()。如果其中某个不存在或者无法调用，返回就会失败。d 的析构会（按顺序）调用四个析构函数：M2:: M2()、M1:: M1()、B2:: B2() 和 B1:: B1()。如果其中某个不存在或者无法调用，d 的析构就会失败。这些构造函数和析构函数中的每一个，都可以是用户定义的，也可以是生成的。

如果类有用户定义的构造函数，就不会定义（生成）那个隐式的（编译器生成的）默认构造函数。

A.12.5　位域

位域是一种机制，用来把许多小值打包进一个字，或者用来匹配外部强加的位布局格式（例如设备寄存器）。例如：

```
struct PPN { // R6000 的物理页号
    unsigned int PFN : 22 ;          // 页框号
    int : 3 ;                        // 未使用
    unsigned int CCA : 3 ;           // 缓存一致性算法
    bool nonreachable : 1 ;
    bool dirty : 1 ;
    bool valid : 1 ;
    bool global : 1 ;
};
```

把位域从左到右打包进一个字，会得到字中各位的如下布局（见 §25.5.5）：



位域不一定需要名字；但如果它没有名字，你就无法访问它。

令人意外的是，把许多小值打包进一个字，未必能节省空间。事实上，与用 char 或 int 来表示哪怕一个二进制位相比，使用这些打包值中的某一个常常反而浪费空间。原因在于，从一个字中取出某一位、或者在不改动其他位的前提下写入某一位，需要若干条指令（而这些指令本身总得存放在内存的某处）。除非你需要大量带有极小数据字段的对象，否则不要试图用位域来节省空间。

A.12.6　联合体

联合体（union）是一种类，它的所有成员都从同一个地址开始分配。一个联合体一次只能存放一个元素，而且读出的成员必须与最后一次写入的成员相同。例如：

```
union U {
    int x;
    double d;
}

U a;
a.x = 7;
int x1 = a.x;      // 正确
a.d = 7.7;
int x2 = a.x;     // 糟糕
```

“读写必须一致”这条规则并不由编译器检查。我可提醒过你了。

A.13　模板

模板是由一组类型和/或整数参数化的类或函数：

```
template<typename T>
class vector {
public:
    // . . .
    int size() const;
private:
    int sz;
    T* p;
};

template<class T>
int vector<T>::size() const
{
    return sz;
}
```

在模板实参列表中，class 表示类型；typename 是等价的替代写法。模板类的成员函数隐式地就是一个模板函数，其模板实参与所属类的模板实参相同。

整数模板实参必须是常量表达式：

```
template<typename T, int sz>
class Fixed_array {
public:
    T a[sz];
    // . . .
    int size() const { return sz; };
};

Fixed_array<char,256> x1;    // 正确
int var = 226;
Fixed_array<char,var> x2;   // 错误：模板实参不是常量表达式
```

A.13.1　模板实参

模板类的实参在每次使用该类名字时给出：

```
vector<int> v1;                  // 正确
vector v2;                       // 错误：缺少模板实参
vector<int,2> v3;                // 错误：模板实参过多
vector<2> v4;                    // 错误：应给出类型模板实参
```

模板函数的实参通常由函数实参推导出来：

```
template<class T>
T find(vector<T>& v, int i)
{
    return v[i];
}

vector<int> v1;
vector<double> v2;

// . . .
int x1 = find(v1,2);          // find() 的 T 是 int
int x2 = find(v2,2);          // find() 的 T 是 double
```

我们有可能定义这样一个模板函数：无法从它的函数实参推导出它的模板实参。这种情况下必须显式指定缺失的模板实参（与类模板完全一样）。例如：

```
template<class T, class U> T* make(const U& u) { return new T{u}; }
int* pi = make<int>(2);
Node* pn = make<Node>(make_pair("hello",17));
```

只要 Node 能由一个 pair<const char *,int> 初始化，这段代码就能工作（§B.6.3）。在显式指定实参时，只有末尾的模板实参才可以省略（留给推导）。

A.13.2　模板实例化

针对一组特定模板实参的模板版本称为一个特化（specialization）。由模板和一组实参生成特化的过程称为模板实例化。通常，编译器根据模板和一组模板实参生成特化，但程序员也可以自行定义某个特定的特化。当通用模板不适用于某组特定实参时，通常就这样做。例如：

```
template<class T> struct Compare {        // 通用比较
    bool operator()(const T& a, const T& b) const
    {
        return a<b;
    }
};

template<> struct Compare<const char*> {     // 比较 C 风格字符串
    bool operator()(const char* a, const char* b) const
    {
        return strcmp(a,b)==0;
    }
};

Compare<int> c2;                             // 通用比较
Compare<const char*> c;                      // C 风格字符串比较
```

```
bool b1 = c2(1,2);                          // 使用通用比较
bool b2 = c("asd","dfg");                   // 使用 C 风格字符串比较
```

对函数而言，大致等价的做法是通过重载来实现：

```
template<class T> bool compare(const T& a, const T& b)
{
    return a<b;
}

bool compare (const char* a, const char* b)   // 比较 C 风格字符串
{
    return strcmp(a,b)==0;
}

bool b3 = compare(2,3);                                  // 使用通用比较
bool b4 = compare("asd","dfg");                          // 使用 C 风格字符串比较
```

模板的分离编译（即只在头文件中保留声明，而把唯一定义放在 .cpp 文件中）无法可移植地工作，因此如果一个模板要在多个 .cpp 文件中使用，就把它的完整定义放在头文件里。

A.13.3　模板成员类型

一个模板既可以有类型成员，也可以有非类型成员（例如数据成员和成员函数）。这意味着一般来说很难判断某个成员名指的是类型还是非类型。出于语言技术上的原因，编译器必须知道这一点，所以偶尔我们必须告诉它。为此我们使用关键字 typename。例如：

```
template<class T> struct Vec {
    typedef T value_type;     // 一个成员类型
    static int count;                 // 一个数据成员
    // . . .
};

template<class T> void my_fct(Vec<T>& v)
{
    int x = Vec<T>::count;      // 默认情况下，成员名
                                // 被假定为非类型
    v.count = 7;                        // 引用非类型成员的更简单方式
    typename Vec<T>::value_type xx = x;         // 这里需要 typename
    // . . .
}
```

关于模板的更多信息，见第 19 章。

A.14　异常

异常用于（配合 throw 语句）把无法就地处理的错误告知调用者。例如，把 Bad_size 从 Vector 中移出来：

```
struct Bad_size {
    int sz;
    Bad_size(int s) : ss{s} { }
};

class Vector {
    Vector(int s) { if (s<0 || maxsize<s) throw Bad_size{s}; }
    // . . .
};
```

通常，我们抛出的类型是专门为表示某个特定错误而定义的。调用者可以捕获异常：

```
void f(int x)
{
    try {
        Vector v(x);   // 可能抛出异常
        // . . .
    }
    catch (Bad_size bs) {
        cerr << "Vector with bad size (" << bs.sz << ")\n";
        // . . .
    }
}
```

可以用“捕获所有”（catch all）的子句来捕获每一个异常：

```
try {
    // . . .
} catch (...) {  // 捕获所有异常
    // . . .
}
```

通常，RAII（“资源获取即初始化”）技术比到处写显式的 try 和 catch 更好（更简单、更易用、更可靠）；见 §19.5。

不带实参的 throw（即 throw;）会重新抛出当前异常。例如：

```
try {
    // . . .
} catch (Some_exception& e) {
    // 做局部清理
    throw;          // 让我的调用者完成剩下的工作
}
```

你可以定义自己的类型用作异常。标准库也定义了少数几种异常类型，你也可以使用它们；见§B.2.1。绝不要用内置类型作异常（别人可能已经这么做过，你的异常就会与他们的混淆）。

当异常被抛出时，C++ 的运行时支持系统会“沿调用栈向上”查找类型与被抛出对象类型相匹配的 catch 子句；也就是说，它先查看抛出异常的那个函数中的 try 语句，再查看调用该函数的函数，再查看调用那个函数的函数，依此类推，直到找到匹配。如果找不到匹配，程序就会终止。在这个查找匹配 catch 子句的过程中遇到的每个函数、以及途经的每个作用域，都会调用析构函数进行清理。这个过程称为栈展开（stack unwinding）。

一个对象一旦其构造函数执行完毕，就被认为已经构造完成，随后它将在栈展开期间、或在任何其他离开其作用域的时刻被销毁。这意味着部分构造的对象（有些成员或基类已构造、有些尚未构造）、数组以及作用域中的变量都能被正确处理。子对象当且仅当已经被构造时才会被销毁。

不要抛出异常让它离开析构函数。这意味着析构函数不应该失败。例如：

```
X::~X() { if (in_a_real_mess()) throw Mess{}; }          // 绝不要这样做!
```

这条严厉建议的主要理由是：如果析构函数在栈展开期间抛出异常（而且自身没有捕获它），我们就不知道该处理哪一个异常了。值得花很大力气去避免析构函数因抛出而退出，因为我们不知道有什么系统化的办法能写出允许这种情况发生的正确代码。特别要说明的是，一旦发生这种情况，没有任何标准库设施能保证正常工作。

A.15　命名空间

命名空间把相关的声明组织在一起，用来防止名字冲突：

```
int a;

namespace Foo {
    int a;
    void f(int i)
    {
        a+= i;  // 这是 Foo 的 a (Foo::a)
    }
}

void f(int);

int main()
{
    a = 7;                  // 这是全局的 a (::a)
```

```
    f(2);                    // 这是全局的 f (::f)
    Foo::f(3);         // 这是 Foo 的 f
    ::f(4);                  // 这是全局的 f (::f)
}
```

名字可以用其命名空间名显式限定（例如 Foo::f(3)），也可以用 :: 限定（例如 ::f(2)），后者表示全局作用域。

命名空间中的所有名字（这里是标准库命名空间 std）都可以通过一条命名空间指令变得可访问：

```
using namespace std;
```

使用 using 指令要有节制。using 指令提供的书写便利，是以潜在的名字冲突为代价换来的。特别要避免在头文件中使用 using 指令。命名空间中的单个名字可以通过命名空间声明变为可用：

```
using Foo::g;
g(2);              // 这是 Foo 的 g (Foo::g)
```

关于命名空间的更多信息，见 §8.7。

A.16　别名

我们可以为一个名字定义别名；也就是说，我们可以定义一个符号名，对于该名字的大多数用法来说，它的含义与它所指向的东西完全相同：

```
using Pint = int*;                          // Pint 表示指向 int 的指针

namespace Long_library_name { /* . . . */ }
namespace Lib = Long_library_name;  // Lib 表示 Long_library_name

int x = 7;
int& r = x;                                                 // r 表示 x
```

引用（§8.5.5、§A.8.3）是一种运行期机制，它指向对象。using（§20.5）和命名空间别名则是编译期机制，它们指向名字。特别要说明的是，using 并不引入新类型，只是为某个类型引入一个新名字。例如：

```
using Pchar = char*;          // Pchar 是 char* 的名字
Pchar p = "Idefix";                 // 正确：p 是 char*
char* q = p;                             // 正确：p 和 q 都是 char*
int x = strlen(p);                     // 正确：p 是 char*
```

较早的代码使用关键字 typedef（§27.3.1），而不是（C++ 的）using 写法，来定义类型别名。例如：

```
typedef char* Pchar;           // Pchar 是 char* 的名字
```

A.17　预处理指令

每个 C++ 实现都包含一个预处理器。原则上，预处理器在真正的编译器之前运行，把我们写下的源代码变换成编译器所看到的形式。在现实中，这一动作已经集成到编译器中，除非它引起问题，否则并不引人注意。以 # 开头的每一行都是一条预处理指令。

A.17.1　#include

我们已经大量使用预处理器来包含头文件。例如：

```
#include "file.h"
```

这条指令告诉预处理器：把 file.h 的内容包含到源文本中出现该指令的位置。对于标准头文件，我们也可以用 <...> 代替 "..."。例如：

```
#include<vector>
```

这是包含标准头文件的推荐写法。

A.17.2　#define

预处理器实现了一种字符处理机制，称为宏替换（macro substitution）。例如，我们可以为一个字符串定义名字：

```
#define FOO bar
```

现在，只要出现 FOO，它就会被替换成 bar：

```
int FOO = 7;
int FOOL = 9;
```

于是，编译器看到的将是

```
int bar = 7;
int FOOL = 9;
```

注意，预处理器对 C++ 名字有足够的了解，不会去替换 FOOL 中作为其一部分出现的 FOO。

你也可以定义带参数的宏：

```
#define MAX(x,y) (((x)>(y))?(x) : (y))
```

我们可以这样使用它：

```
int xx = MAX(FOO+1,7);
int yy = MAX(++xx,9);
```

它将被展开成

```
int xx = (((bar+1)>( 7))?(bar+1) : (7));
int yy = (((++xx)>( 9))?(++xx) : (9));
```

注意，为了让 FOO+1 得到正确结果，那些括号是必需的。还要注意，xx 以一种非常不直观的方式被自增了两次。宏极其流行——主要是因为 C 程序员几乎没有别的选择。常见的头文件定义了成千上万个宏。我可提醒过你了！

如果必须使用宏，惯例是用全大写字母（ALL_CAPITAL_LETTERS）来命名。任何普通名字都不应该全部使用大写字母。不要指望别人听从这条合理的建议。例如，我们在一个本来声誉不错的头文件里发现了一个名为 max 的宏。

另见 §27.8。

附录 C　Visual Studio 入门

"The universe is not only queerer than we imagine,
it's queerer than we can imagine."
（宇宙不仅比我们想象的更奇异，
而且比我们能够想象的更奇异。）
——J. B. S. Haldane

本附录说明：用 Microsoft Visual Studio 输入程序、编译程序并让它运行起来，需要经过哪些步骤。

C.1　让程序运行起来
C.2　安装 Visual Studio
C.3　创建并运行程序
　C.3.1　新建项目
　C.3.2　使用 `std_lib_facilities.h` 头文件
　C.3.3　向项目添加 C++ 源文件
　C.3.4　输入源代码
　C.3.5　生成可执行程序
　C.3.6　执行程序
　C.3.7　保存程序
C.4　后续

C.1　让程序运行起来

要让一个程序运行起来，你得设法把相关文件放在一起（这样当一个文件引用另一个文件时——比如你的源文件引用某个头文件——它就能找到对方）。然后你需要调用编译器和链接器（至少要把程序与 C++ 标准库链接起来），最后再运行（执行）程序。做到这一点有若干种办法，不同的系统（例如 Windows 和 Linux）有不同的惯例和工具集。不过，只要使用任一主流工具集，你就能在所有主流系统上运行本书的全部例子。本附录讲解在一个流行系统上怎么做到这一点，即 Microsoft 的 Visual Studio。

就我们自己而言，很少有哪种练习比在一个陌生而新奇的环境里让第一个程序跑起来更让人沮丧。这种任务确实值得请人帮忙。不过，如果真有人帮你，一定要让帮忙的人教会你怎么做，而不是替你做完。

C.2　安装 Visual Studio

Visual Studio 是 Windows 上的一个交互式开发环境（interactive development environment，IDE）。如果你的计算机上还没有安装 Visual Studio，可以购买一套并按照随附的说明安装，也可以从 www.microsoft.com/express/download 下载并安装免费的 Visual C++ Express。这里的说明基于 Visual Studio 2005，其他版本可能略有差异。

C.3　创建并运行程序

步骤如下：

1. 新建一个项目。
2. 向项目中添加一个 C++ 源文件。
3. 输入源代码。
4. 生成一个可执行文件。
5. 执行程序。
6. 保存程序。

C.3.1　新建项目

在 Visual Studio 中，“项目”（project）是一组文件，它们合在一起提供在 Windows 下创建和运行程序（也叫应用程序）所需的一切。具体步骤如下：

1. 单击 Microsoft Visual Studio 2005 图标打开 Visual C++ IDE，或者从 Start > Programs > Microsoft Visual Studio 2005 > Microsoft Visual Studio 2005 中选择它。
2. 打开 File 菜单，指向 New，然后单击 Project。
3. 在 Project Types 下选择 Visual C++。
4. 在 Templates 区域选择 Win32 Console Application。
5. 在 Name 文本框中键入项目的名称，例如 Hello,World!。
6. 为项目选择一个目录。默认目录 `C:\Documents and Settings\Your Name\My Documents\Visual Studio 2005\Projects` 通常是个好选择。
7. 单击 OK。
8. 此时应出现 WIN32 Application Wizard。
9. 在对话框左侧选择 Application Settings。
10. 在 Additional Options 下选择 Empty Project。
11. 单击 Finish。这样，控制台项目的所有编译器设置就都初始化好了。

C.3.2　使用 `std_lib_facilities.h` 头文件

对于最初几个程序，我们强烈建议你使用来自 `www.stroustrup.com/Programming/std_lib_facilities.h` 的自定义头文件 `std_lib_facilities.h`。把它的一份副本放到你在 C.3.1 节第 6 步选定的目录中（注意：存成文本文件，不要存成 HTML）。要使用它，你的程序中需要有下面这一行：

```
#include "../../std_lib_facilities.h"
```

这一行要写进你的程序。“../../”告诉编译器：你把头文件放在了 `C:\Documents and Settings\Your Name\My Documents\Visual Studio 2005\Projects`，这样你的所有项目都能使用它；而不是把它放在某个项目的源文件旁边——那样每个项目都得复制一份。

C.3.3　向项目添加 C++ 源文件

你的程序中至少需要一个源文件（通常需要很多个）：

1. 单击菜单栏上的 Add New Item 图标（通常是左起第二个图标）。这会打开 Add New Item 对话框。在 Visual C++ 类别下选择 Code。
2. 在模板窗口中选择 C++ File (.cpp) 图标。在 Name 文本框中键入程序文件名（Hello,World!），然后单击 Add。

这样你就创建了一个空的源代码文件。现在可以开始键入源代码程序了。

C.3.4　输入源代码

此时，你可以直接在 IDE 中键入源代码，也可以从其他来源复制粘贴过来。

C.3.5　生成可执行程序

当你确信已经正确输入了程序的源代码，就转到 Build 菜单选择 Build Solution，或者单击 IDE 窗口顶部附近图标列表中那个指向右侧的三角形图标。IDE 会尝试编译并链接你的程序。如果成功，Output 窗口中应出现下面的消息：

```
Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped
```

否则会出现若干条错误消息。调试程序，改正错误，然后再次执行 Build Solution。

如果你用的是那个三角形图标，那么只要没有错误，程序就会自动开始运行（执行）。如果你用的是 Build Solution 菜单项，就必须显式地启动程序，具体做法见 C.3.6 节。

C.3.6　执行程序

所有错误都消除之后，转到 Debug 菜单选择 Start Without Debugging 来执行程序。

C.3.7　保存程序

在 File 菜单下单击 Save All。如果你忘了保存而想要关闭 IDE，IDE 会提醒你。

C.4　后续

IDE 的功能和选项多得像是无穷无尽。起步阶段不要为它们分心——否则你会彻底迷失。如果你万一把一个项目弄得一团糟，让它“行为古怪”，就向有经验的朋友求助，或者干脆从头新建一个项目。随着时间推移，再慢慢尝试新的功能和选项。